

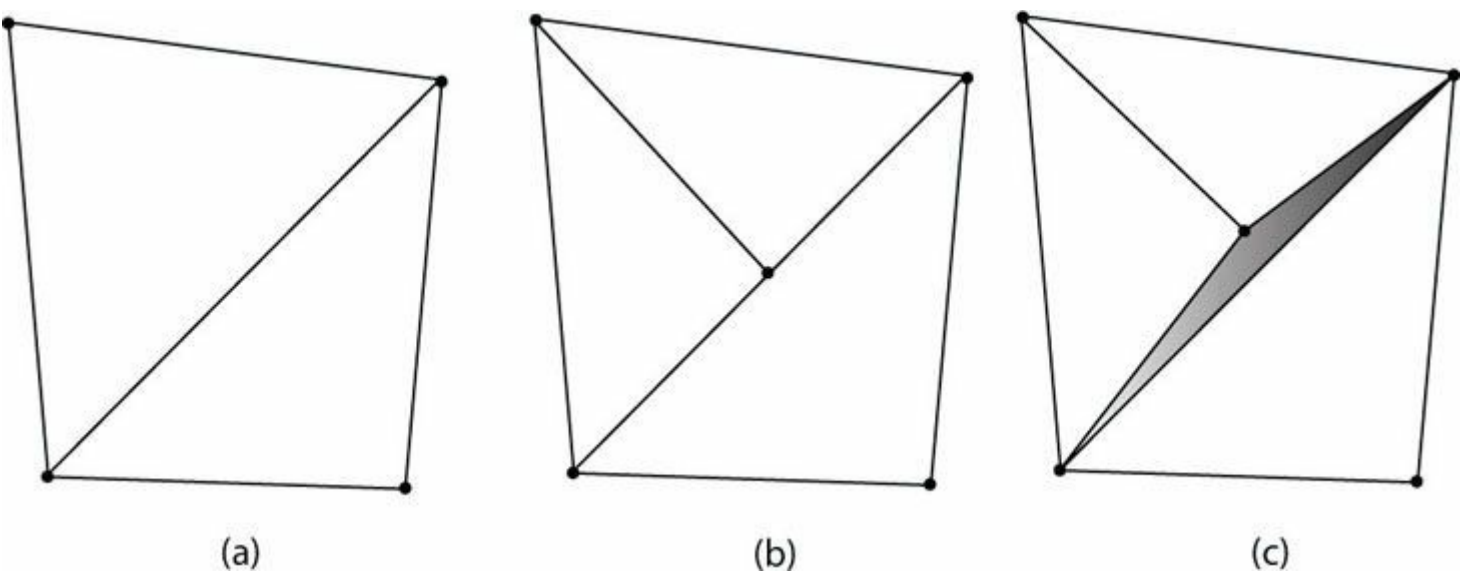


그림 18.10. (a) 한 변을 공유하는 두 개의 삼각형. (b) 상단 삼각형은 테셀레이션되어 변을 따라 추가 정점이 도입된다. 하단 삼각형은 테셀레이션되지 않는다. (c) 변위 매핑을 통해 새로 생성된 정점이 이동된다. 이로 인해 한때 변을 공유했던 두 삼각형 사이에 틈(음영 처리된 영역으로 표시)이 생겨난다.

제어점 헬 셰이더는 여러 제어점을 입력으로 받고 여러 제어점을 출력으로 낸다는 점을 상기하십시오. 제어점 헬 셰이더는 출력된 제어점마다 한 번씩 호출됩니다. 우리의 변위 매핑 데모에서 제어점 헬 셰이더는 단순히 "통과" 셰이더입니다:

```
struct HullOut
{
    float3 PosW          : 위치; float3 법선W :
    법선; float3 접선W : 접선; float2 텍스처   : 텍
    스처 좌표;
};

[도메인("삼각형")] [분할("소수점_홀수")] [출력토폴로
지("삼각형_시계방향")] [출력제어점(3)] [패치상수함수
("PatchHS")]
HullOut HS(InputPatch<VertexOut,3> p,
            uint i : SV_OutputControlPointID, uint patchId :
            SV_PrimitiveID)
{
    HullOut hout;

    // 셰이더 통과. hout.PosW          =
    p[i].PosW; hout.NormalW = p[i].NormalW;
    hout.TangentW = p[i].TangentW; hout.Tex
    = p[i].Tex;

    return hout;
}
```

18.8.4 도메인 셰이더

도메인 셰이더는 테셀레이션 단계에서 생성된 각 버텍스에 대해 호출됩니다. 헬 셰이더는 본질적으로 테셀레이션된 패치의 버텍스 셰이더입니다. 여기서 높이맵(노멀 맵의 알파 채널에 저장됨)을 샘플링하고, §18.7에서 논의된 공식에 따라 노멀 방향으로 버텍스를 오프셋하여 변위 매핑을 수행합니다.

```
struct DomainOut
{
    float4 PosH          : SV_POSITION; float3
    PosW                  : POSITION;
```

```

float3 NormalW : NORMAL; float3
TangentW : TANGENT; float2 Tex :
TEXCOORD;

};

// 도메인 셰이더는 테셀레이터가 생성한 모든 버텍스에 대해 호출됩니다.
// 테셀레이션 후의 버텍스 셰이더와 유사합니다. [domain("tri")]

DomainOut DS(PatchTess patchTess,
              float3 bary : SV_DomainLocation, const
              OutputPatch<HullOut,3> tri)
{
    DomainOut dout;

    // 패치 속성을 생성된 버텍스에 보간합니다. dout.PosW =
    bary.x*tri[0].PosW +
        bary.y*tri[1].PosW +
        bary.z*tri[2].PosW;
    dout.NormalW = bary.x*tri[0].NormalW +
        bary.y*tri[1].NormalW + bary.z*tri[2].NormalW;
    dout.TangentW = bary.x*tri[0].TangentW + bary.y*tri[1].TangentW +
        bary.z*tri[2].TangentW;
    dout.Tex = bary.x*tri[0].Tex +
        bary.y*tri[1].Tex +
        bary.z*tri[2].Tex;

    // 보간된 법선은 비정규화될 수 있으므로 정규화합니다. dout.NormalW =
    normalize(dout.NormalW);

    //
    // 변위 매핑.
    //

    // 시점과의 거리에 따라 밍맵 레벨 선택;
    // 구체적으로 MipInterval 단위마다 다음 미프 레벨을 선택하고,
    // mip 수준을 [0,6] 범위에서 클램프합니다. const
    float MipInterval = 20.0f; float mipLevel = clamp(
        (distance(dout.PosW, gEyePosW) - MipInterval) / MipInterval, 0.0f, 6.0f);

    // 높이 맵 샘플링 (알파 채널에 저장됨).
    float h = gNormalMap.SampleLevel(samLinear, dout.Tex, mipLevel).a;

    // 노멀 방향으로 버텍스 오프셋.
    dout.PosW += (gHeightScale*(h-1.0))*dout.NormalW;

    // 동차 클립 공간으로 투영.
    dout.PosH = mul(float4(dout.PosW, 1.0f), gViewProj);

    return dout;
}

```

한 가지 언급할 점은 도메인 셰이더에서 직접 밍맵 레벨 선택을 수행해야 한다는 것입니다. 일반적인 픽셀 셰이더 메서드인 **Texture2D::Sample**은 도메인 셰이더에서 사용할 수 없으므로 **Texture2D::SampleLevel** 메서드를 사용해야 합니다. 이 메서드는 밍맵 레벨을 명시적으로 지정해야 합니다.

 본 장의 데모에는 노멀 매핑과 변위 매핑 구현이 포함되어 있습니다. 주요 컨트롤은 다음과 같습니다.

1. 와이어프레임 모드: 키 '1' 누르기

2. 기본 렌더링을 위해 '2'를 누릅니다.
3. '3'을 눌러 노멀 매핑 렌더링을 실행합니다.
4. '4'를 눌러 변위 매핑 렌더링을 실행합니다.

18.9 요약

1. 노멀 매핑의 전략은 폴리곤에 노멀 맵을 텍스처링하는 것입니다. 이를 통해 픽셀 단위 노멀을 얻게 되며, 이는 표면의 미세한 디테일(올통불통함, 굽힘, 틈새 등)을 포착합니다. 이후 조명 계산 시 보관된 버텍스 노멀 대신 노멀 맵에서 추출한 이 픽셀 단위 노멀을 사용합니다.
2. 노멀 맵은 텍스처이지만, 각 텍셀에 RGB 데이터를 저장하는 대신 압축된 x좌표, y좌표, z좌표를 각각 빨간색, 녹색, 파란색 성분에 저장합니다. 노멀 맵 생성을 위해 <http://developer.nvidia.com/nvidia-texture-tools-adobe-photoshop> 및 <http://www.crazybump.com> 등에 위치한 다양한 도구를 사용합니다.
3. 노멀 맵 내 노멀의 좌표는 텍스처 공간 좌표계에 상대적입니다. 따라서 조명 계산을 수행하려면 노멀을 텍스처 공간에서 월드 공간으로 변환하여 조명과 노멀이 동일한 좌표계에 위치하도록 해야 합니다. 각 버텍스에 구축된 TBN 기반은 텍스처 공간에서 월드 공간으로의 변환을 용이하게 합니다.
4. 변위 매핑의 개념은 표면의 요철과 틈새를 묘사하는 높이맵이라는 추가 맵을 활용하는 것입니다. 메시를 테셀레이션할 때 도메인 세이더에서 높이맵을 샘플링하여 정점들을 법선 벡터 방향으로 오프셋함으로써 메시에 기하학적 디테일을 추가합니다.

18.10 연습문제

1. NVIDIA 노멀 맵 플러그인(http://developer.nvidia.com/object/nv_texture_tools.html)을 다운로드하고 이를 활용해 다양한 노멀 맵을 만들어 보세요. 본 장의 데모 애플리케이션에서 제작한 노멀 맵을 테스트해 보십시오.
2. *CrazyBump* 평가판(<http://www.crazybump.com/>)을 다운로드하세요. 컬러 이미지를 불러온 후 노멀 맵과 변위 맵을 생성해 보세요. 생성한 맵을 본 장의 데모 애플리케이션에서 테스트해 보세요.
3. 변위 매핑 변수에 대한 다양한 값을 실험해 보세요:

```
cbuffer cbPerFrame
{
    ...
    float gHeightScale;
    float gMaxTessDistance; float
    gMinTessDistance; float
    gMinTessFactor; float
    gMaxTessFactor;
};
```

4. 회전 텍스처 변환을 적용하는 경우, 탄젠트 공간 좌표계도 그에 따라 회전시켜야 합니다. 그 이유를 설명하세요. 특히 이는 세계 공간에서 **N**을 중심으로 **T**를 회전시켜야 함을 의미하며, 이는 비용이 많이 드는 삼각함수 계산(더 정확히 말하면 임의의 축 **N**을 중심으로 한 회전 변환)이 필요합니다. 다른 해결책은 세계 공간에서 탄젠트 공간으로 **T**를 변환한 후, 텍스처 변환 행렬을 직접 사용하여 **T**를 회전시키고 다시 세계 공간으로 변환하는 것입니다.
5. 월드 공간에서 조명을 처리하는 대신, 시점과 광원 벡터를 월드 공간에서 탄젠트 공간으로 변환하여 해당 공간에서 모든 조명 계산을 수행할 수 있습니다. 노멀 매핑 세이더를 수정하여 탄젠트 공간에서 조명 계산을 수행하도록 합니다.
6. 변위 매핑을 활용해 바다 파도를 구현할 수 있습니다. 평평한 버텍스 그리드 위에 두 개(또는 그 이상)의 높이맵을 서로 다른 속도와 방향으로 스크롤하는 방식입니다. 그리드의 각 버텍스에 대해 높이맵을 샘플링하고 높이를 합산합니다. 합산된 높이가 해당 시점의 버텍스 높이(즉, y좌표)가 됩니다. 하이츠맵을 스크롤함으로써 파도가 지속적으로 형성되고 사라지며 바다 파도의 환상을 만들어 냅니다([그림 18.11](#) 참조). 이 연습에서는 본 장에서 다운로드 가능한 두 개의 바다 파도 하이츠맵(및 해당 노멀 맵)을 사용하여 방금 설명한 바다 파도 효과를 구현하십시오([그림 18.12](#)). 파도를 자연스럽게 보이게 하는 몇 가지 팁은 다음과 같습니다:

하이트맵을 서로 다르게 적용하여 한 세트는 진폭이 큰 넓은 저주파 파도를 모델링하고, 다른 세트는 진폭이 작은 고주파의 작은 잔물결을 모델링할 수 있도록 합니다. 따라서 하이트맵에 대해 두 세트의 텍스처 좌표와 두 개의 텍스처 변환이 필요합니다.



그림 18.11. 높이맵, 노멀맵, 환경 매핑으로 모델링된 해양 파도.

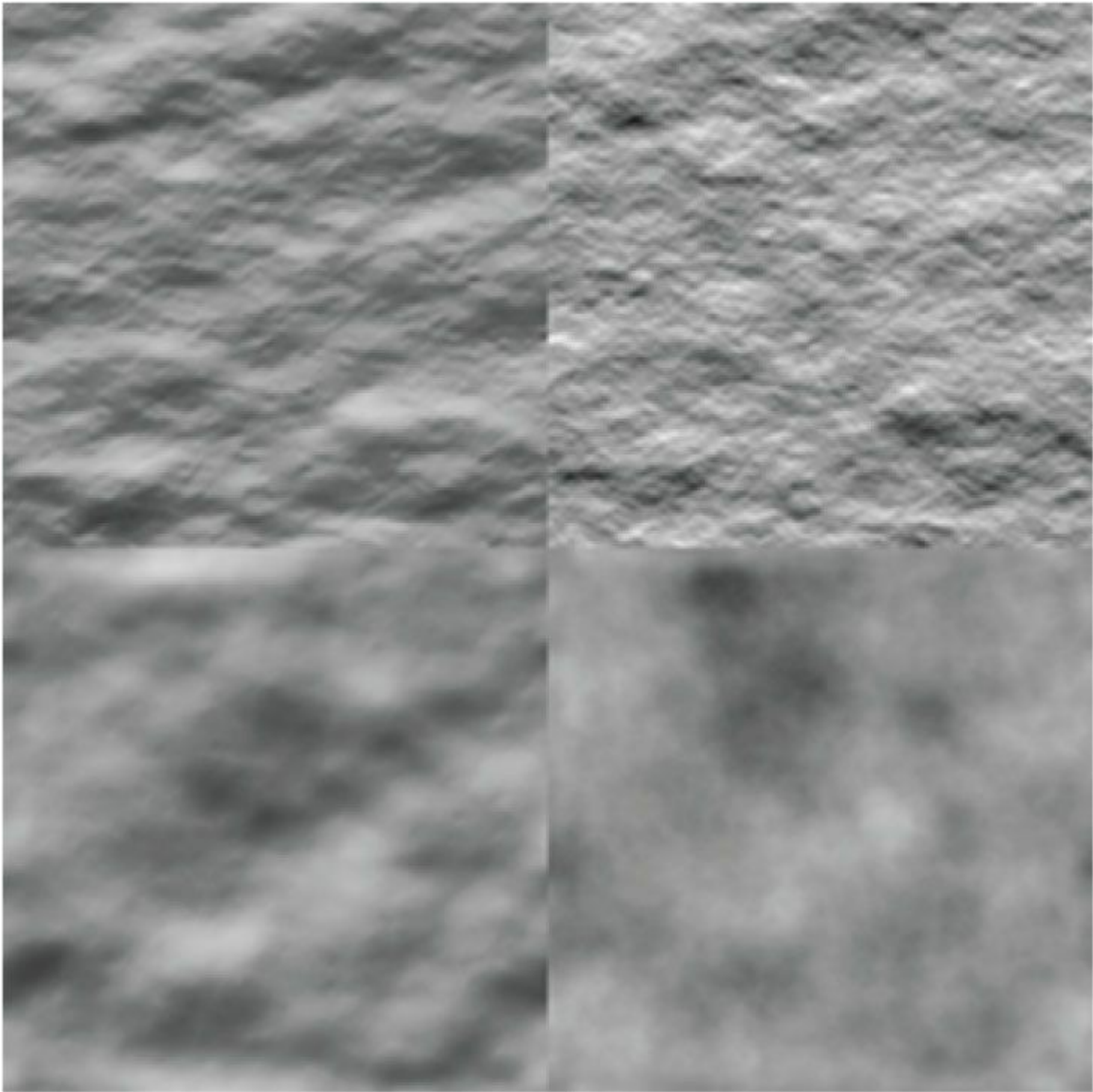


그림 18.12. (위쪽 행) 고주파의 거친 파도를 위한 해양 파도 노멀 맵과 하이트맵. (아래쪽 행) 저주파의 넓은 파도를 위한 해양 파도 노멀 맵과 하이트맵.

노멀 맵 텍스처는 하이트맵 텍스처보다 더 많이 타일링되어야 합니다. 하이트맵은 파도의 형태를 제공하며, 노멀 맵은 픽셀 단위로 파도를 조정하는 데 사용됩니다. 하이트맵과 마찬가지로 노멀 맵도 시간이 지남에 따라 그리고 다양한 방향으로 변환되어 새로운 파도가 생겨나고 사라지는 듯한 착시를 주어야 합니다. 그런 다음 다음과 유사한 코드를 사용하여 두 노멀을 결합할 수 있습니다:

```
float3 normalMapSample0 = gNormalMap0.Sample(samLinear, pin.WaveNormalTex0).rgb; float3 bumpedNormalW0 =
NormalSampleToWorldSpace(
    normalMapSample0, pin.NormalW, pin.TangentW);

float3 normalMapSample1 = gNormalMap1.Sample(samLinear, pin.WaveNormalTex1).rgb; float3 bumpedNormalW1 =
NormalSampleToWorldSpace(
    normalMapSample1, pin.NormalW, pin.TangentW);

float3 bumpedNormalW = normalize(bumpedNormalW0 + bumpedNormalW1);
```

파도의 재질을 바다색으로 더 강조하고 환경 맵에서 반사 효과를 일부 유지합니다.

지형 렌더링의 개념은 평평한 격자(그림 19.1 상단)로 시작하는 것입니다. 그런 다음 정점의 높이(즉, y좌표)를 조정하여 메쉬가 산에서 계곡으로의 부드러운 전환을 모델링하도록 하여 지형을 시뮬레이션합니다(그림 19.1 중간). 물론, 모래사장, 푸른 언덕, 바위 절벽, 눈 덮인 산을 표현하기 위해 멋진 텍스처를 적용합니다(그림 19.1 하단).

목표:

1. 산과 계곡 사이의 부드러운 전환을 구현하는 지형의 높이 정보를 생성하는 방법을 학습한다.
2. 지형에 텍스처를 적용하는 방법을 알아보기.
3. 하드웨어 테셀레이션을 적용하여 연속적인 디테일 수준으로 지형을 렌더링하는 방법.
4. 카메라나 다른 물체를 지형 표면에 고정시키는 방법을 발견하기 위해.

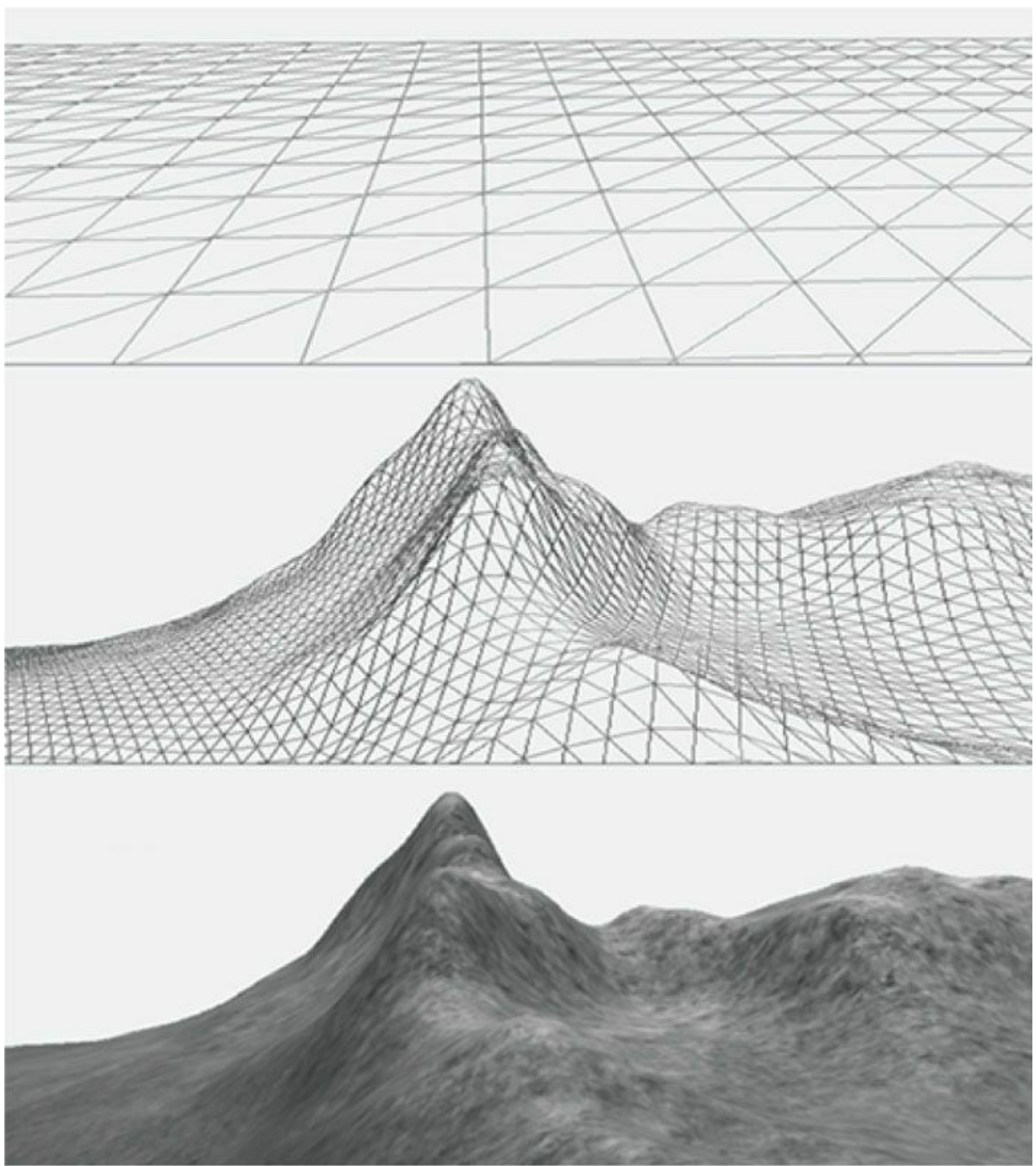


그림 19.1. (위) 삼각형 그리드. (가운데) 언덕과 계곡을 생성하기 위해 부드러운 높이 전환을 적용한 삼각형 그리드. (아래) 조명과 텍스처가 적용된 지형.

19.1 고도맵

지형의 언덕과 골짜기를 표현하기 위해 *높이맵*을 사용합니다. 높이맵은 행렬로, 각 요소는 지형 그리드 내 특정 정점의 높이를 지정합니다. 즉, 그리드 정점마다 높이맵에 항목이 존재하며, ij 번째 높이맵 항목은 ij 번째 정점의 높이를 제공합니다. 일반적으로 높이맵은 이미지 편집기에서 흑백 맵으로 그래픽 표현되며, 검정은 가장 낮은 높이를, 흰색은 가장 높은 높이를, 회색 음영은 그 사이의 높이를 나타냅니다.

[그림](#)

[19.2](#) 고도맵과 이를 통해 생성된 지형의 몇 가지 예시를 보여줍니다.

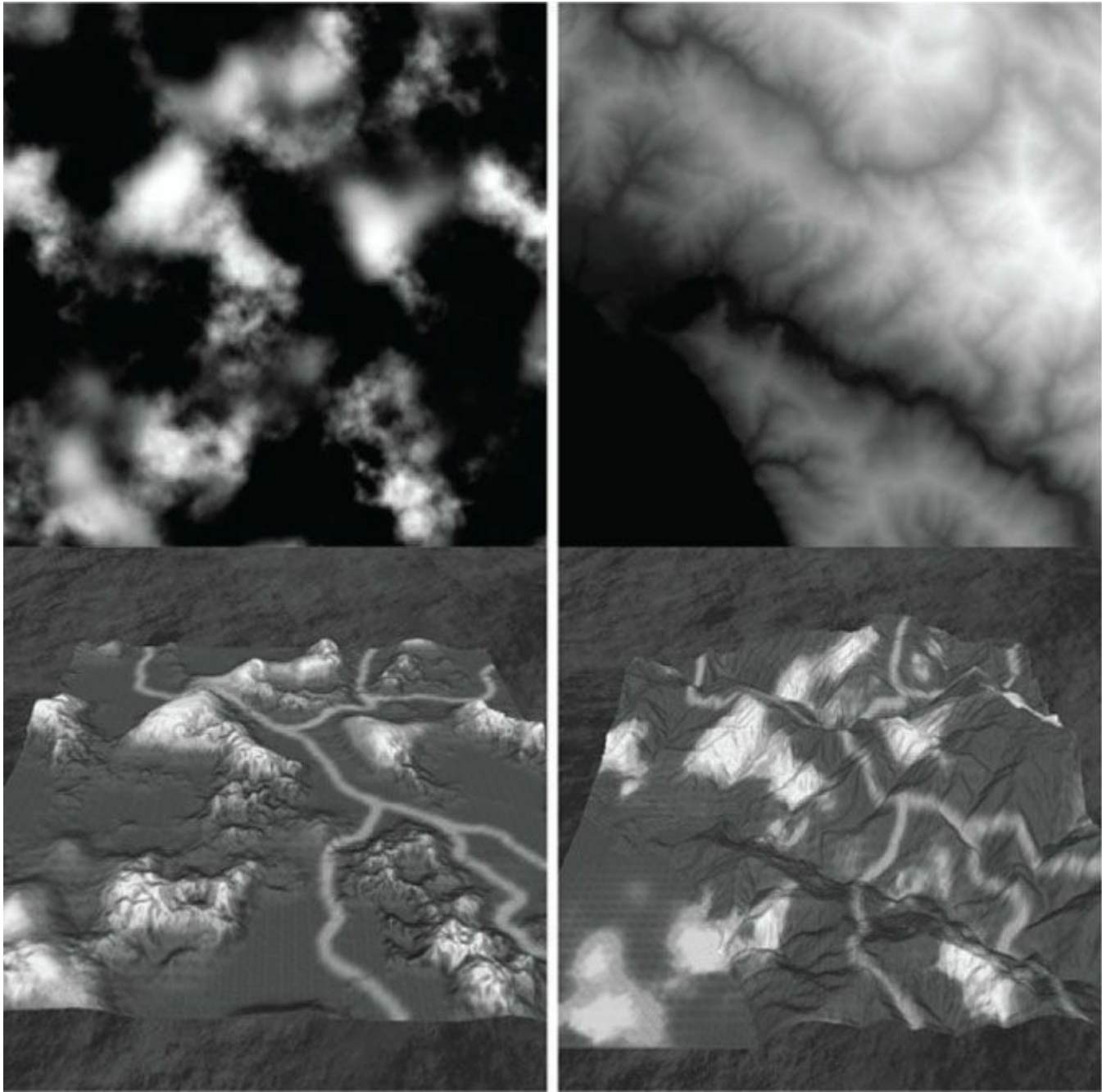


그림 19.2. 높이맵의 예시. 높이맵이 묘사하는 높이가 어떻게 서로 다른 지형 표면을 형성하는지 살펴보십시오.

디스크에 하이트맵을 저장할 때 일반적으로 하이트맵의 각 요소에 대해 1바이트의 메모리를 할당하므로 높이는 0부터 255까지의 범위를 가집니다. 0부터 255까지의 범위는 지형의 높이 간 전환을 표현하기에 충분하지만, 3D 세계의 스케일에 맞추기 위해 0부터 255 범위를 벗어나 스케일링해야 할 수도 있습니다. 예를 들어, 3D 세계의 측정 단위가 피트라면 0부터 255까지는 흥미로운 것을 표현하기에 충분한 값을 제공하지 못합니다. 이러한 이유로 데이터를 애플리케이션에 로드할 때 각 높이 요소에 대해 **부동 소수점(float)**을 할당합니다. 이를 통해 필요한 모든 스케일에 맞추기 위해 0부터 255 범위를 훨씬 벗어난 스케일링이 가능해집니다. 또한 높이맵을 필터링하고 정수값 사이의 높이 값을 생성할 수 있게 합니다.

§6.11에서는 수학적 함수를 사용하여 "지형"을 생성했습니다. 이는 절차적으로 생성된 지형()의 한 예시입니다. 그러나 원하는 지형을 정확히 묘사하는 함수를 고안해내는 것은 어렵습니다. 하이트맵(heightmap)은
는 화가나 높이맵 편집 도구를 통해 편집할 수 있기 때문에 더 많은 유연성을 제공합니다.

19.1.1 하이트맵 생성

하이트맵은 절차적으로 생성하거나 Adobe Photoshop과 같은 이미지 편집기에서 만들 수 있습니다. 페인트 필터를 사용하여 다양한 무질서한 하이트맵 패턴을 생성하는 것이 좋은 시작점이 될 수 있습니다. 그런 다음 페인트 편집기의 도구를 활용하여 하이트맵을 수동으로 조정할 수 있습니다. 블러 필터를 적용하면 하이트맵의 거친 가장자리를 부드럽게 하는 데 유용합니다.

Terragen 프로그램(<http://www.planetside.co.uk/>)은 절차적으로 높이맵을 생성할 수 있으며, (또는 높이맵을 내보낸 후 포토샵 같은 별도의 페인트 프로그램에서 가져와 수정할 수 있음). Bryce(<http://www.daz3d.com/i.x/software/bryce/>) 프로그램 역시 높이맵 생성을 위한 다양한 절차적 알고리즘을 보유하고 있으며

고도맵 생성 알고리즘과 내장 고도맵 편집기를 제공합니다. Dark Tree(<http://www.darksim.com/>)는 강력한 절차적 텍스처 제작 프로그램으로, 특히 그레이스케일 고도맵 생성에도 활용할 수 있습니다. 하이맵 작성을 완료한 후에는 8비트 RAW 파일로 저장해야 합니다. RAW 파일은 단순히 이미지를 차례로 불러옵니다. 이렇게 하면 프로그램에서 이미지를 쉽게 읽어올 수 있습니다. 사용 중인 소프트웨어에서 헤더를 포함한 RAW 파일 저장을 요청할 수 있습니다. 헤더를 지정하지 않도록 설정하세요. **그림 19.3**은 *Terragen*의 내보내기 대화 상자를 보여줍니다.

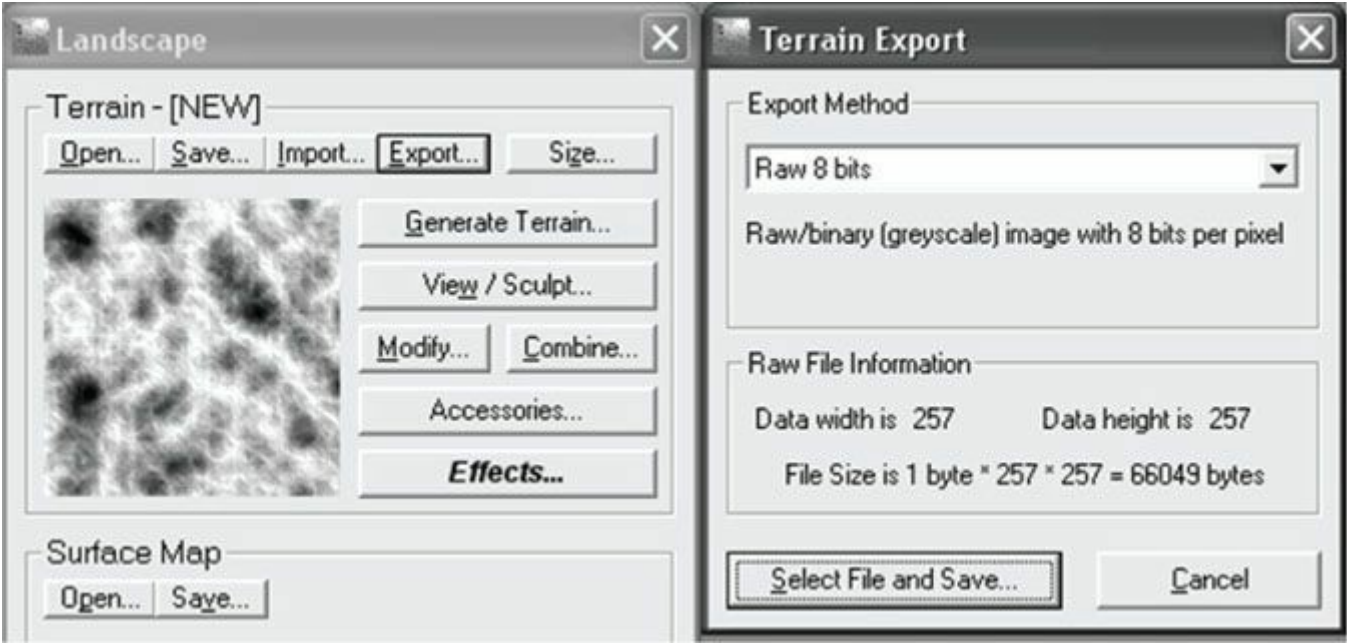


그림 19.3. (왼쪽) 지형 생성기를 사용하면 절차적으로 무작위 지형을 생성할 수 있으며 브러시 도구로 지형을 수동으로 조각할 수도 있습니다. (오른쪽) *Terragen*의 내보내기 대화 상자. 선택된 내보내기 방법이 8비트 RAW 형식임을 확인하십시오.

RAW 형식으로 높이맵을 저장할 필요는 없습니다. 필요에 맞는 어떤 형식이라도 사용하시면 됩니다. RAW 형식은 우리가 사용할 수 있는 형식 중 하나의 예시일 뿐입니다. RAW 형식을 선택한 이유는 많은 이미지 편집기가 이 형식으로 내보낼 수 있고, RAW 파일의 데이터를 프로그램 데모에 불러오는 것이 매우 쉽기 때문입니다. 이 책의 데모는 8비트 RAW 파일(즉, 높이맵의 각 요소가 8비트 정수임)을 사용합니다.

256 높이 단계가 너무 거칠다면, 각 높이 항목이 16비트 정수로 표현되는 16비트 높이맵 저장을 고려해 볼 수 있습니다. Terragen은 16비트 RAW 높이맵을 내보낼 수 있습니다.

19.1.2 RAW 파일 불러오기

RAW 파일은 연속된 바이트 블록(각 바이트가 높이맵 항목)에 불과하므로, 다음 메서드에서 수행하는 것처럼 `std::ifstream::read` 호출 하나로 메모리 블록을 쉽게 읽어올 수 있습니다:

```
void Terrain::LoadHeightmap()
{

    // 각 정점에 대한 높이 std::vector<unsigned char>
    in(
        mInfo.HeightmapWidth * mInfo.HeightmapHeight);

    // 파일 열기. std::ifstream inFile;
    inFile.open(mInfo.HeightMapFilename.c_str(), std::ios_base::binary);

    if(inFile)
    {

        // 원시 바이트를 읽습니다.
        inFile.read((char*)&in[0], (std::streamsize)in.size());

        // 파일 처리 완료.
```

```

        inFile.close();
    }

    // 배열 데이터를 부동 소수점 배열로 복사하고 크기를 조정합니다. mHeightmap.resize(mInfo.HeightmapHeight *
    mInfo.HeightmapWidth, 0); for(UINT i = 0; i < mInfo.HeightmapHeight * mInfo.HeightmapWidth; ++i)
    {
        mHeightmap[i] = (in[i] / 255.0f)*mInfo.HeightScale;
    }
}

```

Terrain 클래스의 멤버 변수인 mInfo는 지형의 다양한 속성을 설명하는 다음 구조체의 인스턴스입니다:

```

struct InitInfo
{
    // RAW 높이맵 데이터 파일명. std::wstring HeightMapFilename;

    // 지형 텍스처링에 사용되는 텍스처 파일명. std::wstring LayerMapFilename0;
    std::wstring LayerMapFilename1;    std::wstring
    LayerMapFilename2;                  std::wstring
    LayerMapFilename3;                  std::wstring
    LayerMapFilename4;                  std::wstring
    BlendMapFilename;

    // 높이맵에서 로드된 높이값에 적용할 스케일. float HeightScale;

    // 높이맵에서 로드된 후 적용할 높이 스케일.
    float HeightScale;

    // 높이맵의 크기. UINT HeightmapWidth;
    UINT HeightmapHeight;

    // x축 및 z축을 따라 셀 간격 (그림 (19.4) 참조). float CellSpacing;
};

```

 *그리드 생성에 대해서는 §5.11 을 참고하시기 바랍니다.*

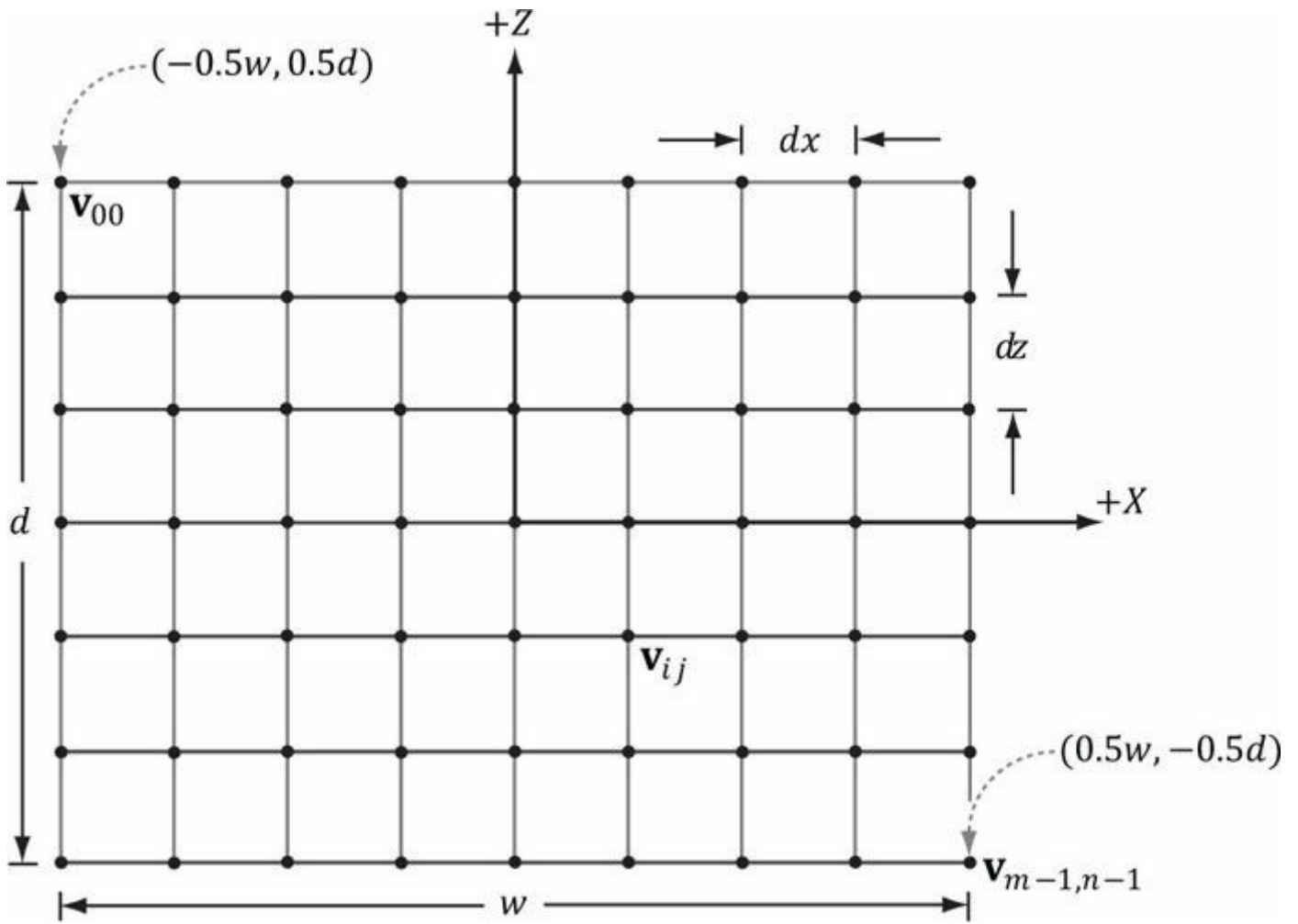


그림 19.4. 그리드 속성.

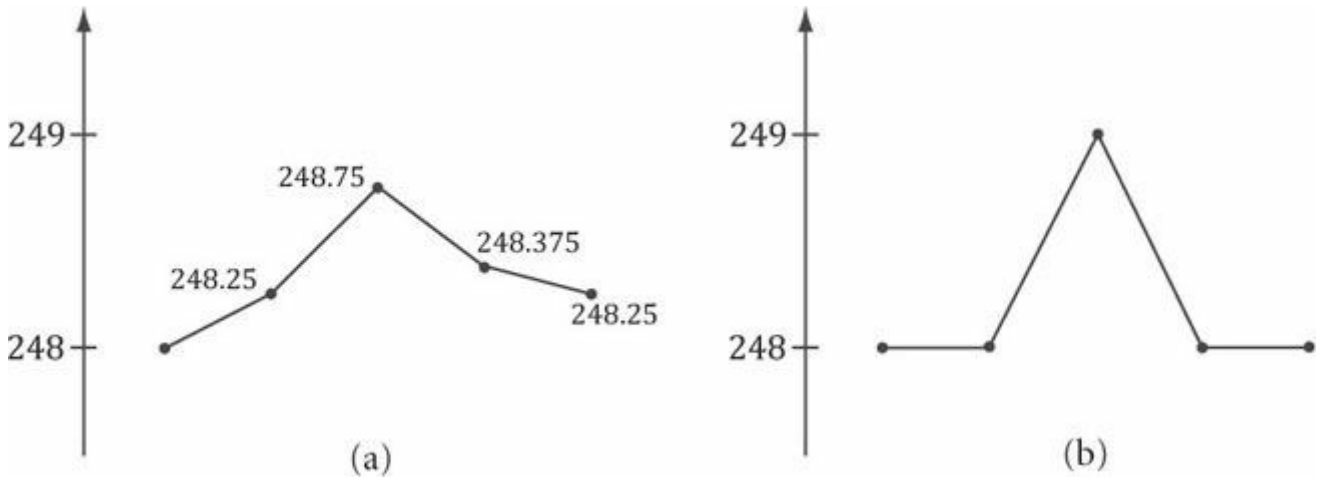


그림 19.5. (a) [0, 255] 범위의 부동 소수점 높이 값. (b) 가장 가까운 정수로 클램핑된 높이 값.

19.1.3 스무딩

8비트 높이맵 사용의 문제점 중 하나는 256개의 이산적인 높이 단계만 표현할 수 있다는 점입니다. 따라서 그림 19.5a에 표시된 높이 값을 모델링할 수 없으며, 대신 그림 19.5b와 같은 결과가 나옵니다. 이러한 절단은 의도하지 않은 "거친" 지형을 생성합니다. 물론, 일단 잘라내면 원래 높이 값을 복구할 수 없지만, 그림 19.5b를 평활화하면 19.5a에 더 가까운 결과를 얻을 수 있습니다.

따라서 우리가 하는 작업은 원시 바이트를 읽어 높이맵을 메모리에 로드하는 것입니다. 그런 다음 바이트 배열을 부동 소수점 배열로 복사하여 우리는 부동 소수점 정밀도를 사용합니다. 그런 다음 부동 소수점 높이맵에 필터를 적용하여 높이맵을 부드럽게 만듭니다. 이로 인해 인접한 요소들 사이의 높이 차이가 그렇게 극심하지 않게 됩니다. 우리가 사용하는 필터링 알고리즘은 상당히 기본적인입니다. 새로운 필터링된 높이맵 픽셀은 그 자체와 여덟 개의 이웃 픽셀을 평균하여 계산됩니다(그림 19.6):

$$\tilde{h}_{i,j} = \frac{h_{i-1,j-1} + h_{i-1,j} + h_{i-1,j+1} + h_{i,j-1} + h_{i,j} + h_{i,j+1} + h_{i+1,j-1} + h_{i+1,j} + h_{i+1,j+1}}{9}$$

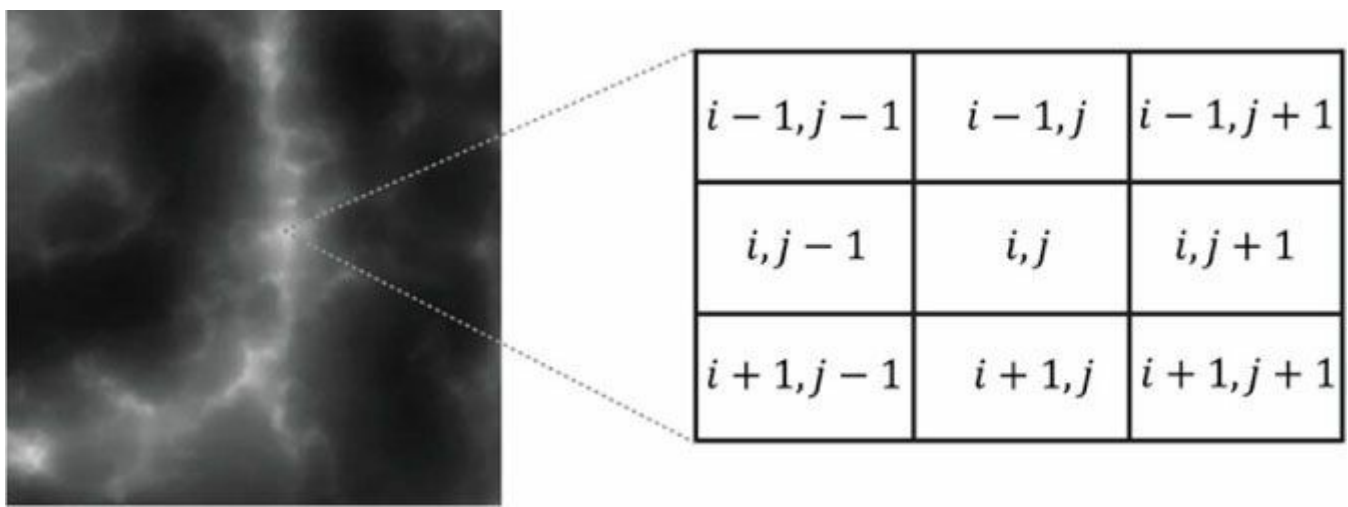


그림 19.6. i,j 번째 정점의 높이는 i,j 번째 높이맵 항목과 그 주변 8개 높이맵 항목의 평균값으로 구합니다.

높이맵의 가장자리에서 픽셀이 8개의 인접 픽셀을 갖지 않는 경우, 가능한 한 많은 인접 픽셀을 사용하여 평균을 구합니다.

하이츠맵에서 i 번째, j 번째 픽셀의 평균값을 계산하는 함수의 구현은 다음과 같습니다:

```
bool Terrain::InBounds(int i, int j)
{
    // ij가 유효한 인덱스일 경우 true, 그렇지 않으면 false를 반환합니다. return
    i >= 0 && i < (int)mInfo.HeightmapHeight &&
    j >= 0 && j < (int)mInfo.HeightmapWidth;
}

float Terrain::Average(int i, int j)
{
    // 함수는 ij 요소의 평균 높이를 계산합니다.
    // 해당 픽셀과 그 주변 8개 픽셀의 평균을 구합니다.
    // 인접 픽셀이 없는 경우 해당 픽셀은 평균 계산에 포함되지 않음
    // 즉, 가장자리 픽셀은 인접 픽셀이 없습니다.
    //
    // _____
    // | 1| 2| 3|
    // _____
    // |4 |ij| 6|
    // _____
    // | 7| 8| 9|
    // _____

    float avg = 0.0f; float num
    = 0.0f;

    // 음수를 허용하려면 int를 사용하세요. UINT를 사용하면 @ i=0, m=i-1=UINT_MAX
    // 외부 for 루프의 반복이 발생하지 않습니다. for(int m = i-1; m <= i+1;
    ++m)
    {
        for(int n = j-1; n <= j+1; ++n)
        {
            if(InBounds(m,n))
            {
                avg += mHeightmap[m*mInfo.HeightmapWidth + n]; num += 1.0f;
            }
        }
    }

    return avg / num;
}
```

```
    }
```

inBounds 함수는 해당 항목이 높이맵에 있을 경우 true를 반환하고 그렇지 않으면 false를 반환합니다. 따라서 높이맵에 속하지 않는 가장자리에서 항목에 인접한 요소를 샘플링하려고 시도하면 **inBounds**가 false를 반환하며, 우리는 이를 평균에 포함하지 않습니다

— 존재하지 않으므로 포함하지 않습니다.

전체 높이맵을 평활화하려면 각 높이맵 항목에 **평균**을 적용하기만 하면 됩니다:

```
void Terrain::Smooth()
{
    std::vector<float> dest(mHeightmap.size()); for(UINT i = 0; i <
mInfo.HeightmapHeight; ++i)
    {
        for(UINT j = 0; j < mInfo.HeightmapWidth; ++j)
        {
            dest[i*mInfo.HeightmapWidth+j] = Average(i,j);
        }
    }
    // 기존 하이트맵을 필터링된 하이트맵으로 대체합니다. mHeightmap = dest;
}
```

19.1.4 하이트맵 셰이더 리소스 뷰

다음 섹션에서 살펴보겠지만, 테셀레이션과 변위 매핑을 지원하려면 셰이더 프로그램에서 높이맵을 샘플링해야 합니다. 따라서 높이맵에 대한 셰이더 리소스 뷰를 생성해야 합니다. 이 작업은 이제 익숙할 것입
니다. 유일한 요령은 메모리를 절약하기 위해 32비트 부동소수점 대신 16비트 부동소수점을 사용한다는 점입니다. 32비트 부동 소수점을 16비트 부동 소수점으로 변환하려면 XNA 수학 함수
XMConvertFloatToHalf를 사용합니다.

```
void Terrain::BuildHeightmapSRV(ID3D11Device* device)
{
    D3D11_TEXTURE2D_DESC texDesc;
    texDesc.Width = mInfo.HeightmapWidth; texDesc.Height =
mInfo.HeightmapHeight; texDesc.MipLevels = 1;
    texDesc.ArraySize = 1;
    texDesc.Format = DXGI_FORMAT_R16_FLOAT;
    텍스처 설명.샘플 설명.카운트 = 1;

    텍스처 설명.샘플 설명.품질 = 0; 텍스처 설명.사용법 =
D3D11_USAGE_DEFAULT;
    텍스처 설명자.바인딩 플래그 = D3D11_BIND_SHADER_RESOURCE; 텍스처 설명자.CPU 액세스
스 플래그 = 0;

    텍스처 설명자.기타 플래그 = 0;

    // HALF는 xnamath.h에 정의되어 있으며, 16비트 부동소수점 값을 저장합니다.
    std::vector<HALF> hmap(mHeightmap.size()); std::transform(mHeightmap.begin(),
mHeightmap.end(),
                        hmap.begin(), XMConvertFloatToHalf);

    D3D11_SUBRESOURCE_DATA data;
    data.pSysMem = &hmap[0];
    data.SysMemPitch = mInfo.HeightmapWidth*sizeof(HALF); data.SysMemSlicePitch = 0;
    ID3D11Texture2D* hmapTex = 0;
    HR(device->CreateTexture2D(&texDesc, &data, &hmapTex));

    D3D11_SHADER_RESOURCE_VIEW_DESC srvDesc;
    srvDesc.Format = texDesc.Format;
    srvDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D; srvDesc.Texture2D.MostDetailedMip = 0;
    srvDesc.Texture2D.MipLevels = -1; HR(device-
>CreateShaderResourceView(
                        hmapTex, &srvDesc, &mHeightMapSRV));
}
```

```
// SRV는 참조를 저장합니다.  
ReleaseCOM(hmapTex);  
}
```

19.2 지형 테셀레이션

지형은 넓은 영역을 차지하므로 이를 구성하는 데 필요한 삼각형 수가 많습니다. 일반적으로 지형에는 *디테일 수준*(LOD) 시스템이 필요합니다. 즉, 카메라에서 멀리 떨어진 지형 부분은 디테일이 눈에 띄지 않으므로 그만큼 많은 삼각형이 필요하지 않습니다. [그림 19.7](#)을 참조하십시오.

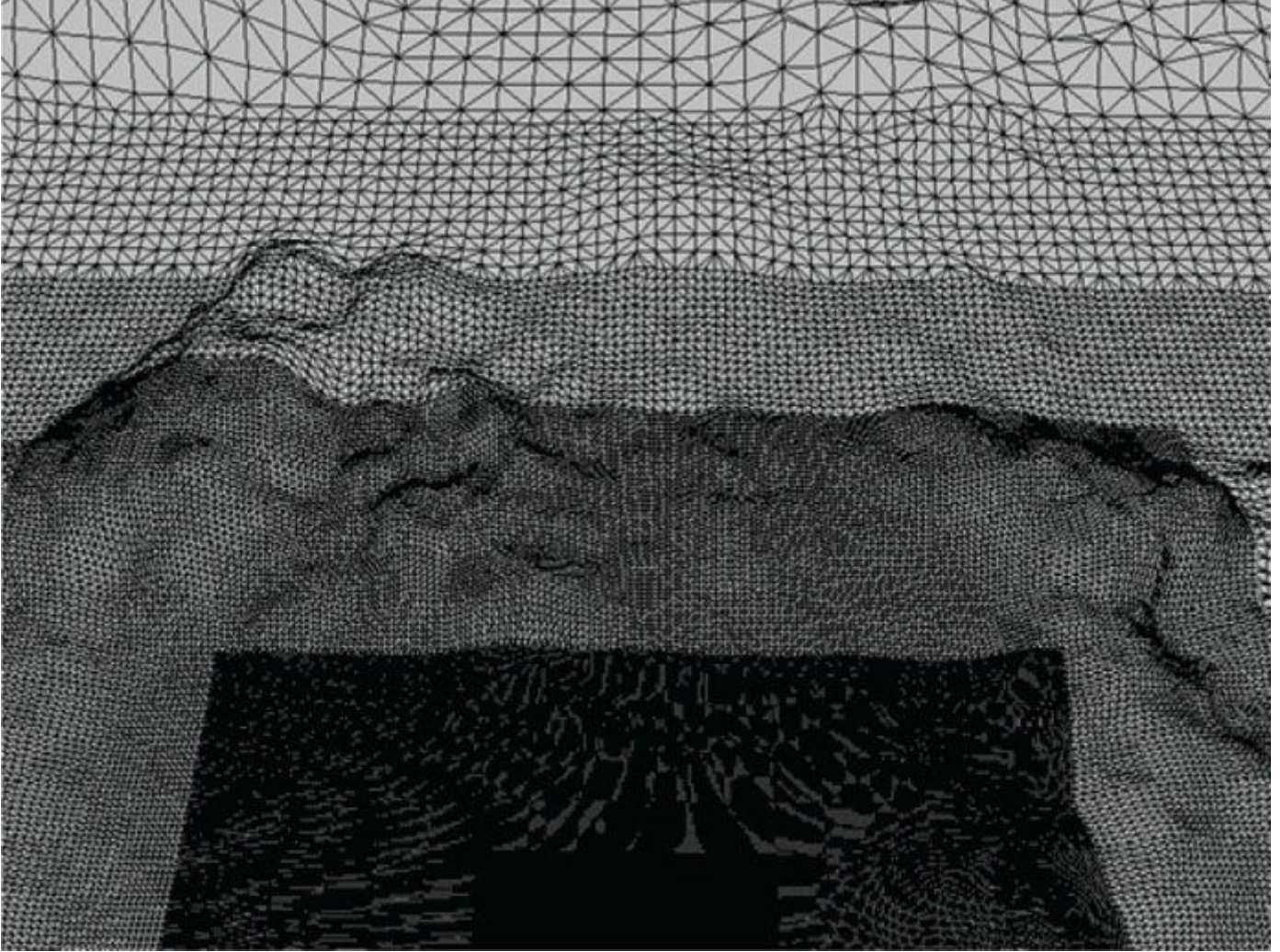


그림 19.7. 카메라로부터의 거리에 따라 디테일 수준이 감소합니다. 지형 테셀레이션을 위한 우리의 전략은 다음과 같습니다:

1. 사각 패치 그리드를 배치합니다.
2. 패치를 카메라로부터의 거리에 따라 테셀레이션합니다.
3. 하이츠맵을 셰이더 리소스로 바인딩합니다. 도메인 셰이더에서 생성된 정점들을 적절한 높이로 오프셋하기 위해 하이츠맵으로부터 변위 매핑을 수행합니다.

19.2.1 그리드 생성

하이츠맵의 크기가 $(2^m + 1) \times (2^n + 1)$ 이라고 가정합니다. 이 하이츠맵으로 생성할 수 있는 가장 상세한 지형은 $(2^m + 1) \times (2^n + 1)$ 개의 정점으로 구성된 그리드입니다. 따라서 이는 최대 테셀레이션 지형 그리드 (T_0 이라 칭함)를 나타냅니다. T_0 의 정점 간 셀 간격은 **InitInfo::CellSpacing** 속성(§9.1.2)으로 주어집니다. 즉, 셀을 언급할 때는

가장 세분화된 격자 T_0 의 셀을 의미합니다.

지형을 패치 격자로 분할하여 각 패치가 T_0 의 65×65 정점 블록을 덮도록 합니다([그림 19.8](#) 참조).

65를 선택한 이유는 최대 테셀레이션 계수가 64이기 때문입니다. 따라서 패치를 64×64 셀로 분할할 수 있으며, 이 셀들은 65×65 정점과 교차합니다. 따라서 패치가 최대 테셀레이션되면 생성된 모든 정점에 필요한 높이맵 정보를 갖게 됩니다. 패치의 테셀레이션 계수가 1인 경우, 해당 패치는 세분화되지 않고 단순히 두 개의 삼각형으로 렌더링됩니다. 따라서 패치 격자는

지형의 가장 거칠게 테셀레이션된 버전으로 간주할 수 있습니다.

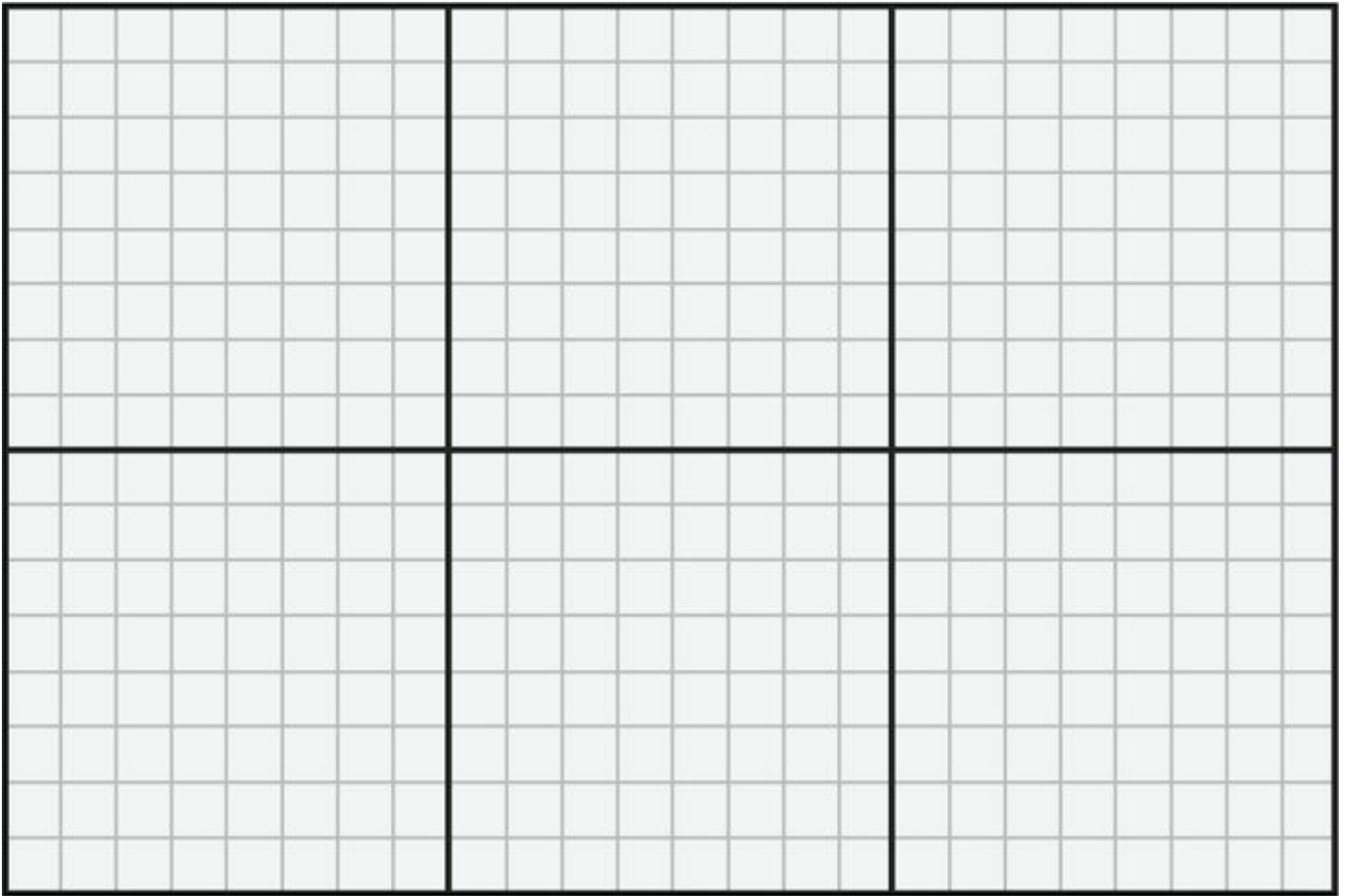


그림 19.8. 설명을 위해 더 작은 숫자를 사용합니다. 최대 테셀레이션 지형 그리드는 17×25 개의 정점과 16×24 개의 셀을 가집니다. 각 패치가 8×8 개의 셀 또는 9×9 개의 정점을 덮도록 그리드를 패치 그리드로 분할합니다. 이는 2×3 의 패치 그리드를 유도합니다.

패치 정점 그리드 크기는 다음과 같이 계산됩니다:

```
static const int CellsPerPatch = 64;
```

```
// 높이맵을 각 패치가 CellsPerPatch를 갖도록 패치로 분할합니다. mNumPatchVertRows = ((mInfo.HeightmapHeight-1) / CellsPerPatch) + 1;  
mNumPatchVertCols = ((mInfo.HeightmapWidth-1) / CellsPerPatch) + 1;
```

그리고 패치 정점과 쿼드 패치 프리미티브의 총 개수는 다음과 같이 계산됩니다:

```
mNumPatchVertices = mNumPatchVertRows*mNumPatchVertCols; mNumPatchQuadFaces = (mNumPatchVertRows-1)*(mNumPatchVertCols-1);
```

지형 패치 버텍스 구조는 다음과 같습니다:

```
struct Terrain  
{  
    XMFLOAT3 Pos;  
    XMFLOAT2 Tex;  
    XMFLOAT2 BoundsY;  
};
```

BoundsY 속성은 §19.3에서 설명됩니다. 쿼드 패치 버텍스 및 인덱스 버퍼를 생성하는 코드는 다음과 같습니다:

```
float Terrain::GetWidth()const  
{  
    // 총 지형 너비.  
    return (mInfo.HeightmapWidth-1)*mInfo.CellSpacing;  
}
```

```

float Terrain::GetDepth()const
{
    // 총 지형 깊이.
    return (mInfo.HeightmapHeight-1)*mInfo.CellSpacing;
}

void Terrain::BuildQuadPatchVB(ID3D11Device* device)
{
    std::vector<Vertex::Terrain> patchVertices(mNumPatchVertRows*mNumPatchVertCols);

    float halfWidth = 0.5f*GetWidth(); float halfDepth =
    0.5f*GetDepth();

    float patchWidth = GetWidth() / (mNumPatchVertCols-1); float patchDepth = GetDepth()
    / (mNumPatchVertRows-1); float du = 1.0f / (mNumPatchVertCols-1);
    float dv = 1.0f / (mNumPatchVertRows-1);

    for(UINT i = 0; i < mNumPatchVertRows; ++i)
    {
        float z = halfDepth - i*patchDepth;
        for(UINT j = 0; j < mNumPatchVertCols; ++j)
        {
            float x = -halfWidth + j*patchWidth; patchVertices[i*mNumPatchVertCols+j].Pos = XMFLOAT3(x, 0.0f, z);

            // 텍스처를 그리드에 늘림. patchVertices[i*mNumPatchVertCols+j].Tex.x = j*du;
            patchVertices[i*mNumPatchVertCols+j].Tex.y = i*dv;
        }
    }

    // 좌표축에 정렬된 바운딩 박스의 y 경계를 패치 좌측 상단 모서리에 저장합니다. for(UINT i = 0; i <
    mNumPatchVertRows-1; ++i)
    {
        for(UINT j = 0; j < mNumPatchVertCols-1; ++j)
        {
            UINT patchID = i*(mNumPatchVertCols-1)+j; patchVertices[i*mNumPatchVertCols+j].BoundsY =
            mPatchBoundsY[patchID];
        }
    }

    D3D11_BUFFER_DESC vbd;
    vbd.Usage = D3D11_USAGE_IMMUTABLE;
    vbd.ByteWidth = sizeof(Vertex::Terrain) * patchVertices.size(); vbd.BindFlags =
    D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = 0;

    vbd.MiscFlags = 0;
    vbd.StructureByteStride = 0;

    D3D11_SUBRESOURCE_DATA vinitData; vinitData.pSysMem =
    &patchVertices[0];
    HR(device->CreateBuffer(&vbd, &vinitData, &mQuadPatchVB));
}

void Terrain::BuildQuadPatchIB(ID3D11Device* device)
{
    std::vector<USHORT> indices(mNumPatchQuadFaces*4); // 4개의 패치 사면당 인덱스

    // 각 쿼드를 순회하며 인덱스 계산 int k = 0;
    for(UINT i = 0; i < mNumPatchVertRows-1; ++i)
    {

```

```

for(UINT j = 0; j < mNumPatchVertCols-1; ++j)
{
    // 2x2 쿼드 패치의 최상단 행 indices[k] =
    i*mNumPatchVertCols+j;
    indices[k+1] = i*mNumPatchVertCols+j+1;

    // 2x2 쿼드 패치의 하단 행
    indices[k+2] = (i+1)*mNumPatchVertCols+j; indices[k+3] =
    (i+1)*mNumPatchVertCols+j+1; k += 4; // 다음 쿼드
}
}

D3D11_BUFFER_DESC ibd;
ibd.Usage = D3D11_USAGE_IMMUTABLE;
ibd.ByteWidth = sizeof(USHORT) * indices.size(); ibd.BindFlags =
D3D11_BIND_INDEX_BUFFER; ibd.CPUAccessFlags = 0;
ibd.MiscFlags = 0;
ibd.StructureByteStride = 0;

D3D11_SUBRESOURCE_DATA iinitData; iinitData.pSysMem =
&indices[0];
HR(device->CreateBuffer(&ibd, &iinitData, &mQuadPatchIB));
}

```

19.2.2 지형 버텍스 셰이더

테셀레이션을 사용하기 때문에 버텍스 셰이더는 제어점별로 작동합니다. 우리의 버텍스 셰이더는 패치 제어점에 대해 높이맵 값을 읽어 디스플래스먼트 매핑을 수행한다는 점을 제외하면 거의 단순한 통과형 셰이더입니다. 이는 제어점의 y좌표를 적절한 높이에 위치시킵니다. 이렇게 하는 이유는 헐 셰이더에서 각 패치와 시점 사이의 거리를 계산할 예정이기 때문입니다. 패치 모서리를 적절한 높이로 오프셋하면 패치를 xz 평면에 배치하는 것보다 이 거리 계산이 더 정확해집니다.

```

Texture2D gHeightMap; SamplerState

samHeightmap
{
    Filter = MIN_MAG_LINEAR_MIP_POINT;

    AddressU = CLAMP; AddressV = CLAMP;
};

struct VertexIn
{
    float3 PosL          : POSITION; float2 텍스
                          : TEXCOORD0; float2
    BoundsY : TEXCOORD1;
};

struct VertexOut
{
    float3 PosW          : 위치; float2 텍스처
                          : 텍스처 좌표0; float2
    BoundsY : 텍스처 좌표1;
};

VertexOut VS(VertexIn vin)
{

```

```
VertexOut vout;

// 지형은 세계 좌표계에서 직접 지정됨. vout.PosW = vin.PosL;

// 패치 모서리를 월드 공간으로 변위시킵니다. 이는
// 패치까지의 거리 계산 정확도를 높이기 위함.
vout.PosW.y = gHeightMap.SampleLevel(samHeightmap, vin.Tex, 0).r;

// 버텍스 속성을 다음 단계로 출력합니다. vout.Tex = vin.Tex;

vout.BoundsY = vin.BoundsY;

return vout;
}
```

19.2.3 테셀레이션 계수

헐 셰이더 상수 함수는 각 패치에 대한 테셀레이션 계수를 계산하여 각 패치를 얼마나 세분화할지 결정합니다. 또한 헐 셰이더 상수 함수를 사용하여 GPU에서 프러스텀 컬링을 수행할 수 있습니다. GPU 기반 패치 프러스텀 컬링은 §19.3에서 설명하겠습니다.

우리는 눈 위치와 각 패치 가장자리의 중간점 사이의 거리를 계산하여 가장자리 테셀레이션 계수를 도출합니다.
내부 테셀레이션 계수의 경우, 눈 위치와 패치 중간점 사이의 거리를 계산합니다. 거리를 기반으로 테셀레이션 계수를 도출하기 위해 다음 코드를 사용합니다:

```
// 거리가 최소일 때 테셀레이션은 최대입니다.

// 거리가 최대일 때, 테셀레이션은 최소입니다. float gMinDist;

float gMaxDist;

// 2의 거듭제곱 테셀레이션을 위한 지수. 테셀레이션
// 범위는 [2^(gMinTess), 2^(gMaxTess)]입니다. 최대
// 테셀레이션은 64이므로, gMaxTess는 최대 64가 될 수 있습니다.
// (2^6 = 64)이며, gMinTess는 최소 0이어야 함 (2^0 = 1). float gMinTess;

float gMaxTess;

float CalcTessFactor(float3 p)
{

    float d = distance(p, gEyePosW);

    float s = saturate((d - gMinDist) / (gMaxDist - gMinDist)); return pow(2, (lerp(gMaxTess,

    gMinTess, s)));
}
```

이 공식은 [18장에서](#) 변위 매핑을 수행할 때 사용한 공식과 다릅니다. 2의 거듭제곱을 사용하는 이유는 각 세부 수준에서 세분화 수가 두 배로 증가함을 의미하기 때문입니다. 예를 들어, 세부 수준이 $2^3 = 8$ 이라
고 가정해 보겠습니다.

다음 정밀화 단계에서는 세분화 횟수가 $2^4 = 16$ 으로 두 배가 되며, 더 낮은 디테일 레벨은 세분화 횟수가 절반인 $2^2 = 4$ 가 됩니다. 2의 거듭제곱 함수를 사용하면 거리와 함께 디테일 레벨이 더 잘 분산됩니다.
이제 상수 헐 셰이더 함수에서, 패치 중심점과 패치 가장자리 중심점에 이 함수를 적용하여
테셀레이션 계수를 계산합니다:

```
struct PatchTess
{

    float EdgeTess[4] : SV_TessFactor;
    float InsideTess[2] : SV_InsideTessFactor;
};

PatchTess ConstantHS(InputPatch<VertexOut, 4> patch,
    uint patchID : SV_PrimitiveID)
```

```
{
    PatchTess pt;

    //
    // 프러스텀 컬링
    //

    [... 프러스텀 컬링 코드 생략]

    //
    // 거리에 기반한 일반적인 테셀레이션을 수행합니다.
    // else
    {
        // 테셀레이션 계수 계산이 중요합니다
        // 가장자리 속성을 기반으로 하여
        // 하나 이상의 패치에 의해 공유되는 예지는 동일한
        // 테셀레이션 계수를 동일하게 유지합니다. 그렇지 않으면 틀이 생길 수 있습니다.

        // 예지의 중간점과 패치 중심점 계산 float3 e0 = 0.5f*(patch[0].PosW +
        patch[2].PosW); float3 e1 = 0.5f*(patch[0].PosW + patch[1].PosW); float3 e2 =
        0.5f*(patch[1].PosW + patch[3].PosW); float3 e3 = 0.5f*(patch[2].PosW +
        patch[3].PosW); float3 c = 0.25f*(patch[0].PosW + patch[1].PosW +
        patch[2].PosW + patch[3].PosW);

        pt.EdgeTess[0] = CalcTessFactor(e0); pt.EdgeTess[1] =
        CalcTessFactor(e1);          pt.EdgeTess[2] =
        CalcTessFactor(e2);          pt.EdgeTess[3] =
        CalcTessFactor(e3);

        pt.InsideTess[0] = CalcTessFactor(c); pt.InsideTess[1] =
        pt.InsideTess[0];

        return pt;
    }
}
```

19.2.4 변위 매핑

도메인 셰이더는 테셀레이션을 위한 버텍스 셰이더와 유사하다는 점을 상기하십시오. 도메인 셰이더는 생성된 각 버텍스에 대해 평가됩니다. 도메인 셰이더에서의 우리의 작업은 테셀레이션된 버텍스 위치의 파라메트릭 (u, v) 좌표를 사용하여 제어점 데이터를 보간하여 실제 버텍스 위치와 텍스처 좌표를 도출하는 것입니다. 또한 높이맵을 샘플링하여 변위 매핑을 수행합니다.

```
struct DomainOut
{
    float4 PosH          : SV_POSITION; float3
    PosW                  : 위치; float2 텍스 :
    TEXCOORD0; float2 TiledTex : TEXCOORD1;
};

// 텍스처 레이어를 타일링할 비율. float2 gTexScale = 50.0f;

[domain("quad")]
DomainOut DS(PatchTess patchTess,
              float2 uv : SV_DomainLocation,
              const OutputPatch<HullOut, 4> quad)
{
    DomainOut dout;
```

```

// 이차 보간. dout.PosW = lerp(
    lerp(quad[0].PosW, quad[1].PosW, uv.x),
    lerp(quad[2].PosW, quad[3].PosW, uv.x), uv.y);

dout.Tex = lerp(
    lerp(quad[0].Tex, quad[1].Tex, uv.x),
    lerp(quad[2].Tex, quad[3].Tex, uv.x), uv.y);

// 지형 위에 레이어 텍스처를 타일링합니다. dout.TiledTex =
dout.Tex*gTexScale;

// 변위 매핑
dout.PosW.y = gHeightMap.SampleLevel(samHeightmap, dout.Tex, 0).r;

// 참고: 도메인 셰이더에서 노멀을 계산하려고 시도했으나
// 유한 차분법을 사용했지만 정점들이 지속적으로 이동합니다
// fractional_even은 눈에 띄는 빛의 반짝임을 생성합니다
// 따라서, 우리는
// 계산을 픽셀 셰이더로 옮겼습니다.

// 균일 클립 공간으로 투영합니다.
dout.PosH = mul(float4(dout.PosW, 1.0f), gViewProj);

return dout;
}

```

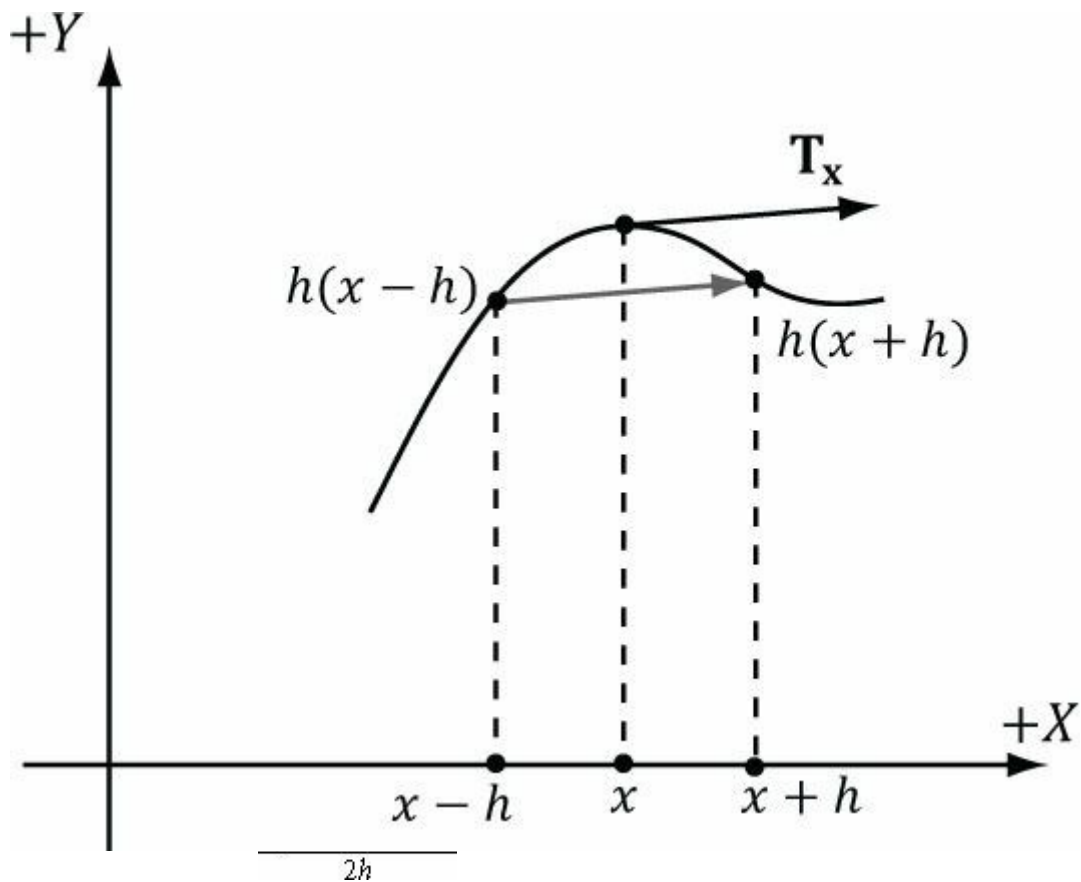


그림 19.9. 중앙 차분.

x 방향 접선 벡터의 기울기를 제공합니다. 이 차분을 점 x 에서의 접선 벡터 추정값으로 사용합니다.

19.2.5 접선 및 법선 벡터 추정

픽셀 셰이더에서 높이맵을 활용해 중앙 차분(그림 19.9 참조)을 통해 접선 벡터($+x$ 방향과 $-x$ 방향 각 1개)를 실시간으로 추정합니다:

$$\mathbf{T_x}(x,z)=\left(1,\frac{\partial h}{\partial x},0\right)\approx\left(1,\frac{h(x+h,z)-h(x-h,z)}{2h},0\right)$$

$$\mathbf{T_{-z}}(x,z)=\left(0,-\frac{\partial h}{\partial z},-1\right)\approx\left(0,\frac{h(x,z-h)-h(x,z+h)}{2h},-1\right)$$

텍스처 공간의 v 축에 해당하므로 z 축의 음의 방향을 취합니다. 따라서 이 벡터들은 노멀 매핑을 위한 탄젠트 공간 형성에도 기여합니다. 양의 x 방향과 음의 z 방향의 탄젠트 벡터 추정값을 얻으면, 외적 연산을 통해 법선 벡터를 계산합니다:

```
// 정규화된 UV 공간 [0,1] 내 높이 값 간 간격. float gTexelCellSpaceU;
float gTexelCellSpaceV;

// 월드 공간에서의 셀 간 간격. float gWorldCellSpace;

//
// 중앙 차분을 사용하여 정상과 접선을 추정합니다.
//
float2 leftTex = pin.Tex + float2(-gTexelCellSpaceU, 0.0f); float2 rightTex = pin.Tex +
float2(gTexelCellSpaceU, 0.0f); float2 bottomTex = pin.Tex + float2(0.0f, gTexelCellSpaceV);
float2 topTex = pin.Tex + float2(0.0f, -gTexelCellSpaceV);

float leftY = gHeightMap.SampleLevel(samHeightmap, leftTex, 0).r; float rightY =
gHeightMap.SampleLevel(samHeightmap, rightTex, 0).r;
float bottomY = gHeightMap.SampleLevel(samHeightmap, bottomTex, 0).r; float topY =
gHeightMap.SampleLevel(samHeightmap, topTex, 0).r;

float3 tangent = normalize( float3(2.0f*gWorldCellSpace, rightY - leftY, 0.0f));
float3 bitan = normalize(
float3(0.0f, bottomY - topY, -2.0f*gWorldCellSpace)); float3 normalW = cross(tangent,
bitan);
```

도메인 셰이더에서 중앙 차분법을 사용해 법선을 계산해 보았으나, **fractional_even** 테셀레이션으로 인해 정점들이 지속적으로 이동하면서 법선이 변할 때 눈에 띄는 빛 번짐 현상이 발생했습니다. 따라서 계산 작업을 픽셀 셰이더로 옮겼습니다. 셰이더 내에서 법선 벡터를 계산하는 대신, 오프라인에서 하이트맵으로부터 법선 맵을 생성하는 방법도 있습니다. 이는 GPU 계산 시간을 메모리 사용량과 교환하는 방식입니다. 텍스처 맵을 사용하기로 결정했다면, $\frac{\partial h / \partial x}{\|\mathbf{T_x}\|}$ 와 $\frac{\partial h / \partial z}{\|\mathbf{T_{-z}}\|}$ 만 저장하여 메모리를 절약할 수 있습니다. 그런 다음 셰이더에서 $\frac{\mathbf{T_x}}{\|\mathbf{T_x}\|}$ 와 $\frac{\mathbf{T_{-z}}}{\|\mathbf{T_{-z}}\|}$ 및 법선 벡터를 재구성할 수 있습니다. 이는 요소 당 2바이트가 필요하며, 법선 벡터 외에도 탄젠트 벡터를 함께 얻을 수 있습니다.

19.3 프러스텀 컬링 패치

지형은 일반적으로 광대한 영역을 차지하며, 카메라에 포착되지 않는 패치들이 다수 존재합니다. 이는 프러스텀 컬링이 효과적인 최적화 기법이 될 것임을 시사합니다. 패치의 테셀레이션 계수가 모두 0인 경우, GPU는 해당 패치를 추가 처리에서 제외합니다. 이는 삼각형들이 나중에 클리핑 단계에서 제거될 것임을 알면서도 패치를 테셀레이션하는 데 노력을 낭비하지 않도록 합니다.

프러스트럼 컬링을 수행하려면 두 가지 요소가 필요합니다. 뷰 프러스트럼 평면과 각 패치에 대한 바운딩 볼륨이 필요합니다.

각 패치에 대한 경계 볼륨이 필요합니다. [제15장](#) 연습문제 2에서는 뷰 프러스텀 평면을 추출하는 방법을 설명했습니다. 이 연습문제의 해결책을 구현한 코드는 다음과 같습니다(*d3dUtil.h/d3dUtil.cpp* [0](#)// 구현됨):

```
void ExtractFrustumPlanes(XMFLOAT4 planes[6], CXMMATRIX M)
{
    //
    // 왼쪽
    //
```

```

    planes[0].x = M(0,3) + M(0,0);
    planes[0].y = M(1,3) + M(1,0);
    planes[0].z = M(2,3) + M(2,0);
    planes[0].w = M(3,3) + M(3,0);

//
// 우측
//
planes[1].x = M(0,3) - M(0,0);
planes[1].y = M(1,3) - M(1,0);
planes[1].z = M(2,3) - M(2,0);
planes[1].w = M(3,3) - M(3,0);

//
// 바닥면
//
planes[2].x = M(0,3) + M(0,1);
planes[2].y = M(1,3) + M(1,1);
planes[2].z = M(2,3) + M(2,1);
planes[2].w = M(3,3) + M(3,1);

//
// 상단
//
planes[3].x = M(0,3) - M(0,1);
planes[3].y = M(1,3) - M(1,1);
planes[3].z = M(2,3) - M(2,1);
planes[3].w = M(3,3) - M(3,1);

//
// 근접
//
planes[4].x = M(0,2);
planes[4].y = M(1,2);
planes[4].z = M(2,2);
planes[4].w = M(3,2);

//
// 원거리
//
planes[5].x = M(0,3) - M(0,2);
planes[5].y = M(1,3) - M(1,2);
planes[5].z = M(2,3) - M(2,2);
planes[5].w = M(3,3) - M(3,2);

// 평면 방정식 정규화. for(int i = 0; i < 6; ++i)
{
    XMVECTOR v = XMPlaneNormalize(XMLoadFloat4(&planes[i])); XMStoreFloat4(&planes[i], v);
}

```

다음으로 각 패치에 대한 경계 볼륨이 필요합니다. 각 패치는 직사각형이므로 축에 정렬된 경계 상자를 경계 볼륨으로 선택합니다. 패치 제어점은 본질적으로 x-z 평면에서 직사각형으로 패치를 구성하기 때문에 x 및 z 좌표 경계를 인코딩합니다. y 좌표 경계는 어떻게 될까요? y 좌표 경계를 얻으려면 사전 처리 단계를 수행해야 합니다. 각 패치는 높이맵의 65×65 요소를 덮습니다. 따라서 각 패치에 대해 해당 패치가 덮는 높이맵 항목들을 스캔하고 최소 및 최대 y좌표를 계산합니다. 그런 다음 이 값들을 패치의 왼쪽 상단 제어점에 저장하여 상수 텍 세이더에서 패치의 y 경계값에 접근할 수 있도록 합니다. 다음 코드는 각 패치의 y 경계값을 계산하는 과정을 보여줍니다:

```

// x-stores minY, y-stores maxY.

```

```
std::vector<XMFLOAT2> mPatchBoundsY;
```

```
void Terrain::CalcAllPatchBoundsY()
```

```
{
    mPatchBoundsY.resize(mNumPatchQuadFaces);

    // 각 패치에 대해
    for(UINT i = 0; i < mNumPatchVertRows-1; ++i)
    {
        for(UINT j = 0; j < mNumPatchVertCols-1; ++j)
        {
            CalcPatchBoundsY(i, j);
        }
    }
}
```

```
void Terrain::CalcPatchBoundsY(UINT i, UINT j)
```

```
{
    // 이 패치가 포함하는 높이맵 값을 스캔하고
    // 최소/최대 높이를 계산합니다.

    UINT x0 = j*CellsPerPatch; UINT x1 =
(j+1)*CellsPerPatch;

    UINT y0 = i*CellsPerPatch; UINT y1 =
(i+1)*CellsPerPatch;

    float minY = +MathHelper::Infinity; float maxY = -
MathHelper::Infinity; for(UINT y = y0; y <= y1; ++y)
    {
        for(UINT x = x0; x <= x1; ++x)
        {
            UINT k = y*mInfo.HeightmapWidth + x;
            minY = MathHelper::Min(minY, mHeightmap[k]); maxY =
MathHelper::Max(maxY, mHeightmap[k]);
        }
    }

    UINT patchID = i*(mNumPatchVertCols-1)+j; mPatchBoundsY[patchID] =
XMFLOAT2(minY, maxY);
}
```

```
void Terrain::BuildQuadPatchVB(ID3D11Device* device)
```

```
{
    [...]

    // 좌표축에 정렬된 바운딩 박스의 y 경계를 패치 좌측 상단 모서리에 저장합니다. for(UINT i = 0; i <
mNumPatchVertRows-1; ++i)
    {
        for(UINT j = 0; j < mNumPatchVertCols-1; ++j)
        {
            UINT patchID = i*(mNumPatchVertCols-1)+j; patchVertices[i*mNumPatchVertCols+j].BoundsY =
mPatchBoundsY[patchID];
        }
    }
    [...]
}
```

이제 상수 선체 셰이더에서 축 정렬 바운드 박스를 구성하고, 박스/프러스텀 교차 테스트를 수행하여 박스가 프러스텀 외부에 있는지 확인할 수 있습니다. [여기서는 15장에서](#) 설명한 것과 다른 박스/평면 교차 테스트를 사용합니다. 이는 실제로 [15장](#) 연습문제 4에서 설명한 OBB/평면 테스트의 특수한 경우입니다. 박스가 축 정렬되어 있으므로 *반지름 r*의 공식은

은 단순화됩니다:

$$\begin{aligned}a_0\mathbf{r}_0 &= \left(a_0,0,0\right) \\ a_1\mathbf{r}_1 &= \left(0,a_1,0\right) \\ a_2\mathbf{r}_2 &= \left(0,0,a_2\right) \\ r &= \left|a_0\mathbf{r}_0 \cdot \mathbf{n}\right| + \left|a_1\mathbf{r}_1 \cdot \mathbf{n}\right| + \left|a_2\mathbf{r}_2 \cdot \mathbf{n}\right| \\ &= a_0\left|n_x\right| + a_1\left|n_y\right| + a_2\left|n_z\right|\end{aligned}$$

이 테스트는 §15.2.3.3의 테스트보다 조건문이 포함되지 않아 선호됩니다.

```
float4 gWorldFrustumPlanes[6];

// 박스가 평면의 완전히 뒤쪽(음의 반공간)에 있을 때 true 반환
// 반공간에 위치할 때 true를 반환합니다.
bool AabbBehindPlaneTest(float3 center, float3 extents, float4 plane)
{
    float3 n = abs(plane.xyz); // (|n.x|, |n.y|, |n.z|)

    // 항상 양수입니다. float r = dot(extents,
    n);

    // 중심점에서 평면까지의 부호 있는 거리. float s = dot(float4(center, 1.0f),
    plane);

    // 박스의 중심점이 평면 뒤에 e 이상의 거리에 있을 경우
    // (이 경우 평면 뒤쪽이므로 s는 음수),
    // 박스는 평면의 음의 반공간에 완전히 위치합니다. return (s + e) < 0.0f;
}

// 박스가 프러스텀 밖에 완전히 위치할 경우 true를 반환합니다.
bool AabbOutsideFrustumTest(float3 center, float3 extents, float4 frustumPlanes[6])
{
    for(int i = 0; i < 6; ++i)
    {
        // 박스가 어느 한 프러스텀 평면 뒤에 완전히 위치해 있다면
        // 그러면 원뿔 투영 영역 밖에 있음. if(AabbBehindPlaneTest(center, extents,
        frustumPlanes[i]))
        {
            return true;
        }
    }
    return false;
}

PatchTess ConstantHS(InputPatch<VertexOut, 4> patch, uint patchID : SV_PrimitiveID)
{
    PatchTess pt;

    //
    // 프러스텀 컬링
    //

    // 패치의 BoundsY를 첫 번째 제어점에 저장합니다. float minY = patch[0].BoundsY.x;
    float maxY = patch[0].BoundsY.y;
```

```

// 축 정렬 경계 상자 생성. patch[2]는 왼쪽 아래 모서리
// patch[1]은 우측 상단 모서리입니다.
float3 vMin = float3(patch[2].PosW.x, minY, patch[2].PosW.z); float3 vMax =
float3(patch[1].PosW.x, maxY, patch[1].PosW.z);

// 중심/범위 표현.
float3 boxCenter = 0.5f * (vMin + vMax); float3 boxExtents
= 0.5f * (vMax - vMin);
if(AabbOutsideFrustumTest(boxCenter, boxExtents, gWorldFrustumPlanes))
{

    pt.EdgeTess[0]      =      0.0f;
    pt.EdgeTess[1]      =      0.0f;
    pt.EdgeTess[2]      =      0.0f;
    pt.EdgeTess[3] = 0.0f;

    pt.InsideTess[0] = 0.0f; pt.InsideTess[1] =
0.0f;

    return pt;
}
//
// 거리에 기반한 일반적인 테셀레이션을 수행합니다.
// else
{
    [...]
}

```

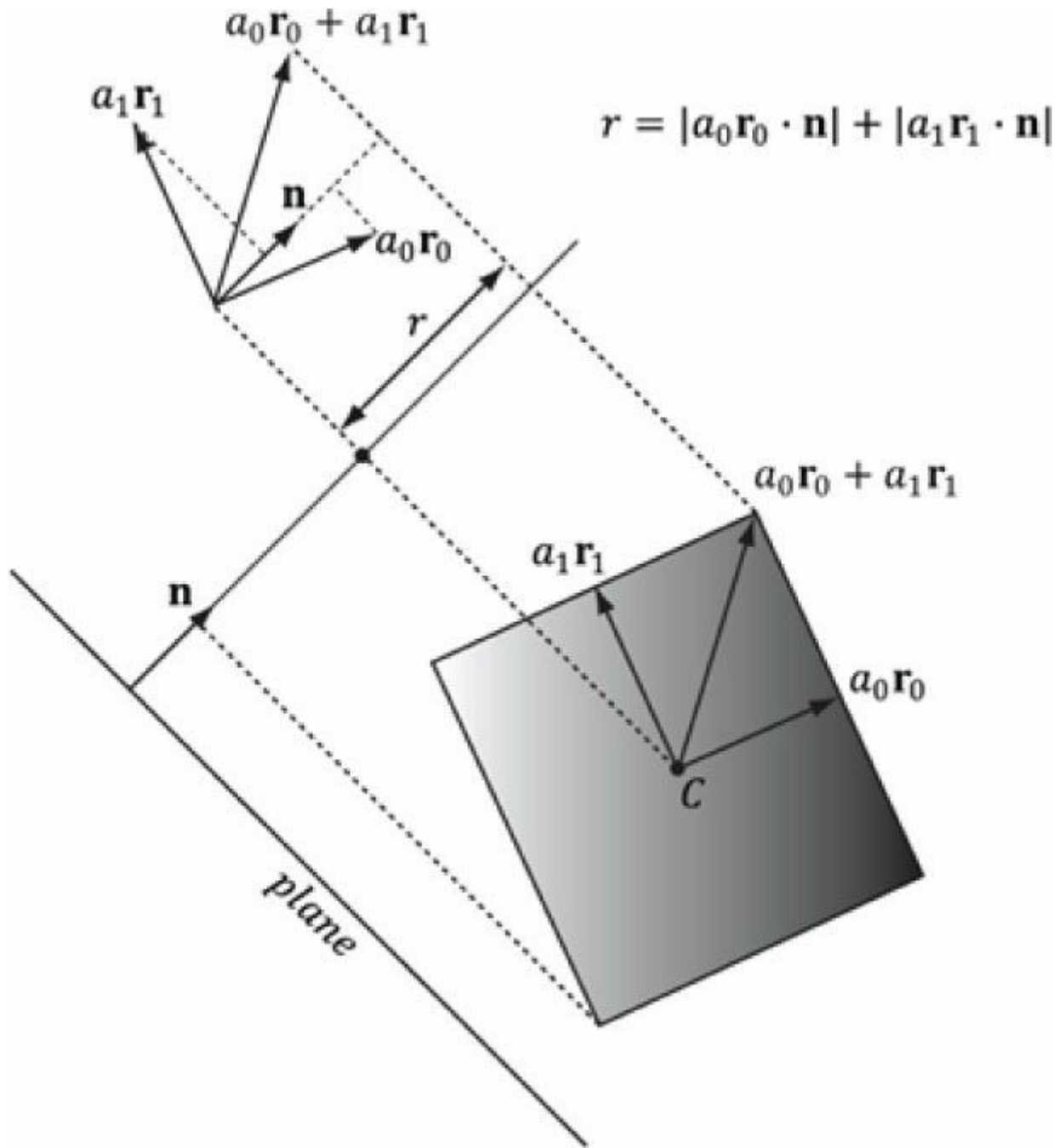


그림 19.10. OBB/평면 교차 테스트. AABB는 OBB의 특수한 경우이므로 동일한 테스트를 사용할 수 있습니다. 또한 AABB의 경우 r 에 대한 공식이 단순화됩니다.

19.4 텍스처링

§8.10 절을 떠올려 보자. 우리는 언덕 위에 잔디 텍스처를 타일링했다. 텍스처를 타일링한 것은 해상도를 높이기 위함이었다(즉, 육지 삼각형을 덮는 텍셀 샘플 수를 늘리기 위함). 여기서도 동일한 작업을 수행하고자 한다. 다만 단일 잔디 텍스처에 국한되고 싶지는 않다. 모래, 잔디, 흙, 바위, 눈 등 동시에 묘사하는 지형을 만들고 싶다. 모래, 풀, 흙 등을 포함한 하나의 대형 텍스처를 생성하여 지형에 늘어 놓는 방법을 제안할 수도 있습니다. 그러나 이는 해상도 문제로 되돌아가는 결과를 초래합니다—지형 지오메트리가 너무 커서 적절한 해상도를 얻기 위해 충분한 색상 샘플을 확보하려면 비현실적으로 큰 텍스처가 필요해진다. 대신 투명도 알파 블렌딩과 유사하게 작동하는 멀티텍스처링 방식을 채택합니다.

각 지형 레이어마다 별도의 텍스처를 사용하는 것이 핵심입니다(예: 풀, 흙, 바위 등). 이 텍스처들은 고해상도를 위해 지형 위에 타일링됩니다. 예를 들어, 3개의 지형 레이어(잔디, 흙, 바위)가 있다고 가정하면, 이 레이어들은 [그림 19.11](#)과 같이 결합됩니다.

이 과정은 투명도 알파 블렌딩을 연상케 합니다. 블렌드 맵은 소스 레이어의 불투명도를 나타내므로, 소스 레이어가 기존 지형 색상을 얼마나 덮어쓸지 제어할 수 있습니다. 이를 통해 지형의 일부를 풀로, 일부를 흙으로, 또 다른 부분을 눈으로 채색하거나 세 가지 모두를 다양한 비율로 혼합할 수 있습니다.

Terrain RGB

Layer Maps

Blend Maps

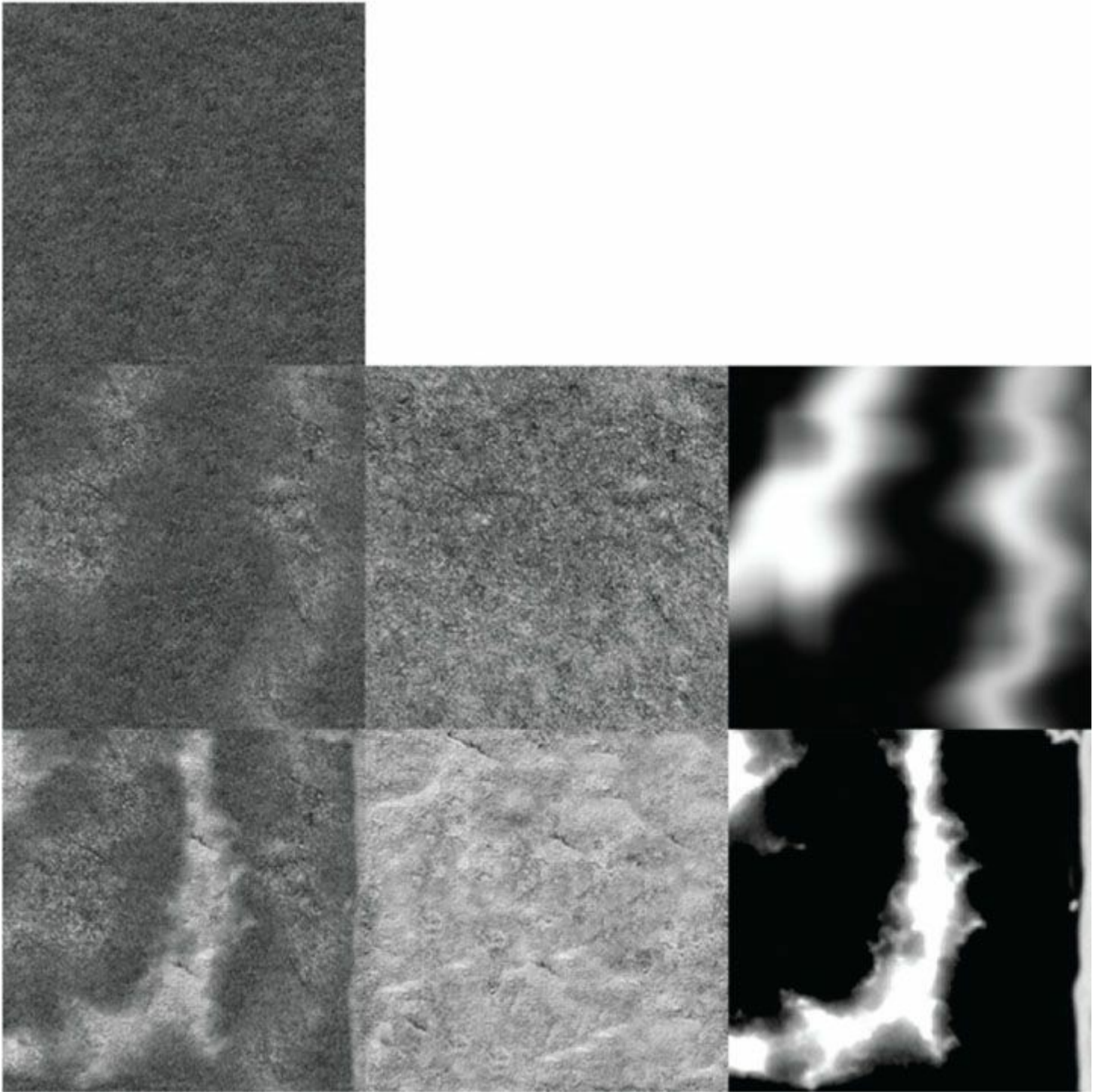


그림 19.11. (상단) 먼저 0번째 레이어(잔디)를 현재 지형 색상으로 배치합니다. (중간) 이제 투명도 알파 블렌딩 방식을 통해 현재 지형 색상과 1번째 레이어(흙)를 혼합합니다; 블렌드 맵이 소스 알파 성분을 제공합니다. (아래) 마지막으로 현재 지형 색상을 투명도 알파 블렌딩 방식을 통해 2번째 레이어(바위)와 혼합합니다. 블렌드 맵이 소스 알파 성분을 제공합니다.

그림 19.11은 3개의 컬러 맵을 사용한 과정을 보여줍니다. 우리의 코드에서는 5개를 사용합니다. 5개의 컬러 맵을 결합하기 위해서는 4개의 그레이스케일 *블렌드 맵*이 필요합니다. 이 4개의 그레이스케일 블렌드 맵을 단일 RGBA 텍스처에 압축할 수 있습니다(빨간색 채널은 첫 번째 블렌드 맵, 녹색 채널은 두 번째 블렌드 맵, 파란색 채널은 세 번째 블렌드 맵, 알파 채널은 네 번째 블렌드 맵을 저장함). 따라서 이를 구현하려면 총 6개의 텍스처가 필요합니다. 다음 지형 픽셀 셰이더 코드는 텍스처 블렌딩 구현 방식을 보여줍니다.

```
// 텍스처 배열에서 레이어 샘플링.
```

```
float4 c0 = gLayerMapArray.Sample(samLinear, float3(pin.TiledTex, 0.0f)); float4 c1 =  
gLayerMapArray.Sample(samLinear, float3(pin.TiledTex, 1.0f)); float4 c2 = gLayerMapArray.Sample(samLinear,  
float3(pin.TiledTex, 2.0f)); float4 c3 = gLayerMapArray.Sample(samLinear, float3(pin.TiledTex, 3.0f)); float4 c4 =  
gLayerMapArray.Sample(samLinear, float3(pin.TiledTex, 4.0f));
```

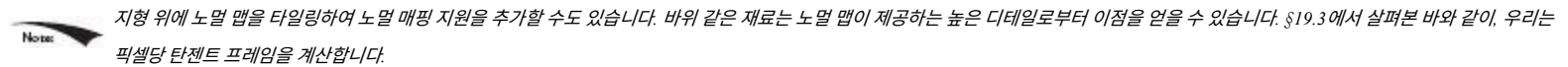
```
// 블렌드 맵 샘플링.
float4 t = gBlendMap.Sample(samLinear, pin.Text);

// 레이어를 서로 위에 블렌딩합니다. float4 texColor = c0;

텍스처 색상 = lerp(텍스처 색상, c1, t.r); 텍스처 색상
= lerp(텍스처 색상, c2, t.g); 텍스처 색상 = lerp(텍스
처 색상, c3, t.b); 텍스처 색상 = lerp(텍스처 색상, c4,
t.a);
```

블렌드 맵은 일반적으로 지형의 하이트맵에 의존합니다. 따라서 블렌드 맵을 컬러 맵의 알파 채널에 내장하지 않습니다. 그렇게 하면 컬러 맵이 특정 하이트맵에서만 작동하게 됩니다. 컬러 맵과 블렌드 맵을 분리함으로써 다양한 지형에 컬러 맵을 재사용할 수 있습니다. 반면 각 지형마다 다른 블렌드 맵이 생성됩니다.

레이어 텍스처와 달리 블렌드 맵은 타일링되지 않습니다. 이는 블렌드 맵을 지형 표면 전체에 걸쳐 눌러 적용하기 때문입니다. 이는 필수적입니다. 블렌드 맵은 특정 텍스처가 드러나길 원하는 지형 영역을 표시하는 데 사용되므로, 블렌드 맵은 전역적이어야 하며 전체 지형을 커버해야 합니다. 이것이 허용 가능한지, 아니면 과도한 확대 현상이 발생하는지 궁금할 수 있습니다. 실제로 확대 현상은 발생하며, 블렌드 맵은 전체 지형에 걸쳐 늘어나면서 텍스처 필터링에 의해 왜곡될 것입니다. 하지만 블렌드 맵은 디테일을 얻는 곳이 아닙니다(디테일은 타일링된 텍스처에서 얻습니다). 블렌드 맵은 특정 텍스처가 기여하는(그리고 어느 정도 기여하는) 지형의 일반적인 영역만을 표시합니다. 따라서 블렌드 맵이 왜곡되고 흐려져도 최종 결과에 큰 영향을 미치지 않습니다. 예를 들어, 약간의 흙이 약간의 풀과 혼합될 수 있으며, 이는 실제로 레이어 간 더 부드러운 전환을 제공합니다.



19.5 지형 높이

일반적인 작업은 x 및 z 좌표를 주어 지형 표면의 높이를 구하는 것입니다. 이는 지형 표면에 오브젝트를 배치하거나, 플레이어가 지형을 걷는 것을 시뮬레이션하기 위해 카메라를 지형 표면보다 약간 위에 배치하는 데 유용합니다.

하이트맵은 격자점에서의 지형 정점 높이를 제공합니다. 그러나 정점 사이의 지형 높이는 알 수 없습니다. 따라서 이산적인 높이맵 샘플링을 바탕으로 지형을 표현하는 연속적인 표면 $y = h(x, z)$ 를 형성하기 위해 보간을 수행해야 합니다. 지형이 삼각형 메쉬로 근사화되므로, 높이 함수가 기본 지형 메쉬 기하학과 일치하도록 선형 보간을 사용하는 것이 합리적입니다.

이를 해결하기 위한 첫 번째 목표는 x 및 z 좌표가 어느 셀에 속하는지 파악하는 것입니다. (참고: x 및 z 좌표는 지형의 국소 공간을 기준으로 합니다.) 다음 코드는 이를 수행하며, x 및 z 좌표가 위치한 셀의 행과 열을 알려줍니다. x 와 z 좌표가 지형의 로컬 공간을 기준으로 함을 가정합니다.) 다음 코드는 이를 수행하며, x - z 좌표가 위치한 셀의 행과 열을 알려줍니다.

```
// 지형 로컬 공간에서 "셀" 공간으로 변환. float c = (x + 0.5f*width()) /
mInfo.CellSpacing;
float d = (z - 0.5f * depth()) / -mInfo.CellSpacing;

// 현재 위치의 행과 열을 구합니다. int row =
(int)floorf(d);
int col = (int)floorf(c);
```

그림 19.12ab는 이 코드의 동작을 설명합니다. 기본적으로 좌표계를 변환하여 좌표계의 원점을 지형 최상단 좌측 꼭짓점으로 설정하고, 양의 z 축은 아래 방향으로 향하며, 각 단위를 한 셀 간격에 대응하도록 스케일링합니다. 이제 이 좌표계에서 **그림 19.12b**를 통해 셀의 행과 열은 각각 **floor(z)**와 **floor(x)**로 간단히 주어짐이 명확합니다. 그림 예시에서 점은 4행 1열에 위치합니다(0부터 시작하는 인덱스 사용). (**floor(t)**는 t 보다 작거나 같은 가장 큰 정수로 평가됨을 상기하십시오.) 또한 row와 col이 셀의 좌상단 꼭짓점 인덱스를 제공함을 관찰하십시오.

t보다 작거나 같은 가장 큰 정수임을 상기하십시오.) 또한 row와 col이 셀의 왼쪽 상단 꼭짓점의 인덱스를 제공함을 관찰하십시오. 현재 위치하는 셀을 확인했으므로, 높이맵에서 네 개의 셀 꼭짓점 높이를 가져옵니다:

```
// 현재 위치 셀의 높이값을 가져옵니다.
// A*B
// | |
// | |
// C*D
float A = mHeightmap[row*mInfo.HeightmapWidth + col];
```

```
float B = mHeightmap[row*mInfo.HeightmapWidth + col + 1]; float C =
mHeightmap[(row+1)*mInfo.HeightmapWidth + col]; float D =
mHeightmap[(row+1)*mInfo.HeightmapWidth + col + 1];
```

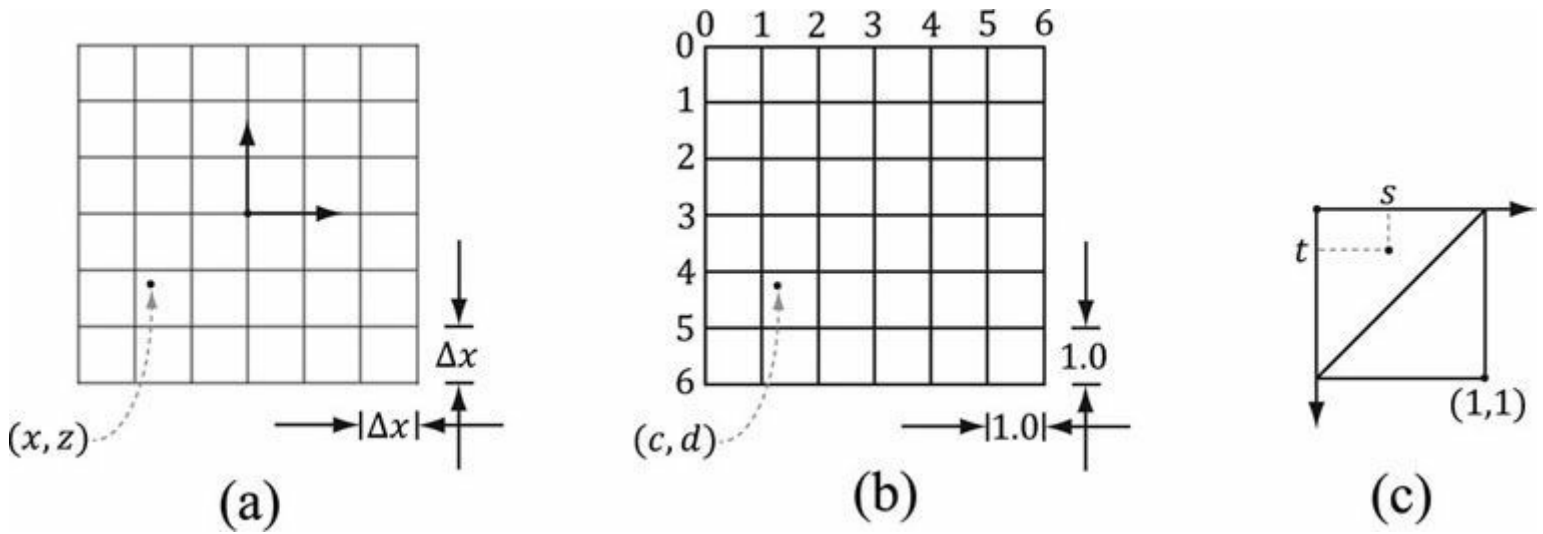


그림 19.12. (a) 지형 좌표계에 대한 xz 평면상의 점은 좌표 (x, z) 를 가진다. (b) 좌표계의 원점을 좌측 상단 그리드 꼭짓점으로 설정하고, 양의 z 축은 아래로 향하며, 각 단위가 하나의 셀 공간에 대응하도록 스케일링된 새로운 좌표계를 선택한다. 이 점은 이 좌표계에 대해 (c, d) 좌표를 가진다. 이 변환은 평행 이동과 크기 조절을 포함합니다. 이 새로운 좌표계에서 우리가 위치한 셀의 행과 열을 찾는 것은 간단합니다. (c) 셀의 왼쪽 위 꼭짓점을 원점으로 하는 세 번째 좌표계를 도입합니다. 해당 점은 이 좌표계에 대해 (s, t) 좌표를 가집니다. 좌표를 이 좌표계로 변환하는 것은 좌표를 오프셋하기 위한 단순한 평행 이동만 포함합니다. $s + t \leq 1$ 이면 "상부" 삼각형에 있고, 그렇지 않으면 "하부" 삼각형에 있음을 관찰하십시오.

이 시점에서 우리는 현재 위치하는 셀과 해당 셀의 네 꼭짓점 높이를 알고 있습니다. 이제 특정 x - 및 z -좌표에서의 지형 표면 높이(y -좌표)를 찾아야 합니다. 셀이 여러 방향으로 기울어질 수 있기 때문에 이 작업은 다소 까다롭습니다; 그림 19.13을 참조하십시오.

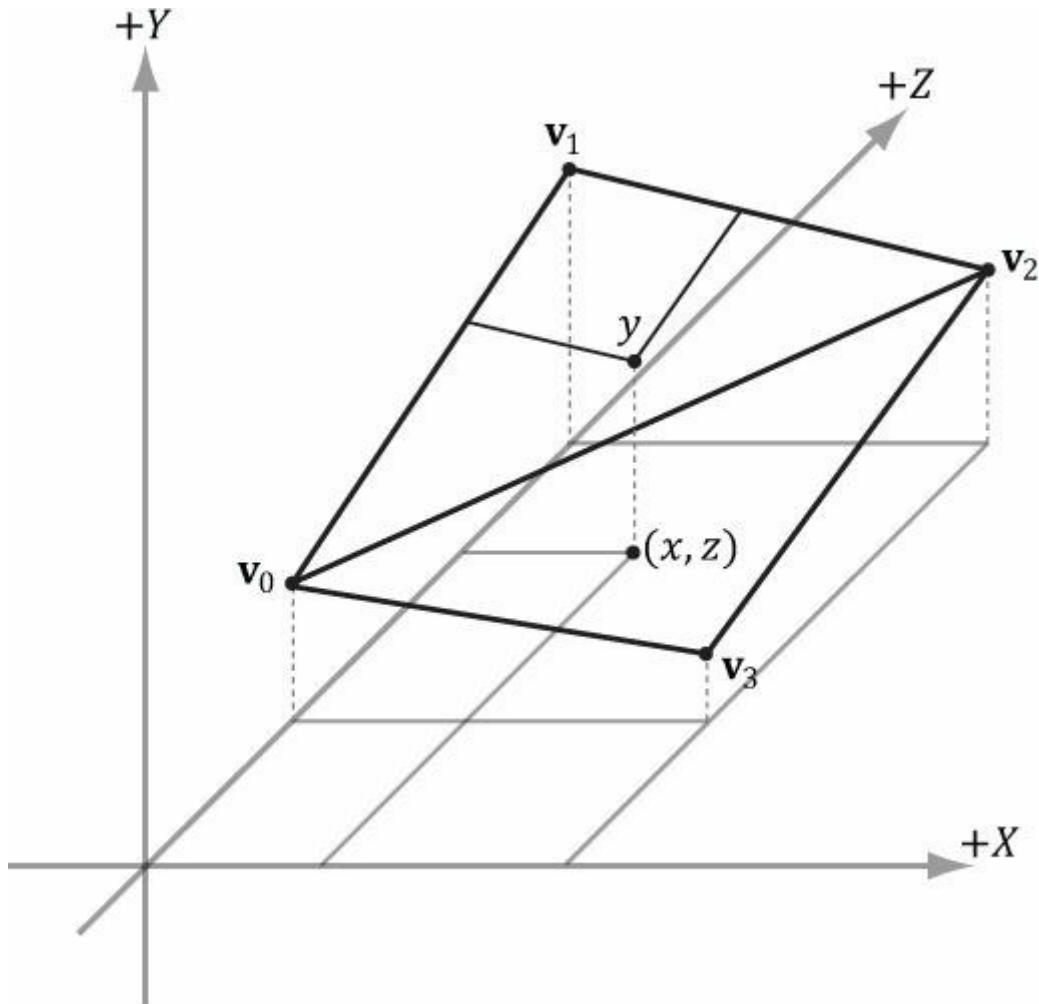


그림 19.13. 특정 x 좌표와 z 좌표에서의 지형 표면 높이(y 좌표).

높이를 구하기 위해서는 우리가 속한 셀의 삼각형이 어느 것인지 알아야 합니다(셀이 두 개의 삼각형으로 렌더링된다는 점을 상기하십시오).

현재 위치하는 삼각형을 찾기 위해, 좌표 (c, d) 가 셀 좌표계에 상대적으로 표현되도록 좌표계를 변경할 것입니다(그림 19.12c 참조). 이 간단한 좌표계 변경은 평행 이동만 포함하며 다음과 같이 수행됩니다:

```
float s = c - (float)col; float t = d -  
(float)row;
```

그런 다음, $s + t \leq 1$ 이면 우리는 "상부" 삼각형 ABC 에 위치하며, 그렇지 않으면 "하부" 삼각형 DCB 에 위치합니다.

이제 "상부" 삼각형에 있을 때 높이를 찾는 방법을 설명합니다. "하부" 삼각형의 과정도 유사하며, 물론 양쪽에 대한 코드는 곧 제공됩니다.

물론 양쪽에 대한 코드는 곧 이어집니다. "상부" 삼각형에 있을 때 높이를 구하려면 먼저 두 벡터 $\mathbf{u} = (Ax, B$

$- A, 0)$ 및 $\mathbf{v} = (0, C - A, - Az)$ 를 삼각형의 변에 대해, 종점 Q 에서 시작하는 경우를 그림 19.14a가 보여준다. 그런 다음 $\mathbf{u}$ 방향으로는 s 에 따라 선형 보간하고, $\mathbf{v}$ 방향으로는 t 에 따라 선형 보간합니다. 그림 19.14b는

이러한 보간 과정을 보여줍니다. 점 $Q + \mathbf{su} + \mathbf{tv}$ 의 y 좌표는 주어진 x 좌표와 z 좌표를 기반으로 한 높이를 제공합니다(벡터 덧셈의 기하학적 해석을 떠올려 보세요

을 떠올리면 이를 확인할 수 있음).

우리가 보간된 높이 값만 고려하므로 y 성분만 보간하고 다른 성분들은 무시할 수 있습니다. 따라서 높이는 $A + \mathbf{su}(y) + \mathbf{tv}(y)$ 의 합으로 구해집니다.

따라서 높이는 $A + \mathbf{su}_y + \mathbf{tv}_y$ 의 합으로 구해집니다.

따라서 `Terrain::GetHeight` 코드의 결론은 다음과 같습니다:

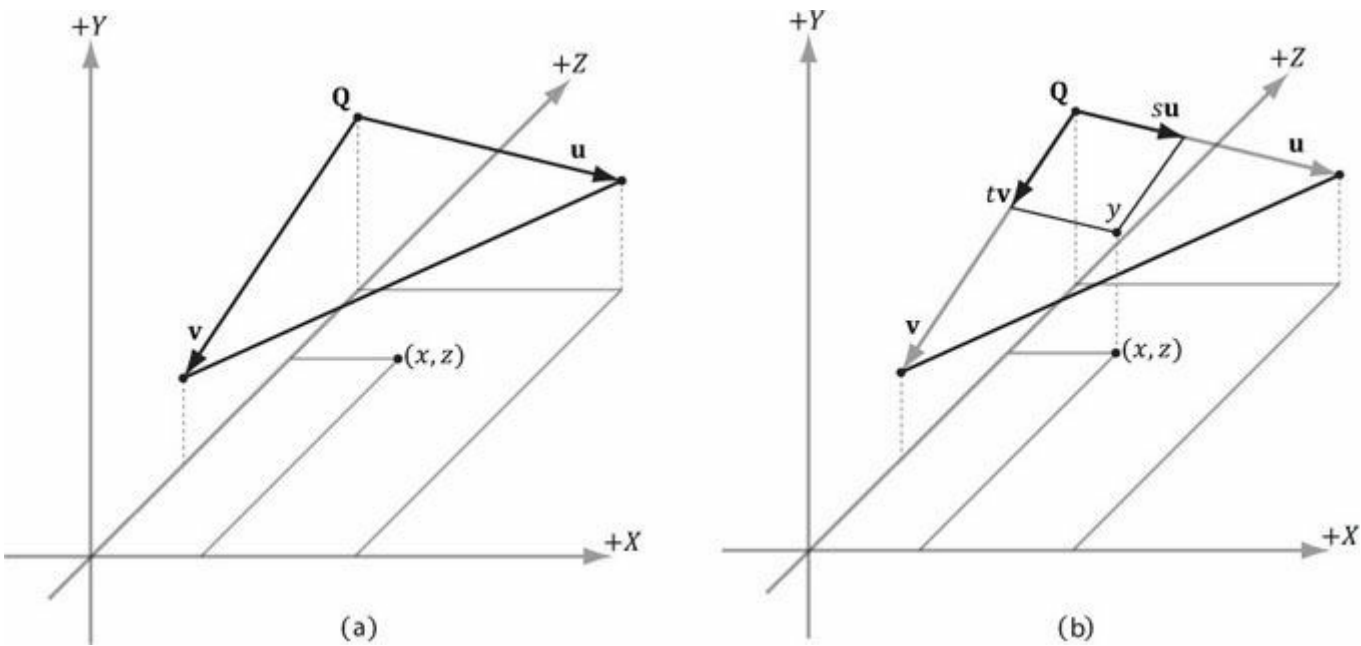


그림 19.14. (a) 상부 삼각형 모서리에서 두 벡터 계산. (b) 높이는 벡터의 y 좌표.

```
// 상부 삼각형 ABC인 경우 if(s + t <= 1.0f)  
{  
  
    float uy = B - A; float vy  
    = C - A;  
    return A + s*uy + t*vy;  
}  
else // 하부 삼각형 DCB.  
{  
  
    float uy = C - D; float vy =  
    B - D;  
    return D + (1.0f-s)*uy + (1.0f-t)*vy;  
}
```

이제 플레이어가 지형 위를 걷는 것처럼 시뮬레이션하기 위해 카메라를 지형 상단으로 클램프할 수 있습니다:

```
void TerrainApp::UpdateScene(float dt)  
{  
  
    //  
    // 카메라 제어.  
    //
```

```

        if(GetAsyncKeyState('W') & 0x8000)
            mCam.Walk(10.0f*dt); if(GetAsyncKeyState('S') &
0x8000)
            mCam.Walk(-10.0f*dt); if(GetAsyncKeyState('A') &
0x8000)
            mCam.Strafe(-10.0f*dt); if(GetAsyncKeyState('D') &
0x8000)
            mCam.Strafe(10.0f*dt);

//
// 걷기/비행 모드
//
if(GetAsyncKeyState('2') & 0x8000)
    mWalk CamMode = true; if(GetAsyncKeyState('3')
& 0x8000)
        mWalk CamMode = false;

//
// 걷기 모드에서 카메라를 지형 표면에 고정합니다.
//
if(mWalk
CamMode)
{
    XMFLOAT3 camPos = mCam.GetPosition();
    float y = mTerrain.GetHeight(camPos.x, camPos.z); mCam.SetPosition(camPos.x, y +
2.0f, camPos.z);
}

mCam.UpdateViewMatrix();
}

```

19.6 요약

1. 삼각형 그리드를 사용하여 지형을 모델링할 수 있으며, 각 정점의 높이를 지정하여 언덕과 계곡을 시뮬레이션합니다.
2. 하이츠맵은 각 요소가 지형 그리드 내 특정 정점의 높이를 지정하는 행렬입니다. 그리드 정점마다 하이츠맵에 항목이 존재하며, ij번째 하이츠맵 항목은 ij번째 정점의 높이를 제공합니다. 하이츠맵은 일반적으로 흑백 맵으로 시각화되며, 검정은 가장 낮은 높이를, 흰색은 가장 높은 높이를, 회색 음영은 그 사이의 높이를 나타냅니다.
3. 지형은 넓은 영역을 덮기 때문에 이를 구축하는 데 필요한 삼각형의 수가 많습니다. 균일하게 분할된 그리드를 사용하면 원근 투영의 특성상 지형의 화면 공간 삼각형 밀도가 거리 증가에 따라 높아집니다. 이는 실제로 우리가 원하는 것과 반대입니다: 디테일이 눈에 띄는 카메라 근처에서는 삼각형 밀도가 높아야 하고, 디테일이 알아채기 어려운 카메라에서 멀리 떨어진 곳에서는 삼각형 밀도가 낮아야 합니다. 하드웨어 테셀레이션을 사용하여 카메라와의 거리에 기반한 연속적인 디테일 레벨(LOD)을 구현할 수 있습니다. 지형 테셀레이션의 전반적인 전략은 다음과 같이 요약할 수 있습니다:

사각 패치 그리드를 배치합니다.

패치를 카메라로부터의 거리에 따라 분류합니다.

하이츠맵을 셰이더 리소스로 바인딩합니다. 도메인 셰이더에서 생성된 점점들을 적절한 높이로 오프셋하기 위해 하이츠맵으로부터 변위 매핑을 수행합니다.

4. GPU에서 프러스텀 컬링을 구현하여 프러스텀 외부의 쿼드 패치를 제거할 수 있습니다. 이는 프러스텀 외부의 패치에 대해 모든 테셀레이션 계수를 0으로 설정하는 상수 헬 셰이더 함수에서 수행됩니다.
5. 지형 텍스처링은 여러 레이어(예: 풀, 흙, 바위, 눈)를 서로 블렌딩하여 수행합니다. 블렌드 맵은 각 레이어가 최종 지형 이미지에 기여하는 정도를 제어하는 데 사용됩니다.
6. 고도맵은 격자점에서의 지형 정점 높이를 제공하지만, 정점 사이의 지형 높이도 필요합니다. 따라서 이산적인 고도맵 샘플링을 주어진 지형을 표현하는 연속적인 표면 $y = h(x, z)$ 를 형성하기 위해 보간을 수행해야 합니다. 지형이 삼각형 메쉬로 근사화되므로, 우리의 높이 함수가 기본 지형 메쉬 기하학과 일치하도록 선형 보간을 사용하는 것이 합리적입니다. 지형의 높이 함수를 확보하면 지형 표면에 오브젝트를 배치하거나, 카메라를 지형 표면 바로 위에 위치시켜 플레이어가 지형을 걷는 모습을 시뮬레이션하는 데 유용합니다.

19.7 연습문제

1. 데모에서는 이전 장에서와 같이 카메라를 "조준" 방향으로 이동시킨 후 `Terrain::GetHeight`를 적용합니다. 카메라를 지형 표면보다 약간 위에 고정하여 카메라가 지형 위를 걷는 것처럼 시뮬레이션하는 가능합니다. 그러나 [그림 19.15](#)에서는 카메라를 "조준" 방향이 아닌 지형 표면에 접하는 방향으로 이동시켜야 함을 보여줍니다. 플레이어가 언덕을 오르내릴 때 부자연스러운 속도 변화가 발생하지 않도록 지형 위에서 카메라가 적절히 이동하도록 "Terrain" 데모를 수정하세요.
2. 일반적으로 지형의 일부는 거칠고 구릉지이며 디테일이 풍부한 반면, 다른 부분은 평평하고 매끄럽습니다. 고주파 영역을 포함하는 패치에 더 많은 테셀레이션이 적용되도록 테셀레이션을 편향시키는 것이 일반적입니다. 이렇게 하면 추가 디테일이 잘 드러납니다. 마찬가지로 평평한 영역은 테셀레이션이 덜 필요합니다. 예를 들어 평평한 사각형 패치는 전혀 세분화할 필요가 없습니다. 패치별 거칠기 계수 계산 방법을 연구하십시오. 디스플레이먼트 맵의 고주파 영역을 나타내는 밀도 맵을 계산하는 SDK 샘플 "DetailTessellation11"이 좋은 출발점이 될 것입니다.
3. *Terragen*에서 직접 높이맵을 생성하여 Terrain 데모에서 사용해 보세요. 자신만의 블렌드 맵을 제작해 보세요.

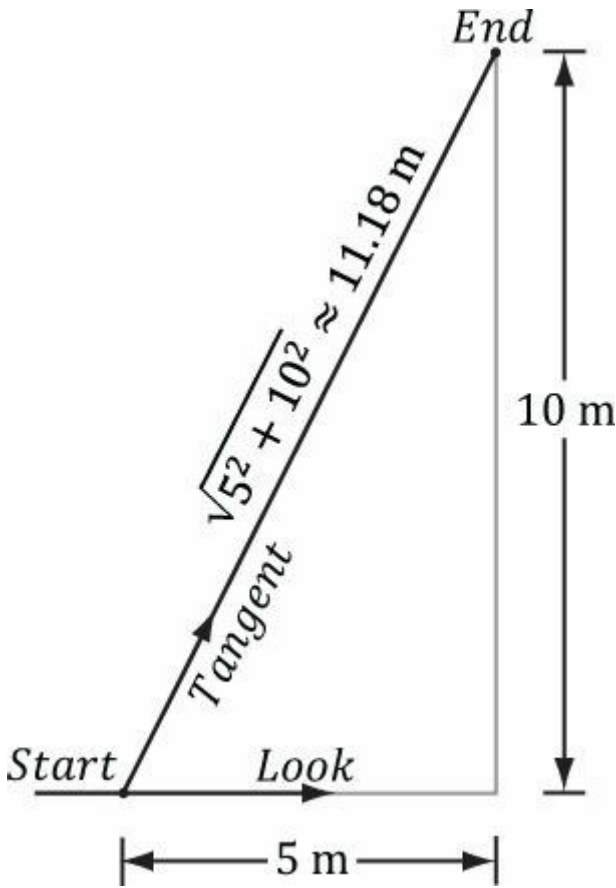


그림 19.15. 카메라가 "시작" 위치에 있고 "조준" 방향으로 이동한다고 가정합니다. 또한 카메라 속도가 5m/s(초당 미터)라고 가정합니다. 1초 동안 카메라는 "조준" 방향으로 5m 이동하지만, 이후 높이가 "종료"로 조정되어 카메라가 지형 위에 머무르게 됩니다. 따라서 실제로는 카메라가 1초 만에 5m 이동한 것이 아니라 총 11.18m를 이동한 것이며, 이는 비정상적인 가속 현상입니다. 이를 해결하기 위해 표면에 접선 방향으로 항상 5m/s 속도로 카메라를 이동시킵니다.

4. 우리는 `fractional_even` 테셀레이션 모드와 함께 연속적인 레벨 오브 디테일(LOD)을 사용합니다. 디버깅을 위해 `CalcTessFactor` 함수에 다음과 같은 변경을 가해 지형의 다양한 LOD 레벨을 더 쉽게 확인할 수 있도록 합니다.

```
float CalcTessFactor(float3 p)
{
    // xz 평면 내 최대 노름 (상세 수준 확인에 유용함
    // 새의 눈높이에서).
    float d = max(abs(p.x-gEye PosW.x), abs(p.z-gEye PosW.z));

    float s = saturate((d - gMinDist) / (gMaxDist - gMinDist));
```

```
    return pow(2, round(lerp(gMaxTess, gMinTess, s)));  
}
```

두 가지 변경 사항을 적용했습니다. 첫째, xz 평면에서 최대 노름을 사용하여 거리를 측정합니다. 이를 통해 y 좌표를 무시하므로 거리 값에 영향을 주지 않고 새의 눈높이 시점으로 확대/축소할 수 있습니다. 둘째, 지수 부분을 정수로 반올림하여 테셀레이션 계수가 항상 2의 정수 지수가 되도록 했습니다. 이로 인해 정확히 일곱 개의 이산적인 LOD 레벨($2^0, 2^1, \dots, 2^6$). 이 연습에서는 CalcTessFactor 함수에 위 변경 사항을 적용하고, 확대/축소하여 [그림 19.7](#)과 유사한 화면을 확인하세요. 하늘 그리기 기능을 비활성화하고 지형을 와이어프레임 모드로 검정색으로 표시하면 도움이 될 수 있습니다.

제20장 입자 시스템과 스트림 아웃

이 장에서는 유사하면서도 다소 무작위적인 방식으로 행동하는 (보통 작은) 입자 집합을 모델링하는 작업에 대해 다룹니다. 이러한 입자 집합을 *입자 시스템*이라고 부릅니다. 입자 시스템은 불, 비, 연기, 폭발, 스프링클러, 마법 주문 효과, 투사체 등 다양한 현상을 시뮬레이션하는 데 사용될 수 있습니다.

목표:

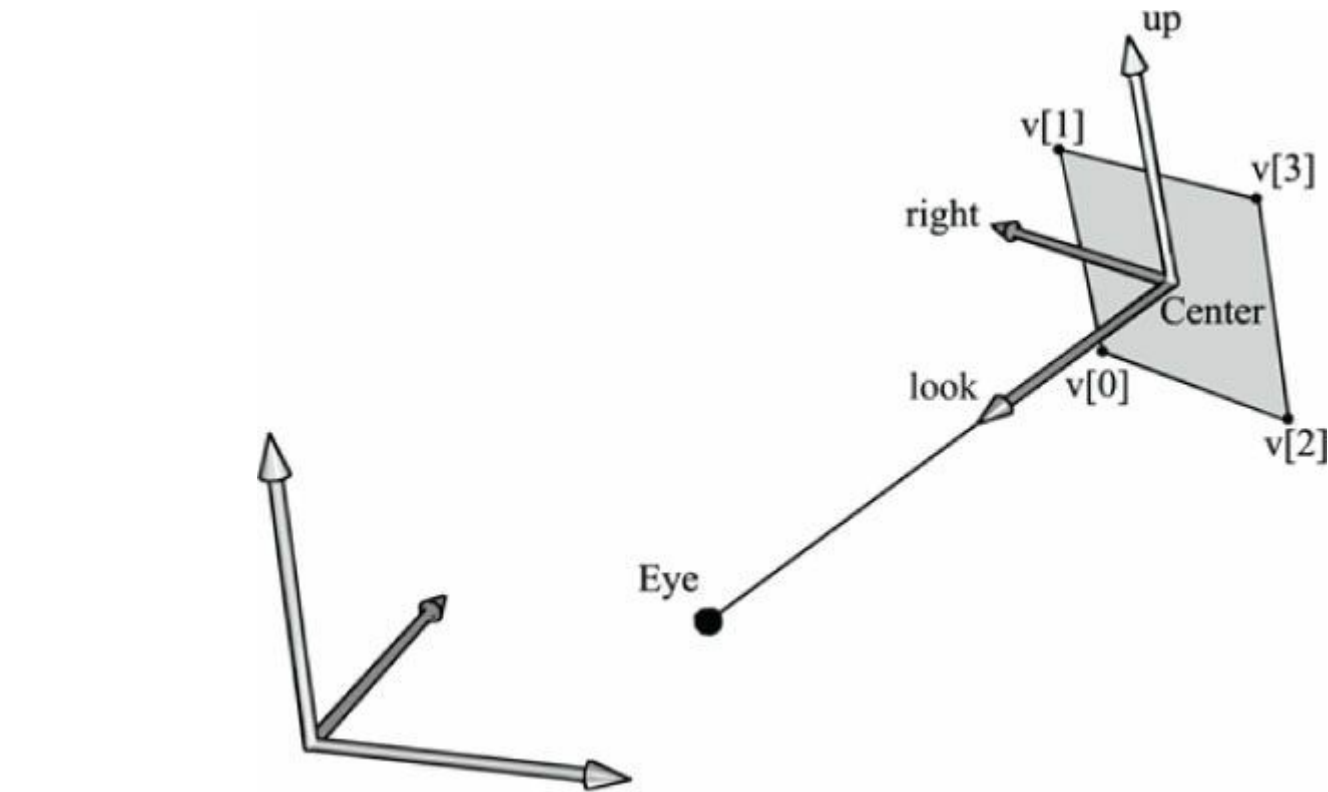
- 1. 지오메트리 셰이더와 스트림 아웃 기능을 활용하여 입자를 효율적으로 저장하고 렌더링하는 방법을 학습한다.
- 2. 기본 물리학 개념을 활용하여 입자들이 물리적으로 현실적인 방식으로 움직이도록 만드는 방법을 알아보기 위함입니다.
- 3. 새로운 맞춤형 입자 시스템을 쉽게 생성할 수 있도록 유연한 입자 시스템 프레임워크를 설계하기 위함입니다.

20.1 입자 표현

*입자*는 일반적으로 수학적으로 점으로 모델링되는 매우 작은 객체입니다. 따라서 점 프리미티브(D3D11_PRIMITIVE_TOPOLOGY_POINTLIST)가 입자 표시용으로 적합한 후보가 될 수 있습니다. 그러나 점 프리미티브는 단일 픽셀로 래스터화됩니다. 이는 다양한 크기의 입자를 구현하거나 심지어 전체 텍스처를 입자에 매핑하려는 우리의 요구에 비해 유연성이 부족합니다. 따라서 11장에서 나무 빌보드에 적용했던 전략과 유사한 방식을 채택하겠습니다. 점으로 입자를 저장한 후, 지오메트리 셰이더에서 카메라를 향하도록 사각형(쿼드)으로 확장합니다. 세계 좌표계의 y축에 정렬되었던 나무 빌보드와 달리, 입자 빌보드는 카메라를 완전히 향하게 됩니다(그림 20.1 참조).

세계 좌표계의 세계 상향 벡터 $\mathbf{j}$, 빌보드 중심 위치 $\mathbf{C}$, 그리고 시점 위치 $\mathbf{E}$ 를 알고 있다면 빌보드의 로컬 프레임을 세계 좌표계로 표현할 수 있으며, 이는 빌보드의 세계 행렬을 제공합니다:

$$\begin{aligned} \mathbf{w} &= \frac{\mathbf{E} - \mathbf{C}}{\|\mathbf{E} - \mathbf{C}\|} \\ \mathbf{u} &= \frac{\mathbf{j} \times \mathbf{w}}{\|\mathbf{j} \times \mathbf{w}\|} \\ \mathbf{v} &= \mathbf{w} \times \mathbf{u} \\ \mathbf{W} &= \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ C_x & C_y & C_z & 1 \end{bmatrix} \end{aligned}$$



그림/20.1. 세계 좌표계와 빌보드 좌표계. 빌보드는 시점 위치 E를 향합니다.

위치와 크기 외에도, 우리의 입자들은 다른 속성을 가질 것입니다. 입자를 위한 우리의 정점 구조는 다음과 같습니다:

```
struct Particle
{
    XMFLOAT3      InitialPos;
    XMFLOAT3      InitialVel;
    XMFLOAT2 Size;
    float Age; unsigned int
    Type;
};
```

반드시 점을 사각형으로 확장할 필요는 없습니다. 예를 들어, 비를 렌더링할 때 선 목록을 사용하는 방법이 꽤 효과적입니다. 다른 지오메트리 셰이더를 사용해 점을 선으로 확장할 수 있습니다.

기본적으로 우리 시스템에서는 각 파티클 시스템이 자체 효과 파일을 갖게 됩니다. 이 효과 파일은 해당 파티클 시스템 유형에 특화된 세부 사항을 구현합니다.



20.2 입자 운동

입자들이 물리적으로 현실적인 방식으로 움직이도록 하고자 합니다. 단순화를 위해 이 책에서는 일정한 순가속도(예: 중력에 의한 가속도)로 제한합니다. (바람과 같은 다른 힘에 의한 가속도도 일정하게 설정하여 대략적인 근사치를 만들 수도 있습니다.) 또한 입자들 간의 충돌 검출은 수행하지 않습니다.

$p(t)$ 가 곡선을 따라 움직이는 입자의 위치(시간 t 에서)를 나타낸다고 하자. 시간 t 에서 입자의 순간 속도는 다음과 같다:

$$v(t) = p'(t)$$

시간 t 에서의 입자의 순간 가속도는 다음과 같다:

$$a(t) = v'(t) = p''(t)$$

미적분학에서 다음을 상기하자:

- 함수 $f(t)$ 의 원함수란 $F(t)$ 의 미분이 $f(t)$ 가 되는 함수 $F(t)$ 를 말한다. 즉, $F'(t) = f(t)$ 이다.
- 만약 $F(t)$ 가 $f(t)$ 의 어떤 적분함수이고 c 가 어떤 상수라면, $F(t) + c$ 역시 $f(t)$ 의 적분함수이다. 또한 $f(t)$ 의 모든 적분함수는 $F(t) + c$ 의 형태를 가진다.

3. 함수 $f(t)$ 의 임의의 부정적분을 나타내기 위해, 우리는 적분 기호

$$\int f(t) dt = F(t) + c.$$

속도와 가속도의 정의로부터, 속도 함수는 가속도 함수의 부정적분이며, 위치 함수는 속도 함수의 부정적분임이 명백합니다. 따라서 우리는 다음과 같이 표현할 수 있습니다:

$$\mathbf{p}(t)=\int \mathbf{v}(t) dt$$

$$\mathbf{v}(t)=\int \mathbf{a}(t) dt$$

이제 가속도가 일정하다고 가정하자(즉, 시간에 따라 변하지 않음). 또한 시간 $t=0$ 에서 입자의 초기 속도 $\mathbf{v}(0)=\mathbf{v}_0$ 과 초기 위치 $\mathbf{p}(0)=\mathbf{p}_0$ 를 안다고 가정하자. 그러면 속도 함수는 일정한 가속도를 적분하여 구할 수 있다:

$$\mathbf{v}(t)=\int \mathbf{a}(t) dt = t\mathbf{a} + \mathbf{c}$$

상수 $\mathbf{c}$ 를 구하기 위해 초기 속도를 사용합니다:

$$\mathbf{v}(0)=0\cdot \mathbf{a} + \mathbf{c} = \mathbf{c} = \mathbf{v}_0$$

따라서 속도 함수는

$$\mathbf{v}(t)= t\mathbf{a} + \mathbf{v}_0$$

위치 함수를 구하기 위해 방금 구한 속도 함수를 적분합니다:

$$\mathbf{p}(t)=\int \mathbf{v}(t) dt = \int (t\mathbf{a} + \mathbf{v}_0) dt = \frac{1}{2}t^2\mathbf{a} + t\mathbf{v}_0 + \mathbf{k}$$

상수 $\mathbf{k}$ 를 구하기 위해 초기 위치를 사용합니다:

$$\mathbf{p}(0)=\frac{1}{2}\cdot 0\cdot \mathbf{a} + 0\cdot \mathbf{v}_0 + \mathbf{k} = \mathbf{k} = \mathbf{p}_0$$

따라서 위치 함수는 다음과 같습니다.

$$\mathbf{p}(t)=\frac{1}{2}t^2\mathbf{a} + t\mathbf{v}_0 + \mathbf{p}_0$$

다시 말해, 입자의 궤적 $\mathbf{p}(t)$ (즉, $t\geq 0$ 인 모든 시간에서의 위치)는 초기 위치, 초기 속도, 가속도 상수에 의해 완전히 결정됩니다. 이는 당연한 결과입니다. 왜냐하면 우리가 출발 지점, 초기 속도 및 방향, 그리고 시간에 따른 가속도 변화를 알고 있다면, 우리가 따라온 경로를 계산해낼 수 있어야 하기 때문입니다.

예를 들어 보자. 좌표계의 원점에 위치한 소형 대포가 있다고 가정하자. 이 대포는 x축으로부터 30° 각도로

(그림 20.2 참조). 따라서 이 좌표계에서 $\mathbf{p}_0=(0,0,0)$ (즉, 포탄의 초기 위치는 원점에 있음)이며, 중력에 의한 일정한 가속도는 $\mathbf{a}=(0,-9.8,0)$ m/s²입니다 (즉, 중력 가속도는 초당 9.8미터 제곱입니다). 또한, 이전 실험을 통해 대포가 발사되는 순간 포탄의 초기 속도가 초당 50미터임을 확인했다고 가정합니다. 따라서 초기 속도는 $\mathbf{v}_0=50(\cos 30^\circ, \sin 30^\circ, 0)$ m/s $\approx (43.3, 25.0, 0)$ m/s [속도는 속도와 방향이므로 속도에 방향을 곱해야 함]이다.

단위 방향 벡터 $(\cos 30^\circ, \sin 30^\circ, 0)$. 따라서 포탄의 궤적은 다음과 같이 주어집니다:

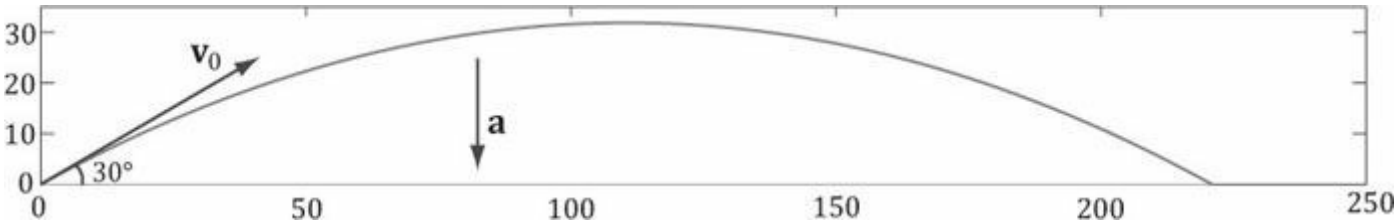


그림 20.2. 초기 위치와 속도를 가졌으며 중력에 의한 일정한 가속도를 받는 입자가 xy 평면에서 시간에 따라 따라가는 경로(시간 차원은 표시되지 않음).

$$\begin{aligned}\mathbf{p}(t) &= \frac{1}{2}t^2\mathbf{a} + t\mathbf{v}_0 + \mathbf{p}_0 \\ &= \frac{1}{2}t^2(0,-9.8,0)\text{m/s}^2 + t(43.3,25.0,0)\text{m/s}\end{aligned}$$

이를 xy 평면(z 좌표는 항상 0)에 그래프로 그리면 [그림 20.2](#)와 같이 중력 하에서 예상되는 궤적을 얻을 수 있다.

또한 이전에 유도한 함수를 사용하지 않을 수도 있습니다. 입자들이 따라가길 원하는 궤적 함수 $\mathbf{p}(t)$ 를 이미 알고있다면  , 이를 직접 프로그래밍하면 됩니다. 예를 들어 입자들이 타원 궤적을 따라가길 원한다면, $p(t)$ 에 대해 타원의 매개변수 방정식을 사용하면 됩니다. 타원 궤도를 따라가게 하려면, $\mathbf{p}(t)$ 에 타원의 매개변수 방정식을 사용하면 됩니다.

20.3 무작위성

입자 시스템에서는 입자들이 비슷하게 행동하되 완전히 동일하지는 않게 하고자 합니다. 즉, 시스템에 약간의 무작위성을 부여하려는 것입니다. 예를 들어, 빗방울을 모델링할 때 빗방울이 정확히 동일한 방식으로 떨어지길 원하지 않습니다. 서로 다른 위치에서, 약간 다른 각도로, 그리고 약간 다른 속도로 떨어지길 원합니다. 입자 시스템에 필요한 무작위성 기능을 구현하기 위해 *MathHelper.h/cpp*에 구현된 **RandF** 및 **RandUnitVec3** 함수를 사용합니다:

```
// [0, 1) 범위의 난수 부동소수점 값을 반환합니다. static
float RandF()
{
    return (float)(rand()) / (float)RAND_MAX;
}

// [a, b) 범위의 난수 부동소수점 값을 반환합니다. static
float RandF(float a, float b)
{
    return a + RandF()*(b-a);
}

XMVECTOR MathHelper::RandUnitVec3()
{
    XMVECTOR One = XMVectorSet(1.0f, 1.0f, 1.0f, 1.0f);
    XMVECTOR Zero = XMVectorZero();

    // 반구 위/안에 점이 나올 때까지 계속 시도합니다. while(true)
    {
        // 정육면체 [-1,1]^3 내 임의의 점 생성. XMVECTOR v = XMVectorSet(
            MathHelper::RandF(-1.0f, 1.0f), MathHelper::RandF(-
            1.0f, 1.0f), MathHelper::RandF(-1.0f, 1.0f), 0.0f);

        // 단위 구면 외부의 점은 무시하여
        // 단위 구면 전체에 균일한 분포를 얻기 위함. 그렇지 않으면
        // 점들은 정육면체의 모서리 근처 구면에서 더 뭉쳐질 것이다
        //

        if(XMVector3Greater(XMVector3LengthSq(v), One)) continue;

        return XMVector3Normalize(v);
    }
}
```

이전 함수들은 C++ 코드에서 작동하지만, 셰이더 코드에서도 난수가 필요합니다. 셰이더에서 난수를 생성하는 것은 셰이더 전용 난수 생성기가 없기 때문에 더 까다롭습니다. 따라서 우리가 하는 일은 네 개의 부동 소수점 성분을 가진 1차원 텍스처(DXGI_FORMAT_R32G32B32A32_FLOAT)를 생성하는 것입니다. 이 텍스처에 [-1,1] 구간 내 좌표를 가진 4차원 벡터를 무작위로 채웁니다. 텍스처는 랩 주소 모드로 샘플링되므로 [0, 1] 구간 밖의 무한 텍스처 좌표도 사용할 수 있습니다. 이후 셰이더 코드가 이 텍스처를 샘플링하여 난수를 얻습니다. 랜덤 텍스처를 샘플링하는 방법은 여러 가지가 있습니다. 각 파티클이 서로 다른 x좌표를 가질 경우, x좌표를 텍스처 좌표로 사용하여 랜덤 값을 얻을 수 있습니다. 그러나 많은 파티클이 동일한 x좌표를 가질 경우, 모두 텍스처에서 동일한 값을 샘플링하게 되어 랜덤성이 떨어집니다. 다른 접근법은 현재 게임 시간 값을 텍스처 좌표로 사용하는 것입니다. 이렇게 하면 서로 다른 시간에 생성된 파티클이 서로 다른 무작위 값을 얻게 됩니다. 그러나 이는 동시에 생성된 파티클이 동일한 값을 가지게 된다는 의미입니다. 파티클 시스템이 한 번에 여러 파티클을 방출해야 하는 경우 문제가 될 수 있습니다. 동시에 많은 입자를 생성할 때는 게임 시간에 서로 다른 텍스처 좌표 오프셋 값을 추가하여 텍스처 맵의 다른 지점을 샘플링함으로써 서로 다른 무작위 값을 얻을 수 있습니다. 예를 들어, 20개의 입자를 생성하기 위해 20번 반복하는 경우, 반복 횟수(적절히 스케일링된)를 사용하여 무작위 텍스처를 샘플링하는데 사용되는 텍스처 좌표를 오프셋할 수 있습니다. 이렇게 하면 20개의 서로 다른 무작위 값을 얻을 수 있습니다.

다음 코드는 무작위 텍스처를 생성하는 방법을 보여줍니다:

```
ID3D11ShaderResourceView* d3dHelper::CreateRandomTexture1DSRV( ID3D11Device* device)
{
    //
    // 무작위 데이터 생성.
    //
    XMFLOAT4 randomValues[1024];

    for(int i = 0; i < 1024; ++i)
    {
        randomValues[i].x = MathHelper::RandF(-1.0f, 1.0f); randomValues[i].y =
        MathHelper::RandF(-1.0f, 1.0f); randomValues[i].z = MathHelper::RandF(-1.0f,
        1.0f); randomValues[i].w = MathHelper::RandF(-1.0f, 1.0f);
    }

    D3D11_SUBRESOURCE_DATA initData;
    initData.pSysMem = randomValues; initData.SysMemPitch =
    1024*sizeof(XMFLOAT4); initData.SysMemSlicePitch = 0;

    //
    // 텍스처 생성.
    //
    D3D11_TEXTURE1D_DESC texDesc;
    texDesc.Width = 1024;
    texDesc.MipLevels = 1;
    텍스처 설명자 형식 = DXGI_FORMAT_R32G32B32A32_FLOAT; 텍스처 설명자 사용법 =
    D3D11_USAGE_IMMUTABLE; 텍스처 설명자 바인딩 플래그 =
    D3D11_BIND_SHADER_RESOURCE; 텍스처 설명자 CPU 접근 플래그 = 0;
    텍스처 설명자.기타 플래그 = 0;
    텍스처 설명자.배열 크기 = 1;

    ID3D11Texture1D* randomTex = 0;
    HR(device->CreateTexture1D(&texDesc, &initData, &randomTex));

    //
    // 리소스 뷰 생성.
    //
    D3D11_SHADER_RESOURCE_VIEW_DESC 뷰디스크립션 뷰디스크립션;
    viewDesc.Format = texDesc.Format;
    viewDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE1D; viewDesc.Texture1D.MipLevels = texDesc.MipLevels;
    viewDesc.Texture1D.MostDetailedMip = 0;
```

```
ID3D11ShaderResourceView* randomTexSRV = 0;
HR(device->CreateShaderResourceView(randomTex, &viewDesc,
&randomTexSRV));

ReleaseCOM(randomTex); return

randomTexSRV;

}
```

무작위 텍스처의 경우 단일 mip 맵 레벨만 필요합니다. 단일 mip 맵으로 텍스처를 샘플링하려면 **SampleLevel** 내장 함수를 사용합니다. 이 함수를 통해 샘플링할 mip 맵 레벨을 명시적으로 지정할 수 있습니다. 첫 번째 매개변수는 샘플러, 두 번째 매개변수는 텍스처 좌표(1차원 텍스처의 경우 단일 좌표), 세 번째 매개변수는 mip 맵 레벨입니다(mip 맵 레벨이 하나뿐인 텍스처의 경우 0이어야 함). 다음 셰이더 함수는 단위 구상에서 랜덤 벡터를 얻는 데 사용됩니

```
float3 RandUnitVec3(float offset)
{
    // 게임 시간에 오프셋을 더해 무작위 텍스처를 샘플링합니다. float u = ( gameTime +
offset);

    // 좌표 범위 [-1,1]
    float3 v = gRandomTex.SampleLevel(samLinear, u, 0).xyz;

    // 단위 구에 투영 return normalize(v);
}
```

20.4 블렌딩과 파티클 시스템

입자 시스템은 일반적으로 어떤 형태의 블렌딩을 통해 렌더링됩니다. 불과 마법 주문 같은 효과에서는 입자 위치에서 색상 강도가 밝아지길 원합니다. 이 효과에는 가산 블렌딩이 효과적입니다. 즉, 소스 색상과 대상 색상을 단순히 합산하는 방식입니다. 그러나 입자는 일반적으로 투명하기도 하므로, 소스 입자 색상을 불투명도에 따라 스케일링해야 합니다. 즉, 다음과 같은 블렌드 매개변수를 사용합니다:

```
SrcBlend = SRC_ALPHA; DestBlend =
ONE; BlendOp = ADD;
```

이를 통해 블렌딩 방정식은 다음과 같이 표현됩니다:

$$C = a_s C_{src} + C_{dst}$$

다시 말해, 소스 입자가 합계에 기여하는 색상 양은 그 불투명도에 의해 결정됩니다: 입자가 불투명할수록 더 많은 색상을 기여합니다. 대안적인 접근법은 텍스처를 그 불투명도(알파 채널로 표현됨)로 미리 곱하여 텍스처 색상이 불투명도에 따라 희석되도록 하는 것입니다. 그런 다음 희석된 텍스처를 사용합니다. 이 경우 다음과 같은 블렌딩 매개변수를 사용할 수 있습니다:

```
SrcBlend = ONE; DestBlend =
ONE; BlendOp = ADD;
```

이는 본질적으로 C_{src} 를 사전 *계산하여* 텍스처 데이터에 직접 내장했기 때문입니다. 가산 혼합은 또한 입자 농도에 비례하여 해당 영역을 밝게 하는 효과도 있습니다(가산적 색상 누적 때문). 색상이 누적되기 때문임). 따라서 농도가 높은 영역은 특히 밝게 보이는데, 이는 일반적으로 원하는 효과입니다([그림 20.3](#) 참조). 연기와 같은 것의 경우, 겹쳐진 여러 연기 입자의 색상을 더하면 색상이 혼합되어 버리기 때문에 가산 혼합은 효과가 없습니다.

결국 연기를 밝게 만들어 어둡지 않게 합니다. 연기의 경우 감산 연산자(D3D11_BLEND_OP_REV_SUBTRACT)로 블렌딩하는 것이 더 효과적이며, 이때 연기 입자는 대상 영역의 색상을 빼게 됩니다. 이렇게 하면 연기 입자의 농도가 높을수록 색이 더 검게 나타나 짙은 연기의 느낌을 주고, 농도가 낮을수록 연한 색조가 나타나 옅은 연기의 느낌을 줍니다. 그러나 이 방법은 검은 연기에 효과적이지만 밝은 회색 연거나 증기에는 적합하지 않습니다. 연기에 대한 또 다른 가능성은 투명도 블렌딩을 사용하는 것입니다. 여기서 연기 입자를 반투명 오브젝트로 간주하고 투명도 블렌딩을 사용하여 렌더링합니다. 투명도 블렌딩의 주요 문제는 시스템 내 입자를 시점 기준 후면-전면 순서로 정렬해야 한다는 점입니다. 이는

비용이 많이 들고 실용적이지 않을 수 있습니다. 파티클 시스템의 무작위 특성으로 인해 이 규칙이 깨져도 눈에 띄는 렌더링 오류가 발생하지 않는 경우가 있습니다. 단, 한 장면에 많은 파티클 시스템이 존재할 경우 시스템 자체는 여전히 후면-전면 순서로 정렬되어야 하며, 단지 시스템 내 파티클 간 정렬은 생략하면 됩니다. 블렌딩을 사용할 때는 §9.5.4 및 §9.5.5의 논의 사항이 적용됨을 유의하십시오.



그림 20.3. 가산 혼합 방식에서는 더 많은 입자가 겹쳐지고 합쳐지는 소스 지점 근처에서 강도가 더 강해집니다. 입자가 퍼져 나갈수록 겹쳐지고 합쳐지는 입자가 줄어들기 때문에 강도가 약해집니다.

20.5 스트림-아웃

GPU가 텍스처에 쓰기 작업을 수행할 수 있다는 것은 잘 알려져 있습니다. 예를 들어, GPU는 깊이/스텐실 버퍼와 백 버퍼에 대한 쓰기 작업을 처리합니다. Direct3D 10에서 도입된 기능 중 하나는 스트림 아웃(SO) 단계입니다. SO 단계는 GPU가 실제로ジオ메트리(버텍스 리스트 형태)를 파이프라인의 SO 단계에 바인딩된 버텍스 버퍼 *VO* 쓸 수 있도록 합니다. 구체적으로,ジオ메트리 셰이더에서 출력된 버텍스는 *VO* 쓰여지거나 스트림 아웃됩니다. *VO 저장된*ジオ메트리는 이후에 렌더링될 수 있습니다. [그림 20.4](#)는 이 개념을 설명합니다(테셀레이션 단계는 생략됨). 스트림 아웃은 우리의 파티클 시스템 프레임워크에서 중요한 역할을 할 것입니다.

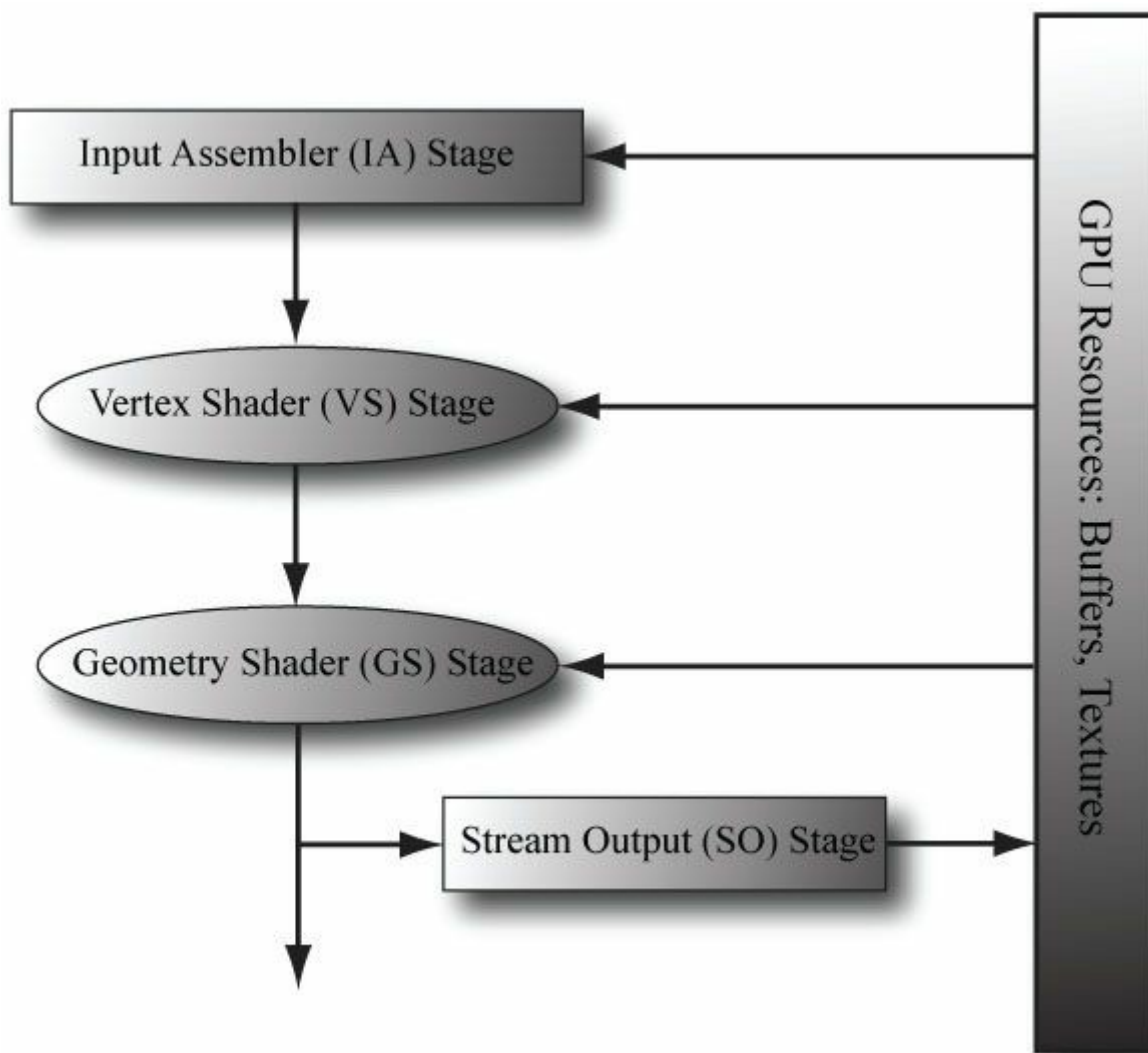


그림 20.4. 프리미티브가 파이프라인을 통과합니다. 지오메트리 셰이더는 프리미티브를 출력하며, 이는 GPU 메모리의 버텍스 버퍼로 스트림 아웃됩니다.

20.5.1 스트림 아웃용 지오메트리 셰이더 생성

스트림 아웃을 사용할 때는 지오메트리 셰이더를 특별히 생성해야 합니다. 다음 코드는 이 작업을 효과 파일에서 수행하는 방법을 보여줍니다.

```

GeometryShader gsStreamOut = ConstructGSWithSO( CompileShader(gs_5_0, GS()),
    "POSITION.xyz; VELOCITY.xyz; SIZE.xy; AGE.x; TYPE.x");

technique11 SOTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(gsStreamOut); SetPixelShader(CompileShader(ps_5_0, PS()));
    }
}
  
```

ConstructGSWithSO의 첫 번째 매개변수는 컴파일된 지오메트리 셰이더 자체입니다. 두 번째 매개변수는 스트림 아웃되는 버텍스의 형식(즉, 지오메트리 셰이더가 출력하는 버텍스의 형식)을 설명하는 문자열입니다. 이전 예시에서 버텍스 형식은 다음과 같습니다.

```

struct Vertex
{
    float3 InitialPosW : POSITION; float3
    InitialVelW : VELOCITY; float2 SizeW : SIZE;
}
  
```

```
float Age : AGE;
uint Type : TYPE;
};
```

20.5.2 스트림 아웃 전용

일반적인 조건에서 스트림 아웃을 사용할 경우, 지오메트리 셰이더 출력은 버텍스 버퍼로 스트림 아웃되며 렌더링 파이프라인의 다음 단계(래스터화)로 계속 진행됩니다. 데이터를 스트림 아웃만 하고 렌더링하지 않는 기법을 원한다면 픽셀 셰이더와 깊이/스텐실 버퍼를 비활성화해야 합니다. (픽셀 셰이더와 깊이/스텐실 버퍼를 비활성화하면 래스터화가 비활성화됩니다.) 다음 기법은 이를 구현하는 방법을 보여줍니다 :

```
DepthStencilState DisableDepth
{
    DepthEnable = FALSE; DepthWriteMask =
    ZERO;
};

GeometryShader gsStreamOut = ConstructGSWithSO( CompileShader(gs_5_0, StreamOutGS()),
    "POSITION.xyz; VELOCITY.xyz; SIZE.xy; AGE.x; TYPE.x");

technique10 StreamOutTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, StreamOutVS())); SetGeometryShader(gsStreamOut);

        // 스트림 아웃 전용으로 픽셀 셰이더 비활성화 SetPixelShader(NULL);

        // 스트림 아웃 전용으로 깊이 버퍼도 비활성화해야 함 SetDepthStencilState(DisableDepth,
        0);
    }
}
```

또는 렌더링 파이프라인의 출력 병합 단계에 null 렌더 타깃과 null 깊이/스텐실 버퍼를 바인딩할 수도 있습니다.
당사 파티클 시스템에서는 스트림 아웃 전용 기법을 사용할 예정입니다. 이 기법은 파티클 생성 및 소멸(즉, 파티클 시스템 업데이트)에만 사용됩니다.
(즉, 파티클 시스템 업데이트). 매 프레임마다:

1. 현재 파티클 리스트는 스트림 아웃 전용으로 렌더링됩니다. 래스터라이제이션 유닛이 비활성화되어 있으므로 화면에 파티클이 표시되지 않습니다.
2. 이 패스의 지오메트리 셰이더는 파티클 시스템마다 다른 다양한 조건에 따라 파티클을 생성하거나 소멸시킵니다.
3. 업데이트된 파티클 목록은 버텍스 버퍼로 스트리밍됩니다.

그런 다음 애플리케이션은 다른 렌더링 기법을 사용하여 해당 프레임에 대해 업데이트된 파티클 목록을 그립니다. 두 가지 기법을 사용하는 주된 이유는 지오메트리 셰이더가 서로 다른 작업을 수행하기 때문입니다. 스트림 아웃 전용 기법의 경우, 지오메트리 셰이더는 파티클을 입력받아 업데이트한 후 파티클을 출력합니다. 렌더링 기법의 경우, 지오메트리 셰이더는 카메라를 향하는 사각형으로 점을 확장하는 작업을 수행합니다. 따라서 지오메트리 셰이더들은 동일한 유형의 프리미티브를 출력하지도 않으므로, 두 개의 지오메트리 셰이더가 필요합니다.
요약하자면, GPU에서 파티클 시스템을 렌더링하기 위해 두 가지 기법이 필요합니다:

1. 입자 시스템을 업데이트하는 데 사용되는 스트림 아웃 전용 기법.
2. 입자 시스템을 그리는 데 사용되는 렌더링 기법.

*스트림 아웃 전용 패스에서 입자 물리도 점진적으로 업데이트될 수 있습니다. 그러나 우리의 설정에서는 명시적인 위치 함수 $p(t)$ 가 존재합니다. 따라서 스트림 아웃 전용 패스에서 입자의 위치/속도를 점진적으로 업데이트할 필요가 없습니다. 다만 *ParticlesGS SDK* 샘플은 다른 물리 모델을 사용하기 때문에 물리 업데이트에 스트림 아웃 전용 패스를 활용합니다.*

Note:

20.5.3 스트림 아웃용 버텍스 버퍼 생성

GPU가 버텍스를 쓸 수 있도록 버텍스 버퍼를 SO 단계에 바인딩하려면 **D3D11_BIND_STREAM_OUTPUT** 바인딩 플래그로 생성해야 합니다. 일반적으로 스트림 아웃 대상으로 사용되는 버텍스 버퍼는 나중에 파이프라인의 입력으로도 사용됩니다(즉, 내용을 렌더링할 수 있도록 IA 단계에 바인딩됨). 따라서 **D3D11_BIND_VERTEX_BUFFER** 바인딩 플래그도 반드시 지정해야 합니다. 다음 코드 조각은 스트림 아웃 용도로 버텍스 버퍼를 생성하는 예시를 보여줍니다:

```
D3D11_BUFFER_DESC vbd;
vbd.Usage = D3D11_USAGE_DEFAULT; vbd.ByteWidth = sizeof(Vertex) *
MAX_VERTICES;
vbd.BindFlags = D3D11_BIND_VERTEX_BUFFER| D3D11_BIND_STREAM_OUTPUT;
vbd.CPUAccessFlags = 0;
vbd.MiscFlags = 0;

HR(md3dDevice->CreateBuffer(&vbd, 0, &mStreamOutVB));
```

버퍼 메모리는 초기화되지 않은 상태로 남겨집니다. 이는 GPU가 버텍스 데이터를 여기에 직접 기록하기 때문입니다. *또한 버퍼는 유한한 크기를 가지므로, 최대 용량을 초과할 정도로 많은 버텍스를 스트림 아웃하지 않도록 주의해야 합니다.*

20.5.4 SO 스테이지에 바인딩

D3D11_BIND_STREAM_OUTPUT 바인드 플래그로 생성된 버텍스 버퍼는 다음 메서드를 사용하여 파이프라인의 SO 단계에 바인딩하여 쓰기 작업에 사용할 수 있습니다:

```
void ID3D11DeviceContext::SOSetTargets( UINT NumBuffers,
ID3D11Buffer *const *ppSOTargets, const UINT *pOffsets);
```

- 1. **NumBuffers**: SO 단계에 타겟으로 바인딩할 버텍스 버퍼의 개수입니다. 최대 4개까지 가능합니다.
- 2. **ppSOTargets**: SO 단계에 바인딩할 버텍스 버퍼 배열.
- 3. **pOffsets**: 각 버텍스 버퍼마다 하나씩, SO 단계가 버텍스 쓰기를 시작해야 하는 시작 위치를 나타내는 오프셋 배열입니다.

 스트림 아웃을 위한 네 개의 출력 슬롯이 있습니다. SO 단계에 네 개 미만의 버퍼가 바인딩된 경우, 나머지 슬롯은 **null**로 설정됩니다. 예를 들어, 슬롯 0(첫 번째 슬롯)에만 바인딩하면 슬롯 1, 2, 3은 **null**로 설정됩니다.

20.5.5 스트림 아웃 스테이지에서 바인딩 해제

정점 버퍼로 정점을 스트림 아웃한 후, 해당 정점이 정의하는 프리미티브를 그리려는 경우가 있을 수 있습니다. 그러나 정점 버퍼는 SO 스테이지와 IA 스테이지에 동시에 바인딩될 수 없습니다. SO 스테이지에서 정점 버퍼를 언바인딩하려면, 그 자리에 다른 버퍼(null 버퍼도 가능)를 SO 스테이지에 바인딩하기만 하면 됩니다. 다음 코드는 null 버퍼를 바인딩하여 슬롯 0의 버텍스 버퍼를 언바인딩합니다:

```
ID3D11Buffer* bufferArray[1] = {0};
md3dDeviceContext->SOSetTargets(1, bufferArray, &offset);
```

20.5.6 자동 드로우

버텍스 버퍼로 스트리밍된 지오메트리는 가변적일 수 있습니다. 그렇다면 몇 개의 버텍스를 그리는 걸까요? 다행히 Direct3D는 내부적으로 개수를 추적하며, SO로 버텍스 버퍼에 기록된 지오메트리를 그리기 위해 **ID3D11DeviceContext::DrawAuto** 메서드를 사용할 수 있습니다:

```
void ID3D11DeviceContext::DrawAuto();
```

 **DrawAuto 호출 전에, 먼저 스트림 출력 대상으로 사용된 버텍스 버퍼를**

- 1. **IA 스테이지의 입력 슬롯 0에 바인딩해야 합니다. DrawAuto 메서드는 D3D11_BIND_STREAM_OUTPUT 바인드 플래그가 IA 스테이지의 입력 슬롯 0에 바인딩된 경우에만 사용해야 합니다.**
- 2. **DrawAuto로 드로잉할 때에도 스트림 아웃 버텍스 버퍼 내 버텍스의 버텍스 입력 레이아웃을 반드시 지정해야 합니다.**
- 3. **DrawAuto는 인덱스를 사용하지 않습니다. 지오메트리 셰이더는 버텍스 리스트로 정의된 전체 프리미티브만 출력하기 때문입니다.**

20.5.7 핑퐁 버텍스 버퍼

앞서 언급한 바와 같이, 버텍스 버퍼는 OS 단계와 IA 단계에 동시에 바인딩될 수 없습니다. 따라서 핑퐁 방식이 사용됩니다. 스트림 아웃으로 드로잉할 때는 두 개의 버텍스 버퍼를 사용합니다. 하나는 입력 버퍼로, 다른 하나는 출력 버퍼로 사용됩니다. 다음 렌더링 프레임에서는 두 버퍼의 역할이 반전됩니다. 방금 스트림 아웃된 버퍼가 새로운 입력 버퍼가 되고, 기존 입력 버퍼는 새로운 스트림 아웃 대상이 됩니다. 다음 표는 정점 버퍼 V_0 과 V_1 을 사용한 세 번의 반복을 보여줍니다.

	IA 스테이지에 바인딩된 입력 버텍스 버퍼	출력 버텍스 버퍼 (SO 스테이지에 바인딩)
프레임 i	V_0	V_1
프레임 $i+1$	V_1	V_0
프레임 $i+2$	V_0	V_1

20.6 GPU 기반 파티클 시스템

입자 시스템은 일반적으로 시간이 지남에 따라 입자를 생성하고 소멸시킵니다. 이를 수행하는 가장 자연스러운 방법은 동적 버텍스 버퍼를 사용하고 CPU에서 생성 및 소멸되는 입자를 추적하는 것입니다. 그런 다음 버텍스 버퍼는 현재 생존 중인 입자로 채워져 렌더링됩니다. 그러나 이전 섹션에서 살펴본 바와 같이, 별도의 스트림-아웃 전용 패스로 이 생성/소멸 업데이트 사이클을 GPU에서 완전히 처리할 수 있습니다. 이는 효율성 때문입니다—CPU가 GPU로 데이터를 업로드할 때마다 일부 오버헤드가 발생합니다. 또한 이 작업은 CPU에서 GPU로 작업을 이동시켜 CPU가 AI나 물리 계산 같은 다른 작업에 집중할 수 있게 합니다. 이 섹션에서는 당사 파티클 시스템 프레임워크의 일반적인 세부 사항을 설명합니다.

20.6.1 입자 효과

특정 파티클 시스템의 동작 방식에 대한 세부 사항은 효과 파일에서 구현됩니다. 즉, 각 파티클 시스템(예: 비, 불, 연기 등)마다 서로 다르지만 유사한 효과 파일이 존재합니다. 파티클이 생성, 소멸, 렌더링되는 방식의 세부 사항은 시스템마다 다르기 때문에 해당 효과 파일에 모두 스크립트로 작성됩니다. 예시:

1. 빗방울 입자는 지면에 닿으면 소멸되지만, 불 입자는 몇 초 후에 소멸될 수 있습니다.
2. 연기 입자는 시간이 지남에 따라 서서히 사라질 수 있지만, 비 입자는 그렇지 않을 수 있습니다. 마찬가지로 연기 입자의 크기는 시간이 지남에 따라 커질 수 있지만, 비 입자는 그렇지 않을 수 있습니다.
3. 선 프리미티브는 일반적으로 비 모델링에 효과적이며, 반면 빌보드 쿼드는 불/연기 입자에 사용됩니다. 서로 다른 입자 시스템에 다른 효과를 적용함으로써, 지오메트리 셰이더가 비에는 점을 선으로, 불/연기에는 점을 쿼드로 확장하도록 할 수 있습니다.
4. 비와 연기 입자의 초기 위치와 속도는 분명히 다를 것입니다.

다시 말해, 이러한 입자 시스템별 세부 사항은 효과 파일에서 구현할 수 있습니다. 우리 시스템에서는 셰이더 코드가 입자의 생성, 소멸, 업데이트를 처리하기 때문입니다. 이 설계는 매우 편리합니다. 새로운 입자 시스템을 추가하려면 그 동작을 설명하는 새로운 효과 파일을 스크립팅하기만 하면 되기 때문입니다.

20.6.2 입자 시스템 클래스

다음에 제시된 클래스는 파티클 시스템 생성, 업데이트, 렌더링과 관련된 C++ 코드를 처리합니다. 이 코드는 일반적이며 우리가 생성하는 모든 파티클 시스템에 적용됩니다.

```
class ParticleSystem
{
public:

    ParticleSystem();
    ~ParticleSystem();

    // 시스템이 재설정된 이후 경과 시간. float GetAge()const;

    void SetEyePos(const XMFLOAT3& eyePosW); void SetEmitPos(const XMFLOAT3& emitPosW);
```

```
void SetEmitDir(const XMFLOAT3& emitDirW);

void Init(ID3D11Device* device, ParticleEffect* fx,
          ID3D11ShaderResourceView* texArraySRV,
          ID3D11ShaderResourceView* randomTexSRV, UINT maxParticles);

void Reset();
void Update(float dt, float gameTime);
void Draw(ID3D11DeviceContext* dc, const Camera& cam);
```

```
private:
    void BuildVB(ID3D11Device* device);

    ParticleSystem(const ParticleSystem& rhs); ParticleSystem& operator=(const
    ParticleSystem& rhs);
```

```
private:

    UINT mMaxParticles; bool
    mFirstRun;

    float mGameTime; float
    mTimeStep; float mAge;

    XMFLOAT3 mEyePosW;
    XMFLOAT3 mEmitPosW; XMFLOAT3
    mEmitDirW;

    ParticleEffect* mFX;

    ID3D11Buffer* mInitVB; ID3D11Buffer*
    mDrawVB; ID3D11Buffer* mStreamOutVB;

    ID3D11ShaderResourceView* mTexArraySRV; ID3D11ShaderResourceView*
    mRandomTexSRV;
};
```

드로우 메서드를 제외하고, 파티클 시스템 클래스 메서드들의 구현은 이제 상당히 일상적입니다(예: 버텍스 버퍼 생성); 따라서 이 책에서는 이를 보여주지 않습니다(해당 장의 소스 코드를 참조하십시오).

입자 시스템은 입자 텍스처링을 위해 텍스처 배열을 사용합니다. 모든 입자가 정확히 동일하게 보이길 원하지 않을 수 있다는 개념입니다. 예를 들어 연기 입자 시스템을 구현할 때, 다양성을 더하기 위해 여러 연기 텍스처를 사용하고 싶을 수 있습니다. 프라이머리 ID를 픽셀 셰이더에서 사용해 텍스처 배열 내 연기 텍스처들을 번갈아 적용할 수 있습니다.

 **Note:**

20.6.3 방출기 입자

지오메트리 셰이더가 파티클 생성/파괴를 담당하기 때문에, 특별한 이미터 파티클이 필요합니다. 이미터 파티클은 렌더링될 수도 있고 아닐 수도 있습니다. 예를 들어, 이미터 파티클을 보이지 않게 하려면 파티클 렌더링 지오메트리 셰이더가 이를 출력하지 않도록 설정하기만 하면 됩니다. 이미터 파티클은 다른 파티클과 달리 시스템 내에서 다른 파티클을 생성할 수 있다는 점에서 다르게 동작합니다. 예를 들어, 이미터 파티클은 경과 시간을 추적하다가 특정 시점에 도달하면 새 파티클을 방출할 수 있습니다. 이렇게 시간이 지남에 따라 새로운 파티클이 생성됩니다. Vertex::Particle 구조체의 **Type** 멤버를 사용하여 이미터 파티클을 표시합니다. 또한, 어떤 입자가 다른 입자를 생성할 수 있는지 제한함으로써 입자 생성 방식을 어느 정도 제어할 수 있습니다. 예를 들어, 단 하나의 에미터 입자만 존재하도록 하면 프레임당 생성되는 입자 수를 쉽게 제어할 수 있습니다. 스트림 전용 지오메트리 셰이더는 항상 최소한 하나의 에미터 입자를 출력해야 합니다. 입자 시스템이 모든 에미터를 잃으면 사실상 작동이 중단되기 때문입니다. 다만 일부 입자 시스템에서는 이것이 의도된 결과일 수도 있습니다.

이 책에서 시연하는 파티클 시스템은 파티클 시스템의 수명 동안 단 하나의 이미터 파티클만 사용합니다. 그러나 파티클 시스템 프레임워크는 필요 시 더 많은 요소를 사용하도록 확장될 수 있습니다. 또한 다른 입자들도 입자를 방출할 수 있습니다. 예를 들어, SDK의 *ParticlesGS* 데모에는 런처 입자(셀 입자를 생성하는 보이지 않는 입자), 셀 입자(시간이 지나면 새로운 불씨 입자로 폭발하는 미폭발 불꽃), 그리고 이 불씨들 중 일부는 2차 폭발을 생성하기 위해 새로운 2차 불씨 입자로 폭발하기도 합니다. 이러한 의미에서, 발사체 입자는 다른 발사체 입자를 방출할 수 있습니다. 연습 1에서는 이러한 발사체 입자를 생성하는 방법을 묻습니다.

불꽃이 시간이 지나면 새로운 불씨 입자로 폭발하는 것)이 있으며, 이 불씨 중 일부는 다시 새로운 2차 불씨 입자로 폭발하여 2차 폭발을 생성합니다. 이러한 의미에서 방출기 입자는 다른 방출기 입자를 방출할 수 있습니다. 연습 1에서는 이를 탐구해 보도록 요청합니다.

20.6.4 초기화 버텍스 버퍼

우리의 입자 시스템에는 특별한 초기화 버텍스 버퍼가 있습니다. 이 버텍스 버퍼에는 단 하나의 발사체 입자만 포함됩니다. 이 버텍스 버퍼를 먼저 렌더링하여 파티클 시스템을 시작합니다. 이후 이 발사체 파티클이 후속 프레임에서 다른 파티클을 생성하기 시작합니다. 초기화 버텍스 버퍼는 시스템이 재설정되는 경우를 제외하고 단 한 번만 렌더링된다는 점에 유의하십시오. 단일 발사체 파티클로 파티클 시스템이 초기화된 후, 두 개의 스트림-아웃 버텍스 버퍼를 핑퐁 방식으로 사용합니다.

초기화 버텍스 버퍼는 시스템을 처음부터 재시작해야 할 때도 유용합니다. 이 코드를 사용해 :

```
void ParticleSystem::Reset()
{
    mFirstRun = true; mAge = 0.0f;
}

// ...

mFirstRun을 true로 설정하면 입자 시스템이 다음 드로우 호출에서 초기화 버텍스 버퍼를 그리도록 지시하여, 단일 이미터 입자로 입자 시스템을 재시작합니다.
```

20.6.5 Update/Draw 메서드

GPU에서 파티클 시스템을 렌더링하려면 두 가지 기법이 필요합니다:

- 1. 입자 시스템을 업데이트하는 데 사용되는 스트림 아웃 전용 기법.
- 2. 입자 시스템을 그리는 데 사용되는 렌더링 기법.

다음 코드는 두 버텍스 버퍼의 핑퐁 방식을 처리하는 것 외에도 이를 수행합니다:

```
void ParticleSystem::Draw(ID3D11DeviceContext* dc, const Camera& cam)
{
    XMATRIX VP = cam.ViewProj();

    //
    // 상수 설정.
    //
    mFX->SetViewProj(VP);
    mFX->SetGameTime(mGameTime); mFX->SetTimeStep(mTimeStep); mFX->SetEyePosW(mEyePosW); mFX->SetEmitPosW(mEmitPosW); mFX->SetEmitDirW(mEmitDirW); mFX->SetTexArray(mTexArraySRV);
    mFX->SetRandomTex(mRandomTexSRV);

    //
    // IA 스테이지 설정.
    //
    dc->IASetInputLayout(InputLayouts::Particle);
    dc->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_POINTLIST); UINT stride = sizeof(Vertex::Particle);
    UINT 오프셋 = 0;

    // 첫 번째 패스에서는 초기화 VB를 사용합니다. 그렇지 않으면
    // 현재 파티클 목록을 포함하는 VB를 사용합니다. if( mFirstRun )
        dc->IASetVertexBuffers(0, 1, &mInitVB, &stride, &offset); else
        dc->IASetVertexBuffers(0, 1, &mDrawVB, &stride, &offset);

    //
    // 스트림 아웃만 사용하여 현재 파티클 목록을 업데이트하며 렌더링합니다.
```

```

// 업데이트된 정점 정보는 대상 버텍스 버퍼로 스트림 아웃됩니다.
//
dc->SOSetTargets(1, &mStreamOutVB, &offset);

D3DX11_TECHNIQUE_DESC techDesc;
mFX->StreamOutTech->GetDesc(&techDesc); for(UINT p = 0; p <
techDesc.Passes; ++p)
{
    mFX->StreamOutTech->GetPassByIndex( p )->Apply(0, dc);

    if(mFirstRun)
    {
        dc->Draw(1, 0); mFirstRun = false;

    }
    else
    {
        dc->DrawAuto();

    }
}

// 스트리밍 완료-버텍스 버퍼 해제 ID3D11Buffer* bufferArray[1] = {0};
dc->SOSetTargets(1, bufferArray, &offset);

// 버텍스 버퍼를 핑퐁 처리 std::swap(mDrawVB, mStreamOutVB);

//
// 방금 스트리밍한 업데이트된 파티클 시스템을 렌더링합니다.
//
dc->IASetVertexBuffers(0, 1, &mDrawVB, &stride, &offset);

mFX->DrawTech->GetDesc(&techDesc); for(UINT p = 0; p <
techDesc.Passes; ++p)
{
    mFX->DrawTech->GetPassByIndex(p)->Apply(0, dc);

    dc->DrawAuto();

}
}

```

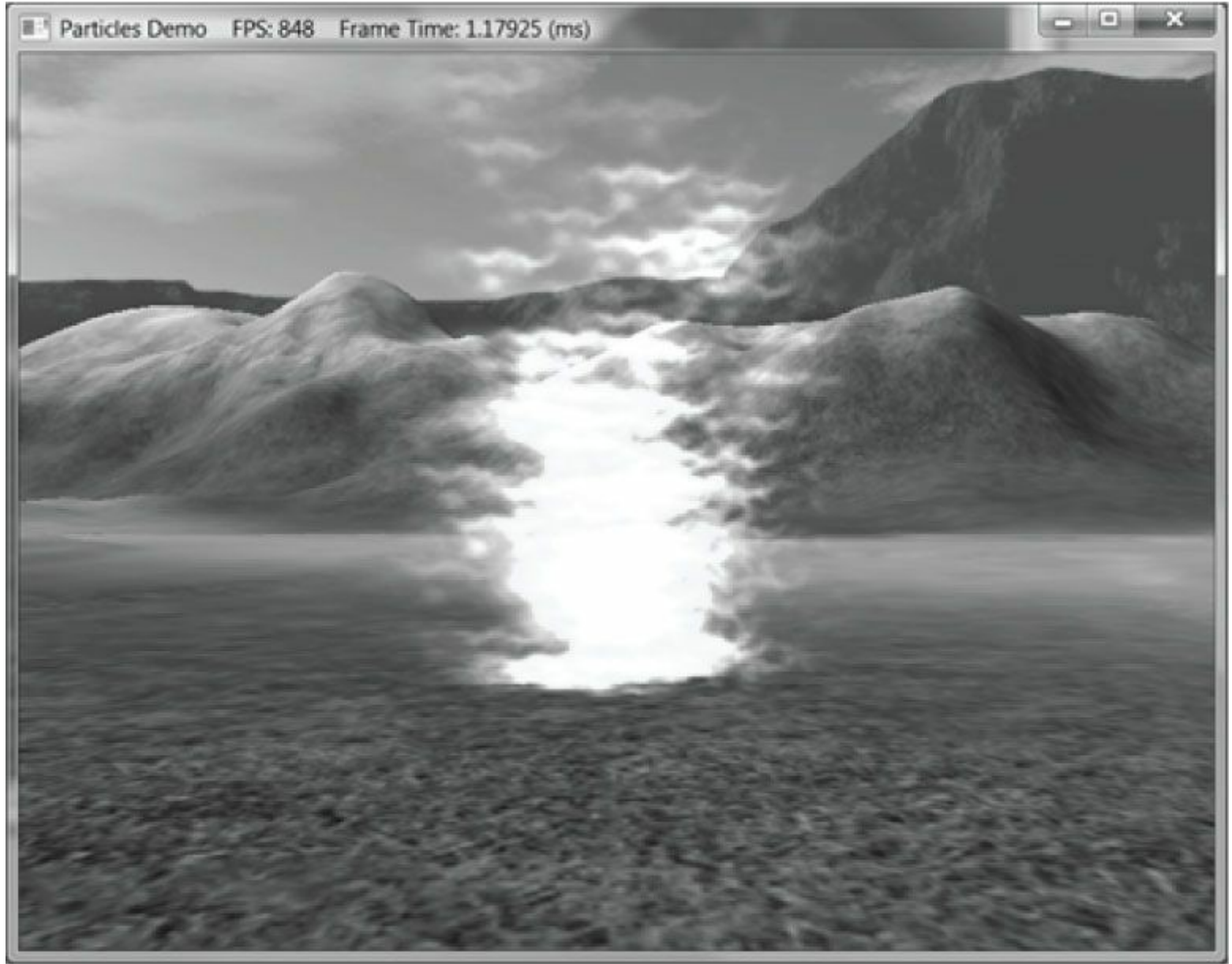


그림 20.5. 불을 보여주는 파티클 시스템 데모의 스크린샷.

20.7 FIRE

다음은 불 파티클 시스템을 렌더링하는 효과입니다. 두 가지 기법으로 구성됩니다.

1. 입자 시스템을 업데이트하는 데 사용되는 스트림 아웃 전용 기법.
2. 파티클 시스템을 그리는 데 사용되는 렌더링 기법.

파괴/생성/렌더링 규칙이 다르기 때문에 이 두 기법에 프로그래밍된 로직은 일반적으로 파티클 시스템마다 다릅니다. 불꽃 파티클은 방출 위치에서 배출되지만, 불덩이를 생성하기 위해 불꽃이 퍼지도록 무작위 초기 속도가 부여됩니다.

```

//*****

// 전역 변수 *
//*****

cbuffer cbPerFrame
{
    float3 gEyePosW;

    // 방출 위치/방향이 변동될 때 사용 float3 gEmitPosW;
    float3 gEmitDirW; float
    gGameTime;

```

```

        float gTimeStep; float4x4
        gViewProj;
    };

cbuffer cbFixed
{
    // 입자 가속을 위한 순수 상수 가속도. float3 gAccelIW = {0.0f, 7.8f, 0.0f};

    // 쿼드 위에 텍스처를 늘리기 위해 사용되는 텍스처 좌표
    // 점 입자를 사각형으로 확장할 때 사용됩니다. float2 gQuadTexC[4] =
    {
        float2(0.0f, 1.0f),
        float2(1.0f, 1.0f),
        float2(0.0f, 0.0f), float2(1.0f, 0.0f)
    };
};

// 입자 텍스처링용 텍스처 배열. Texture2DArray gTexArray;

// 셰이더에서 난수를 생성하는 데 사용되는 랜덤 텍스처. Texture1D gRandomTex;

SamplerState samLinear
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU =
    WRAP;
    AddressV = WRAP;
};

DepthStencilState DisableDepth
{
    DepthEnable = FALSE; DepthWriteMask =
    ZERO;
};

DepthStencilState NoDepthWrites
{
    DepthEnable = TRUE; DepthWriteMask =
    ZERO;
};

BlendState AdditiveBlending
{
    AlphaToCoverageEnable = FALSE; BlendEnable[0] = TRUE;
    SrcBlend = SRC_ALPHA; DestBlend =
    ONE; BlendOp = ADD; SrcBlendAlpha
    = ZERO; DestBlendAlpha = ZERO;
    BlendOpAlpha = ADD;
    RenderTargetWriteMask[0] = 0x0F;
};

//*****

// 헬퍼 함수 *
//***** float3 RandUnitVec3(float
offset)
{
    // 게임 시간에 오프셋을 더해 무작위 텍스처를 샘플링합니다.

```

```

float u = (gGameTime + offset);

// 좌표 범위 [-1,1]
float3 v = gRandomTex.SampleLevel(samLinear, u, 0).xyz;

// 단위 구에 투영 return normalize(v);
}

//*****

// 스트림아웃 기술 *
//*****

#define PT_EMITTER 0
#define PT_FLARE 1

struct Particle
{
    float3 InitialPosW : POSITION; float3 InitialVelW
    : VELOCITY; float2 SizeW : SIZE;
    float Age : AGE; uint Type :
    TYPE;
};

Particle StreamOutVS(Particle vin)
{
    return vin;
}

// 스트림 아웃 GS는 단순히 방출만 담당합니다
// 새로운 파티클 생성 및 기존 파티클 파괴.
// 여기에서 프로그래밍된 로직은 일반적으로 파티클 시스템마다 다릅니다.
// 생성 규칙이 다르기 때문입니다.
// 다르기 때문입니다.
[maxvertexcount(2)]
void StreamOutGS(point Particle gin[1],
                 inout PointStream<Particle> ptStream)
{
    gin[0].Age += gTimeStep;

    if(gin[0].Type == PT_EMITTER)
    {
        // 새 입자를 방출할 시간인가? if(gin[0].Age >
        0.005f)
        {
            float3 vRandom = RandUnitVec3(0.0f); vRandom.x *= 0.5f;
            vRandom.z *= 0.5f;

            Particle p;
            p.InitialPosW = gEmitPosW.xyz; p.InitialVelW =
            4.0f*vRandom; p.SizeW =
            float2(3.0f, 3.0f); p.Age = 0.0f;
            p.Type = PT_FLARE;

            ptStream.Append(p);

            // 방출 시간을 초기화 gin[0].Age = 0.0f;
        }

        // 항상 방출기를 유지

```

```

        ptStream.Append(gin[0]);
    }
    else
    {
        // 입자를 유지할 조건을 지정합니다; 시스템마다 다를 수 있습니다

        // 시스템마다 다를 수 있음. if(gin[0].Age
        <= 1.0f)

            ptStream.Append(gin[0]);
    }
}

GeometryShader gsStreamOut = ConstructGSWithSO( CompileShader(gs_5_0, StreamOutGS()),
    "POSITION.xyz; VELOCITY.xyz; SIZE.xy; AGE.x; TYPE.x");

technique11 StreamOutTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, StreamOutVS())); SetGeometryShader(gsStreamOut);

        // 스트림 아웃 전용으로 픽셀 셰이더 비활성화 SetPixelShader(NULL);

        // 스트림 아웃 전용으로 깊이 버퍼도 비활성화해야 함 SetDepthStencilState(DisableDepth,
        0);
    }
}

//*****
// DRAW TECH *
//*****

struct VertexOut
{
    float3 PosW : POSITION; float2 SizeW
    : SIZE; float4 Color : COLOR; uint
    Type : TYPE;
};

VertexOut DrawVS(Particle vin)
{
    VertexOut vout; float t =

    vin.Age;

    // 일정한 가속도 방정식
    vout.PosW = 0.5f*t*t*gAccelW + t*vin.InitialVelW + vin.InitialPosW;

    // 시간에 따른 색상 페이드
    float opacity = 1.0f - smoothstep(0.0f, 1.0f, t/1.0f); vout.Color = float4(1.0f, 1.0f,
    1.0f, opacity);

    vout.SizeW = vin.SizeW; vout.Type =
    vin.Type;

    return vout;
}

struct GeoOut
{
    float4 PosH : SV_Position; float4 Color :
    COLOR;

```

```

float2 Tex : TEXCOORD;
};

// 드로우 GS는 단순히 점을 카메라를 향한 사각형으로 확장합니다. [maxvertexcount(4)]
void DrawGS(point VertexOut gin[1],
            inout TriangleStream<GeoOut> triStream)
{
    // 이미터 입자는 그리지 않습니다. if(gin[0].Type !=
    PT_EMITTER)
    {
        //
        // 빌보드가 카메라를 향하도록 월드 매트릭스를 계산합니다.
        //
        float3 look = normalize(gEyePosW.xyz - gin[0].PosW); float3 right =
        normalize(cross(float3(0,1,0), look)); float3 up = cross(look, right);

        //
        // 월드 공간에서 삼각형 스트림 정점(사각형)을 계산합니다.
        //
        float halfWidth = 0.5f*gin[0].SizeW.x; float halfHeight =
        0.5f*gin[0].SizeW.y;

        float4 v[4];
        v[0] = float4(gin[0].PosW + halfWidth*right - halfHeight*up, 1.0f); v[1] = float4(gin[0].PosW +
        halfWidth*right + halfHeight*up, 1.0f); v[2] = float4(gin[0].PosW - halfWidth*right - halfHeight*up,
        1.0f); v[3] = float4(gin[0].PosW - halfWidth*right + halfHeight*up, 1.0f);

        //
        // 사각형 정점을 월드 공간으로 변환하고
        // 삼각형 스트림으로 출력합니다.
        //
        GeoOut gout; [unroll]
        for(int i = 0; i < 4; ++i)
        {
            gout.PosH = mul(v[i], gViewProj); gout.Tex =
            gQuadTexC[i]; gout.Color = gin[0].Color;
            triStream.Append(gout);
        }
    }
}

float4 DrawPS(GeoOut pin) : SV_TARGET
{
    return gTexArray.Sample(samLinear, float3(pin.Tex, 0))*pin.Color;
}

technique11 DrawTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, DrawVS()));
        SetGeometryShader(CompileShader(gs_5_0, DrawGS()));
        SetPixelShader(CompileShader(ps_5_0, DrawPS()));

        SetBlendState(AdditiveBlending, float4(0.0f, 0.0f, 0.0f, 0.0f), 0xffffffff); SetDepthStencilState(NoDepthWrites, 0);
    }
}

```

20.8 비

우린 또한 비 입자 시스템을 구현합니다. 비 입자 시스템의 동작은 비 효과(*rain.fx*)에 의해 지정됩니다. *fire.fx*와 유사한 패턴을 따르지만, 파괴/생성/렌더링 규칙은 다릅니다. 예를 들어, 우리 비방울들은 약간의 각도로 아래로 가속되는 반면, 불 입자들은 위로 가속됩니다. 또한 비 입자들은 사각형 대신 선으로 확장됩니다([그림 20.6](#) 참조). 빗방울은 카메라 상공의 무작위 위치에서 방출됩니다. 비는 항상 카메라를 "추적"하므로 세계 전체에 빗방울을 방출할 필요가 없습니다. 즉, 카메라 근처에서만 빗방울을 방출해도 비가 내리는 듯한 착각을 줄 수 있습니다. 비 시스템은 블렌딩을 전혀 사용하지 않는다는 점에 유의하십시오.

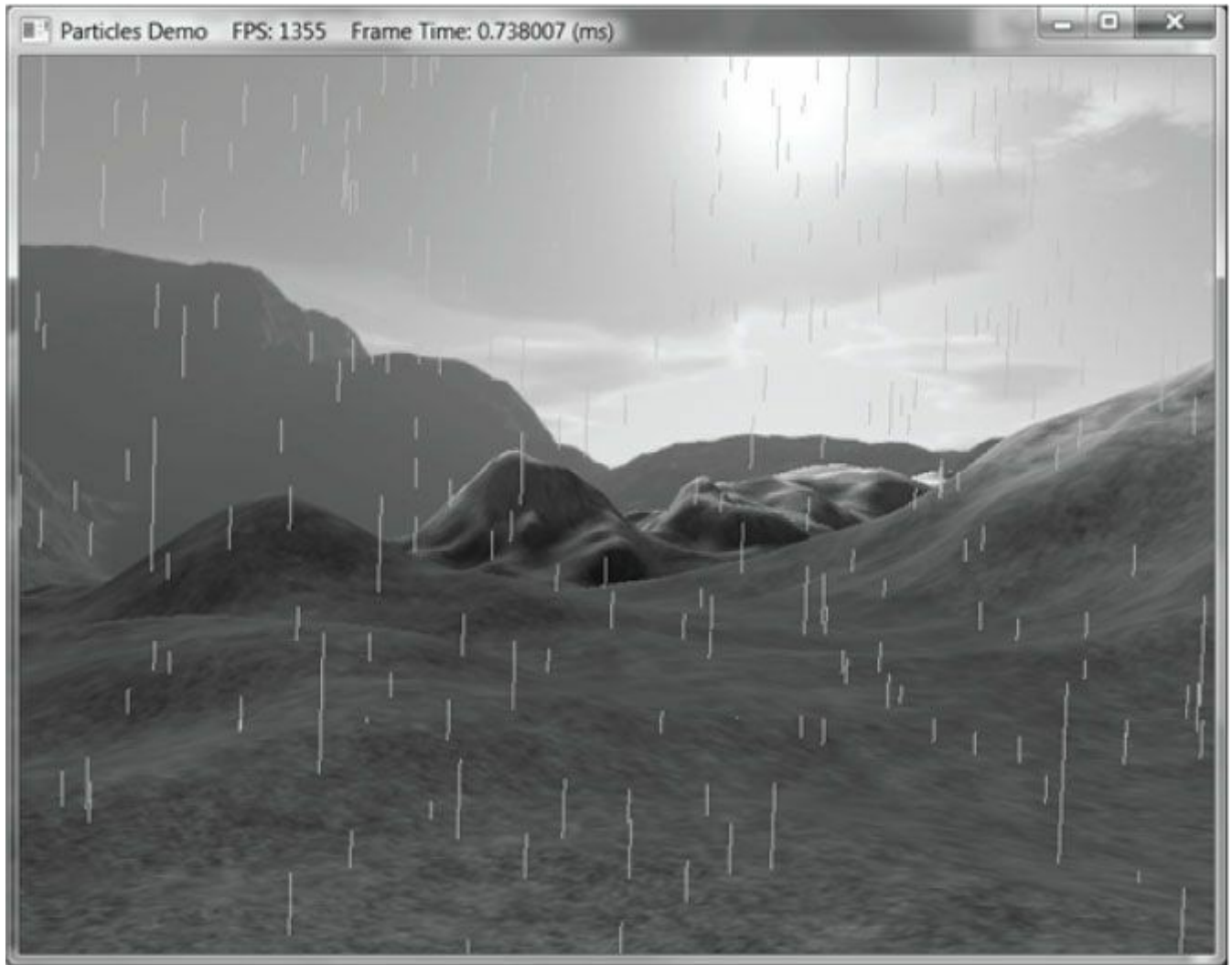


그림 20.6. 비를 보여주는 파티클 시스템 데모 스크린샷.

```
//*****  
// 전역 변수 *  
//*****  
  
cbuffer cbPerFrame  
{  
    float3 gEyePosW;  
  
    // 방출 위치/방향이 변할 때 사용 float3 gEmitPosW;  
    float3 gEmitDirW;  
  
    float gGameTime; float  
    gTimeStep; float4x4  
    gViewProj;  
};  
  
cbuffer cbFixed  
{
```

```

// 입자 가속을 위한 순수 상수 가속도. float3 gAccelW = {-1.0f, -9.8f, 0.0f};
};

// 입자 텍스처링용 텍스처 배열. Texture2DArray gTexArray;

// 셰이더에서 난수를 생성하는 데 사용되는 랜덤 텍스처. Texture1D gRandomTex;

SamplerState samLinear
{
    Filter = MIN_MAG_MIP_LINEAR; AddressU =
    WRAP;

    AddressV = WRAP;
};

DepthStencilState DisableDepth
{
    DepthEnable = FALSE; DepthWriteMask =
    ZERO;
};

DepthStencilState NoDepthWrites
{
    DepthEnable = TRUE; DepthWriteMask =
    ZERO;
};

//*****
// 헬퍼 함수 *
//***** float3 RandUnitVec3(float
offset)
{
    // 게임 시간에 오프셋을 더해 무작위 텍스처를 샘플링합니다. float u = (gGameTime +
offset);

    // 좌표 범위 [-1,1]
    float3 v = gRandomTex.SampleLevel(samLinear, u, 0).xyz;

    // 단위 구에 투영 return normalize(v);
}

float3 RandVec3(float offset)
{
    // 게임 시간에 오프셋을 더해 무작위 텍스처를 샘플링합니다. float u = (gGameTime +
offset);

    // 좌표 범위 [-1,1]
    float3 v = gRandomTex.SampleLevel(samLinear, u, 0).xyz;

    return v;
}

//*****
// STREAM-OUT TECH *
//*****

#define PT_EMITTER 0
#define PT_FLARE 1

struct Particle
{
    float3 InitialPosW : POSITION; float3 InitialVelW
: VELOCITY;

```

```

        float2 SizeW : SIZE; float Age
        : AGE; uint Type : TYPE;
};

Particle StreamOutVS(Particle vin)
{
    return vin;
}

// 스트림 아웃 GS는 단순히
// 새 입자를 생성하고 기존 입자를 소멸시키는 역할만 합니다.
// 여기에서 프로그래밍된 로직은 일반적으로 파티클 시스템마다 다릅니다.
// 파괴/생성 규칙이 다르기 때문입니다.
// 다르기 때문입니다.
[maxvertexcount(6)]
void StreamOutGS(point Particle gin[1], inout
    PointStream<Particle> ptStream)
{
    gin[0].Age += gTimeStep;

    if(gin[0].Type == PT_EMITTER)
    {
        // 새로운 입자를 방출할 시간인가? if(gin[0].Age >
        0.002f)
        {
            for(int i = 0; i < 5; ++i)
            {
                // 카메라 위쪽으로 빔방울을 퍼뜨립니다. float3 vRandom =
                35.0f*RandVec3((float)i/5.0f); vRandom.y = 20.0f;

                Particle p;
                p.InitialPosW = gEmitPosW.xyz + vRandom; p.InitialVelW =
                float3(0.0f, 0.0f, 0.0f); p.SizeW = float2(1.0f, 1.0f);
                p.Age = 0.0f;
                p.Type = PT_FLARE;

                ptStream.Append(p);
            }

            // 방출 시간을 초기화 gin[0].Age = 0.0f;
        }

        // 항상 방출기를 유지
        ptStream.Append(gin[0]);
    }
    else
    {
        // 입자를 유지할 조건 지정; 시스템마다 다를 수 있음
        // 시스템마다 다를 수 있음. if( gin[0].Age
        <= 3.0f ) ptStream.Append(gin[0]);
    }
}

GeometryShader gsStreamOut = ConstructGSWithSO( CompileShader(gs_5_0, StreamOutGS()),
    "POSITION.xyz; VELOCITY.xyz; SIZE.xy; AGE.x; TYPE.x");

technique11 StreamOutTech

```

```
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, StreamOutVS())); SetGeometryShader(gsStreamOut);

        // 스트림 아웃 전용으로 픽셀 셰이더 비활성화 SetPixelShader(NULL);

        // 스트림 아웃 전용으로 깊이 버퍼도 비활성화해야 함 SetDepthStencilState(DisableDepth,
        0);
    }
}
```

```
//*****
// DRAW TECH *
//*****
```

```
struct VertexOut
{
    float3 PosW : 위치; uint Type : 유형;
};

VertexOut DrawVS(Particle vin)
{
    VertexOut vout; float t =
    vin.Age;

    // 일정한 가속도 방정식
    vout.PosW = 0.5f*t*t*gAccelW + t*vin.InitialVelW + vin.InitialPosW; vout.Type = vin.Type;

    return vout;
}
```

```
struct GeoOut
{
    float4 PosH : SV_Position; float2 Tex :
    TEXCOORD;
};
```

// 드로우 GS는 단순히 점을 선으로 확장합니다. [maxvertexcount(2)]

```
void DrawGS(point VertexOut gin[1],
    inout LineStream<GeoOut> lineStream)
{
    // 이미터 입자는 그리지 않습니다. if(gin[0].Type !=
    PT_EMITTER)
    {
        // 가속 방향으로 기울어진 선. float3 p0 = gin[0].PosW;

        float3 p1 = gin[0].PosW + 0.07f*gAccelW;

        GeoOut v0;
        v0.PosH = mul(float4(p0, 1.0f), gViewProj); v0.Tex = float2(0.0f,
        0.0f); lineStream.Append(v0);

        GeoOut v1;
        v1.PosH = mul(float4(p1, 1.0f), gViewProj); v1.Tex = float2(1.0f,
        1.0f);
```

```
        lineStream.Append(v1);
    }
}

float4 DrawPS(GeoOut pin) : SV_TARGET
{
    return gTexArray.Sample(samLinear, float3(pin.Text, 0));
}

technique11 DrawTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, DrawVS()));
        SetGeometryShader(CompileShader(gs_5_0, DrawGS()));
        SetPixelShader(CompileShader(ps_5_0, DrawPS()));

        SetDepthStencilState(NoDepthWrites, 0);
    }
}
```

20.9 요약

1. 입자 시스템은 유사하면서도 다소 무작위적인 방식으로 행동하는 (일반적으로 작은) 입자들의 집합체입니다. 입자 시스템은 불, 비, 연기, 폭발, 스프링클러, 마법 주문 효과 등 다양한 현상을 시뮬레이션하는 데 활용될 수 있습니다.
2. 우리는 입자를 점으로 모델링하지만, 렌더링 전에 지오메트리 셰이더에서 카메라를 향하도록 쿼드로 확장합니다. 이는 점으로 작업할 때의 효율성(더 작은 메모리 사용량, 네 개의 쿼드 버텍스 대신 하나의 점 버텍스만 물리 효과를 적용하면 됨)을 얻을 수 있을 뿐만 아니라, 나중에 점을 쿼드로 확장함으로써 다양한 크기의 입자를 만들고 텍스처를 매핑할 수 있는 능력도 얻게 됨을 의미합니다. 포인트를 사각형으로 확장하는 것은 필수가 아닙니다. 예를 들어, 선분으로도 빗방울을 상당히 잘 모델링할 수 있습니다. 포인트를 선분으로 확장하기 위해 다른 지오메트리를 사용할 수 있습니다.
3. 일정한 가속도를 받는 입자의 궤적은 다음 식으로 주어집니다:
$$\mathbf{p}(t) = \frac{1}{2}t^2\mathbf{a} + t\mathbf{v}_0 + \mathbf{p}_0$$
 여기서 $\mathbf{a}$ 는 일정한 가속도 벡터, $\mathbf{v}_0$ 는 입자의 초기 속도(즉, 시간 $t = 0$ 에서의 속도), $\mathbf{p}_0$ 는 입자의 초기 위치(즉, 시간 $t = 0$ 에서의 위치)입니다. 이 방정식을 사용하면 $t \geq 0$ 인 모든 시간 t 에서 함수를 평가하여 입자의 위치를 구할 수 있습니다.
4. 입자 시스템의 강도가 입자 밀도에 비례하도록 하려면 가산 혼합을 사용하십시오. 투명한 입자의 경우 투명도 혼합을 사용하십시오. 투명한 입자 시스템을 뒤에서 앞으로 정렬하지 않는 것이 문제가 될 수도 있고 아닐 수도 있습니다(즉, 문제가 눈에 띄지 않을 수도 있음). 일반적으로 입자 시스템에서는 입자들이 서로 가리지 않도록 깊이 쓰기가 비활성화됩니다. 그러나 비입자 오브젝트가 입자를 가릴 수 있도록 깊이 테스트는 여전히 활성화됩니다.
5. 스트림 아웃(SO) 단계에서는 GPU가 지오메트리(버텍스 리스트 형태)를 파이프라인의 SO 단계에 바인딩된 *버텍스 버퍼 V*에 쓸 수 있습니다. 구체적으로 지오메트리 셰이더에서 출력된 버텍스가 *V*에 쓰여 지거나(스트림 아웃됨) 나중에 *V*의 지오메트리를 렌더링할 수 있습니다. 이 스트림 아웃 기능을 활용하여 GPU에서 완전히 실행되는 파티클 시스템을 구현합니다. 이를 위해 두 가지 기법을 사용합니다:

입자 시스템을 업데이트하는 데 사용되는 스트림 아웃 전용 기법입니다. 이 렌더링 패스에서 지오메트리 셰이더는 입자 시스템마다 다른 다양한 조건에 따라 입자를 생성/파괴/업데이트합니다. 이후 생존한 입자들은 버텍스 버퍼로 스트림 아웃됩니다.

입자 시스템을 그리는 데 사용되는 렌더링 기법입니다. 이 렌더링 패스에서는 스트림 아웃된 생존 입자들을 그립니다.

20.10 연습 문제

1. 임미터가 무작위 방향으로 n 개의 셀 입자를 방출하는 폭발 입자 시스템을 구현하세요. 잠시 후 각 셀 입자는 m 개의 입자로 폭발해야 합니다. 각 셀이 정확히 동시에 폭발할 필요는 없습니다. 다양성을 더하기 위해 서로 다른 카운트다운 시간을 부여할 수 있습니다. 만족스러운 결과를 얻을 때까지 다양한 n 과 m 을 실험해 보세요. 임미터가 셀 입자를 생성한 후에는 더 이상 셀이 방출되지 않도록 임미터를 파괴해야 합니다.
2. 분수 입자 시스템을 구현하십시오. 입자들은 소스 지점에서 발생하여 원뿔 형태로 대기 중으로 무작위 방향으로 방출되어야 합니다.

공중으로 발산해야 합니다. 결국 중력에 의해 다시 지면으로 떨어지게 합니다. 참고: 초기 중력을 극복할 수 있을 만큼 충분히 큰 초기 속도 크기를 입자에 부여하십시오.

제21장 그림자 매핑

그림자는 관찰자에게 빛의 발원지를 알려주고 장면 내 물체들의 상대적 위치를 전달하는 데 도움을 줍니다. 본 장에서는 게임 및 3D 애플리케이션에서 동적 그림자를 모델링하는 데 널리 사용되는 기본 그림자 매핑 알고리즘을 소개합니다. 입문서인 만큼 기본 색도 매핑 알고리즘에만 초점을 맞춥니다. 더 정교한 그림자 기법(예: 더 나은 품질의 결과를 제공하는 캐스케이딩 색도 맵[Engel06])은 기본 색도 매핑 알고리즘을 확장하여 구축됩니다.

목표:

- 1. 기본 색도 매핑 알고리즘을 알아보기.
- 2. 투영 텍스처링의 작동 방식을 학습하기 위해.
- 3. 직교 투영법에 대해 알아보기.
- 4. 색도 맵의 에일리어싱 문제와 이를 해결하기 위한 일반적인 전략을 이해하기 위한.

21.1 씬 깊이 렌더링

색도 매핑 알고리즘은 광원의 시점에서 장면 깊이를 렌더링하는 것에 의존합니다. 이는 본질적으로 §12.7.2에서 처음 설명된 렌더-투-텍스처의 변형입니다. "장면 깊이 렌더링"이란 광원의 시점에서 깊이 버퍼를 구축하는 것을 의미합니다. 따라서 광원의 시점에서 장면을 렌더링한 후에는 광원에 가장 가까운 픽셀 프래그먼트를 알 수 있습니다. 이러한 프래그먼트는 그림자에 있을 수 없습니다. 이 섹션에서는 광원의 관점에서 장면 깊이를 저장하는 데 도움이 되는 ShadowMap이라는 유틸리티 클래스를 살펴봅니다. 이 클래스는 단순히 깊이/스텐실 버퍼, 필요한 뷰, 뷰포트를 캡슐화합니다. 색도우 매핑에 사용되는 깊이/스텐실 버퍼를 *색도우 맵*이라고 합니다.

```
class ShadowMap
{
public:
    ShadowMap(ID3D11Device* device, UINT width, UINT height);
    ~ShadowMap();

    ID3D11ShaderResourceView* DepthMapSRV();

    void BindDsvAndSetNullRenderTarget(ID3D11DeviceContext* dc); private:
    ShadowMap(const ShadowMap& rhs); ShadowMap& operator=(const
    ShadowMap& rhs);

private:
    UINT mWidth; UINT
    mHeight;

    ID3D11ShaderResourceView* mDepthMapSRV; ID3D11DepthStencilView*
    mDepthMapDSV;

    D3D11_VIEWPORT mViewport;
};
```

생성자는 지정된 크기, 뷰, 뷰포트를 가진 텍스처를 생성합니다. 색도우 맵의 해상도는 그림자의 품질에 영향을 미치지만, 동시에 높은 해상도의 색도우 맵은 렌더링 비용이 더 많이 들고 더 많은 메모리를 필요로 합니다.

```
ShadowMap::ShadowMap(ID3D11Device* device, UINT width, UINT height)
: mWidth(width), mHeight(height), mDepthMapSRV(0), mDepthMapDSV(0)
```

```

{
    // 새도우 맵 크기와 일치하는 뷰포트. mViewport.TopLeftX = 0.0f;

    mViewport.TopLeftY = 0.0f;
    mViewport.Width          = static_cast<float>(width);
    mViewport.Height = static_cast<float>(height); mViewport.MinDepth =
    0.0f; mViewport.MaxDepth = 1.0f;

    // DSV가 비트를

    // SRV는 DXGI_FORMAT_R24_UNORM_X8_TYPELESS로 해석할 것이므로

    // DXGI_FORMAT_R24_UNORM_X8_TYPELESS로 해석할 예정이기 때문입니다. D3D11_TEXTURE2D_DESC texDesc;
    texDesc.Width = mWidth; texDesc.Height =
    mHeight; texDesc.MipLevels = 1;

    texDesc.ArraySize = 1;
    texDesc.Format      = DXGI_FORMAT_R24G8_TYPELESS;
    texDesc.SampleDesc.Count    = 1;

    텍스처 설명.샘플 설명.품질 = 0;

    texDesc.Usage
                = D3D11_USAGE_DEFAULT; texDesc.BindFlags
                = D3D11_BIND_DEPTH_STENCIL |

                D3D11_BIND_SHADER_RESOURCE;

    texDesc.CPUAccessFlags = 0;
    texDesc.MiscFlags      = 0;

    ID3D11Texture2D* depthMap = 0;
    HR(device->CreateTexture2D(&texDesc, 0, &depthMap));

    D3D11_DEPTH_STENCIL_VIEW_DESC dsvDesc;
    dsvDesc.Flags = 0;
    dsvDesc.Format = DXGI_FORMAT_D24_UNORM_S8_UINT; dsvDesc.ViewDimension = D3D11_DSV_DIMENSION_TEXTURE2D;
    dsvDesc.Texture2D.MipSlice = 0;
    HR(device->CreateDepthStencilView(depthMap, &dsvDesc, &mDepthMapDSV));

    D3D11_SHADER_RESOURCE_VIEW_DESC srvDesc;
    srvDesc.Format = DXGI_FORMAT_R24_UNORM_X8_TYPELESS; srvDesc.ViewDimension =
    D3D11_SRV_DIMENSION_TEXTURE2D; srvDesc.Texture2D.MipLevels = texDesc.MipLevels;
    srvDesc.Texture2D.MostDetailedMip = 0;
    HR(device->CreateShaderResourceView(depthMap, &srvDesc, &mDepthMapSRV));

    // 뷰는 텍스처에 대한 참조를 저장하므로 참조를 해제할 수 있습니다.

    // 참조를 해제할 수 있습니다.
    ReleaseCOM(depthMap);
}

```

나중에 살펴보겠지만, 새도 매핑 알고리즘은 두 번의 렌더 패스가 필요합니다. 첫 번째 패스에서는 빛의 시점에서 장면의 깊이를 새도 맵에 렌더링합니다. 두 번째 패스에서는 "플레이어" 카메라의 시점에서 장면을 백 버퍼에 정상적으로 렌더링하지만, 새도 매핑 알고리즘을 구현하기 위해 새도 맵을 셰이더 입력으로 사용합니다. 새도 맵에 대한 셰이더 리소스 뷰에 접근하는 메서드를 제공합니다.

```

ID3D11ShaderResourceView* ShadowMap::DepthMapSRV()
{
    return mDepthMapSRV;
}

```

마지막으로, 다음 메서드는 새도우 맵으로 렌더링하기 위한 OM 스테이지를 준비합니다. 렌더 타깃을 null로 설정하여 색상 쓰기를 비활성화하는 점에 유의하십시오. 이는 새도우 맵으로 장면을 렌더링할 때, 우리가 관심 있는 것은 장면의 깊이 값이 광원에 대한 상대적 위치뿐이기 때문입니다. 그래픽 카드는 깊이만 그리는데 최적화되어 있으며, 깊이 전용 렌더 패스는 색상과 깊이를 함께 그리는 것보다 훨씬 빠릅니다.

```
void ShadowMap::BindDsvAndSetNullRenderTarget(ID3D11DeviceContext* dc)
{
    dc->RSSetViewports(1, &mViewport);

    // 깊이 버퍼에만 그릴 것이므로 렌더 타겟을 null로 설정합니다.
    // 깊이 버퍼에만 렌더링할 예정입니다. 렌더 타겟을 null로 설정하면
    // 컬러 쓰기를 비활성화합니다.
    ID3D11RenderTargetView* renderTargets[1] = {0};
    dc->OMSetRenderTargets(1, renderTargets, mDepthMapDSV);

    dc->ClearDepthStencilView(mDepthMapDSV, D3D11_CLEAR_DEPTH, 1.0f, 0);
}
```

새도 맵에 대한 렌더링이 완료된 후, 호출 애플리케이션은 백 버퍼를 렌더 타겟으로 복원하고, 원래의 깊이/스텐실 버퍼 및 원래 뷰포트를 복원할 책임이 있습니다.

여러 렌더 타겟을 처리하기 위한 더 나은 설계는 스택을 구현하여 렌더 타겟을 푸시(push)하고 팝(pop)할 수 있도록 하는 것입니다. 팝하면 이전 렌더 타겟, 깊이/스텐실 버퍼 및 뷰포트가 자동으로 복원됩니다.

뷰포트를 자동으로 복원합니다.

21.2 직교 투영

지금까지 이 책에서는 원근 투영법을 사용해 왔습니다. 원근 투영법의 핵심 특성은 물체가 관찰자로부터 멀어질수록 작아져 보인다는 점입니다. 이는 우리가 현실에서 사물을 인지하는 방식과 일치합니다. 또 다른 투영법은 직교 투영법입니다. 이러한 투영법은 주로 3D 과학 또는 공학 응용 분야에서 사용되며, 투영 후에도 평행선이 평행하게 유지되기를 원하는 경우에 적합합니다. 그러나 직교 투영법은 평행광이 생성하는 그림자를 모델링하는 데 활용될 수 있습니다. 직교 투영에서 시야 볼륨은 시공간 축과 정렬된 상자 형태로, 너비 w , 높이 h , 근거리 평면 n , 원거리 평면 f 를 가지며 시공간의 양의 z 축을 따라 아래를 향합니다(그림 21.1 참조). 시공간 좌표계에 상대적으로 정의된 이 수치들은 상자형 시야 볼륨을 규정합니다.

정사영법에서는 투영선이 시공간 z 축과 평행합니다(그림 21.2). 그리고 2차원 점 (x, y, z) 의 투영은 단순히 (x, y) 임을 알 수 있습니다.

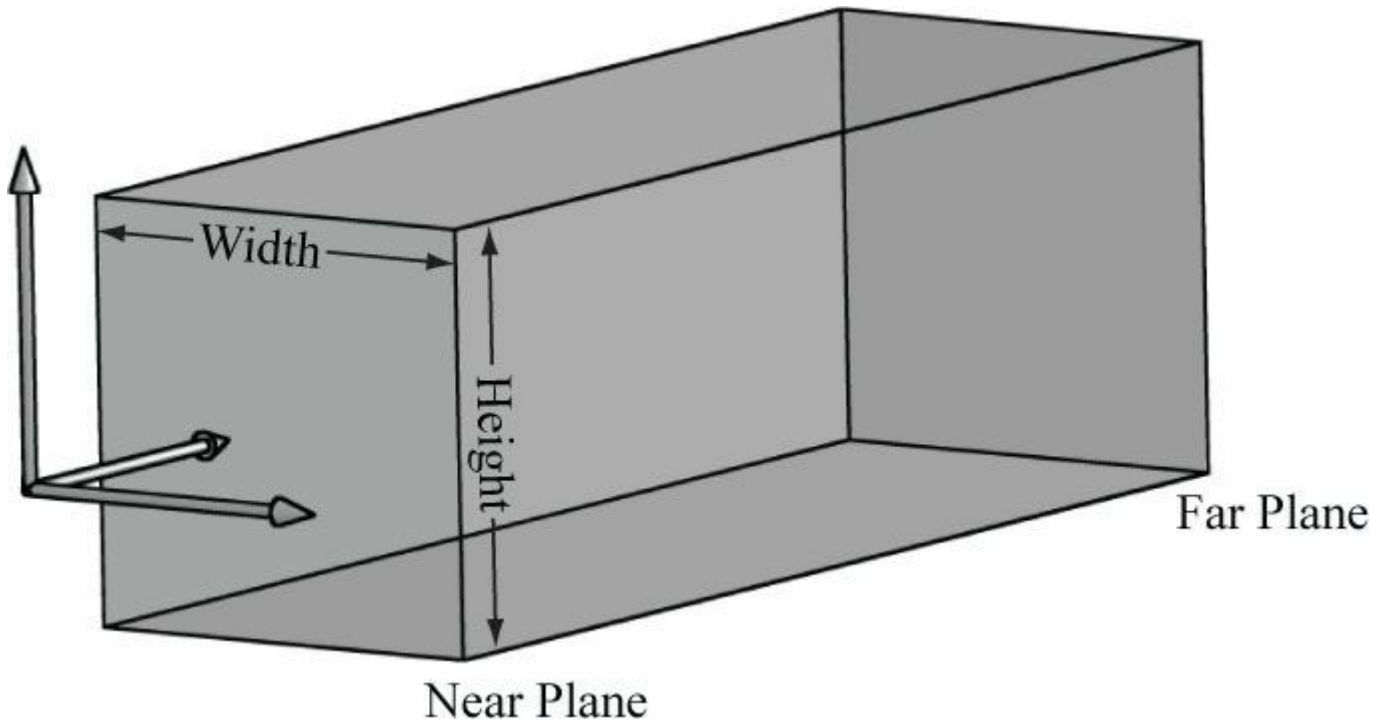


그림 21.1. 직교 투영 시야 공간은 시야 좌표계에 축 정렬된 상자 형태이다.

원근 투영과 마찬가지로 상대적 깊이 정보를 유지하고 정규화된 장치 좌표(NDC)를 원합니다. 뷰 볼륨을 뷰 공간에서 NDC 공간으로 변환하려면, 뷰 공간의 뷰 볼륨을 NDC 공간의 뷰 볼륨에 매핑하기 위해 재스케일링과 시프트가 필요합니다. 이 매핑은

$$\left[-\frac{w}{2}, \frac{w}{2}\right] \times \left[-\frac{h}{2}, \frac{h}{2}\right] \times [n, f] \quad \rightarrow \quad [-1, 1] \times [-1, 1] \times [0, 1]$$

좌표별로 작업하여 결정할 수 있습니다. 처음 두 좌표의 경우, 구간이 단순히 스케일링 계수만 다를 뿐이라는 점을 쉽게 알 수 있습니다:

$$\frac{2}{w} \cdot \left[-\frac{w}{2}, \frac{w}{2} \right] = [-1, 1]$$

$$\frac{2}{h} \cdot \left[-\frac{h}{2}, \frac{h}{2} \right] = [-1, 1]$$

세 번째 좌표에 대해 $[n, f] \rightarrow [0, 1]$ 로 매핑해야 합니다. 매핑이 $g(z) = az + b$ 형태(즉, 크기 조정과 평행 이동)를 취한다고 가정합니다. $g(n) = 0$ 및 $g(f) = 1$ 이라는 조건이 주어지므로 a 와 b 를 구할 수 있습니다:

$$an + b = 0$$

$$af + b = 1$$

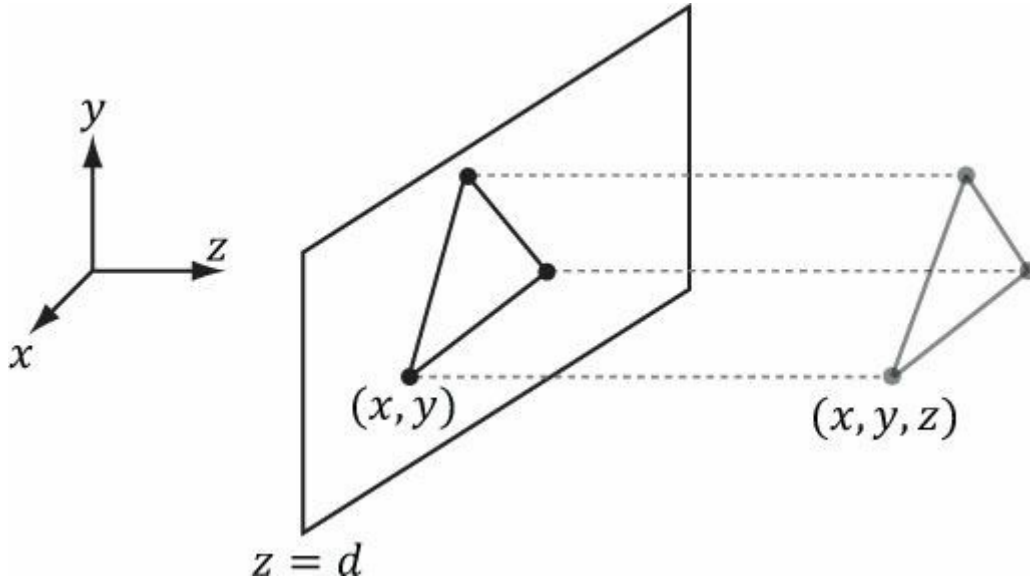


그림 21.2. 투영 평면 상의 점에 대한 직교 투영. 직교 투영에서는 투영선이 시점 공간 z 축과 평행하다.

첫 번째 방정식은 $b = -an$ 을 의미한다. 이를 두 번째 방정식에 대입하면:

$$af - an = 1$$

$$a = \frac{1}{f - n}$$

따라서:

$$-\frac{n}{f - n} = b$$

따라서,

$$g(z) = \frac{z}{f - n} - \frac{n}{f - n}$$

독자는 $f > n$ 을 만족하는 다양한 n 과 f 에 대해 정의역 $[n, f]$ 위에서 함수 $g(z)$ 를 그래프로 그리기를 원할 수 있다.

마지막으로, 시점 공간 좌표 (x, y, z) 에서 NDC 공간 좌표 (x', y', z) 로의 직교 변환은 다음과 같습니다:

$$x' = \frac{2}{w} x$$

$$y' = \frac{2}{h} y$$

$$z' = \frac{z}{f-n} - \frac{n}{f-n}$$

또는 행렬로 표현하면:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \begin{bmatrix} \frac{2}{w} & 0 & 0 & 0 \\ 0 & \frac{2}{h} & 0 & 0 \\ 0 & 0 & \frac{1}{f-n} & 0 \\ 0 & 0 & \frac{n}{n-f} & 1 \end{bmatrix}$$

위 방정식의 4×4 행렬은 *직교 투영 행렬*입니다.

기억하시다시피, 투시 투영 변환에서는 이를 두 부분으로 나누어야 했습니다: 투영 행렬로 설명되는 선형 부분과 w 로 나누기로 설명되는 비선형 부분입니다. 반면 정사 투영 변환은 완전히 선형적입니다— w 로 나누는 과정이 없습니다. 정사 투영 행렬을 곱하면 바로 NDC 좌표계로 이동합니다.

21.3 투영 텍스처 좌표

투영 텍스처링은 슬라이드 프로젝터처럼 텍스처를 임의의 기하학적 형상에 투영할 수 있게 해주기 때문에 그렇게 불립니다. [그림 21.3](#)은 투영 텍스처링의 예를 보여줍니다.

[그림 21.3](#)은 투영 텍스처링의 예를 보여줍니다.

투영 텍스처링은 슬라이드 프로젝터 조명을 모델링하는 데 단독으로도 유용할 수 있지만, §21.4에서 살펴볼게 될 것처럼 색도 매핑의 중간 단계로도 사용됩니다. 중간 단계로도 사용됩니다.

투영 텍스처링의 핵심은 적용된 텍스처가

기하학 구조에 투영된 것처럼 보이도록 텍스처 좌표를 생성하는 데 있습니다. 이렇게 생성된 텍스처 좌표를 *투영 텍스처 좌표*라고 부릅니다.

[그림 21.4](#)에서 볼 수 있듯이, 텍스처 좌표 (u, v) 는 3차원 점 $\mathbf{p}$ 에 투영되어야 할 텍셀을 식별합니다. 그러나

좌표 (u, v) 는 정확히 투영 창 상의 텍스처 공간 좌표계에 상대적인 투영 창 상의 $\mathbf{p}$ 의 투영을 식별합니다. 따라서 투영 텍스처 좌표를 생성하는 전략은 다음과 같습니다:

1. 점 $\mathbf{p}$ 를 빛의 투영 창에 투영하고 좌표를 NDC 공간으로 변환합니다.
2. 투영된 좌표를 NDC 공간에서 텍스처 공간으로 변환하여, 이를 효과적으로 텍스처 좌표로 전환합니다.

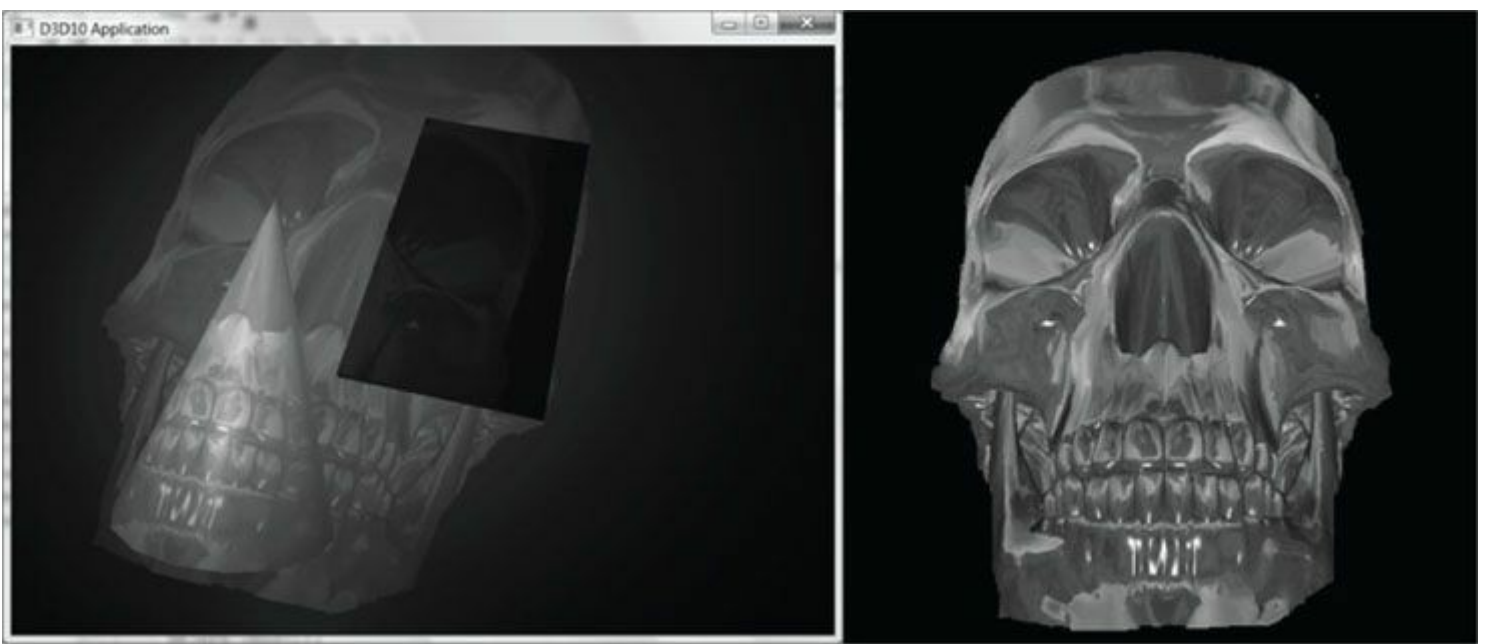


그림 21.3. 두개골 텍스처(오른쪽)가 장면ジオ메트리(왼쪽)에 투영됩니다.

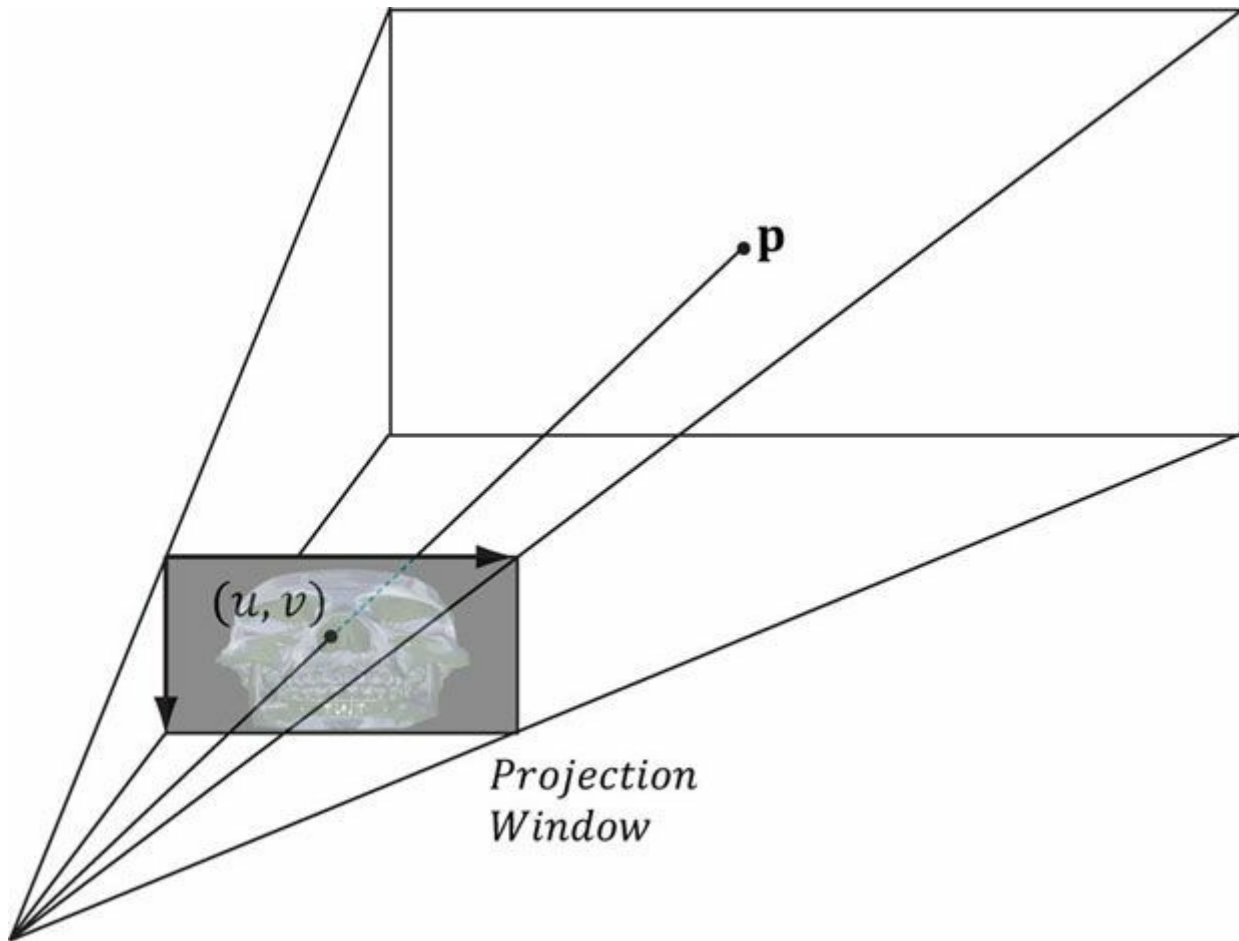


그림 21.4. 투영 창에서 텍스처 공간에 대한 좌표 (u, v) 로 식별된 텍셀은 광원 원점에서 점 p 까지의 시선 방향을 따라 점 p 에 투영됩니다.

단계 1은 광 프로젝터를 카메라로 간주하여 구현할 수 있습니다. 광 프로젝터에 대한 뷰 행렬 V 와 투영 행렬 P 를 정의합니다. 이 두 행렬은 함께 광 프로젝터의 위치, 방향 및 프러스텀을 세계 공간에서 정의합니다. 행렬 V 는 좌표를 세계 공간에서 광 프로젝터의 좌표계로 변환합니다. 좌표가 광 좌표계에 상대적이 되면, 투영 행렬과 동차 분할을 함께 사용하여 정점을 §5.6.3.5에서 언급했듯이, 동차 분할 후 좌표는 NDC 공간에 위치합니다.

단계 2는 다음 좌표 변환을 통해 NDC 공간에서 텍스처 공간으로 변환함으로써 수행됩니다:

$$u = 0.5x + 0.5$$

$$v = -0.5y + 0.5$$

여기서 $u, v \in [0, 1]$ 이며 $x, y \in [-1, 1]$ 을 만족합니다. NDC 좌표계에서 양의 y 축이 텍스처 좌표계의 양의 v 축과 반대 방향으로 향하기 때문에 축을 반전시키기 위해 y 좌표를 음수로 스케일링합니다. 텍스처 공간 변환은 행렬로 표현할 수 있습니다(3장의 연습문제 21을 참고하십시오).

행렬로 표현할 수 있습니다(제3장 연습문제 21을 참고하십시오):

$$\begin{bmatrix} x & y & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & -0.5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0.5 & 0.5 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u & v & 0 & 1 \end{bmatrix}$$

이전 행렬을 NDC 공간에서 텍스처 공간으로 변환하는 "텍스처 행렬" T라고 부르자. 이를 통해 월드 공간에서 직접 텍스처 공간으로 이동하는 복합 변환 VPT를 구성할 수 있다. 이 변환을 곱한 후에도 변환을 완료하려면 원근 분할을 수행해야 한다. 텍스처 변환 후 원근 분할을 수행할 수 있는 이유는 5장 연습문제 8을 참조하라.

21.3.1 코드 구현

투영 텍스처 좌표를 생성하는 코드는 다음과 같습니다:

```
struct VertexOut
{
    float4 PosH : SV_POSITION; float3 PosW :
    POSITION; float3 TangentW : TANGENT;
    float3 NormalW : NORMAL; float2 Tex :
    TEXCOORD0; float4 ProjTex : TEXCOORD1;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout; [...]

    // 빛의 투영 공간으로 변환.
    vout.ProjTex = mul(float4(vIn.posL, 1.0f),
        gLightWorldViewProjTexture);

    [...]
    return vout;
}

float4 PS(VertexOut pin) : SV_Target
{
    // w로 나눈 값을 사용하여 투영 완료 pin.ProjTex.xyz /= pin.ProjTex.w;

    // NDC 공간에서의 깊이. float depth =
    pin.ProjTex.z;

    // 투영 텍스처 좌표를 사용하여 텍스처를 샘플링합니다. float4 c =
    gTextureMap.Sample(sampler, pin.ProjTex.xy);

    [...]
}
```

21.3.2 프러스텀 외부 점

렌더링 파이프라인에서, 프러스텀(frustum) 외부의 지오메트리는 클리핑됩니다. 그러나 빛 프로젝터의 시점에서 지오메트리를 투영하여 투영 텍스처 좌표를 생성할 때는 클리핑이 수행되지 않습니다—단순히 정점을 투영할 뿐입니다. 결과적으로,

프로젝터의 프러스텀 외부 지오메트리는 $[0, 1]$ 범위를 벗어난 투영 텍스처 좌표를 받게 됩니다. $[0, 1]$ 범위를 벗어난 투영 텍스처 좌표는 텍스처 샘플링 시 사용되는 활성화된 주소 모드(§8.8 참조)에 기반하여 $[0, 1]$ 범위를 벗어난 일반 텍스처 좌표와 동일하게 작동합니다.

일반적으로 프로젝트의 프러스텀 외부 지오메트리에 텍스처를 적용하는 것은 의미가 없으므로 원하지 않습니다(예: 기하학적 영역은 프로젝트로부터 빛을 받지 않습니다). 경계색 주소 모드에 제로 색상을 사용하는 것이 일반적인 해결책입니다. 다른 전략은 프로젝트에 스포트라이트(§7.10 참조)를 연결하여 스포트라이트의 시야 원뿔 영역 밖에 있는 모든 것이 조명되지 않도록 하는 것입니다(즉, 표면이 투사된 빛을 받지 않음). 스포트라이트 사용의 장점은 프로젝트의 광도가 스포트라이트 원뿔 중심에서 가장 강하며, $-L$ 과 d 사이의 각도 ϕ 가 증가함에 따라(여기서 L 은 표면 점까지의 광 벡터이고 d 는 스포트라이트 방향) 부드럽게 감쇠된다는 점입니다.

21.3.3 정사영 투영

지금까지 투영 텍스처링을 설명할 때 원근 투영(프러스텀 형태의 볼륨)을 사용했습니다. 그러나 투영 과정에 원근 투영 대신 정사투영을 사용할 수도 있습니다. 이 경우 텍스처는 빛의 z 축 방향을 따라 상자 형태의 공간을 통과하여 투영됩니다.

투영 텍스처 좌표에 대해 논의한 모든 내용은 정사영 투영을 사용할 때에도 동일하게 적용됩니다. 단, 몇 가지 사항을 제외하면 모두 적용됩니다. 첫째, 정사영 투영에서는 투영기 볼륨 외부 점을 처리하는 스포트라이트 전략이 작동하지 않습니다. 이는 스포트라이트 원뿔이 어느 정도 원뿔형 볼륨을 근사화하지만, 상자 형태를 근사화하지 않기 때문입니다. 그러나 텍스처 주소 모드를 사용해 프로젝트 볼륨 외부 점을 처리하는 것은 여전히 가능합니다. 정사영 투영도 NDC 좌표를 생성하며, NDC 공간 내 점 (x, y, z) 이 볼륨 내에 존재하는 조건은 다음과 같기 때문입니다:

$$-1 \leq x \leq 1$$

$$-1 \leq y \leq 1$$

$$0 \leq z \leq 1$$

둘째, 직교 투영에서는 w 로 나누는 연산이 필요하지 않습니다. 즉, 다음 줄이 필요하지 않습니다:

```
// w로 나누어 투영을 완료합니다. pin.ProjTex.xyz /= pin.ProjTex.w;
```

이는 직교 투영 후 좌표가 이미 NDC 공간에 있기 때문입니다. 이는 원근 투영에 필요한 픽셀별 나눗셈을 피하므로 더 빠릅니다. 반면, 나눗셈을 그대로 두어도 문제가 되지 않습니다. 왜냐하면 1로 나누기 때문이죠(직교 투영은 w 좌표를 변경하지 않으므로 w 는 1이 됩니다). 셰이더 코드에서 w 로 나누는 연산을 그대로 두면, 셰이더 코드가 원근 투영과 직교 투영 모두에 일관되게 적용됩니다. 다만, 이러한 일관성의 대가로 직교 투영 시 불필요한 나눗셈 연산을 수행하게 됩니다.

21.4 새도 매핑

21.4.1 알고리즘 설명

새도 매핑 알고리즘의 개념은 빛의 시점에서 장면의 깊이를 *세도우 맵이라* 불리는 깊이 버퍼에 렌더링-투-텍스처하는 것입니다. 이 작업이 완료되면 세도우 맵에는 빛의 관점에서 보이는 모든 픽셀의 깊이 값이 포함됩니다. (다른 픽셀에 가려진 픽셀은 깊이 테스트에 실패하여 덮어쓰기되거나 아예 기록되지 않으므로 세도우 맵에 포함되지 않습니다.)

빛의 시점에서 장면을 렌더링하려면, 좌표를 월드 좌표계에서 변환하는 빛 뷰 매트릭스를 정의해야 합니다. 빛의 공간과 빛 투영 행렬을 공간에 매핑합니다. 이 행렬은 세계 내에서 빛이 방출되는 부피를 설명합니다. 이는 원뿔형 부피(투시 투영) 또는 상자형 부피(정사투영)일 수 있습니다. 원뿔형 빛 부피는 스포트라이트 원뿔을 원뿔형 부피 안에 포함시켜 스포트라이트를 모델링하는 데 사용할 수 있습니다. 박스 광원은 평행광을 모델링하는 데 사용할 수 있습니다. 그러나 평행광은 이제 상자 볼륨으로 제한되어 통과하므로 장면의 일부 영역에만 도달할 수 있습니다(그림 21.5 참조). 장면 전체에 도달하는 광원(예: 태양)의 경우, 장면 전체를 포함할 수 있을 만큼 광 볼륨을 충분히 크게 설정할 수 있습니다.

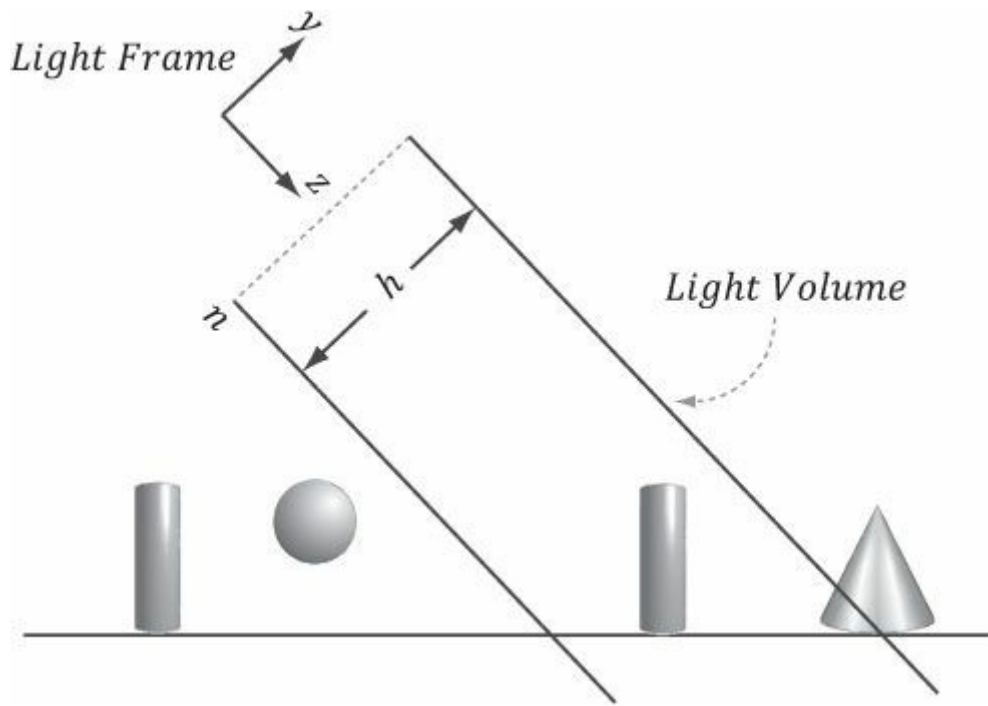


그림 21.5. 평행 광선은 광량체를 통과하므로, 광량체 내부의 장면 일부만 빛을 받습니다. 광원이 장면 전체에 닿아야 한다면, 광량체 크기를 장면 전체를 포함하도록 설정할 수 있습니다.

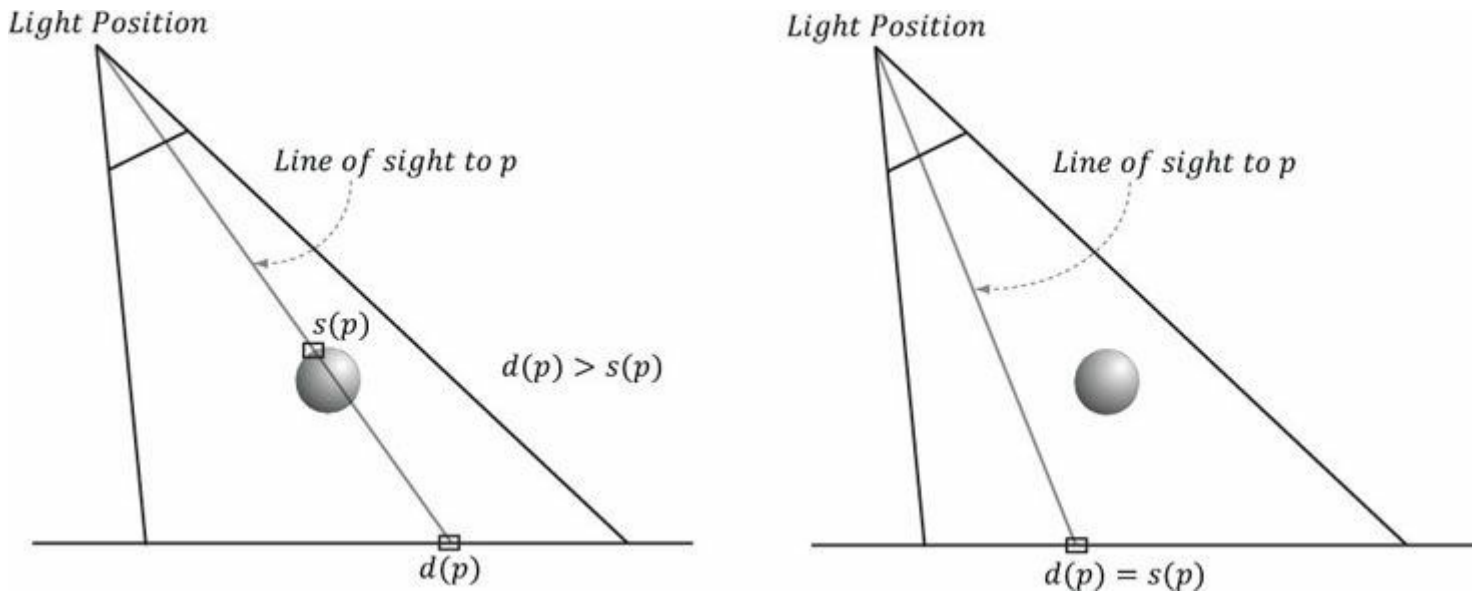


그림 21.6. 왼쪽에서, 픽셀 p 가 빛으로부터의 깊이는 $d(p)$ 이다. 그러나 동일한 시선 방향을 따라 빛에 가장 가까운 픽셀의 깊이는 $s(p)$ 이며, $d(p) > s(p)$ 이다. 따라서 우리는 빛의 관점에서 p 앞에 물체가 존재한다고 결론지을 수 있으며, 따라서 p 는 그림자에 있다. 오른쪽 그림에서, 픽셀 p 의 광원으로부터의 깊이는 $d(p)$ 이며, 동시에 시선 상에서 광원에 가장 가까운 픽셀이기도 하다. 즉, $s(p) = d(p)$ 이므로 p 는 그림자에 있지 않다고 결론지을 수 있다.

새도 맵을 구축한 후에는 "플레이어" 카메라의 시점에서 평소처럼 장면을 렌더링합니다. 렌더링되는 각 픽셀 p 에 대해, 우리는 또한 광원으로부터의 깊이 $d(p)$ 를 계산합니다. 또한 투영 텍스처링을 사용하여, 광원에서 픽셀 p 까지의 시선 방향을 따라 새도 맵을 샘플링하여 새도 맵에 저장된 깊이 값 $s(p)$ 를 얻습니다. 이 값은 빛의 위치에서 p 까지의 시선 상에서 빛에 가장 가까운 픽셀의 깊이입니다. 그런 다음 그림 21.6에서 볼 수 있듯이, 픽셀 p 가 그림자에 있는 것은 $d(p) > s(p)$ 일 때와만 같습니다. 따라서 픽셀이 그림자에 있지 않은 것은 $d(p) \leq s(p)$ 일 때와만 같습니다.

비교되는 깊이 값은 NDC 좌표계에 있습니다. 이는 덤스 버퍼인 새도 맵이 깊이 값을 NDC 좌표계로 저장하기 때문입니다. 정확한 구현 방식은 코드를 살펴보면 명확해질 것입니다.

21.4.2 바이어싱과 에일리어싱

새도 맵은 연관된 광원에 대해 가장 가시적인 픽셀의 깊이를 저장합니다. 그러나 새도 맵은 유한한 해상도만을 가집니다. 따라서 각 새도 맵 텍셀은 장면의 특정 영역에 대응합니다. 즉, 새도 맵은 단순히 광원 관점에서 장면 깊이를 이산적으로 샘플링한 것에 불과합니다. 이로 인해 '새도 여드름(shadow acne)'이라 불리는 에일리어싱 문제가 발생합니다(그림 21.7 참조).

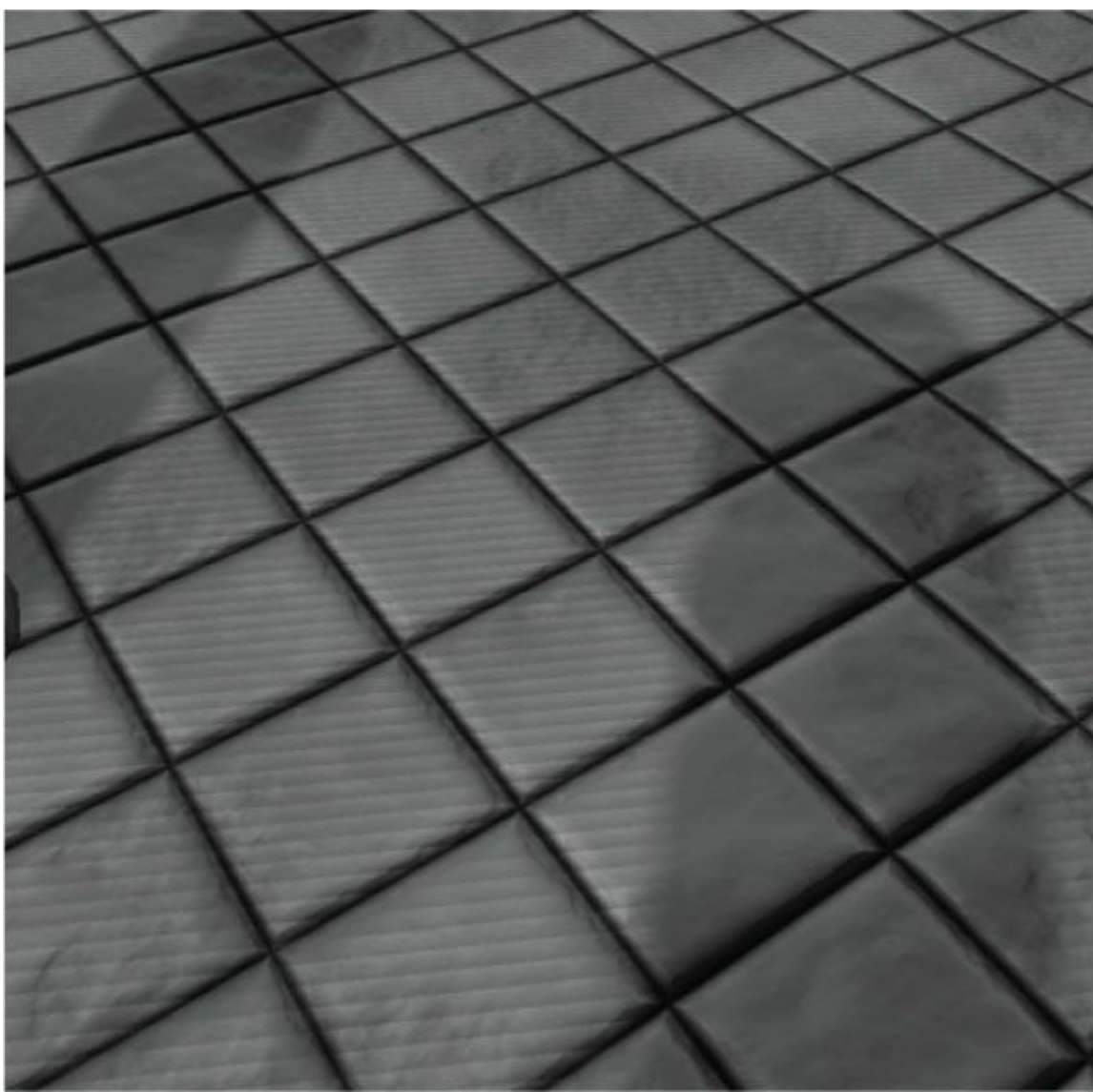


그림 21.7. 바닥 평면에서 빛과 그림자가 계단식 교대로 나타나는 앨리어싱 현상을 확인하십시오. 이 앨리어싱 오류는 흔히 그림자 여드름(shadow acne)이라 불립니다.

그림 21.8은 그림자 여드름이 발생하는 원리를 설명하는 간단한 다이어그램입니다. 간단한 해결책은 그림자 맵 깊이에 일정한 편향을 적용하여 오프셋하는 것입니다. 그림 21.9는 이 방법으로 문제가 수정되는 모습을 보여줍니다.

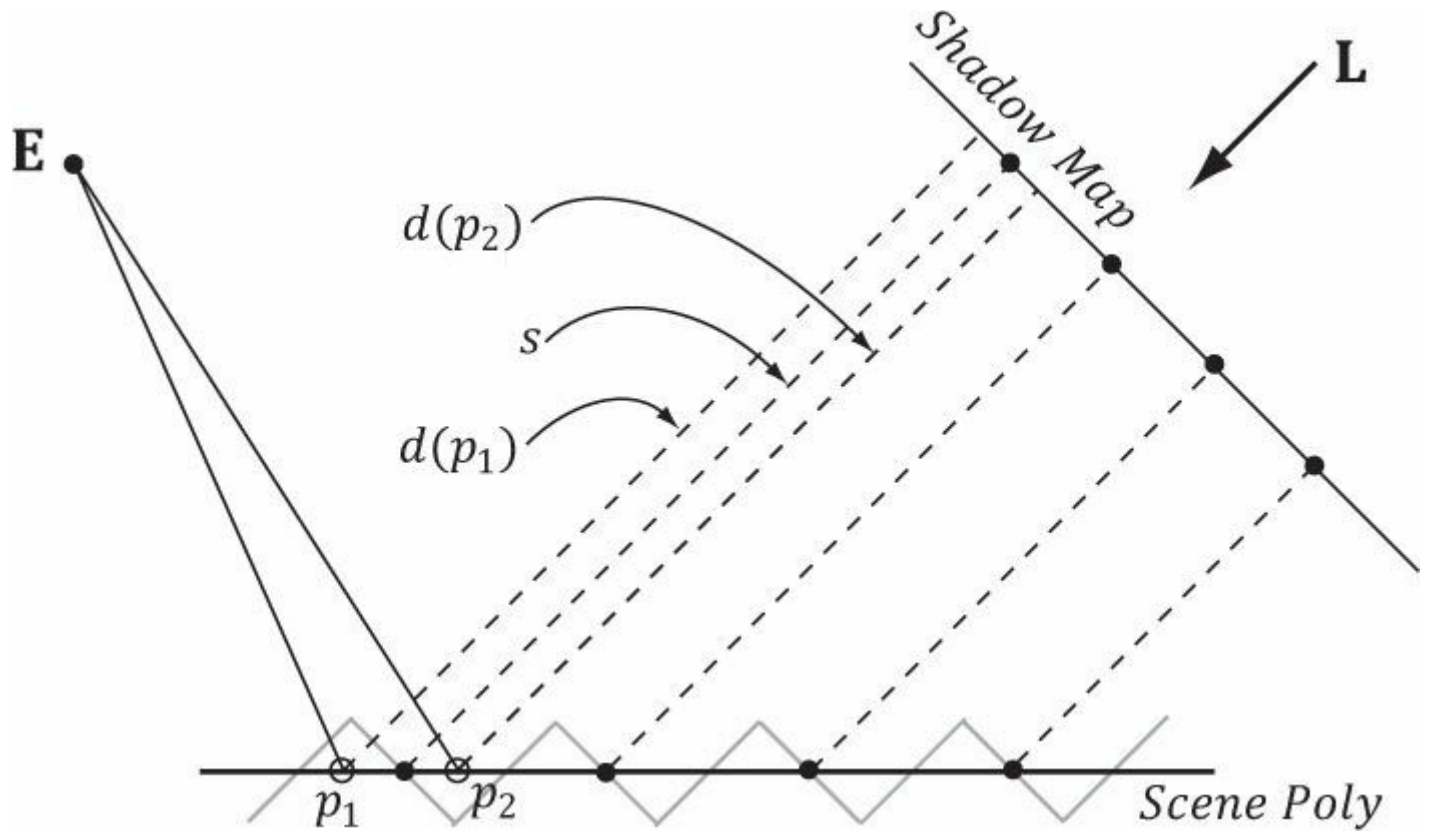


그림 21.8. 섀도 맵은 장면의 깊이를 샘플링합니다. 섀도 맵의 유한한 해상도로 인해 각 섀도 맵 텍셀은 장면의 특정 영역에 대응함을 알 수 있습니다. 관찰자 E 는 서로 다른 화면 픽셀에 대응하는 장면상의 두 점 p_1 과 p_2 를 봅니다. 그러나 광원의 관점에서 두 점 모두 동일한 섀도 맵 텍셀에 의해 덮여 있습니다(즉, $s(p_1) = s(p_2) = s$). 섀도우 맵 테스트를 수행할 때 $d(p_1) > s$ 이고 $d(p_2) \leq s$ 입니다. 따라서 p_1 은 그림자에 있는 것처럼 채색되고, p_2 는 그림자에 있지 않은 것처럼 채색됩니다. 이로 인해 그림자 여드름 현상이 발생합니다.

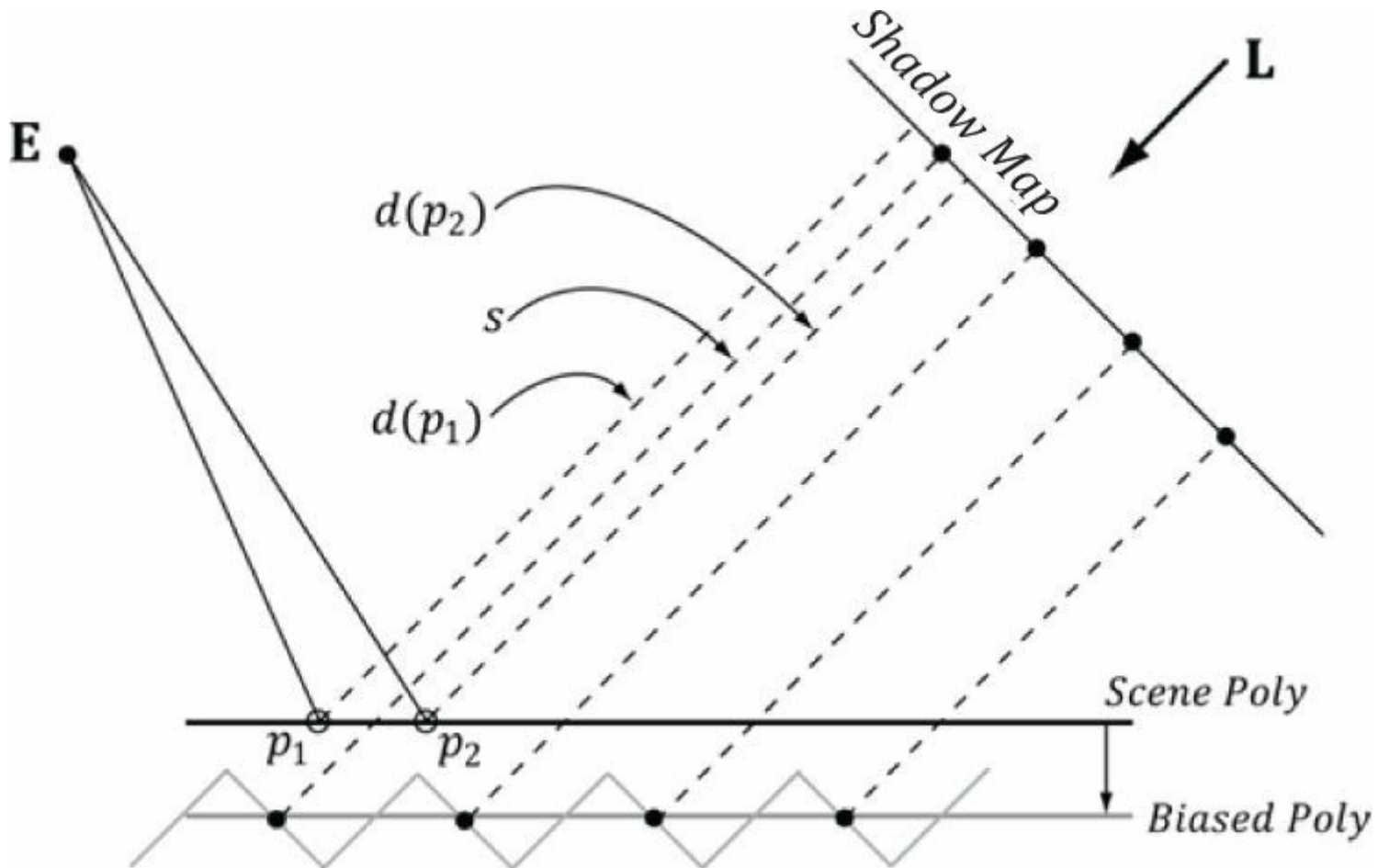


그림 21.9. 섀도 맵의 깊이 값에 바이어스를 적용함으로써 가짜 그림자가 발생하지 않습니다. $d(p_1) \leq s$ 및 $d(p_2) \leq s$ 가 성립합니다. 적절한 깊이 바이어스 값은 일반적으로 실험을 통해 찾습니다.

과도한 바이어싱은 그림자가 물체에서 분리되어 보이는 *피터 팬* 현상이라는 아티팩트를 유발합니다(그림

그림 21.10 참조).



그림 21.10. 피터 팬닝—심도 바이어스가 너무 커서 그림자가 기둥에서 분리되어 보임.

안타깝게도 고정된 편향값이 모든 기하학적 구조에 적용되지는 않습니다. 특히 [그림 21.11](#)은 (광원에 대해) 큰 경사각을 가진 삼각형에는 더 큰 편향값이 필요함을 보여줍니다. 모든 경사각을 처리할 수 있을 만큼 충분히 큰 값이 편향값을 선택하고 싶은 유혹이 있습니다. 그러나 [그림 21.10](#)에서 보았듯이, 이는 피터 팬닝 현상을 유발합니다.

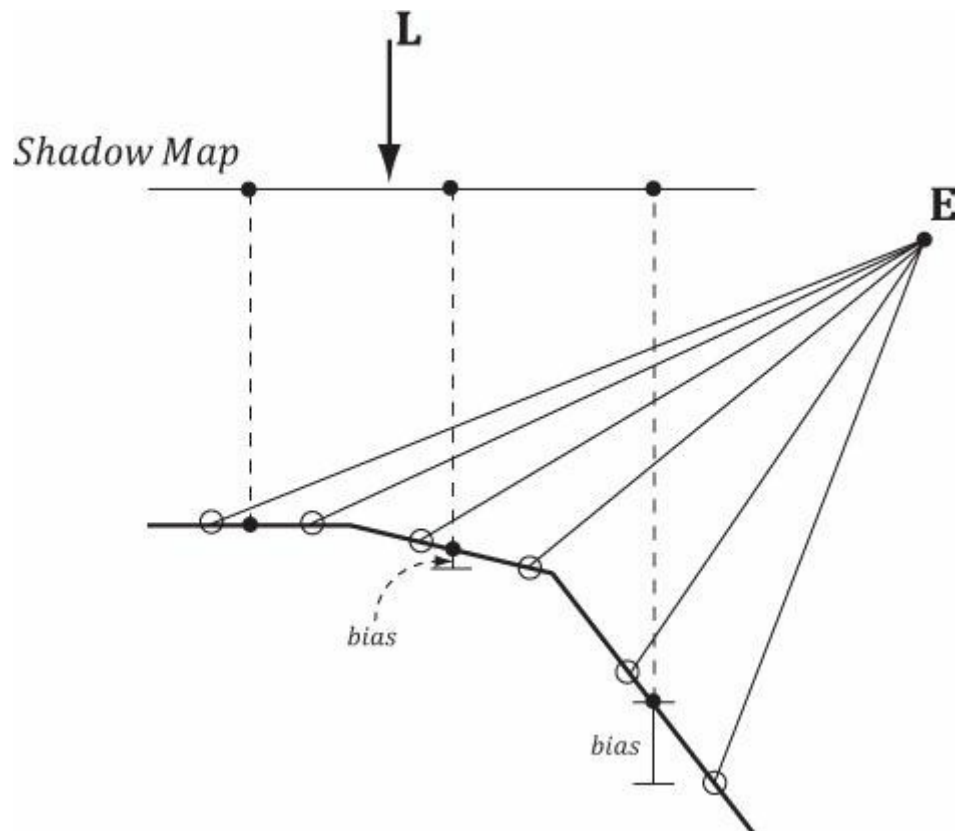


그림 21.11. 광원에 대한 경사가 큰 다각형은 경사가 작은 다각형보다 더 많은 바이어스가 필요합니다.

우리가 원하는 것은 광원에 대한 다각형 경사도를 측정하고 경사가 큰 다각형에 더 많은 바이어스를 적용하는 방법이다. 다행히 그래픽스 하드웨어는 소위 *경사 스케일링 바이어스* 래스터화 상태 속성을 통해 이를 본질적으로 지원한다.

속성을 통해 이를 내재적으로 지원합니다:

```
typedef struct D3D11_RASTERIZER_DESC { [...]  
    INT DepthBias;  
    FLOAT DepthBiasClamp; FLOAT  
    SlopeScaledDepthBias; [...]  
} D3D11_RASTERIZER_DESC;
```

- 1. **DepthBias**: 적용할 고정 바이어스 값입니다. 이 정수 값이 UNORM 깊이 버퍼 형식에서 어떻게 사용되는지는 아래 설명을 참조하십시오.
- 2. **DepthBiasClamp**: 허용되는 최대 깊이 바이어스 값입니다. 이를 통해 깊이 바이어스에 상한을 설정할 수 있습니다. 매우 가파른 경사에서는 경사 스케일링된 바이어스가 지나치게 커져 피터 팬 현상 같은 아티팩트를 유발할 수 있기 때문입니다.
- 3. **SlopeScaledDepthBias**: 다각형 경사도에 따라 바이어스 적용량을 조절하는 스케일 계수입니다. 공식은 후속 주석을 참조하십시오.

참고: *썬을 새도우 맵에 렌더링할 때* 경사 스케일링된 바이어스를 적용합니다. 이는 *광원에 대한* 폴리곤 경사에 따라 바이어스를 적용하기 위함입니다. 결과적으로 새도우 맵 값에 바이어스가 적용됩니다.

데모에서는 다음 값을 사용합니다:

```
RasterizerState Depth  
{  
    // [MSDN 출처]  
    // 출력 병합 단계에 현재 바인딩된 깊이 버퍼가  
    // UNORM 형식일 경우 또는 바인딩된 맵스 버퍼가 없을 경우, 바이어스 값은  
    // 다음과 같이 계산됩니다:  
    //  
    // 편향값 = (float)DepthBias * r + SlopeScaledDepthBias * MaxDepthSlope;  
    //  
    // 여기서 r은  
    //  
    // [/End MSDN]  
    //  
    // 24비트 깊이 버퍼의 경우, r = 1 / 2^24입니다.  
    //  
    // 예시: DepthBias = 100000 ==> 실제 DepthBias = 100000/2^24 = 0.006  
  
    // 이 값들은 장면에 따라 크게 달라지므로,  
    // 최적의 값을 찾기 위해 장면에 맞게 실험해 보셔야 합니다.  
    // 최적값을 찾아야 합니다. DepthBias  
    // = 100000; DepthBiasClamp = 0.0f;  
    // SlopeScaledDepthBias = 1.0f;  
};
```

 깊이 바이어스는 래스터화 단계(클리핑 이후)에서 발생하므로 기하학적 클리핑에는 영향을 미치지 않습니다.

 깊이 바이어스에 대한 자세한 내용은 SDK 문서에서 "Depth Bias"를 검색하면 적용 규칙과 부동 소수점 깊이 버퍼에서의 작동 방식을 모두 확인할 수 있습니다.

21.4.3 PCF 필터링

그림자 맵을 샘플링하는 데 사용되는 투영 텍스처 좌표 (u, v) 는 일반적으로 그림자 맵의 텍셀과 일치하지 않습니다. 보통 네 개의 텍셀 사이에 위치합니다. 컬러 텍스처링에서는 이 문제가 이차원 보간법(§8.4.1)으로 해결됩니다. 그러나 [Kilgard01]은 깊이 값을 평균화해서는 안 된다고 지적합니다. 이는 픽셀이 그림자에 표시된 것으로 잘못 판단되는 결과를 초래할 수 있기 때문입니다. (같은 이유로 그림자 맵에 대한 밍 맵도 생성할 수 없습니다.) 깊이 값을 보간하는 대신 결과값을 보간하는데, 이를 *퍼센티지 클로저 필터링(PCF)*이라고 합니다. 즉, 포인트 필터링(MIN_MAG_MIP_POINT)을 사용하고 (u, v) , $(u + \Delta x, v)$, $(u, v + \Delta x)$, $(u + \Delta x, v + \Delta x)$ 좌표로 텍스처를 샘플링합니다. 여기서 $\Delta x = 1/\text{SHADOW_MAP_SIZE}$ 입니다. 포인트 샘플링을 사용하기 때문에, 이 네 점은 (u, v) 를 둘러싼 가장 가까운 네 텍셀 s_0, s_1, s_2, s_3 에 각각 닿게 됩니다.

그림 21.12. 그런 다음 샘플링된 각 깊이에 대해 색도 맵 테스트를 수행하고 색도 맵 결과를 이차 보간합니다.

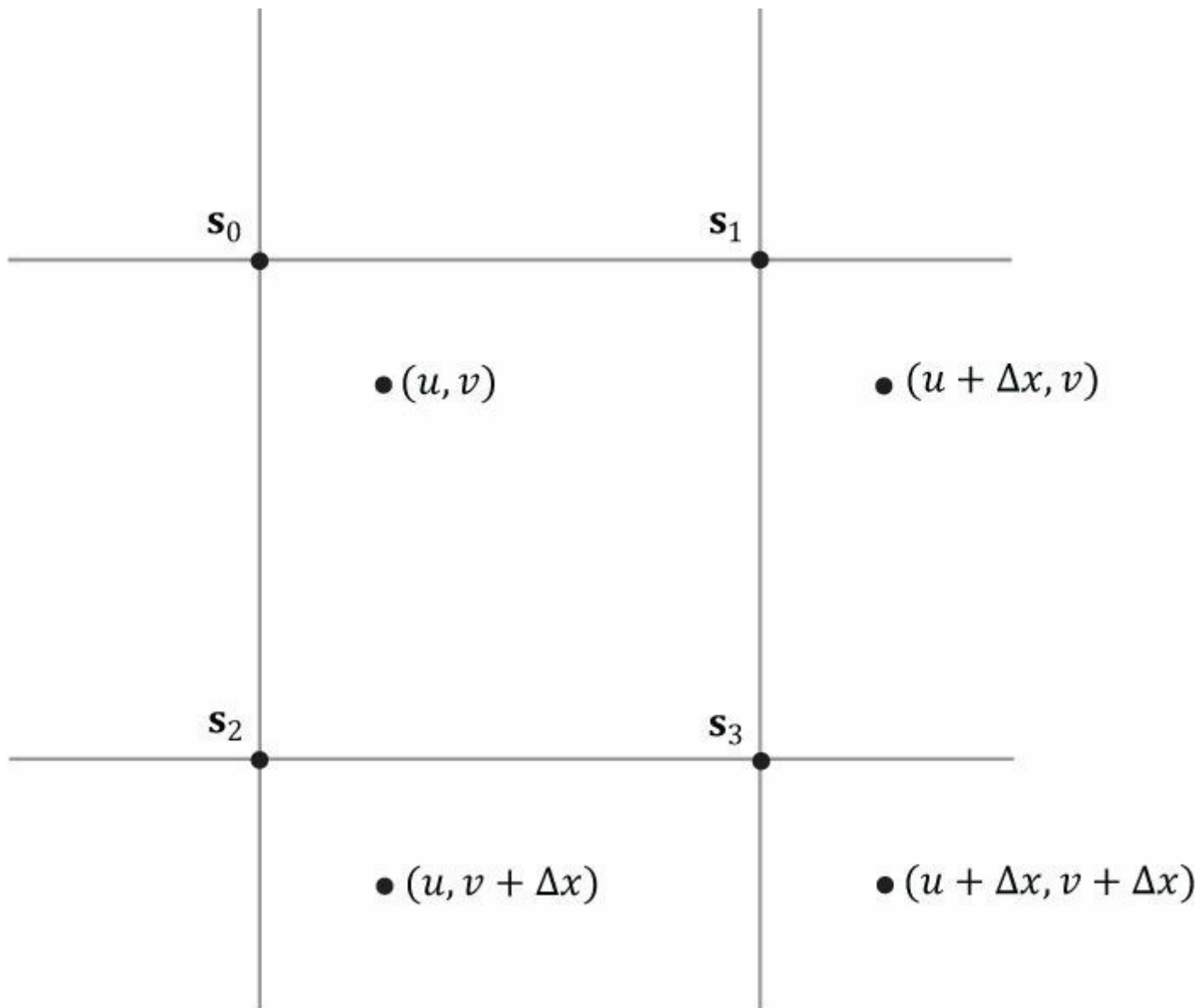


그림 21.12. 네 개의 색도 맵 샘플을 채취합니다.

```
static const float SMAP_SIZE = 2048.0f;
static const float SMAP_DX = 1.0f / SMAP_SIZE;

...

// 빛에 가장 가까운 깊이를 얻기 위해 색도 맵 샘플링. float s0 =
    gShadowMap.Sample(gShadowSam,
        projTexC.xy).r;
float s1 = gShadowMap.Sample(gShadowSam, projTexC.xy +
    float2(SMAP_DX, 0)).r;
float s2 = gShadowMap.Sample(gShadowSam, projTexC.xy + float2(0,
    SMAP_DX)).r;
float s3 = gShadowMap.Sample(gShadowSam, projTexC.xy +
    float2(SMAP_DX, SMAP_DX)).r;

// 픽셀 깊이가 색도 맵 값보다 작은가? float result0 = depth <=
    s0;
float result1 = depth <= s1; float result2 =
    depth <= s2; float result3 = depth <= s3;

// 텍셀 공간으로 변환.
float2 texelPos = SMAP_SIZE*projTexC.xy;
```

```
// 보간량을 결정합니다. float2 t = frac(texelPos);
```

```
// 보간 결과 계산.
```

```
return lerp(lerp(result0, result1, t.x),  
            lerp(result2, result3, t.x), t.y);
```

이렇게 하면 모든 픽셀이 완전히 그림자에 속하거나 완전히 그림자 밖에 속하는 상황이 아닙니다. 예를 들어, 두 개의 샘플이 그림자에 속하고 두 개가 그림자 밖에 속하면 해당 픽셀은 50% 그림자에 속하게 됩니다. 이는 그림자 픽셀에서 그림자 없는 픽셀로의 더 부드러운 전환을 만들어 줍니다(그림 21.13 참조).

HLSL **frac** 함수는 부동 소수점 숫자의 소수 부분(즉, 맨티사)을 반환합니다. 예를 들어, **SMAP_SIZE** = 1024이고 **projTex.xy** = (0.23, 0.68)일 때, **texelPos** = (235.52, 696.32)이며 **frac(texelPos)** = (0.52, 0.32)입니다. 이 분수 값들은 샘플들 사이에서 얼마나 보간해야 하는지를 알려줍니다. HLSL의 **lerp(x, y, s)** 함수는 선형 보간 함수로, $x + s(y - x) = (1 - s)x + sy$ 를 반환합니다.

필터링을 적용해도 그림자는 여전히 매우 딱딱하며, 가까이서 보면 앨리어싱 아티팩트가 여전히 만족스럽지 않을 수 있습니다.

더 공격적인 방법을 사용할 수 있습니다. 예를 들어 [Uralsky05]를 참조하십시오. 또한 고해상도 새도우 맵을 사용하는 것이 도움이 되지만 비용이 너무 많이 들 수 있다는 점도 언급합니다.

앞서 설명한 PCF 필터링의 주요 단점은 텍스처 샘플링이 네 번 필요하다는 점이다. 현대 GPU에서 텍스처 샘플링은 메모리 대역폭과 메모리 지연 시간이 GPU의 순수 연산 성능만큼 크게 개선되지 않았기 때문에 상대적으로 비용이 많이 드는 작업 중 하나이다[Möller08]. 다행히 DirectX 11 그래픽스 하드웨어는 **SampleCmpLevelZero** 메서드를 통해 PCF를 내장 지원한다:

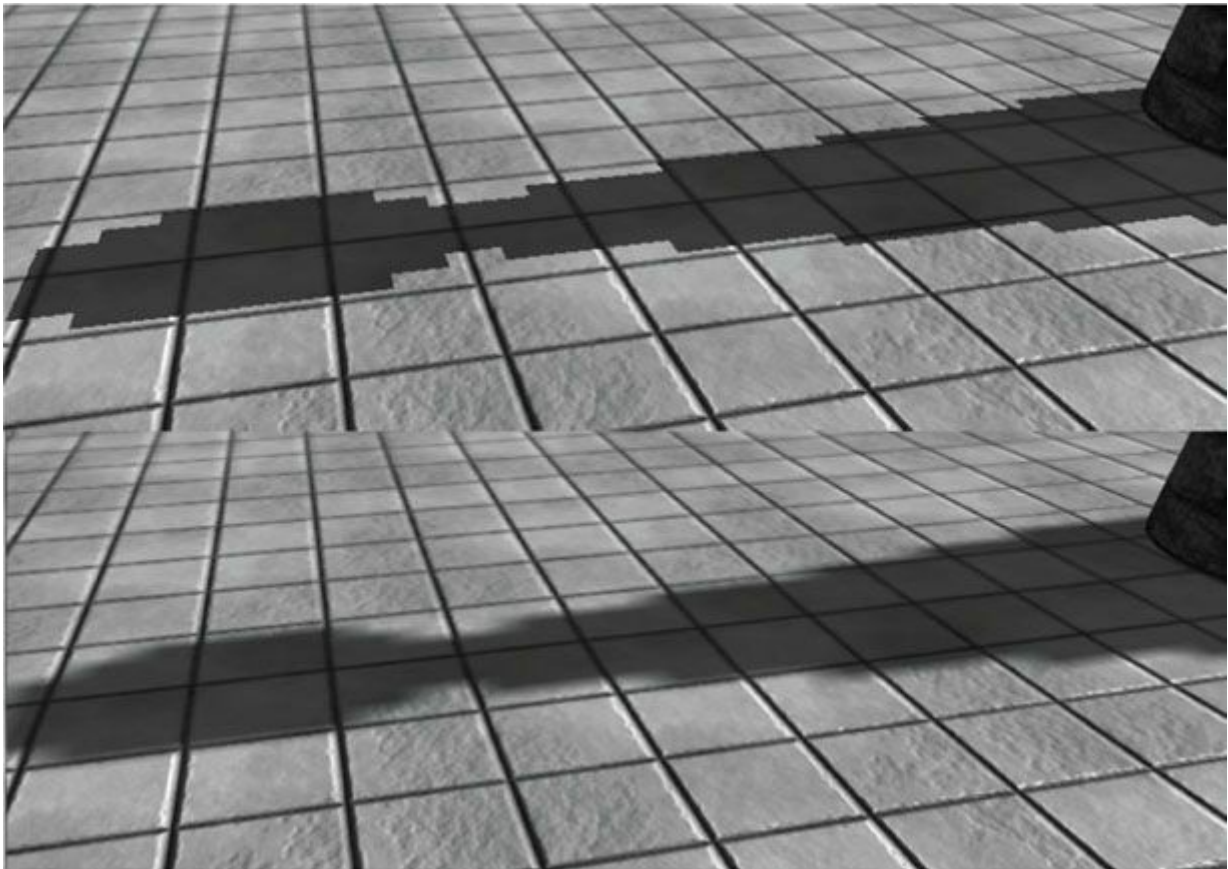


그림 21.13. 상단 이미지에서 그림자 경계에 나타나는 "계단 현상" 아티팩트를 관찰하십시오. 하단 이미지에서는 필터링으로 이러한 앨리어싱 아티팩트가 다소 부드럽게 처리되었습니다.

```
Texture2D gShadowMap; SamplerComparisonState
```

```
samShadow  
{
```

```
    Filter = COMPARISON_MIN_MAG_LINEAR_MIP_POINT;
```

```
    // 라이트 프러스텀 외부 포인트에 대해 0 반환
```

```
    // 그림자에 넣기 위해. AddressU = BORDER;
```

```
AddressV = BORDER; AddressW
= BORDER;
BorderColor = float4(0.0f, 0.0f, 0.0f, 0.0f);

ComparisonFunc = LESS_EQUAL;
};

// 자동으로 4-tap PCF를 수행합니다. 비교 퍼센트Lit =
shadowMap.SampleCmpLevelZero( samShadow, shadowPosH.xy,
depth).r;
```

메서드 이름의 **LevelZero** 부분은 최상위 mip맵 레벨만 확인한다는 의미입니다. 이는 색도우 매핑에 필요한 사항이므로 적합합니다(색도우 맵용 mip맵 체인을 생성하지 않음). 이 메서드는 일반적인 샘플러 객체가 아닌 *비교 샘플러*를 사용합니다. 이는 하드웨어가 결과 필터링 전에 수행해야 하는 색도우 맵 비교 테스트를 처리할 수 있도록 하기 위함입니다. PCF의 경우 **COMPARISON_MIN_MAG_LINEAR_MIP_POINT** 필터를 사용하고 비교 함수를 **LESS_EQUAL**로 설정해야 합니다(깊이 값에 편향을 적용하므로 LESS도 작동합니다). 첫 번째와 두 번째 매개변수는 각각 비교 샘플러와 텍스처 좌표입니다. 세 번째 매개변수는 색도우 맵 샘플과 비교할 값입니다. 따라서 비교 값을 **깊이**로 설정하고 비교 함수를 **LESS_EQUAL**로 설정하면 다음과 같은 비교를 수행하게 됩니다:

```
float result0 = depth <= s0; float result1 =
depth <= s1; float result2 = depth <= s2;
float result3 = depth <= s3;

그런 다음 하드웨어가 결과를 이차선형 보간하여 PCF를 완성합니다.
```

Note

From *SDK SDK 문서에서, 유일한 다음 다음 형식 지원 비교 필터:*

R32_FLOAT_X8X24_TYPELESS, R32_FLOAT, R24_UNORM_X8_TYPELESS, R16_UNORM.

지금까지 이 섹션에서는 4-탭 PCF 커널을 사용했습니다. 더 큰 커널을 사용하면 그림자의 가장자리를 더 크게 만들고 더 부드럽게 만들 수 있지만, 추가적인 **SampleCmpLevelZero** 호출이 필요합니다. 본 데모에서는 3×3 박스 필터 패턴으로 **SampleCmpLevelZero**를 호출합니다. 각 **SampleCmpLevelZero** 호출이 4-tap PCF를 수행하므로, 그림자 맵에서 4×4 고유 샘플 포인트를 사용합니다(패턴에 따라 샘플 포인트 일부가 중첩됨). 큰 필터링 커널 사용은 그림자 여드름 현상이 재발할 수 있습니다. 그 이유와 해결책은 §21.5에서 설명합니다.

한 가지 관찰 사항은 PCF가 실제로는 그림자 가장자리에서만 수행되면 된다는 점입니다. 그림자 내부에서는 블렌딩이 발생하지 않으며, 그림자 영역 밖에서는 블렌딩이 발생하지 않습니다. 이 관찰을 바탕으로 그림자 경계에서만 PCF를 수행하는 방법이 고안되었습니다. [Isidoro06b]는 이를 구현하는 한 가지 방식을 설명합니다. 이러한 기법은 셰이더 코드에 동적 분기 구조를 요구합니다: "그림자 경계에 있을 경우 비용이 큰 PCF를 수행하고, 그렇지 않으면 그림자 맵 샘플 하나만 채취한다."

다만, 이러한 방법의 추가 비용은 PCF 커널이 충분히 크고(예: 5×5 이상) 효율성이 입증된 경우에만 정당화됩니다. 그러나 이는 일반적인 조건일 뿐이며, 비용 대비 효과를 검증하려면 프로파일링이 필요합니다. 마지막으로, PCF 커널이 반드시 박스 필터 그리드일 필요는 없습니다. 무작위로 선택하는 방법에 관한 많은 글이 작성되었습니다. PCF 커널에 포함될 포인트들.

21.4.4 색도우 맵 구축

색도우 매핑의 첫 단계는 색도우 맵을 생성하는 것입니다. 이를 위해 **ShadowMap** 인스턴스를 생성합니다:

```
static const int SMapSize = 2048;
mSmap = new ShadowMap(md3dDevice, SMapSize, SMapSize);

그런 다음 라이트 뷰 행렬과 투영 행렬(라이트 프레임과 뷰 볼륨을 나타냄)을 정의합니다. 라이트 뷰 행렬은 주 광원에서 파생되며, 라이트 뷰 볼륨은 전체 장면의 바운딩 스피어에 맞도록 계산됩니다.

// 장면 경계 구체는 장면 구성 방식을 알고 있으므로 수동으로 추정합니다.
// 그리드는 "가장 넓은 객체"로
// 너비 20, 깊이 30.0f이며 월드 공간 원점에 중심을 둔다.
// 원점에 위치합니다. 일반적으로 모든 월드 공간
// 정점 위치를 순회하며 경계 구를 계산해야 합니다.

struct BoundingSphere
{
    BoundingSphere() : Center(0.0f, 0.0f, 0.0f), Radius(0.0f) {} XMFLOAT3 Center;
```

```

    float Radius;

};

BoundingSphere mSceneBounds; mSceneBounds.Center = XMFLOAT3(0.0f, 0.0f,
0.0f);
mSceneBounds.Radius = sqrtf(10.0f*10.0f + 15.0f*15.0f);

void ShadowsApp::UpdateScene(float dt)
{
    [...]
    //
    // 조명(그리고 그에 따른 그림자)을 애니메이션 처리합니다.
    //

    mLightRotationAngle += 0.1f*dt;

    XMMATRIX R = XMMatrixRotationY(mLightRotationAngle); for(int i = 0; i < 3; ++i)
    {
        XMVECTOR lightDir = XMLoadFloat3(&mOriginalLightDir[i]); lightDir =
        XMVector3TransformNormal(lightDir, R); XMStoreFloat3(&mDirLights[i].Direction, lightDir);
    }

    BuildShadowTransform();
}

void ShadowsApp::BuildShadowTransform()
{
    // 첫 번째 "메인" 조명만 그림자를 생성합니다.
    XMVECTOR lightDir = XMLoadFloat3(&mDirLights[0].Direction); XMVECTOR lightPos = -
    2.0f*mSceneBounds.Radius*lightDir; XMVECTOR targetPos = XMLoadFloat3(&mSceneBounds.Center);
    XMVECTOR up = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);

    XMMATRIX V = XMMatrixLook AtLH(lightPos, targetPos, up);

    // 경계 구체(bounding sphere)를 광원 공간(light space)으로 변환.
    XMFLOAT3 sphereCenterLS;
    XMStoreFloat3(&sphereCenterLS, XMVector3TransformCoord(targetPos, V));

    // 라이트 공간의 정사각형 프러스텀이 장면을 둘러쌉니다.
    float l = sphereCenterLS.x - mSceneBounds.Radius; float b = sphereCenterLS.y -
    mSceneBounds.Radius; float n = sphereCenterLS.z - mSceneBounds.Radius;
    float r = sphereCenterLS.x + mSceneBounds.Radius; float t = sphereCenterLS.y
    + mSceneBounds.Radius; float f = sphereCenterLS.z + mSceneBounds.Radius;
    XMMATRIX P = XMMatrixOrthographicOffCenterLH(l, r, b, t, n, f);

    // NDC 공간 [-1,+1]^2을 텍스처 공간 [0,1]^2으로 변환 XMMATRIX T(
        0.5f, 0.0f, 0.0f, 0.0f,
        0.0f, -0.5f, 0.0f, 0.0f,
        0.0f, 0.0f, 1.0f, 0.0f,
        0.5f, 0.5f, 0.0f, 1.0f); XMMATRIX S
        = V*P*T;

    XMStoreFloat4x4(&mLightView, V); XMStoreFloat4x4(&mLightProj,
    P); XMStoreFloat4x4(&mShadowTransform, S);
}

```

핀을 새도우 맵에 렌더링하는 방법은 다음과 같습니다:

```
mSmap->BindDsvAndSetNullRenderTarget(md3dImmediateContext); DrawSceneToShadowMap();
```

```
md3dImmediateContext->RSSetState(0);
```

```
//
```

```
// 백 버퍼와 깊이 버퍼를 OM 스테이지로 복원합니다.
```

```
//
```

```
ID3D11RenderTargetView* renderTargets[1] = {mRenderTargetView}; md3dImmediateContext->OMSetRenderTargets(1, renderTargets, mDepthStencilView); md3dImmediateContext->RSSetViewports(1, &mScreenViewport);
```

조명 관점에서 장면을 렌더링하는 데 사용하는 이펙트 파일은 새도 맵만 구축하기 때문에 매우 간단합니다. 따라서 복잡한 픽셀 셰이더 작업이 필요하지 않습니다.

```
cbuffer cbPerFrame
```

```
{
```

```
    float4x4 gLightWVP;
```

```
};
```

```
// 숫자가 아닌 값은 cbuffer에 추가할 수 없습니다. Texture2D gDiffuseMap;
```

```
SamplerState samLinear
```

```
{
```

```
    필터 = MIN_MAG_MIP_LINEAR; 주소U = 랩;
```

```
    AddressV = Wrap;
```

```
};
```

```
struct VertexIn
```

```
{
```

```
    float3 PosL : POSITION; float3 Normall :  
    NORMAL; float2 Tex : TEXCOORD;
```

```
};
```

```
struct VertexOut
```

```
{
```

```
    float4 PosH : SV_POSITION; float2 Tex :  
    TEXCOORD;
```

```
};
```

```
VertexOut VS(VertexIn vin)
```

```
{
```

```
    VertexOut vout;
```

```
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj); vout.Tex = mul(float4(vin.Tex,  
    0.0f, 1.0f), gTexTransform).xy;
```

```
    return vout;
```

```
}
```

```
// 이 변환은 알파 커트아웃 지오메트리 전용으로, 그림자가
```

```
// 올바르게 표시되도록 합니다. 텍스처를 샘플링할 필요가 없는 지오메트리는
```

```
// 텍스처 샘플링이 필요 없는 지오메트리는 깊이 패스에 NULL 픽셀 셰이더를 사용할 수 있
```

```
습니다. void PS(VertexOut pin)
```

```
{
```

```
    float4 diffuse = gDiffuseMap.Sample(samLinear, pin.Tex);
```

```
    // 새도 맵에 투명한 픽셀을 쓰지 않습니다. clip(diffuse.a - 0.15f);
```

```
}
```

```
RasterizerState Depth
```

```

{
// [MSDN에서 발췌]

// 출력 병합 단계에 현재 바인딩된 깊이 버퍼가 UNORM 형식인 경우
// 형식이나 깊이 버퍼가 바인딩되지 않은 경우, 바이어스 값은 다음과 같이 계산됩니다:
//
// 편향값 = (float)DepthBias * r + SlopeScaledDepthBias * MaxDepthSlope;
//
// 여기서 r은 깊이 버퍼 형식에서 표현 가능한 최소 값(> 0)이며
// float32로 변환된 값입니다.
// [/End MSDN]
//
// 24비트 깊이 버퍼의 경우, r = 1 / 2^24입니다.
//
// 예시: DepthBias = 100000 ==> 실제 DepthBias = 100000/2^24 = 0.006

// 장면에 맞게 이 값들을 실험해 보아야 합니다. DepthBias = 100000;
DepthBiasClamp = 0.0f;
SlopeScaledDepthBias = 1.0f;
};

technique11 BuildShadowMapTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetGeometryShader(NULL); SetPixelShader(NULL);

        SetRasterizerState(Depth);
    }
}

technique11 BuildShadowMapAlphaClipTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
        SetPixelShader(CompileShader(ps_5_0, PS()));
    }
}

```

픽셀 셰이더는 깊이 값만 출력하면 되므로 값을 반환하지 않는다는 점에 유의하십시오. 픽셀 셰이더는 오로지 알파 값이 0이거나 낮은 픽셀 프래그먼트를 클리핑하는 데 사용되며, 이는 완전한 투명도를 나타낸다고 가정합니다. 예를 들어, [그림 21.14](#)의 나무 앞 텍스처를 고려해 보십시오. 여기서는 흰색 알파 값을 가진 픽셀만 색도 맵에 그리기를 원합니다. 이를 용이하게 하기 위해 두 가지 기법을 제공합니다: 알파 클리핑 작업을 수행하는 기법과 수행하지 않는 기법입니다. 알파 클리핑이 필요하지 않다면, 텍스처만 샘플링하고 클리핑 작업을 수행하는 픽셀 셰이더를 바인딩하는 것보다 더 빠른 무효 픽셀 셰이더를 바인딩할 수 있습니다.

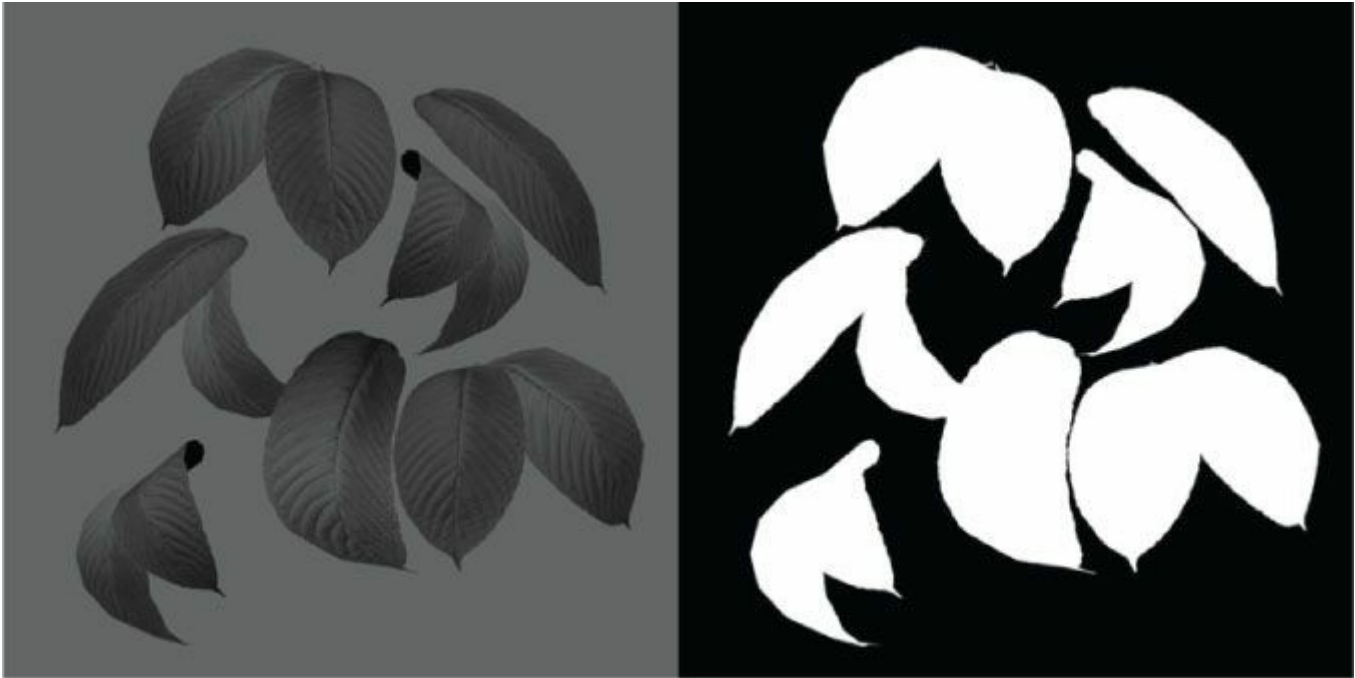


그림 21.14. 잎사귀 텍스처.

Note: 간결함을 위해 생략되었지만, 테셀레이션된 지오메트리의 깊이를 렌더링하는 셰이더는 조금 더 복잡합니다. 테셀레이션된 지오메트리를 색도우 맵에 그릴 때는 백 버퍼에 그릴 때와 동일한 방식으로 지오메트리를 테셀레이션해야 합니다(즉, 플레이어의 눈으로부터의 거리를 기준으로). 이는 일관성을 위한 것입니다.

눈이 보는 기하학 구조는 빛이 보는 것과 동일해야 한다. 그렇다고 해도, 테셀레이션된 기하학 구조가 너무 많이 변위되지 않는다면 이동되지 않는다면, 그림자 영역에서 변위가 눈에 띄지 않을 수도 있습니다. 따라서 그림자 맵을 렌더링할 때 지오메트리를 테셀레이션하지 않는 것이 가능한 최적화 방법일 수 있습니다. 이 최적화는 정확도를 속도와의 교환으로 삼습니다.

21.4.5 색도우 팩터

그림자 계수는 조명 방정식에 추가하는 새로운 계수입니다. 그림자 계수는 0에서 1 사이의 스칼라 값입니다. 0은 점이 그림자 영역에 있음을, 1은 그림자 영역 밖에 있음을 나타냅니다. PCF (§21.4.3)를 사용하면 점이 부분적으로 그림자에 있을 수도 있으며, 이 경우 그림자 계수는 0과 1 사이의 값을 가집니다. CalcShadowFactor 구현은 LightHelper.fx 파일에 있습니다.

```
static const float SMAP_SIZE = 2048.0f;
static const float SMAP_DX = 1.0f / SMAP_SIZE;
float CalcShadowFactor(SamplerComparisonState samShadow,
                      Texture2D shadowMap, float4
                      shadowPosH)
{
    // w로 나누어 투영을 완료합니다. shadowPosH.xyz /= shadowPosH.w;

    // NDC 공간에서의 깊이.
    float depth = shadowPosH.z;

    // 텍셀 크기.
    const float dx = SMAP_DX;

    float percentLit = 0.0f; const float2
    offsets[9] =
    {
        float2(-dx, -dx), float2(0.0f, -dx), float2(dx, -dx),
        float2(-dx, 0.0f), float2(0.0f, 0.0f), float2(dx, 0.0f), float2(-dx, +dx),
        float2(0.0f, +dx), float2(dx, +dx)
    };

    // 3×3 박스 필터 패턴. 각 샘플은 4-탭 PCF를 수행합니다. [unroll]
    for(int i = 0; i < 9; ++i)
    {
```

```
    percentLit += shadowMap.SampleCmpLevelZero(samShadow, shadowPosH.xy + offsets[i], depth).r;
}
```

```
    // 샘플 평균화. return percentLit /= 9.0f;
```

```
}
```

우리 모델에서 그림자 계수는 확산 조명과 반사 조명 항에 곱해집니다.

```
// 첫 번째 빛만 그림자를 생성합니다. float3 shadow =
float3(1.0f, 1.0f, 1.0f);
shadow[0] = CalcShadowFactor(samShadow, gShadowMap, pin.ShadowPosH);
```

```
// 각 광원으로부터의 빛 기여도를 합산합니다. [unroll]
```

```
for(int i = 0; i < gLightCount; ++i)
{
    float4 A, D, S; ComputeDirectionalLight(gMaterial, gDirLights[i],
bumpedNormalW, toEye, A, D, S);
```

```
    ambient += A;
    diffuse += shadow[i]*D; spec +=
    shadow[i]*S;
}
```

새도 팩터는 간접광인 앰비언트 라이트에는 영향을 미치지 않으며, 환경 맵에서 오는 반사광에도 영향을 미치지 않습니다.

21.4.6 새도우 맵 테스트

이제 새도우 맵이 구축된 후 플레이어 카메라 시점에서 장면을 렌더링하는 효과 코드를 보여드립니다. 핵심은 각 픽셀 p 에 대해 $d(p)$ 와 $s(p)$ 를 계산하는 것입니다. $d(p)$ 값은 해당 점을 광원의 NDC 공간으로 변환한 후 z 좌표가 광원으로부터의 정규화된 깊이 값을 제공함으로써 구합니다. 이 값은 투영 텍스처링을 통해 그림자 맵을 빛의 뷰 볼륨을 거쳐 장면에 투영함으로써 구합니다. 이 설정에서는 $d(p)$ 와 $s(p)$ 모두 빛의 NDC 공간에서 측정되므로 비교가 가능하다는 점에 유의하십시오. 변환 행렬 **gShadowTransform**은 로컬 공간을 새도 맵 텍스처 공간으로 변환합니다 (§21.3).
관련 코드는 굵게 표시했습니다.

```
#include "LightHelper.fx" cbuffer cbPerFrame
{
    DirectionalLight gDirLights[3]; float3 gEyePosW;
```

```
    float gFogStart; float
    gFogRange; float4
    gFogColor;
};

cbuffer cbPerObject
{
    float4x4 gWorld;
    float4x4 gWorldInvTranspose; float4x4
    gWorldViewProj; float4x4 gTexTransform;
    float4x4 gShadowTransform; Material
    gMaterial;
};
```

```
// 비수치 값은 cbuffer에 추가할 수 없습니다. Texture2D gShadowMap;
```

```
Texture2D gDiffuseMap;
```

```
Texture2D gNormalMap;
```

```
TextureCube gCubeMap;
```

```
샘플러 상태 samLinear
```

```
{  
    Filter = MIN_MAG_MIP_LINEAR; AddressU =  
    WRAP;  
    AddressV = WRAP;  
};
```

```
샘플러비교상태 samShadow
```

```
{  
    필터 = COMPARISON_MIN_MAG_LINEAR_MIP_POINT;  
    AddressU = BORDER; AddressV =  
    BORDER; AddressW = BORDER;  
    BorderColor = float4(0.0f, 0.0f, 0.0f, 0.0f);  
  
    ComparisonFunc = LESS_EQUAL;  
};
```

```
struct VertexIn
```

```
{  
    float3 PosL : POSITION; float3 NormalL :  
    NORMAL; float2 Tex : TEXCOORD; float3  
    TangentL : TANGENT;  
};
```

```
struct VertexOut
```

```
{  
    float4 PosH : SV_POSITION; float3 PosW :  
    POSITION; float3 NormalW : NORMAL;  
    float3 TangentW : TANGENT; float2 Tex :  
    TEXCOORD0;  
    float4 그림자위치H : TEXCOORD1;  
};
```

```
VertexOut VS(VertexIn vin)
```

```
{  
    VertexOut vout;  
  
    // 월드 공간으로 변환.  
    vout.PosW = mul(float4(vin.PosL, 1.0f), gWorld).xyz; vout.NormalW = mul(vin.NormalL,  
    (float3x3)gWorldInvTranspose); vout.TangentW = mul(vin.TangentL, (float3x3)gWorld);  
  
    // 동차 클립 공간으로 변환.  
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);  
  
    // 삼각형 간 보간을 위한 정점 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),  
    gTexTransform).xy;  
  
    // 그림자 맵을 장면에 투영하기 위한 투영 텍스처 좌표 생성. vout.ShadowPosH = mul(float4(vin.PosL, 1.0f),  
    gShadowTransform);  
  
    return vout;  
}
```

```
float4 PS(VertexOut pin, uniform int  
    gLightCount, uniform bool  
    gUseTexture, uniform bool gAlphaClip,
```

```

uniform bool gFogEnabled,
uniform bool gReflectionEnabled) : SV_Target
{

// 보관된 노멀은 비정규화될 수 있으므로 정규화합니다. pin.NormalW =
normalize(pin.NormalW);

// toEye 벡터는 조명 계산에 사용됩니다. float3 toEye =
gEyePosW - pin.PosW;

// 이 표면 점에서 눈까지의 거리를 캐시합니다. float distToEye = length(toEye);

// 정규화합니다.
toEye /= distToEye;

// 기본값으로 곱셈 항등원 사용. float4 texColor =
float4(1, 1, 1, 1); if(gUseTexure)
{
    // 텍스처 샘플링.
    texColor = gDiffuseMap.Sample(samLinear, pin.Tex);

    f(gAlphaClip)
    {
        // 텍스처 알파가 0.1 미만인 픽셀은 버립니다. 이 테스트는
        // 이 테스트를 가능한 한 빨리 수행하여
        // 셰이더를 조기에 종료할 수 있도록 하여
        // 나머지 셰이더 코드를 건너뛸 수 있도록 합니다.
        clip(texColor.a - 0.1f);
    }
}

//
// 노멀 매핑
//

float3 normalMapSample = gNormalMap.Sample(samLinear, pin.Tex).rgb; float3 bumpedNormalW =
NormalSampleToWorldSpace(
    normalMapSample, pin.NormalW, pin.TangentW);

//
// 조명.
//

float4 litColor = texColor; if(gLightCount >
0)
{
    // 합계를 0으로 시작합니다.
    float4 ambient = float4(0.0f, 0.0f, 0.0f, 0.0f); float4 diffuse =
    float4(0.0f, 0.0f, 0.0f, 0.0f); float4 spec = float4(0.0f, 0.0f, 0.0f, 0.0f);

    // 첫 번째 빛만 그림자를 생성합니다. float3 shadow =
    float3(1.0f, 1.0f, 1.0f);
    shadow[0] = CalcShadowFactor(samShadow, gShadowMap, pin.ShadowPosH);

    // 각 광원의 빛 기여도를 합산합니다. [unroll]
    for(int i = 0; i < gLightCount; ++i)
    {
        float4 A, D, S; ComputeDirectionalLight(gMaterial, gDirLights[i],
        bumpedNormalW, toEye, A, D, S);
    }
}

```

```

        ambient += A;
        diffuse += shadow[i]*D; spec +=
        shadow[i]*S;
    }

    litColor = texColor*(ambient + diffuse) + spec; if(gReflectionEnabled)
    {
        float3 incident = -toEye;
        float3 reflectionVector = reflect(incident, bumpedNormalW); float4 reflectionColor =
        gCubeMap.Sample(
            samLinear, reflectionVector);

        litColor += gMaterial.Reflect*reflectionColor;

    }
}

//
// 포깅
//

if(gFogEnabled)
{
    float fogLerp = saturate((distToEye - gFogStart) / gFogRange);

    // 안개 색상과 조명 색상을 블렌딩합니다. litColor = lerp(litColor,
    gFogColor, fogLerp);
}

// 디퓨즈 머티리얼과 텍스처에서 알파를 가져오는 공통 처리. litColor.a = gMaterial.Diffuse.a *
texColor.a;

return litColor;
}

```

21.4.7 새도우 맵 렌더링

이 데모에서는 화면 오른쪽 하단 모서리를 차지하는 사각형(quad)에 새도우 맵을 렌더링합니다. 이를 통해 각 프레임마다 새도우 맵이 어떻게 보이는지 확인할 수 있습니다. 그림자 맵은 **D3D11_BIND_SHADER_RESOURCE** 플래그가 설정된 깊이 버퍼 텍스처일 뿐이므로, 표면에 텍스처링하는 데에도 사용할 수 있습니다. 그림자 맵은 각 픽셀에 1차원 값(깊이 값)을 저장하기 때문에 그레이 스케일 이미지로 렌더링됩니다. [그림 21.15](#)는 "Shadow Map" 데모의 스크린샷입니다.

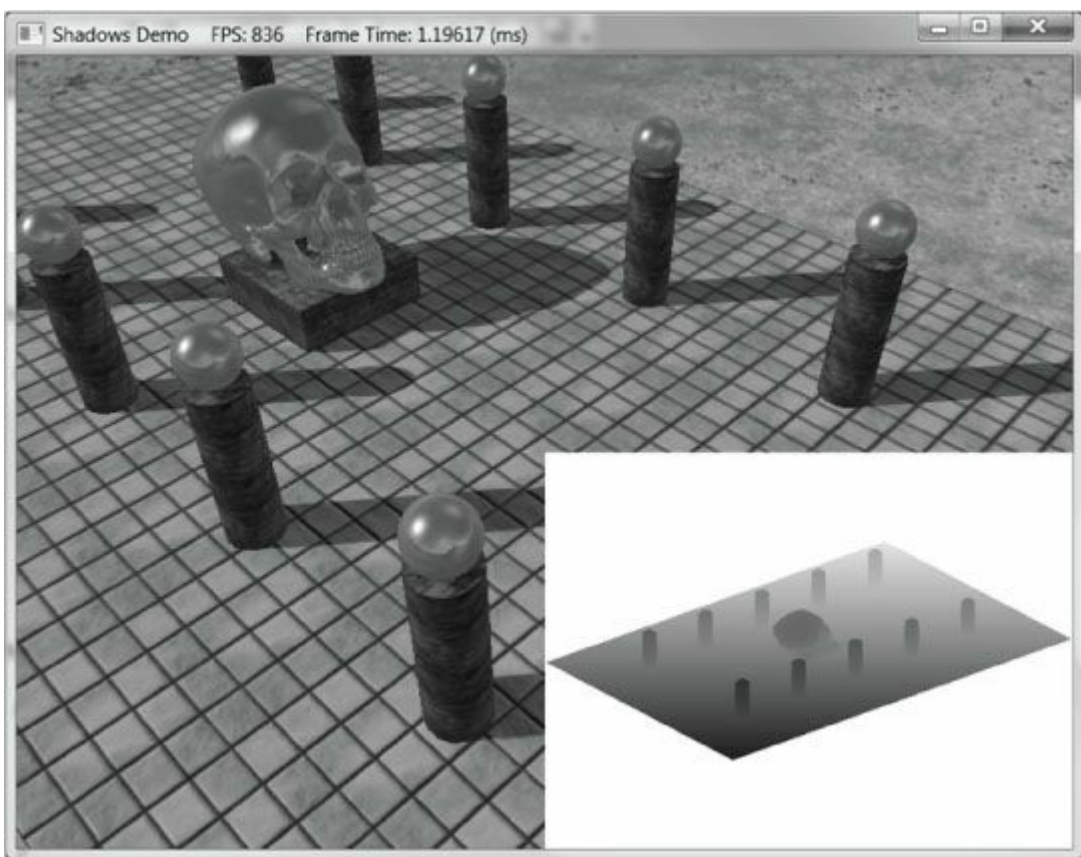


그림 21.15. "새도우 맵" 데모의 스크린샷.

21.5 대형 PCF 커널

이 섹션에서는 대형 PCF 커널 사용 시 문제가 발생하는 경우를 논의합니다. 우리의 데모는 대형 PCF 커널을 사용하지 않으므로 이 섹션은 어느 정도 선택적이지만, 몇 가지 흥미로운 아이디어를 소개합니다.

그림 21.16을 참조하십시오. 여기서 우리는 눈으로 보이는 픽셀 p 에 대한 그림자 테스트를 계산하고 있습니다. PCF 없이 우리는

거리 $d = d(p)$ 를 광원으로부터의 p 의 거리로 계산하고, 이를 대응하는 새도우 맵 값 $s_0 = s(p)$ 와 비교합니다. PCF를 사용하면 인접한 새도우 맵 값 s_{-1} 과 s_1 도 d 와 비교합니다. 그러나 d 를 s_{-1} 및 s_1 과 비교하는 것은 유효하지 않습니다. 텍셀 s_{-1} 과 s_1 은 p 와 동일한 폴리곤에 속할 수도 있고 아닐 수도 있는 장면의 서로 다른 영역들의 깊이를 나타냅니다.

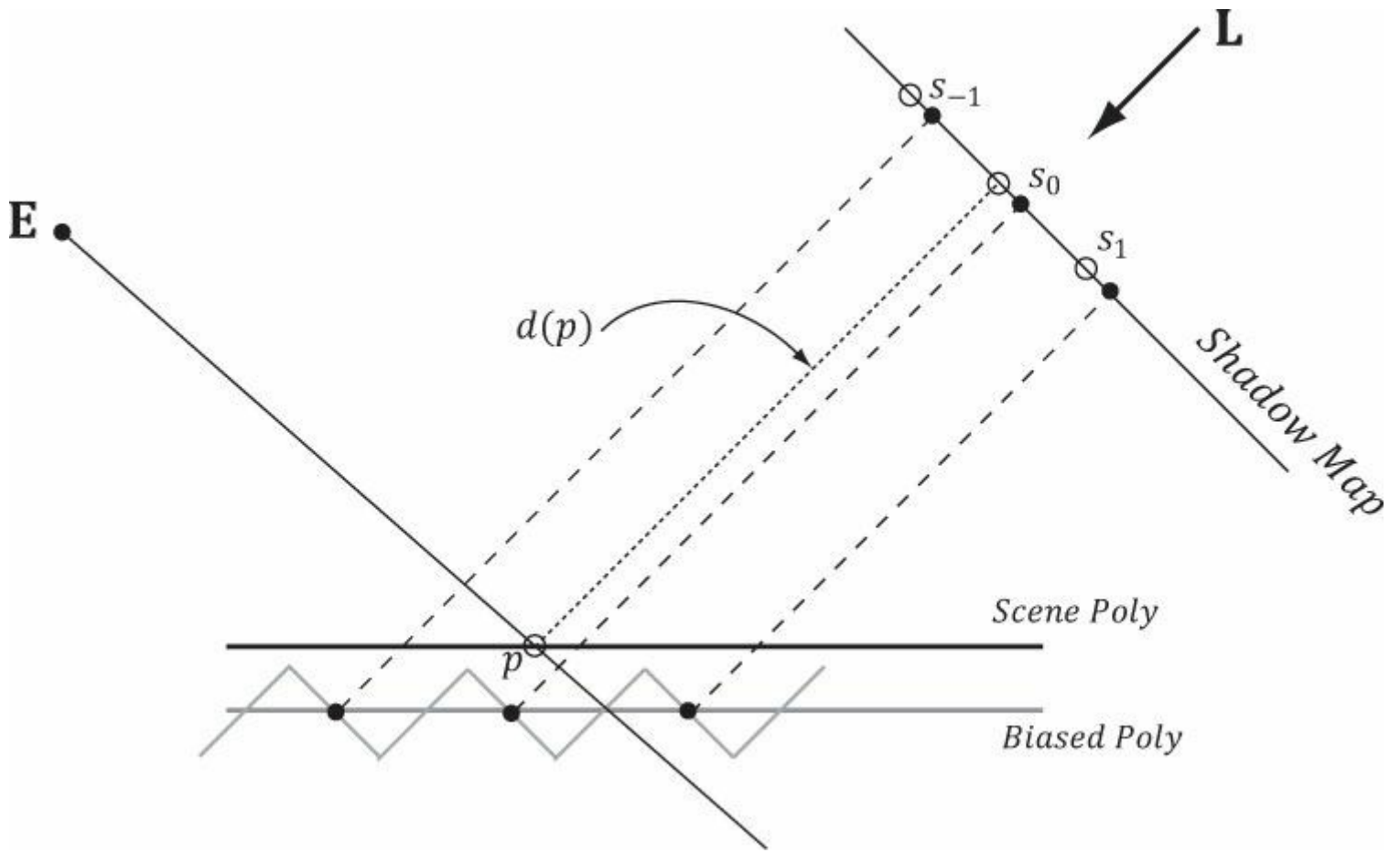


그림 21.16. 깊이 $d(p)$ 를 s_0 과 비교하는 것은 올바른데, 텍셀 s_0 이 장면 영역 p 가 포함된 영역을 덮기 때문이다. 그러나 $d(p)$ 를 s_{-1} 및 s_1 과 비교하는 것은 올바르지 않은데, 해당 텍셀들은 p 와 무관한 장면 영역을 덮기 때문이다.

그림 21.16의 시나리오는 실제로 PCF에서 오류를 발생시킵니다. 구체적으로, 색도 맵 테스트를 수행할 때 다음과 같이 계산합니다.

$$lit_0 = d \leq s_0(\text{true})$$

$$lit_{-1} = d \leq s_{-1}(\text{true})$$

$$lit_1 = d \leq s_1(\text{false})$$

결과를 보간하면 p 가 그림자 영역의 1/3에 위치한다고 계산되는데, p 를 가리는 요소가 전혀 없으므로 이는 잘못된 결과입니다.

그림 21.16에서 볼 수 있듯이 더 많은 깊이 바이어싱이 오류를 수정할 수 있습니다. 그러나 이 예시에서는 다음 샘플만 채취하고 있습니다.

색도 맵에서 인접 텍셀을 처리할 때, PCF 커널을 넓히면 더 많은 편향이 필요해집니다. 따라서 작은 PCF 커널의 경우, §21.4.2에서 설명한 대로 깊이 편향만 적용해도 이 문제를 해결할 수 있으며 걱정할 필요가 없습니다. 그러나 소프트 색도우 생성에 사용되는 5×5 나 9×9 같은 큰 PCF 커널의 경우, 이는 심각한 문제가 될 수 있습니다.

21.5.1 DDX 및 DDY 함수

이 문제에 대한 근사적 해결책을 살펴보기 전에 먼저 **ddx** 및 **ddy** HLSL 함수에 대해 논의해야 합니다. 이 함수들은 각각 $\partial p / \partial x$ 와 $\partial p / \partial y$ 를 추정합니다. 여기서 x 는 화면 공간의 x 축, y 는 화면 공간의 y 축을 의미합니다. 이 함수들을 사용하면 픽셀당 양 p 가 픽셀마다 어떻게 변화하는지 파악할 수 있습니다. 미분 함수의 활용 예시:

1. 픽셀별로 색상이 어떻게 변화하는지 추정합니다.
2. 픽셀별로 깊이가 어떻게 변화하는지 추정합니다.
3. 노멀이 픽셀별로 어떻게 변화하는지 추정합니다.

하드웨어가 이러한 편미분을 추정하는 방식은 복잡하지 않습니다. 하드웨어는 픽셀을 2×2 사각형 단위로 동시에 병렬 처리합니다. 그러면 x 방향의 편미분은 전방 차분 방정식 $q_{(x, (+1, y))} - q_{x, y}$ (양 q 가 픽셀 (x, y) 에서 픽셀 $(x + 1, y)$ 로 어떻게 변화하는지 추정)로 추정할 수 있으며, y 방향의 편미분도 마찬가지로 추정됩니다.

21.5.2 대형 PCF 커널 문제의 해결법

본문에서 설명하는 해결책은 [Tuft10]에서 비롯되었습니다. 이 전략은 p 와 인접 픽셀들이 p 와 동일한 평면에 위치한다는 가정을 전제로 합니다. 이 가정이 항상 성립하는 것은 아니지만, 현재로서는 가장 효과적인 접근법입니다.

$\mathbf{p} = (u, v, z)$ 를 광원 공간의 좌표로 정의하자. 좌표 (u, v) 는 새도우 맵에 인덱싱하는 데 사용되며, 값 z 는

$(\Delta x, \Delta y)$ 광원으로부터의 거리로 그림자 맵 테스트에 사용된다. 다각형의 접평면에 위치한 dx 와 dy 를 통해 접선 벡터 $\frac{\partial \mathbf{p}}{\partial x} = \left(\frac{\partial u}{\partial x}, \frac{\partial v}{\partial x}, \frac{\partial z}{\partial x} \right)$ 및 $\frac{\partial \mathbf{p}}{\partial y} = \left(\frac{\partial u}{\partial y}, \frac{\partial v}{\partial y}, \frac{\partial z}{\partial y} \right)$

를 계산할 수 있다. 이는 화면 공간에서 이동할 때 광 공간에서 어떻게 이동하는지 알려준다. 화면 공간에서 이동할 때 특히 접선 벡터 방향으로 화면 공간에서 이동할 때 화면 공간에서 이동할 때 화면 공간에서 이동할 때

$\Delta x \left(\frac{\partial u}{\partial x}, \frac{\partial v}{\partial x}, \frac{\partial z}{\partial x} \right) + \Delta y \left(\frac{\partial u}{\partial y}, \frac{\partial v}{\partial y}, \frac{\partial z}{\partial y} \right)$ 단위를 광학 공간에서 접선 벡터 방향으로 이동합니다. 깊이 항은 잠시 무시하고, 화면 공간에서 $(\Delta x, \Delta y)$ 단위를 이동하면, uv 평면에서 광학 공간으로 $\Delta x \left(\frac{\partial u}{\partial x}, \frac{\partial v}{\partial x} \right) + \Delta y \left(\frac{\partial u}{\partial y}, \frac{\partial v}{\partial y} \right)$ 단위를 이동합니다. 이는 행렬 방정식으로 표현할 수 있습니다:

$$[\Delta x, \Delta y] \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial v}{\partial x} \\ \frac{\partial u}{\partial y} & \frac{\partial v}{\partial y} \end{bmatrix} = \Delta x \begin{pmatrix} \frac{\partial u}{\partial x} & \frac{\partial v}{\partial x} \end{pmatrix} + \Delta y \begin{pmatrix} \frac{\partial u}{\partial y} & \frac{\partial v}{\partial y} \end{pmatrix} = [\Delta u, \Delta v]$$

따라서,

$$[\Delta x, \Delta y] = [\Delta u, \Delta v] \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial v}{\partial x} \\ \frac{\partial u}{\partial y} & \frac{\partial v}{\partial y} \end{bmatrix}^{-1} \quad (\text{식 21.1})$$

$$= [\Delta u, \Delta v] \frac{1}{\frac{\partial u}{\partial x} \frac{\partial v}{\partial y} - \frac{\partial v}{\partial x} \frac{\partial u}{\partial y}} \begin{bmatrix} \frac{\partial v}{\partial y} & -\frac{\partial v}{\partial x} \\ -\frac{\partial u}{\partial y} & \frac{\partial u}{\partial x} \end{bmatrix}$$

 제2장에서 살펴본 바와 같이

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}^{-1} = \frac{1}{A_{11}A_{22} - A_{12}A_{21}} \begin{bmatrix} A_{22} & -A_{12} \\ -A_{21} & A_{11} \end{bmatrix}$$

이 새로운 방정식은 uv 평면에서 빛 공간으로 $(\Delta u, \Delta v)$ 단위만큼 이동하면 스크린 공간에서 $(\Delta x, \Delta y)$ 단위만큼 이동함을 알려줍니다. 그렇다면 방정식 21.1이 왜 중요한가요? 음, PCF 커널을 구축할 때 새도우 맵에서 인접한 값을 샘플링하기 위해 텍스처 좌표를 오프셋합니다:

```
// 텍셀 크기.
const float dx = SMAP_DX;

float percentLit = 0.0f; const float2
offsets[9] =
{
    float2(-dx, -dx), float2(0.0f, -dx), float2(dx, -dx),
    float2(-dx, 0.0f), float2(0.0f, 0.0f), float2(dx, 0.0f), float2(-dx, +dx),
    float2(0.0f, +dx), float2(dx, +dx)
};

// 3x3 박스 필터 패턴. 각 샘플은 4.샘 PCF를 수행합니다. [unroll]
for(int i = 0; i < 9; ++i)
```

```

{
    percentLit += shadowMap.SampleCmpLevelZero(samShadow, shadowPosH.xy + offsets[i], depth).r;
}

```

다시 말해, 우리는 uv 평면에서 빛 공간으로 얼마나 변위하는지 알고 있습니다—우리는 $(\Delta u, \Delta v)$ 를 알고 있습니다. 방정식 21.1은 빛 공간에서 $(\Delta u, \Delta v)$ 단위를 이동할 때 화면 공간에서 $(\Delta x, \Delta y)$ 만큼 이동한다는 것을 알려줍니다.

이제 우리가 무시해왔던 깊이 향으로 돌아가 보자. 화면 공간에서 $(\Delta x, \Delta y)$ 만큼 이동하면, 빛 공간 깊이는

$$\Delta z = \Delta x \frac{\partial z}{\partial x} + \Delta y \frac{\partial z}{\partial y}$$

따라서 PCF를 수행하기 위해 텍스처 좌표를 오프셋할 때, 깊이 테스트에 사용되는 깊이 값을 다음과 같이 수정할 수 있습니다: (그림 21.17 참조).

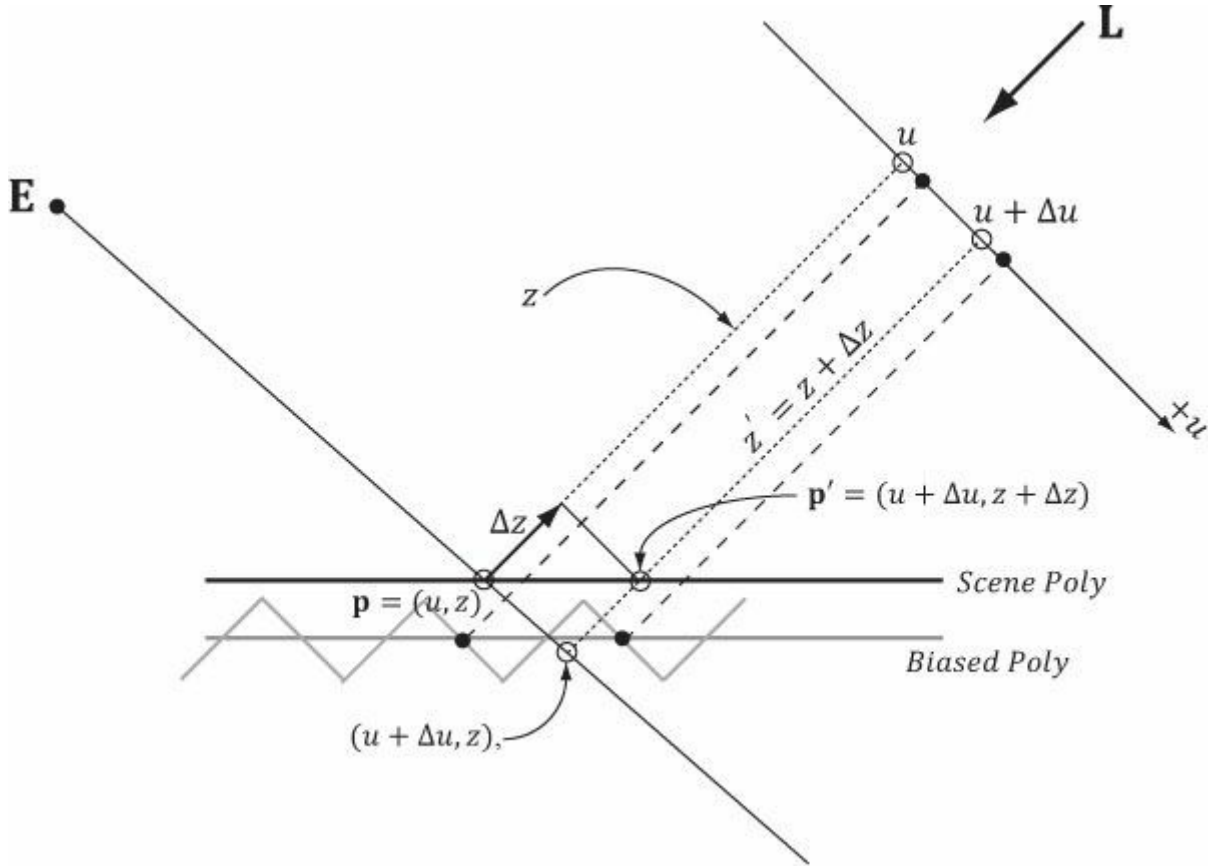
$$z' = z + \Delta z$$


그림 21.17. 단순화를 위해 2차원으로 설명합니다. $p = (u, z)$ 에서 u 방향으로 Δu 만큼 오프셋하여 $(u + \Delta u, z)$ 를 얻으면, 다각형 위에 머물기 위해 Δz 만큼 오프셋하여 $p' = (u + \Delta u, z + \Delta z)$ 를 얻어야 합니다.

요약하자면:

1. PCF 구현에서 우리는 색도우 맵의 인접 값을 샘플링하기 위해 텍스처 좌표를 오프셋합니다. 따라서 각 샘플에 대해 $(\Delta u, \Delta v)$ 를 알 수 있습니다.
2. 방사 공간에서 $(\Delta u, \Delta v)$ 단위를 오프셋할 때 화면 공간 오프셋 $(\Delta x, \Delta y)$ 을 구하기 위해 방정식 21.1을 사용할 수 있습니다.
3. $(\Delta x, \Delta y)$ 를 구한 후, $\Delta z = \Delta x \frac{\partial z}{\partial x} + \Delta y \frac{\partial z}{\partial y}$ 을 적용하여 광학 공간 깊이 변화를 계산합니다.

The “CascadedShadowMaps11” 데모 에서 의 DirectX 11 SDK 는 이 메소드를 를 CalculateRightAndUpTexelDepthDeltas 및 CalculatePCFPercentLit 함수에서

CalculateRightAndUpTexelDepthDeltas 및 CalculatePCFPercentLit 함수에서

21.5.3 대형 PCF 커널 문제에 대한 대안적 해결책

[Isidoro06]에 제시된 이 해법은 이전 절과 동일한 취지이지만, 약간 다른 접근법을 취합니다.

$p = (u, v, z)$ 를 라이트 스페이스의 좌표로 정의하자. 좌표 (u, v) 는 색도우 맵 인덱싱에 사용되며, z 값은

거리 from the light source used in the shadow map test. We can compute

$$\frac{\partial \mathbf{p}}{\partial x} = \left(\frac{\partial u}{\partial x}, \frac{\partial v}{\partial x}, \frac{\partial z}{\partial x} \right) \text{ and}$$

$$\frac{\partial \mathbf{p}}{\partial y} = \left(\frac{\partial u}{\partial y}, \frac{\partial v}{\partial y}, \frac{\partial z}{\partial y} \right) \text{ with } \mathbf{d}\mathbf{x} \text{ and } \mathbf{d}\mathbf{y}.$$

특히, 우리가 이러한 미분을 취할 수 있다는 사실은

$$u = u(x, y), v = v(x, y), \text{ 그리고 } z = z(x, y) \text{ 모두}$$

x 와 y 의 함수임을 의미합니다. 그러나 z 를 u 와 v 의 함수로도 생각할 수 있습니다—즉, $z = z(u, v)$. u -방향과 v -방향으로 빛 공간을 이동함에 따라 깊이 z 는 다각형 평면을 따라 변화합니다. 연쇄 법칙에 따라 다음과 같습니다:

$$\begin{aligned}\frac{\partial z}{\partial x} &= \frac{\partial z}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial z}{\partial v} \frac{\partial v}{\partial x} \\ \frac{\partial z}{\partial y} &= \frac{\partial z}{\partial u} \frac{\partial u}{\partial y} + \frac{\partial z}{\partial v} \frac{\partial v}{\partial y}\end{aligned}$$

또는 행렬 표기법으로:

$$\begin{bmatrix} \frac{\partial z}{\partial x} & \frac{\partial z}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{\partial z}{\partial u} & \frac{\partial z}{\partial v} \end{bmatrix} \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} \end{bmatrix}$$

역행렬을 취하면:

$$\begin{aligned}\begin{bmatrix} \frac{\partial z}{\partial u} & \frac{\partial z}{\partial v} \end{bmatrix} &= \begin{bmatrix} \frac{\partial z}{\partial x} & \frac{\partial z}{\partial y} \end{bmatrix} \begin{bmatrix} \frac{\partial u}{\partial x} & \frac{\partial u}{\partial y} \\ \frac{\partial v}{\partial x} & \frac{\partial v}{\partial y} \end{bmatrix}^{-1} \\ &= \frac{\begin{bmatrix} \frac{\partial z}{\partial x} & \frac{\partial z}{\partial y} \end{bmatrix}}{\frac{\partial u}{\partial x} \frac{\partial v}{\partial y} - \frac{\partial u}{\partial y} \frac{\partial v}{\partial x}} \begin{bmatrix} \frac{\partial v}{\partial y} & -\frac{\partial u}{\partial y} \\ -\frac{\partial v}{\partial x} & \frac{\partial u}{\partial x} \end{bmatrix}\end{aligned}$$

이제 $\frac{\partial z}{\partial u}$ 와 $\frac{\partial z}{\partial v}$ 을 직접 구했습니다(방정식 우변의 모든 항은 알려져 있음). uv 평면에서 광 공간 상으로 $(\Delta u, \Delta v)$ 만큼 이동하면, 광 공간 깊이는 $\Delta z = \Delta u \frac{\partial z}{\partial u} + \Delta v \frac{\partial z}{\partial v}$ 만큼 이동합니다.

따라서 이 접근법에서는 화면 공간으로 변환할 필요가 없으며, 광원 공간에 머물 수 있습니다. 그 이유는 u 와 v 가 변할 때 깊이가 어떻게 변하는지 직접 파악했기 때문입니다. 반면 이전 섹션에서는 화면 공간에서 x 와 y 가 변할 때 깊이가 어떻게 변하는지만 알 수 있었습니다.

u 와 v 가 변할 때 깊이가 어떻게 변하는지 직접 알아냈기 때문입니다. 반면 이전 섹션에서는 화면 공간에서 x 와 y 가 변할 때 깊이가 어떻게 변하는지만 알았습니다.

21.6 요약

- 백 버퍼가 항상 렌더 타겟일 필요는 없습니다. 다른 텍스처로 렌더링할 수 있습니다. 텍스처 렌더링은 GPU가 런타임에 텍스처 내용을 효율적으로 업데이트할 수 있는 방법을 제공합니다. 텍스처로 렌더링한 후에는 해당 텍스처를 셰이더 입력으로 바인딩하고 지오메트리에 매핑할 수 있습니다. 섀도우 맵, 물 시뮬레이션, 범용 GPU 프로그래밍과 같은 많은 특수 효과는 텍스처 렌더링 기능이 필요합니다.
- 정사영 투영에서는 시야 공간이 너비 w , 높이 h , 근거리 평면 n , 원거리 평면 f 를 가진 상자 형태([그림 21.1](#) 참조)이며, 투영 선은 시야 공간 z 축과 평행합니다. 이러한 투영은 주로 평행선이 투영 후에도 평행하게 유지되어야 하는 3D 과학 또는 공학 응용 분야에서 사용됩니다. 그러나 평행광원이 생성하는 그림자를 모델링하기 위해 정사영 투영을 사용할 수 있습니다.
- 투영 텍스처링은 슬라이드 프로젝터처럼 텍스처를 임의의 기하형상에 투영할 수 있게 해주기 때문에 이 이름이 붙었습니다. 투영 텍스처링의 핵심은 적용된 텍스처가 기하형상에 투영된 것처럼 보이도록 각 픽셀에 대한 텍스처 좌표를 생성하는 것입니다. 이러한 텍스처 좌표를 투영 텍스처 좌표라고 합니다. 우리는 픽셀을 프로젝터의 투영 평면에 투영한 다음 텍스처 좌표계로 매핑함으로써 투영 텍스처 좌표를 얻습니다.

텍스처 좌표는 해당 픽셀을 프로젝트의 투영 평면에 투영한 후 텍스처 좌표계로 매핑하여 구합니다.

4. 새도 매핑은 실시간 그림자 표현 기법으로, 임의의 기하학적 구조에 그림자를 투사합니다(평면 그림자에 국한되지 않음). 새도 매핑의 핵심은 빛의 시점에서 장면의 깊이 정보를 새도 맵에 렌더링하는 것입니다. 따라서 새도 맵은 빛의 시점에서 보이는 모든 픽셀의 깊이 정보를 저장하게 됩니다. 그런 다음 카메라 시점에서 장면을 다시 렌더링하고, 투영 텍스처링을 사용하여 새도우 맵을 장면에 투영합니다. 그림자 맵에서 픽셀 p 로 투영된 깊이 값을 $s(p)$, 광원으로부터의 픽셀 깊이를 $d(p)$ 라고 하자. 이때 $d(p) > s(p)$, 즉 픽셀의 깊이가 투영된 픽셀 깊이 $s(p)$ 보다 클 경우, p 를 가리는 광원에 더 가까운 픽셀이 존재하여 p 를 그림자에 빠뜨리게 된다.
5. 엘리어싱은 새도우 맵의 가장 큰 과제입니다. 새도우 맵은 연관된 광원에 대한 가시적 픽셀 중 가장 가까운 픽셀의 깊이를 저장합니다. 그러나 새도우 맵은 유한한 해상도만을 가집니다. 따라서 각 새도우 맵 텍셀은 장면의 특정 영역에 대응합니다. 즉, 새도우 맵은 광원 관점에서 장면 깊이의 이산적 샘플링에 불과합니다. 이로 인해 '그림자 여드름(shadow acne)'으로 알려진 에일리어싱 문제가 발생합니다. 래스터화 렌더링 상태에서 그래픽스 하드웨어의 내장 기능인 경사 스케일링 바이어스(slope-scaled-bias)를 사용하는 것이 그림자 여드름을 해결하는 일반적인 전략입니다. 그림자 맵의 유한한 해상도는 그림자 경계에서도 에일리어싱을 유발합니다. PCF(Precise Convolution Filter)는 이에 대한 널리 사용되는 해결책입니다. 에일리어싱 문제에 활용되는 보다 진보된 해결책으로는 캐스케이드 새도우 맵(cascaded shadow maps)과 분산 새도우 맵(variance shadow maps)이 있습니다.

21.7 연습문제

1. 텍스처를 장면에 투영하여 슬라이드 프로젝터를 시뮬레이션하는 프로그램을 작성하세요. 투시 투영과 직교 투영을 모두 실험해 보세요.
2. 텍스처 주소 모드를 사용하여 프로젝트의 프러스텀(frustum) 외부 지점이 빛을 받지 않도록 이전 연습 문제의 해결책을 수정하세요.
3. 연습문제 1의 해결책을 수정하여 스포트라이트를 사용하도록 하되, 스포트라이트 원뿔 영역 밖에 있는 점들은 프로젝트로부터 빛을 전혀 받지 않도록 하십시오.
4. 이 장의 데모 애플리케이션을 원근 투영을 사용하여 수정하십시오. 정사 투영에서 작동했던 경사-스케일-바이어스(slope-scaled-bias)가 원근 투영에서는 제대로 작동하지 않을 수 있다는 점에 유의하십시오. 원근 투영을 사용할 때, 깊이 맵이 흰색(1.0)으로 크게 편향되어 있음을 확인하십시오. 그림 5.25의 그래프를 고려할 때 이것이 타당해 보입니까?
5. 다음 그림자 맵 해상도로 실험해 보십시오: 4096×4096 , 1024×1024 , 512×512 , 256×256 . 크기에 따라 달라지는 효과 상수도 반드시 업데이트하십시오:
- ```
static const float SMAP_SIZE = 2048.0f;
static const float SMAP_DX = 1.0f / SMAP_SIZE;
```
6.  $[l, r] \times [b, t] \times [n, f] \rightarrow [-1, 1] \times [-1, 1] \times [0, 1]$ 로 매핑하는 행렬을 유도하라. 이는 "오프 센터" 정사영 뷰 볼륨이다(즉, 상자가 뷰 공간 원점을 중심으로 하지 않음). 반면, §21.2에서 유도한 정사영 행렬은 "중심에 위치한" 정사영 뷰 볼륨이다.
7. 16장에서 우리는 투시 투영 행렬을 이용한 피킹에 대해 배웠다. 중심이 어긋난 직교 투영에 대한 피킹 공식을 유도하라.
8. "Shadow Demo"를 수정하여 단일 점 샘플링 그림자 테스트(즉, PCF 없음)를 사용하도록 하십시오. 경계가 뚜렷한 그림자와 들쭉날쭉한 그림자 가장자리를 관찰할 수 있어야 합니다.
9. 경사 스케일링 바이어스를 끄고 그림자 여드름 현상을 관찰하십시오.
10. 경사 스케일링 바이어스를 매우 크게 수정하여 피터 팬 현상을 관찰하십시오.
11. BuildShadowMap.fx의 테셀레이션 코드를 연구하여 그림자 맵으로 렌더링되는 지오메트리를 테셀레이션하는 방법을 학습하십시오.
12. 방향광의 새도우 맵 생성에 정사영 투영을, 스포트라이트의 새도우 맵 생성에 원근 투영을 사용할 수 있습니다. 점광원의 새도우 맵 생성에 큐브 맵과 6개의  $90^\circ$  시야각 원근 투영을 활용하는 방법을 설명하세요.



(17장에서 동적 큐브 맵이 어떻게 생성되었는지 떠올려 보십시오.) 큐브 맵을 사용하여 새도 맵 테스트가 어떻게 수행되는지 설명하십시오.

# 제22장

## 앰비언트 오클루전

성능 제약으로 인해 실시간 조명 모델은 간접광(즉, 장면 내 다른 물체에 반사된 빛)을 고려하지 않는 경우가 흔합니다. 그러나 현실 세계에서 우리가 보는 빛의 상당수는 간접광입니다. 제7장에서 조명 방정식에 앰비언트 항을 도입했습니다:

$$\mathbf{A} = \mathbf{l}_a \otimes \mathbf{m}_a$$

색상  $\mathbf{l}_a$ 는 표면이 광원에서 받는 간접(주변)광의 총량을 지정합니다. 주변 재료 색상  $\mathbf{m}_a$ 는 표면이 반사하고 흡수하는 주변광의 양을 지정합니다. 주변광은 단순히 물체를 균일하게 약간 밝게 할 뿐이며, 실제 물리적 계산은 전혀 이루어지지 않습니다. 간접광은 장면 내에서 여러 번 산란되고 반사되어 모든 방향에서 균일하게 물체에 도달한다는 개념입니다. 그림 22.1은 주변광 항만 사용하여 모델을 그릴 경우 일정한 색상으로 렌더링된다는 것을 보여줍니다.

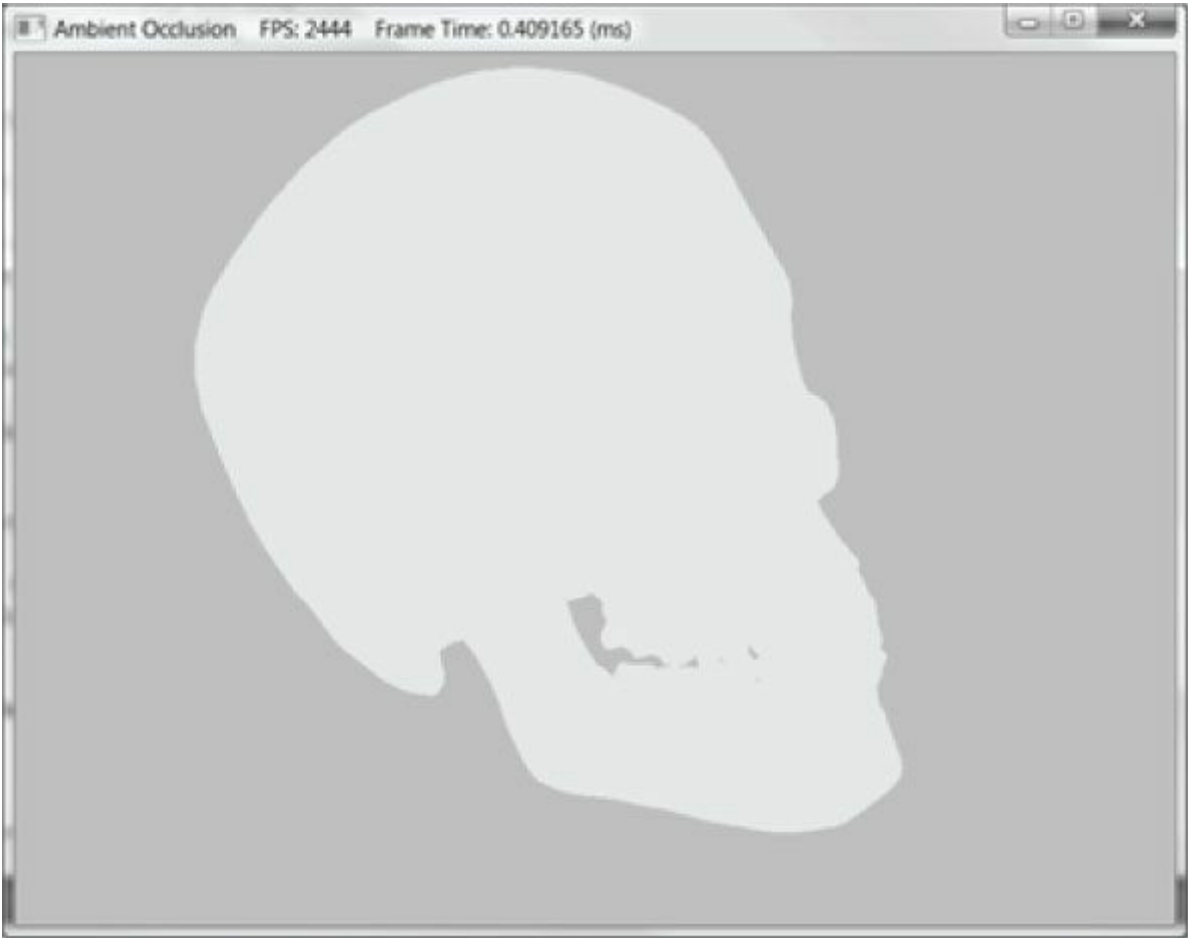


그림 22.1. 주변광 항만 적용된 메쉬는 단색으로 렌더링됩니다.

그림 22.1은 우리의 주변광 항이 개선이 필요함을 분명히 보여줍니다. 이 장에서는 주변광 항을 개선하기 위한 널리 사용되는 기법인 주변광 차단(Ambient Occlusion)에 대해 논의합니다.

목표:

- 1. 앰비언트 오클루전의 기본 개념과 레이 캐스팅을 통한 구현 방법을 이해한다.
- 2. 스크린 스페이스 앰비언트 오클루전(SSAO)이라 불리는 스크린 스페이스에서 앰비언트 오클루전의 실시간 근사값 구현 방법을 학습한다.

### 22.1 레이 캐스팅을 통한 앰비언트 오클루전

앰비언트 오클루전(Ambient Occlusion)의 개념은 표면상의 점  $p$ 가 받는 간접광의 양이  $p$ 를 중심으로 한 반구 영역의 입사광에 대한 가림 정도에 비례한다는 것이다—그림 22.2 참조.

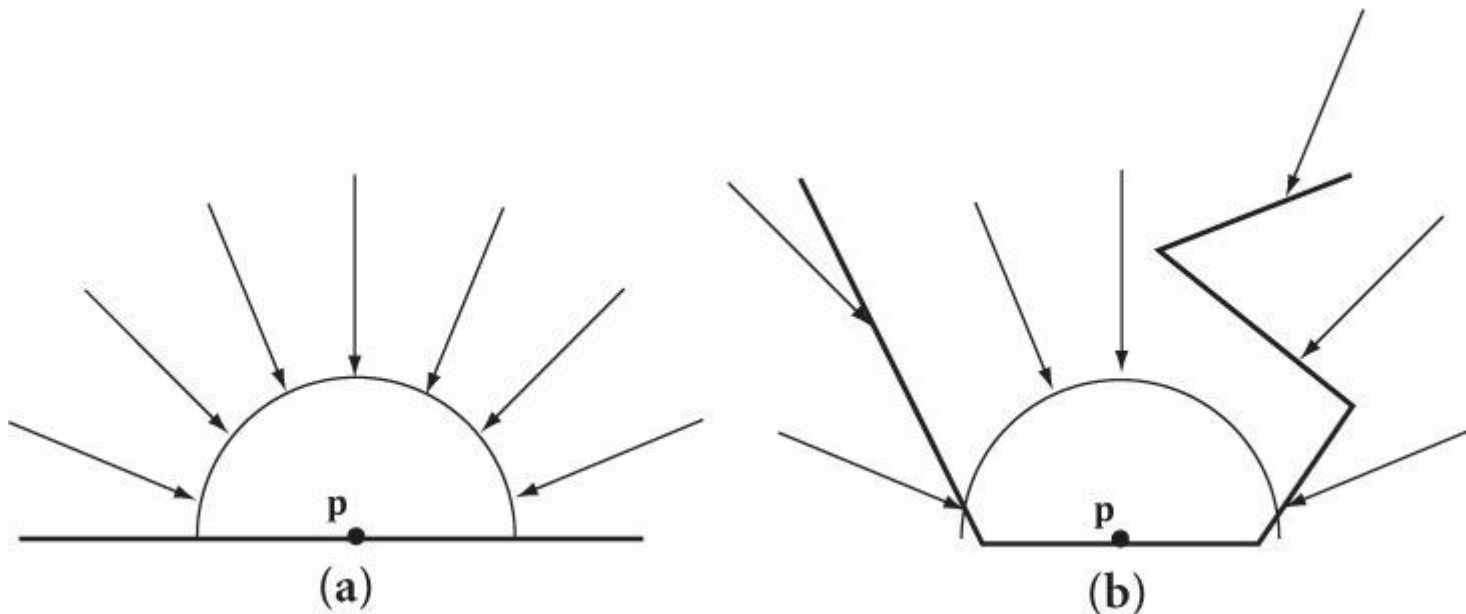


그림 22.2. (a) 점  $p$ 는 완전히 가려지지 않아  $p$ 를 중심으로 한 반구 상의 모든 입사광이  $p$ 에 도달한다. (b) 지오메트리가  $p$ 를 부분적으로 가려  $p$ 를 중심으로 한 반구 상의 입사광선을 차단한다.

점  $p$ 의 가림 정도를 추정하는 한 가지 방법은 광선 투사(ray casting)를 이용하는 것이다.  $p$ 를 중심으로 한 반구 영역에 무작위로 광선을 투사하고, 메쉬와의 교차 여부를 확인한다(그림 22.3).  $N$ 개의 광선을 투사했을 때  $h$ 개가 메쉬와 교차한다면, 해당 점의 가림 값은 다음과 같다:

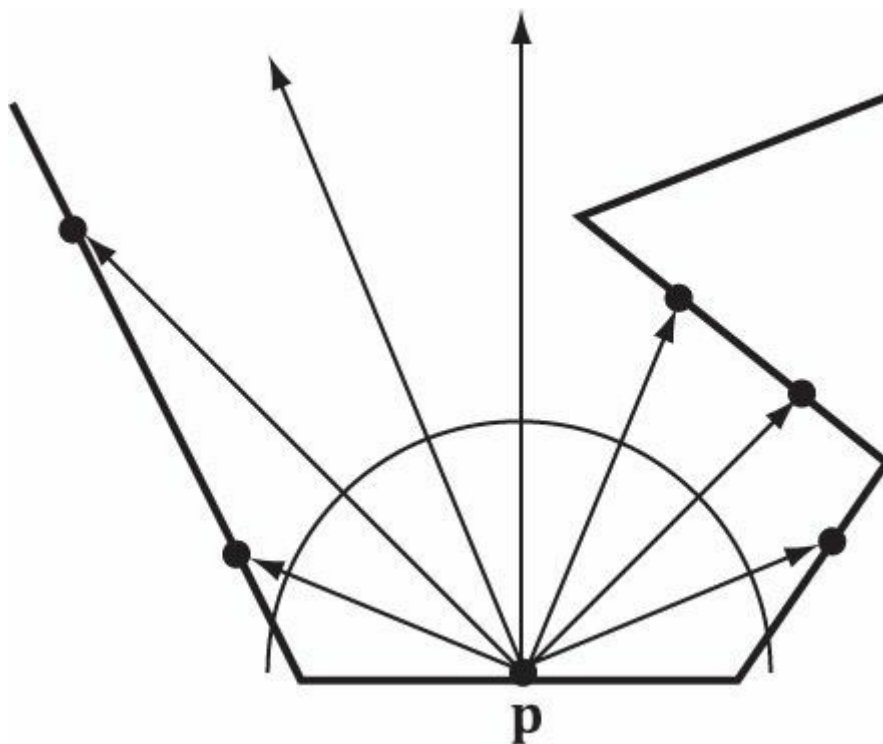


그림 22.3. 광선 투사를 통한 주변 음영 추정.

$$occlusion = \frac{h}{N} \in [0,1]$$

교차점  $q$ 가  $p$ 로부터의 거리가 임계값  $d$ 보다 작은 광선만이 가림 추정값에 기여해야 합니다. 이는  $p$ 로부터 멀리 떨어진 교차점  $q$ 는  $p$ 를 가리기에는 너무 멀기 때문입니다.

차폐 계수는 해당 점이 얼마나 차폐되었는지(즉, 얼마나 많은 빛을 받지 못하는지)를 측정한다.

계산을 위해 우리는 이 값의 역수를 다루고자 합니다. 즉, 해당 점이 얼마나 많은 빛을 받는지 알고 싶어 합니다—이를

접근성(또는 주변 접근성이라 부름)은 가림 현상에서 파생되며 다음과 같이 정의됩니다:

$$\text{접근성} = 1 - \text{오클루전} \in [0, 1]$$

"앰비언트 오클루전" 데모는 삼각형마다 레이 캐스트를 수행한 후, 해당 삼각형을 공유하는 정점들의 오클루전 결과를 평균화합니다. 레이의 기점은 삼각형의 중심이며, 삼각형의 반구 영역에 걸쳐 무작위 레이 방향을 생성합니다.

```
void AmbientOcclusionApp::BuildVertexAmbientOcclusion(
 std::vector<Vertex::AmbientOcclusion>& vertices, const std::vector<UINT>&
 indices)
{
 UINT vcount = vertices.size(); UINT tcount =
 indices.size()/3;

 std::vector<XMVECTOR> positions(vcount); for(UINT i = 0; i < vcount;
 ++i)
 positions[i] = vertices[i].Pos;

 Octree octree; octree.Build(positions, indices);

 // 각 정점에 대해, 해당 정점을 포함하는 삼각형의 개수를 계산합니다. std::vector<int> vertexSharedCount(vcount);

 // 각 삼각형에 대해 광선을 투사하고, 이 삼각형을 공유하는 정점들과 함께 삼각형 가림 0// 의 평
 균을 구합니다.
 for(UINT i = 0; i < tcount; ++i)
 {
 UINT i0 = indices[i*3+0]; UINT i1 =
 indices[i*3+1]; UINT i2 =
 indices[i*3+2];

 XMVECTOR v0 = XMLoadFloat3(&vertices[i0].Pos); XMVECTOR v1 =
 XMLoadFloat3(&vertices[i1].Pos); XMVECTOR v2 =
 XMLoadFloat3(&vertices[i2].Pos);

 XMVECTOR edge0 = v1 - v0; XMVECTOR
 edge1 = v2 - v0;

 XMVECTOR normal = XMVector3Normalize(XMVector3Cross(edge0, edge1));

 XMVECTOR centroid = (v0 + v1 + v2)/3.0f;

 // 자기 교차 방지 오프셋. centroid += 0.001f*normal;

 const int NumSampleRays = 32; float
 numUnoccluded = 0;
 for(int j = 0; j < NumSampleRays; ++j)
 {
 XMVECTOR randomDir = MathHelper::RandHemisphereUnitVec3(normal);

 // 무작위 광선이 장면 메쉬와 교차하는지 테스트.
 //
 // TODO: 기술적으로 삼각형을 가리는 것처럼 멀리 있는 교차점은
 // 삼각형을 가리는 것으로 간주해서는 안 되지만
 // 데모용으로는 괜찮습니다. if(!octree.RayOctreeIntersect(centroid, randomDir))
 {
 numUnoccluded++;
 }
 }
 }
```

```
float ambientAccess = numUnoccluded / NumSampleRays;
```

```
// 이 면을 공유하는 정점들과 평균을 냅니다. vertices[i0].AmbientAccess += ambientAccess;
```

```
vertices[i1].AmbientAccess += ambientAccess; vertices[i2].AmbientAccess +=
ambientAccess;
```

```
vertexSharedCount[i0]++;
```

```
vertexSharedCount[i1]++;
```

```
vertexSharedCount[i2]++;
```

```
}
```

```
// 추가한 샘플 수로 나누어 평균을 완성하고,
```

```
// 버텍스 속성에 저장합니다. for(UINT i = 0; i <
```

```
vcount; ++i)
```

```
{
```

```
vertices[i].AmbientAccess /= vertexSharedCount[i];
```

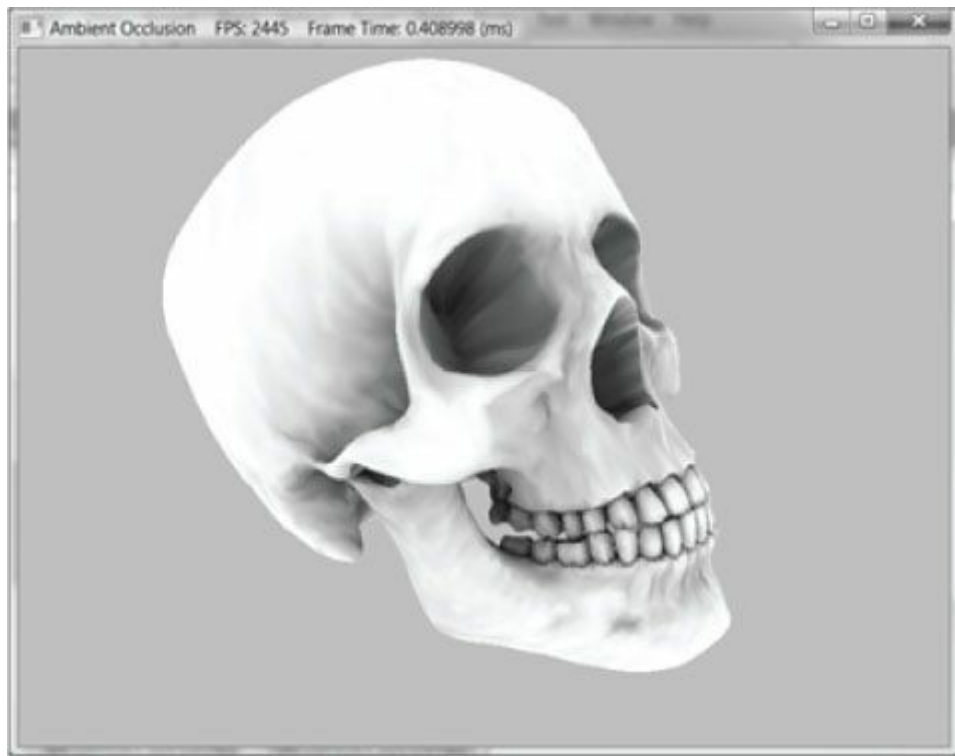
```
}
```

```
}
```

"앰비언트 오클루전" 데모의 세부 사항은 다루지 않겠습니다. 우리가 진정으로 원하는 앰비언트 오클루전 기법은 다음 섹션에서 논의됩니다. 다만 데모에 대해 두 가지 언급할 점이 있습니다.

1. 이 데모는 광선/삼각형 교차 테스트 속도를 높이기 위해 옥트리(octree)를 사용합니다. 수천 개의 삼각형으로 구성된 메쉬의 경우, 무작위 광선 하나하나를 모든 메쉬 삼각형과 테스트하는 것은 매우 느릴 것입니다. 옥트리는 삼각형을 공간적으로 정렬하므로, 광선과 교차할 가능성이 높은 삼각형만 빠르게 찾을 수 있습니다. 이는 광선/삼각형 교차 테스트 횟수를 크게 줄여줍니다. 옥트리는 고전적인 공간 데이터 구조이며, 연습문제 1에서는 이를 더 깊이 연구해 보도록 요청합니다.
2. 데모는 §15.2.1에서 처음 언급된 *xnacollision.h/cpp* 유틸리티 라이브러리를 사용합니다. 해당 파일은 *Microsoft DirectX SDK (2010년 6월)\Samples\C++\Misc\Collision*에 위치합니다. 이 파일들은 광선/삼각형 교차, 광선/박스 교차, 박스/박스 교차, 박스/평면 교차, 박스/프리스트럼, 구체/프리스트럼 등 일반적인 기하학적 원시체 교차 테스트에 대한 빠른 XNA 수학 구현을 제공합니다.

[그림 22.4](#)는 이전 알고리즘으로 생성된 앰비언트 오클루전만으로 렌더링된 모델의 스크린샷입니다(씬 내에는 광원이 없습니다). 앰비언트 오클루전은 초기화 단계에서 사전 계산되어 정점 속성으로 저장됩니다. [그림 22.1](#)과 비교하면 엄청난 개선을 확인할 수 있습니다—이제 모델이 실제로 3D처럼 보입니다.



*그림 22.4*. 메쉬는 앰비언트 오클루전만으로 렌더링되었으며, 장면 조명은 존재하지 않습니다. 틸새가 더 어두운 것을 확인할 수 있습니다. 이는 광선을 투사할 때 기하학적 구조와 교차할 가능성이 높아 오클루전에 기여하기 때문입니다. 반면 두 개골 덮개는 흰색(오클루전되지 않음)입니다. 두개골 덮개 상의 점들에 대해 반구 위로 광선을 투사할 때, 이 광선들은 두개골의 어떤 기하학적 구조와도 교차하지 않기 때문입니다.

정적 모델에는 앰비언트 오클루전 사전 계산이 효과적입니다. 앰비언트 오클루전 데이터를 저장하는 텍스처인 *앰비언트 오클루전 맵*을 생성하는 도구(<http://www.xnormal.net>)도 존재합니다. 그러나 애니메이션 모델의 경우 이러한 정적 접근법은 한계에 부딪힙니다.

다운로드. "앰비언트 오클루전" 데모를 로드하여 실행하면 단 하나의 모델에 대한 앰비언트 오클루전을 사전 계산하는 데 몇 초가 소요됨을 알 수 있습니다. 따라서 런타임에 광선을 발사하여 동적 앰비언트 오클루전을 구현하는 것은 불가능합니다. 다음 섹션에서는 화면 공간 정보를 사용하여 실시간으로 앰비언트 오클루전을 계산하는 널리 사용되는 기법을 살펴보겠습니다.

## 22.2 스크린 스페이스 앰비언트 오클루전

*화면 공간 앰비언트 오클루전(SSAO)* 전략은 매 프레임마다 장면 뷰 공간의 법선과 깊이 정보를 전체 화면 렌더 타깃에 렌더링한 후, 이 렌더 타깃만을 입력으로 사용하여 각 픽셀의 앰비언트 오클루전을 추정하는 방식입니다.

### 22.2.1 법선 및 깊이 패스 렌더링

장면을 화면 크기의 `DXGI_FORMAT_R16G16B16A16_FLOAT` 노멀/깊이 텍스처 맵에 렌더링합니다. 여기서 RGB는 뷰 스페이스 노멀을 저장하고 알파 채널은 뷰 스페이스 깊이(*z*-좌표)를 저장합니다. 이 패스에 사용되는 버텍스/픽셀 셰이더는 다음과 같습니다:

```
VertexOut VS(VertexIn vin)
{
 VertexOut vout;

 // 뷰 공간으로 변환.
 vout.PosV = mul(float4(vin.PosL, 1.0f), gWorldView).xyz;
 vout.NormalV = mul(vin.NormalL, (float3x3)gWorldInvTransposeView);

 // 동차 클립 공간으로 변환.
 vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);

 // 삼각형 간 보간을 위한 정점 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
 gTexTransform).xy; return vout;
}

float4 PS(VertexOut pin, uniform bool gAlphaClip) : SV_Target
{
 // 보간된 법선은 비정규화될 수 있으므로 정규화합니다. pin.NormalV =
 normalize(pin.NormalV);

 if(gAlphaClip)
 {
 float4 texColor = gDiffuseMap.Sample(samLinear, pin.Tex); clip(texColor.a - 0.1f);
 }
 return float4(pin.NormalV, pin.PosV.z);
}
```

버텍스 셰이더는 단순히 법선 벡터와 위치를 뷰 공간으로 변환합니다. 그런 다음 뷰 공간 법선과 뷰 공간 *z* 좌표가 픽셀 셰이더에서 렌더 타깃에 기록됩니다. 우리가 부동 소수점 렌더 타깃에 기록하고 있으므로 임의의 부동 소수점 데이터를 기록하는 데 아무런 문제가 없습니다.

### 22.2.2 앰비언트 오클루전 패스

뷰 스페이스 노멀과 깊이 정보를 렌더 타깃에 기록한 후, 덤스 버퍼를 비활성화합니다(앰비언트 오클루전 텍스처 생성에 필요하지 않음). 이후 전체 화면 쿼드를 그려 각 픽셀에서 SSAO 픽셀 셰이더를 호출합니다. 픽셀 셰이더는 노멀/덤스 버퍼를 활용해 각 픽셀의 앰비언트 접근성 값을 생성합니다. 이 패스에서 생성된 텍스처 맵을 *SSAO 맵이라* 부릅니다. 정상/깊이 맵은 전체 화면 해상도(즉, 백 버퍼의 해상도)로 렌더링하지만, 성능상의 이유로 SSAO 맵에는 백 버퍼 너비와 높이의 절반 크기로 렌더링합니다. 앰비언트 오클루전은 저주파 효과이므로 절반 크기로 렌더링해도 품질에 큰 영향을 미치지 않습니다. 다음 하위 절에서는 [그림 22.5](#)를 참조하십시오.

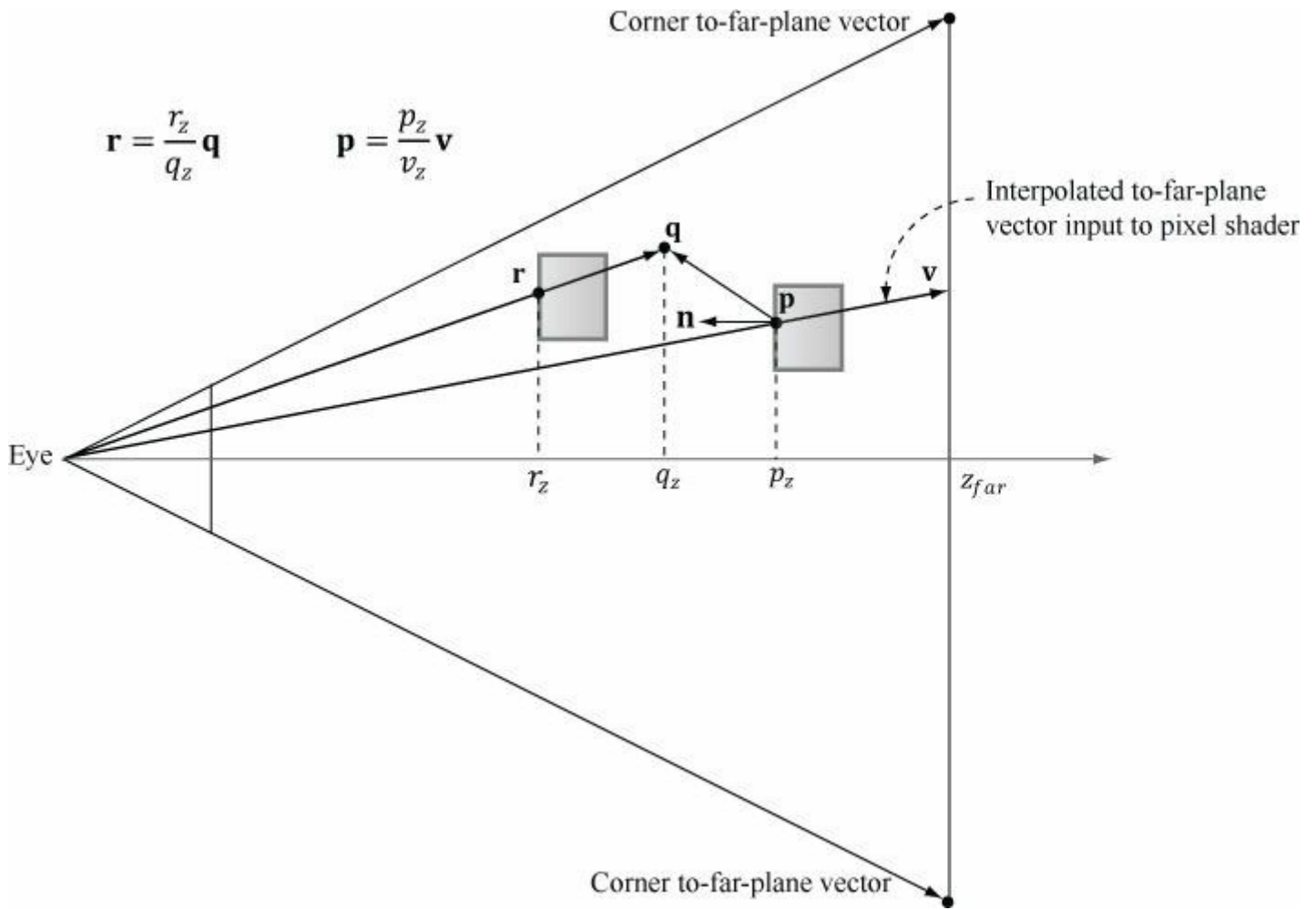


그림 22.5. SSAAO에 관여하는 점들. 점  $p$ 는 현재 처리 중인 픽셀에 해당하며, 법선/깊이 맵에 저장된 깊이 값과 보간된 원거리 평면 벡터  $v$ 로부터 재구성됩니다. 점  $q$ 는  $p$ 의 반구 내 임의의 점입니다. 점  $r$ 은 시점에서  $q$ 까지의 광선 상에서 가장 가까운 가시점에 해당합니다. 점  $r$ 은  $|p_z, r_z|$ 이 충분히 작고  $r \cdot p$ 와  $n$  사이의 각도가  $90^\circ$  미만일 때  $p$ 의 오클루전(occlusion)에 기여한다. 데모에서는 14개의 무작위 샘플 점을 취하고 각 점의 오클루전을 평균화하여 화면 공간에서 앰비언트 오클루전을 추정한다.

### 22.2.2.1 시점 공간 위치 재구성

SSAO 맵의 각 픽셀에서 SSAAO 픽셀 셰이더를 호출하기 위해 전체 화면 사각형을 그릴 때, 우리는 사각형의 네 꼭짓점 각각에 원근 투영체 원근면 모서리 배열에 대한 인덱스를 저장합니다. 우리는 버텍스 셰이더에서 원근 투영체 모서리 벡터를 가져와 픽셀 셰이더로 전달합니다. 이 원근면 방향 벡터들은 보간되어 각 픽셀에 대해 시점에서 원근면까지의 벡터  $v$ 를 제공합니다.

```
void Ssao::BuildFrustumFarCorners(float fovy, float farZ)
{
 float aspect = (float)mRenderTargetWidth / (float)mRenderTargetHeight;

 float halfHeight = farZ * tanf(0.5f*fovy); float halfWidth =
 aspect * halfHeight;

 mFrustumFarCorner[0] = XMFLOAT4(-halfWidth, -halfHeight, farZ, 0.0f); mFrustumFarCorner[1] =
 XMFLOAT4(-halfWidth, +halfHeight, farZ, 0.0f); mFrustumFarCorner[2] = XMFLOAT4(+halfWidth, +halfHeight,
 farZ, 0.0f); mFrustumFarCorner[3] = XMFLOAT4(+halfWidth, -halfHeight, farZ, 0.0f);
}

cbuffer cbPerFrame
{
 float4 gFrustumCorners[4];

 ...
};

VertexOut VS(VertexIn vin)
{

```

```
VertexOut vout;

// 이미 NDC 공간에 있음. vout.PosH =
float4(vin.PosL, 1.0f);

// 정사각형의 모서리 인덱스를 범선 x 좌표 슬롯에 저장합니다. vout.ToFarPlane =
gFrustumCorners[vin.ToFarPlaneIndex.x].xyz;

// 픽셀 셰이더로 전달합니다. vout.Tex =
vin.Tex;

return vout;
}
```

이제 각 픽셀에 대해 노멀/덱스 맵을 샘플링하여 시점에 가장 가까운 가시점의 뷰 스페이스 노멀  $\mathbf{n}$ 과  $z$ 좌표  $p_z$ 를 얻습니다. 목표는 샘플링된 뷰 스페이스  $z$ -좌표  $p_z$ 와 보간된 원근면 벡터  $\mathbf{v}$ 로부터 뷰 스페이스 위치  $\mathbf{p} = (p_x, p_y, p_z)$ 를 재구성하는 것입니다. 이 재구성은 다음과 같이 수행됩니다. 원근 평면까지의 벡터  $\mathbf{v}$ 는  $\mathbf{p}$ 를 통과하므로,  $\mathbf{p} = t\mathbf{v}$ 가 성립하는  $t$ 가 존재합니다. 특히  $p_z = tv_z$ 이므로  $t = p_z/v_z$ 입니다. 따라서  $\mathbf{p} = \frac{p_z}{v_z} \mathbf{v}$ 입니다.

### 22.2.2.2 랜덤 샘플 생성

이 단계는 반구 전체에 대한 무작위 광선 투사와 유사합니다. 우리는  $\mathbf{p}$ 의 정면 방향에 위치하며 지정된 가림 반경 내에 있는  $\mathbf{p}$  주변의  $N$ 개 점  $\mathbf{q}$ 를 무작위로 샘플링합니다. 가림 반경은  $\mathbf{p}$ 로부터 얼마나 멀리 떨어진 지점에서 무작위 샘플링 점을 취할지 제어하는 예술적 매개변수입니다.  $\mathbf{p}$ 의 정면 방향에 있는 점만 샘플링하기로 선택하는 것은 광선 투사를 반구 전체가 아닌 전체 구면이 아닌 반구에만 광선을 캐스팅하는 것과 유사합니다.

다음 질문은 무작위 샘플을 생성하는 방법입니다. 무작위 벡터를 생성하여 텍스처 맵에 저장할 수 있으며, 그런 다음 이 텍스처 맵을  $N$ 개의 서로 다른 위치에서 샘플링하여  $N$ 개의 무작위 벡터를 얻습니다. 그러나 이 벡터들이 무작위이므로 샘플링된 벡터들이 균일하게 분포될 것이라는 보장은 없습니다—모두 대략 같은 방향으로 뭉쳐질 수도 있습니다. 이를 극복하기 위해 다음과 같은 트릭을 사용합니다. 우리의 구현에서는  $N = 14$ 개의 샘플을 사용하며, C++ 코드에서 균등하게 분포된 14개의 벡터를 생성합니다:

```
void Ssao::BuildOffsetVectors()
{
 // 14개의 균일 분포 벡터로 시작합니다. 정육면체의
 // 정육면체의 8개 모서리와 각 면을 따라 위치한 6개의 중심점을 선택합니다
 // 정육면체 면을 따라 6개의 중심점을 선택합니다. 정육면체의 반대편 면에 있는 점들은 항상 번갈아 가며 선택합니다.
 // 이렇게 하면 14개 미만의 샘플을 사용하기로 선택하더라도 벡터가 여전히 넓게 퍼져 있음
 // 14개 미만의 샘플을 사용하기로 선택하더라도 벡터가 고르게 분포되도록 합니다.

 // 8개의 정육면체 모서리
 mOffsets[0] = XMVECTOR4(+1.0f, +1.0f, +1.0f, 0.0f); mOffsets[1] = XMVECTOR4(-1.0f, -1.0f, -1.0f, 0.0f);

 mOffsets[2] = XMVECTOR4(-1.0f, +1.0f, +1.0f, 0.0f); mOffsets[3] = XMVECTOR4(+1.0f, -1.0f, -1.0f, 0.0f);

 mOffsets[4] = XMVECTOR4(+1.0f, +1.0f, -1.0f, 0.0f); mOffsets[5] = XMVECTOR4(-1.0f, -1.0f, +1.0f, 0.0f);

 mOffsets[6] = XMVECTOR4(-1.0f, +1.0f, -1.0f, 0.0f); mOffsets[7] = XMVECTOR4(+1.0f, -1.0f, +1.0f, 0.0f);

 // 6개의 정육면체 면 중심점
 mOffsets[8] = XMVECTOR4(-1.0f, 0.0f, 0.0f, 0.0f); mOffsets[9] = XMVECTOR4(+1.0f, 0.0f, 0.0f, 0.0f);

 mOffsets[10] = XMVECTOR4(0.0f, -1.0f, 0.0f, 0.0f); mOffsets[11] = XMVECTOR4(0.0f, +1.0f, 0.0f, 0.0f);

 mOffsets[12] = XMVECTOR4(0.0f, 0.0f, -1.0f, 0.0f); mOffsets[13] = XMVECTOR4(0.0f, 0.0f, +1.0f, 0.0f);
}
```

```

for(int i = 0; i < 14; ++i)
{
 // [0.25, 1.0] 범위에서 임의의 길이를 생성합니다. float s =
 MathHelper::RandF(0.25f, 1.0f);

 XMVECTOR v = s * XMVector4Normalize(XMLoadFloat4(&mOffsets[i])); XMStoreFloat4(&mOffsets[i], v);
}
}

```

**Note:** 4차원 동차 벡터를 사용하는 이유는 오프셋 벡터 배열을 효과에 할당할 때 정렬 문제를 걱정하지 않아도 되기 때문입니다.

이제 픽셀 셰이더에서 무작위 벡터 텍스처 맵을 한 번 샘플링한 후, 이를 14개의 균등하게 분포된 벡터를 반영하는 데 사용합니다. 이로 인해 14개의 *균등하게 분포된 무작위 벡터*가 생성됩니다.

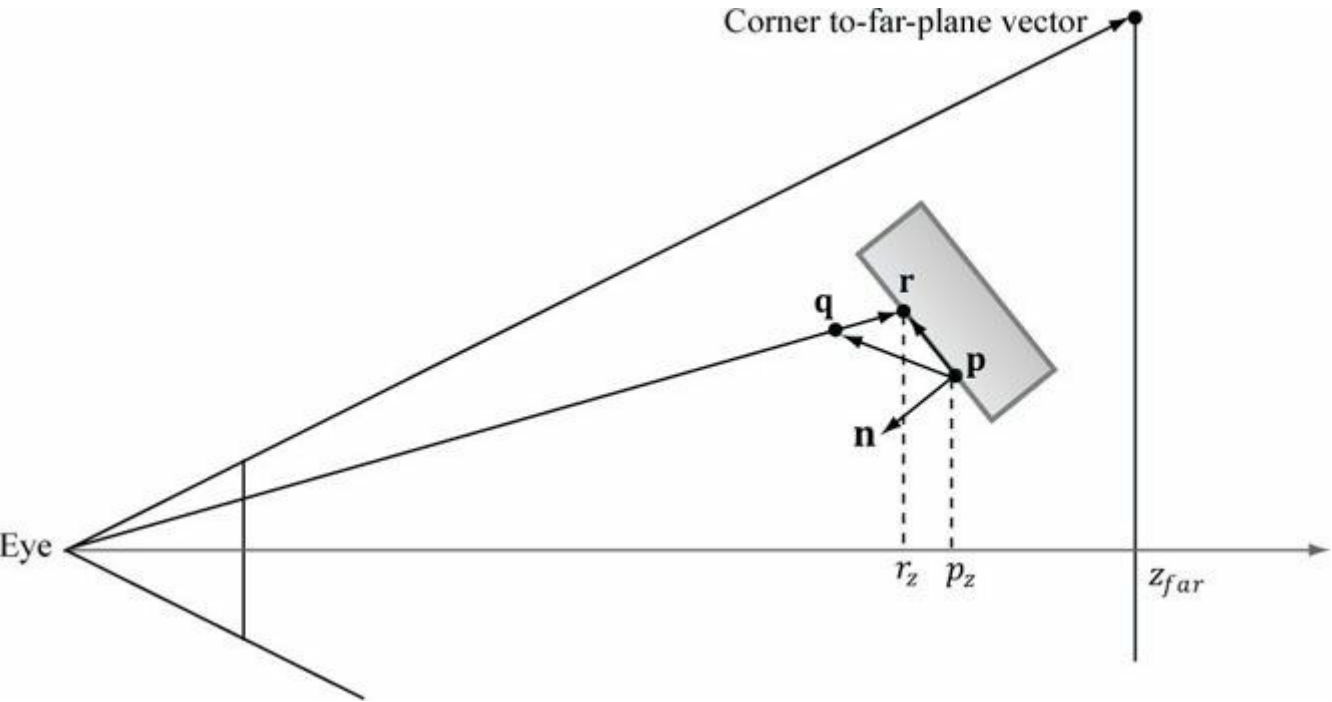
### 22.2.2.3 잠재적 가림 지점 생성

이제 **p**를 둘러싼 임의의 샘플 점 **q**들을 확보했습니다. 그러나 이 점들이 공허한 공간을 차지하는지, 아니면 고체 물체를 차지하는지에 대해서는 아무것도 알지 못합니다. 따라서 이 점들을 **p**를 가리는지 테스트하는 데 사용할 수 없습니다. 잠재적인 가림 지점을 찾으려면 노멀/덱스 맵으로부터의 깊이 정보가 필요합니다. 따라서 우리가 하는 일은 카메라를 기준으로 각 **q**에 대해 투영 텍스처 좌표를 생성하고, 이를 이용해 노멀/덱스 맵을 샘플링하여 눈에서 **q**까지의 광선 상에서 가장 가까운 가시 픽셀의 깊이  $r_z$ 를 얻는 것입니다.  $z$ 좌표  $r_z$ 를 알면, §22.2.2.1에서 했던 것과 유사한 방식으로 전체 3D 뷰 공간 위치 **r**을 재구성할 수 있습니다. 눈에서 **q**까지의 벡터를 **r**을 통과하므로  $\mathbf{r} = t\mathbf{q}$ 를 만족하는  $t$ 가 존재합니다. 특히  $r_z = tq_z$ 이므로  $t = r_z/q_z$ 입니다. 따라서  $\mathbf{r} = \frac{r_z}{q_z} \mathbf{q}$ 입니다. 각 무작위 샘플 점 **q**마다 생성된 점 **r**은 잠재적 가림점입니다.

### 22.2.2.4 가림 테스트 수행

잠재적 가림점 **r**을 확보했으므로, 이제 가림 테스트를 수행하여 **p**를 가리는지 추정할 수 있습니다. 이 테스트는 두 가지 양에 의존합니다:

1. 시점 공간 깊이 거리  $|p_z - r_z|$ . 거리가 증가함에 따라 가림 효과를 선형적으로 축소합니다. 왜냐하면 멀리 있는 점일수록 가림 효과가 약해지기 때문입니다. 거리가 특정 최대 거리를 초과하면 가림이 발생하지 않습니다. 또한 거리가 매우 작을 경우 **p**와 **q**가 동일한 평면에 있다고 가정하므로 **q**가 **p**를 가릴 수 없습니다.
2.  $\mathbf{n}$ 과  $\mathbf{r} - \mathbf{p}$  사이의 각도  $\max\left(\mathbf{n} \cdot \left(\frac{\mathbf{r} - \mathbf{p}}{\|\mathbf{r} - \mathbf{p}\|}\right), 0\right)$ 로 측정). 이는 자기 교차(self-intersection)를 방지하기 위함입니다([그림 22.6](#) 참조).



**그림 22.6.**  $\mathbf{r}$ 이 **p**와 동일한 평면에 위치할 경우, 거리  $|p_z - r_z|$ 이 충분히 작아  $\mathbf{r}$ 이 **p**를 가리는 첫 번째 조건을 통과할 수 있습니다. 그러나 그림에서 보듯이 이는 잘못된 것으로,  $\mathbf{r}$ 과 **p**가 동일한 평면에 있으므로  $\mathbf{r}$ 이 **p**를 가리지 않습니다. 가림 현상을  $\max\left(\mathbf{n} \cdot \left(\frac{\mathbf{r} - \mathbf{p}}{\|\mathbf{r} - \mathbf{p}\|}\right), 0\right)$  이러한 상황을 방지합니다.

상황을 방지합니다.

### 22.2.2.5 계산 완료

각 샘플의 오클루전을 합산한 후, 샘플 수로 나누어 평균 오클루전을 계산합니다. 그런 다음 앰비언트 액세스를 계산하고, 마지막으로 앰비언트 액세스를 제공하여 대비를 높입니다. 앰비언트 맵의 밝기를 높이기 위해 강도를 증가시키는 값을 추가할 수도 있습니다. 다양한 대비/밝기 값으로 실험해 볼 수 있습니다.

```
occlusionSum /= gSampleCount; float access = 1.0f -
occlusionSum;

// SSAO 맵의 대비를 선명하게 하여 SSAO 효과가
// 효과를 더욱 극적으로 만듭니다.
return saturate(pow(access, 4.0f));
```

### 22.2.2.6 구현

이전 섹션에서는 SSAO 맵 생성을 위한 핵심 요소를 설명했습니다. 다음은 전체 효과 코드입니다.

```
//=====
// Ssao.fx by Frank Luna (C) 2011 All Rights Reserved.
//
// SSAO 맵을 계산합니다.
//=====

cbuffer cbPerFrame
{
 float4x4 gViewToTexSpace; // Proj*Texture float4 gOffsetVectors[14];
 float4 gFrustumCorners[4];

 // 뷰 공간 좌표로 주어진. float gOcclusionRadius =
 0.5f; float gOcclusionFadeStart = 0.2f; float
 gOcclusionFadeEnd = 2.0f; float gSurfaceEpsilon =
 0.05f;
};

// 비수치형 값은 cbuffer에 추가할 수 없습니다. Texture2D gNormalDepthMap;
Texture2D gRandomVecMap;

SamplerState samNormalDepth
{
 Filter = MIN_MAG_LINEAR_MIP_POINT;

 // 샘플링이 NormalDepth 맵 외부에서 이루어질 경우 매우 먼 깊이 값을 설정하여
 // AddressU = BORDER;
 AddressV = BORDER;
 BorderColor = float4(0.0f, 0.0f, 0.0f, 1e5f);
};

SamplerState samRandomVec
{
 Filter = MIN_MAG_LINEAR_MIP_POINT;
 AddressU = WRAP;
 AddressV = WRAP;
};

struct VertexIn
{
 float3 PosL : POSITION;
```

```
float3 ToFarPlaneIndex : NORMAL; float2 Tex :
 텍스처 좌표;

};

struct VertexOut
{
 float4 PosH : SV_POSITION; float3
 ToFarPlane : TEXCOORD0; float2 Tex :
 TEXCOORD1;
};

VertexOut VS(VertexIn vin)
{
 VertexOut vout;

 // 이미 NDC 공간에 있음. vout.PosH =
 float4(vin.PosL, 1.0f);

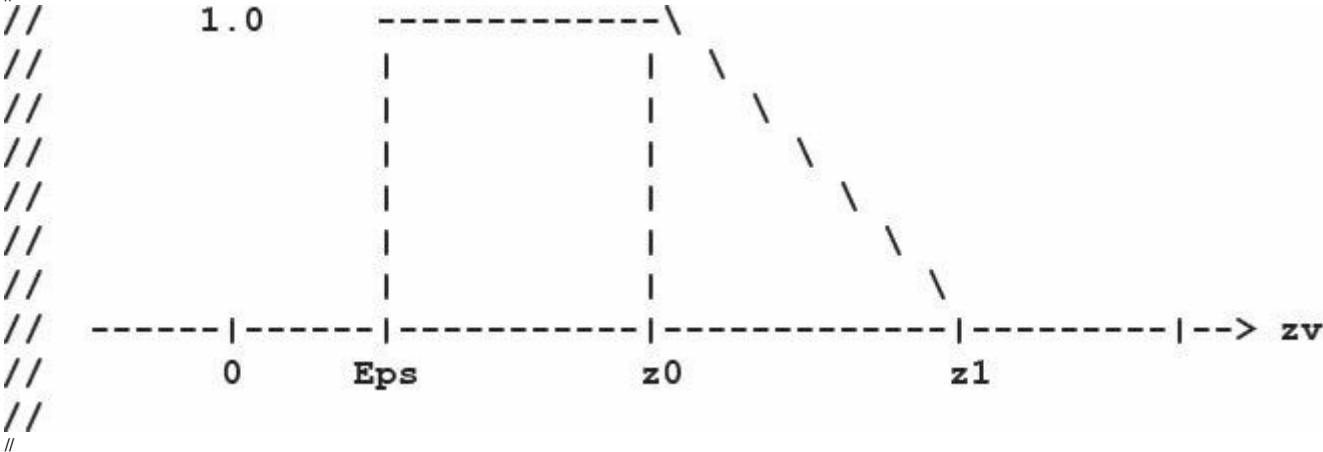
 // 원근 투영체 모서리 인덱스를 법선 x 좌표 슬롯에 저장합니다. vout.ToFarPlane =
 gFrustumCorners[vin.ToFarPlaneIndex.x].xyz;

 // 픽셀 셰이더로 전달합니다. vout.Tex =
 vin.Tex;

 return vout;
}
```

// 샘플 점 q가 점 p를 얼마나 가리는지 distZ 함수로 결정

```
//
float OcclusionFunction(float distZ)
{
 //
 // depth(q)가 depth(p)보다 "뒤에" 있으면 q는 p를 가릴 수 없음.
 // 또한 depth(q)와 depth(p)가 충분히 가까울 경우,
 // q가 p를 가릴 수 없다고 가정합니다. q는
 // p 앞에 epsilon만큼 위치하여 p를 가려야 한다.
 //
 // 가림 여부를 판단하기 위해 다음 함수를 사용합니다.
 //
 //
```



```
float occlusion = 0.0f; if(distZ >
gSurfaceEpsilon)
{
 float fadeLength = gOcclusionFadeEnd - gOcclusionFadeStart;

 // distZ가 gOcclusionFadeStart에서 gOcclusionFadeEnd로 갈수록
 // gOcclusionFadeStart에서 gOcclusionFadeEnd까지.
 occlusion = saturate((gOcclusionFadeEnd-distZ)/fadeLength);
}
```

```

 }

 return occlusion;
}

float4 PS(VertexOut pin, uniform int gSampleCount) : SV_Target
{
 // p -- 앰비언트 오클루전을 계산 중인 점.
 // n -- p 지점의 법선 벡터.
 // q -- p에서 무작위로 선택된 오프셋.
 // r -- p를 가릴 수 있는 잠재적 가림체.

 // 이 픽셀의 뷰 스페이스 법선과 z 좌표를 가져옵니다.
 // 우리가 그린 전체 화면 쿼드의 텍스처 좌표는 이미 uv 공간에 있습니다. float4
 normalDepth = gNormalDepthMap.SampleLevel(
 samNormalDepth, pin.Tex, 0.0f);

 float3 n = normalDepth.xyz; float pz =
 normalDepth.w;

 //
 // 전체 뷰 스페이스 위치 (x,y,z) 재구성.
 // p = t*pin.ToFarPlane. 조건을 만족하는 t 찾기.
 // p.z = t*pin.ToFarPlane.z
 // t = p.z / pin.ToFarPlane.z
 //
 float3 p = (pz/pin.ToFarPlane.z)*pin.ToFarPlane;

 // 무작위 벡터 추출 및 [0,1]에서 [-1, +1]로 매핑. float3 randVec =
 2.0f*gRandomVecMap.SampleLevel(
 samRandomVec, 4.0f*pin.Tex, 0.0f).rgb - 1.0f; float occlusionSum = 0.0f;

 // 반구 내 p 주변 인접 점 샘플링
 // n 방향으로 지향됨.
 [unroll]
 for(int i = 0; i < gSampleCount; ++i)
 {
 // 오프셋 벡터가 고정되고 균일하게
 // 분포되어 있는지 확인 (오프셋 벡터들이
 // 동일한 방향으로 뭉치지 않도록). 만약 우리가
 // 무작위 벡터에 대해 반사하면
 // 오프셋 벡터의 무작위 균일 분포를 얻을 수 있습니다. float3 offset =
 reflect(gOffsetVectors[i].xyz, randVec);

 // 평면 뒤에 있는 경우 오프셋 벡터를 뒤집기
 // (p, n)으로 정의된 평면 뒤에 위치할 경우.
 float flip = sign(dot(offset, n));

 // 가림 반경 내에서 p 근처의 점을 샘플링합니다. float3 q = p + flip *
 gOcclusionRadius * offset;

 // q를 투영하고 투영 텍스처 좌표를 생성합니다.
 float4 projQ = mul(float4(q, 1.0f), gViewToTexSpace); projQ /= projQ.w;

 // 시선 방향으로 가장 가까운 깊이 값을 찾음
 // (이는 q의 깊이가 아님, q는
 // p 근처 임의의 점이며,
 // 빈 공간). 가장 가까운 깊이를 찾기 위해
 // 깊이맵에서 찾아봅니다.

```

```
float rz = gNormalDepthMap.SampleLevel(samNormalDepth, projQ.xy, 0.0f).a;
```

```
// 전체 뷰 공간 위치 r = (rx, ry, rz)를 재구성합니다.
```

```
// r이 q의 광선상에 있음을 알므로, t가 존재하여
```

```
// r = t*q를 만족하는 t가 존재합니다.
```

```
// r.z = t*q.z ==> t = r.z / q.z
```

```
float3 r = (rz / q.z) * q;
```

```
//
```

```
// r이 p를 가리는지 테스트합니다.
```

```
// * 곱積 dot(n, normalize(r - p))는
```

```
// 평면(p,n) 앞에서 오를루더 점이
```

```
// 더 앞쪽에 위치할수록 더 많은 가림 현상이 발생합니다.
```

```
// 가중치를 부여합니다. 이는 또한 자체 그림자를 방지합니다
```

```
// 예를 들어 각이진 평면 (p,n) 위의 점 r이
```

```
// 서로 다른 깊이 값을 가지므로
```

```
// 따라서 시점에 대해 서로 다른 깊이 값을 가집니다.
```

```
// * 오를루전의 가중치는 어떻게
```

```
// 차단체가 계산 중인 지점에서 얼마나 멀리 떨어져 있는지에 따라 조정됩니다.
```

```
// 오를루더 r이 멀리 떨어져 있을수록
```

```
// 점 p로부터 멀리 떨어져 있다면, 이를 가리지 않습니다.
```

```
//
```

```
float distZ = p.z - r.z;
```

```
float dp = max(dot(n, normalize(r - p)), 0.0f); float occlusion = dp *
```

```
OcclusionFunction(distZ);
```

```
occlusionSum += occlusion;
```

```
}
```

```
occlusionSum /= gSampleCount; float access = 1.0f
```

```
- occlusionSum;
```

```
// SSAO 맵의 대비를 선명하게 하여 SSAO 효과가
```

```
// 효과를 더욱 극적으로 만들기 위해 SSAO의 대비를 선명하게 합니다.
```

```
return saturate(pow(access, 4.0f));
```

```
}
```

```
technique11 Ssao
```

```
{
```

```
 패스 P0
```

```
 {
```

```
 SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
```

```
 SetPixelShader(CompileShader(ps_5_0, PS(14)));
```

```
 }
```

```
}
```

## 22.2.3 블러 패스

[그림 22.7](#)은 현재 우리의 앰비언트 오를루전 맵이 어떻게 보이는지 보여줍니다. 노이즈는 우리가 단지 몇 개의 무작위 샘플만 채취했기 때문입니다. 노이즈를 숨길 만큼 충분한 샘플을 채취하는 것은 실시간으로 실현하기 어렵습니다. 일반적인 해결책은 SSAO 맵에 에지 보존 블러(즉, 양면 블러)를 적용하여 매끄럽게 만드는 것입니다. 에지 보존 블러를 사용하지 않으면 날카로운 불연속성이 부드러워지면서 장면의 선명도가 손실됩니다. 에지 보존 블러는 [12장에서](#) 구현한 블러와 유사하지만, 에지(법선/심도 맵에서 탐지됨)를 가로지르지 않도록 조건문을 추가합니다:

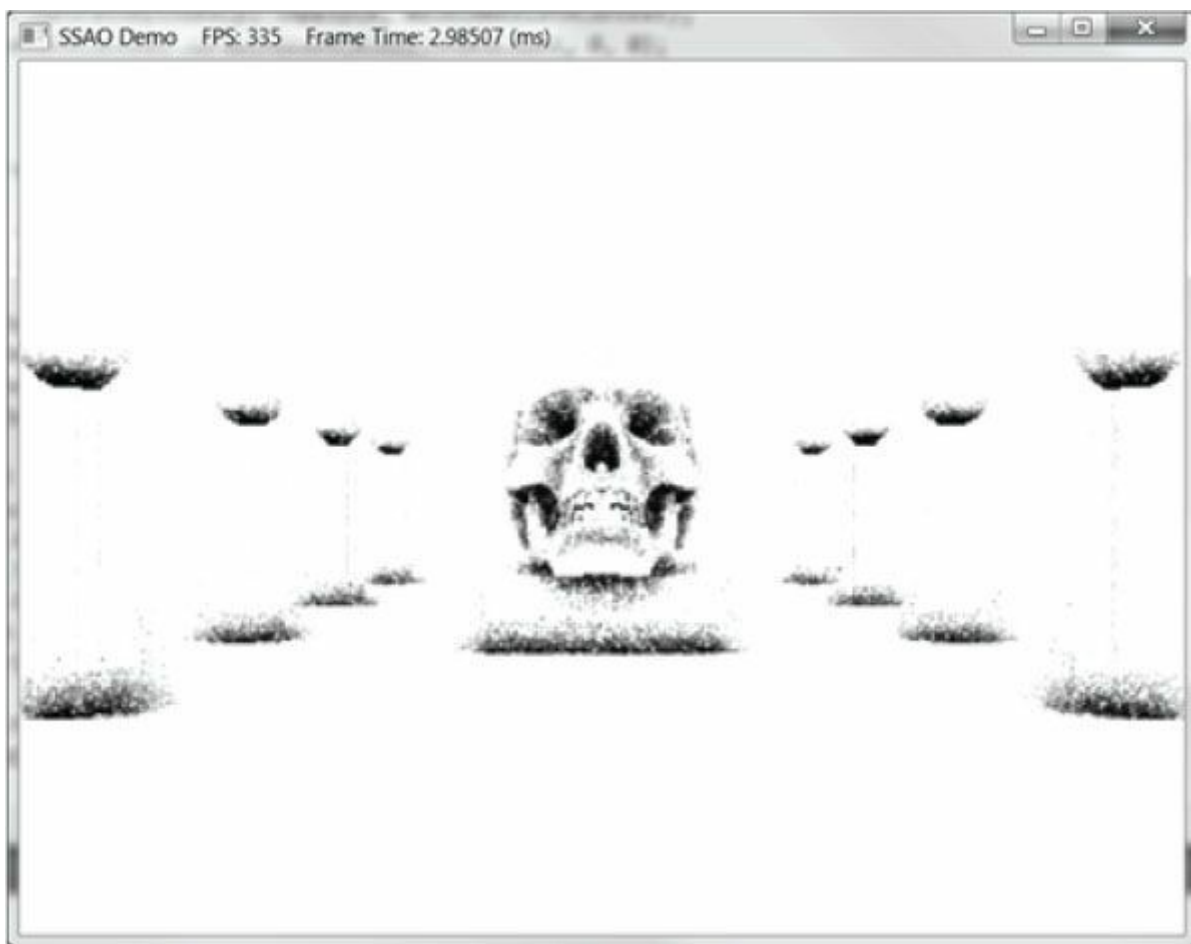


그림 22.7. 무작위 샘플링 횟수가 적어 SSAO에 노이즈가 발생하는 모습.

```
//=====
// SsaoBlur.fx by Frank Luna (C) 2011 All Rights Reserved.
//
// 앰비언트 맵에 대해 양방향 에지 보존 블러를 수행합니다. 컴퓨터 셰이더 대신
// 컴퓨터 셰이더 대신 픽셀 셰이더를 사용하여
// 컴퓨터 모드에서 렌더링 모드로 전환하는 것을 피하기 위해 컴퓨터 셰이더 대신 픽셀 셰이더를 사용합니다. 텍스처 캐시는 공유 메모리 부재로 인한 일부 손실을 보완합니다. 앰비언
// 공유 메모리 부재로 인한 손실 일부를 보완합니다. 앰비언트 맵은
// 16비트 텍스처 형식으로 크기가 작으므로
// 캐시에 많은 텍셀을 담을 수 있을 것입니다.
//=====

cbuffer cbPerFrame
{
 float gTexelWidth; float
 gTexelHeight;
};

cbuffer cbSettings
{
 float gWeights[11] =
 {
 0.05f, 0.05f, 0.1f, 0.1f, 0.1f, 0.2f, 0.1f, 0.1f, 0.1f, 0.05f, 0.05f
 };
};

cbuffer cbFixed
{
 static const int gBlurRadius = 5;
};

// cbuffer에는 숫자가 아닌 값을 추가할 수 없습니다. Texture2D
gNormalDepthMap;
```

```
Texture2D gInputImage;
```

```
SamplerState samNormalDepth
```

```
{
 Filter = MIN_MAG_LINEAR_MIP_POINT;

 AddressU = CLAMP; AddressV = CLAMP;
};
```

```
샘플러 상태 samInputImage
```

```
{
 필터 = MIN_MAG_LINEAR_MIP_POINT;

 AddressU = CLAMP; AddressV = CLAMP;
};
```

```
struct VertexIn
```

```
{
 float3 PosL : POSITION; float3 NormalL :
 NORMAL; float2 Tex : TEXCOORD;
};
```

```
struct VertexOut
```

```
{
 float4 PosH : SV_POSITION; float2 Tex :
 TEXCOORD;
};
```

```
VertexOut VS(VertexIn vin)
```

```
{
 VertexOut vout;

 // 이미 NDC 공간에 있음. vout.PosH =
 float4(vin.PosL, 1.0f);

 // 픽셀 셰이더로 전달. vout.Tex = vin.Tex;

 return vout;
}
```

```
float4 PS(VertexOut pin, uniform bool gHorizontalBlur) : SV_Target
```

```
{
 float2 texOffset; if(gHorizontalBlur)
 {
 텍스처 오프셋 = float2(g텍셀 너비, 0.0f);
 }
 else
 {
 texOffset = float2(0.0f, gTexelHeight);
 }

 // 중심 값은 항상 합계에 기여합니다. float4 color =
 gWeights[5]*gInputImage.SampleLevel(
 samInputImage, pin.Tex, 0.0); float totalWeight
 = gWeights[5];
 float4 centerNormalDepth = gNormalDepthMap.SampleLevel(samNormalDepth, pin.Tex, 0.0f);
 for(float i = -gBlurRadius; i <=gBlurRadius; ++i)
 {
```

```

// 이미 중심 가중치를 추가했습니다. if(i == 0)
계속;

float2 tex = pin.Tex + i*texOffset;

float4 neighborNormalDepth = gNormalDepthMap.SampleLevel(samNormalDepth, tex, 0.0f);

//
// 중심값과 이웃값의 차이가 너무 크다면
// (법선 또는 깊이 중 하나에서) 차이가 너무 크다면,
// 불연속성을 가로지르는 샘플링이라고 가정합니다.
// 이러한 샘플은 블러 처리에서 제외합니다.
//

if(dot(neighborNormalDepth.xyz, centerNormalDepth.xyz) >= 0.8f && abs(neighborNormalDepth.a - centerNormalDepth.a) <=
0.2f)
{
 float weight = gWeights[i+gBlurRadius];

 // 이웃 픽셀을 블러에 추가합니다.
 color += weight*gInputImage.SampleLevel(samInputImage, tex, 0.0);

 totalWeight += weight;
}
}

// 총 가중치 합이 1이 되도록 버려진 샘플을 보정합니다. return color / totalWeight;
}

```

technique11 수평 블러

```

{
 pass P0
 {
 SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
 SetPixelShader(CompileShader(ps_5_0, PS(true)));
 }
}

```

technique11 VertBlur

```

{
 P0 통과
 {
 SetVertexShader(CompileShader(vs_5_0, VS())); SetGeometryShader(NULL);
 SetPixelShader(CompileShader(ps_5_0, PS(false)));
 }
}

```

그림 22.8은 에지 보존 블러 처리 후의 앰비언트 맵을 보여줍니다.

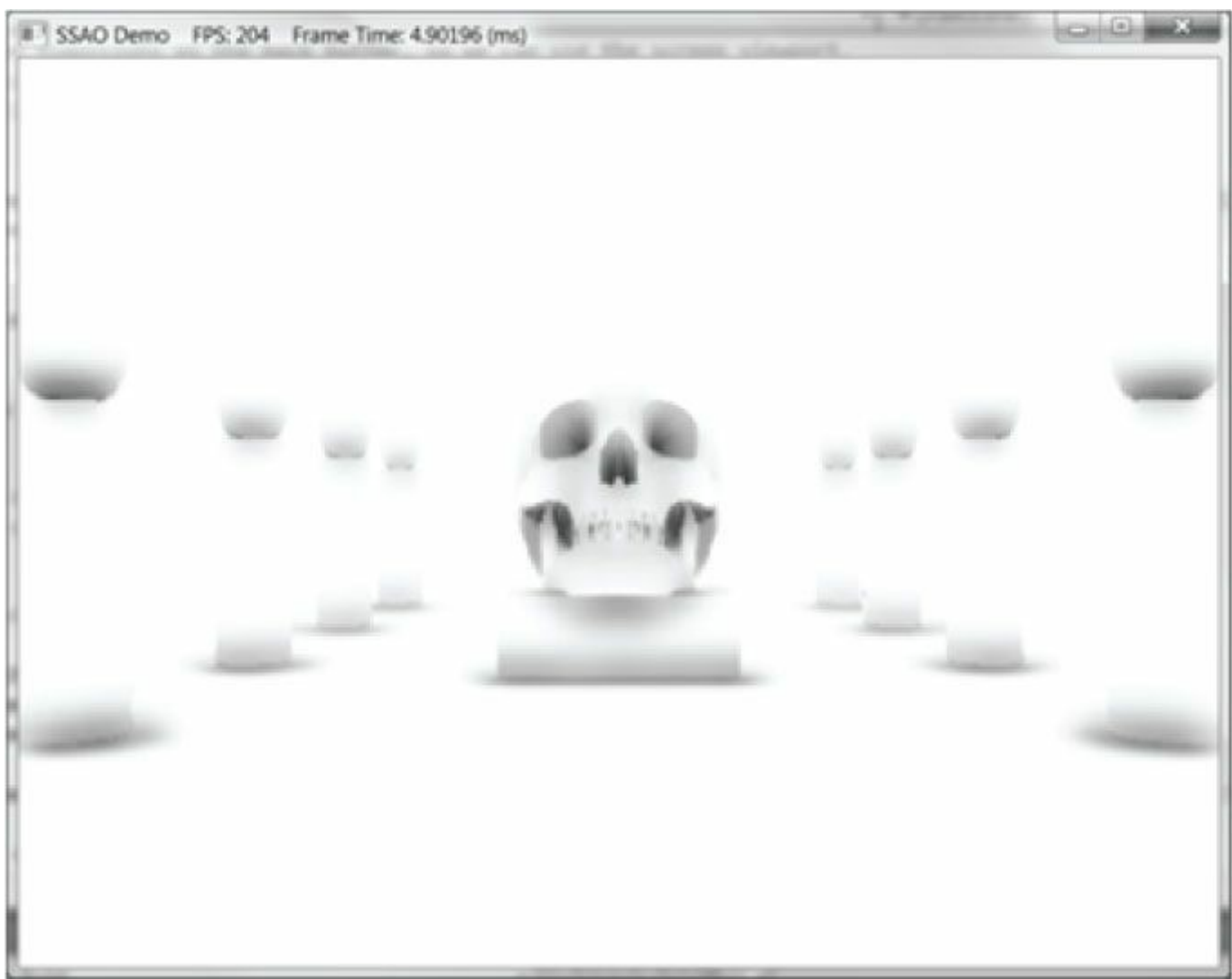


그림 22.8. 에지 보존 블러는 노이즈를 부드럽게 합니다. 데모에서는 이미지를 4회 블러 처리합니다.

## 22.2.4 앰비언트 오클루전 맵 사용

지금까지 훌륭한 앰비언트 오클루전 맵을 구축했습니다. 마지막 단계는 이를 장면에 적용하는 것입니다. 알파 블렌딩을 사용하여 백 버퍼와 앰비언트 맵을 번조하는 방법을 생각할 수 있습니다. 그러나 이렇게 하면 앰비언트 맵이 앰비언트 항뿐만 아니라 조명 방정식의 확산 및 반사 항까지 수정하게 되어 잘못된 결과를 초래합니다. 대신 장면을 백 버퍼에 렌더링할 때 앰비언트 맵을 셰이더 입력으로 바인딩합니다. 그런 다음 카메라를 기준으로 한 투영 텍스처 좌표를 생성하고, SSAO 맵을 샘플링하여 조명 방정식의 앰비언트 항에만 적용합니다:

```
// 버텍스 셰이더에서 투영 텍스처 좌표 생성

// SSAO 맵을 장면에 투영하기 위해.
vout.SsaoPosH = mul(float4(vin.PosL, 1.0f), gWorldViewProjTex);

// 픽셀 셰이더에서 텍스처 투영을 완료하고 SSAO 맵을 샘플링합니다. pin.SsaoPosH /= pin.SsaoPosH.w;

float ambientAccess = gSsaoMap.Sample(samLinear, pin.SsaoPosH.xy, 0.0f).r;

// 조명 방정식의 환경광 항을 스케일링합니다. ambient +=
ambientAccess*A;
diffuse += shadow[i]*D; spec
+= shadow[i]*S;
```

그림 22.9는 SSAO 맵이 적용된 장면을 보여줍니다. SSAO는 미묘할 수 있으며, 장면이 충분한 주변광을 반사해야 ambient-access로 스케일링했을 때 눈에 띄는 차이를 만들 수 있습니다. 원한다면 확산 성분에도 ambient-access를 곱하여 효과를 더 극적으로 만들 수 있지만, 이는 정확도가 떨어집니다.

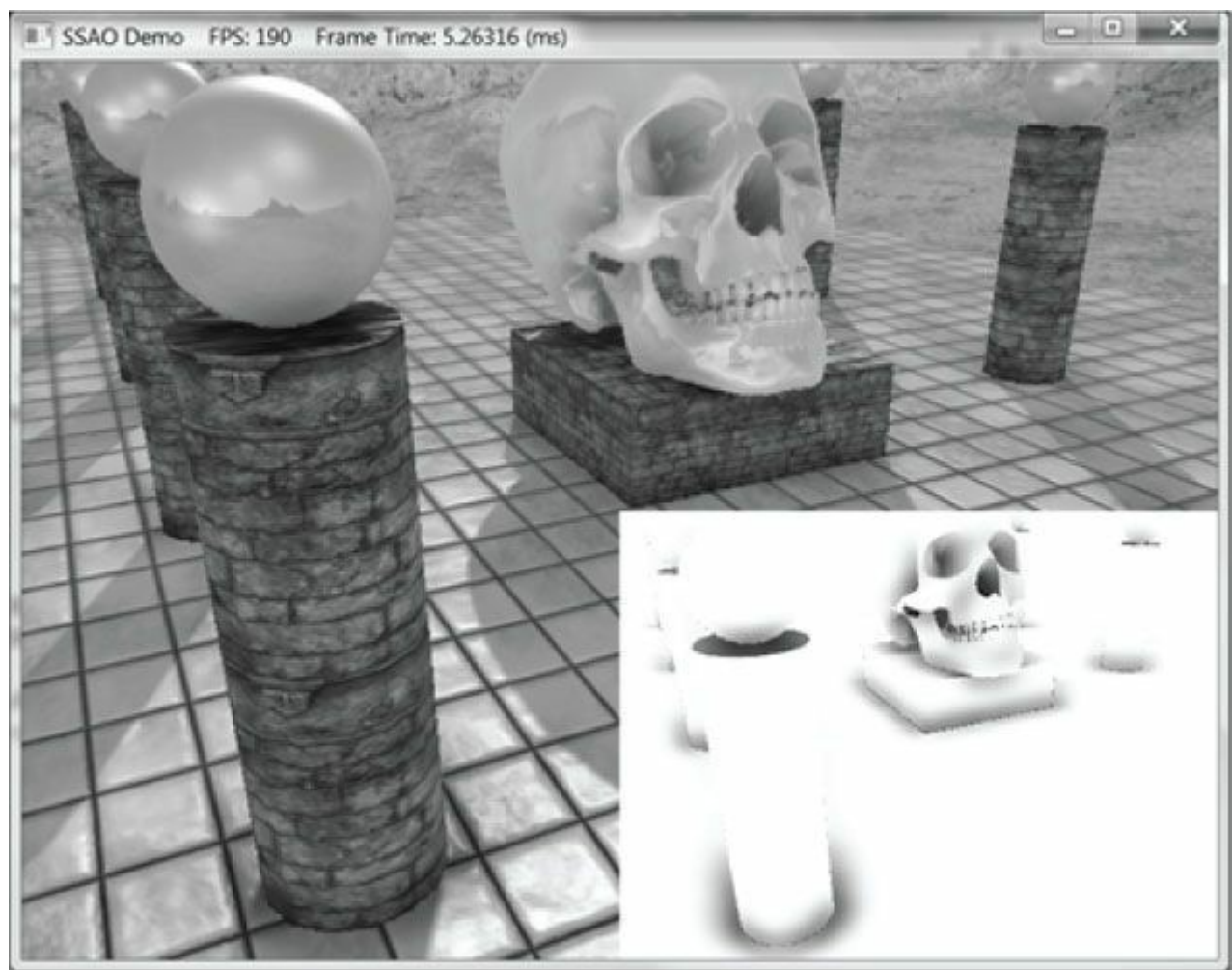


그림 22.9. 데모 스크린샷. 효과는 주변광 향에만 영향을 주기 때문에 미묘하지만, 기둥과 상자 바닥, 구체 아래, 두개골 주변이 어두워진 것을 확인할 수 있습니다.

SSAO의 장점은 물체가 그림자에 있을 때 가장 두드러집니다. 그림자 속 물체는 확산 및 반사 성분이 사라지므로 주변광 성분만 나타납니다. SSAO가 없으면 그림자 속 물체가 일정한 주변광으로 평평하게 비춰지지만, SSAO를 적용하면 입체감을 유지합니다.

장면을 노멀/딥스 렌더 타겟에 렌더링할 때, 해당 장면의 딥스 버퍼도 함께 구축됩니다. 따라서 SSAO 맵과 함께 장면을 두 번째로 렌더링할 때, 깊이 비교 테스트를 "EQUALS"로 수정합니다. 이는 가장 가까운 가시 픽셀만 이 깊이 비교 테스트를 통과하므로 두 번째 렌더링 패스에서 오버드로우를 방지합니다. 또한, 두 번째 렌더링 패스는 깊이 버퍼에 쓰기 작업을 수행할 필요가 없습니다. 왜냐하면 우리는 이미 노멀/깊이 렌더 타겟 패스에서 장면을 깊이 버퍼에 기록했기 때문입니다.

```
D3D11_DEPTH_STENCIL_DESC equalsDesc; ZeroMemory(&equalsDesc,
sizeof(D3D11_DEPTH_STENCIL_DESC)); equalsDesc.DepthEnable = true;
equalsDesc.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ZERO; equalsDesc.DepthFunc = D3D11_COMPARISON_EQUAL;
```

```
HR(device->CreateDepthStencilState(&equalsDesc, &EqualsDSS));
```

```
...
```

```
//
```

```
// 뷰 스페이스 노멀과 깊이를 렌더링합니다. 이 렌더 타겟은
```

```
// 백 버퍼와 동일한 크기를 가지므로 화면 뷰포트를 사용할 수 있습니다.
```

```
뷰포트를 사용할 수 있습니다.
```

```
// 이 렌더 패스는 앰비언트 오클루전(암부 음영) 계산을 위해 필요합니다.
```

```
// 이 패스에서는 메인 깊이/스텐실 버퍼를 사용한다는 점에 유의하십시오.
```

```
//
```

```
md3dImmediateContext->ClearDepthStencilView(mDepthStencilView, D3D11_CLEAR_DEPTH|D3D11_CLEAR_STENCIL, 1.0f, 0);
```

```
md3dImmediateContext->RSSetViewports(1, &mScreenViewport);
```

```
mSsao->SetNormalDepthRenderTarget(mDepthStencilView); DrawSceneToSsaoNormalDepthMap();
```

```
//
```

```
// 이제 앰비언트 오클루전을 계산합니다.
```

```
//
```

```
mSsao->ComputeSsao(mCam); mSsao->BlurAmbientMap(6);
```

```
//
```

```
// 백 버퍼, 깊이 버퍼 및 뷰포트를 OM 스테이지로 복원합니다.
```

```
//
```

```
ID3D11RenderTargetView* renderTargets[1] = {mRenderTargetView}; md3dImmediateContext->OMSetRenderTargets(1, renderTargets, mDepthStencilView); md3dImmediateContext->RSSetViewports(1, &mScreenViewport);
```

```
md3dImmediateContext->ClearRenderTargetView(mRenderTargetView, reinterpret_cast<const float*>(&Colors::Silver));
```

```
// 우리는 이미
```

```
// 따라서 깊이 비교를 설정할 수 있습니다.
```

```
// "EQUALS"로 테스트합니다. 이는 렌더링 패스에서 오버드로우를 방지합니다.
```

```
// 패스에서 오버드로우를 방지합니다. 가장 가까운 가시 픽셀만이 이 깊이
```

```
// 비교 테스트를 통과합니다.
```

```
md3dImmediateContext->OMSetDepthStencilState(RenderStates::EqualsDSS, 0);
```

```
// ...썸을 백 버퍼에 렌더링
```



*Note:* 이 데모에서는 테셀레이션되지 않은 지오메트리로부터 SSAO를 계산했습니다. 이 단순화는 장면의 범위량에 따라 작동할 수 있습니다. 테셀레이션된 지오메트리를 사용하여 SSAO 맵을 계산하려면,

그런 다음 해당 지오메트리를 노멀/덱스 렌더 타겟에 렌더링할 때 테셀레이션해야 합니다.

## 22.3 요약

1. 조명 방정식의 환경광 항은 간접광을 모델링합니다. 우리의 조명 모델에서 환경광 항은 단순히 상수 값입니다. 따라서 물체가 그림자에 있고 표면에 환경광만 적용될 때, 모델은 입체감이 전혀 없는 평평한 모습으로 보입니다. 앰비언트 오클루전의 목표는 환경광 항에 대한 더 나은 추정값을 찾아, 환경광 항만 적용된 상태에서도 물체가 여전히 3차원적으로 보이도록 하는 것입니다.
2. 앰비언트 오클루전(Ambient Occlusion)의 개념은 표면상의 점  $p$ 가 받는 간접광의 양이  $p$ 를 중심으로 한 반구 영역 내 입사광에 대한 가림 정도에 비례한다는 것이다. 점  $p$ 의 가림 정도를 추정하는 한 가지 방법은 광선 투사(Ray Casting)를 이용하는 것이다.  $p$ 를 중심으로 한 반구 영역에 무작위로 광선을 투사하고, 메쉬와의 교차 여부를 확인합니다. 광선이 어떤 기하학적 구조물과도 교차하지 않으면 해당 점은 완전히 가려지지 않은 상태입니다. 그러나 교차점이 많을수록  $p$ 는 더 많이 가려진 상태라고 볼 수 있습니다.
3. 레이 캐스팅 기반 앰비언트 오클루전은 동적 오브젝트에 대해 실시간으로 수행하기에는 비용이 너무 높습니다. 스크린 스페이스 앰비언트 오클루전(SSAO)은 뷰 스페이스의 법선/깊이 값을 기반으로 한 실시간 근사법입니다. 분명히 결함이나 잘못된 결과를 내는 상황을 발견할 수 있지만, 제한된 정보로 작업해야 하는 현실에서는 결과가 매우 우수합니다.

## 22.4 연습문제

1. 웹, KD-트리, 쿼드트리, 옥트리 연구.
2. "SSAO" 데모를 수정하여 예지 보존 블러 대신 가우시안 블러를 적용하세요. 어느 쪽이 더 마음에 드나요?
3. SSAO를 컴퓨터 셰이더에서 구현할 수 있나요? 가능하다면 구현 방안을 개략적으로 제시하세요.
4. [그림 22.10](#)은 자기 교차 검사(§22.2.4)를 포함하지 않을 경우 SSAO 맵에 발생하는 현상을 보여줍니다. "SSAO" 데모를 수정하여 자기 교차 검사를 제거하고 [그림 22.10](#)의 결과를 재현하세요.

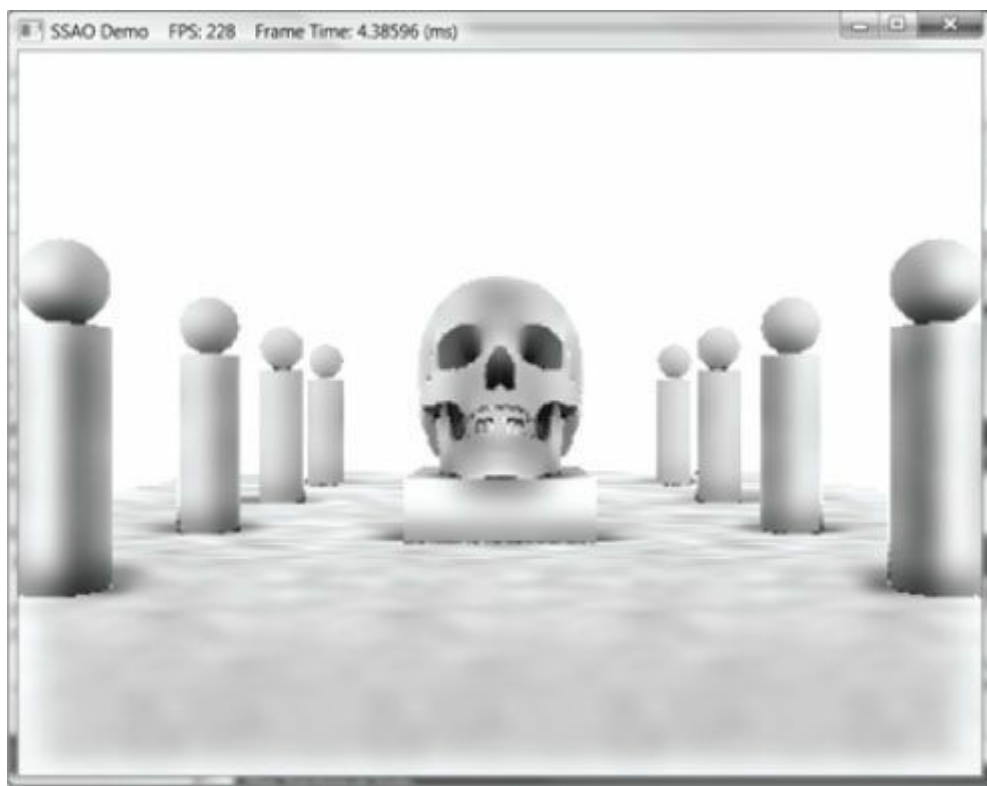


그림 22.10. 모든 곳에 나타나는 잘못된 그림 현상.

5. 노멀/덱스 패스에는 `DXGI_FORMAT_R16G16B16A16_FLOAT` 텍스처 맵을 사용했습니다. 여기서 RGB는 뷰 스페이스 노멀을 저장하고 알파 채널은 뷰 스페이스 깊이( $z$ -좌표)를 저장합니다. 이는 `DXGI_FORMAT_R8G8B8A8_UNORM`과 같은 32비트 픽셀 형식 대비 픽셀당 64비트입니다. 64비트 형식은 텍스처 쓰기 및 읽기 시 두 배의 대역폭을 소모합니다(또한 앰비언트 맵 블러링에는 깊이/법선 맵 읽기가 필요함을 유의하십시오). "SSAO" 데모를 수정하여 32비트 픽셀 형식 `DXGI_FORMAT_R8G8B8A8_UNORM`을 사용하도록 합니다. 여기서 RG는 노멀 벡터의  $x$ - 및  $y$ -좌표(일반 맵과 마찬가지로  $[0, 1]$ 로 시프트 및 바이어스 적용됨)를 저장하고, BA는 결합되어 16비트 깊이 값을 저장합니다. 노멀의  $z$ 좌표는 셰이더에서 다음과 같이 구성할 수 있습니다

$n_z = -\sqrt{1 - x^2 - y^2}$ . 노멀은 길이 1이며, 뷰 공간에서 정면을 향한 가시적 오브젝트의 노멀은  $z$ 좌표가 음수임을 알기 때문입니다. 뷰 공간의 깊이를 두 개의 8비트 UNORM 채널에 저장하려면,  $z$ 를 먼저  $[0, 1]$ 로 정규화해야 합니다. 이를 위해 멀리 있는 평면의 깊이  $z_{far}$ 와 같은 스케일링 계수로 나누면 됩니다.

$$z' = \frac{z}{z_{far}} \in [0, 1]$$

이진수에서  $z'$ 의 16비트 버전은  $0.b_1 \dots b_8 \cdot b_9 \dots b_{16}$ 과 같으며, 이를 두 개의 8비트 구성 요소로 분배해야 합니다.  $z'$ 를 직접 정색 구성 요소에 할당할 수 있지만, 이는 최상위 8자리만 기록하고 나머지는 잘라냅니다:

$$b = z' = 0.b_1 \dots b_8$$

남은 8비트를 저장하려면 8비트 왼쪽 시프트(256으로 곱하는 방식으로 구현 가능)를 수행하고 소수 부분을 알파 채널에 기록합니다:

$$a = \text{frac}(256 \cdot z') = \text{frac}(b_1 \dots b_8 \cdot b_9 \dots b_{16}) = 0.b_9 \dots b_{16}$$

코드 표현:

```
z = depth / farZ;
normalDepth.ba = float2(z, frac(256*z));
```

값을 재구성하는 것은 역과정입니다. 알파 채널을 256으로 나누어 오른쪽으로 8비트 오른쪽 이동합니다:

$$a' = \frac{a}{256} = 0.00000000b_9 \dots b_{16}$$

그러면

$$z' = \mathbf{b} + \mathbf{a}' = \mathbf{0} . \mathbf{b_1} \dots \mathbf{b_8} + \mathbf{0.00000000b_9} \dots \mathbf{b_{16}} = \mathbf{0.b_1} \dots \mathbf{b_8 b_9} \dots \mathbf{b_{16}}$$

코드:

```
float4 normalDepth = normalDepthTex.Sample(sampler, uv); z = normalDepth.b +
normalDepth.a/256.0f;
depth = z*farZ;
```

전체 화면 노멀/덱스 텍스처 대신, 또 다른 메모리 최적화 방법은 화면 해상도의 1/4 크기로 노멀/덱스 텍스처를 사용하는 것입니다. SSAO 맵에서 했던 방식과 동일합니다. 그러나 전체 화면 노멀/덱스 텍스처가 필요한 알고리즘(예: 라이트 프리패스(light

 프리패스 <http://diaryofagraphicsprogrammer.blogspot.com/2008/03/light-pre-pass-renderer.html>)과 같이 전체 화면 노멀/덱스 맵이 필요한 경우, 짧은 픽셀 셰이더를 작성하여 전체 화면 노멀/덱스 맵을 1/4 크기로 복사 렌더링한 후, 이 작은 맵을 SSAO에 사용할 수 있습니다.

## 제23장 1인칭 시점 카메라

지금까지 이 책에서는 절차적으로 생성된 메쉬(예: 원통, 격자, 구체)와 파일에서 불러온 간단한 메쉬 데이터(예: 두개골 메쉬)를 다루었습니다. 이 짧은 장에서는 사용 중인 *.m3d* 형식을 확장하여 머티리얼 및 기타 정보를 지원하도록 합니다. [25장에서는](#) 이 형식을 다시 확장하여 캐릭터 애니메이션을 지원할 것입니다.

*.m3d* 형식은 표준 메시 형식이 아닙니다. 이 책에서 비교적 간단하게 로드할 수 있도록 개발한 형식입니다.

메쉬를 데모에 통합합니다. 스튜디오들은 일반적으로 자체 애플리케이션과 렌더링 엔진에 특화된 맞춤형 모델 형식을 사용합니다. 대부분의 3D 모델링 프로그램은 플러그인 작성 도구를 제공하여 모델링 프로그램의 데이터를 사용자 정의 형식으로 내보낼 수 있게 합니다. 또 다른 접근법은 표준 모델 형식에서 사용자 정의 모델 형식으로 변환기를 작성하는 것입니다. 예를 들어, "MeshFromOBJ10" SDK 샘플은 *.obj* 메쉬를 로드하여 Direct3D로 렌더링하는 방법을 보여줍니다. 두 번째 예로, XNA 게임 스튜디오 콘텐츠 파이프라인은 *.x* 및 *.fbx* 파일 형식을 지원합니다. 콘텐츠 파이프라인은 데이터를 게임에 바로 사용할 수 있는 XNA 모델 클래스로 로드하며, 이 클래스는 XNA 프레임워크로 렌더링할 준비가 되어 있습니다. 자체 *.x* 또는 *.fbx* 변환기를 작성하지 않으려면 XNA를 사용하여 파일을 XNA **Model** 클래스에 로드한 후 원하는 데이터를 추출하고 사용자 정의 형식으로 저장할 수 있습니다. 다양한 표준 모델 형식 로드를 지원하는 *Open Asset Import Library*(<http://assimp.sourceforge.net/index.html>)에 관심이 있을 수 있습니다. 데이터가 로드되면 라이브러리의 데이터 구조를 탐색하여 원하는 데이터를 추출하고 사용자 정의 형식으로 저장할 수 있습니다. 플러그인이나 변환기를 직접 작성하든, 이는 상당한 노력이 필요하고 견고하게 만드는 것이 어려울 수 있습니다. 따라서 XNA 게임 스튜디오 콘텐츠 파이프라인이나 *오픈 애셋 임포트 라이브러리* 같은 기존 솔루션을 사용하면 상당한 시간을 절약할 수 있습니다.

### 목표:

- 1. *.m3d* 형식에서 메쉬와 재질을 불러오는 방법을 학습합니다.
- 2. 메쉬를 서로 다른 하위 집합으로 분할하는 방법을 이해하기 위해.

### 23.1 .M3D 형식

*.m3d*는 수동으로 쉽게 읽고 편집할 수 있도록 텍스트 기반입니다. 실제로는 바이너리 형식을 사용하는 것이 메모리 효율성이 더 높습니다.

#### 23.1.1 헤더

포맷은 로더가 읽어야 할 데이터 양을 알려주는 글로벌 정보를 지정하는 헤더로 시작합니다:

```
*****m3d-파일-헤더***** #자료 2

#정점 2258
#삼각형 1674
#본즈 0
#애니메이션클립 0
```

- 1. **#자료**: 메쉬가 사용하는 고유한 자료의 수.
- 2. **#정점**: 메시의 정점 수.
- 3. **#삼각형**: 메시 내 삼각형의 수.
- 4. **#본즈**: 메시 내 본즈 개수—이 값은 [25장에서](#) 사용됩니다. 현재로서는 본즈가 있는 메시가 없으므로 이 값들은 0입니다.
- 5. **#애니메이션클립**: 메시에 포함된 애니메이션 클립 수—이 값은 [25장에서](#) 사용됩니다. 현재로서는 메시에 애니메이션 데이터가 없으므로 이 값들은 0입니다.

헤더 데이터는 다음과 같이 로드합니다:

```
bool M3DLoader::LoadM3d(const std::string& filename,
```

```
std::vector<Vertex::PosNormalTexTan>& vertices, std::vector<USHORT>& indices,
std::vector<MeshGeometry::Subset>& subsets, std::vector<M3dMaterial>& mats)
{

std::ifstream fin(filename);

UINT numMaterials = 0; UINT
numVertices = 0; UINT
numTriangles = 0; UINT numBones
= 0;
UINT numAnimationClips = 0; std::string ignore;

if(fin)
{
 fin >> ignore; // 파일 헤더 텍스트 fin >> ignore
 >> numMaterials; fin >> ignore >>
 numVertices; fin >> ignore >> numTriangles;
 fin >> ignore >> numBones;
 fin >> ignore >> numAnimationClips;
}
```

## 23.1.2 재료

.m3d 형식의 다음 "청크"는 머티리얼 목록입니다. 이 특정 예시에서 메쉬에는 두 개의 머티리얼이 있습니다:

```
*****재료***** 앰비언트: 0.4 0.4 0.4

확산: 0.9 0.9 0.9
반사: 0.4 0.4 0.4

반사 전력: 16
반사율: 0 0 0

알파 클립: 1 효과: 일반

확산 맵: tree01-leaves_diffuse.dds 노멀 맵: tree01-
leaves_normal.dds

앰비언트: 0.4 0.4 0.4

확산: 0.9 0.9 0.9

반사율: 0.4 0.4 0.4

반사 전력: 16
반사율: 0 0 0

알파 클립: 0 효과: 일반

확산 맵: tree01-bark_diffuse.dds
노멀맵: tree01-bark_normal.dds
이 파일에는 우리가 익숙한 머티리얼 데이터(앰비언트, 디퓨즈, 스펙큘러 등)뿐만 아니라 적용할 텍스처, 알파 클리핑 적용 필요 여부, 효과 이름과 같은 추가 정보도 포함되어 있습니다.
적용할 텍스처, 알파 클리핑 적용 필요 여부, 효과 이름 등이 포함됩니다. 이 책에서는 효과 이름을 사용하지 않지만, 특정 지오메트리를 렌더링할 때 필요한 효과를 지정하는 데 사용할 수 있습니다. 예를 들어, 어
떤 지오메트리는 변위 매핑이 필요하고, 다른 지오메트리는 노멀 매핑이 필요하며, 또 다른 지오메트리는 환경 매핑이 필요할 수 있습니다.
원하는 정보를 저장하기 위해 일치하는 머티리얼 구조를 정의하고, 다음과 같이 머티리얼 데이터를 로드합니다:

 struct M3dMaterial
 {
 Material Mat; bool
 AlphaClip;
 std::string 효과 유형 이름; std::wstring 확산 맵 이름
 ; std::wstring 노멀 맵 이름;
 };

void M3DLoader::ReadMaterials(std::ifstream& fin, UINT
numMaterials, std::vector<M3dMaterial>& mats)
```

```

{
 std::string ignore; mats.resize(numMaterials);

 std::string diffuse MapName; std::string
 normalMapName;

 fin >> ignore; // 재료 헤더 텍스트 for(UINT i = 0; i < numMaterials; ++i)
 {
 fin >> ignore >> mats[i].Mat.Ambient.x >> mats[i].Mat.Ambient.y >>
 mats[i].Mat.Ambient.z;
 fin >> ignore >> mats[i].Mat.Diffuse.x >> mats[i].Mat.Diffuse.y >> mats[i].Mat.Diffuse.z;
 fin >> ignore >> mats[i].Mat.Specular.x >> mats[i].Mat.Specular.y >>
 mats[i].Mat.Specular.z;
 fin >> ignore >> mats[i].Mat.Specular.w; fin >> ignore >>
 mats[i].Mat.Reflect.x >>
 mats[i].Mat.Reflect.y >> mats[i].Mat.Reflect.z; fin >> ignore >>
 mats[i].AlphaClip;
 fin >> 무시 >> mats[i].EffectType Name; fin >> 무시 >> diffuse
 MapName;
 fin >> ignore >> normalMapName;

 mats[i].Diffuse MapName.resize(diffuse MapName.size(), ' '); mats[i].NormalMapName.resize(normalMapName.size(), ' ');

 // wstring으로 변환 std::copy(diffuse MapName.begin(),
 diffuse MapName.end(), mats[i].Diffuse
 MapName.begin());
 std::copy(normalMapName.begin(), normalMapName.end(),
 mats[i].NormalMapName.begin());
 }
}

```

### 23.1.3 서브셋

메쉬는 하나 이상의 서브셋으로 구성됩니다. *서브셋*은 동일한 머티리얼을 사용하여 모두 렌더링할 수 있는 메쉬 내 삼각형 그룹입니다. 여기서 동일한 *머티리얼*이란 동일한 효과, 텍스처 및 렌더링 상태를 의미합니다. [그림 23.1](#)은 자동차를 나타내는 메쉬가 여러 서브셋으로 어떻게 분할될 수 있는지 보여줍니다.

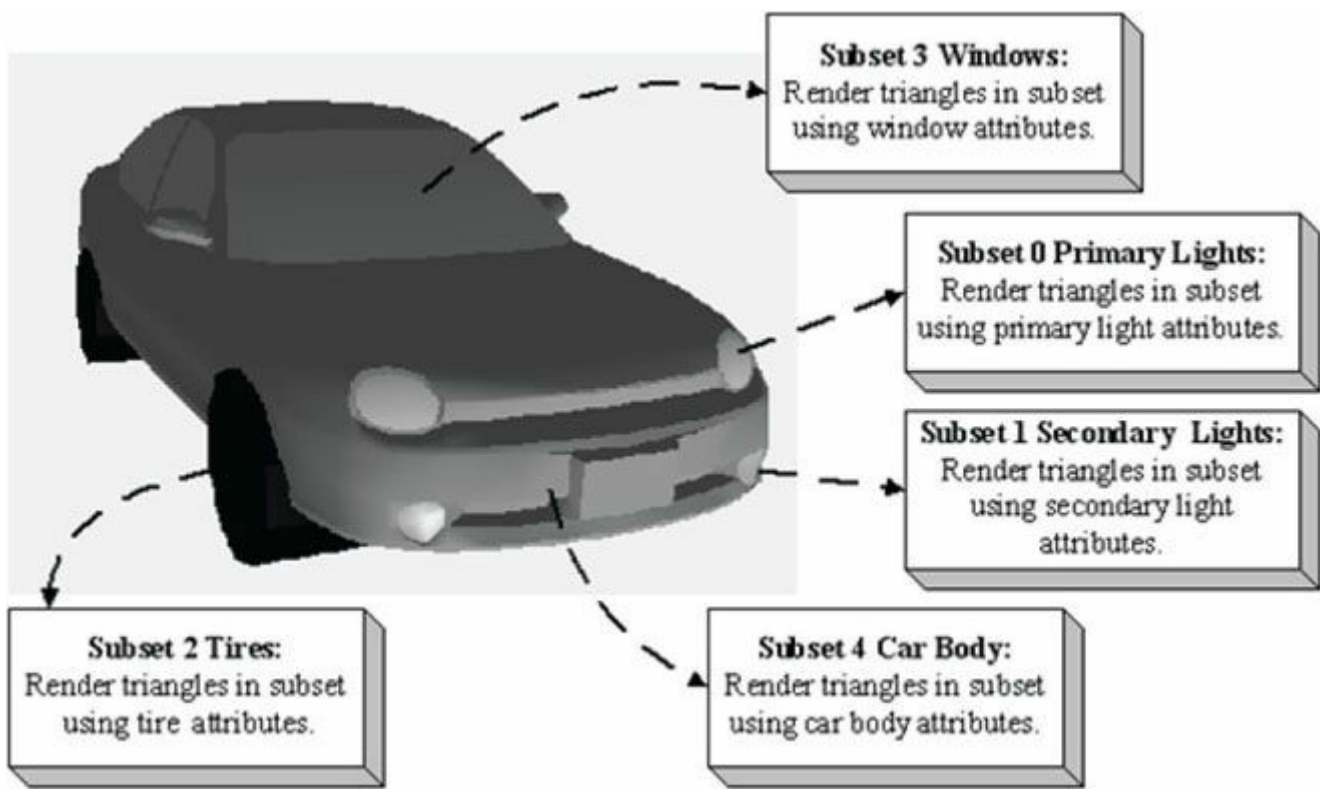


그림 23.1. 하위 집합별로 분할된 자동차. 여기서는 하위 집합별 재료만 다르지만, 텍스처가 추가되어 서로 다를 수도 있다고 가정할 수 있습니다. 또한 렌더링 상태도 다를 수 있습니다. 예를 들어, 유리창은 투명도를 위해 알파 블렌딩으로 렌더링될 수 있습니다.

각 재질에 대응하는 서브셋이 존재하며,  $i$ 번째 서브셋은  $i$ 번째 재질에 대응한다.  $i$ 번째 서브셋은  $i$ 번째 재질로 렌더링되어야 하는 연속적인 기하학적 블록을 정의한다.

\*\*\*\*\*서브셋 테이블\*\*\*\*\*

SubsetID: 0 VertexStart: 0 VertexCount: 1600 FaceStart: 0 FaceCount: 800

부분집합ID: 1 정점시작: 1600 정점수: 658 면시작: 800 면수: 874

이전 예시에서, 메시의 첫 800개 삼각형(정점 0–1599 참조)은 재료 0으로 렌더링되어야 하며, 다음 874개 삼각형(정점 1600–2257 참조)은 재료 1로 렌더링되어야 합니다.

```
struct Subset
{
 Subset() :
 Id(-1),
 VertexStart(0), VertexCount(0), FaceStart(0),
 FaceCount(0)
 {
 }

 UINT Id;
 UINT 정점시작; UINT 정점개
 수; UINT 면시작; UINT 면개
 수;
};

void M3DLoader::ReadSubsetTable(std::ifstream& fin, UINT numSubsets,
 std::vector<MeshGeometry::Subset>& subsets)
{
 std::string ignore; subsets.resize(numSubsets);

 fin >> ignore; // 부분집합 헤더 텍스트 for(UINT i = 0; i <
 numSubsets; ++i)
 {
 fin >> ignore >> subsets[i].Id;
 fin >> ignore >> subsets[i].VertexStart;
```

```
 fin >> ignore >> subsets[i].VertexCount; fin >> ignore >>
 subsets[i].FaceStart;
 fin >> ignore >> subsets[i].FaceCount;
 }
}
```

### 23.1.4 정점 및 삼각형 인덱스

마지막 두 데이터 블록은 단순히 정점과 인덱스 목록입니다(삼각형당 3개의 인덱스):

```
*****정점***** 위치: 0.9207547 10.77502 -
1.320696
접선: -0.2406725 0.05866166 -0.968832 1
법선: 0.020704 0.9982551 0.05530001
텍스처 좌표: 1 0

위치: -0.08739337 10.78203 -1.069832
접선: -0.2406725 0.05866166 -0.968832 1
법선: 0.020704 0.9982551 0.05530001
텍스처 좌표: 1 1
...

*****삼각형***** 0 1 2

3 0 2
4 5 6
...
```

우리의 .m3d 형식의 한 가지 제한 사항은 모든 정점이 동일한 형식(위치, 법선, 접선, 텍스처 좌표)을 갖도록 강제한다는 점입니다. 애플리케이션에는 서로 다른 정점 형식을 사용하는 모델이 있을 수 있습  
니다. 그러나 이러한 유연성을 형식에 추가하면 복잡해지며, 우리 형식의 목표는 데모 목적으로 간단하게 유지하는 것입니다.  
버텍스와 인덱스를 로드하는 코드는 다음과 같습니다:

```
void M3DLoader::ReadVertices(std::ifstream& fin, UINT numVertices, std::vector<Vertex::PosNormalTexTan>& vertices)
{
 std::string ignore;
 vertices.resize(numVertices);

 fin >> ignore; // 정점 헤더 텍스트 for(UINT i = 0; i <

 numVertices; ++i)
{
 fin>>ignore >> vertices[i].Pos.x >>
 정점[i].Pos.y >> 정점[i].Pos.z;
 fin>>ignore >> vertices[i].TangentU.x >> vertices[i].TangentU.y >>
 vertices[i].TangentU.z >> vertices[i].TangentU.w;
 fin>>ignore >> vertices[i].Normal.x >> vertices[i].Normal.y >>
 vertices[i].Normal.z;
 fin>>ignore >> vertices[i].Tex.x >> vertices[i].Tex.y;
 }
}

void M3DLoader::ReadTriangles(std::ifstream& fin, UINT numTriangles,
 std::vector<USHORT>& indices)
{
 std::string ignore; indices.resize(numTriangles*3);

 fin >> ignore; // 삼각형 헤더 텍스트 for(UINT i = 0; i <
 numTriangles; ++i)
```

```

 {
 fin >> indices[i*3+0] >> indices[i*3+1] >> indices[i*3+2];
 }
}

```

## 23.2 메쉬 지오메트리

메쉬 데이터를 구성하기 위해, 우리는 정점 및 인덱스 버퍼와 하위 집합을 캡슐화하는 저수준 클래스인 **MeshGeometry**를 정의합니다.

```

class MeshGeometry
{
public:
 struct Subset { ... };

public:
 MeshGeometry();
 ~MeshGeometry();

 template <typename VertexType>
 void SetVertices(ID3D11Device* device, const VertexType*
vertices, UINT count);

 void SetIndices(ID3D11Device* device, const USHORT*
indices, UINT count);

 void SetSubsetTable(std::vector<Subset>& subsetTable); void

 Draw(ID3D11DeviceContext* dc, UINT subsetId);

private:
 MeshGeometry(const MeshGeometry& rhs); MeshGeometry& operator=(const
MeshGeometry& rhs);

private:
 ID3D11Buffer* mVB; ID3D11Buffer* mIB;

 DXGI_FORMAT mIndexBufferFormat; // 항상 16비트

 // IASetVertexBuffers에 전달할 버텍스 타입 캐시 크기 sizeof(VertexType) UINT
mVertexStride;

 std::vector<Subset> mSubsetTable;
};

```

멤버 함수 구현은 간단하므로 **Draw** 호출을 제외한 코드는 생략합니다. 이 함수는 서브셋 테이블을 사용하여 지정된 서브셋에 속하는 삼각형만 렌더링합니다.

```

void MeshGeometry::Draw(ID3D11DeviceContext* dc, UINT subsetId)
{
 UINT offset = 0;

 dc->IASetVertexBuffers(0, 1, &mVB, &mVertexStride, &offset); dc->IASetIndexBuffer(mIB,
mIndexBufferFormat, 0);

 dc->DrawIndexed(mSubsetTable[subsetId].FaceCount*3,
mSubsetTable[subsetId].FaceStart*3, 0);
}

```

# 23.3 기본 모델

상위 수준에서 **BasicModel** 클래스를 정의합니다. 이 클래스는 모델의 기하 구조를 지정하는 MeshGeometry 인스턴스와 메쉬를 그리기 위해 필요한 재질을 포함합니다. 또한 시스템 메모리 복사본을 유지합니다. 시스템 메모리 복사본은 경계 볼륨 계산, 피킹 또는 충돌 감지와 같은 메쉬 데이터 읽기가 필요할 때 사용됩니다.

```
class BasicModel
{
public:
 BasicModel(ID3D11Device* device, TextureMgr& texMgr,
 const std::string& modelName, const std::wstring&
 texturePath);
 ~BasicModel();

 UINT 서브셋 카운트; std::vector<Material>
 매트;
 std::vector<ID3D11ShaderResourceView*> DiffuseMapSRV;
 std::vector<ID3D11ShaderResourceView*> NormalMapSRV;

 // 읽기용으로 메쉬 데이터의 CPU 복사본을 유지합니다.
 std::vector<Vertex::PosNormalTexTan> Vertices; std::vector<USHORT> Indices;
 std::vector<MeshGeometry::Subset> Subsets;

 MeshGeometry ModelMesh;
};

BasicModel::BasicModel(ID3D11Device* device,
 TextureMgr& texMgr,
 const std::string& modelName, const std::wstring&
 texturePath)
{
 std::vector<M3dMaterial> mats; M3DLoader
 m3dLoader;
 m3dLoader.LoadM3d(modelName, Vertices, Indices, Subsets, mats);

 ModelMesh.SetVertices(device, &Vertices[0], Vertices.size()); ModelMesh.SetIndices(device,
 &Indices[0], Indices.size()); ModelMesh.SetSubsetTable(Subsets);

 SubsetCount = mats.size();

 for(UINT i = 0; i < SubsetCount; ++i)
 {
 Mat.push_back(mats[i].Mat);

 ID3D11ShaderResourceView* diffuseMapSRV = texMgr.CreateTexture(texturePath +
 mats[i].DiffuseMapName);
 DiffuseMapSRV.push_back(diffuseMapSRV);

 ID3D11ShaderResourceView* normalMapSRV = texMgr.CreateTexture(texturePath +
 mats[i].NormalMapName);
 NormalMapSRV.push_back(normalMapSRV);
 }
}
```

이제 **BasicModel**을 렌더링하는 것은 해당 서브셋을 순회하며, i번째 서브셋의 머티리얼을 설정한 후, i번째 서브셋의 지오메트리를 렌더링 파이프라인에 제출하는 과정으로 이루어집니다.

```
for(UINT subset = 0; subset < mModelInstances[modelIndex].Model->SubsetCount; ++subset)
{
```

```
Effects::NormalMapFX->SetMaterial(Model->Mat[subset]); Effects::NormalMapFX->SetDiffuseMap(Model->DiffuseMapSRV[subset]); Effects::NormalMapFX->SetNormalMap(Model->NormalMapSRV[subset]);
```

```
tech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); Model->ModelMesh.Draw(md3dImmediateContext, subset);
```

```
}
```

일부 메쉬가 동일한 텍스처를 공유할 수 있으므로, 중복을 방지하기 위한 주의가 없다면 이러한 텍스처가 여러 번 로드될 수 있습니다. 중복을 방지하기 위해 텍스처 생성을 위한 **TextureMgr** 클래스를 도입합니다. 이 클래스는 **std::map** 데이터 구조를 사용하여 텍스처가 두 번 로드되거나 생성되는 것을 방지합니다. 사용자가 텍스처를 생성하려고 할 때, 관리자는 먼저 해당 텍스처가 맵에 이미 존재하는지 확인합니다. 존재할 경우, 관리자는 텍스처를 다시 로드하지 않고 해당 텍스처에 대한 포인터만 반환합니다. 그렇지 않으면 관리자가 텍스처를 로드한 후 해당 포인터를 반환합니다. TextureMgr 클래스는 텍스처 리소스의 소멸도 처리하므로, 메인 애플리케이션 코드에서는 이를 수행할 필요가 없습니다. TextureMgr 클래스는 TextureMgr.h/.cpp에 정의되어 있으며 Common 디렉터리에 위치합니다.

리소스의 소멸도 처리하므로 메인 애플리케이션 코드에서 이를 수행할 필요가 없습니다. **TextureMgr** 클래스는 TextureMgr.h/.cpp에 정의되어 있으며 Common 디렉터리에 위치합니다.

**기본 모델(BasicModel)**을 여러 위치에 반복적으로 그려 인스턴스화할 수 있습니다. 이를 위해 다음과 같은 간단한 구조를 사용합니다:

구조를 사용합니다:

```
struct BasicModelInstance
{
 BasicModel* 모델; XMFLOAT4X4
 월드;
};
```

## 23.4 메쉬 뷰어 데모

**그림 23.2**는 본 장의 데모 화면을 보여줍니다. 본 장에서 논의한 로딩 코드와 데이터 구조를 사용하여 여러 메쉬를 로드하고 렌더링합니다.



**그림 23.2.** "메쉬 뷰어" 데모의 스크린샷.

## 23.5 요약

---

1. 메쉬는 하나 이상의 서브셋으로 구성됩니다. 서브셋은 동일한 효과, 텍스처 및 렌더링 상태를 사용하여 모두 렌더링할 수 있는 메쉬 내 삼각형 그룹입니다.
2. 메쉬 데이터를 구성하기 위해, 우리는 정점 및 인덱스 버퍼를 캡슐화하고 메쉬의 하위 집합을 정의하는 저수준 클래스인 **MeshGeometry**를 정의합니다.
3. **BasicModel** 클래스는 메시를 그리기 위해 필요한 재료로서 **MeshGeometry** 인스턴스를 캡슐화합니다. 또한 메시의 시스템 메모리 복사본을 유지합니다. 시스템 메모리 복사본은 경계 볼륨 계산, 피킹 또는 충돌 감지와 같이 메시 데이터를 읽어야 할 때 필요합니다.

## 23.6 연습 문제

---

1. 수동으로 *m3d* 파일을 생성하고 큐브 데이터로 채우십시오. 각 면의 지오메트리를 별도의 서브셋에 배치하여 총 6개의 서브셋이 되도록 합니다. 각 큐브 면에 서로 다른 텍스처를 적용하십시오.
2. Direct3D10 SDK 데모인 "MeshFromOBJ10"을 연구하여 *.obj* 파일 형식을 로드하고 렌더링하는 방법을 확인하세요.
3. Open Asset Import Library(<http://assimp.sourceforge.net/index.html>)를 다운로드하여 설정하십시오. 라이브러리가 지원하는 무료 3D 모델 형식을 웹에서 찾거나(또는 모델링 프로그램이 있다면 직접 모델을 제작하세요), Direct3D 11을 사용하여 해당 모델 뷰어를 작성하세요. 특히, 이 라이브러리는 Blender 3D 형식(.blend)을 지원하며, Blender 3D(<http://www.blender.org/>)는 무료 오픈 소스 모델링 프로그램입니다.

# 제24<sup>장</sup> 쿼터니언 제24장

제1장에서는 벡터라는 새로운 종류의 수학적 대상을 소개했습니다. 특히, 3차원 벡터는 실수의 순서된 3원조로 구성되며, 기하학적으로 유용한 벡터 연산들을 정의했습니다. 마찬가지로 2장에서는 행렬을 소개했는데, 이는 실수의 직사각형 표로 유용한 연산들이 정의되어 있습니다. 예를 들어 행렬이 선형 변환과 아핀 변환을 어떻게 표현하는지, 행렬 곱셈이 변환 합성에 어떻게 대응하는지 살펴보았습니다. 이 장에서는 또 다른 수학적 대상인 쿼터니언에 대해 배웁니다. 단위 쿼터니언이 3차원 회전을 표현하는 데 사용될 수 있으며 편리한 보간 특성을 지닌다는 점을 살펴볼 것입니다. 쿼터니언(및 회전)에 대한 포괄적인 설명을 원하는 독자를 위해 [Kuipers99]의 해당 주제에 전념한 책을 추천합니다.

## 학습 목표:

- 복소수를 복습하고, 복소수 곱셈이 평면에서 어떻게 회전을 수행하는지 상기한다.
- 쿼터니언과 그 위에 정의된 연산에 대한 이해를 얻기 위해.
- 단위 쿼터니언 집합이 3차원 회전을 어떻게 표현하는지 알아보기.
- 다양한 회전 표현 간 변환 방법을 파악한다.
- 단위 쿼터니언 사이의 보간법을 학습하고, 이것이 기하학적으로 3차원 방향 사이의 보간과 동등함을 이해한다.
- XNA Math 라이브러리의 쿼터니언 함수와 클래스에 익숙해지기.

## 24.1 복소수 복습

쿼터니언은 복소수의 일반화로 볼 수 있으며, 이는 쿼터니언보다 먼저 복소수를 연구하도록 동기를 부여한다. 특히, 이 절의 주요 목표는 복소수  $\mathbf{p}$ (2차원 벡터 또는 점으로 간주)에 단위 복소수를 곱하면  $\mathbf{p}$ 의 회전을 발생함을 보여주는 것이다. 이후 §24.3에서는 단위 사원수를 포함한 특별한 사원수 곱셈이 벡터 또는 점  $\mathbf{p}$ 의 3차원 회전을 초래함을 증명할 것이다.

### 24.1.1 정의

복소수를 소개하는 방법은 다양하다. 우리는 복소수를 2차원 점이나 벡터로 즉시 생각하게 하는 방식으로 소개한다.

실수 쌍  $\mathbf{z} = (a, b)$ 은 복소수이다. 첫 번째 성분을 실수부, 두 번째 성분을 허수부라고 부른다. 성분은 *허수* 부분이라고 부릅니다. 또한, 등호, 덧셈, 뺄셈, 곱셈, 나눗셈은 다음과 같이 정의됩니다:

- $(a, b) = (c, d)$  다음과 같은 경우에  $a = c$  그리고  $b = d$
- $(a, b) \pm (c, d) = (a \pm c, b \pm d)$  만 성립한다.
- $(a, b)(c, d) = (ac - bd, ad + bc)$
- $\frac{(a, b)}{(c, d)} = \left( \frac{ac + bd}{c^2 + d^2}, \frac{bc - ad}{c^2 + d^2} \right)$  if  $(c, d) \neq (0, 0)$

결합법칙, 분배법칙 등); 연습문제 1을 참조하십시오.

복소수가  $(x,0)$  형태라면, 이를 실수  $x$ 로 식별하고  $x = (x,0)$ 로 표기하는 것이 관례이다. 따라서 모든 실수는 허수 성분이 0인 복소수로 생각할 수 있다. 그러면 실수와 복소수의 곱은  $x(a,b) = (x,0)(a,b) = (xa,xb) = (a,b)(x,0) = (a,b)x$ 로 주어지며, 이는 스칼라-벡터 곱셈을 연상케 한다.

우리는 정의한다 *허수* 단위  $i = (0,1)$  사용 우리의 정의 정의 복합 곱셈에 대한 관찰하다 다음과 같다

$i^2 = (0,1)(0,1) = (-1,0) = -1$ , 이는  $i = \sqrt{-1}$   $z = (a,b)$  이는  $i$ 가 방정식  $\bar{z} = (a,-b)$ 를 만족시킨다는 것을 알려준다.

복소수의 **복소공액**은  $\bar{z}$ 로 표기되며 다음 식으로 주어집니다. 간단한 방법은

복소수 나눗셈 공식을 기억하는 간단한 방법은 분자와 분모를 분모의 공액으로 곱하여 분모를 실수로 만드는 것이다:

$$\frac{(a,b)}{(c,d)} = \frac{(a,b)(c,-d)}{(c,d)(c,-d)} = \frac{(ac+bd,bc-ad)}{c^2+d^2} = \left(\frac{ac+bd}{c^2+d^2}, \frac{bc-ad}{c^2+d^2}\right)$$

다음으로, 복소수  $(a,b)$ 가  $a+ib$  형태로 표현될 수 있음을 보여줍니다. 우리는  $a = (a,0)$   $b = (b,0)$ 이며, 따라서

$i = (0,1)$  따라서  $a+ib = (a,0) + (0,1)(b,0) = (a,0) + (0,b) = (a,b)$

$a+ib$  형태를 사용하여 덧셈, 뺄셈, 곱셈, 나눗셈에 대한 공식을 다음과 같이 재구성할 수 있습니다:

1.  $(a+ib) \pm (c+id) = (a+c) + i(b+d)$
2.  $(a+ib)(c+id) = (ac-bd) + i(ad+bc)$
3.  $\frac{a+ib}{c+id} = \frac{ac+bd}{c^2+d^2} + i\frac{bc-ad}{c^2+d^2}$  if  $(c,d) \neq (0,0)$ 로 주어집니다.

24.1.2 기하학적 해석

복소수의 순서쌍 형태  $a+ib = (a,b)$ 은 자연스럽게 복소수를 복소평면 상의 2차원 점 또는 벡터로 기하학적으로 생각하도록 유도한다. 실제로 복소수 덧셈의 정의는 벡터 덧셈의 정의와 일치한다

; 그림 24.1을 참조하라. 복소수 곱셈에 대한 기하학적 해석은 다음 절에서 제시할 것이다.

복소수  $a+ib$ 의 **절대값**(또는 **크기**)은 이를 나타내는 벡터의 길이로 정의된다(그림 24.2). 이는 다음과 같이 주어짐을 알고 있다:

$$|a+ib| = \sqrt{a^2+b^2}$$

복소수의 크기가 1인 경우, 이를 **단위 복소수**라고 한다.

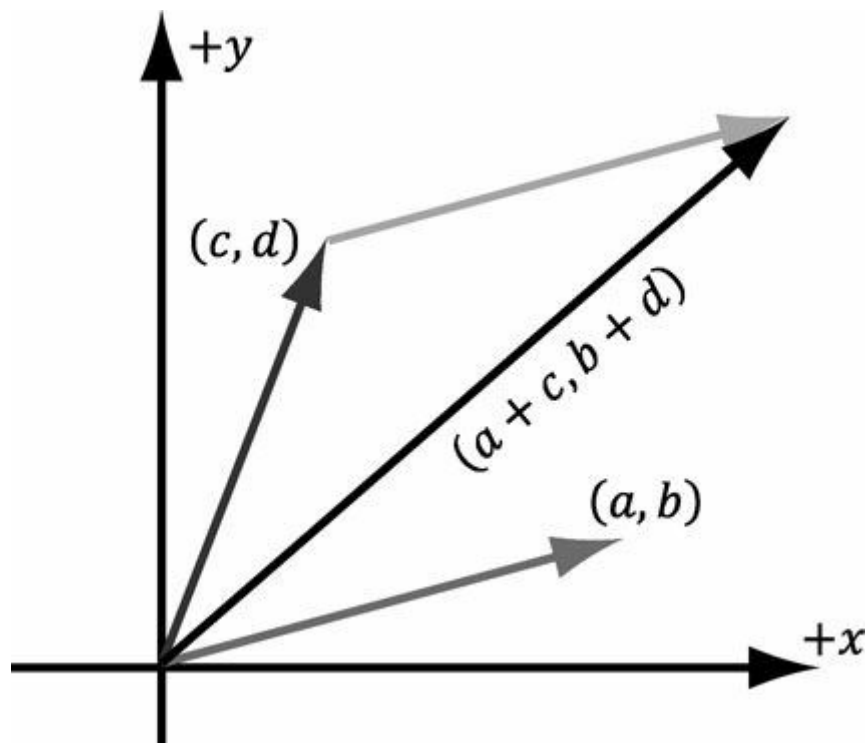


그림 24.1. 복소수 덧셈은 평면에서의 벡터 덧셈을 연상시킨다.

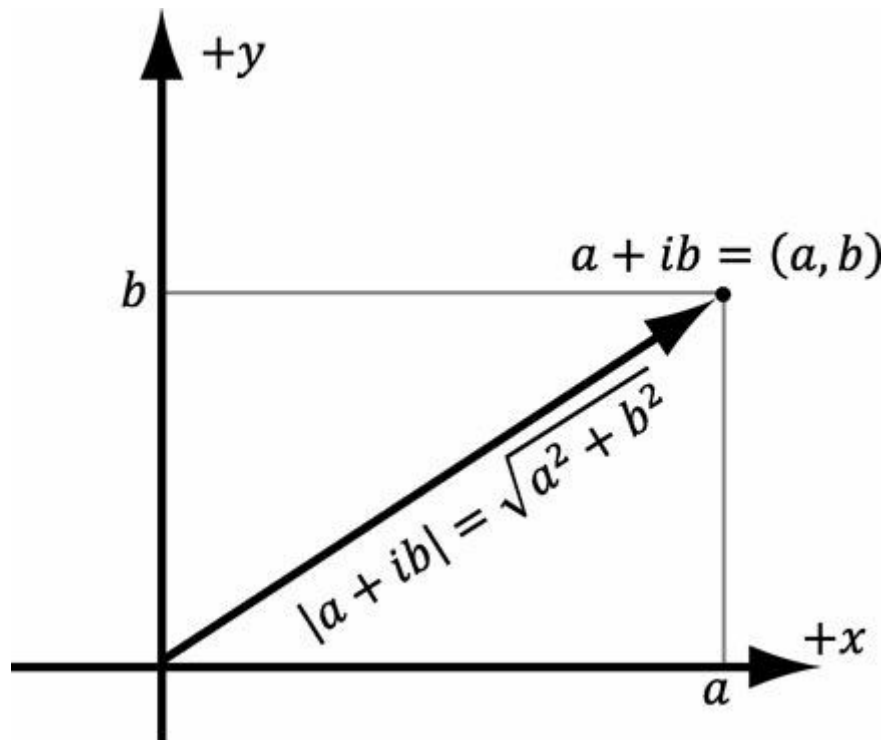


그림 24.2. 복소수의 크기.

### 24.1.3 극좌표 표현과 회전

복소수는 2차원 복소 평면상의 점 또는 벡터로 볼 수 있으므로, 극좌표계를 사용하여 그 성분을 표현할 수도 있습니다(그림 24.3 참조):

$$r = |a + ib|$$

$$a + ib = r \cos \theta + i r \sin \theta = r (\cos \theta + i \sin \theta)$$

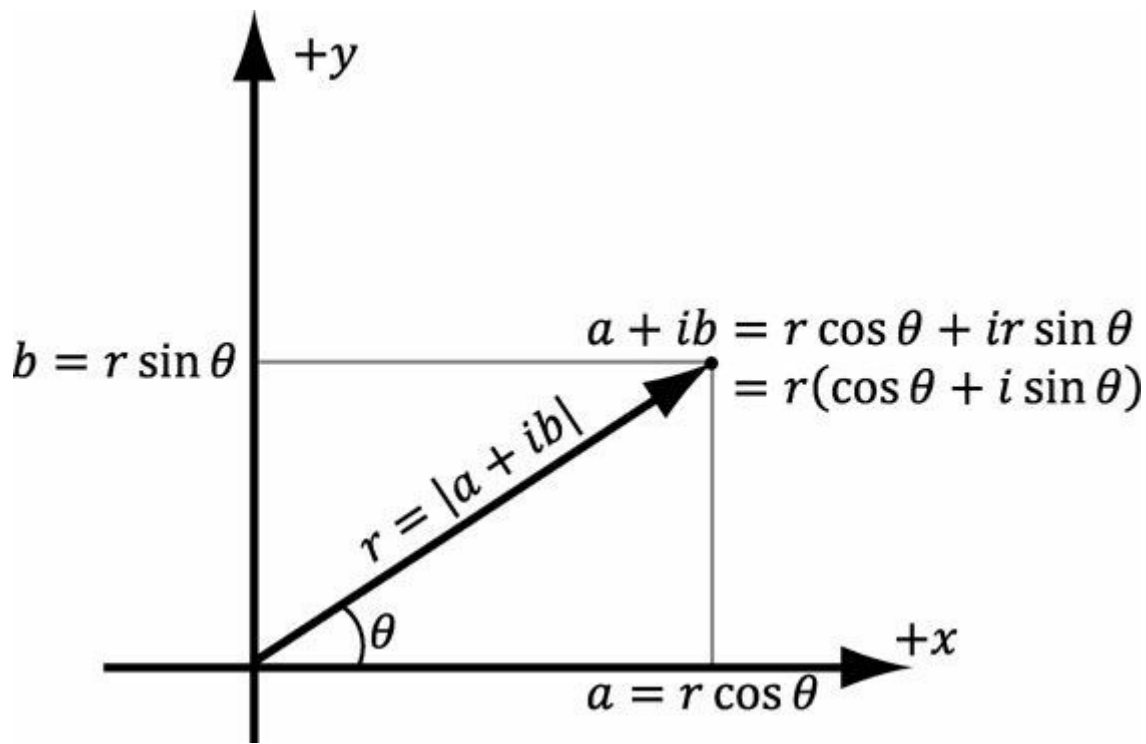


그림 24.3. 복소수의 극표현.

방정식의 우변은 복소수의 극표현이라고 한다.

다음과 같이 정의한다. 우변을

곱하자

두

복소수

수

곱하기

극좌표

형태로  $\mathbf{z}_1 = r_1 (\cos \theta_1 + i \sin \theta_1)$  다음

과 같이 정의한다.  $\mathbf{z}_2 = r_2 (\cos \theta_2 + i \sin \theta_2)$  라고 하자. 그러면

$$\begin{aligned} \mathbf{z}_1 \mathbf{z}_2 &= r_1 r_2 (\cos \theta_1 \cos \theta_2 - \sin \theta_1 \sin \theta_2 + i (\cos \theta_1 \sin \theta_2 + \sin \theta_1 \cos \theta_2)) \\ &= r_1 r_2 (\cos(\theta_1 + \theta_2) + i \sin(\theta_1 + \theta_2)) \end{aligned}$$

여기서 삼각함수 항등식을 사용했습니다

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$$

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta$$

따라서 기하학적으로, 곱  $\mathbf{z}_1 \mathbf{z}_2$  는 크기  $r_1 r_2$  를 가지며 실수축과  $\theta_1 + \theta_2$  의 각도를 이루는 벡터를 나타내는 복소수이다.

특히, 만약  $r_2 = 1$  이라면  $\mathbf{z}_1 \mathbf{z}_2 = r_1 (\cos(\theta_1 + \theta_2) + i \sin(\theta_1 + \theta_2))$  이며, 이는 기하학적으로  $\mathbf{z}_1$  을 각도  $\theta_2$  만큼 회전시킵니다. [그림 24.4](#)를 참조하십시오. 따

라서, 복소수  $\mathbf{z}_1$  (2차원 벡터 또는 점으로 간주)에 단위 복소수  $\mathbf{z}_2$  를 곱하면  $\mathbf{z}_1$  의 회전이 발생합니다.

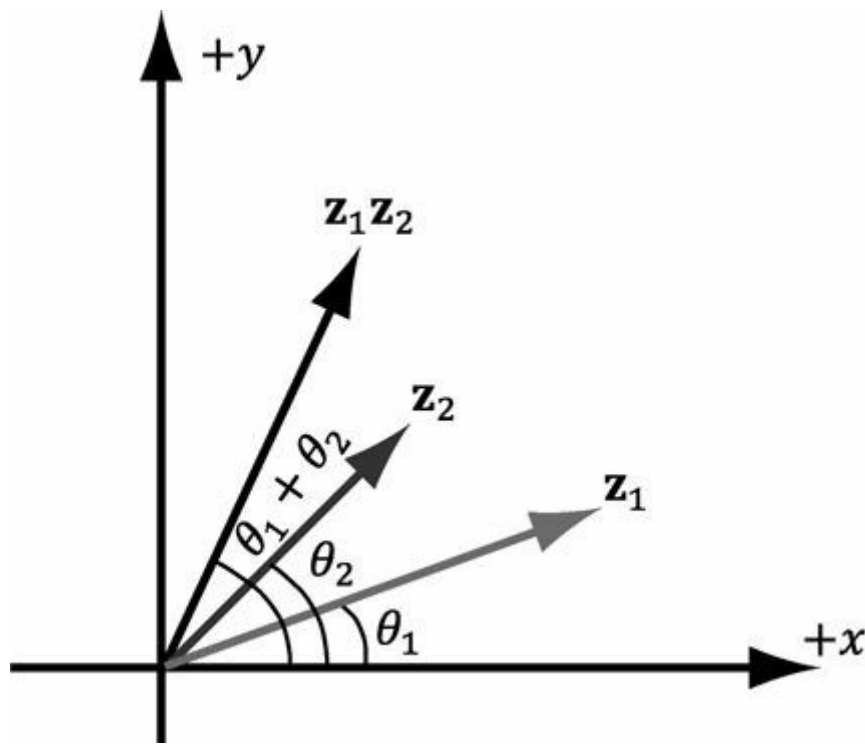


그림 24.4. 단위 복소수 벡터( $\mathbf{z}_1 = r_1(\cos \theta_1 + i \sin \theta_1)$ ,  $\mathbf{z}_2 = (\cos \theta_2 + i \sin \theta_2)$ ). 벡터  $\mathbf{z}_1\mathbf{z}_2$ 의 끝은  $\mathbf{z}_1$ 을 각도  $\theta_2$ 만큼 회전시킨다.

## 24.2 쿼터니언 대수

### 24.2.1 정의와 기본 연산

실수의 순서된 4원조

$\mathbf{q} = (x, y, z, w) = (q_1, q_2, q_3, q_4)$ 는 사원수이다. 이는 일반적으로 다음과 같이 약칭된다.

$\mathbf{q} = (\mathbf{u}, w) = (x, y, z, w)$  그리고 우리는  $\mathbf{u} = (x, y, z)$  허수 벡터 성분을  $i$ , 실수 성분을  $w$ 라 부릅니다. 또한 등호, 덧셈, 뺄셈, 곱셈, 나눗셈은 다음과 같이 정의됩니다:

1.  $(\mathbf{u}, a) = (\mathbf{v}, b)$  다음과 같은 경우에  $\mathbf{u} = \mathbf{v}$  그리고  $a = b$
2.  $(\mathbf{u}, a) \pm (\mathbf{v}, b) = (\mathbf{u} \pm \mathbf{v}, a \pm b)$ 만 성립한다.
3.  $(\mathbf{u}, a)(\mathbf{v}, b) = (a\mathbf{v} + b\mathbf{u} + \mathbf{u} \times \mathbf{v}, ab - \mathbf{u} \cdot \mathbf{v})$

곱셈의 순서는 중요하며,  $(\mathbf{u}, a)(\mathbf{v}, b) \neq (\mathbf{v}, b)(\mathbf{u}, a)$ 입니다—그리고 이 정의는 유용한 것으로 밝혀졌습니다.

$$\begin{aligned} \mathbf{p} &= (\mathbf{u}, p_4) = (p_1, p_2, p_3, p_4) & \mathbf{q} &= (\mathbf{v}, q_4) = (q_1, q_2, q_3, q_4) \\ \mathbf{u} \times \mathbf{v} &= (p_2q_3 - p_3q_2, p_3q_1 - p_1q_3, p_1q_2 - p_2q_1) & \mathbf{u} \times \mathbf{v} &= (p_2q_3 - p_3q_2, p_3q_1 - p_1q_3, p_1q_2 - p_2q_1) \\ \mathbf{u} \cdot \mathbf{v} &= p_1q_1 + p_2q_2 + p_3q_3 & \mathbf{u} \cdot \mathbf{v} &= p_1q_1 + p_2q_2 + p_3q_3 \\ \mathbf{r} &= \mathbf{p}\mathbf{q} \\ r_1 &= p_4q_1 + q_4p_1 + p_2q_3 - p_3q_2 = q_1p_4 - q_2p_3 + q_3p_2 + q_4p_1 \\ r_2 &= p_4q_2 + q_4p_2 + p_3q_1 - p_1q_3 = q_1p_3 + q_2p_4 - q_3p_1 + q_4p_2 \\ r_3 &= p_4q_3 + q_4p_3 + p_1q_2 - p_2q_1 = -q_1p_2 + q_2p_1 + q_3p_4 + q_4p_3 \\ r_4 &= p_4q_4 - p_1q_1 - p_2q_2 - p_3q_3 = -q_1p_1 - q_2p_2 - q_3p_3 + q_4p_4 \end{aligned}$$

이는 행렬 곱으로 다음과 같이 쓸 수 있습니다:

$$\mathbf{pq} = \begin{bmatrix} p_4 & -p_3 & p_2 & p_1 \\ p_3 & p_4 & -p_1 & p_2 \\ -p_2 & p_1 & p_4 & p_3 \\ -p_1 & -p_2 & -p_3 & p_4 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix}$$

 행 벡터-행렬 곱셈을 선호한다면, 단순히 전치를 취하면 됩니다:

$$\left( \begin{bmatrix} p_4 & -p_3 & p_2 & p_1 \\ p_3 & p_4 & -p_1 & p_2 \\ -p_2 & p_1 & p_4 & p_3 \\ -p_1 & -p_2 & -p_3 & p_4 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix} \right)^T = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix}^T \begin{bmatrix} p_4 & -p_3 & p_2 & p_1 \\ p_3 & p_4 & -p_1 & p_2 \\ -p_2 & p_1 & p_4 & p_3 \\ -p_1 & -p_2 & -p_3 & p_4 \end{bmatrix}^T$$

### 24.2.2 특수 곱셈

$\mathbf{i} = (1, 0, 0, 0)$ ,  $\mathbf{j} = (0, 1, 0, 0)$ ,  $\mathbf{k} = (0, 0, 1, 0)$ 를 사원수라고 합시다. 그러면 다음과 같은 특수 곱셈이 존재하며, 일부는 외적의 특성을 연상시킵니다:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$$

$$\mathbf{ij} = \mathbf{k} = -\mathbf{ji}$$

$$\mathbf{jk} = \mathbf{i} = -\mathbf{kj}$$

$$\mathbf{ki} = \mathbf{j} = -\mathbf{ik}$$

이러한 등식은 사원수 곱셈 정의에서 직접 도출됩니다. 예를 들어,

$$\mathbf{ij} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \mathbf{k}$$

### 24.2.3 성질

사원수 곱셈은 교환법칙을 따르지 않습니다. 예를 들어, §24.2.2에서  $\mathbf{ij} = -\mathbf{ji}$ 임을 보였습니다. 그러나 사원수 곱셈은 결합법칙을 따릅니다. 이는 사원수 곱셈을 행렬 곱셈으로 표현할 수 있고, 행렬 곱셈은 결합법칙을 따르기 때문에 알 수 있습니다. 사원수  $\mathbf{e} = (0, 0, 0, 1)$ 는 곱셈 항등원 역할을 합니다:

$$\mathbf{pe} = \mathbf{ep} = \begin{bmatrix} p_4 & -p_3 & p_2 & p_1 \\ p_3 & p_4 & -p_1 & p_2 \\ -p_2 & p_1 & p_4 & p_3 \\ -p_1 & -p_2 & -p_3 & p_4 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \end{bmatrix} = \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \end{bmatrix}$$

우리는 또한 다음을 얻습니다: 즉, 쿼터니언 곱셈 분배법칙을 따릅니다. 쿼터니언 덧셈에 대해서도  $\mathbf{p}(\mathbf{q} + \mathbf{r}) = \mathbf{pq} + \mathbf{pr}$  그리고

$(\mathbf{q} + \mathbf{r})\mathbf{p} = \mathbf{qp} + \mathbf{rp}$  이를 확인하려면 사원수 곱셈과 덧셈을 행렬 형태로 작성하고, 행렬이

곱셈은 행렬 덧셈에 대해 분배법칙을 따른다.

24.2.4 변환

실수, 벡터(또는 점), 그리고 쿼터니언을 다음과 같이 연관시킵니다. 실수  $s$ 와 벡터  $\mathbf{u} = (x, y, z)$ 가 주어졌을 때, 다음이 성립합니다.

1.  $s = (0,0,0,s)$
2.  $\mathbf{u} = (x, y, z) = (\mathbf{u}, 0) = (x, y, z, 0)$
- 즉, 모든 실수는 벡터 성분이 0인 사원수로, 모든 벡터는 실수 성분이 0인 사원수로 생각할 수 있다. 특히, 항등 사원수에 대해  $\mathbf{1} = (0,0,0,1)$ 임을 유의하라. 실수 성분이 0인 사원수는 순수 사원수

라 부른다.

부분을 가진 사원수를 순수 사원수라고부름니다.

쿼터니언 곱셈의 정의를 사용하여, 실수에 쿼터니언을 곱하는 것은 단순히 "스칼라 곱셈"이며  
가 교환법칙을 따른다:

$$s(p_1, p_2, p_3, p_4) = (0,0,0,s)(p_1, p_2, p_3, p_4) = \begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & s \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \end{bmatrix} = \begin{bmatrix} sp_1 \\ sp_2 \\ sp_3 \\ sp_4 \end{bmatrix}$$

마찬가지로,

$$(p_1, p_2, p_3, p_4)s = (p_1, p_2, p_3, p_4)(0,0,0,s) = \begin{bmatrix} p_4 & -p_3 & p_2 & p_1 \\ p_3 & p_4 & -p_1 & p_2 \\ -p_2 & p_1 & p_4 & p_3 \\ -p_1 & -p_2 & -p_3 & p_4 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ s \end{bmatrix} = \begin{bmatrix} sp_1 \\ sp_2 \\ sp_3 \\ sp_4 \end{bmatrix}$$

24.2.5 공액과 노름

쿼터니언  $\mathbf{q} = (q_1, q_2, q_3, q_4) = (\mathbf{u}, q_4)$ 의 복소공액은  $\mathbf{q}^*$ 로 표기하며 다음과 같이 정의됩니다

$$\mathbf{q}^* = -q_1 - q_2 - q_3 + q_4 = (-\mathbf{u}, q_4)$$

즉, 사원수의 허수 벡터 부분만 부호만 바꾸면 됩니다. 이는 복소수의 복소수 공액과 유사합니다. 공액은 다음과 같은 성질을 가집니다:

1.  $(\mathbf{pq})^* = \mathbf{q}^* \mathbf{p}^*$
2.  $(\mathbf{p} + \mathbf{q})^* = \mathbf{p}^* + \mathbf{q}^*$
3.  $(\mathbf{q}^*)^* = \mathbf{q}$
4.  $(s\mathbf{q})^* = s\mathbf{q}^* \quad s \in \mathbb{R}$
5.  $\mathbf{q} + \mathbf{q}^* = (\mathbf{u}, q_4) + (-\mathbf{u}, q_4) = (0, 2q_4) = 2q_4$
6.  $\mathbf{qq}^* = \mathbf{q}^* \mathbf{q} = q_1^2 + q_2^2 + q_3^2 + q_4^2 = ||\mathbf{u}||^2 + q_4^2$
- 노름(또는 크기)은 다음

$$||\mathbf{q}|| = \sqrt{\mathbf{qq}^*} = \sqrt{q_1^2 + q_2^2 + q_3^2 + q_4^2} = \sqrt{||\mathbf{u}||^2 + q_4^2}$$

노름이 1인 사원수를 단위 사원수라고합니다. 노름은 다음과 같은 성질을 가집니다:

1.  $||\mathbf{q}||^* = ||\mathbf{q}||$
  2.  $||\mathbf{pq}|| = ||\mathbf{p}|| ||\mathbf{q}||$
- 특히, 성질 2는 두 단위 사원수의 곱이 단위 사원수임을 알려줍니다. 또한  $||\mathbf{p}|| = 1$ 이라면,
- $||\mathbf{pq}|| = ||\mathbf{q}||$
- 공액과 노름의 성질은 정의로부터 직접 유도될 수 있다. 예를 들어,

$$\begin{aligned} \left(\mathbf{q}^*\right)^* &= \left(-\mathbf{u}, q_4\right)^* = \left(\mathbf{u}, q_4\right) = \mathbf{q} \\ ||\mathbf{q}^*|| &= ||\left(-\mathbf{u}, q_4\right)|| = \sqrt{||-\mathbf{u}||^2 + q_4^2} = \sqrt{||\mathbf{u}||^2 + q_4^2} = ||\mathbf{q}|| \\ ||\mathbf{pq}||^2 &= (\mathbf{pq})(\mathbf{pq})^* \\ &= \mathbf{pq}\mathbf{q}^*\mathbf{p}^* \\ &= \mathbf{p}||\mathbf{q}||^2\mathbf{p}^* \\ &= \mathbf{p}\mathbf{p}^*||\mathbf{q}||^2 \\ &= ||\mathbf{p}||^2||\mathbf{q}||^2 \end{aligned}$$

독자는 다른 성질들을 유도해 보아야 한다(연습문제 참조).

### 24.2.6 역원

행렬과 마찬가지로, 사원수의 곱셈은 교환법칙을 따르지 않으므로 나눗셈 연산자를 정의할 수 없습니다. (우리는 곱셈이 교환법칙을 따를 때에만 나눗셈을 정의하는 것을 선호합니다. 그래야 다음과 같은 성질을 얻을 수 있기 때문입니다.

$$\frac{a}{b} = ab^{-1} = b^{-1}a$$

그러나 모든 0이 아닌 쿼터니언은

$$\mathbf{q} = \left(q_1, q_2, q_3, q_4\right) = \left(\mathbf{u}, q_4\right)$$

가 0이 아닌 사원수라면, 그 역원은  $\mathbf{q}^{-1}$

로 표기되며 다음과 같이 주어집니다:

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{||\mathbf{q}||^2}$$

이것이 실제로 역원임을 쉽게 확인할 수 있습니다. 왜냐하면 다음이 성립하기 때문입니다:

$$\begin{aligned} \mathbf{q}\mathbf{q}^{-1} &= \frac{\mathbf{q}\mathbf{q}^*}{||\mathbf{q}||^2} = \frac{||\mathbf{q}||^2}{||\mathbf{q}||^2} = 1 = (0,0,0,1) \\ \mathbf{q}^{-1}\mathbf{q} &= \frac{\mathbf{q}^*\mathbf{q}}{||\mathbf{q}||^2} = \frac{||\mathbf{q}||^2}{||\mathbf{q}||^2} = 1 = (0,0,0,1) \end{aligned}$$

$\mathbf{q}$ 가 단위 사원수일 경우  $||\mathbf{q}||^2 = 1$ 이므로  $\mathbf{q}^{-1} = \mathbf{q}^*$ 임을 관찰할 수 있다.

쿼터니언 역원에 대해 다음 성질이 성립한다:

1.  $\left(\mathbf{q}^{-1}\right)^{-1} = \mathbf{q}$
2.  $\left(\mathbf{pq}\right)^{-1} = \mathbf{q}^{-1}\mathbf{p}^{-1}$

### 24.2.7 극좌표 표현

$$\mathbf{q} = \left(q_1, q_2, q_3, q_4\right) = \left(\mathbf{u}, q_4\right)$$

가 단위 사원수일 때,

$$||\mathbf{q}||^2 = ||\mathbf{u}||^2 + q_4^2 = 1$$

이  $\theta \in [0, \pi]$ 에 대해  $q_4 = \cos \theta$  이고  $||\mathbf{u}|| = \sin \theta$  이므로  $q_1, q_2, q_3$ 는  $\sin \theta$ 를 곱한 단위 벡터를 나타낼 수 있다. 이는  $q_4^2 \leq 1 \Leftrightarrow |q_4| \leq 1 \Leftrightarrow -1 \leq q_4 \leq 1$  그림 24.5는  $\theta$ 에 대해 각도  $\theta \in [0, \pi]$ 가 존재함을 보여준다.

$q_4 = \cos \theta$  삼각함수 항등식  $\sin^2 \theta + \cos^2 \theta = 1$ 를 이용하면, 우리는

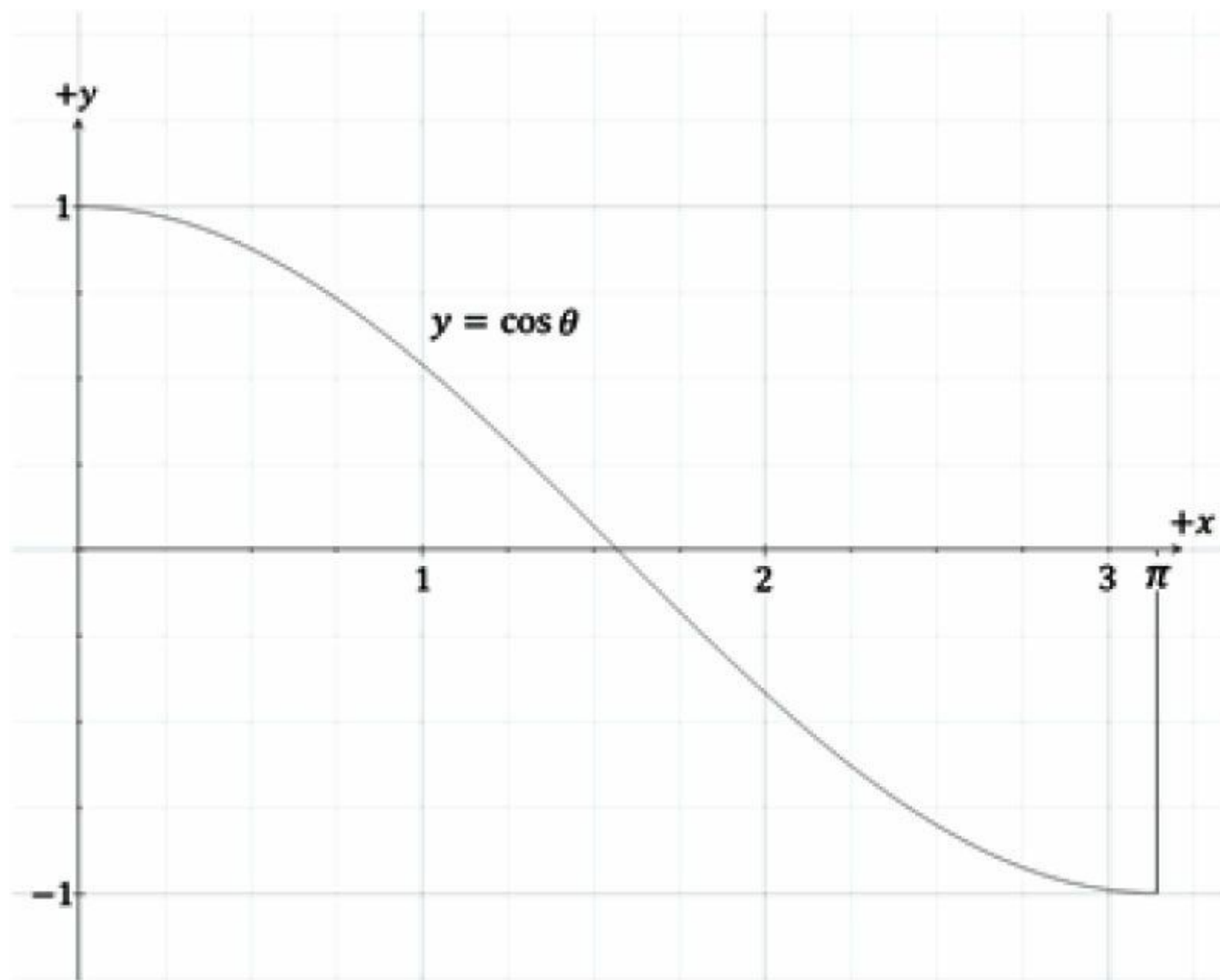


그림 24.5. 수  $y \in [-1, 1]$ 에 대해  $y = \cos \theta$ 을 만족하는 각도  $\theta$ 가 존재한다.

$$\sin^2 \theta = 1 - \cos^2 \theta = 1 - q_4^2 = ||\mathbf{u}||^2$$

이는 다음을 의미한다

$$||\mathbf{u}|| = |\sin \theta| = \sin \theta \quad \text{for } \theta \in [0, \pi]$$

이제  $\mathbf{u}$ 와 같은 방향의 단위 벡터를  $\mathbf{n}$ 으로 표시하자:

$$\mathbf{n} = \frac{\mathbf{u}}{||\mathbf{u}||} = \frac{\mathbf{u}}{\sin \theta}$$

따라서,  $\mathbf{u} = \sin \theta \mathbf{n}$ 이며, 따라서 단위 사원수  $\mathbf{q} = (\mathbf{u}, a_4)$ 를 다음과 같은 극좌표 표현으로 쓸 수 있다. 여기서  $\mathbf{n}$ 은 단위 벡터이다.  
는 단위 벡터입니다:

$$\mathbf{q} = (\sin \theta \mathbf{n}, \cos \theta) \quad \text{for } \theta \in [0, \pi]$$

예를 들어, 쿼터니언이 주어졌다고 가정하자.

$$\mathbf{q} = \left(0, \frac{1}{2}, 0, \frac{\sqrt{3}}{2}\right) \quad \text{극좌표 표현으로 변환하려면, 우리는 다음을 구합니다}$$



참고로, 벡터의 외적 정의에 따라  $(\mathbf{v} \times \mathbf{u})$ 는  $\mathbf{u}$ 에 직교하므로  $\mathbf{u} \cdot (\mathbf{v} \times \mathbf{u}) = 0$ 입니다.

벡터 부분

$$\begin{aligned}
 & a\mathbf{n} + b\mathbf{m} + \mathbf{m} \times \mathbf{n} \\
 &= w(\mathbf{w}\mathbf{v} - \mathbf{v} \times \mathbf{u}) + (\mathbf{v} \cdot \mathbf{u})\mathbf{u} + \mathbf{u} \times (\mathbf{w}\mathbf{v} - \mathbf{v} \times \mathbf{u}) \\
 &= w^2\mathbf{v} - \mathbf{w}\mathbf{v} \times \mathbf{u} + (\mathbf{v} \cdot \mathbf{u})\mathbf{u} + \mathbf{u} \times \mathbf{w}\mathbf{v} + \mathbf{u} \times (\mathbf{u} \times \mathbf{v}) \\
 &= w^2\mathbf{v} + \mathbf{u} \times \mathbf{w}\mathbf{v} + (\mathbf{v} \cdot \mathbf{u})\mathbf{u} + \mathbf{u} \times \mathbf{w}\mathbf{v} + \mathbf{u} \times (\mathbf{u} \times \mathbf{v}) \\
 &= w^2\mathbf{v} + 2(\mathbf{u} \times \mathbf{w}\mathbf{v}) + (\mathbf{v} \cdot \mathbf{u})\mathbf{u} + \mathbf{u} \times (\mathbf{u} \times \mathbf{v}) \\
 &= w^2\mathbf{v} + 2(\mathbf{u} \times \mathbf{w}\mathbf{v}) + (\mathbf{v} \cdot \mathbf{u})\mathbf{u} + (\mathbf{u} \cdot \mathbf{v})\mathbf{u} - (\mathbf{u} \cdot \mathbf{u})\mathbf{v} \\
 &= (w^2 - \mathbf{u} \cdot \mathbf{u})\mathbf{v} + 2w(\mathbf{u} \times \mathbf{v}) + 2(\mathbf{u} \cdot \mathbf{v})\mathbf{u} \\
 &= (w^2 - \mathbf{u} \cdot \mathbf{u})\mathbf{v} + 2(\mathbf{u} \cdot \mathbf{v})\mathbf{u} + 2w(\mathbf{u} \times \mathbf{v})
 \end{aligned}$$

여기서 3중곱 항등식  $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \cdot \mathbf{c})\mathbf{b} - (\mathbf{a} \cdot \mathbf{b})\mathbf{c}$ 을  $\mathbf{u} \times (\mathbf{u} \times \mathbf{v})$ 에 적용하였습니다.

다음과 같이 증명하였습니다:

$$\mathbf{q}\mathbf{p}\mathbf{q}^* = \left( (w^2 - \mathbf{u} \cdot \mathbf{u})\mathbf{v} + 2(\mathbf{u} \cdot \mathbf{v})\mathbf{u} + 2w(\mathbf{u} \times \mathbf{v}), 0 \right) \text{ (eq. 24.1)}$$

이 결과는 벡터나 점으로 나타남을 알 수 있습니다. 실수 성분이 0이기 때문이며(이 연산자가 벡터나 점을 회전시키려면 반드시 벡터나 점으로 평가되어야 함), 따라서 이후 방정식에서는 실수 성분을 생략합니다.

이제  $\mathbf{q}$ 가 단위 사원수이므로 다음과 같이 쓸 수 있다.

$$\mathbf{q} = (\sin\theta\mathbf{n}, \cos\theta) \quad \text{for} \quad \|\mathbf{n}\|=1 \quad \text{and} \quad \theta \in [0, \pi]$$

이를 방정식 24.1에 대입하면 다음과 같습니다:

$$\begin{aligned}
 \mathbf{q}\mathbf{p}\mathbf{q}^* &= (\cos^2\theta - \sin^2\theta)\mathbf{v} + 2(\sin\theta\mathbf{n} \cdot \mathbf{v})\sin\theta\mathbf{n} + 2\cos\theta(\sin\theta\mathbf{n} \times \mathbf{v}) \\
 &= (\cos^2\theta - \sin^2\theta)\mathbf{v} + 2\sin^2\theta(\mathbf{n} \cdot \mathbf{v})\mathbf{n} + 2\cos\theta\sin\theta(\mathbf{n} \times \mathbf{v})
 \end{aligned}$$

이를 더 단순화하기 위해 삼각함수 항등식을 적용합니다:

$$\cos^2\theta - \sin^2\theta = \cos(2\theta)$$

$$2\cos\theta\sin\theta = \sin(2\theta)$$

$$\cos(2\theta) = 1 - 2\sin^2\theta$$

$$\begin{aligned}
 \mathbf{q}\mathbf{p}\mathbf{q}^* &= (\cos^2\theta - \sin^2\theta)\mathbf{v} + 2\sin^2\theta(\mathbf{n} \cdot \mathbf{v})\mathbf{n} + 2\cos\theta\sin\theta(\mathbf{n} \times \mathbf{v}) \\
 &= \cos(2\theta)\mathbf{v} + (1 - \cos(2\theta))(\mathbf{n} \cdot \mathbf{v})\mathbf{n} + \sin(2\theta)(\mathbf{n} \times \mathbf{v})
 \end{aligned}$$

(식 24.2)

이제 방정식 24.2를 축-각도 회전 방정식 3.5와 비교해 보면, 이것이 바로 회전 공식  $\mathbf{R}_{\mathbf{n}}(\mathbf{v})$ 임을 알 수 있습니다. 즉, 벡터(또는 점)  $\mathbf{v}$ 를 축  $\mathbf{n}$ 을 중심으로 각도  $2\theta$ 만큼 회전시키는 것입니다.

$$\mathbf{R}_{\mathbf{n}}(\mathbf{v}) = \cos\theta\mathbf{v} + (1 - \cos\theta)(\mathbf{n} \cdot \mathbf{v})\mathbf{n} + \sin\theta(\mathbf{n} \times \mathbf{v})$$

따라서, 우리는 사원수 회전 연산자를 다음과 같이 정의한다:

$$\begin{aligned}
 R_{\mathbf{q}}(\mathbf{v}) &= \mathbf{q}\mathbf{v}\mathbf{q}^{-1} \\
 &= \mathbf{q}\mathbf{v}\mathbf{q}^* \\
 &= \cos(2\theta)\mathbf{v} + (1 - \cos(2\theta))(\mathbf{n} \cdot \mathbf{v})\mathbf{n} + \sin(2\theta)(\mathbf{n} \times \mathbf{v})
 \end{aligned}
 \tag{eq. 24.3}$$

우리는 사원수 회전 연산자  $R_{\mathbf{q}}(\mathbf{v}) = \mathbf{q}\mathbf{v}\mathbf{q}^{-1}$ 가 벡터(또는 점)  $\mathbf{v}$ 를 축  $\mathbf{n}$ 을 중심으로 각도  $2\theta$ 만큼 회전시킨다는 것을 보여주었다.

따라서 축  $\mathbf{n}$ 과 축  $\mathbf{n}$ 을 중심으로 회전할 각도  $\theta$ 가 주어졌다고 가정합니다. 이에 대응하는 회전 쿼터니언은 다음과 같이 구성합니다:

$$\mathbf{q} = \left( \sin\left(\frac{\theta}{2}\right)\mathbf{n}, \cos\left(\frac{\theta}{2}\right) \right)$$

그런 다음 다음 공식을 적용합니다:  $R_{\mathbf{q}}(\mathbf{v})$ . 2로 나누는 것은  $2\theta$ 를 보상하기 위한 것입니다. 왜냐하면 우리는  $2\theta$ 가 아닌  $\theta$  각도로 회전시키고 싶기 때문입니다.

### 24.3.2 쿼터니언 회전 연산자를 행렬로 변환

$\mathbf{q} = (\mathbf{u}, w) = (q_1, q_2, q_3, q_4)$ 를 단위 쿼터니언이라 하자. 방정식 24.1로부터 우리는

$$\mathbf{r} = R_{\mathbf{q}}(\mathbf{v}) = \mathbf{q}\mathbf{p}\mathbf{q}^* = (w^2 - \mathbf{u} \cdot \mathbf{u})\mathbf{v} + 2(\mathbf{u} \cdot \mathbf{v})\mathbf{u} + 2w(\mathbf{u} \times \mathbf{v})$$

$$q_1^2 + q_2^2 + q_3^2 + q_4^2 = 1 \Rightarrow q_4^2 - 1 = -q_1^2 - q_2^2 - q_3^2 \text{을 함의하므로, 따라서}$$

$$\begin{aligned}
 (w^2 - \mathbf{u} \cdot \mathbf{u})\mathbf{v} &= (q_4^2 - q_1^2 - q_2^2 - q_3^2)\mathbf{v} \\
 &= (2q_4^2 - 1)\mathbf{v}
 \end{aligned}$$

$R_{\mathbf{q}}(\mathbf{v})$ 의 세 항은 행렬로 표현할 수 있다:

$$(w^2 - \mathbf{u} \cdot \mathbf{u})\mathbf{v} = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \begin{bmatrix} 2q_4^2 - 1 & 0 & 0 \\ 0 & 2q_4^2 - 1 & 0 \\ 0 & 0 & 2q_4^2 - 1 \end{bmatrix}$$

$$2(\mathbf{u} \cdot \mathbf{v})\mathbf{u} = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \begin{bmatrix} 2q_1^2 & 2q_1q_2 & 2q_1q_3 \\ 2q_1q_2 & 2q_2^2 & 2q_2q_3 \\ 2q_1q_3 & 2q_2q_3 & 2q_3^2 \end{bmatrix}$$

$$2w(\mathbf{u} \times \mathbf{v}) = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \begin{bmatrix} 0 & 2q_4q_3 & -2q_4q_2 \\ -2q_4q_3 & 0 & 2q_4q_1 \\ 2q_4q_2 & -2q_4q_1 & 0 \end{bmatrix}$$

항들을 합하면 다음과 같습니다:

$$R_q(\mathbf{v}) = \mathbf{v}Q = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \begin{bmatrix} 2q_1^2 + 2q_4^2 - 1 & 2q_1q_2 + 2q_3q_4 & 2q_1q_3 - 2q_2q_4 \\ 2q_1q_2 - 2q_3q_4 & 2q_2^2 + 2q_4^2 - 1 & 2q_2q_3 + 2q_1q_4 \\ 2q_1q_3 + 2q_2q_4 & 2q_2q_3 - 2q_1q_4 & 2q_3^2 + 2q_4^2 - 1 \end{bmatrix}$$

$\mathbf{q}$ 의 단위 길이 성질  $q_1^2 + q_2^2 + q_3^2 + q_4^2 = 1$ 은 다음을 함의합니다:

$$2q_1^2 + 2q_4^2 = 2 - 2q_2^2 - 2q_3^2$$

$$2q_2^2 + 2q_4^2 = 2 - 2q_1^2 - 2q_3^2$$

$$2q_3^2 + 2q_4^2 = 2 - 2q_1^2 - 2q_2^2$$

따라서 이 행렬 방정식을 다음과 같이 다시 쓸 수 있습니다:

$$R_q(\mathbf{v}) = \mathbf{v}Q = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \begin{bmatrix} 1 - 2q_2^2 - 2q_3^2 & 2q_1q_2 + 2q_3q_4 & 2q_1q_3 - 2q_2q_4 \\ 2q_1q_2 - 2q_3q_4 & 1 - 2q_1^2 - 2q_3^2 & 2q_2q_3 + 2q_1q_4 \\ 2q_1q_3 + 2q_2q_4 & 2q_2q_3 - 2q_1q_4 & 1 - 2q_1^2 - 2q_2^2 \end{bmatrix} \quad (\text{식 24.4})$$

많은 그래픽스 교재에서는 벡터 변환에 행렬-열 벡터 순서를 사용합니다. 따라서 많은 그래픽스 교재에서 행렬  $\mathbf{Q}$ 의 전치 행렬을 사용하여

$$R_q(\mathbf{v}) = \mathbf{Q}^T \mathbf{v}^T.$$

### 24.3.3 행렬에서 쿼터니언 회전 연산자로

주어진 회전 행렬

$$\mathbf{R} = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix}$$

우리는 쿼터니언  $\mathbf{q} = (q_1, q_2, q_3, q_4)$ 를 찾고자 한다. 즉,  $\mathbf{q}$ 로부터 방정식 24.4의 행렬  $\mathbf{Q}$ 를 구성했을 때  $\mathbf{R}$ 을 얻도록 하는 것이다. 따라서 우리의 전략은

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix} = \begin{bmatrix} 1 - 2q_2^2 - 2q_3^2 & 2q_1q_2 + 2q_3q_4 & 2q_1q_3 - 2q_2q_4 \\ 2q_1q_2 - 2q_3q_4 & 1 - 2q_1^2 - 2q_3^2 & 2q_2q_3 + 2q_1q_4 \\ 2q_1q_3 + 2q_2q_4 & 2q_2q_3 - 2q_1q_4 & 1 - 2q_1^2 - 2q_2^2 \end{bmatrix}$$

$q_1, q_2, q_3, q_4$ 에 대해 풀기 위한 것입니다.  $\mathbf{R}$ 이 주어졌으므로 방정식의 좌변에 있는 모든 성분은 알려져 있습니다.

먼저 대각선 요소들을 합산합니다(이는 행렬의 트레이스라고 합니다):

$$\begin{aligned}
\text{trace}(\mathbf{R}) &= R_{11} + R_{22} + R_{33} \\
&= 1 - 2q_2^2 - 2q_3^2 + 1 - 2q_1^2 - 2q_3^2 + 1 - 2q_1^2 - 2q_2^2 \\
&= 3 - 4q_1^2 - 4q_2^2 - 4q_3^2 \\
&= 3 - 4(q_1^2 + q_2^2 + q_3^2) \\
&= 3 - 4(1 - q_4^2) \\
&= -1 + 4q_4^2 \\
\therefore q_4 &= \frac{\sqrt{\text{trace}(\mathbf{R}) + 1}}{2}
\end{aligned}$$

이제 대각선 반대편 원소들을 결합하여  $q_1, q_2, q_3$ 를 구합니다(항을 제거하기 때문입니다):

$$\begin{aligned}
R_{23} - R_{32} &= 2q_2q_3 + 2q_1q_4 - 2q_2q_3 + 2q_1q_4 \\
&= 4q_1q_4 \\
\therefore q_1 &= \frac{R_{23} - R_{32}}{4q_4}
\end{aligned}$$

$$\begin{aligned}
R_{31} - R_{13} &= 2q_1q_3 + 2q_2q_4 - 2q_1q_3 + 2q_2q_4 \\
&= 4q_2q_4 \\
\therefore q_2 &= \frac{R_{31} - R_{13}}{4q_4}
\end{aligned}$$

$$\begin{aligned}
R_{12} - R_{21} &= 2q_1q_2 + 2q_3q_4 - 2q_1q_2 + 2q_3q_4 \\
&= 4q_3q_4 \\
\therefore q_3 &= \frac{R_{12} - R_{21}}{4q_4}
\end{aligned}$$

$q_4 = 0$  일 때 이 방정식들은 정의되지 않습니다. 이 경우,  $\mathbf{R}$  행렬의 대각선 요소 중 가장 큰 값을 나누는 데 사용하고, 행렬 요소의 다른 조합을 선택합니다.  $R_{11}$ 이 최대 대각선 요소라고 가정하면:

$$\begin{aligned}
R_{11} - R_{22} - R_{33} &= 1 - 2q_2^2 - 2q_3^2 - 1 + 2q_1^2 + 2q_3^2 - 1 + 2q_1^2 + 2q_2^2 \\
&= -1 + 4q_1^2 \\
\therefore q_1 &= \frac{\sqrt{R_{11} - R_{22} - R_{33} + 1}}{2}
\end{aligned}$$

$$\begin{aligned}
 R_{12} + R_{21} &= 2q_1q_2 + 2q_3q_4 + 2q_1q_2 - 2q_3q_4 \\
 &= 4q_1q_2 \\
 \therefore q_2 &= \frac{R_{12} + R_{21}}{4q_1} \\
 R_{13} + R_{31} &= 2q_1q_3 - 2q_2q_4 + 2q_1q_3 + 2q_2q_4 \\
 &= 4q_1q_3 \\
 \therefore q_3 &= \frac{R_{13} + R_{31}}{4q_1} \\
 R_{23} - R_{32} &= 2q_2q_3 + 2q_1q_4 - 2q_2q_3 + 2q_1q_4 \\
 &= 4q_1q_4 \\
 \therefore q_4 &= \frac{R_{23} - R_{32}}{4q_1}
 \end{aligned}$$

$R_{22}$  또는  $R_{33}$ 이 최대 대각선 요소인 경우에도 유사한 패턴을 적용합니다.

### 24.3.4 합성

$\mathbf{p}$ 와  $\mathbf{q}$ 가 각각  $R_p$ 와  $R_q$ 로 주어진 회전 연산자를 가진 단위 사원수라고 가정하자.  $\mathbf{v}' = R_p(\mathbf{v})$ 로 두면, 합성은 다음과 같이 주어진다:

$$R_q(R_p(\mathbf{v})) = R_q(\mathbf{v}') = \mathbf{q}\mathbf{v}'\mathbf{q}^{-1} = \mathbf{q}(\mathbf{p}\mathbf{v}\mathbf{p}^{-1})\mathbf{q}^{-1} = (\mathbf{qp})\mathbf{v}(\mathbf{p}^{-1}\mathbf{q}^{-1}) = (\mathbf{qp})\mathbf{v}(\mathbf{qp})^{-1}$$

$\mathbf{p}$ 와  $\mathbf{q}$ 가 모두 단위 사원수이므로, 곱  $\mathbf{pq}$  역시 단위 사원수이다. 왜냐하면

; 따라서 사원수

곱  $\mathbf{pq}$  역시 회전을 나타내며, 즉 합성 연산으로 주어지는 순회전량이다

$$\|\mathbf{pq}\| = \|\mathbf{p}\| \|\mathbf{q}\| = 1$$

에 의해 주어지는 순회전을 나타

낸다.

$$R_q(R_p(\mathbf{v}))$$

## 24.4 사원수 보간법

쿼터니언은 실수의 4원조이므로 기하학적으로 4차원 벡터로 시각화할 수 있습니다. 특히 단위 쿼터니언은 4차원 단위 구 위에 놓인 4차원 단위 벡터입니다. 외적(3차원 벡터에만 정의됨)을 제외하면, 우리의 벡터 연산은 4차원 공간, 나아가  $n$ 차원 공간으로 일반화됩니다. 구체적으로, 내적은 쿼터니언에 대해 성립합니다.

$$\mathbf{p} = (\mathbf{u}, s) \text{ 이고 } \mathbf{q} = (\mathbf{v}, t) \text{ 그러면:}$$

$$\mathbf{p} \cdot \mathbf{q} = \mathbf{u} \cdot \mathbf{v} + st = \|\mathbf{p}\| \|\mathbf{q}\| \cos \theta$$

여기서  $\theta$ 는 두 쿼터니언 사이의 각도입니다. 쿼터니언  $\mathbf{p}$ 와  $\mathbf{q}$ 가 단위 길이이면,  $\mathbf{p} \cdot \mathbf{q} = \cos \theta$ 입니다. 내적은 두 쿼터니언 사이의 각도를 단위 구상에서 서로 얼마나 "가까운"지를 측정하는 척도로 설명할 수 있게 해줍니다.

애니메이션을 위해 우리는 한 방향으로 보간하고자 합니다. 사원수를 보간하기 위해 우리는 단위 구의 호 상에서 보간하여, 보간된 사원수도 단위 사원수가 되도록 하고자 합니다.

이러한 공식을 도출하기 위해 그림 24.6을 고려하자. 여기서 우리는  $\mathbf{a}$ 에서  $\mathbf{b}$  사이를 각도  $t\theta$ 로 보간하고자 한다. 우리는 다음과 같은 가중치  $c_1$ 과  $c_2$ 를 구하고자 한다.

$c_1$  및  $c_2$ 를 구하고자 한다.

$$\mathbf{p} = c_1\mathbf{a} + c_2\mathbf{b} \text{ 여기서 } \|\mathbf{p}\| = \|\mathbf{a}\| = \|\mathbf{b}\| \text{ 두 미지수에 대해 다음과 같이 두 방정식을 세웁니다:}$$

$$\mathbf{a} \cdot \mathbf{p} = c_1\mathbf{a} \cdot \mathbf{a} + c_2\mathbf{a} \cdot \mathbf{b}$$

$$\cos(t\theta) = c_1 + c_2 \cos(\theta)$$

$$\mathbf{p} \cdot \mathbf{b} = c_1 \mathbf{a} \cdot \mathbf{b} + c_2 \mathbf{b} \cdot \mathbf{b}$$

$$\cos((1-t)\theta) = c_1 \cos(\theta) + c_2$$

이를 통해 행렬 방정식이 도출됩니다:

$$\begin{bmatrix} 1 & \cos(\theta) \\ \cos(\theta) & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} \cos(t\theta) \\ \cos((1-t)\theta) \end{bmatrix}$$

행렬 방정식  $\mathbf{Ax} = \mathbf{b}$  를 고려하자. 여기서  $\mathbf{A}$  는 가역 행렬이다. 그러면 크래머의 법칙에 따라  $x_i = \det \mathbf{A}_i / \det \mathbf{A}$  이 성립한다. 여기서  $\mathbf{A}_{(i)}$  는  $\mathbf{A}$  의  $i$  번째 열 벡터를  $\mathbf{b}$  와 교환하여 구한다. 따라서:

$$c_1 = \frac{\det \begin{bmatrix} \cos(t\theta) & \cos(\theta) \\ \cos((1-t)\theta) & 1 \end{bmatrix}}{\det \begin{bmatrix} 1 & \cos(\theta) \\ \cos(\theta) & 1 \end{bmatrix}} = \frac{\cos(t\theta) - \cos(\theta) \cos((1-t)\theta)}{1 - \cos^2(\theta)}$$

$$c_2 = \frac{\det \begin{bmatrix} 1 & \cos(t\theta) \\ \cos(\theta) & \cos((1-t)\theta) \end{bmatrix}}{\det \begin{bmatrix} 1 & \cos(\theta) \\ \cos(\theta) & 1 \end{bmatrix}} = \frac{\cos((1-t)\theta) - \cos(\theta) \cos(t\theta)}{1 - \cos^2(\theta)}$$

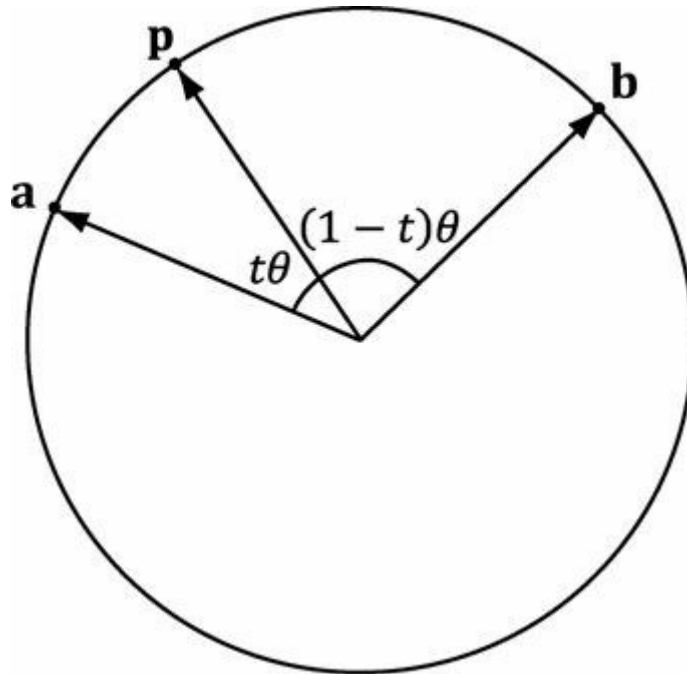


그림 24.6. 4차원 단위 구상에서 a에서 b까지 각도  $t\theta$ 로 보간하는 과정. a와 b 사이의 각도는  $\theta$ , a와 p 사이의 각도는  $t\theta$ , p와 b 사이의 각도는  $(1-t)\theta$ 이다.

삼각함수 피타고라스 항등식과 합성 공식으로부터 다음을 얻습니다:

$$1 - \cos^2(\theta) = \sin^2(\theta)$$

$$\cos((1-t)\theta) = \cos(\theta - t\theta) = \cos(\theta)\cos(t\theta) + \sin(\theta)\sin(t\theta)$$

$$\sin((1-t)\theta) = \sin(\theta - t\theta) = \sin(\theta)\cos(t\theta) - \cos(\theta)\sin(t\theta)$$

따라서,

$$\begin{aligned} c_1 &= \frac{\cos(t\theta) - \cos(\theta) [\cos(\theta)\cos(t\theta) + \sin(\theta)\sin(t\theta)]}{\sin^2(\theta)} \\ &= \frac{\cos(t\theta) - \cos(\theta)\cos(\theta)\cos(t\theta) - \cos(\theta)\sin(\theta)\sin(t\theta)}{\sin^2(\theta)} \\ &= \frac{\cos(t\theta)(1 - \cos^2(\theta)) - \cos(\theta)\sin(\theta)\sin(t\theta)}{\sin^2(\theta)} \\ &= \frac{\cos(t\theta)\sin^2(\theta) - \cos(\theta)\sin(\theta)\sin(t\theta)}{\sin^2(\theta)} \\ &= \frac{\sin(\theta)\cos(t\theta) - \cos(\theta)\sin(t\theta)}{\sin(\theta)} \\ &= \frac{\sin((1-t)\theta)}{\sin(\theta)} \end{aligned}$$

그리고

$$\begin{aligned} c_2 &= \frac{\cos(\theta)\cos(t\theta) + \sin(\theta)\sin(t\theta) - \cos(\theta)\cos(t\theta)}{\sin^2(\theta)} \\ &= \frac{\sin(t\theta)}{\sin(\theta)} \end{aligned}$$

따라서 구형 보간 공식은 다음과 같이 정의됩니다.

$$\text{slerp}(\mathbf{a}, \mathbf{b}, t) = \frac{\sin((1-t)\theta)\mathbf{a} + \sin(t\theta)\mathbf{b}}{\sin\theta} \quad \text{for } t \in [0, 1]$$

단위 쿼터니언을 4차원 단위 벡터로 생각하면 쿼터니언 사이의 각도를 구할 수 있습니다.  $\theta = \arccos(\mathbf{a} \cdot \mathbf{b})$

$\mathbf{a}$ 와  $\mathbf{b}$  사이의 각도  $\theta$ 가 0에 가까우면  $\sin \theta$ 도 0에 가까워지며, 유한한 수치 정밀도로 인해 나눗셈 문제가 발생할 수 있습니다

정밀도로 인해 문제가 발생할 수 있습니다. 이 경우, 쿼터니언 사이에 선형 보간을 수행하고 결과를 정규화하십시오. 이는 실제로 작은  $\theta$ 에 대해 좋은 근사값입니다([그림 24.7 참조](#)).

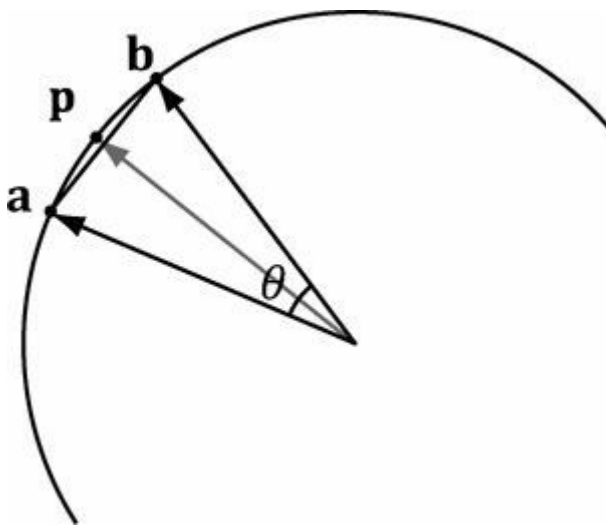


그림 24.7. a와 b 사이의 각도  $\theta$ 가 작을 때, 선형 보간은 구형 보간에 대한 좋은 근사값입니다. 그러나 선형 보간을 사용할 경우, 보간된 쿼터니언은 더 이상 단위 구 위에 위치하지 않으므로, 결과를 정규화하여 다시 단위 구 위에 투영해야 합니다.

그림 24.8에서 볼 수 있듯이, 선형 보간 후 보간된 쿼터니언을 단위 구면에 다시 투영하면 비선형 회전 속도가 발생합니다. 따라서 큰 각도에 선형 보간을 사용하면 회전 속도가 가속 및 감속됩니다. 이 효과는 종종 바람직하지 않으며, 일정한 속도로 회전하는 구형 보간이 선호되는 이유 중 하나입니다.

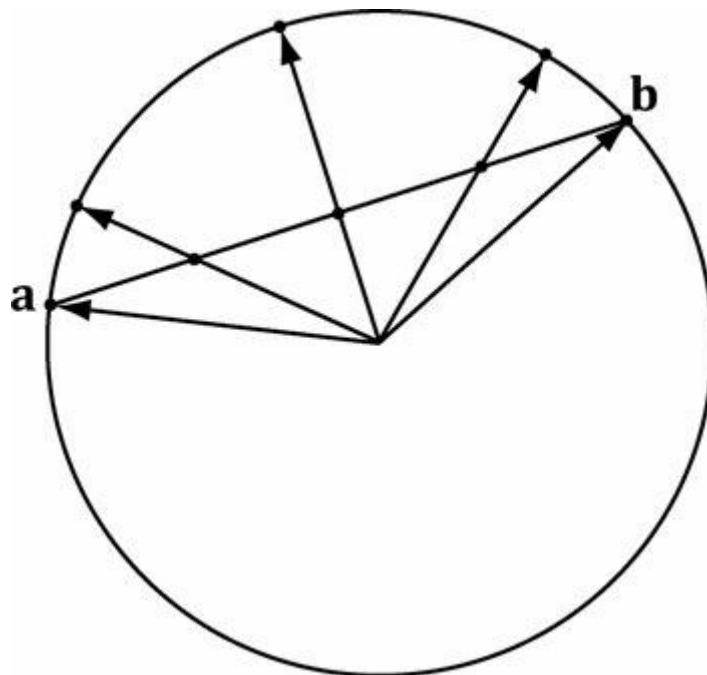


그림 24.8. 선형 보간은 정규화 후 단위 구면 상에서 비선형 보간을 초래합니다. 이는 보간 과정에서 회전 속도가 일정하지 않고 가속 및 감속함을 의미합니다.

이제 사원수의 흥미로운 성질을 지적하고자 합니다.  $(s\mathbf{q})^* = s\mathbf{q}^*$ 이며 스칼라-사원수 곱셈이 교환법칙을 따르므로 다음이 성립합니다:

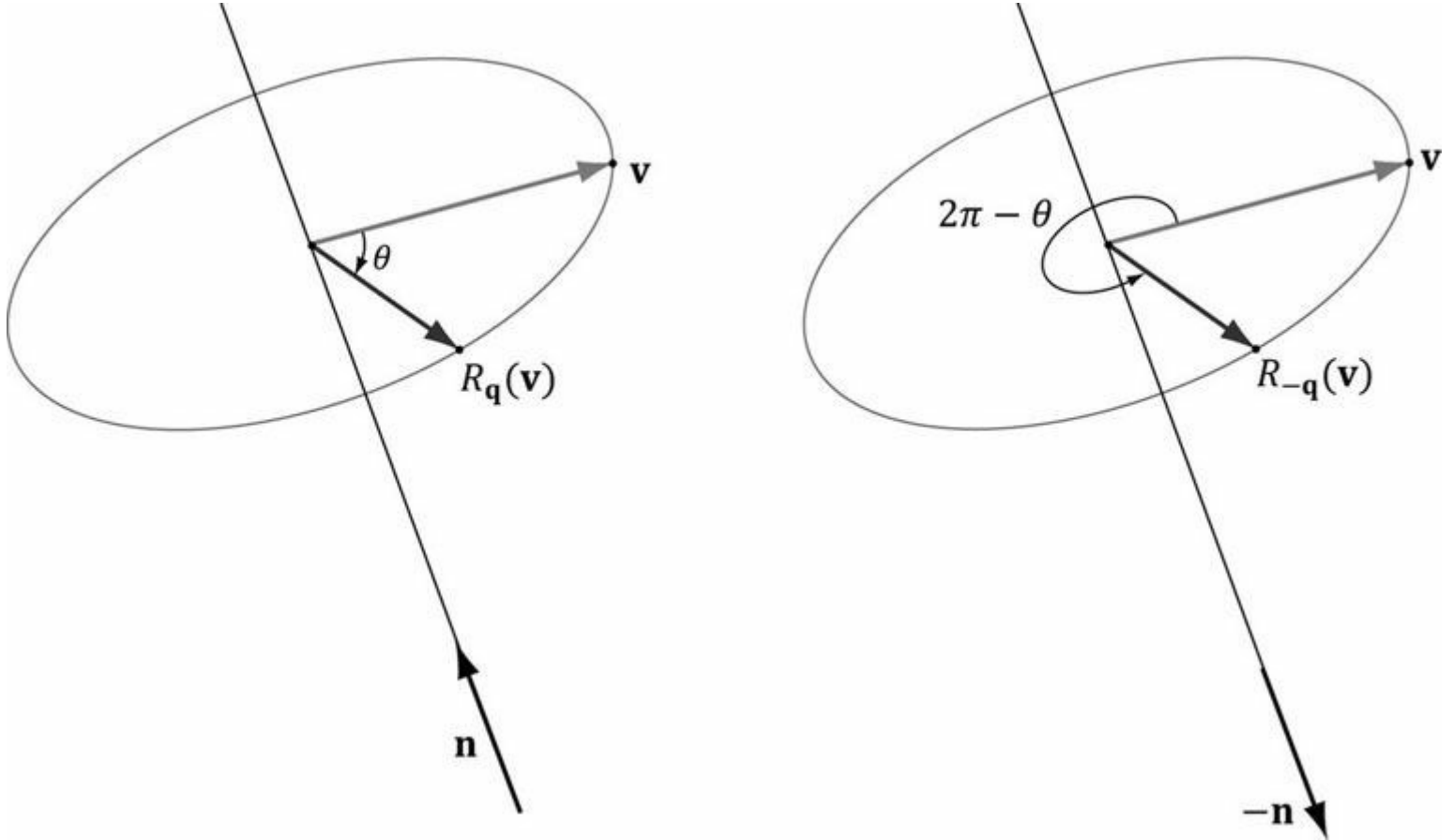
$$\begin{aligned} R_{-\mathbf{q}}(\mathbf{v}) &= -\mathbf{q}\mathbf{v}(-\mathbf{q})^* \\ &= (-1)\mathbf{q}\mathbf{v}(-1)\mathbf{q}^* \\ &= \mathbf{q}\mathbf{v}\mathbf{q}^* \end{aligned}$$

따라서  $\mathbf{q}$ 와  $-\mathbf{q}$ 는 동일한 회전을 나타냅니다. 다른 방식으로 이를 확인하려면, 만약

$$\mathbf{q} = \left( n \sin \frac{\theta}{2}, \cos \frac{\theta}{2} \right), \text{ 그러면}$$

$$\begin{aligned}
 -\mathbf{q} &= \left( -\mathbf{n} \sin \frac{\theta}{2}, -\cos \frac{\theta}{2} \right) \\
 &= \left( -\mathbf{n} \sin \left( \pi - \frac{\theta}{2} \right), \cos \left( \pi - \frac{\theta}{2} \right) \right) \\
 &= \left( -\mathbf{n} \sin \left( \frac{2\pi - \theta}{2} \right), \cos \left( \frac{2\pi - \theta}{2} \right) \right)
 \end{aligned}$$

즉,  $R_{\mathbf{q}}$ 는 축  $\mathbf{n}$ 을 중심으로  $\theta$ 만큼 회전시키고,  $R_{-\mathbf{q}}$ 는 축  $-\mathbf{n}$ 을 중심으로  $2\pi - \theta$ 만큼 회전시킵니다. 기하학적으로, 4차원 단위 구상의 단위 사원수  $\mathbf{q}$ 와 그 극대칭인  $-\mathbf{q}$ 는 동일한 방향성을 나타냅니다. [그림 24.9](#)는 이 두 회전이 동일한 위치로 이동함을 보여줍니다. 그러나 한 회전은 더 짧은 각도로, 다른 회전은 더 긴 각도로 이동함을 알 수 있습니다.



**그림 24.9.**  $R_{\mathbf{q}}$ 는 축  $\mathbf{n}$ 을 중심으로  $\theta$ 만큼 회전시키고,  $R_{-\mathbf{q}}$ 는 축  $-\mathbf{n}$ 을 중심으로  $2\pi - \theta$ 만큼 회전시킨다.

$\mathbf{b}$ 와  $-\mathbf{b}$ 는 동일한 방향을 나타내므로 보간법으로  $\text{slerp}(\mathbf{a}, \mathbf{b}, t)$  또는  $\text{slerp}(\mathbf{a}, -\mathbf{b}, t)$  중 하나를 선택할 수 있습니다. 하나는 회전을 최소화하는 가장 직접적인 방식으로 방향 사이를 보간하고([그림 24.9a](#)와 유사), 다른 하나는 우회 경로를 취합니다([그림 24.9b](#)와 유사). [그림 24.10](#)을 참조하면, 4차원 단위 구면 상에서 더 짧은 호를 따라 보간하는 쪽을 기준으로  $\mathbf{b}$  또는  $-\mathbf{b}$ 를 선택해야 합니다. 더 짧은 호를 선택하면 가장 직접적인 경로를 통해 보간하게 되며, 더 긴 호를 선택하면 물체가 먼 길을 돌아가며 회전하게 되어 추가적인 회전(스핀닝)이 발생합니다 [Eberly01].

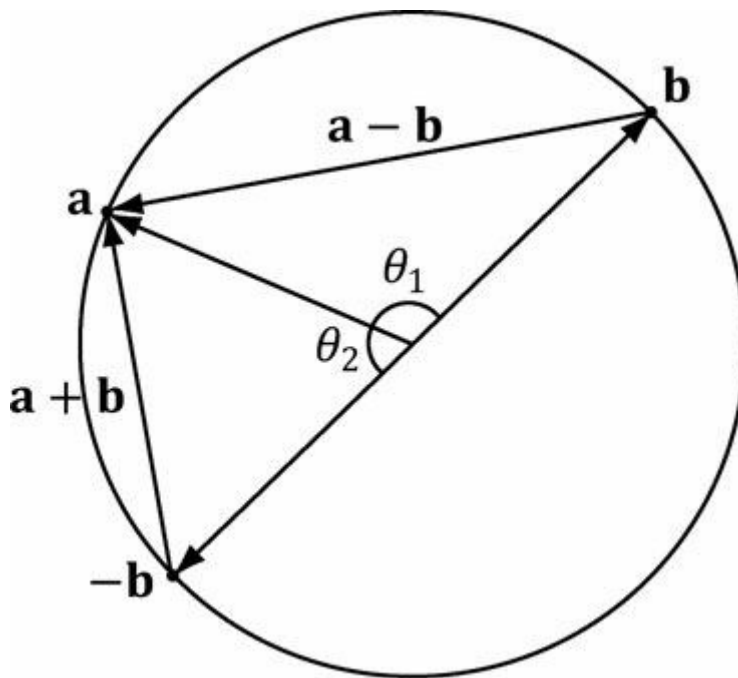


그림 24.10. a에서 b로 보간하면 4차원 단위 구면 상의 더 큰 호  $\theta_1$ 을 따라 보간되는 반면, a에서 -b로 보간하면 4차원 단위 구면 상의 더 짧은 호  $\theta_2$ 를 따라 보간된다. 우리는 4차원 단위 구면 상에서 가장 짧은 호를 선택하고자 한다.

[Watt92]에 따르면, 4차원 단위 구면에서 가장 짧은 호를 제공하는 사원수를 찾기 위해 다음을 비교합니다:  $\|\mathbf{a} - \mathbf{b}\|^2$  그리고  $\|\mathbf{a} - (-\mathbf{b})\|^2 = \|\mathbf{a} + \mathbf{b}\|^2$ 을 비교합니다. 만약  $\|\mathbf{a} + \mathbf{b}\|^2 < \|\mathbf{a} - \mathbf{b}\|^2$  라면, -b가 a에 더 가깝고 따라서 더 짧은 호를 제공하므로 보간을 위해 b 대신 -b를 선택합니다.

```
// 선형 보간법 (작은 theta에 대해).
public static Quaternion LerpAndNormalize(Quaternion p, Quaternion q, float s)
{
 // 단위 쿼터니언이 되도록 정규화합니다. return Normalize((1.0f - s)*p + s*q);
}

public static Quaternion Slerp(Quaternion p, Quaternion q, float s)
{
 // q와 -q는 동일한 방향을 나타내지만, 두 방향 사이를 보간할 때
 // 두 방향 사이의 보간은 다릅니다: 하나는 가장 짧은 호를, 다른 하나는
 // 긴 호를 취합니다. 가장 짧은 호를 찾으려면 p-q의 크기가 p+q인
 p-q의 크기와 p-(-q) = p+q의 크기를 비교합니다.

 if(LengthSq(p-q) > LengthSq(p+q)) q = -q;

 float cosPhi = DotP(p, q);
 // 매우 작은 각도의 경우 선형 보간법을 사용합니다. if(cosPhi > (1.0f - 0.001))
 return LerpAndNormalize(p, q, s);

 // 두 쿼터니언 사이의 각도를 구합니다. float phi =
 (float)Math.Acos(cosPhi);

 float sinPhi = (float)Math.Sin(phi);

 // 4차원 단위 구와 단위 구의 원점을 지나는 평면이 교차하여 형성된 호를 따라 보간합니다.
 // p, q 및 단위 구의 원점을 지나는 평면이 교차하여 형성된 호를 따라 보간합니다. return
 ((float)Math.Sin(phi*(1.0-s))/sinPhi)*p +
 ((float)Math.Sin(phi*s)/sinPhi)*q;
}
```

## 24.5 XNA 수학 쿼터니언 함수

XNA 수학 라이브러리는 쿼터니언을 지원합니다. 쿼터니언의 "데이터"가 4개의 실수이므로, XNA 수학은 쿼터니언 저장을 위해 **XMVECTOR** 타입을 사용합니다. 정의된 일반적인 쿼터니언 함수 중 일부는 다음과 같습니다:

```
// 쿼터니언 내적을 반환합니다.
XMVECTOR XMQuaternionDot(XMVECTOR Q1, XMVECTOR Q2);

// 신분 사원수를 반환합니다.
XMVECTOR XMQuaternionIdentity();

// 쿼터니언 Q의 공액체를 반환합니다.
XMVECTOR XMQuaternionConjugate(XMVECTOR Q);

// 쿼터니언 Q의 노름을 반환합니다.
XMVECTOR XMQuaternionLength(XMVECTOR Q);

// 쿼터니언을 4차원 벡터로 간주하여 정규화합니다.
XMVECTOR XMQuaternionNormalize(XMVECTOR Q);

// 쿼터니언 곱 $Q_1 Q_2$ 을 계산합니다.
XMVECTOR XMQuaternionMultiply(XMVECTOR Q1, XMVECTOR Q2);

// 축-각도 회전 표현으로부터 쿼터니언을 반환합니다.
XMVECTOR XMQuaternionRotationAxis(XMVECTOR Axis, FLOAT Angle);

// 축-각도 회전 표현에서 쿼터니언을 반환합니다. 여기서 축
// 벡터가 정규화된 상태입니다. XMQuaternionRotationAxis보다 빠릅니다. XMVECTOR XMQuaternionRotationNormal(XMVECTOR
NormalAxis, FLOAT Angle);

// 회전 행렬로부터 쿼터니언을 반환합니다.
XMVECTOR XMQuaternionRotationMatrix(XMMATRIX M);

// 단위 쿼터니언으로부터 회전 행렬을 반환합니다.
XMMATRIX XMMatrixRotationQuaternion(XMVECTOR Quaternion);

// 쿼터니언 Q에서 축과 각도 회전 표현을 추출합니다.
VOID XMQuaternionToAxisAngle(XMVECTOR *pAxis, FLOAT *pAngle, XMVECTOR Q);

// slerp(Q1, Q2, t)를 반환합니다.
XMVECTOR XMQuaternionSlerp(XMVECTOR Q0, XMVECTOR Q1, FLOAT t);
```

## 24.6 회전 데모

이 장의 데모에서는 간단한 장면에서 해골 메시를 애니메이션 처리합니다. 메시의 위치, 방향, 크기가 애니메이션됩니다. 해골의 방향을 표현하기 위해 쿼터니언을 사용하고, 방향 간 보간에는 슬립(slerp)을 적용합니다. 위치와 크기 간 보간에는 선형 보간을 사용합니다. 이 데모는 다음 장의 캐릭터 애니메이션을 위한 애니메이션 '준비 운동' 역할도 합니다. 일반적인 애니메이션 형태 중 하나는 키 프레임 애니메이션입니다. *키 프레임*은 특정 시점에서의 객체 위치, 방향, 크기를 지정합니다. 본 데모(*AnimationHelper.h/cpp*)에서는 다음과 같은 키 프레임 구조를 정의합니다:

```
struct Keyframe
{
 Keyframe();
 ~Keyframe();

 float TimePos; XMFLOAT3
 Translation; XMFLOAT3 Scale;
 XMFLOAT4 RotationQuat;
};
```

*애니메이션*은 시간 순으로 정렬된 키 프레임 목록입니다.

```
struct BoneAnimation
{
 float GetStartTime()const;
```

```
float GetEndTime()const;
```

```
void Interpolate(float t, XMFLOAT4X4& M)const; std::vector<Keyframe> Keyframes;
```

```
};
```

"본(bone)"이라는 용어를 사용하는 이유는 다음 섹션에서 명확히 설명될 것입니다. 지금은 단일 본을 애니메이션하는 것을 단일 객체를 애니메이션하는 것으로 생각하시면 됩니다. **GetStartTime** 메서드는 첫 번째 키 프레임의 시간을 반환합니다. 예를 들어, 객체가 타임라인 기준 10초가 지나야 애니메이션을 시작할 수도 있습니다. 마찬가지로 **GetEndTime** 메서드는 마지막 키 프레임의 시간을 반환합니다.

키 프레임의 시간을 반환합니다. 이는 애니메이션이 언제 종료되는지 파악하여 애니메이션을 중지할 수 있게 해주므로 유용합니다.

이제 애니메이션의 대략적인 전체 모습을 정의하는 키 프레임 목록이 있습니다. 그렇다면 키 프레임 사이의 시간  $t$ 에서는 애니메이션이 어떻게 보일까요?

? 여기서 보간이 등장합니다. 두 키 프레임(예:  $K_i$ 와  $K_{i+1}$ ) 사이의 시간  $t$ 에 대해, 우리는 두 키 프레임  $K_i$ 와  $K_{(i+1)}$  사이를 보간합니다.

```
void BoneAnimation::Interpolate(float t, XMFLOAT4X4& M)const
```

```
{
 // t는 애니메이션 시작 전이므로 첫 번째 키 프레임을 반환합니다. if(t <= Keyframes.front().TimePos)
```

```
{
```

```
 XMVECTOR S = XMLoadFloat3(&Keyframes.front().Scale); XMVECTOR P =
 XMLoadFloat3(&Keyframes.front().Translation); XMVECTOR Q =
 XMLoadFloat4(&Keyframes.front().RotationQuat);
```

```
 XMVECTOR zero = XMVectorSet(0.0f, 0.0f, 0.0f, 1.0f); XMStoreFloat4x4(&M,
 XMMatrixAffineTransformation(S, zero, Q, P));
```

```
}
```

```
// t는 애니메이션 종료 후이므로 마지막 키 프레임을 반환합니다. else if(t >=
Keyframes.back().TimePos)
```

```
{
```

```
 XMVECTOR S = XMLoadFloat3(&Keyframes.back().Scale); XMVECTOR P =
 XMLoadFloat3(&Keyframes.back().Translation); XMVECTOR Q =
 XMLoadFloat4(&Keyframes.back().RotationQuat);
```

```
 XMVECTOR zero = XMVectorSet(0.0f, 0.0f, 0.0f, 1.0f); XMStoreFloat4x4(&M,
 XMMatrixAffineTransformation(S, zero, Q, P));
```

```
}
```

```
// t는 두 키 프레임 사이에 있으므로 보간합니다. else
```

```
{
```

```
 for(UINT i = 0; i < Keyframes.size()-1; ++i)
```

```
{
```

```
 if(t >= Keyframes[i].TimePos && t <= Keyframes[i+1].TimePos)
 {
```

```
 float lerpPercent = (t - Keyframes[i].TimePos) / (Keyframes[i+1].TimePos - Keyframes[i].TimePos);
```

```
 XMVECTOR s0 = XMLoadFloat3(&Keyframes[i].Scale); XMVECTOR s1 =
 XMLoadFloat3(&Keyframes[i+1].Scale);
```

```
 XMVECTOR p0 = XMLoadFloat3(&Keyframes[i].Translation); XMVECTOR p1 =
 XMLoadFloat3(&Keyframes[i+1].Translation);
```

```
 XMVECTOR q0 = XMLoadFloat4(&Keyframes[i].RotationQuat); XMVECTOR q1 =
 XMLoadFloat4(&Keyframes[i+1].RotationQuat);
```

```
 XMVECTOR S = XMVectorLerp(s0, s1, lerpPercent); XMVECTOR P = XMVectorLerp(p0, p1,
 lerpPercent); XMVECTOR Q = XMQuaternionSlerp(q0, q1, lerpPercent);
```

```
 XMVECTOR zero = XMVectorSet(0.0f, 0.0f, 0.0f, 1.0f); XMStoreFloat4x4(&M,
 XMMatrixAffineTransformation(S, zero, Q, P));
```

```

 break;
 }
}
}

```

그림 24.11은 키 1에서 키2로보간하여 생성된 중간 프레임을 보여줍니다.

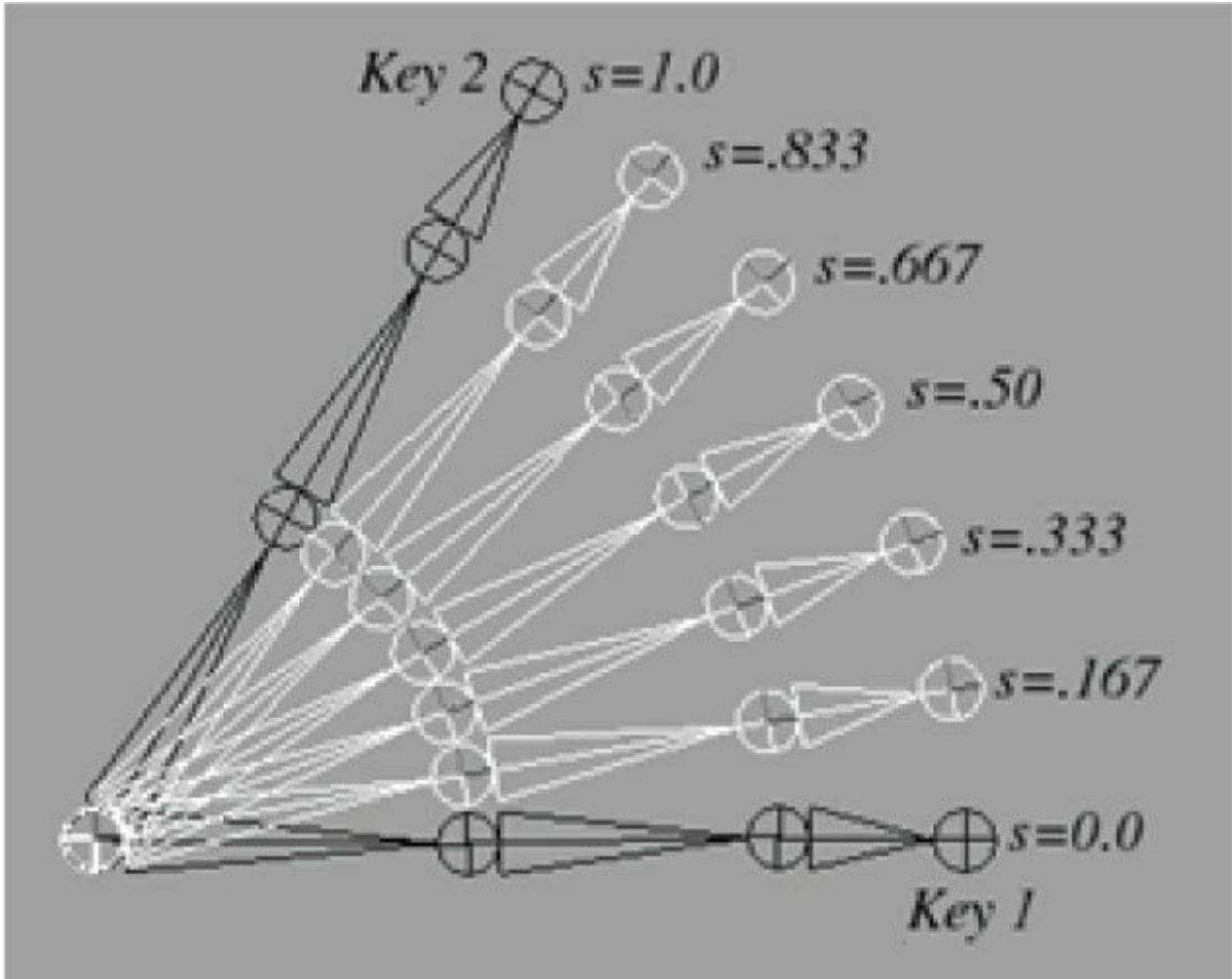


그림 24.11. 키 프레임 보간. 키 프레임은 애니메이션의 "키" 포즈를 정의합니다. 보간된 값은 키 프레임 사이의 값을 나타냅니다.



Permission:

그림 24.11은 Frank D. Luna의 저서 『DirectX 9.0c를 이용한 3D 게임 프로그래밍 입문: 셰이더 접근법』 (2006년, Jones and Bartlett Learning, Burlington, MA)에 수록된 내용입니다. [www.jblearning.com](http://www.jblearning.com). 허가를 받아 재인쇄함.

보간 후에는 변환 행렬을 구성합니다. 이는 셰이더 프로그램에서 변환을 수행할 때 궁극적으로 행렬을 사용하기 때문입니다. **XMMatrixAffineTransformation** 함수는 다음과 같이 선언됩니다.

```

XMMATRIX XMMatrixAffineTransformation(XMVECTOR Scaling,
 XMVECTOR RotationOrigin, XMVECTOR
 RotationQuaternion, XMVECTOR Translation);

```

이제 간단한 애니메이션 시스템이 구축되었으므로, 데모의 다음 단계는 몇 가지 키 프레임을 정의하는 것입니다.

// 멤버 데이터

```

float mAnimTimePos; BoneAnimation
mSkullAnimation;

```

//

// 생성자에서 애니메이션 키 프레임 정의

```
//
```

```
XMVECTOR q0 = XMQuaternionRotationAxis(XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f), XMConvertToRadians(30.0f));
XMVECTOR q1 = XMQuaternionRotationAxis(XMVectorSet(1.0f, 1.0f, 2.0f, 0.0f), XMConvertToRadians(45.0f));
XMVECTOR q2 = XMQuaternionRotationAxis(XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f), XMConvertToRadians(-30.0f));
XMVECTOR q3 = XMQuaternionRotationAxis(XMVectorSet(1.0f, 0.0f, 0.0f, 0.0f), XMConvertToRadians(70.0f));
```

```
mSkullAnimation.Keyframes.resize(5); mSkullAnimation.Keyframes[0].TimePos = 0.0f;
mSkullAnimation.Keyframes[0].Translation = XMFLOAT3(-7.0f, 0.0f, 0.0f); mSkullAnimation.Keyframes[0].Scale
= XMFLOAT3(0.25f, 0.25f, 0.25f); XMStoreFloat4(&mSkullAnimation.Keyframes[0].RotationQuat, q0);
```

```
mSkullAnimation.Keyframes[1].TimePos = 2.0f; mSkullAnimation.Keyframes[1].Translation = XMFLOAT3(0.0f, 2.0f, 10.0f);
mSkullAnimation.Keyframes[1].Scale = XMFLOAT3(0.5f, 0.5f, 0.5f);
XMStoreFloat4(&mSkullAnimation.Keyframes[1].RotationQuat, q1);
```

```
mSkullAnimation.Keyframes[2].TimePos = 4.0f; mSkullAnimation.Keyframes[2].Translation = XMFLOAT3(7.0f, 0.0f, 0.0f);
mSkullAnimation.Keyframes[2].Scale = XMFLOAT3(0.25f, 0.25f, 0.25f);
XMStoreFloat4(&mSkullAnimation.Keyframes[2].RotationQuat, q2);
```

```
mSkullAnimation.Keyframes[3].TimePos = 6.0f; mSkullAnimation.Keyframes[3].Translation = XMFLOAT3(0.0f, 1.0f, -
10.0f); mSkullAnimation.Keyframes[3].Scale = XMFLOAT3(0.5f, 0.5f, 0.5f);
XMStoreFloat4(&mSkullAnimation.Keyframes[3].RotationQuat, q3);
```

```
mSkullAnimation.Keyframes[4].TimePos = 8.0f; mSkullAnimation.Keyframes[4].Translation = XMFLOAT3(-7.0f, 0.0f, 0.0f);
mSkullAnimation.Keyframes[4].Scale = XMFLOAT3(0.25f, 0.25f, 0.25f);
XMStoreFloat4(&mSkullAnimation.Keyframes[4].RotationQuat, q0);
```

키 프레임은 두개골을 장면 내 다양한 위치, 방향, 크기로 배치합니다. 이 데모에서 직접 키 프레임을 추가하거나 키 프레임 값을 변경하며 실험해 보세요. 예를 들어 모든 회전과 크기를 동일값으로 설정하면 위치만 애니메이션 처리된 모습을 확인할 수 있습니다.

애니메이션을 작동시키기 위한 마지막 단계는 보간을 수행하여 새로운 두개골 월드 매트릭스를 얻는 것입니다. 이 매트릭스는

:

```
void QuatApp::UpdateScene(float dt)
{
 ...

 // 시간 위치를 증가시킵니다. mAnimTimePos
 += dt;
 if(mAnimTimePos >= mSkullAnimation.GetEndTime())
 {
 // 애니메이션을 처음으로 되감습니다. mAnimTimePos = 0.0f;
 }

 // 이 시간 순간의 두개골 월드 매트릭스를 가져옵니다. mSkullAnimation.Interpolate(mAnimTimePos, mSkullWorld);
}
```

두개골의 세계 행렬은 이제 애니메이션을 위해 매 프레임마다 변경됩니다.

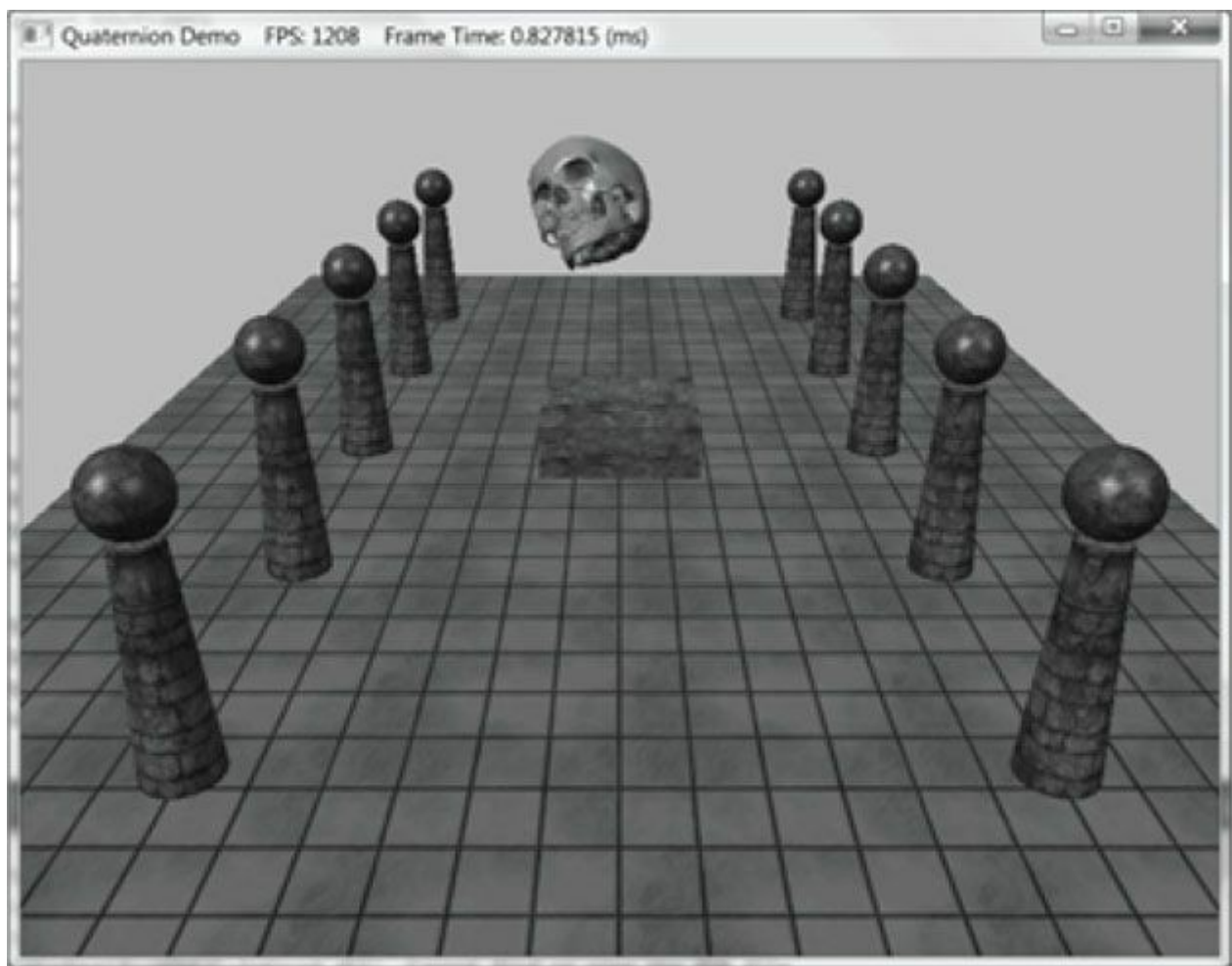


그림 24.12. 쿼터니언 데모의 스크린샷.

## 24.7 요약

- 실수의 순서된 4-튜플  $\mathbf{q} = (x, y, z, w) = (q_1, q_2, q_3, q_4)$  는 쿼터니언입니다. 일반적으로  $\mathbf{q} = (\mathbf{u}, w) = (x, y, z, w)$  로 약칭하며,  $\mathbf{u} = (x, y, z)$  를 허수 벡터 부분,  $w$  를 실수 부분이라 부른다. 또한 등호, 덧셈, 뺄셈, 곱셈, 나눗셈은 다음과 같이 정의된다:  

$$\mathbf{u} = \mathbf{v} \text{ 일 때만 성립하며 } a = b$$
- 쿼터니언 곱셈은 교환법칙을 따르지 않지만 결합법칙을 따른다. 쿼터니언은  $\mathbf{e} = (0, 0, 0, 1)$  는 곱셈 항등원 역할을 합니다. 사원수  $\mathbf{p}(\mathbf{q} + \mathbf{r}) = \mathbf{p}\mathbf{q} + \mathbf{p}\mathbf{r}$  그리고  $(\mathbf{q} + \mathbf{r})\mathbf{p} = \mathbf{q}\mathbf{p} + \mathbf{r}\mathbf{p}$  의 곱셈은 사원수의 덧셈에 대해 분배법칙을 따릅니다.
- 실수  $s$  를 쿼터니언 공간으로 변환하려면  $\mathbf{s} = (0, 0, 0, s)$  로 표기하며, 벡터  $\mathbf{u}$  를 쿼터니언 공간으로 변환하려면  $\mathbf{u} = (\mathbf{u}, 0)$  로 표기합니다. 실수 부분이 0인 쿼터니언을 순수 쿼터니언 (pure quaternion)이라 합니다. 이때 스칼라와 쿼터니언을 곱할 수 있으며, 그 결과는  $\mathbf{s}(p_1, p_2, p_3, p_4) = (sp_1, sp_2, sp_3, sp_4) = (p_1, p_2, p_3, p_4)\mathbf{s}$  입니다. 스칼라 곱셈의 특수한 경우는 교환법칙을 따른다.
- 스칼라 곱셈의 공액의 사원수의  $\mathbf{q} = (q_1, q_2, q_3, q_4) = (\mathbf{u}, q_4)$  사원수의 기호  $\mathbf{q}^*$  로 정의된 다음과 같이  $\mathbf{q}^* = -q_1 - q_2 - q_3 + q_4 = (-\mathbf{u}, q_4)$  의 사원수의 정의된다 다음과 같이 정의됩니다:  

$$\|\mathbf{q}\| = \sqrt{\mathbf{q}\mathbf{q}^*} = \sqrt{q_1^2 + q_2^2 + q_3^2 + q_4^2} = \sqrt{\|\mathbf{u}\|^2 + q_4^2}$$

(또는 크기)의 사원수의

우리는 사원수가 단위 사원수라고 말합니다.

하나의 규범.

$$\mathbf{q} = (q_1, q_2, q_3, q_4) = (\mathbf{u}, q_4)$$

5. 가  
어집니다:

가 0이 아닌 사원수일 때, 역원은 다음과 같이 표시됩니다.

$q^{-1}$ 로 표시하며 다음 식으로 주

- $\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|^2}$ . 만약  $\mathbf{q}$ 가 단위 사원수라면  $\mathbf{q}^{-1} = \mathbf{q}^*$ 이다.
6. 단위 사원수  $\mathbf{q} = (\mathbf{u}, q_4)$  는 극좌표 표현  $\mathbf{q} = (\sin \theta \mathbf{n}, \cos \theta)$ 로 쓸 수 있으며, 여기서  $\mathbf{n}$ 은 단위 벡터이다.
7.  $\mathbf{q}$ 가 단위 사원수라면,  $\mathbf{q} = (\sin \theta \mathbf{n}, \cos \theta) \parallel \mathbf{n} \parallel = 1$  및  $\theta \in [0, \pi]$ 에 대해 성립한다. 사원수 회전 연산자는 다음과 같이 정의된다:  

$$\mathbf{R}_{\mathbf{q}}(\mathbf{v}) = \mathbf{q}\mathbf{v}\mathbf{q}^{-1} = \mathbf{q}\mathbf{v}\mathbf{q}^*$$
 로 정의되며, 점/벡터  $\mathbf{v}$ 를 축  $\mathbf{n}$  주위로 각도  $2\theta$ 만큼 회전시킨다.  $\mathbf{q}$ 는 행렬을 가집니다.
- 표현이며, 모든 회전 행렬은 해당 회전을 나타내는 쿼터니언으로 변환될 수 있습니다.
8. 애니메이션에서 흔히 수행하는 작업은 두 방향 사이를 보간하는 것입니다. 각 방향을 단위 쿼터니언으로 표현하면, 구형 보간을 사용하여 단위 쿼터니언을 보간하여 보간된 방향을 구할 수 있습니다.

## 24.8 연습문제

1. 지정된 복소수 연산을 수행하세요.
2. 복소수  $(-1, 3)$ 를 극좌표법으로 표기하세요.
3. 벡터  $(2, 1)$ 를 복소수 곱셈을 사용하여  $30^\circ$  회전시키세요.
4. 복소수 나눗셈의 정의를 사용하여  $\frac{a+ib}{a+ib} = 1$ 임을 증명하십시오.
5.  $\mathbf{z} = a + ib$ 라 하자.  $|\mathbf{z}|^2 = \mathbf{z}\overline{\mathbf{z}}$ 임을 증명하라.
6.  $\mathbf{M}$ 을  $2 \times 2$  행렬이라 하자. 다음이 성립함을 증명하라:  $\det \mathbf{M} = 1, \mathbf{M}^{-1} = \mathbf{M}^T$  이는  $\mathbf{M}$ 이 회전 행렬인 경우에 한하여 성립한다. 이는 행렬이 회전 행렬인지 테스트하는 방법을 제공한다. 즉,  $\mathbf{M}$ 이 회전 행렬일 때만 성립함을 증명하라. 이는 행렬이 회전 행렬인지 테스트하는 방법을 제공한다.
7.  $\mathbf{p} = (1, 2, 3, 4)$ 와  $\mathbf{q} = (2, -1, 1, -2)$ 를 쿼터니언으로 하자. 표시된 쿼터니언 연산을 수행하라.

$\mathbf{q}$   
\*  
\*

$\mathbf{p}$   
 $\mathbf{p} \parallel$   
 $\mathbf{q} \parallel$   
1  
1

8. 단위 사원수를  $\mathbf{q} = \left(\frac{1}{2}, \frac{1}{2}, 0, \frac{1}{\sqrt{2}}\right)$  in 극좌표법으로 작성하십시오.
9. 단위 쿼터니언을  $\mathbf{q} = \left(\frac{\sqrt{3}}{2}, 0, 0, -\frac{1}{2}\right)$  극좌표법으로 표기하십시오.
10. 축을 중심으로  $45^\circ$  회전하는 단위 쿼터니언을 구하세요  $(1, 1, 1)$
11. 축을 중심으로  $60^\circ$  회전하는 단위 쿼터니언을 구하라  $(0, 0, -1)$

12. Let  $\mathbf{p} = \left(\frac{1}{2}, 0, 0, \frac{\sqrt{3}}{2}\right)$  그리고  $\mathbf{q} = \left(\frac{\sqrt{3}}{2}, 0, 0, \frac{1}{2}\right)$   $\text{slerp}\left(\mathbf{p}, \mathbf{q}, \frac{1}{2}\right)$ 을 계산하고 그것이 단위 사원수임을 확인하라.
13. 사원수가  $\left(x, y, z, w\right)$  다음 형태로 표현될 수 있음을 보여라  $x\mathbf{i} + y\mathbf{j} + z\mathbf{k} + w$ 의 형태로 쓸 수 있음을 보여라.
14. 다음이 성립함을 증명하라.  $\mathbf{q}\mathbf{q}^* = \mathbf{q}^*\mathbf{q} = q_1^2 + q_2^2 + q_3^2 + q_4^2 = ||\mathbf{u}||^2 + q_4^2$ .
15. 다음과 같이 하자.  $\mathbf{p} = \left(\mathbf{u}, 0\right)$  와  $\mathbf{q} = \left(\mathbf{v}, 0\right)$  순수 사원수(즉, 실수 부분 0)라고 하자. 다음을 증명하라.  $\mathbf{pq} = \left(\mathbf{p} \times \mathbf{q}, -\mathbf{p} \cdot \mathbf{q}\right)$
16. 다음 성질을 증명하라.

$$\mathbf{s} \in \mathbb{R}$$

17. 증명하라  $\mathbf{a} \cdot \frac{\sin\left(\left(1-t\right)\theta\right)\mathbf{a} + \sin\left(t\theta\right)\mathbf{b}}{\sin\theta} = \cos\left(t\theta\right)$  대수적으로 증명하라.
18. 다음과 같이 하자. 3차원 벡터라 하자. 다음 등식을 증명하라:  $\mathbf{a}, \mathbf{b}, \mathbf{c}$

# 제25<sup>장</sup> 캐릭터 애니메이션



본 장의 일부 내용은 Frank D. Luna의 저서 『DirectX 9.0c를 이용한 3D 게임 프로그래밍 입문: 셰이더 접근법』(2006년, Jones and Bartlett Learning, Burlington, MA)에 수록된 내용입니다. [www.jblearning.com](http://www.jblearning.com) 허가를 받아 재인쇄합니다.

이 장에서는 인간이나 동물과 같은 복잡한 캐릭터를 애니메이션화하는 방법을 배웁니다. 캐릭터가 복잡한 이유는 동시에 움직이는 여러 움직이는 부위를 가지고 있기 때문입니다. 달리는 인간을 생각해 보세요—모든 뼈가 어떤 식으로든 움직이고 있습니다. 이렇게 복잡한 애니메이션을 수작업으로 만드는 것은 실용적이지 않으며, 이를 위한 특별한 모델링 및 애니메이션 도구가 존재합니다. 이미 캐릭터와 그에 대응하는 애니메이션 데이터가 생성되었다고 가정할 때, 이 장에서는 Direct3D를 사용하여 이를 애니메이션화하고 렌더링하는 방법을 배울 것입니다.

## 목표:

1. 애니메이션된 스킨링 메시(skinned mesh)의 용어에 익숙해지기.
2. 메쉬 계층 구조 변환의 수학적 원리와 트리 기반 메쉬 계층 구조 탐색 방법을 학습한다.
3. 버텍스 블렌딩의 개념과 수학적 원리를 이해한다.
4. .m3d 형식에서 애니메이션 데이터를 불러오는 방법을 알아봅니다.
5. Direct3D에서 캐릭터 애니메이션을 구현하는 방법을 알아보기.

## 25.1 프레임 계층 구조

많은 객체는 부모-자식 관계로 구성된 부분들로 이루어져 있으며, 하나 이상의 자식 객체는 자체적으로 독립적으로 움직일 수 있지만(물리적 동작 제약이 있을 수 있음—예: 인간의 관절은 특정 가동 범위를 가짐), 부모가 움직일 때 함께 움직여야 합니다. 예를 들어, 상완, 전완, 손으로 나뉜 팔을 생각해 보십시오. 손은 손목 관절을 중심으로 독립적으로 회전할 수 있습니다. 그러나 팔뚝이 팔꿈치 관절을 중심으로 회전하면 손도 함께 회전해야 합니다. 마찬가지로, 상완이 어깨 관절을 중심으로 회전하면 팔뚝이 함께 회전하며, 팔뚝이 회전하면 손도 함께 회전합니다(그림 25.1 참조). 따라서 명확한 객체 계층 구조가 나타납니다: 손은 팔뚝의 자식 객체이며, 팔뚝은 상완의 자식 객체입니다. 상황을 확장하면 상완은 몸통의 자식 객체가 되고, 이렇게 계속 이어져 골격 전체를 완성하게 됩니다(그림 25.2는 더 복잡한 계층 구조 예시를 보여줍니다).

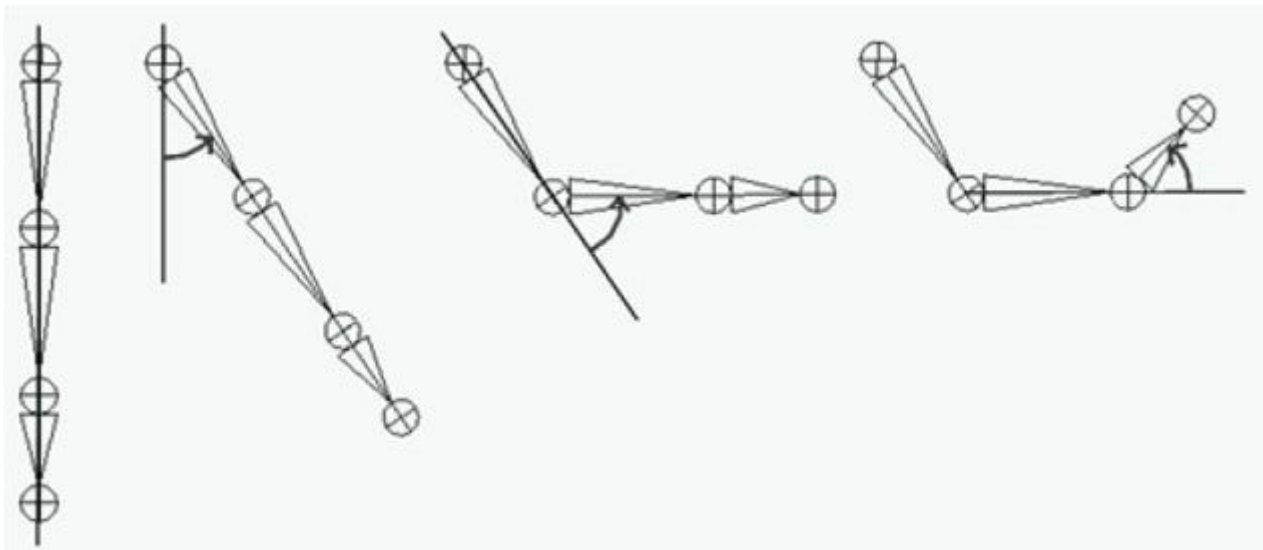


그림 25.1. 계층 변환; 한 뼈의 부모 변환이 자신과 모든 자식에게 영향을 미친다는 점을 관찰하십시오.

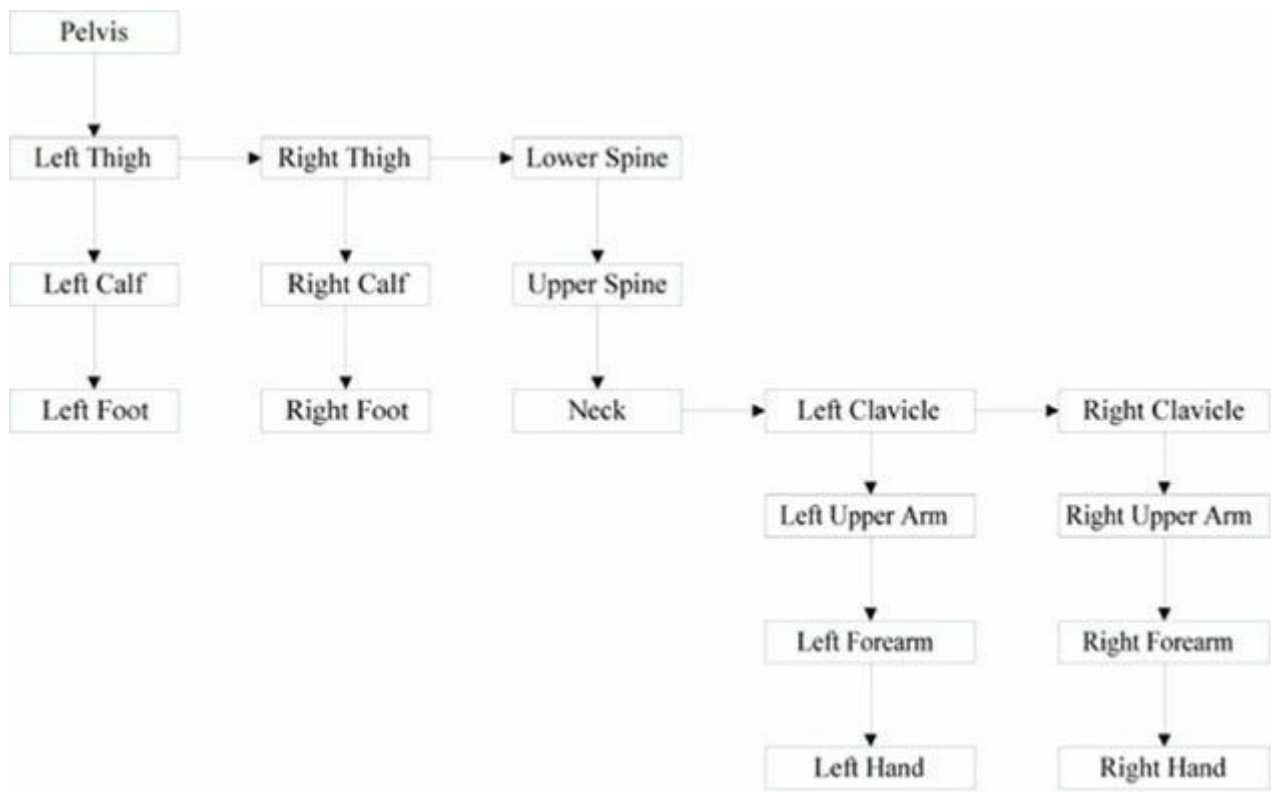


그림 25.2. 이쪽 보행 휴머노이드 캐릭터를 모델링하기 위한 더 복잡한 트리 계층 구조. 아래쪽 화살표는 "첫 번째 자식" 관계를, 오른쪽 화살표는 "동급 자식" 관계를 나타냅니다. 예를 들어, "왼쪽 허벅지", "오른쪽 허벅지", "하부 척추"는 모두 "골반" 본의 자식입니다.

이 섹션의 목적은 객체의 위치와 그 조상(즉, 부모, 조부모, 증조부모 등)의 위치를 기반으로 객체를 장면에 배치하는 방법을 보여주는 것이다.

## 25.1.1 수학적 공식화

*reader may wish to review [Chapter 3](#) of this book, specifically the topic of change-of-coordinate transformations.*

**Note** 간단하고 구체적으로 설명하기 위해, 우리는 상완(근부), 전완, 손의 계층 구조를 각각 Bone 0, Bone 1, Bone 2로 표시하여 작업합니다(그림 25.3 참조).

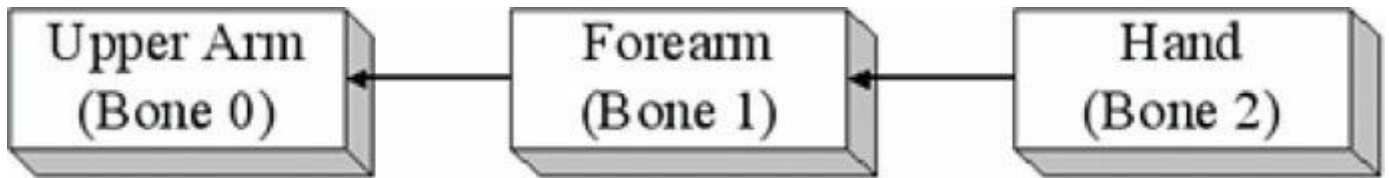


그림 25.3. 간단한 계층 구조.

기본 개념을 이해한 후에는 더 복잡한 상황을 처리하기 위해 간단한 일반화 기법을 사용합니다. 계층 구조 내의 객체가 주어졌을 때, 이를 월드 공간으로 올바르게 변환하려면 어떻게 해야 할까요? 당연히 객체를 단순히 월드 공간으로 직접 변환할 수는 없습니다. 상위 객체들의 변환도 고려해야 하기 때문입니다. 상위 객체들의 변환 역시 해당 객체의 장면 내 위치에 영향을 미치기 때문입니다.

계층 구조 내 각 객체는 회전을 용이하게 하기 위해 피벗 관절을 원점으로 하여 자체 로컬 좌표계에 대해 모델링됩니다 (그림 25.4 참조).

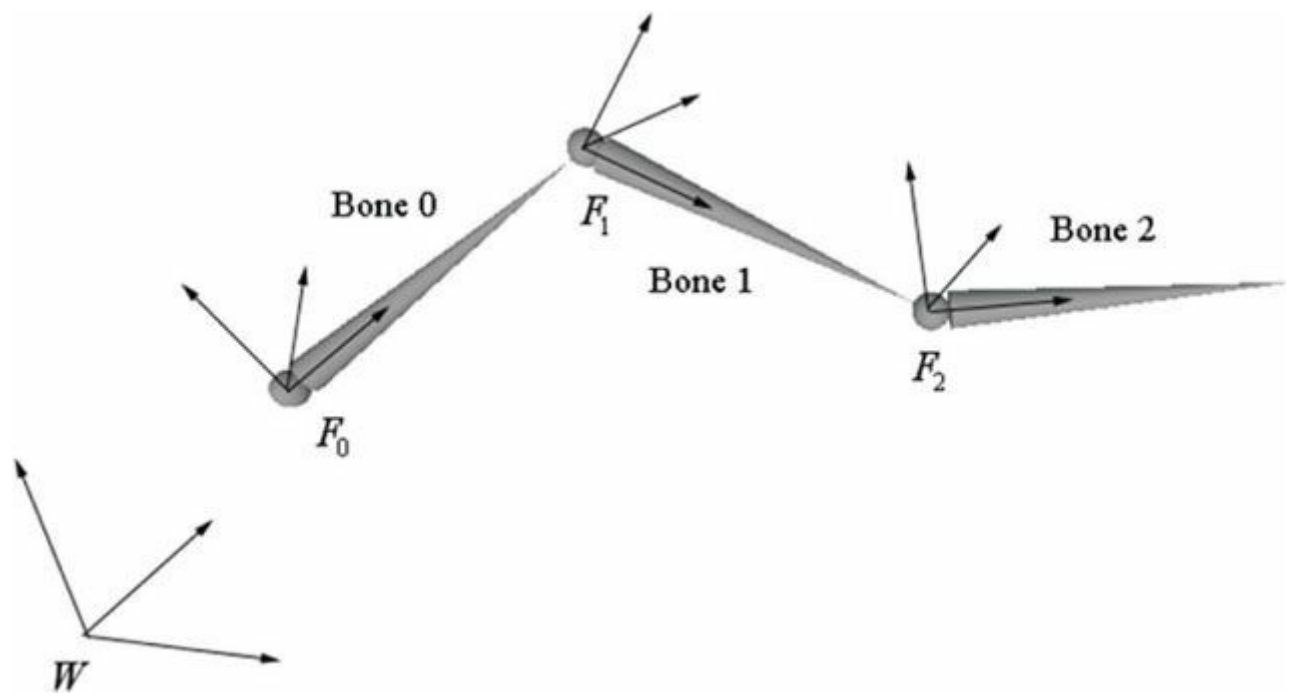


그림 25.4. 각 본의 기하학 구조는 자체 로컬 좌표계에 상대적으로 표현됩니다. 또한 모든 좌표계가 동일한 공간에 존재하므로 서로를 연관시킬 수 있습니다.

모든 좌표계가 동일한 우주에 존재하기 때문에, 우리는 이들을 서로 연관시킬 수 있습니다. 특히, 임의의 순간(일반적으로 이러한 메쉬 계층 구조는 애니메이션 처리되어 관계가 시간 함수로 변화하므로, 시간을 고정하고 스냅샷을 연구합니다)에 대해, 각 좌표계를 상위 좌표계에 상대적으로 설명합니다. (루트 프레임  $F_0$ 의 부모 좌표계는 월드 공간 좌표계 *월드*. 즉, 좌표계  $F_0$ 은 월드 좌표계를 기준으로 설명됩니다.) 자식 및 부모 좌표계를 연관지었으므로, 이제 변환 행렬을 사용하여 자식 공간에서 부모 공간으로 변환할 수 있습니다. (이는 로컬-월드 변환과 동일한 개념입니다. 다만 로컬 공간에서 월드 공간으로 변환하는 대신, 로컬 공간에서 부모 공간으로 변환합니다.)  $A_2$ 를 프레임  $F_2$ 의 기하학을  $F_1$ 로 변환하는 행렬이라 하자.  $A_1$ 을 프레임  $F_1$ 에서  $F_0$ 으로 변환하는 행렬,  $A_0$ 을 프레임  $F_0$ 에서  $W$ 로 변환하는 행렬이라 하자. ( $A_i$ 는 자식 좌표계에서 부모 좌표계로 기하학적 구조를 변환하므로 '부모로 가는 행렬(*to-parent matrix*)'이라 부른다.) 그러면 팔 계층 구조 내  $i$ 번째 객체를 세계 좌표계로 변환하는 행렬  $M_i$ 는 다음과 같이 정의된다:

$$M_i = A_i A_{i-1} \cdots A_1 A_0 \quad (\text{eq. 25.1})$$

구체적으로, 본 예시에서  $M_2 = A_2 A_1 A_0$ ,  $M_1 = A_1 A_0$ ,  $M_0 = A_0$ 은 각각 손을 세계 좌표계로, 팔뚝을 세계 좌표계로, 상완을 세계 좌표계로 변환합니다. 객체는 상위 객체의 변환을 상속한다는 점을 유의하십시오. 예를 들어 상완이 움직이면 손이 움직이는 원리가 바로 이것입니다.

그림 25.5는 방정식 25.1이 말하는 내용을 그래픽으로 보여줍니다. 기본적으로 팔 계층 구조 내의 객체를 변환하려면

해당 객체와 모든 상위 객체(상승 순서)에 대한 '부모 변환'을 적용하여 좌표계 계층 구조를 상향 퍼콜레이션(percolation) 시키면, 객체가 세계 공간에 도달하게 됩니다.

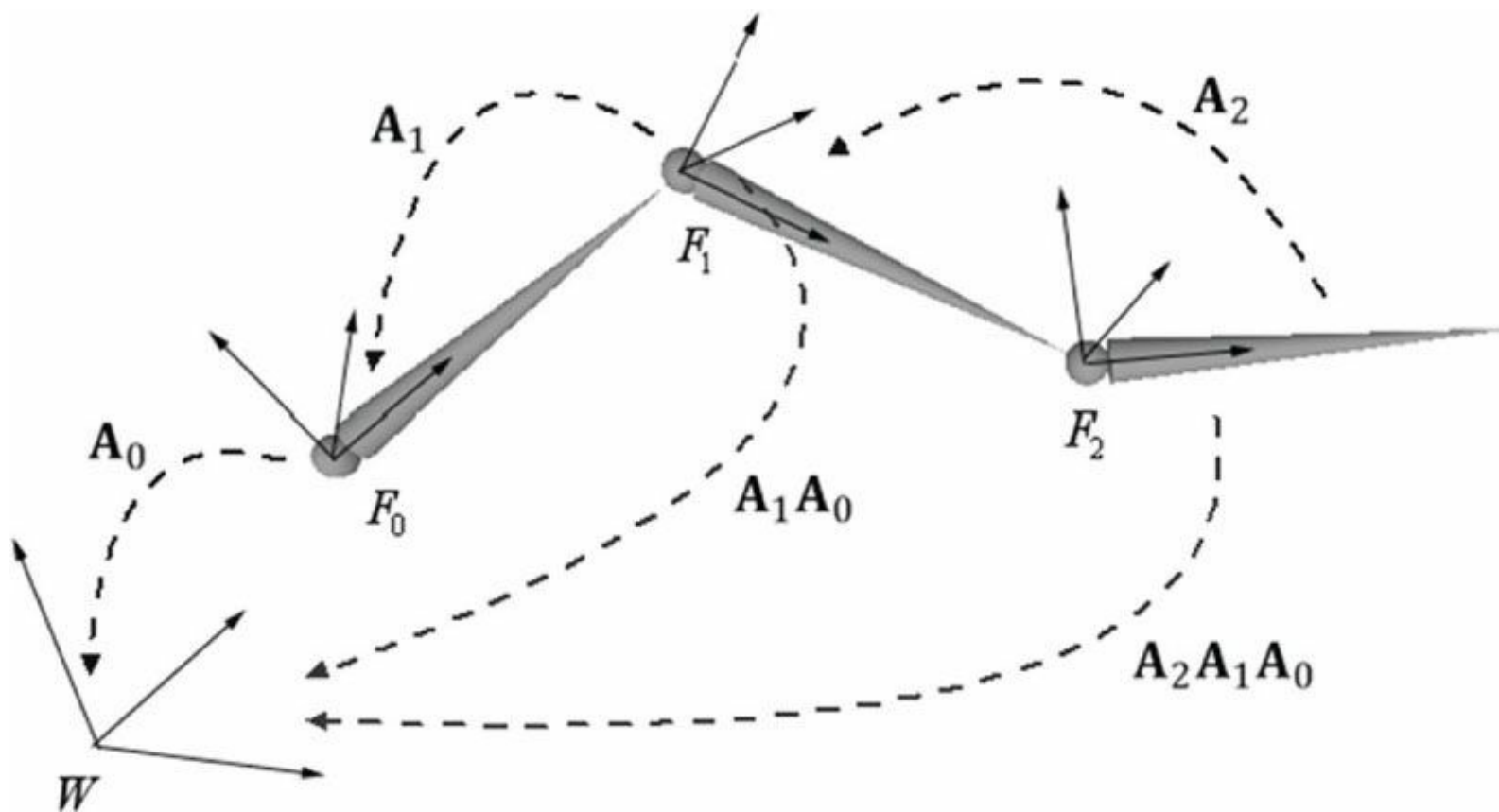


그림 25.5. 좌표계들이 동일한 우주에 존재하기 때문에 서로 연관시킬 수 있으며, 따라서 한 좌표계에서 다른 좌표계로 변환할 수 있습니다. 특히, 각 뼈의 좌표계를 그 부모 뼈의 좌표계에 상대적으로 설명함으로써 연관시킵니다. 이를 통해 뼈의 기하학적 구조를 로컬 좌표계에서 부모 좌표계로 변환하는 *부모* 변환 행렬을 구성할 수 있습니다. 부모 좌표계로 변환된 후에는 부모의 부모 변환 행렬을 적용하여 조부모 좌표계로 변환하고, 이 과정을 반복하여 각 조상의 좌표계를 거쳐 최종적으로 월드 공간에 도달합니다.

지금까지 다룬 예시는 단순한 선형 계층 구조입니다. 그러나 동일한 개념은 트리 계층 구조로도 일반화됩니다. 즉, 계층 구조 내 모든 객체의 월드 스페이스 변환은 해당 객체와 모든 조상 객체(상승 순서)의 부모 변환을 적용하여 좌표계 계층 구조를 상향 퍼콜레이션(percolate)함으로써 월드 스페이스에 도달할 때까지 계산됩니다(다시 한번 루트의 부모 좌표계가 월드 스페이스임을 가정합니다). 유일한 실질적 차이는 선형 리스트 대신 탐색해야 할 더 복잡한 트리 데이터 구조를 갖는다는 점입니다.

예를 들어, [그림 25.2](#)의 왼쪽 새골을 살펴보자. 이는 목뼈의 형제 노드이므로 상부 척추의 자식 노드이다. 상부 척추는 하부 척추의 자식이며, 하부 척추는 골반의 자식이다. 따라서 왼쪽 새골의 월드 트랜스폼은 왼쪽 새골의 부모 트랜스폼, 상부 척추의 부모 트랜스폼, 하부 척추의 부모 트랜스폼, 골반의 부모 트랜스폼을 순차적으로 연결하여 형성된다.

## 25.2 스킨 처리된 메쉬

### 25.2.1 정의

[그림 25.6](#)은 캐릭터 메쉬를 보여줍니다. 그림에서 강조 표시된 뼈 사슬을 *스켈레톤*이라고 합니다. 스켈레톤은 캐릭터 애니메이션 시스템을 구동하기 위한 자연스러운 계층 구조를 제공합니다. 스켈레톤은 *외부 스킨*으로 둘러싸여 있으며, 우리는 이를 3D 지오메트리(정점과 다각형)로 모델링합니다. 초기에는 스킨 정점이 *바인드 공간*(전체 스킨이 정의되는 로컬 좌표계, 일반적으로 루트 좌표계)에 상대적입니다. 스켈레톤의 각 뼈는 실제와 마찬가지로, 그 뼈가 영향을 미치는 피부의 부분 집합(즉, 그 뼈가 영향을 미치는 정점들)의 모양과 위치에 영향을 미칩니다. 따라서 스켈레톤을 애니메이션화하면, 연결된 피부는 스켈레톤의 현재 포즈를 반영하도록 그에 따라 애니메이션됩니다.

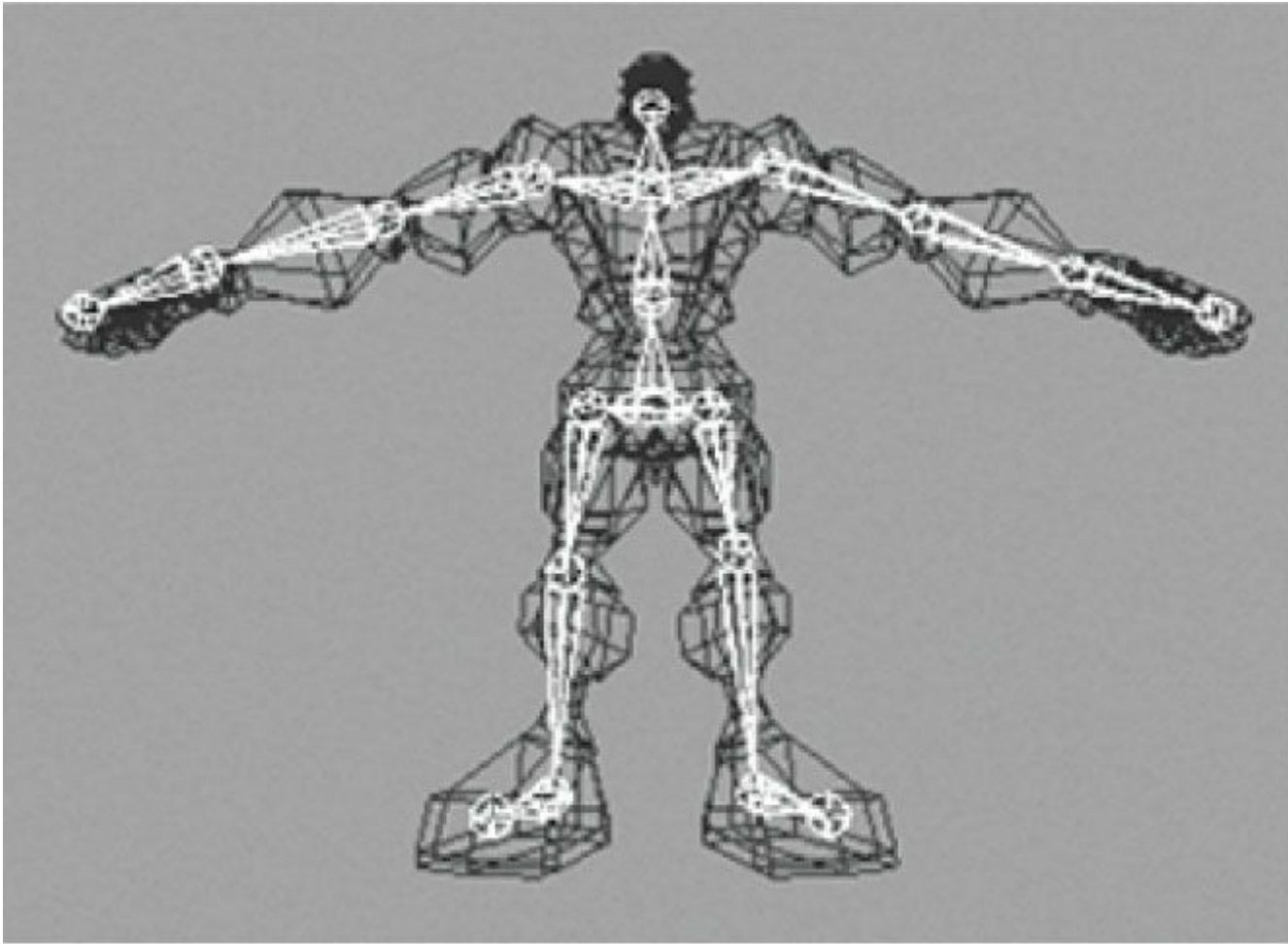


그림 25.6. 캐릭터 메쉬. 강조 표시된 본 체인은 캐릭터의 골격을 나타냅니다. 어두운 색의 폴리곤은 캐릭터의 피부를 나타냅니다. 피부 정점들은 *바인드 공간*(bind space)을 기준으로 하며, 이는 메쉬가 모델링된 좌표계입니다.

## 25.2.2 본즈 투 루트 변환 재구성

§25.1과의 차이점 중 하나는 여기서는 루트 좌표계에서 월드 좌표계로의 변환을 별도의 단계로 수행한다는 점입니다. 따라서 각 본에 대한 월드 변환 행렬을 찾는 대신, 각 본에 대한 *루트* 변환 행렬(즉, 본의 로컬 좌표계에서 루트 본의 좌표계로의 변환)을 찾습니다.

두 번째 차이점은 §25.1에서는 노드의 계보를 하향식(bottom-up)으로 탐색했는데, 이는 뼈에서 시작하여 상위 노드로 이동하는 방식이었습니다. 그러나 실제로는 상향식 접근법(식 25.2 참조)이 더 효율적입니다. 이 방식에서는 루트에서 시작하여 트리 아래로 이동합니다.  $n$ 개의 뼈에 정수 0, 1, ...,  $n-1$ 을 할당하면,  $i$  번째 뼈의 루트 변환을 표현하는 다음 공식이 성립합니다:

$$toRoot_i = toParent_i \cdot toRoot_p \quad (\text{eq. 25.2})$$

여기서  $p$ 는 뼈  $i$ 의 부모 뼈의 레이블입니다. 이해가 되시나요? 실제로  $toRoot_p$ 는 기하학적 구조를 뼈  $p$ 의 좌표계에서 루트 좌표계로 직접 매핑해 줍니다. 따라서 루트 좌표계로 이동하려면, 뼈  $i$ 의 좌표계에서 그 부모 뼈  $p$ 의 좌표계로 기하학적 구조를 변환하기만 하면 되며,  $toParent_{(i)}$ 가 바로 그 역할을 수행합니다.

유일한 문제는 이 기능이 작동하려면  $i$ 번째 본을 처리할 때 이미 해당 부모의 루트 변환을 계산해 놓아야 한다는 점입니다. 변환이 미리 계산되어 있어야 한다는 점입니다. 그러나 트리를 상향식(top-down)으로 탐색한다면, 부모의 루트 변환은 항상 자식들의 루트 변환보다 먼저 계산됩니다.

또한 이것이 왜 더 효율적인지 알 수 있습니다. 하향식 접근법에서는 어떤 뼈  $i$ 에 대해서도 이미 부모의 루트 변환 행렬을 가지고 있으므로, 뼈  $i$ 에 대한 루트 변환까지 단 한 단계만 남게 됩니다. 상향식 기법에서는 각 뼈마다 전체 계보를 탐색해야 하며, 많은 행렬 곱셈이 발생합니다.

변환 행렬을 이미 가지고 있으므로, 뼈  $i$ 의 루트 변환까지 단 한 단계만 남게 됩니다. 반면 상향식 기법에서는 각 뼈마다 전체 계보를 탐색해야 하며, 공통 조상을 공유하는 뼈들 사이에서 많은 행렬 곱셈이 중복될 것입니다.

## 25.2.3 오프셋 변환

뼈에 의해 영향을 받는 정점들이 해당 뼈의 좌표계에 상대적이지 않다는 사실(메쉬가 모델링된 좌표계인 *바인드 공간*에 상대적임)에서 비롯된 미묘한 차이가 존재합니다. 따라서 방정식 15.2를 적용하기 전에, 먼저 정점들을 바인드 공간에서 해당 정점들에 영향을 미치는 뼈의 공간으로 변환해야 합니다. 소위 *오프셋 변환*이 이를 수행합니다; 그림 25.7을 참조하십시오.

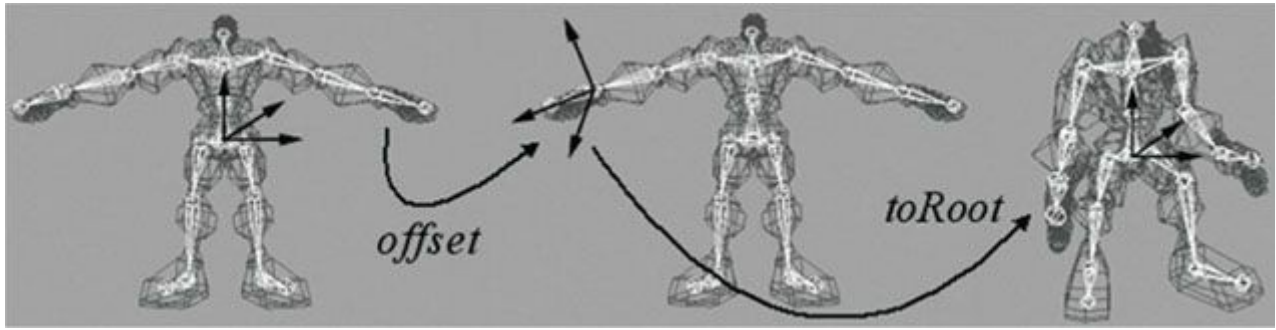


그림 25.7. 먼저 바인드 공간에서 뼈의 영향을 받는 점들을 오프셋 변환을 통해 영향을 주는 뼈의 공간으로 변환합니다. 그런 다음 해당 뼈의 공간에 도달하면, 뼈의 루트 변환을 적용하여 점들을 뼈의 공간에서 루트 뼈의 공간으로 변환합니다. 최종 변환은 오프셋 변환과 루트 변환의 조합입니다.

따라서 임의의 뼈  $B$ 의 오프셋 행렬로 점들을 변환함으로써, 점들을 바인드 공간에서  $B$ 의 뼈 공간으로 이동시킵니다. 그런 다음 점들이  $B$ 의 뼈 공간에 위치하게 되면,  $B$ 의 루트 변환을 사용하여 현재 애니메이션 포즈에서 캐릭터 공간으로 다시 위치시킬 수 있습니다.

이제 **최종 변환**이라고 부르는 새로운 변환을 소개합니다. 이는 본의 오프셋 변환과 루트 변환을 결합한 것입니다.

변환: 수학적으로,  $i$ 번째 뼈  $F_i$ 의 최종 변환 행렬은 다음과 같이 주어진다:

$$F_i = offset_i \cdot toRoot_i \quad (eq. 25.3)$$

## 25.2.4 스켈레톤 애니메이션

지난 장의 데모에서는 단일 객체의 애니메이션 방법을 보여주었습니다. *키 프레임*이 특정 시점에서의 객체 위치, 방향, 크기를 지정하며, *애니메이션*은 시간순으로 정렬된 키 프레임 목록으로 전체 애니메이션의 외형을 대략 정의한다고 설명했습니다. 이후 키 프레임 사이의 보간을 통해 중간 시점의 객체 위치를 계산하는 방법을 보여주었습니다. 이제 애니메이션 시스템을 스켈레톤 애니메이션으로 확장합니다. 이 애니메이션 클래스들은 본 장의 "스킨드 메시" 데모에 있는 *SkinnedData.h/cpp*에 정의되어 있습니다.

스켈레톤 애니메이션은 단일 객체 애니메이션보다 크게 어렵지 않습니다. 단일 객체를 하나의 뼈로 생각할 수 있는 반면,

스켈레톤은 연결된 뼈들의 집합체에 불과합니다. 각 뼈가 독립적으로 움직일 수 있다고 가정하겠습니다. 따라서 스켈레톤을 애니메이션화하려면 각 뼈를 로컬로 애니메이션화하기만 하면 됩니다. 그런 다음 각 뼈가 로컬 애니메이션을 수행한 후, 상위 뼈들의 움직임을 고려하여 루트 공간으로 변환합니다.

*애니메이션 클립*은 특정 골격 애니메이션을 형성하기 위해 함께 작동하는 애니메이션 목록(골격의 각 뼈마다 하나씩)으로 정의됩니다.

예를 들어, "걷기", "달리기", "싸우기", "엎드리기", "뛰기" 등이 애니메이션 클립의 예시입니다.

```

///<summary>
/// AnimationClip의 예시로는 "걷기", "달리기", "공격", "방어"가 있습니다.
/// 애니메이션 클립은 애니메이션 클립을 형성하기 위해 모든 뼈에 대해 BoneAnimation이 필요합니다.
/// 애니메이션 클립을 구성합니다.
///</summary>
struct AnimationClip
{
 // 이 클립 내 모든 본 중 가장 작은 종료 시간. float
 GetClipStartTime()const;

 // 이 클립 내 모든 본에 대한 최대 종료 시간. float GetClipEndTime()const;

 // 클립 내 각 BoneAnimation을 순회하며
 // 애니메이션을 보간합니다.
 void Interpolate(float t, std::vector<XMFLOAT4X4>& boneTransforms)const;

 // 각 본에 대한 애니메이션. std::vector<BoneAnimation>
 BoneAnimations;
};

```

캐릭터는 일반적으로 애플리케이션에서 수행해야 하는 모든 애니메이션을 위해 여러 애니메이션 클립을 가집니다. 그러나 모든 애니메이션 클립은 동일한 스켈레톤에서 작동하므로 동일한 수의 본을 사용합니다(특정 애니메이션에 대해 일부 본은 고정될 수 있음). 모든 애니메이션 클립을 저장하고 읽기 쉬운 이름으로 애니메이션 클립을 참조하기 위해 **맵** 데이터 구조를 사용할 수 있습니다:

```
std::map<std::string, AnimationClip> mAnimations; AnimationClip& clip =
mAnimations["attack"];
```

마지막으로 앞서 언급한 바와 같이, 각 본은 바인드 공간의 정점을 본의 공간으로 변환하기 위한 오프셋 변환이 필요합니다. 또한 스켈레톤 계층 구조를 표현할 방법이 필요합니다(배열을 사용합니다—자세한 내용은 다음 섹션 참조). 이를 통해 스켈레톤 애니메이션 데이터를 저장할 최종 데이터 구조를 얻을 수 있습니다:

```
class SkinnedData
{
public:

 UINT BoneCount()const;

 float GetClipStartTime(const std::string& clipName)const; float GetClipEndTime(const
std::string& clipName)const;

 void Set(
 std::vector<int>& boneHierarchy, std::vector<XMFLOAT4X4>& boneOffsets,
 std::map<std::string, AnimationClip>& animations);

 // 실제 프로젝트에서는 결과가 캐시되도록 하는 것이 바람직합니다.
 // 동일한 클립 이름으로 동일한 시간 위치에서 여러 번 호출할 가능성이 있었습니다
 // 동일한 clipName과 timePos로 여러 번 호출했을 가능성이 있습니다.
 void GetFinalTransforms(const std::string& clipName, float timePos, std::vector<XMFLOAT4X4>& finalTransforms)const;

private:

 // i번째 본의 부모 인덱스를 반환합니다. i번째 본은
 // 애니메이션 클립 내 i번째 BoneAnimation에 해당합니다.
 std::vector<int> mBoneHierarchy;

 // i번째 본의 오프셋 변환. std::vector<XMFLOAT4X4> mBoneOffsets;

 std::map<std::string, AnimationClip> mAnimations;
};
```

## 25.2.5 최종 변환 계산

캐릭터의 프레임 계층 구조는 일반적으로 [그림 25.2](#)와 유사한 트리 형태입니다. 우리는 정수 배열로 계층 구조를 모델링하여 배열의 *i*번째 요소가 *i*번째 본의 부모 인덱스를 제공하도록 합니다. 또한 *i*번째 요소는 작업 중인 애니메이션 클립의 *i*번째 **BoneAnimation**에 해당하고, *i*번째 요소는 *i*번째 오프셋 변환에 해당합니다. 루트 본은 항상 요소 0에 위치하며 부모가 없습니다. 예를 들어, 뼈 *i*의 조부모 뼈의 애니메이션과 오프셋 변환은 다음과 같이 구합니다:

```
int 부모인덱스 = mBoneHierarchy[i];
int grandParentIndex = mBoneHierarchy[parentIndex]; XMFLOAT4X4 offset =
mBoneOffsets[grandParentIndex];

AnimationClip& clip = mAnimations["attack"];
BoneAnimation& anim = clip.BoneAnimations[grandParentIndex];
```

따라서 각 본에 대한 최종 변환을 다음과 같이 계산할 수 있습니다:

```
void SkinnedData::GetFinalTransforms(const std::string& clipName, float timePos,
std::vector<XMFLOAT4X4>& finalTransforms)const
{

 UINT numBones = mBoneOffsets.size(); std::vector<XMFLOAT4X4>

 toParentTransforms(numBones);

 // 주어진 시간 인스턴스에서 이 클립의 모든 본을 보간합니다.
```

```

auto clip = mAnimations.find(clipName);
clip->second.Interpolate(timePos, toParentTransforms);

//
// 계층 구조를 탐색하여 모든 본을 루트 공간으로 변환합니다.
//

std::vector<XMFLOAT4x4> toRootTransforms(numBones);

// 루트 본은 인덱스 0을 가집니다. 루트 본은 부모가 없으므로
// toRootTransform은 단순히 로컬 본 변환입니다. toRootTransforms[0] =
toParentTransforms[0];

// 이제 자식들의 toRootTransform을 찾습니다. for(UINT i = 1; i < numBones;
++i)
{

 XMATRIX toParent = XMLoadFloat4x4(&toParentTransforms[i]); int parentIndex =

 mBoneHierarchy[i];
 XMATRIX parentToRoot = XMLoadFloat4x4(&toRootTransforms[parentIndex]); XMATRIX toRoot =

 XMMatrixMultiply(toParent, parentToRoot);

 XMStoreFloat4x4(&toRootTransforms[i], toRoot);
}
// 최종 변환을 얻기 위해 본 오프셋 변환을 선폴하기합니다. for(UINT i = 0; i < numBones; ++i)
{
 XMATRIX offset = XMLoadFloat4x4(&mBoneOffsets[i]); XMATRIX toRoot =
 XMLoadFloat4x4(&toRootTransforms[i]);
 XMStoreFloat4x4(&finalTransforms[i], XMMatrixMultiply(offset, toRoot));
}
}

```

이 작업을 수행하기 위해 필요한 조건이 하나 있습니다. 루프에서 본을 순회할 때, 해당 본의 부모 본에 대한 루트 변환을 조회합니다:

```

int parentIndex = mBoneHierarchy[i];
XMATRIX parentToRoot = XMLoadFloat4x4(&toRootTransforms[parentIndex]);

```

이 작업은 루프 초반에 부모 본의 루트 변환이 이미 처리되었음이 보장될 때만 가능합니다. 실제로 본을 항상 부모 본이 자식 본 앞에 오도록 배열에 정렬하면 이 보장을 할 수 있습니다. 샘플 3D 데이터는 이러한 조건을 충족하도록 생성되었습니다. 다음은 특정 캐릭터 모델의 계층 구조 배열에서 상위 10개 본의 샘플 데이터입니다:

```

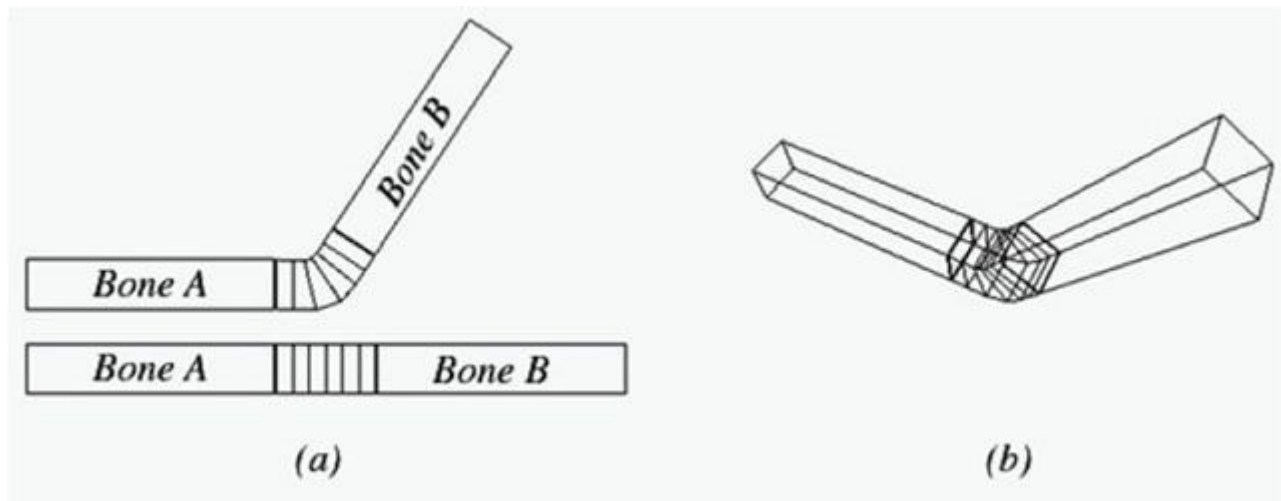
ParentIndexOfBone0: -1
ParentIndexOfBone1: 0
ParentIndexOfBone2: 0
ParentIndexOfBone3: 2
ParentIndexOfBone4: 3
ParentIndexOfBone5: 4
ParentIndexOfBone6: 5
ParentIndexOfBone7: 6
ParentIndexOfBone8: 5
ParentIndexOfBone9: 8

```

Bone9를 예로 들어 보겠습니다. Bone9의 부모는 Bone8이며, Bone8의 부모는 Bone5, Bone5의 부모는 Bone4, Bone4의 부모는 Bone3, Bone3의 부모는 Bone2, Bone2의 부모는 루트 노트 Bone0입니다. 정렬된 배열에서 자식 본은 부모 본보다 앞에 위치하지 않는다는 점에 유의하십시오.

## 25.3 벡터스 블렌딩

지금까지 스켈레톤 애니메이션을 구현하는 방법을 살펴보았습니다. 이 섹션에서는 스켈레톤을 덮고 있는 정점들의 스킨 애니메이션에 집중하겠습니다. 이를 수행하는 알고리즘을 *정점 블렌딩*이라고 합니다. 버텍스 블렌딩의 전략은 다음과 같습니다. 기본 뼈대 계층 구조가 존재하지만, 피부 자체는 하나의 연속적인 메쉬입니다 (즉, 메쉬를 각 뼈에 대응하도록 분할하여 개별적으로 애니메이션하지 않습니다). 또한 하나 이상의 본이 스킨의 한 정점에 영향을 미칠 수 있으며, 최종 결과는 영향을 미치는 본들의 최종 변환에 가중치를 적용한 평균값으로 결정됩니다(가중치는 모델 제작 시 아티스트가 지정하여 파일에 저장됨). 이러한 설정으로 관절부(일반적으로 문제가 되는 영역)에서 부드러운 전환 블렌딩을 달성할 수 있어 스킨이 탄력적으로 느껴지게 합니다. [그림 25.8](#)을 참조하십시오.



*그림 25.8.* 피부는 두 뼈를 모두 덮는 하나의 연속적인 메쉬입니다. 관절 근처의 정점들은 뼈 A와 뼈 B 모두의 영향을 받아 유연한 피부를 시뮬레이션하기 위한 부드러운 전환 블렌딩을 생성함을 관찰하십시오.

실제 적용 시 [Möller08]에 따르면 일반적으로 정점당 4개 이상의 뼈대 영향은 필요하지 않습니다. 따라서 본 설계에서는 정점당 최대 4개의 영향 뼈대를 고려합니다. 정점 블렌딩 구현을 위해 캐릭터 메쉬의 피부를 하나의 연속된 메쉬로 모델링합니다. 각 정점은 최종 변환 행렬 배열(골격의 각 뼈대마다 하나의 항목)인 *뼈대 행렬 팔레트*를 참조하는 최대 4개의 인덱스를 포함합니다. 또한 각 버텍스는 최대 4개의 가중치를 가지며, 이는 네 개의 영향 뼈대 각각이 해당 버텍스에 미치는 영향의 상대적 양을 나타냅니다. 따라서 버텍스 블렌딩을 위한 버텍스 구조는 다음과 같습니다([그림 25.9](#)):

# Matrix Palette

```
struct PosNormalTexTanSkinned
{
 XMFLOAT3 Pos;
 XMFLOAT3 Normal;
 XMFLOAT2 Tex;
 XMFLOAT4 TangentU;
 XMFLOAT3 Weights;
 BYTE BoneIndices[4];
};
```

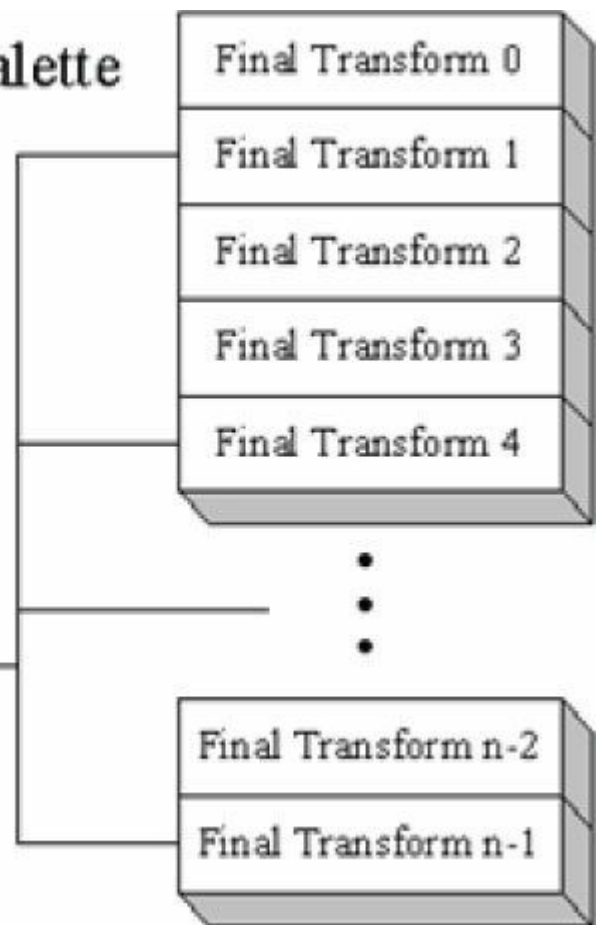


그림 25.9. 매트릭스 팔레트는 각 뼈에 대한 최종 변환을 저장합니다. 네 개의 뼈 인덱스가 매트릭스 팔레트에 어떻게 인덱싱되는지 살펴보세요. 뼈 인덱스는 해당 정점에 영향을 미치는 스켈레톤의 뼈를 식별합니다. 정점이 반드시 네 개의 뼈에 의해 영향을 받는 것은 아닙니다. 예를 들어, 네 개의 인덱스 중 두 개만 사용될 수 있으며, 이는 두 개의 뼈만이 정점에 영향을 미친다는 것을 나타냅니다. 뼈 가중치를 0으로 설정하여 해당 뼈가 정점에 영향을 미치지 않도록 효과적으로 제거할 수 있습니다.

이 형식의 정점을 가진 연속 메시는 정점 블렌딩을 수행할 준비가 된 상태이며, 이를 *스키닝된 메시*라고 부릅니다.

루트 좌표계에 대한 임의의 정점  $\mathbf{v}$ 의 정점 혼합 위치  $\mathbf{v}'$ 는(세계 변환을 루트 좌표계에서 마지막 단계로 수행한다는 점을 기억하십시오) 다음 가중 평균 공식으로 계산할 수 있습니다.

모든 요소가 루트 좌표계에 배치된 후 마지막 단계로 수행됨)은 다음 가중 평균 공식으로 계산할 수 있습니다:

$$\mathbf{v}' = w_0 \mathbf{vF}_0 + w_1 \mathbf{vF}_1 + w_2 \mathbf{vF}_2 + w_3 \mathbf{vF}_3$$

여기서  $w_0 + w_1 + w_2 + w_3 = 1$ , 즉 가중치들의 합은 1이 됩니다.

이 방정식에서 주어진 정점  $\mathbf{v}$ 를 영향을 미치는 모든 최종 뼈 변환(즉,

행렬  $\mathbf{F}_0, \mathbf{F}_1, \mathbf{F}_2, \mathbf{F}_3$ ). 그런 다음 개별적으로 변환된 이 점들의 가중 평균을 취하여 최종 벡스 혼합 위치  $\mathbf{v}'$ 를 계산합니다.

법선과 접선의 변환도 유사하게 수행됩니다:

$$\mathbf{n}' = \text{normalize}(w_0 \mathbf{nF}_0 + w_1 \mathbf{nF}_1 + w_2 \mathbf{nF}_2 + w_3 \mathbf{nF}_3)$$

$$\mathbf{t}' = \text{normalize}(w_0 \mathbf{tF}_0 + w_1 \mathbf{tF}_1 + w_2 \mathbf{tF}_2 + w_3 \mathbf{tF}_3)$$

여기서 변환 행렬  $\mathbf{F}_i$ 에는 비균일 스케일링이 포함되지 않는다고 *가정합니다*. 그렇지 않은 경우, 법선 변환 시 역전치행렬  $(\mathbf{F}^{-1})^T$ 를 사용해야 합니다(§7.2.2 참조).

다음 버텍스 셰이더 프래그먼트는 각 버텍스에 대해 최대 두 개의 본 영향으로 버텍스 블렌딩을 수행하는 핵심 코드를 보여줍니다.

버텍스에 대해 최대 두 개의 본 영향을 가진 버텍스 블렌딩을 수행하는 핵심 코드를 보여줍니다:

```
cbuffer cbSkinned
{
 // 캐릭터당 최대 96개의 본 지원.
 float4x4 gBone Transforms[96];
};
struct 스키닝된 정점 입력
{
```

```
float3 PosL : 위치; float3 노멀L :
NORMAL; float2 Tex : TEXCOORD; float4
TangentL : TANGENT; float3 가중

치 : 가중치;

uint4 BoneIndices : BONEINDICES;

};
```

```
VertexOut SkinnedVS(SkinnedVertexIn vin)
{
```

```
 VertexOut vout;
```

```
 // 배열 초기화하지 않으면 SV_POSITION에 대한 이상한 경고가 발생합니다. float weights[4] = {0.0f,
 0.0f, 0.0f, 0.0f};
 weights[0] = vin.Weights.x; weights[1] =
 vin.Weights.y; weights[2] = vin.Weights.z;
 weights[3] = 1.0f - weights[0] - weights[1] - weights[2];
```

```
 // 버텍스 블렌딩 수행.
```

```
 float3 posL = float3(0.0f, 0.0f, 0.0f); float3 normalL =
 float3(0.0f, 0.0f, 0.0f); float3 tangentL = float3(0.0f, 0.0f,
 0.0f); for(int i = 0; i < 4; ++i)
```

```
 {
```

```
 // 노멀 변환 시 비균일 스케일링이 없다고 가정하므로
```

```
 // 역전치 행렬을 사용할 필요가 없습니다.
```

```
 posL += weights[i]*mul(float4(vin.PosL, 1.0f), gBoneTransforms[vin.BoneIndices[i]]).xyz;
 normalL += weights[i]*mul(vin.NormalL, (float3x3)gBoneTransforms[vin.BoneIndices[i]]);
 tangentL += weights[i]*mul(vin.TangentL.xyz, (float3x3)gBoneTransforms[vin.BoneIndices[i]]);
```

```
 }
```

```
 // 월드 공간으로 변환.
```

```
 vout.PosW = mul(float4(posL, 1.0f), gWorld).xyz; vout.NormalW = mul(normalL,
 (float3x3)gWorldInvTranspose);
 vout.TangentW = float4(mul(tangentL, (float3x3)gWorld), vin.TangentL.w);
```

```
 // 동차 클립 공간으로 변환.
```

```
 vout.PosH = mul(float4(posL, 1.0f), gWorldViewProj);
```

```
 // 삼각형 간 보간을 위한 정점 속성 출력. vout.Tex = mul(float4(vin.Tex, 0.0f, 1.0f),
 gTexTransform).xy;
```

```
 // 쉐도우 새도우 맵을 투영하기 위한 투영 텍스처 좌표 생성. vout.ShadowPosH = mul(float4(posL, 1.0f),
```

```
 gShadowTransform);
```

```
 // SSAO 맵을 장면에 투영하기 위한 투영 텍스처 좌표 생성.
```

```
 vout.SsaoPosH = mul(float4(posL, 1.0f), gWorldViewProjTex); return vout;
```

```
}
```

이전 버텍스 셰이더가 버텍스당 최대 네 개의 본 영향을 사용하여 버텍스 블렌딩을 수행한다면, 왜 버텍스당 네 개가 아닌 세 개의 가중치만 입력할까요? 총 가중치는 반드시 1이 되어야 한다는 점을 떠올려 보  
세요. 따라서 네 개의 가중치에 대해 다음과 같습니다:  $w_0 + w_1 +$

$$w_2 + w_3 = 1 \iff w_3 = 1 - w_0 - w_1 - w_2.$$

## 25.4 .M3D 애니메이션 데이터 로딩 중

이제 .m3d 파일 형식을 확장하여 스킨 처리된 메쉬 데이터를 지원합니다. .m3d 헤더에는 스켈레톤의 뼈 수와 애니메이션 클립 수를 저장하는 슬롯이 있음을 상기하십시오:

```
*****m3d-파일-헤더***** #자료 3

#정점 3121
#삼각형 4062
#Bones 44
#AnimationClips 15
```

스킨 처리된 메시 지원을 위해 .m3d 형식은 다음과 같이 수정되었습니다:

- 1. 버텍스 데이터 청크에 버텍스 블렌드 가중치와 인덱스가 추가되었습니다.
- 2. 본 오프셋 변환(행렬로 저장)을 위한 새로운 청크가 추가되었습니다.
- 3. 계층 구조 배열을 저장하기 위한 새로운 청크가 추가되었습니다.
- 4. 애니메이션 클립을 저장하기 위한 새로운 청크가 추가되었습니다.

### 25.4.1 스킨닝된 버텍스 데이터

첫 번째 변경 사항은 정점 데이터에 블렌드 가중치와 본 인덱스가 포함되므로, 스킨닝된 정점 데이터를 읽기 위한 특별한 메서드가 필요하다는 점입니다:

```
*****정점***** 위치: -14.34667 90.44742 -
12.08929
접선: -0.3069077 0.2750875 0.9111171 1
법선: -0.3731041 -0.9154652 0.150721
텍스처 좌표: 0.21795 0.105219
혼합 가중치: 0.483457 0.483457 0.0194 0.013686
블렌드 인덱스: 3 2 39 34

위치: -15.87868 94.60355 9.362272
접선: -0.3069076 0.2750875 0.9111172 1
법선: -0.3731041 -0.9154652 0.150721
텍스처 좌표: 0.278234 0.091931
블렌드 가중치: 0.4985979 0.4985979 0.002804151 0
블렌드 인덱스: 39 2 3 0
...

void M3DLoader::ReadSkinnedVertices(std::ifstream& fin, UINT numVertices, std::vector<Vertex::PosNormalTexTanSkinned>& vertices)
{
 std::string ignore;
 vertices.resize(numVertices);

 fin >> ignore; // 정점 헤더 텍스트
 int boneIndices[4];

 float weights[4];
 for(UINT i = 0; i < numVertices; ++i)
 {
 fin >> ignore >> vertices[i].Pos.x >>
 vertices[i].Pos.y >> vertices[i].Pos.z;
 fin >> ignore >> vertices[i].TangentU.x >> vertices[i].TangentU.y >>
```

```

 vertices[i].TangentU.z >> vertices[i].TangentU.w;

 fin >> ignore >> vertices[i].Normal.x >>
 vertices[i].Normal.y >> vertices[i].Normal.z;

 fin >> ignore >> vertices[i].Tex.x >>
 vertices[i].Tex.y;
 fin >> ignore >> weights[0] >> weights[1] >> weights[2] >>
 weights[3];

 fin >> 무시 >> boneIndices[0] >> boneIndices[1] >> boneIndices[2] >> boneIndices[3];

 vertices[i].Weights.x = weights[0];
 vertices[i].Weights.y = weights[1];
 vertices[i].Weights.z = weights[2];

 vertices[i].BoneIndices[0] = (BYTE)boneIndices[0]; vertices[i].BoneIndices[1]
= (BYTE)boneIndices[1]; vertices[i].BoneIndices[2] = (BYTE)boneIndices[2];
 vertices[i].BoneIndices[3] = (BYTE)boneIndices[3];

}
}

```

## 25.4.2 본 오프셋 변환

본 오프셋 변환 청크는 본마다 하나의 행렬 목록을 저장합니다.

```

*****BoneOffsets***** BoneOffset0 -0.8669753
0.4982096 0.01187624 0
0.04897417 0.1088907 -0.9928461 0
-0.4959392 -0.8601914 -0.118805 0
-10.94755 -14.61919 90.63506 1

BoneOffset1 1 4.884964E-07 3.025227E-07 0
-3.145564E-07 2.163151E-07 -1 0
4.884964E-07 0.9999997 -9.59325E-08 0
3.284225 7.236738 1.556451 1
...

void M3DLoader::ReadBoneOffsets(std::ifstream& fin, UINT numBones, std::vector<XMFLOAT4X4>& boneOffsets)
{

 std::string ignore; boneOffsets.resize(numBones);

 fin >> ignore; // BoneOffsets 헤더 텍스트 for(UINT i = 0; i < numBones;
 ++i)
 {
 fin >> ignore >>
 boneOffsets[i](0,0) >> boneOffsets[i](0,1) >> boneOffsets[i](0,2) >>
 boneOffsets[i](0,3) >> boneOffsets[i](1,0) >> boneOffsets[i](1,1) >>
 boneOffsets[i](1,2) >> boneOffsets[i](1,3) >> boneOffsets[i](2,0) >>
 boneOffsets[i](2,1) >> boneOffsets[i](2,2) >> boneOffsets[i](2,3) >>
 boneOffsets[i](3,0) >> boneOffsets[i](3,1) >> boneOffsets[i](3,2) >>
 boneOffsets[i](3,3);

 }
}

```

## 25.4.3 계층 구조

계층 구조 청크는 계층 구조 배열을 저장합니다. 이 배열은 정수 배열로, i번째 배열 항목은 i번째 뼈의 부모 인덱스를 제공합니다.

뼈대의 부모 인덱스를 나타냅니다.

```
*****BoneHierarchy***** ParentIndexOfBone0: -1

ParentIndexOfBone1: 0
ParentIndexOfBone2: 0
ParentIndexOfBone3: 2
ParentIndexOfBone4: 3
ParentIndexOfBone5: 4
ParentIndexOfBone6: 5
ParentIndexOfBone7: 6
ParentIndexOfBone8: 5
ParentIndexOfBone9: 8

void M3DLoader::ReadBoneHierarchy(std::ifstream& fin, UINT numBones, std::vector<int>& boneIndexToParentIndex)
{

 std::string ignore; boneIndexToParentIndex.resize(numBones);

 fin >> ignore; // BoneHierarchy 헤더 텍스트 for(UINT i = 0; i <
numBones; ++i)
 {
 fin >> ignore >> boneIndexToParentIndex[i];
 }
}
```

25.4.4 애니메이션 데이터

마지막으로 읽어야 할 부분은 애니메이션 클립입니다. 각 애니메이션은 읽기 쉬운 이름과 스켈레톤의 각 본에 대한 키 프레임 목록을 가지고 있습니다. 각 키 프레임은 시간 위치, 본의 위치를 지정하는 변환 벡터, 본의 크기를 지정하는 스케일링 벡터, 본의 방향을 지정하는 쿼터니언을 저장합니다.

```
*****AnimationClips***** AnimationClip run_loop
{

 Bone0 #키 프레임: 18
 {
 시간: 0 위치: 2.538344 101.6727 -0.52932
 스케일: 1 1 1
 Quat: 0.4042651 0.3919331 -0.5853591 0.5833637
 시간: 0.0666666
 위치: 0.81979 109.6893 -1.575387
 스케일: 0.9999998 0.9999998 0.9999998
 Quat: 0.4460441 0.3467651 -0.5356012 0.6276384
 ...
 }

 Bone1 #키 프레임: 18
 {
 시간: 0
 위치: 36.48329 1.210869 92.7378
 규모: 1 1 1
 Quat: 0.126642 0.1367731 0.69105 0.6983587
 시간: 0.0666666
 위치: 36.30672 -2.835898 93.15854
 스케일: 1 1 1
 Quat: 0.1284061 0.1335271 0.6239273 0.7592083
 ...
 }
 ...
}
```

```
애니메이션 클립 walk_loop
{

 Bone0 #키 프레임: 33
 {
 시간: 0

 위치: 1.418595 98.13201 -0.051082

 크기: 0.9999985 0.999999 0.9999991
 Quat: 0.3164562 0.6437552 -0.6428624 0.2686314
 시간: 0.0333333

 위치: 0.956079 96.42985 -0.047988

 스케일: 0.9999999 0.9999999 0.9999999
 Quat: 0.3250651 0.6395872 -0.6386833 0.2781091
 ...
 }

 Bone1 #키 프레임: 33
 {
 시간: 0

 위치: -5.831432 2.521564 93.75848

 규모: 0.9999995 0.9999995 1
 Quat: -0.033817 -0.000631005 0.9097761 0.4137191
 시간: 0.0333333

 위치: -5.688324 2.551427 93.71078

 스케일: 0.9999998 0.9999998 1
 Quat: -0.033202 -0.0006390021 0.903874 0.426508
 ...
 }
 ...
}
...

void M3DLoader::ReadAnimationClips(std::ifstream& fin, UINT numBones, UINT numAnimationClips,
 std::map<std::string, AnimationClip>& animations)
{
 std::string ignore;
 fin >> ignore; // AnimationClips 헤더 텍스트
 for(UINT clipIndex = 0; clipIndex < numAnimationClips; ++clipIndex)
 {
 std::string clipName;
 fin >> ignore >> clipName; fin >> ignore;
 // {

 AnimationClip clip; clip.BoneAnimations.resize(numBones);

 for(UINT boneIndex = 0; boneIndex < numBones; ++boneIndex)
 {
 ReadBoneKeyframes(fin, numBones, clip.BoneAnimations[boneIndex]);
 }
 fin >> 무시; // }

 animations[clipName] = clip;
 }
}

void M3DLoader::ReadBoneKeyframes(std::ifstream& fin, UINT numBones,
 BoneAnimation& boneAnimation)
{
 std::string ignore;
 UINT numKeyframes = 0;
 fin >> ignore >> ignore >> numKeyframes;
```

```

fin >> ignore; // {

boneAnimation.Keyframes.resize(numKeyframes); for(UINT i = 0; i <
numKeyframes; ++i)
{
 float t = 0.0f;
 XMFLOAT3 p(0.0f, 0.0f, 0.0f);
 XMFLOAT3 s(1.0f, 1.0f, 1.0f);
 XMFLOAT4 q(0.0f, 0.0f, 0.0f, 1.0f);
 fin >> ignore >> t;
 fin >> ignore >> p.x >> p.y >> p.z; fin >> ignore >> s.x
 >> s.y >> s.z;

 fin >> ignore >> q.x >> q.y >> q.z >> q.w;

 boneAnimation.Keyframes[i].TimePos = t; boneAnimation.Keyframes[i].Translation = p;
 boneAnimation.Keyframes[i].Scale = s; boneAnimation.Keyframes[i].RotationQuat = q;
}

fin >> ignore; // }
}

```

## 25.4.5 M3DLoader

*LoadM3D.h/.cpp*에 *스킨 처리된 데이터*(SkinnedData)를 로드하기 위한 LoadM3d 함수의 오버로드가 추가되었습니다:

```

bool M3DLoader::LoadM3d(const std::string& filename,
 std::vector<Vertex::PosNormalTexTanSkinned>& vertices, std::vector<USHORT>& indices,
 std::vector<MeshGeometry::Subset>& subsets, std::vector<M3dMaterial>& mats,
 SkinnedData& skinInfo)
{
 std::ifstream fin(filename);

 UINT numMaterials = 0; UINT
 numVertices = 0; UINT numTriangles
 = 0; UINT numBones = 0;
 UINT numAnimationClips = 0; std::string ignore;

 if(fin)
 {
 fin >> ignore; // 파일 헤더 텍스트 fin >> ignore
 >> numMaterials; fin >> ignore >>
 numVertices; fin >> ignore >> numTriangles;
 fin >> ignore >> numBones;
 fin >> ignore >> numAnimationClips;

 std::vector<XMFLOAT4X4> boneOffsets; std::vector<int>
 boneIndexToParentIndex; std::map<std::string, AnimationClip> animations;

 ReadMaterials(fin, numMaterials, mats); ReadSubsetTable(fin, numMaterials, subsets);
 ReadSkinnedVertices(fin, numVertices, vertices); ReadTriangles(fin, numTriangles,
 indices); ReadBoneOffsets(fin, numBones, boneOffsets);
 ReadBoneHierarchy(fin, numBones, boneIndexToParentIndex);
 }
}

```

```
 ReadAnimationClips(fin, numBones, numAnimationClips, animations); skinInfo.Set(boneIndexToParentIndex, boneOffsets,
 animations);

 return true;
 }
 return false;
}
```

## 25.5 캐릭터 애니메이션 데모

§23.3에서 정의한 **BasicModel**에 대응하는 스킨 처리된 메쉬로 **SkinnedModel**을 정의합니다. 이 클래스는 **BasicModel**과 정확히 동일합니다

와 동일하지만 **SkinnedData** 인스턴스를 보유한다는 점이 다릅니다.

```
class SkinnedModel
{
public:
 SkinnedModel(ID3D11Device* device, TextureMgr& texMgr, const std::string&
 modelFilename,
 const std::wstring& texturePath);
 ~SkinnedModel(); UINT 서브셋

 카운트;

 std::vector<Material> Mat; std::vector<ID3D11ShaderResourceView*> DiffuseMapSRV;
 std::vector<ID3D11ShaderResourceView*> NormalMapSRV;

 // 읽기용으로 메쉬 데이터의 CPU 복사본을 유지합니다.
 std::vector<Vertex::PosNormalTexTanSkinned> Vertices; std::vector<USHORT> Indices;
 std::vector<MeshGeometry::Subset> Subsets;

 MeshGeometry ModelMesh;
 SkinnedData 스킨링데이터;
};
```

우리는 서로 다른 애니메이션 클립과 위치에서 여러 번 렌더링하여 **스키닝된 모델을** 인스턴스화할 수 있습니다.

이를 용이하게 하기 위해 다음과 같은 간단한 구조를 사용합니다:

```
struct SkinnedModelInstance
{
 SkinnedModel* Model; float
 TimePos; std::string ClipName;
 XMFLOAT4X4 World;
 std::vector<XMFLOAT4X4> FinalTransforms;

 void Update(float dt);
};

SkinnedModel 인스턴스는 애니메이션 위치, 애니메이션 중인 클립 이름, 최종 변환을 추적합니다. Update 메서드는 매 프레임 호출되어 시간 위치를 증가시키고, 현재 애니메이션 클립을 기반으로 각 분
의 애니메이션을 보간하며, 최종 변환을 생성합니다. 이 변환은 최종적으로 버텍스 셰이더에서 처리하기 위해 효과에 설정됩니다. 애니메이션이 끝날 때 시간 위치를 다시 0으로 초기화합니다.

void SkinnedModelInstance::Update(float dt)
{
 TimePos += dt;
 Model->SkinnedData.GetFinalTransforms(
```

```
ClipName, TimePos, FinalTransforms);
```

```
// 애니메이션 루프
```

```
if(TimePos > Model->SkinnedData.GetClipEndTime(ClipName)) TimePos = 0.0f;
```

```
}
```

데모에서는 스킨된 모델 인스턴스를 생성하고, SkinnedMeshApp::UpdateScene 메서드에서 **Update** 메서드를 호출하며, 최종 변환 배열을 효과에 설정하는 것을 잊지 않습니다. [그림 25.10](#)은 데모의 스크린샷입니다. 원본 애니메이션 모델과 텍스처는 DirectX SDK에서 가져와 데모 목적으로 .m3d 형식으로 변환했습니다. 이 샘플 모델에는 *Take1 0/1*라는 단일 애니메이션 클립만 포함되어 있습니다.



*그림 25.10. "스킨된 메시" 데모의 스크린샷.*

## 25.6 요약

1. 컴퓨터 프로그램에서 그래픽으로 모델링하고자 하는 많은 실제 세계의 객체들은 부모-자식 관계를 가진 부분들로 구성됩니다. 여기서 자식 객체는 독립적으로 움직일 수 있지만, 부모가 움직일 때 함께 움직여야 합니다. 예를 들어, 전차 포탑은 전차와 독립적으로 회전할 수 있지만, 여전히 전차에 고정되어 전차와 함께 움직입니다. 또 다른 대표적인 예는 해골 구조로, 뼈들이 서로 연결되어 있어 움직일 때 함께 움직여야 합니다. 기차 안의 게임 캐릭터를 생각해 보십시오. 캐릭터는 기차 안에서 독립적으로 움직일 수 있지만, 기차가 움직일 때도 함께 움직입니다. 이 예시는 게임에서 계층 구조가 동적으로 변경되고 업데이트되어야 함을 보여줍니다. 즉, 캐릭터가 기차에 탑승하기 전에는 기차가 캐릭터 계층 구조의 일부가 아니지만, 플레이어가 기차에 탑승하면 기차는 캐릭터 계층 구조의 일부가 됩니다.

캐릭터가 기차의 변환을 상속받음).

- 메시 계층 구조 내 각 객체는 회전을 용이하게 하기 위해 피벗 관절을 원점으로 하여 자체 로컬 좌표계에 대해 모델링됩니다. 좌표계들이 동일한 공간에 존재하므로 서로 연관시킬 수 있으며, 따라서 한 좌표계에서 다른 좌표계로 변환할 수 있습니다. 특히 각 객체의 좌표계를 부모 객체의 좌표계에 상대적으로 설명함으로써 이를 연관시킵니다. 이를 통해 객체의 지오메트리를 로컬 좌표계에서 부모 객체의 좌표계로 변환하는 '부모로 변환(*to-parent*)' 변환 행렬을 구성할 수 있습니다. 부모 객체의 좌표계에 도달한 후에는 부모 객체의 '부모로 변환' 행렬을 적용하여 조부모 객체의 좌표계로 변환하고, 이 과정을 각 상위 객체의 좌표계를 거쳐 최종적으로 월드 공간에 도달할 때까지 반복합니다. 다시 말해, 메시 계층 구조 내 객체를 로컬 공간에서 월드 공간으로 변환하려면 해당 객체와 모든 상위 객체(상승 순서)에 대한 부모 변환을 적용하여 좌표계 계층 구조를 상향 퍼콜레이션(*percolation*)하여 객체가 월드 공간에 도달하도록 합니다. 이렇게 하면 객체는 부모의 변환을 상속받아 부모가 이동할 때 함께 이동합니다.
- $i$ 번째 본의 루트 변환은 재귀 관계로 표현할 수 있습니다:  $toRoot_i = toParent_i \cdot toRoot_p$ , 여기서  $p$ 는  $i$ 번째 본의 부모 본을 가리킵니다.
- 본 오프셋 변환은 바인드 공간의 정점을 해당 본의 공간으로 변환합니다. 스켈레톤 내 각 본마다 오프셋 변환이 존재합니다.
- 버텍스 블렌딩에서는 기본적인 본 계층 구조가 존재하지만, 스킨 자체는 하나의 연속적인 메시이며, 하나 이상의 본이 하나의 버텍스에 영향을 미칠 수 있습니다. 본이 버텍스에 미치는 영향의 정도는 본 가중치에 의해 결정됩니다. 네 개의 본이 있을 때, 스켈레톤의 루트에 대한 변환된 버텍스  $v'$ 는 가중 평균 공식  $v' = w_0vF_0 + w_1vF_1 + w_2vF_2 + w_3vF_3$ 로 주어집니다. 여기서  $w_0 + w_1 + w_2 + w_3 = 1$ 입니다. 연속적인 메시를 사용하고 버텍스당 여러 개의 가중 본 영향을 사용함으로써 더 자연스러운 탄성 피부 효과를 얻을 수 있습니다.<sup>3</sup> = 1입니다. 연속적인 메시와 정점당 여러 개의 가중치 뼈 영향을 사용함으로써 더 자연스러운 탄성 피부 효과를 얻을 수 있습니다.
- 버텍스 블렌딩을 구현하기 위해, 각 본에 대한 최종 변환 행렬의 배열(이 배열을 *행렬 팔레트*라고 함)를 저장합니다. ( $i$ 번째 본의 최종 변환은  $F_i = offset_i \cdot toRoot_i$ 로 정의됩니다. 즉, 본의 오프셋 변환에 이어 루트 변환을 수행합니다.) 그런 다음 각 버텍스에 대해 버텍스 가중치와 행렬 팔레트 인덱스의 목록을 저장합니다. 한 버텍스의 매트릭스 팔레트 인덱스는 해당 버텍스에 영향을 미치는 뼈들의 최종 변환을 식별합니다.

## 25.7 연습 문제

- 수동으로 애니메이션 선형 계층 구조를 모델링하고 렌더링하세요. 예를 들어, 구와 원통 메시로 구성된 간단한 로봇 팔을 모델링할 수 있습니다. 구는 관절을, 원통은 팔을 모델링할 수 있습니다.
- 그림 25.11과 같은 애니메이션 트리 계층 구조를 손으로 모델링하고 렌더링하세요. 다시 구와 원통을 사용할 수 있습니다.

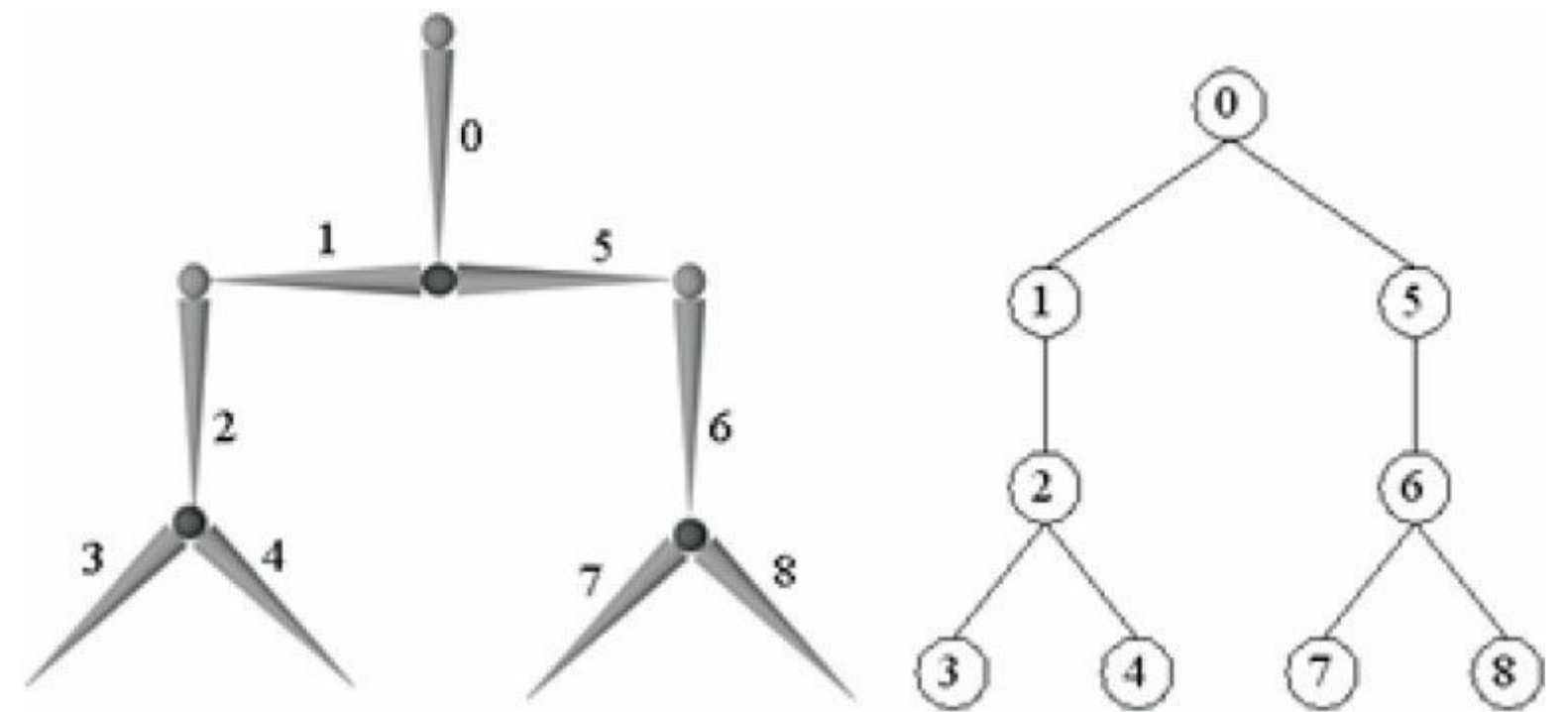


그림 25.11. 간단한 메시 계층 구조.

- 애니메이션 패키지(*블렌더* <http://www.blender.org/> 무료)를 사용할 수 있다면, 본과 블렌드 가중치를 사용해 간단한 애니메이션 캐릭터를 모델링하는 방법을 배워보세요. 데이터를 버텍스 블렌딩을 지원하는 .x 파일 형식으로 내보낸 후,

"Skinned Mesh" 데모에서 렌더링하기 위해 파일 데이터를 *.m3d* 형식으로 변환해 보세요. 이 작업은 간단한 프로젝트가 아닙니다.

# WINDOWS 프로그래밍 소개

## 프로그래밍

Direct3D API(응용 프로그램 프로그래밍 인터페이스)를 사용하려면 3D 장면을 렌더링할 메인 윈도우가 있는 Windows(Win32) 애플리케이션을 생성해야 합니다. 본 부록은 네이티브 Win32 API를 사용하여 Windows 애플리케이션을 작성하는 방법에 대한 소개 역할을 합니다. 대략적으로 Win32 API는 C 프로그래밍 언어에서 노출되는 저수준 함수 및 구조체 집합으로, 이를 통해 Windows 애플리케이션을 생성할 수 있습니다. 예를 들어, 창 클래스를 정의하려면 Win32 API의 **WNDCLASS** 구조체 인스턴스를 작성합니다. 창을 생성하려면 Win32 API의 **CreateWindow** 함수를 사용합니다. 특정 창을 표시하도록 Windows에 알려려면

특정 창을 표시하도록 알려려면 Win32 API 함수 **ShowWindow**를 사용합니다.

윈도우 프로그래밍은 방대한 주제이며, 본 부록에서는 Direct3D 사용에 필요한 최소한의 내용만을 소개합니다.

Win32 API를 활용한 Windows 프로그래밍에 대해 더 깊이 배우고 싶은 독자라면, 찰스 페츨드(Charles Petzold)의 『*Programming Windows*』(현재 5판)이 이 분야의 표준 교재입니다. Microsoft 기술 작업 시 또 다른 귀중한 자료는 MSDN 라이브러리로, 일반적으로 Microsoft Visual Studio에 포함되어 있지만 [www.msdn.microsoft.com](http://www.msdn.microsoft.com)에서 온라인으로도 열람할 수 있습니다. 일반적으로 더 자세히 알고 싶은 Win32 함수나 구조체를 발견하면 MSDN으로 이동하여 해당 함수나 구조체를 검색하여 전체 문서를 확인하십시오. 본 부록에서 Win32 함수나 구조체를 언급하되 자세히 설명하지 않는 경우, 이는 독자가 MSDN에서 해당 함수를 찾아보라는 암묵적인 제안입니다.

### 목표:

1. Windows 프로그래밍에서 사용되는 이벤트 주도형 프로그래밍 모델을 학습하고 이해하는 것.
2. Direct3D를 사용하기 위해 필요한 최소한의 코드를 학습하여 Windows 애플리케이션을 생성하는 방법을 익힙니다.



혼동을 피하기 위해 운영체제(OS)인 Windows를 가리킬 때는 대문자 'W'를 사용하고, Windows 내에서 실행 중인 특정 창을 가리킬 때는 소문자 'w'를 사용합니다.

## A.1 개요

이름에서 알 수 있듯이, Windows 프로그래밍의 주요 주제 중 하나는 창(window) 프로그래밍입니다. Windows 애플리케이션의 많은 구성 요소들은 창입니다. 예를 들어 메인 애플리케이션 창, 메뉴, 도구 모음, 스크롤 바, 버튼, 기타 대화 상자 컨트롤 등이 있습니다. 따라서 Windows 애플리케이션은 일반적으로 여러 개의 창으로 구성됩니다. 다음 하위 섹션들은 보다 완전한 논의를 시작하기 전에 숙지해야 할 Windows 프로그래밍 개념에 대한 간결한 개요를 제공합니다.

### A.1.1 참고 자료

Windows에서는 여러 응용 프로그램이 동시에 실행될 수 있습니다. 따라서 CPU 사이클, 메모리, 심지어 모니터 화면과 같은 하드웨어 리소스는 여러 응용 프로그램이 공유해야 합니다. 여러 응용 프로그램이 아무런 체계 없이 리소스에 접근하거나 수정함으로써 혼란이 발생하는 것을 방지하기 위해, Windows 응용 프로그램은 하드웨어에 직접 접근할 수 없습니다. Windows의 주요 역할 중 하나는 현재 실행 중인 애플리케이션을 관리하고 이들 간에 자원을 분배하는 것입니다. 따라서 우리 애플리케이션이 다른 실행 중인 애플리케이션에 영향을 미칠 수 있는 작업을 수행하려면 반드시 Windows를 거쳐야 합니다. 예를 들어, 창을 표시하려면 Win32 API 함수인 **ShowWindow**를 호출해야 하며, 비디오 메모리에 직접 쓰기 작업을 수행할 수 없습니다.

### A.1.2 이벤트, 메시지 큐, 메시지, 메시지 루프

Windows 애플리케이션은 *이벤트 주도형 프로그래밍 모델*을 따릅니다. 일반적으로 Windows 애플리케이션은 무언가(이벤트)가 발생하기를 기다리며 대기 상태에 있습니다<sup>1</sup>. 이벤트는 다양한 방식으로 생성될 수 있으며, 일반적인 예로는 키 입력, 마우스 클릭, 창 생성, 크기 조정, 이동, 닫기, 최소화, 최대화, 표시 상태 전환 등이 있습니다.

이벤트가 발생하면 Windows는 해당 이벤트가 발생한 애플리케이션에 *메시지*를 전송하고, 이 메시지를 애플리케이션의 메시지 큐에 추가합니다.

응용 프로그램의 *메시지 큐*에 추가합니다. 이 메시지 큐는 단순히 응용 프로그램의 메시지를 저장하는 우선순위 큐입니다. 응용 프로그램은 *메시지 루프*에서 지속적으로 메시지 큐를 확인하며, 메시지를 수신하면 해당 메시지를 *창*으로 전달합니다.

특정 창에 대한 메시지의 *처리* 절차입니다. (한 애플리케이션 내에 여러 창이 포함될 수 있음을 기억하십시오.) 모든 창에는 창 처리 함수(window procedure)라고 불리는 연관된 함수가 있습니다.<sup>2</sup> 창 처리 함수는 특정 메시지에 응답하여 실행될 코드를 포함하는 우리가 구현하는 함수입니다. 예를 들어, Escape 키가 눌렸을 때 창을 파괴하고 싶을 수 있습니다. 창 처리 함수에서 다음과 같이 작성합니다:

```
case WM_KEYDOWN: if(wParam ==
VK_ESCAPE)
DestroyWindow(ghMainWnd); return 0;
```

창이 처리하지 않는 메시지는 기본 창 처리 프로시저로 전달되어야 하며, 이 프로시저가 해당 메시지를 처리합니다.

Win32 API는 **DefWindowProc**라는 기본 창 프로시저를 제공합니다.

요약하자면, 사용자나 애플리케이션이 어떤 작업을 수행하여 이벤트를 생성합니다. 운영체제는 해당 이벤트가

해당 애플리케이션에 응답 메시지를 보냅니다. 이 메시지는 애플리케이션의 메시지 큐에 추가됩니다. 애플리케이션은 지속적으로 메시지 큐를 확인하며 메시지를 수신합니다. 메시지를 수신하면 애플리케이션은 해당 메시지의 대상이 되는 창의 윈도우 프로시저로 메시지를 전달합니다. 마지막으로 윈도우 프로시저는 메시지에 대한 응답으로 명령어를 실행합니다.

[그림 \(A.1\)](#)은 이벤트 주도형 프로그래밍 모델을 요약합니다.

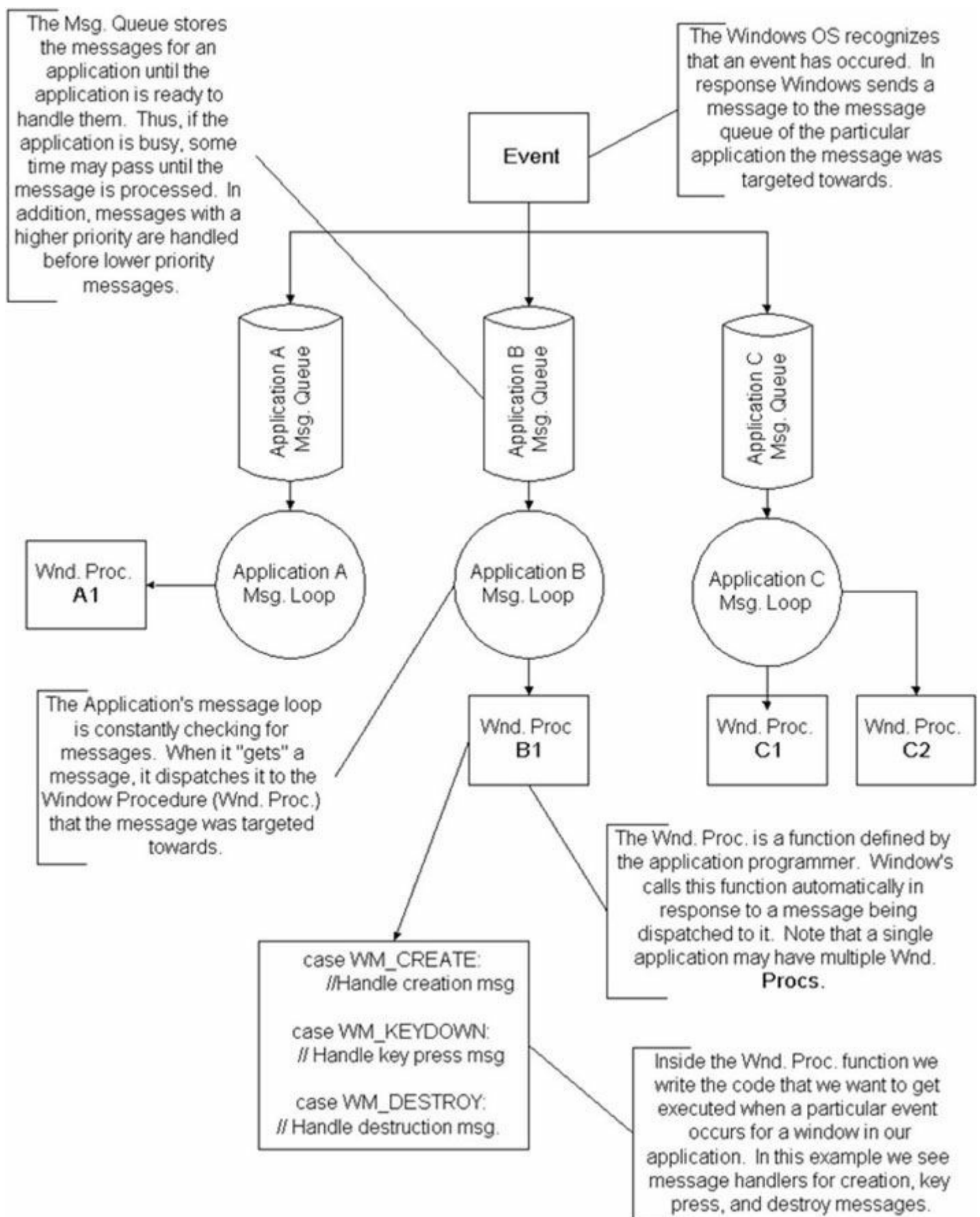


그림 A.1. 이벤트 주도형 프로그래밍 모델.

## A.1.3 GUI

대부분의 Windows 프로그램은 사용자가 작업할 수 있는 GUI(그래픽 사용자 인터페이스)를 제공합니다. 일반적인 Windows 애플리케이션은 하나의 메인 창, 메뉴, 도구 모음 및 기타 컨트롤을 포함합니다. [그림 A.2](#)는 일반적인 GUI 요소를 보여주고 설명합니다. Direct3D 게임 프로그래밍에서는 화려한 GUI가 필요하지 않습니다. 사실, 우리가 필요한 것은 메인 창뿐이며, 클라이언트 영역은 3D 세계를 렌더링하는 데 사용됩니다.

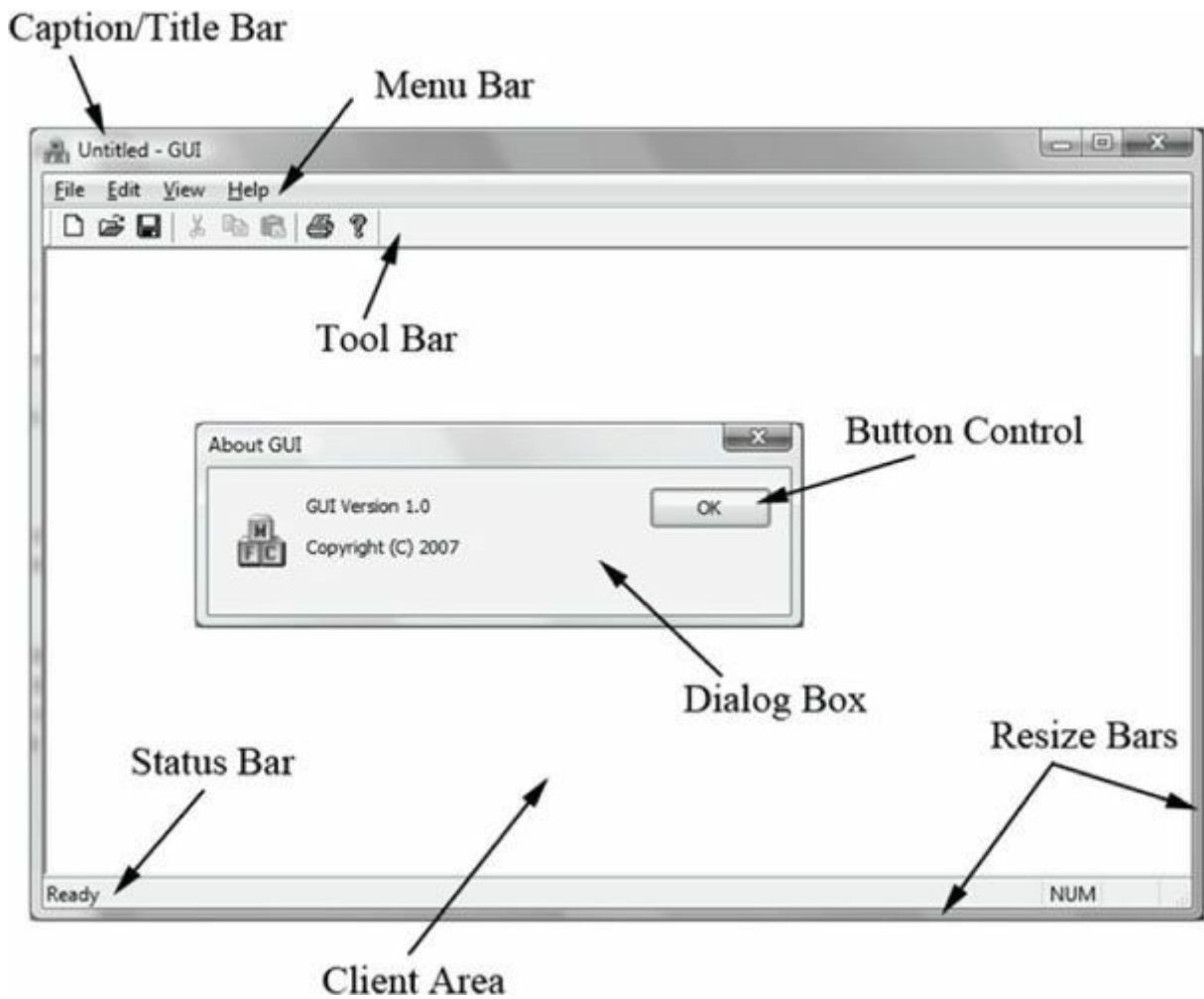


그림 A.2. 전형적인 Windows 애플리케이션 GUI. 클라이언트 영역은 애플리케이션의 전체 큰 흰색 직사각형 공간입니다. 일반적으로 이 영역은 사용자가 프로그램 출력의 대부분을 보는 곳입니다. Direct3D 애플리케이션을 프로그래밍할 때, 우리는 3D 장면을 창의 클라이언트 영역에 렌더링합니다.

## A.1.4 유니코드

기본적으로 유니코드(<http://unicode.org/>)는 16비트 값을 사용하여 문자를 표현합니다. 이를 통해 국제 문자 및 기타 기호를 지원하기 위한 더 큰 문자 집합을 표현할 수 있습니다. C++에서 유니코드를 사용하려면 와이드 문자 유형 `wchar_t`를 사용합니다. 32비트 및 64비트 Windows에서 `wchar_t`는 16비트입니다. 와이드 문자를 사용할 때는 문자열 리터럴 앞에 대문자 L을 붙여야 합니다. 예를 들어:

```
const wchar_t* wstrPtr = L"Hello, World!";
```

L은 컴파일러에게 문자열 리터럴을 와이드 문자열(즉, `char` 대신 `wchar_t`)로 처리하도록 지시합니다. 다른 중요한 점은 문자열 함수의 와이드 문자 버전을 사용해야 한다는 것입니다. 예를 들어 문자열 길이를 구하려면 `strlen` 대신 `wcslen`을, 문자열을 복사하려면 `strcpy` 대신 `wcsncpy`를, 두 문자열을 비교하려면 `strcmp` 대신 `wcsncmp`를 사용해야 합니다. 이러한 함수의 와이드 문자 버전은 `char` 포인터 대신 `wchar_t` 포인터로 작동합니다. C++ 표준 라이브러리도 문자열 클래스의 와이드 문자 버전을 제공합니다: `std::wstring`. Windows 헤더 파일 `WinNT.h`에서도 다음과 같이 정의합니다:

```
typedef wchar_t WCHAR; // wc, 16비트 유니코드 문자
```

## A.2 기본 Windows 애플리케이션

다음은 완전한 기능을 갖추면서도 간단한 Windows 프로그램의 코드입니다. 가능한 한 코드를 따라가며 설명 주석을 읽어보십시오. 다음 섹션에서는 코드를 단계별로 설명합니다. 연습으로 개발 도구를 사용하여 프로젝트를 생성하고, 코드를 직접 입력한 후 컴파일하여 실행해 보십시오. Visual C++의 경우 "Win32 콘솔 애플리케이션 프로젝트"가 아닌 "Win32 애플리케이션 프로젝트"를 생성해야 합니다.

```
//=====
// Win32Basic.cpp by Frank Luna (C) 2008 All Rights Reserved.
//
```

```
// Win32 코드에서 Direct3D 프로그래밍에 필요한 최소한의 양을 보여줍니다.

// Direct3D 프로그래밍에 필요한 최소한의 Win32 코드를 보여줍니다.
//===

// 윈도우 헤더 파일을 포함합니다; 여기에는 프로그래밍에 필요한 모든 Win32 API

// 구조체, 유형 및 함수 선언을 포함합니다.
//
#include <windows.h>

// 메인 윈도우 핸들; 생성된 윈도우를 식별하는 데 사용됩니다.

// 생성된 창을 식별하는 데 사용됩니다.
HWND ghMainWnd = 0;

// Windows 애플리케이션 초기화에 필요한 코드를 감쌉니다.

// 함수는 초기화가 성공하면 true를 반환합니다.

// 성공한 경우 true를 반환하고, 그렇지 않으면 false를 반환합니다.
bool InitWindowsApp(HINSTANCE instanceHandle, int show);

// 메시지 루프 코드를 감쌉니다. int Run();

// 창 프로시저는 창이 수신하는 이벤트를 처리합니다. LRESULT CALLBACK
WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam);

// Windows의 main()에 해당하는 함수 int WINAPI
WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR pCmdLine, int nShowCmd)
{
 // 먼저 래퍼 함수(InitWindowsApp)를 호출하여
 // 메인 애플리케이션 창을 생성하고 초기화합니다.

 // hInstance 및 nShowCmd 값을 인수로 전달합니다. if(!InitWindowsApp(hInstance,
 nShowCmd))

 return 0;

 // 애플리케이션이 생성 및 초기화되면
 // 메시지 루프에 진입합니다. 메시지 루프는 다음 조건이 충족될 때까지 유지됩니다.
 // WM_QUIT 메시지가 수신되어 응용 프로그램이 종료되어야 함을 나타냅니다.
 // 종료되어야 함을 나타냅니다. return
 Run();
}

bool InitWindowsApp(HINSTANCE instanceHandle, int show)
{
 // 창을 생성하기 위한 첫 번째 작업은 WNDCLASS 구조체를 작성하여 창 특성의 일부를 설명하는 것입니다.
 // 특성을 WNDCLASS 구조체를 작성하여 정의하는 것입니다. WNDCLASS wc;

 wc.style = CS_HREDRAW | CS_VREDRAW;
 wc.lpfnWndProc = WndProc;
 wc.cbClsExtra = 0;
 wc.cbWndExtra = 0;
 wc.hInstance = 인스턴스 핸들;
 wc.hIcon = LoadIcon(0, IDI_APPLICATION);
 wc.hCursor = LoadCursor(0, IDC_ARROW); wc.hbrBackground =
(HBRUSH)GetStockObject(WHITE_BRUSH); wc.lpszMenuName = 0;
 wc.lpszClassName = L"BasicWndClass";

 // 다음으로, 이 WNDCLASS 인스턴스를 Windows에 등록합니다.
 // 이를 기반으로 창을 생성할 수 있습니다.
```

```

if(!RegisterClass(&wc))
{
 MessageBox(0, L"클래스 등록 실패", 0, 0); return false;
}

//WNDCLASS 인스턴스가 등록되었으므로

// CreateWindow 함수로 창을 생성할 수 있습니다. 이 함수는
// 생성된 창에 대한 핸들(HWND)을 반환합니다.
// 생성 실패 시 핸들 값은
// 0이 됩니다. 창 핸들은 창을 참조하는 방법입니다.
// 이는 Windows 내부에서 관리됩니다. Win32 API의 많은 함수들은
// 창을 조작하는 Win32 API 함수들은 대부분 HWND를 필요로 합니다.
// 어떤 창에 대해 동작할지 알 수 있도록 합니다.

ghMainWnd = CreateWindow(
 L"BasicWndClass", // 사용할 등록된 WNDCLASS 인스턴스. L"Win32Basic", // 창 제목
 WS_OVERLAPPEDWINDOW, // 스타일 플래그 CW_USEDEFAULT, // x 좌표
 CW_USEDEFAULT, // y-좌표
 CW_USEDEFAULT, // 너비
 CW_USEDEFAULT, // 높이
 0, // 부모 창
 0, // 메뉴 핸들 인스턴스 핸들, //
 앱 인스턴스
 0); // 추가 생성 매개변수 if(ghMainWnd == 0)
{
 MessageBox(0, L"CreateWindow 실패", 0, 0); return false;
}

// 방금 창을 생성했지만, 초기 상태에서는
// 표시됩니다. 따라서 마지막 단계는 방금 생성한
// 방금 생성한 창을 표시하고 업데이트하는 것이며, 이는 다음
// 두 함수 호출로 수행할 수 있습니다. 표시하고 업데이트하려는
// 표시하고 업데이트할 창을 알 수 있도록 합니다.
// 표시 및 업데이트할 창을 인식하도록 합니다. //
ShowWindow(ghMainWnd, show); // UpdateWindow(ghMainWnd);

return true;
}

int Run()
{
 MSG msg = {0};

 // WM_QUIT 메시지를 받을 때까지 반복합니다. 이 함수는
 // GetMessage는 WM_QUIT 메시지가 수신되었을 때만 0(false)을 반환하며
 // 수신되었을 때만 0(거짓)을 반환하며, 이는 효과적으로 루프를 종료합니다.
 // 오류 발생 시 -1을 반환합니다. 또한 GetMessage는
 // 메시지가 있을 때까지 애플리케이션 스레드를 대기 상태로 만듭니다.
 // 메시지가 있을 때까지 애플리케이션 스레드를 잠들
 게 합니다. BOOL bRet =
 1;
 while((bRet = GetMessage(&msg, 0, 0, 0)) != 0)
 {
 if(bRet == -1)
 {
 MessageBox(0, L"GetMessage 실패", L"오류", MB_OK); break;

```

```

 }
 else
 {

 TranslateMessage(&msg);
 DispatchMessage(&msg);
 }
}

return (int)msg.wParam;
}

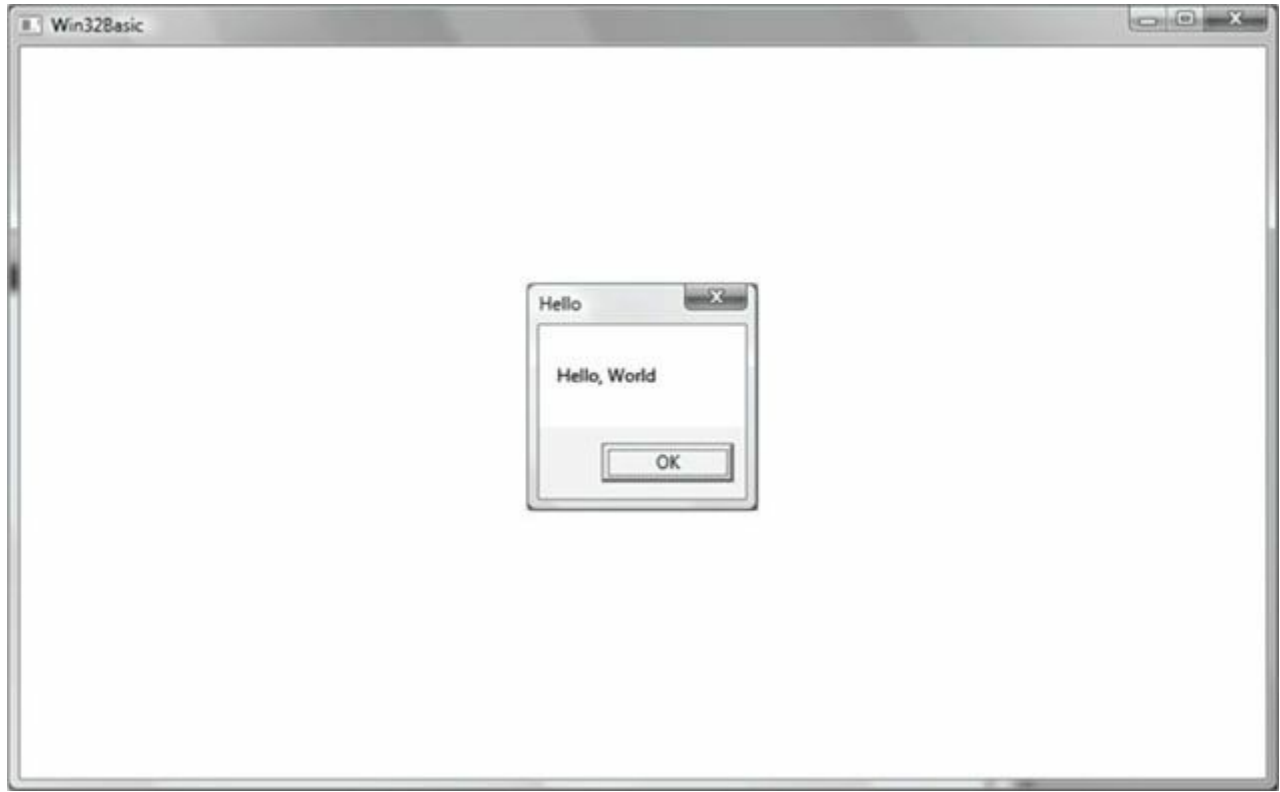
LRESULT 콜백
WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam)
{
 // 특정 메시지를 처리합니다. 메시지를 처리할 경우
 // 메시지를 처리한 후에는 0을 반환해야 합니다.
 switch(msg)
 {
 // 마우스 왼쪽 버튼이 눌린 경우,
 // 메시지 상자를 표시합니다. case
 WM_LBUTTONDOWN:
 MessageBox(0, L"Hello, World", L"Hello", MB_OK); return 0;

 // Esc 키가 눌린 경우,
 // 메인 애플리케이션 창을 파괴합니다. case WM_KEYDOWN:
 if(wParam == VK_ESCAPE) DestroyWindow(ghMainWnd);
 return 0;

 // 파괴 메시지인 경우, 메시지 루프를 종료하는
 // 메시지 루프를 종료하는 종료 메시지를 보냅니다. case WM_DESTROY:
 PostQuitMessage(0); return 0;
 }

 // 위에서 처리하지 않은 다른 메시지는 모두
 // 기본 창 프로시저로 전달합니다. 우리의 창 프로시저는
 // 반드시 DefWindowProc의 반환값을 반환해야 합니다. return DefWindowProc(hWnd,
 msg, wParam, lParam);
}

```



*그림 A.3.* 이전 프로그램의 스크린샷. 마우스 왼쪽 버튼을 창 클라이언트 영역에서 누르면 메시지 상자가 나타남을 확인하십시오. 또한 Esc 키를 눌러 프로그램을 종료해 보십시오.

## A.3 기본 윈도우 애플리케이션 설명

코드를 상단부터 하단까지 검토하며, 호출되는 모든 함수를 단계별로 살펴보겠습니다. 다음 하위 섹션에서는 "기본 Windows 애플리케이션" 섹션의 코드 목록을 참조하십시오.

### A.3.1 포함, 전역 변수 및 프로토타입

가장 먼저 `windows.h` 헤더 파일을 포함합니다. `windows.h` 파일을 포함함으로써 Win32 API의 기본 요소를 사용하기 위해 필요한 구조체, 유형 및 함수 선언을 얻습니다.

```
#include <windows.h>
```

두 번째 문장은 **HWND** 유형의 전역 변수를 선언하는 것입니다. HWND는 "창 핸들" 또는 "창 손잡이"를 의미합니다. Windows 프로그래밍에서는 Windows가 내부적으로 관리하는 객체를 참조하기 위해 핸들을 자주 사용합니다. 이 예제에서는 Windows가 관리하는 메인 애플리케이션 창을 참조하기 위해 **HWND**를 사용할 것입니다. API 호출의 상당수는 해당 호출이 적용될 창 핸들을 전달할 것을 요구하므로, 창 핸들을 보유해야 합니다. 예를 들어 **UpdateWindow** 호출은 업데이트할 창을 지정하는 데 사용되는 **HWND** 유형의 인자 하나를 받습니다. 핸들을 전달하지 않으면 함수가 업데이트할 창을 알 수 없습니다.

```
HWND ghMainWnd = 0;
```

다음 세 줄은 함수 선언입니다. 간단히 말해, **InitWindowsApp**은 메인 애플리케이션 창을 생성하고 초기화합니다; **Run**은 애플리케이션의 메시지 루프를 캡슐화합니다; **WndProc**은 메인 창의 창 처리 함수입니다. 이 함수들은 호출되는 지점에 도달했을 때 더 자세히 살펴보겠습니다.

```
bool InitWindowsApp(HINSTANCE instanceHandle, int show); int Run();
LRESULT CALLBACK
WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam);
```

### A.3.2 WinMain

**WinMain**은 일반적인 C++ 프로그래밍에서 `main` 함수에 해당하는 Windows 함수입니다. **WinMain**의 프로토타입은 다음과 같습니다:

```
int WINAPI
WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, PSTR pCmdLine, int nShowCmd)
```

- 1. **hInstance**: 현재 애플리케이션 인스턴스에 대한 핸들입니다. 이 애플리케이션을 식별하고 참조하는 역할을 합니다. 여러 Windows 애플리케이션이 동시에 실행될 수 있으므로 각각을 참조할 수 있는 방법이 유용합니다.
- 2. **hPrevInstance**: Win32 프로그래밍에서는 사용되지 않으며 0입니다.
- 3. **pCmdLine**: 프로그램을 실행하는 데 사용되는 명령줄 인수 문자열입니다.
- 4. **nCmdShow**: 응용 프로그램의 표시 방식을 지정합니다. 현재 크기 및 위치로 창을 표시하는 **SW\_SHOW**, 최대화 상태로 표시하는 **SW\_SHOWMAXIMIZED**, 최소화 상태로 표시하는 **SW\_SHOWMINIMIZED** 등이 일반적인 명령어입니다. 표시 명령어의 전체 목록은 MSDN 라이브러리를 참조하십시오.

**WinMain**이 성공하면 **WM\_QUIT** 메시지의 **wParam** 멤버를 반환해야 합니다. 메시지 루프에 진입하지 않고 함수가 종료되면 0을 반환해야 합니다. **WINAPI** 식별자는 다음과 같이 정의됩니다:

```
#define WINAPI__ stdcall
```

이것은 함수의 호출 규칙을 지정하며, 이는 함수 인자가 스택에 어떻게 배치되는지를 의미합니다.

### A.3.3 WNDCLASS와 등록

**WinMain** 내부에서 **InitWindowsApp** 함수를 호출합니다. 예상할 수 있듯이 이 함수는 프로그램의 모든 초기화를 수행합니다. 이 함수와 그 구현을 자세히 살펴보겠습니다. **InitWindowsApp**는 초기화가 성공하면 **true**를, 실패하면 **false**를 반환합니다. **WinMain** 정의에서 우리는 애플리케이션 인스턴스의 복사본과 표시 명령 변수를 인수로 **InitWindowsApp**에 전달합니다. 둘 다 **WinMain** 매개변수 목록에서 얻어집니다.

```
if(!InitWindowsApp(hInstance, nShowCmd))
```

창 초기화 시 가장 먼저 수행해야 할 작업은 **WNDCLASS**

(창 클래스) 구조체를 작성하여 창에 대한 기본 속성을 설명하는 것입니다. 그 정의는 다음과 같습니다:

```
typedef struct _WNDCLASS { UINT style;
 WNDPROC lpfnWndProc; int
 cbClsExtra;
 int cbWndExtra; HANDLE hInstance;
 HICON hIcon; HCURSOR hCursor;
 HBRUSH hbrBackground;
 LPCTSTR lpszMenuName; LPCTSTR
 lpszClassName;
} WNDCLASS;
```

**style** : 클래스 스타일을 지정합니다. 본 예제에서는 **CS\_HREDRAW**와 **CS\_VREDRAW**를 조합하여 사용합니다. 이 두 비트 플래그는

- 1. 수평 또는 수직 창 크기가 변경될 때 창을 다시 그리도록 지시합니다. 다양한 스타일의 전체 목록과 설명은 MSDN 라이브러리를 참조하십시오.

```
wc.style = CS_HREDRAW | CS_VREDRAW;
```

**lpfnWndProc**: 이 **WNDCLASS** 인스턴스와 연결할 창 처리 함수 포인터입니다. 이 **WNDCLASS** 인스턴스를 기반으로 생성된 창들은 이 창 처리 함수를 사용합니다. 따라서 동일한

- 2. 창 프로시저에서는 동일한 **WNDCLASS** 인스턴스를 기반으로 두 개의 창을 생성하기만 하면 됩니다. 서로 다른 창 프로시저를 가진 두 개의 창을 생성하려면 두 창 각각에 대해 서로 다른 **WNDCLASS** 인스턴스를 지정해야 합니다. 창 프로시저 함수에 대한 설명은 A.3.6절을 참조하십시오.

```
wc.lpfnWndProc = WndProc;
```

- 3. **cbClsExtra** 및 **cbWndExtra**: 이 메모리 영역은 사용자 목적에 따라 활용할 수 있는 추가 공간입니다. 본 프로그램은 추가 공간이 필요하지 않으므로 두 값 모두 0으로 설정합니다.

```
wc.cbClsExtra = 0;
```

```
wc.cbWndExtra = 0;
```

4. **hInstance**: 이 필드는 애플리케이션 인스턴스에 대한 핸들입니다. 애플리케이션 인스턴스 핸들은 원래 **WinMain**.

**wc.hInstance = instanceHandle;**

**hIcon**: 여기에서는 이 창 클래스를 사용하여 생성된 창에 사용할 아이콘 핸들을 지정합니다. 사용자 정의 아이콘을 사용할 수 있습니다.

5. 디자인된 아이콘이지만 선택할 수 있는 여러 내장 아이콘이 있습니다. 자세한 내용은 MSDN 라이브러리를 참조하십시오. 다음은 기본 애플리케이션 아이콘을 사용하는 예입니다:

**wc.hIcon = LoadIcon(0, IDI\_APPLICATION);**

**hCursor**: **hIcon**과 유사하게, 여기서는 커서가 창 클라이언트 영역 위에 있을 때 사용할 커서 핸들을 지정합니다.

6. 여기에도 여러 내장 커서가 있습니다. 자세한 내용은 MSDN 라이브러리를 참조하십시오. 다음 코드는 표준 "화살표" 커서를 사용합니다.

**wc.hCursor = LoadCursor(0, IDC\_ARROW);**

**hbrBackground**: 이 필드는 창 클라이언트 영역의 배경색을 지정하는 브러시 핸들을 지정하는 데 사용됩니다.

7. 우리 샘플 코드에서는 미리 만들어진 흰색 브러시의 핸들을 반환하는 Win32 함수 **GetStockObject**를 **호출합니다**. 다른 유형의 내장 브러시에 대해서는 MSDN 라이브러리를 참조하십시오.

**wc.hbrBackground = (HBRUSH)GetStockObject(WHITE\_BRUSH);**

8. **lpzMenuName**: 창의 메뉴를 지정합니다. 애플리케이션에 메뉴가 없으므로 이 값을 0으로 설정합니다.

**wc.lpszMenuName = 0;**

**lpzClassName**: 생성 중인 창 클래스 구조체의 이름을 지정합니다. 원하는 이름으로 설정할 수 있습니다. 본

9. 응용 프로그램에서는 "BasicWndClass"로 명명했습니다. 이 이름은 단순히 클래스 구조체를 식별하는 데 사용되며, 나중에 이름을 통해 참조할 수 있도록 합니다.

**wc.lpszClassName = L"BasicWndClass";**

**WNDCLASS** 인스턴스를 작성한 후에는 이를 Windows에 등록해야 해당 클래스를 기반으로 창을 생성할 수 있습니다. 이는 **WNDCLASS** 구조체 포인터를 인수로 받는 **RegisterClass** 함수를 통해 수행됩니다. 이 함수는 실패 시 0을 반환합니다.

```
if(!RegisterClass(&wc))
{
 MessageBox(0, L"RegisterClass 실패", 0, 0); return false;
}
```

### A.3.4 창 생성 및 표시

**WNDCLASS** 변수를 Windows에 등록한 후, 해당 클래스 설명을 기반으로 창을 생성할 수 있습니다. 등록된 **WNDCLASS** 인스턴스는 우리가 부여한 클래스 이름(**lpzClassName**)을 통해 참조할 수 있습니다.

창 생성에 사용하는 함수는 **CreateWindow** 함수로, 다음과 같이 선언됩니다:

```
HWND CreateWindow(LPCTSTR
 lpClassName, LPCTSTR lpWindowName,
 DWORD dwStyle,
 int x,
 int y,
 int nWidth, int
 nHeight,
 HWND hWndParent, HMENU hMenu,
 HANDLE hInstance, LPVOID
 lpParam
);
```

1. **lpClassName**: 생성하려는 창의 일부 속성을 설명하는 등록된 **WNDCLASS** 구조체의 이름입니다.

- 2. **lpWindowName**: 창에 부여할 이름입니다. 이 이름은 창 제목 표시줄에도 표시됩니다.
- 3. **dwStyle**: 창의 스타일을 정의합니다. **WS\_OVERLAPPEDWINDOW**는 여러 플래그의 조합입니다: **WS\_OVERLAPPED**, **WS\_CAPTION**, **WS\_SYSMENU**, **WS\_THICKFRAME**, **WS\_MINIMIZEBOX** 및 **WS\_MAXIMIZEBOX**. 이러한 플래그 이름은 생성되는 창의 특성을 설명합니다. 전체 스타일 목록은 MSDN 라이브러리를 참조하십시오.
- 4. **x**: 화면을 기준으로 한 창 좌측 상단 모서리의 x 좌표. 이 매개변수에 **CW\_USEDEFAULT**를 지정하면 Windows가 적절한 기본값을 선택합니다.
- 5. **y**: 화면을 기준으로 한 창 좌측 상단 모서리의 y 위치. 이 매개변수에 **CW\_USEDEFAULT**를 지정하면 Windows가 적절한 기본값을 선택합니다.
- 6. **nWidth**: 창의 너비(픽셀 단위). 이 매개변수에 **CW\_USEDEFAULT**를 지정하면 Windows가 적절한 기본값을 선택합니다.
- 7. **nHeight**: 창의 높이(픽셀 단위). 이 매개변수에 **CW\_USEDEFAULT**를 지정하면 Windows가 적절한 기본값을 선택합니다.
- 8. **hWndParent**: 이 창의 부모가 될 창의 핸들입니다. 본 창은 다른 창과 아무런 관계가 없으므로 이 값을 0으로 설정합니다.
- 9. **hMenu**: 메뉴 핸들입니다. 본 프로그램은 메뉴를 사용하지 않으므로 이 필드에 0을 지정합니다.
- 10. **hInstance**: 창이 연관될 애플리케이션의 핸들입니다.
- 11. **lpParam**: WM\_CREATE 메시지 핸들러에서 사용할 수 있도록 원하는 사용자 정의 데이터에 대한 포인터입니다. **WM\_CREATE** 메시지는 창이 생성될 때, 그러나 **CreateWindow**가 반환되기 전에 창에 전송됩니다. 창은 생성될 때 특정 작업을 수행하려는 경우(예: 초기화) **WM\_CREATE** 메시지를 처리합니다.

창 위치의 (x, y) 좌표를 지정할 때, 이 좌표는 화면 좌표계(  )의 왼쪽 상단 모서리를 기준으로 합니다. 또한 양의 x 축은 평소처럼 오른쪽으로 향하지만, 양의 y 축은 아래쪽으로 향합니다. 그림 A.4는 이 좌표계를 화면 좌표계(screen coordinates) 또는 화면 공간(screen space)이라고 부릅니다.

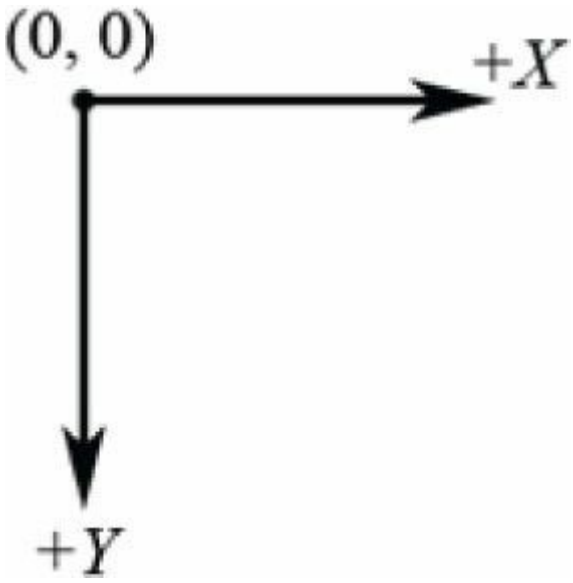


그림 A.4. 화면 공간. **CreateWindow**는 생성한 창에 대한 핸들(HWND)을 반환합니다. 생성 실패 시 핸들은 0 값(널 핸들)을 가집니다. 핸들은 Windows가 관리하는 창을 참조하는 방법임을 기억하십시오. 많은 API 호출은 어떤 창에 동작할지 알기 위해 HWND를 요구합니다.

```
ghMainWnd = CreateWindow(L"BasicWndClass", L"Win32Basic", WS_OVERLAPPEDWINDOW,
 CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT,
 CW_USEDEFAULT,
 0, 0, instanceHandle, 0);
if(ghMainWnd == 0)
{
 MessageBox(0, L"CreateWindow 실패", 0, 0); return false;
}
```

InitWindowsApp 함수의 마지막 두 함수 호출은 창 표시와 관련이 있습니다. 먼저 **ShowWindow**를 호출하여 새로 생성한 창의 핸들을 전달함으로써 Windows가 표시할 창을 알 수 있도록 합니다. 또한 창이 처음에 어떻게 표시될지 정의하는 정수 값(예: 최소화, 최대화 등)도 전달합니다. 이 값은 **WinMain**의 매개변수인 **nShowCmd**여야 합니다. 창을 표시한 후에는 새로 고쳐야 합니다. UpdateWindow가 이를 수행하며, 업데이트하려는 창에 대한 핸들을 하나의 인자로 받습니다.

```
ShowWindow(ghMainWnd, show);
UpdateWindow(ghMainWnd);
```

**InitWindowsApp**에서 여기까지 진행했다면 초기화가 완료된 것입니다. 모든 작업이 성공적으로 수행되었음을 나타내기 위해 **true**를 반환합니다.

## A.3.5 메시지 루프

초기화를 성공적으로 완료했으므로 프로그램의 핵심인 메시지 루프를 시작할 수 있습니다. 기본 Windows 애플리케이션에서는 메시지 루프를 **Run**이라는 함수로 감쌌습니다.

```
int Run()
{
 MSG msg = {0};

 BOOL bRet = 1;
 while((bRet = GetMessage(&msg, 0, 0, 0)) != 0)
 {
 if(bRet == -1)
 {
 MessageBox(0, L"GetMessage 실패", L"오류", MB_OK); break;

 }
 else
 {
 TranslateMessage(&msg);
 DispatchMessage(&msg);

 }
 }
 return (int)msg.wParam;
}
```

**Run**에서 가장 먼저 수행되는 작업은 **MSG** 유형의 **msg**라는 변수를 인스턴스화하는 것입니다. MSG는 Windows 메시지를 나타내는 구조체입니다. 그 정의는 다음과 같습니다:

```
typedef struct tagMSG { HWND hwnd;
 UINT message; WPARAM
 wParam; LPARAM lParam;
 DWORD time; POINT pt;
} MSG;
```

1. **hwnd**: 메시지를 수신할 창 프로시저가 있는 창의 핸들.
2. **message**: 메시지를 식별하는 사전 정의된 상수 값(예: **WM\_QUIT**).
3. **wParam**: 메시지에 대한 추가 정보. 이는 특정 메시지에 따라 달라집니다.
4. **lParam**: 메시지에 대한 추가 정보. 이는 특정 메시지에 따라 다릅니다.
5. **time**: 메시지가 게시된 시간.
6. **pt**: 메시지가 게시되었을 때 마우스 커서의 (x, y) 좌표(화면 좌표계 기준).

다음으로 메시지 루프에 진입합니다. **GetMessage** 함수는 메시지 큐에서 메시지를 가져와 **msg** 인자에 메시지 세부 정보를 채웁니다. **GetMessage**의 두 번째, 세 번째, 네 번째 매개변수는 본 기능에서는 0으로 설정할 수 있습니다. **GetMessage**에서 오류가 발생하면 **â**는 "1"을 반환합니다. **WM\_QUIT** 메시지를 수신하면 **GetMessage**는 0을 반환하여 메시지 루프를 종료합니다. **GetMessage**가 다른 값을 반환하면 두 개의 함수가 추가로 호출됩니다: **TranslateMessage**

**TranslateMessage**는 Windows가 키보드 변환을 수행하도록 합니다. 구체적으로 가상 키를 문자 메시지로 변환합니다. **DispatchMessage**는 최종적으로 해당 메시지를 적절한 윈도우 프로시저로 전달합니다.

**WM\_QUIT** 메시지를 통해 애플리케이션이 성공적으로 종료되면, **WinMain** 함수는 **WM\_QUIT** 메시지의 **wParam**(종료 코드)를 반환해야 합니다.

**WM\_QUIT** 메시지의 **wParam**(종료 코드)를 반환해야 합니다.

### A.3.6 윈도우 프로시저

앞서 언급했듯이, 윈도우 프로시저는 창이 수신한 메시지에 대한 응답으로 실행하고자 하는 코드를 작성하는 곳입니다. 기본 Windows 애플리케이션 프로그램에서 윈도우 프로시저의 이름은 **WndProc**이며, 프로토타입은 다음과 같습니다:

```
LRESULT CALLBACK
WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam);

// 이 함수는 LRESULT 유형(정수로 정의됨)의 값을 반환하며, 이는 함수의 성공 또는 실패를 나타냅니다. CALLBACK 식별자는 이 함수가 콜백 함수임을 지정합니다. 즉, Windows가 프로그램의 코드 공
간 외부에서 이 함수를 호출할 것임을 의미합니다. 기본 Windows 애플리케이션 소스 코드에서 볼 수 있듯이, 우리는
윈도우 프로시저를 직접 호출하지 않습니다. 윈도우가 메시지 처리가 필요할 때 자동으로 호출합니다.
윈도우 프로시저의 시그니처에는 네 가지 매개변수가 있습니다:

1. hWnd: 메시지를 수신하는 창에 대한 핸들입니다.

2. msg: 특정 메시지를 식별하는 사전 정의된 값입니다. 예를 들어 종료 메시지는 WM_QUIT로 정의됩니다. 접두사 WM은 "Window Message"를 의미합니다. 사전 정의된 창 메시지는 100개가 넘으며, 자세
한 내용은 MSDN 라이브러리를 참조하십시오.

3. wParam: 특정 메시지에 따라 달라지는 메시지에 대한 추가 정보입니다.

4. lParam: 특정 메시지에 따라 달라지는 메시지에 대한 추가 정보입니다.
```

우리의 윈도우 프로시저는 세 가지 메시지를 처리합니다: **WM\_LBUTTONDOWN**, **WM\_KEYDOWN**, **WM\_DESTROY** 메시지입니다. **WM\_LBUTTONDOWN** 메시지는 사용자가 윈도우의 클라이
언트 영역에서 마우스 왼쪽 버튼을 클릭할 때 전송됩니다. **WM\_KEYDOWN** 메시지는 키가 눌렸을 때 포커스가 있는 윈도우로 전송됩니다. **WM\_DESTROY** 메시지는 윈도우가 파괴될 때 전송됩니다.

우리 코드는 매우 간단합니다. **WM\_LBUTTONDOWN** 메시지를 수신하면 "Hello, World"를 출력하는 메시지 상자를 표시합니다:

```
case WM_LBUTTONDOWN:
 MessageBox(0, L"Hello, World", L"Hello", MB_OK); return 0;

// ...

우리 창이 WM_KEYDOWN 메시지를 수신하면, Esc 키가 눌렸는지 검사합니다. 눌렀다면 DestroyWindow 함수를 사용하여 메인 애플리케이션 창을 파괴합니다. 창 프로시저로 전달된 wParam은 눌린
특정 키의 가상 키 코드를 지정합니다. 가상 키 코드는 특정 키를 식별하는 고유 식별자라고 생각하면 됩니다. Windows 헤더 파일에는 특정 키를 테스트하는 데 사용할 수 있는 가상 키 코드 상수 목록이 있습니
다. 예를 들어, Esc 키가 눌렸는지 테스트하려면 가상 키 코드 상수 VK_ESCAPE를 사용합니다.
```

```
case WM_KEYDOWN: if(wParam == VK_ESCAPE)
 DestroyWindow(ghMainWnd); return 0;

// ...

wParam 및 lParam 매개변수는 특정 메시지에 대한 추가 정보를 지정하는 데 사용됩니다. WM_KEYDOWN 메시지의 경우 wParam은 눌린 특정 키의 가상 키 코드를 지정합니다. MSDN 라이브러리에
서는 각 Windows 메시지에 대해 wParam 및 lParam 매개변수가 전달하는 정보를 명시합니다.

창이 파괴될 때 PostQuitMessage 함수를 사용하여 WM_QUIT 메시지를 게시합니다(이는
메시지 루프를 종료함):
```

```
case WM_DESTROY: PostQuitMessage(0);
 return 0;

// ...

창 프로시저의 마지막 부분에서 우리는 DefWindowProc라는 다른 함수를 호출합니다. 이 함수는 기본 창 프로시저입니다. 우리의 기본 Windows 애플리케이션 프로그램에서는 세 가지 메시지만 처리합
니다. 우리가 수신하지만 반드시 직접 처리할 필요가 없는 다른 모든 메시지에 대해서는 DefWindowProc에 지정된 기본 동작을 사용합니다. 예를 들어, 기본 Windows 애플리케이션 프로그램은 최소화, 최대화,
크기 조정 및 닫기가 가능합니다. 이러한 기능은 기본 창 프로시저를 통해 제공됩니다. 해당 기능을 수행하는 메시지를 직접 처리하지 않았기 때문입니다.
```

### A.3.7 MessageBox 함수

아직 다루지 않은 마지막 API 함수는 MessageBox 함수입니다. 이 함수는 사용자에게 정보를 제공하고 빠른 입력을 받는 데 매우 유용한 방법입니다. MessageBox 함수의 선언은 다음과 같습니다:

```
int MessageBox(
 HWND hWnd, // 소유자 창 핸들, null로 지정 가능. LPCTSTR lpText, // 메시지 상자에 표시할
 텍스트,
 LPCTSTR lpCaption, // 메시지 상자의 제목으로 표시할 텍스트. UINT uType // 메시지 상자의 스타일.
);
```

MessageBox 함수의 반환 값은 메시지 상자의 유형에 따라 다릅니다. 가능한 반환 값과 스타일 목록은 MSDN 라이브러리를 참조하십시오. 가능한 스타일 중 하나는 예/아니오 메시지 상자입니다. [그림 A.5](#)를 참조하십시오.

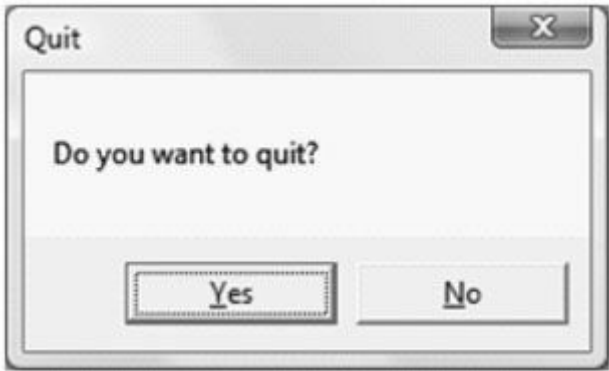


그림 A.5. 예/아니오 메시지 상자.

## A.4 더 나은 메시지 루프

게임은 오피스 애플리케이션이나 웹 브라우저 같은 기존 윈도우 애플리케이션과는 매우 다릅니다. 일반적으로 게임은 메시지를 기다리며 대기하지 않고 지속적으로 업데이트됩니다. 이는 메시지 큐에 메시지가 없을 경우 GetMessage 함수가 스레드를 잠고 메시지를 기다리게 하기 때문에 문제가 됩니다. 게임에서는 이러한 동작을 원하지 않습니다. 처리할 윈도우 메시지가 없다면 자체 게임 코드를 실행해야 합니다. 해결책은 GetMessage 대신 PeekMessage 함수를 사용하는 것입니다. PeekMessage 함수는 메시지가 없으면 즉시 반환됩니다. 새로운 메시지 루프는 다음과 같습니다:

```
int Run()
{
 MSG msg = {0};

 while(msg.message != WM_QUIT)
 {
 // 윈도우 메시지가 있으면 처리합니다. if(Peek Message(&msg, 0, 0, 0,
 PM_REMOVE))
 {
 TranslateMessage(&msg); DispatchMessage(&msg);
 }
 // 그렇지 않으면 애니메이션/게임 작업을 수행합니다. else
 {
 // ...
 }
 }
 return (int)msg.wParam;
}
```

msg를 인스턴스화한 후 무한 루프에 진입합니다. 먼저 메시지 큐에 메시지가 있는지 확인하는 API 함수 PeekMessage를 호출합니다. 매개변수 설명은 MSDN을 참조하십시오. 메시지가 있으면 true를 반환하고 메시지를 처리합니다.

메시지가 없으면 `Peek Message`는 `false`를 반환하고 우리는 자체 게임 코드를 실행합니다.

## A.5 요약

1. Direct3D를 사용하려면 3D 장면을 렌더링할 메인 윈도우를 가진 Windows 애플리케이션을 생성해야 합니다. 또한 게임의 경우 메시지를 확인하는 특수한 메시지 루프를 만듭니다. 메시지가 있으면 이를 처리하고, 없으면 게임 로직을 실행합니다.
2. 여러 Windows 애플리케이션이 동시에 실행될 수 있으므로, Windows는 이들 간 리소스를 관리하고 메시지를 의도된 애플리케이션으로 전달해야 합니다. 애플리케이션에 대한 이벤트(키 입력, 마우스 클릭, 타이머 등)가 발생하면 해당 애플리케이션의 메시지 큐로 메시지가 전송됩니다.
3. 모든 Windows 애플리케이션은 수신 메시지를 저장하는 메시지 큐를 보유합니다. 애플리케이션의 메시지 루프는 지속적으로 큐를 확인하여 메시지를 수신 대상 윈도우 프로시저로 전달합니다. 단일 애플리케이션 내에는 여러 개의 윈도우가 존재할 수 있음을 유의하십시오.
4. 윈도우 프로시저는 응용 프로그램의 창이 메시지를 수신할 때 Windows가 호출하는 특수 콜백 함수로, 우리가 구현합니다. 윈도우 프로시저에서는 응용 프로그램의 창이 특정 메시지를 수신할 때 실행되도록 원하는 코드를 작성합니다. 우리가 특별히 처리하지 않는 메시지는 기본 처리를 위해 기본 윈도우 프로시저로 전달됩니다.

## A.6 연습문제

1. §A.2의 프로그램을 수정하여 다른 아이콘, 커서, 배경색을 사용하도록 하십시오.



MSDN에서 `LoadIcon`, `LoadCursor`, `GetStockObject` 함수를 찾아보십시오.

2. §A.2의 프로그램을 `WM_CLOSE` 메시지를 처리하도록 수정하십시오. 이 메시지는 창이나 응용 프로그램에 창이나 응용 프로그램을 닫아야 함을 알리기 위해 전송됩니다. 메시지 상자 함수를 사용하여 사용자에게 정말 종료할 것인지 묻는 예/아니오 형식의 메시지 상자를 표시하세요. 사용자가 "예"를 선택하면 창을 파괴하고, 그렇지 않으면 창을 파괴하지 마세요. 이 기법을 사용하여 종료 전에 작업을 저장할 것인지 사용자에게 묻는 것도 가능합니다.
3. §A.2의 프로그램을 `WM_CREATE` 메시지를 처리하도록 수정하십시오. 이 메시지는 창이 생성될 때, `CreateWindow`가 반환되기 전에 창에 전송됩니다. 메시지 상자 함수를 통해 창이 생성되었음을 알리는 메시지를 출력하십시오.
4. MSDN에서 `Sleep` 함수를 찾아보고, 이 함수가 수행하는 작업을 자신의 말로 요약하세요.
5. MSDN에서 `WM_SIZE` 및 `WM_ACTIVATE` 메시지를 찾아보고, 메시지가 전송되는 시점을 자신의 말로 요약하세요.

## 참고 사항

<sup>1</sup> 애플리케이션은 유휴 상태에서 처리 작업을 수행할 수 있습니다. 즉, 이벤트가 발생하지 않을 때 특정 작업을 수행할 수 있습니다.

<sup>2</sup> 모든 창에는 창 프로시저가 있지만, 여러 창이 동일한 창 프로시저를 공유할 수 있습니다. 따라서 각 창마다 고유한 창 프로시저를 반드시 작성할 필요는 없습니다. 서로 다른 두 창이 메시지에 다르게 응답하도록 하려면 서로 다른 창 프로시저를 가져야 합니다.

# 고수준 셰이더 언어

## 참조

### 변수 유형

#### 스칼라 유형

1. **bool**: 참 또는 거짓 값. HLSL은 C++과 마찬가지로 **true** 및 **false** 키워드를 제공한다는 점에 유의하십시오.
2. **int**: 32비트 부호 있는 정수.
3. **half**: 16비트 부동 소수점 숫자.
4. **float**: 32비트 부동 소수점 숫자.
5. **double**: 64비트 부동 소수점 수.

 일부 플랫폼에서는 **int**, **half**, **double**을 지원하지 않을 수 있습니다. 이 경우 해당 유형은 **float**을 사용하여 에뮬레이트됩니다.

#### 벡터 유형

1. **float2**: 2차원 벡터로, 각 성분은 **float** 형식입니다.
2. **float3**: 3차원 벡터로, 각 성분은 **float** 형식입니다.
3. **float4**: 4차원 벡터로, 각 성분은 **float** 형식입니다.

 부동 소수점(float) 유형이 아닌 다른 유형의 성분을 가진 벡터를 생성할 수 있습니다. 예를 들어: **int2**, **half3**, **bool4**.

벡터는 배열과 유사한 구문이나 생성자와 유사한 구문을 사용하여 초기화할 수 있습니다:

```
float3 v = {1.0f, 2.0f, 3.0f}; float2 w =
float2(x, y);
float4 u = float4(w, 3.0f, 4.0f); // u = (w.x, w.y, 3.0f, 4.0f)
```

벡터의 성분은 배열 인덱싱 구문을 사용하여 접근할 수 있습니다. 예를 들어, **벡터 vec**의 **i**번째 성분을 설정하려면

다음과 같이 작성합니다:

```
vec[i] = 2.0f;
```

또한 구조체의 멤버에 접근하듯이 정의된 성분 이름 **x**, **y**, **z**, **w**, **r**, **g**, **b**, **a**를 사용하여 **벡터 vec**의 성분에 접근할 수 있습니다.

```
vec.x = vec.r = 1.0f; vec.y =
vec.g = 2.0f; vec.z = vec.b =
3.0f; vec.w = vec.a = 4.0f;
```

**r**, **g**, **b**, **a**라는 이름은 각각 **x**, **y**, **z**, **w**라는 이름과 정확히 동일한 성분을 가리킵니다. 벡터를 사용하여 색상을 표현할 때는 RGBA 표기법이 더 바람직합니다. 이는 벡터가 색상을 표현하고 있다는 사실을 강조하기 때문입니다.

#### 스위즐

벡터  $\mathbf{u} = \begin{pmatrix} u_x, u_y, u_z, u_w \end{pmatrix}$  벡터 **u**를 고려하고, **u**의 성분들을 벡터 **v**로 복사하여 다음 조건을 만족시키려 한다고 가정하자:

$\mathbf{v} = \begin{pmatrix} u_w, u_y, u_y, u_x \end{pmatrix}$ 가장 즉각적인 해결책은  $u$ 의 각 성분을 필요에 따라 개별적으로  $v$ 로 복사하는 것입니다.

그러나 HLSL은 이러한 순서 무관 복사 작업을 위한 특별한 구문인 *스위즐*(*swizzles*)을 제공합니다:

```
float4 u = {1.0f, 2.0f, 3.0f, 4.0f};
float4 v = {0.0f, 0.0f, 5.0f, 6.0f};

v = u.wyyx; // v = {4.0f, 2.0f, 2.0f, 1.0f}
```

또 다른 예시:

```
float4 u = {1.0f, 2.0f, 3.0f, 4.0f};
float4 v = {0.0f, 0.0f, 5.0f, 6.0f};

v = u.wzyx; // v = {4.0f, 3.0f, 2.0f, 1.0f}
```

벡터를 복사할 때 모든 성분을 복사할 필요는 없습니다. 예를 들어, 이 코드 조각이 보여주는 것처럼  $x$  및  $y$  성분만 복사할 수 있습니다:

```
float4 u = {1.0f, 2.0f, 3.0f, 4.0f};
float4 v = {0.0f, 0.0f, 5.0f, 6.0f};

v.xy = u; // v = {1.0f, 2.0f, 5.0f, 6.0f}
```

행렬 유형

다음 구문을 사용하여  $m$ 과  $n$  0/1과 4 사이인  $m \times n$  행렬을 정의할 수 있습니다:

```
float m×n matmxn;
```

예시:

- 1. float2x2:  $2 \times 2$  행렬로, 각 요소의 데이터 유형은 float입니다.
- 2. float3x3:  $3 \times 3$  행렬, 각 요소의 데이터 유형은 float입니다.
- 3. float4x4:  $4 \times 4$  행렬로, 각 요소의 데이터 유형은 float입니다.
- 4. float3x4:  $3 \times 4$  행렬로, 각 요소의 데이터 유형은 float입니다.

 부동 소수점 이외의 유형으로 구성된 행렬을 생성할 수 있습니다. 예: int2x2, half3x3, bool4x4.

이중 배열 인덱싱 구문을 사용하여 행렬의 요소에 접근할 수 있습니다. 예를 들어, **행렬 M**의  $i$ 번째 열과  $j$ 번째 행에 해당하는 요소를 설정하려면 다음과 같이 작성합니다:

```
M[i][j] = 값;
```

또한 구조체의 멤버에 접근하는 방식과 동일하게 **행렬 M**의 요소를 참조할 수 있습니다. 다음 요소 이름들이 정의됩니다:

*1을 기준으로 한 인덱싱:*

```
M._11 = M._12 = M._13 = M._14 = 0.0f; M._21 = M._22 =
M._23 = M._24 = 0.0f; M._31 = M._32 = M._33 = M._34 =
0.0f; M._41 = M._42 = M._43 = M._44 = 0.0f;
```

*0 기반 인덱싱:*

```
M._m00 = M._m01 = M._m02 = M._m03 = 0.0f; M._m10 = M._m11 =
M._m12 = M._m13 = 0.0f; M._m20 = M._m21 = M._m22 = M._m23 =
0.0f; M._m30 = M._m31 = M._m32 = M._m33 = 0.0f;
```

때로는 행렬에서 특정 행 벡터를 참조하고 싶을 때가 있습니다. 단일 배열 인덱싱 구문을 사용하여 이를 수행할 수 있습니다. 예를 들어,  $3 \times 3$  **행렬 M**에서  $i$ 번째 행 벡터를 추출하려면 다음과 같이 작성합니다:

```
float3 ithRow = M[i]; // M의i번째 행 벡터 가져오기
```

다음 예제에서는 행렬의 첫 번째, 두 번째, 세 번째 행에 세 개의 벡터를 삽입합니다:

```
float3 N = normalize(pIn.normalW);
float3 T = normalize(pIn.tangentW - dot(pIn.tangentW, N)*N); float3 B = cross(N,T);
float3x3 TBN;
TBN[0] = T; // 행 1 설정 TBN[1] = B;
// 행 2 설정 TBN[2] = N; // 행 3 설정
```

벡터로부터 행렬을 구성할 수도 있습니다:

```
float3 N = normalize(pIn.normalW);
float3 T = normalize(pIn.tangentW - dot(pIn.tangentW, N)*N); float3 B = cross(N,T);

float3x3 TBN = float3x3(T, B, N);
```



Note

4차원 벡터와 4x4 행렬을 표현하기 위해 float4와 float4x4를 사용하는 대신, 벡터 및 행렬 유형을 동등하게 사용할 수 있습니다.

```
vector u = {1.0f, 2.0f, 3.0f, 4.0f}; matrix M; // 4x4 행렬
```

배열

익숙한 C++ 구문을 사용하여 특정 유형의 배열을 선언할 수 있습니다. 예를 들어:

```
float M[4][4]; half
p[4];
float3 v[12]; // 12개의 3차원 벡터
```

구조체

구조체는 C++에서와 정확히 동일하게 정의됩니다. 그러나 HLSL의 구조체는 멤버 함수를 가질 수 없습니다. 다음은 HLSL의 구조체 예시입니다:

```
struct SurfaceInfo
{
 float3 pos; float3
 normal; float4 diffuse;
 float4 spec;
};
Surface Info v; litColor +=
v.diffuse;

dot(lightVec, v.normal);
float specPower = max(v.spec.a, 1.0f);
```

typedef 키워드

HLSL의 typedef 키워드는 C++에서와 정확히 동일하게 작동합니다. 예를 들어, point라는 이름을 유형에 부여할 수 있습니다.

다음 구문을 사용하여 vector<float, 3>을 정의합니다:

```
typedef float3 point; 그런 다음 다음
```

과 같이 작성하는 대신: float3

```
myPoint;
```

다음과 같이 작성할 수 있습니다:

```
point myPoint;
```

다음은 HLSL의 **const** 키워드(C++과 동일하게 작동)와 함께 **typedef** 키워드를 사용하는 또 다른 예시입니다:

```
typedef const float CFLOAT;
```

## 변수 접두사

다음 키워드는 변수 선언 앞에 접두사로 사용할 수 있습니다.

- 1. **static**: 기본적으로 **extern**의 반대 개념입니다. 이는 셰이더 변수가 C++ 애플리케이션에 노출되지 않음을 의미합니다.

```
static float3 v = {1.0f, 2.0f, 3.0f};
```

- 2. **uniform**: 이는 변수가 버텍스/픽셀마다 변경되지 않음을 의미합니다. C++ 애플리케이션 수준에서 변경할 때까지 모든 버텍스/픽셀에 대해 상수입니다. uniform 변수는 셰이더 프로그램 외부(예: C++ 애플리케이션)에서 초기화됩니다.

- 3. **extern**: 이는 C++ 애플리케이션이 해당 변수를 볼 수 있음을 의미합니다(즉, C++ 애플리케이션 코드가 셰이더 파일 외부에서 변수에 접근할 수 있음). 셰이더 프로그램의 전역 변수는 기본적으로 uniform 및 extern 속성을 가집니다.

- 4. **const**: HLSL의 **const** 키워드는 C++에서의 의미와 동일합니다. 즉, 변수 앞에 **const** 키워드가 붙으면 해당 변수는 상수이며 변경할 수 없습니다.

```
const float pi = 3.14f;
```

## 형 변환

HLSL은 매우 유연한 형변환 방식을 지원합니다. HLSL의 형변환 구문은 C 프로그래밍 언어와 동일합니다. 예를 들어, **부동소수점** 숫자를 **행렬**로 형변환하려면 다음과 같이 작성합니다:

```
float f = 5.0f;
float4x4 m = (float4x4)f; // f를 m의 각 요소에 복사합니다.
```

이 스칼라-행렬 형변환은 스칼라를 행렬의 각 요소에 복사하는 역할을 합니다. 다음 예시를 살펴보겠습니다:

```
float3 n = float3(...); float3 v =
2.0f*n - 1.0f;
```

2.0f\*n은 스칼라-벡터 곱셈으로 명확히 정의됩니다. 그러나 이를 벡터 방정식으로 만들기 위해 스칼라 1.0f는 벡터 (1.0f, 1.0f, 1.0f)로 확장됩니다. 따라서 이전 문장은 다음과 같습니다:

```
float3 v = 2.0f*n - float3(1.0f, 1.0f, 1.0f);
```

이 책의 예제에서는 구문으로부터 형변환의 의미를 추론할 수 있을 것입니다. 형변환 규칙의 전체 목록은 SDK 문서 색인에서 "Casting and Conversion"을 검색하십시오.

## 키워드와 연산자

### 키워드

참고로, HLSL이 정의하는 키워드 목록은 다음과 같습니다:

```
asm, bool, compile, const, decl, do, double, else,
extern, false, float, for, half, if, in, inline, inout, int,
matrix, out, pass, pixelshader, return, sampler, shared, static, string, struct,
technique, texture, true, typedef, uniform, vector, vertexshader, void,
volatile, while
```

다음 키워드 집합은 예약되어 사용되지 않지만 향후 키워드로 지정될 수 있는 식별자를 표시합니다:

```
auto, break, case, catch, char, class,
const_cast, continue, default, delete, dynamic_cast, enum, explicit, friend, goto, long,
mutable, namespace,
new, 연산자, private, protected, public, register, 재해석 캐스트, short, signed, sizeof,
static_cast, switch, 템플릿, this, throw, try, typename, union,
```

연산자

HLSL은 익숙한 C++ 연산자를 다수 지원합니다. 아래에 명시된 몇 가지 예외를 제외하면 C++에서와 동일한 방식으로 사용됩니다. 다음 표는 HLSL 연산자를 나열합니다:

|     |    |   |    |    |     |
|-----|----|---|----|----|-----|
| [ ] | .  | > | <  | <= | >=  |
| !=  | =  | ! | && |    | ? : |
| +   | += | - | -= | *  | *=  |
| /   | /= | % | %= | ++ | --  |
| =   | () | , |    |    |     |

연산자의 동작은 C++과 매우 유사하지만 몇 가지 차이점이 있습니다. 첫째, 나머지 연산자(%)는 정수형과 부동 소수점형 모두에서 작동합니다. 또한 나머지 연산자를 사용하려면 좌변 값과 우변 값이 동일한 부호를 가져야 합니다(예: 양수이거나 음수여야 함).

둘째, HLSL의 많은 연산이 구성 요소 단위로 작동한다는 점을 유의하십시오. 이는 벡터와 행렬이 언어에 내장되어 있으며 이러한 유형이 여러 구성 요소로 이루어져 있기 때문입니다. 연산을 구성 요소 수준에서 수행함으로써 벡터/행렬 덧셈, 벡터/행렬 뺄셈, 벡터/행렬 곱셈, 벡터/행렬 동일성 검사 등의 연산을 스칼라 유형에 사용하는 것과 동일한 연산자로 수행할 수 있습니다. 다음 예시를 참조하십시오.

 연산자는 스칼라에 대해 예상대로 동작합니다. 즉, 일반적인 C++ 방식과 동일합니다.

```
float4 u = {1.0f, 0.0f, -3.0f, 1.0f};
float4 v = {-4.0f, 2.0f, 1.0f, 0.0f};
```

```
// 대응하는 성분을 더함
float4 sum = u + v; // sum = (-3.0f, 2.0f, -2.0f, 1.0f)
```

벡터의 각 성분을 증가시키는 방법:

```
// 증가 전: sum = (-3.0f, 2.0f, -2.0f, 1.0f) sum++; // 증가 후: sum = (-2.0f, 3.0f, -1.0f, 2.0f)
```

벡터의 성분별 곱셈:

```
float4 u = {1.0f, 0.0f, -3.0f, 1.0f};
float4 v = {-4.0f, 2.0f, 1.0f, 0.0f};
```

```
// 대응하는 성분들 곱하기
float4 product = u * v; // product = (-4.0f, 0.0f, -3.0f, 0.0f)
```

 두 개의 행렬이 있는 경우:

```
float4x4 A; float4x4 B;
```

A\*B 구문은 행렬 곱셈이 아닌 성분별 곱셈을 수행합니다. 행렬 곱셈을 위해서는 **mul** 함수를 사용해야 합니다.

비교 연산자 역시 각 성분별로 수행되며, 각 성분이 **bool** 형식인 벡터 또는 행렬을 반환합니다. 결과적인 "bool" 벡터에는 비교된 각 성분의 결과가 포함됩니다. 예를 들어:

```
float4 u = { 1.0f, 0.0f, -3.0f, 1.0f};
float4 v = {-4.0f, 0.0f, 1.0f, 1.0f};

float4 b = (u == v); // b = (false, true, false, true)
```

마지막으로, 이진 연산에서의 변수 승급에 대해 논의하며 결론을 내립니다:

- 1.** 이항 연산에서 좌변과 우변의 차원이 다를 경우, 차원이 작은 쪽이 차원이 큰 쪽과 동일한 차원이 되도록 승격(형변환)됩니다. 예를 들어 `x`가 `float` 형이고 `y`가 `float3` 형인 경우, `(x + y)` 표현식에서 변수 `x`는 `float3`로 승격되며 표현식은 `float3` 형의 값으로 평가됩니다. 형변환은 정의된 캐스트를 사용하여 수행됩니다. 이 경우 스칼라에서 벡터로의 캐스트이므로, `x`가 `float3`로 형변환된 후 `x = (x, x, x)`가 됩니다. 이는 스칼라-벡터 캐스트가 정의하는 바입니다. 캐스트가 정의되지 않은 경우 형변환도 정의되지 않음을 유의하십시오. 예를 들어, `float2`를 `float3`로 형변환할 수 없습니다. 왜냐하면 그러한 정의된 캐스트가 존재하지 않기 때문입니다.
- 2.** 이진 연산의 경우, 좌변과 우변의 유형이 다를 때 유형 해상도가 낮은 쪽이 유형 해상도가 높은 쪽과 동일한 유형이 되도록 승격(캐스트)됩니다. 예를 들어, `x`가 `int` 유형이고 `y`가 `half` 유형인 경우, `(x + y)` 표현식에서 변수 `x`는 `half`로 승격되며 표현식은 `half` 유형의 값으로 평가됩니다.

## 프로그램 흐름

HLSL은 선택, 반복 및 일반적인 프로그램 흐름을 위한 많은 익숙한 C++ 문을 지원합니다. 이러한 문들의 구문은 C++과 정확히 동일합니다.

```
반환문:

return (표현식);

If 및 If...Else 문:

if(조건)
{
 문장(들);
}

if(조건)
{
 문장(들);
}
else
{
 문장(들);
}

for 문:

for(초기값; 조건; 증분)
{
 명령문(들);
}

while 문:

while(조건)
{
 문장(들);
}

do...while 문:

do
{
 명령문;
}while(조건);
```

## 함수

### 사용자 정의 함수

HLSL의 함수는 다음과 같은 특성을 가집니다:

- 1. 함수는 익숙한 C++ 구문을 사용합니다.
- 2. 매개변수는 항상 값으로 전달됩니다.
- 3. 재귀는 지원되지 않습니다.
- 4. 함수는 항상 인라인 처리됩니다.

또한 HLSL은 함수와 함께 사용할 수 있는 몇 가지 추가 키워드를 제공합니다. 예를 들어, HLSL로 작성된 다음 함수를 살펴보십시오:

```
bool foo(in const bool b, // 입력 부울
 out int r1, // 출력 정수
 inout float r2) // 입력/출력 부동 소수점
{
 if(b) // 입력값 테스트
 {
 r1 = 5; // r1을 통해 값 출력
 }
 else
 {
 r1 = 1; // r1을 통해 값 출력
 }

 // r2는 inout이므로 입력으로 사용할 수 있음
 // 값을 계산하고 이를 통해 값을 출력합니다 r2 = r2 * r2 * r2;

 true를 반환합니다;
}
```

이 함수는 **in**, **out**, **inout** 키워드를 제외하면 C++ 함수와 거의 동일합니다.

**in**: *인자* 매개변수로 전달하는 특정 변수)가 함수 시작 전에 매개변수로 복사되어야 함을 지정합니다.

- 1. 함수가 시작되기 전에 매개변수로 **복사되어야 함**을 지정합니다. 매개변수는 기본적으로 **in**이므로 매개변수를 명시적으로 **in**으로 지정할 필요는 없습니다. 예를 들어, 다음은 동등합니다:

```
float square(in float x)
{
 return x * x;
}
```

명시적으로 **in**을 지정하지 않은 경우: `float square(float`

```
x)
{
 return x * x;
}
```

**out**: 함수가 반환될 때 매개변수를 인수로 복사해야 함을 지정합니다. 매개변수를 통해 값을 반환할 때 유용합니다.

- 2. 매개변수를 통해 값을 반환하는 데 유용합니다. HLSL은 참조 전달이나 포인터 전달을 허용하지 않기 때문에 **out** 키워드가 필요합니다. 매개변수가 **out**으로 표시된 경우 함수가 시작되기 전에 인수가 매개변수로 복사되지 않는다는 점에 유의하십시오. 즉,

말하자면, **out** 매개변수는 데이터 출력에만 사용할 수 있으며 입력에는 사용할 수 없습니다.

```
void square(in float x, out float y)
{
 y = x * x;
}
```

여기서는 **x**를 통해 제공할 숫자를 입력받고, 매개변수 **y**를 통해 **x**의 제곱값을 반환합니다.

**inout**: 매개변수를 입력과 **출력** 모두로 지정하는 단축형입니다. 매개변수를 입력과 출력 모두로 사용하려면 **inout**을 지정하십시오.

3. 사용하려면 inout을 지정하십시오.

```
void square(inout float x)
{
 x = x * x;
}
```

여기서 우리는 x를 통해 제공할 숫자를 입력하고 또한 x를 통해 x의 제공을 반환합니다.

내장 함수

HLSL은 3D 그래픽에 유용한 다양한 내장 함수를 제공합니다. 다음은 그 중 일부를 요약한 목록입니다.

- 1. abs(x) – |x|를 반환합니다.
- 2. ceil(x) – x보다 크거나 같으면서 가장 작은 정수를 반환합니다.
- 3. cos(x) – x의 코사인을 반환합니다. 여기서 x는 라디안 단위입니다.
- 4. clamp(x, a, b) – x를 [a, b] 범위로 제한하고 결과를 반환합니다.
- 5. clip(x) – 이 함수는 픽셀 셰이더에서만 호출할 수 있으며, x < 0인 경우 현재 픽셀을 추가 처리에서 제외합니다.
- 6. cross(u, v) – u × v를 반환합니다.
- 7. ddx(p) – 화면 공간 부분 미분 ∂p/∂x를 추정합니다. 이를 통해 화면 공간 x 방향에서 픽셀 간 픽셀별 수량 p의 변화를 확인할 수 있습니다.
- 8. ddy(p) – 화면 공간 부분 미분 ∂p/∂y를 추정합니다. 이를 통해 화면 공간 y 방향에서 픽셀별 양 p가 픽셀마다 어떻게 변화하는지 확인할 수 있습니다.
- 9. degrees(x) – x를 라디안에서 각도로 변환합니다.
- 10. determinant(M) – 행렬의 행렬식을 반환합니다.
- 11. 거리(u, v) – 점 u와 v 사이의 거리 || v - u ||를 반환합니다.
- 12. dot(u, v) – u · v를 반환합니다. |
- 13. floor(x) – x보다 작거나 같은 가장 큰 정수를 반환합니다.
- 14. frac(x) – 이 함수는 부동 소수점 숫자의 소수 부분(즉, 맨티사)을 반환합니다. 예를 들어, x = (235.52, 696.32)이고 frac(x) = (0.52, 0.32)입니다.
- 15. length(v) – || v ||를 반환합니다.
- 16. lerp(u, v, t) – 매개변수 t ∈ [0,1]을 기반으로 u와 v 사이를 선형 보간합니다.
- 17. log(x) – ln(x)를 반환합니다.
- 18. log10(x) – log10 (x)를 반환합니다.
- 19. log2(x) – log2 (x)를 반환합니다.
- 20. max(x, y) – x ≥ y이면 x를 반환하고, 그렇지 않으면 y를 반환합니다.
- 21. min(x, y) – x ≥ y이면 x를 반환하고, 그렇지 않으면 y를 반환합니다.
- 22. mul(M, N) – 행렬 곱 MN을 반환합니다. 행렬 곱 MN이 정의되어야 함에 유의하십시오. M이 벡터인 경우, 벡터-행렬 곱이 정의되도록 행 벡터로 처리됩니다. 마찬가지로 N이 벡터인 경우, 행렬-벡터 곱이 정의되도록 열 벡터로 처리됩니다.
- 23. normalize(v) – v/|| v ||를 반환합니다.
- 24. pow(b, n) – b^n을 반환합니다.
- 25. radians(x) – x를 도(degree)에서 라디안(radian)으로 변환합니다.
- 26. saturate(x) – clamp(x, 0.0, 1.0)을 반환합니다.
- 27. sin(x) – x의 사인(sin)을 반환합니다. 여기서 x는 라디안 단위입니다.
- 28. sincos(in x, out s, out c) – x의 사인과 코사인을 반환합니다. x는 라디안 단위입니다.
- 29. sqrt(x) – √x를 반환합니다.
- 30. reflect(v, n) – 입사 벡터 v와 표면 법선 n을 주어 반사 벡터를 계산합니다.
- 31. refract(v, n, eta) – 입사 벡터 v, 표면 법선 n, 두 물질의 두 굴절률 비 eta를 주어 굴절 벡터를 계산합니다.
- 32. rsqrt(x) – x의 제곱근(1/√x)을 반환합니다.
- 33. tan(x) – x의 탄젠트를 반환합니다. 여기서 x는 라디안 단위입니다.

34. transpose(M) – 전치 행렬 M<sup>(T)</sup>를 반환합니다.
35. Texture2D::Sample(S, texC) – SamplerState 객체 S와 2D 텍스처 좌표 texC를 기반으로 2D 텍스처 맵에서 색상을 반환합니다.
36. Texture2D::SampleLevel(S, texC, mipLevel) – SamplerState 객체 S, 2D 텍스처 좌표 texC 및 mip맵 레벨 mipLevel을 기반으로 2D 텍스처 맵에서 색상을 반환합니다. 이 함수는 세 번째 매개 변수로 사용할 mip맵 레벨을 수동으로 지정한다는 점에서 Texture2D::Sample과 다릅니다. 예를 들어, 최상위 mip맵 LOD에 접근하려면 0을 지정합니다.
37. TextureCube::Sample(S, v) – 샘플러 상태 객체 S와 3D 조희 벡터 v를 기반으로 큐브 맵에서 색상을 반환합니다.
38. Texture2DArray::Sample(S, texC) – 샘플러 상태 객체 S(샘플러 상태는 텍스처 필터와 텍스처 주소 모드를 지정함)와 3차원 텍스처 좌표 texC를 기반으로 2차원 텍스처 배열에서 색상을 반환합니다. 여기서 첫 번째와 두 번째 좌표는 일반적인 2차원 텍스처 좌표이며, 세 번째 좌표는 배열 인덱스를 지정합니다.

대부분의 함수는 해당 함수가 의미 있는 모든 내장형과 함께 작동하도록 중복 정의됩니다. 예를 들어, abs는 모든 스칼라형에 대해 의미가 있으므로 이들 모두에 대해 중복 정의됩니다. 또 다른 예로, cross

Notes

곱셈 연산은 3차원 벡터에만 의미가 있으므로, 어떤 유형의 3차원 벡터(예: 정수형, 부동소수점형, 이중정밀도 부동소수점형 등의 3차원 벡터)에 대해서만 중복 정의됩니다. 반면 선형 보간(lerp)은 스칼라, 2차원, 3차원, 4차원 벡터 모두에 의미가 있으므로 모든 유형에 대해 중복 정의됩니다.

Notes

"스칼라" 함수, 즉 전통적으로 스칼라를 대상으로 하는 함수(예: cos(x))에 비스칼라 유형을 전달하면 함수는 각 성분별로 동작합니다. 예를 들어 다음과 같이 작성하면:

```
float3 v = float3(0.0f, 0.0f, 0.0f); v = cos(v);
```

함수는 각 성분별로 동작합니다.

Notes

추가 참고 자료로, 내장 HLSL 함수의 전체 목록은 DirectX 문서에서 확인할 수 있습니다. 색인에서 "HLSL 내장 함수(HLSL Intrinsic Functions)"를 검색하십시오.

# 일부 해석 기하학

이 부록에서는 더 복잡한 기하학을 위한 구성 요소로 벡터와 점을 사용합니다. 이 주제들은 책에서 다루어지지만 벡터, 행렬, 변환만큼 자주 등장하지는 않습니다. 따라서 본문보다는 부록에 배치하였습니다.

## C.1 광선, 직선, 선분

직선은 그 위에 있는 점  $p_0$ 과 직선과 평행하게 향하는 벡터  $u$ 로 표현될 수 있습니다(그림 C.1 참조). 벡터 직선 방정식은 다음과 같습니다:

$$p(t) = p_0 + tu \quad \text{for } t \in \mathbb{R}$$

$t$ 에 다른 값을 대입함으로써( $t$ 는 어떤 실수라도 가능함) 직선 위의 다른 점들을 얻을 수 있습니다.

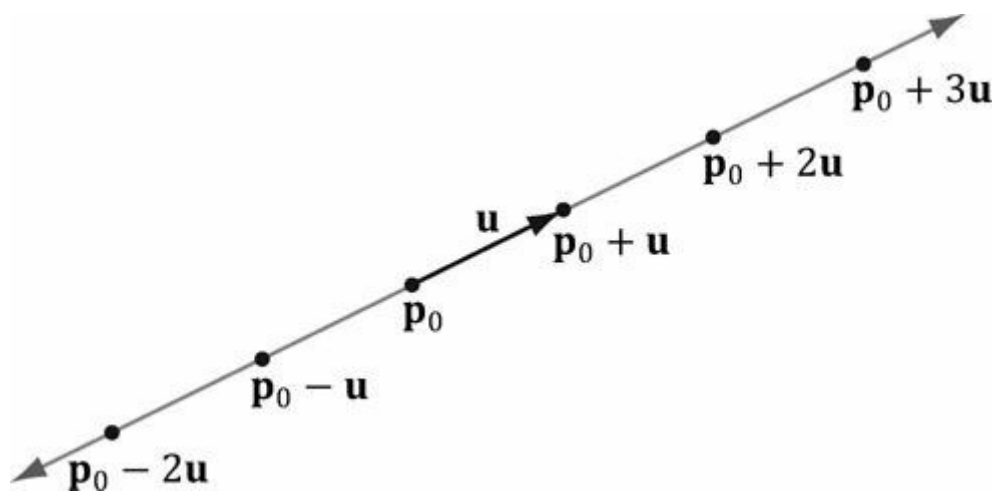


그림 C.1. 직선 위의 점  $p_0$ 과 직선과 평행한 방향을 가지는 벡터  $u$ 로 정의된 직선. 임의의 실수  $t$ 를 대입함으로써 직선 위의 점을 생성할 수 있다.

$t$ 를 비음수로 제한하면, 벡터 직선 방정식의 그래프는 원점  $p_0$ 과 방향  $u$ 를 가진 반직선이다(그림 C.2 참조).

이제 두 끝점  $p_0$ 과  $p_1$ 로 선분을 정의하고자 한다. 먼저 벡터

$p_1$ 까지 이어집니다; 그림 C.3을 참조하십시오. 그런 다음  $t \in [0, 1]$ 에 대해, 방정식의 그래프는

의된 선분이다. 범위를 벗어나면

$u = p_1 - p_0$ 에서  $p(t) = p_0 + tu = p_0 + t(p_1 - p_0)$ 는  $p_0$ 과  $p_1$ 로 정의된 선분이다. 범위를 벗어나면, 선분과 일치하지만 선분 위에 있지 않은 직선상의 점을 얻게 됩니다.

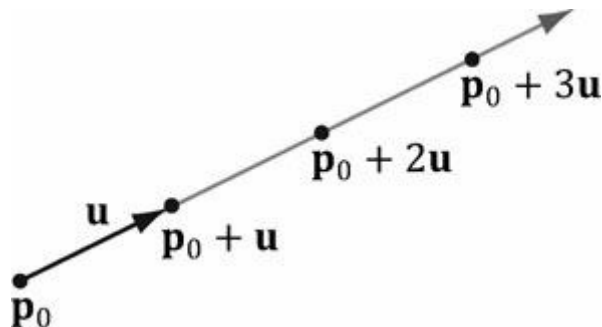


그림 C.2. 원점  $p_0$ 과 방향  $u$ 로 정의된 반직선. 0 이상인 스칼라 값을  $t$ 에 대입함으로써 반직선 위의 점을 생성할 수 있다.

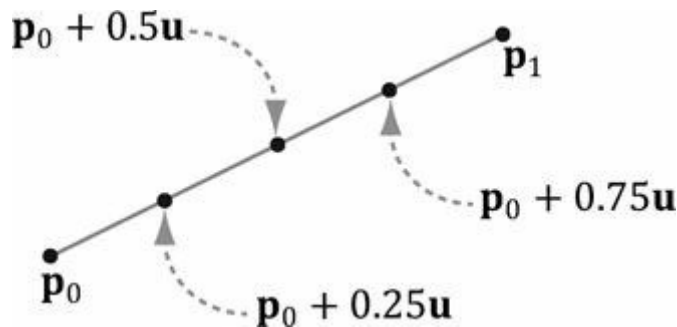


그림 C.3. 우리는  $[0, 1]$  구간에서  $t$ 에 대해 서로 다른 값을 대입하여 선분 위에 점들을 생성합니다. 예를 들어, 선분의 중점은  $t = 0.5$ 에서 주어집니다. 또한  $t = 0$ 일 때 끝점  $p_0$ 를 얻고,  $t = 1$ 일 때 끝점  $p_1$ 를 얻는다는 점에 유의하십시오.  $t = 0$ 일 때 끝점  $p_0$ 를 얻고,  $t = 1$ 일 때 끝점  $p_1$ 를 얻습니다.

## C.2 평행사변형

점  $q$ 와 두 벡터  $u, v$ 가 서로의 스칼라 배가 아닌 경우(즉, 모든 스칼라  $k$ 에 대해  $u \neq kv$ )를 고려하자. 그러면 다음 함수의 그래프는 평행사변형이다(그림 C.4 참조):

$$p(s, t) = q + su + tv \quad \text{for } s, t \in [0, 1]$$

" $u \neq kv$  (모든 스칼라  $k$ 에 대해)"라는 조건의 이유는 다음과 같이 설명할 수 있다. 만약  $u = kv$  라면 다음과 같이 쓸 수 있다:

$$\begin{aligned} p(s, t) &= q + su + tv \\ &= q + skv + tv \\ &= q + (sk + t)v \\ &= q + \bar{t}v \end{aligned}$$

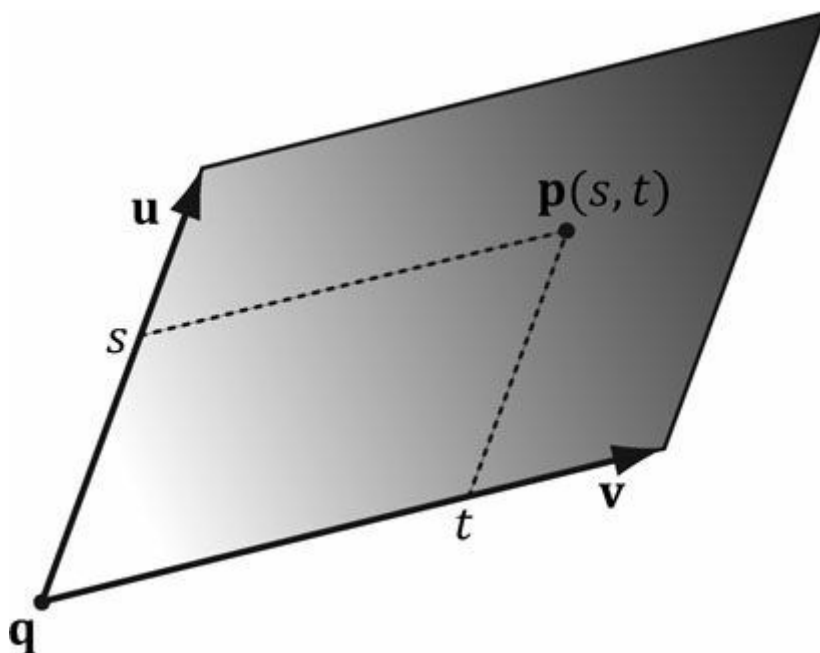


그림 C.4. 평행사변형.  $s, t \in [0, 1]$ 에 다양한 값을 대입함으로써 평행사변형 상의 서로 다른 점들을 생성한다.

이는 단순히 직선의 방정식이다. 즉, 우리는 단 하나의 자유도만을 가진다. 평행사변형과 같은 2차원 도형을 얻으려면 두 개의 자유도가 필요하므로, 벡터  $u$ 와  $v$ 는 서로의 스칼라 배가 되어서는 안 된다.

## C.3 삼각형

삼각형의 벡터 방정식은 평행사변형 방정식과 유사하지만, 매개변수의 정의역을 더 제한한다는 점이 다르다:

$$\mathbf{p}(s, t) = \mathbf{p}_0 + s\mathbf{u} + t\mathbf{v} \quad \text{for} \quad s \geq 0, t \geq 0, s + t \leq 1$$

그림 C.5에서 알 수 있듯이,  $s$ 와  $t$ 에 대한 조건 중 하나라도 성립하지 않으면  $\mathbf{p}(s, t)$ 는 삼각형 "바깥"에 있지만 삼각형 평면 위에 있는 점이 됩니다.

삼각형을 정의하는 세 점을 주어진다면, 그 삼각형의 이전 매개변수 방정식을 구할 수 있다.

세 꼭짓점  $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ 로 정의된 삼각형을 생각해 보자. 그러면  $s \leq 0, t \leq 0, s + t \geq 1$  일 때 삼각형 위의 한 점은 다음과 같이 주어질 수 있다:

$$\mathbf{p}(s, t) = \mathbf{p}_0 + s(\mathbf{p}_1 - \mathbf{p}_0) + t(\mathbf{p}_2 - \mathbf{p}_0)$$

이를 더 발전시켜 스칼라를 분배할 수 있습니다:

$$\begin{aligned} \mathbf{p}(s, t) &= \mathbf{p}_0 + s\mathbf{p}_1 - s\mathbf{p}_0 + t\mathbf{p}_2 - t\mathbf{p}_0 \\ &= (1 - s - t)\mathbf{p}_0 + s\mathbf{p}_1 + t\mathbf{p}_2 \\ &= r\mathbf{p}_0 + s\mathbf{p}_1 + t\mathbf{p}_2 \end{aligned}$$

여기서  $r = (1 - s - t)$ 로 두었다. 좌표  $(r, s, t)$ 를 중점 좌표(barycentric coordinates)라 부른다.  $r + s + t = 1$ 이며, 중점 조합  $\mathbf{p}(r, s, t) = r\mathbf{p}_0 + s\mathbf{p}_1 + t\mathbf{p}_2$ 는 점  $\mathbf{p}$ 를 삼각형의 꼭짓점들의 가중 평균으로 표현한다. 중심 좌표에는 흥미로운 성질들이 있지만, 이 책에서는 필요하지 않습니다. 독자는 중심 좌표에 대해 더 깊이 연구해 볼 수 있습니다.

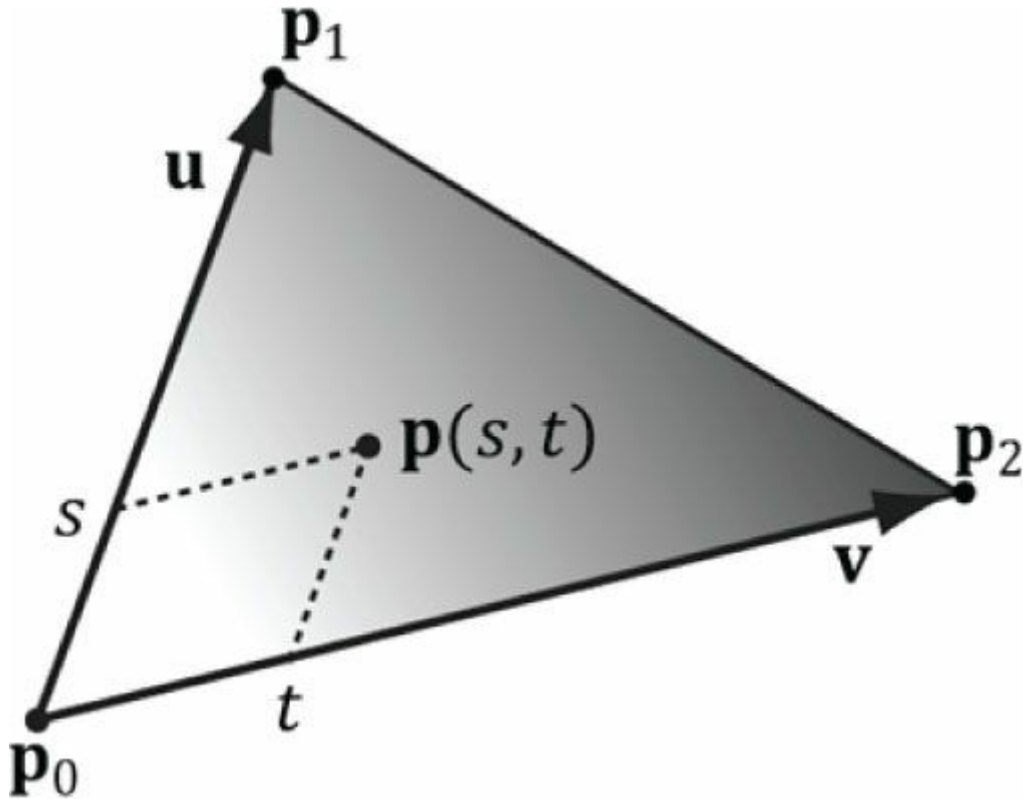


그림 C.5. 삼각형.  $s \leq 0, t \leq 0, s + t \geq 1$ 을 만족하는 다양한  $s, t$  값을 대입함으로써 삼각형 상의 서로 다른 점들을 생성한다.

## C.4 평면

평면은 무한히 얇고, 무한히 넓으며, 무한히 긴 종이 한 장으로 볼 수 있습니다. 평면은 벡터  $\mathbf{n}$ 과 평면 위의 점  $\mathbf{p}_0$ 로 지정할 수 있습니다. 벡터  $\mathbf{n}$ 은 반드시 단위 길이가 아니며, 평면의 **법선 벡터**라고 불리며 평면에 수직입니다(그림 C.6 참조). 평면은 공간을 양의 반공간과 음의 반공간으로 나눕니다. 양의 반공간은 평면 앞쪽의 공간으로, 평면 앞쪽은 법선 벡터가 발산하는 쪽을 의미한다. 음의 반공간은 평면 뒤쪽의 공간이다.

그림 C.6에서 볼 수 있듯이, 평면의 그래프는 평면 방정식을 만족하는 모든 점  $\mathbf{p}$ 로 이루어집니다:

$$\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = 0$$

특정 평면을 설명할 때, 법선  $\mathbf{n}$ 과 평면 상의 알려진 점  $\mathbf{p}_0$ 이 고정되므로, 평면 방정식을 다음과 같이 재표현하는 것이 일반적이다:

$$\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = \mathbf{n} \cdot \mathbf{p} - \mathbf{n} \cdot \mathbf{p}_0 = \mathbf{n} \cdot \mathbf{p} + d = 0$$

$d = -\mathbf{n} \cdot \mathbf{p}_0$ . 만약  $\mathbf{n} = (a, b, c)$ 이고  $\mathbf{p} = (x, y, z)$ 라면, 평면 방정식은 다음과 같이 쓸 수 있습니다:

$$ax + by + cz + d = 0$$

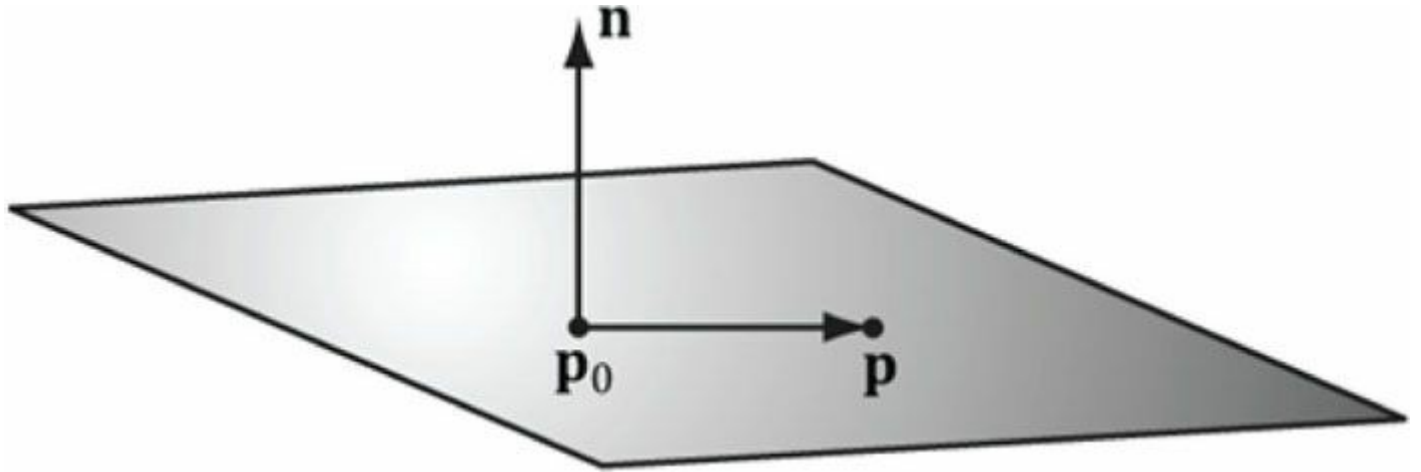
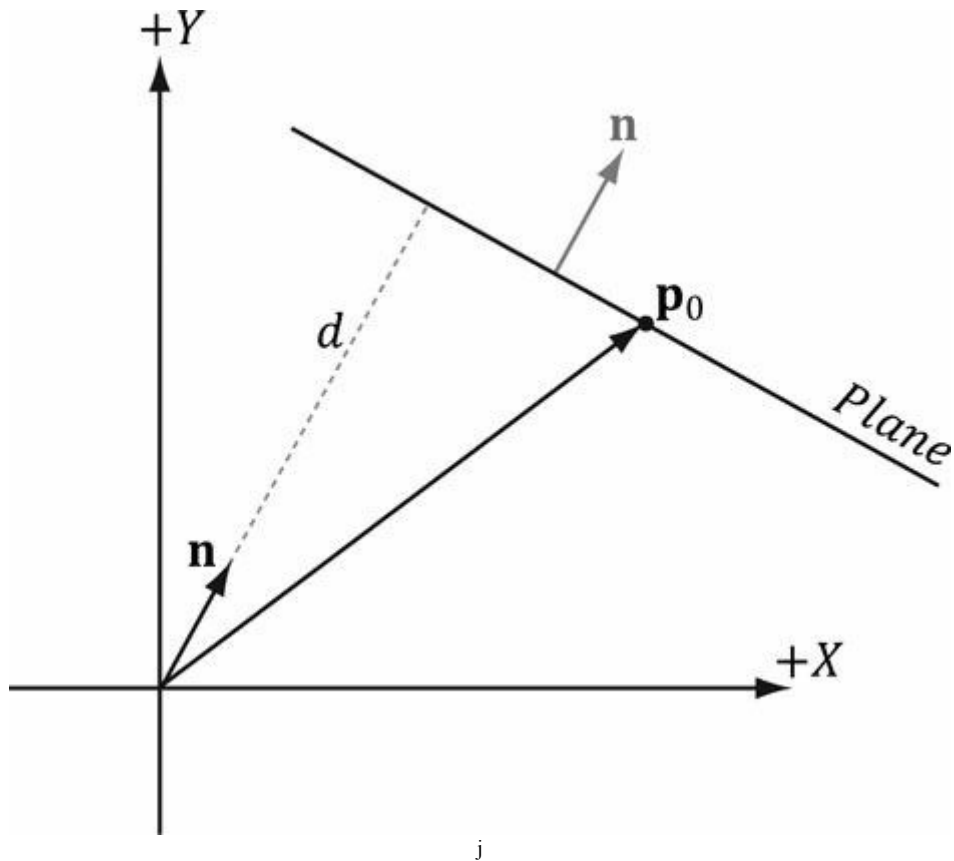


그림 C.6. 법선 벡터  $\mathbf{n}$ 과 평면 상의 점  $\mathbf{p}_0$ 로 정의된 평면.  $\mathbf{p}_0$ 가 평면 상의 점일 때, 점  $\mathbf{p}$ 도 평면 상에 존재하는 것은 벡터  $\mathbf{p} - \mathbf{p}_0$ 가 평면의 법선 벡터에 직교할 때와만 동치이다.



j

그림 C.7. 평면에서 원점까지의 최단 거리.

평면의 법선 벡터  $\mathbf{n}$ 이 단위 길이를 가질 때,  $d = -\mathbf{n} \cdot \mathbf{p}_0$ 는 원점으로부터 평면까지의 최단 부호 거리(signed distance)를 나타낸다(그림 C.7 참조).

그림을 그리기 쉽게 하기 위해, 우리는 때때로 도형을 2차원으로 그리고 평면을 직선으로 표현한다. 수직 법선을 가진 직선은 2차원 공간을 양의 반공간과 음의 반공간으로 나누므로 2차원 평면으로 생각할 수 있다.

양의 반공간과 음의 반공간으로 나누기 때문입니다.

### C.4.1 XNA Math 평면

$(\mathbf{n}, d) = (a, b, c, d)$  평면을 코드로 표현할 때는 법선 벡터  $\mathbf{n}$ 과 상수  $d$ 만 저장하면 충분합니다. 이를 4차원 벡터로 생각하는 것이 유용하며, 우리는 이를 XMVECTOR로 표기합니다. 따라서 XMVECTOR 유형은 부동 소수점 값의 4-튜플을 저장하므로, XNA 수학 라이브러리는 평면도 표현할 수 있도록 XMVECTOR 유형을 오버로드합니다.

### C.4.2 점/평면 공간 관계

임의의 점  $\mathbf{p}$ 에 대해, 그림 C.6과 그림 C.8을 관찰하면

- 다음과 같습니다.  $\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = \mathbf{n} \cdot \mathbf{p} + d > 0$   $\mathbf{p}$ 는 평면 앞에 위치합니다.
- 만약  $\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = \mathbf{n} \cdot \mathbf{p} + d < 0$   $\mathbf{p}$ 는 평면 뒤에 위치한다.

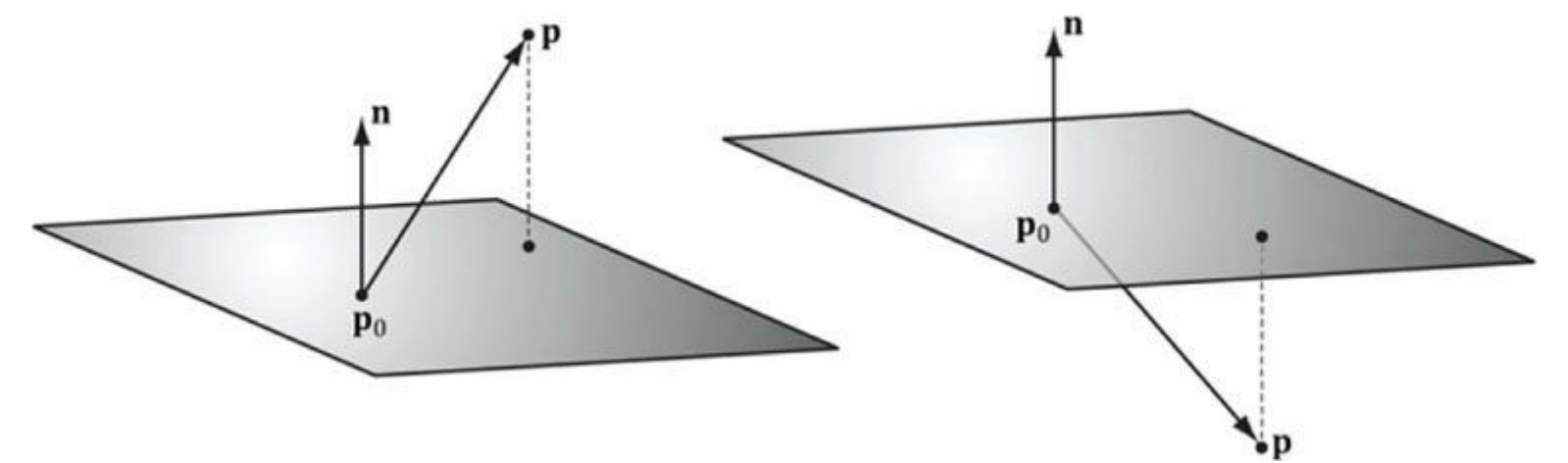


그림 C.8. 점과 평면의 공간적 관계.

- $\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = \mathbf{n} \cdot \mathbf{p} + d = 0$ 이면  $\mathbf{p}$ 는 평면 위에 있다.

이러한 테스트는 평면에 대한 점의 공간적 위치를 테스트하는 데 유용합니다. 다음 XNA 수학 함수는 특정 평면과 점에 대해  $\mathbf{n} \cdot \mathbf{p} + d$ 를 계산합니다:

```
 $\mathbf{n} \cdot \mathbf{p} + d$ XMVECTOR XMPlaneDotCoord(// 각 좌표에 복제된 평면과 점의 벡터 점의 교차점을 반환합니다 XMVECTOR P, // 평면
XMVECTOR P, // w = 1인 점
```

```
// 평면에 대한 점의 위치 테스트
XMVECTOR p = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);

XMVECTOR v = XMVectorSet(3.0f, 5.0f, 2.0f);

float x = XMVectorGetX(XMPlaneDotCoord(p, v));

if(x ≈ 0.0f) // v는 평면과 동일 평면상에 있음. if(x > 0) // v는 양의 반공간에 있음.
if(x < 0) // v는 음의 반공간에 위치합니다.
```

Note: 부동 소수점의 부정확성 때문에 대략적으로 같다고 표현합니다.

유사한 함수는 다음과 같습니다:

```
XMVECTOR XMPlaneDotNormal(XMVECTOR Plane, XMVECTOR Vec);
```

이 함수는 평면 법선 벡터와 주어진 3차원 벡터의 내적을 반환합니다.

### C.4.3 구성

평면 계수  $(\mathbf{n}, d) = (a, b, c, d)$ 를 직접 지정하는 것 외에도, 이 계수들을 두 가지 다른 방법으로 계산할 수 있습니다. 법선  $\mathbf{n}$ 과 평면 상의 알려진 점  $\mathbf{p}_{(0)}$ 이 주어지면  $d$  성분을 구할 수 있습니다:

$$\mathbf{n} \cdot \mathbf{p}_0 + d = 0 \Rightarrow d = -\mathbf{n} \cdot \mathbf{p}_0$$

XNA Math 라이브러리는 점과 법선을 사용하여 평면을 생성하는 다음과 같은 함수를 제공합니다:

```
XMVECTOR XMPlaneFromPointNormal(XMVECTOR Point,
 XMVECTOR Normal);
```

평면을 구성하는 두 번째 방법은 평면 상의 서로 다른 세 점을 지정하는 것이다. 점  $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ 가 주어지면 평면 상에 두 벡터를 형성할 수 있다:

$$\begin{aligned} \mathbf{u} &= \mathbf{p}_1 - \mathbf{p}_0 \\ \mathbf{v} &= \mathbf{p}_2 - \mathbf{p}_0 \end{aligned}$$

이를 통해 평면 상의 두 벡터의 외적(좌손 법칙을 기억하세요)을 구함으로써 평면의 법선을 계산할 수 있습니다.

$$\mathbf{n} = \mathbf{u} \times \mathbf{v}$$

그런 다음  $d = -\mathbf{n} \cdot \mathbf{p}_0$ 를 계산합니다.

XNA Math 라이브러리는 평면 상의 세 점을 주어 평면을 계산하는 다음 함수를 제공합니다:

```
XMVECTOR XMPlaneFromPoints(XMVECTOR
 Point1, XMVECTOR Point2, XMVECTOR
 Point3);
```

### C.4.4 평면의 정규화

평면을 가지고 있고 법선 벡터를 정규화하고 싶은 경우가 있을 수 있습니다. 처음에는 다른 벡터와 마찬가지로 법선 벡터를 정규화하면 될 것 같아 보입니다. 하지만  $d$  성분도 법선 벡터에 의존한다는 점을 기억하세요:  $d = -\mathbf{n} \cdot \mathbf{p}_0$ . 따라서 법선 벡터를 정규화하면  $d$ 도 재계산해야 합니다. 이는 다음과 같이 수행됩니다:

$$d' = \frac{d}{||\mathbf{n}||} = -\frac{\mathbf{n}}{||\mathbf{n}||} \cdot \mathbf{p}_0$$

따라서 평면  $(\mathbf{n}, d)$ 의 법선 벡터를 정규화하는 다음 공식이 있습니다:

$$\frac{1}{||\mathbf{n}||}(\mathbf{n}, d) = \left( \frac{\mathbf{n}}{||\mathbf{n}||}, \frac{d}{||\mathbf{n}||} \right)$$

평면의 법선 벡터를 정규화하기 위해 다음 XNA Math 함수를 사용할 수 있습니다:

```
XMVECTOR XMPlaneNormalize(XMVECTOR P);
```

### C.4.5 평면 변환

[Lengyel02]에 따르면, 평면  $(\mathbf{n}, d)$ 을 4차원 벡터로 간주하고 원하는 변환 행렬의 역전치 행렬을 곱함으로써 평면을 변환할 수 있습니다. 평면의 법선 벡터는 먼저 정규화되어야 합니다. 이를 위해 다음 XNA Math 함수를 사용합니다:

```
XMVECTOR XMPlaneTransform(XMVECTOR P, XMATRIX M);
```

샘플 코드:

```
XMMATRIX T(...); // T를 원하는 변환으로 초기화합니다. XMMATRIX invT =
XMMatrixInverse(XMMatrixDeterminant(T), T); XMMATRIX invTransposeT =
XMMatrixTranspose(invT);

XMVECTOR p = (...); // 평면 초기화.

p = XMPlaneNormalize(p); // 법선 벡터가 정규화되었는지 확인. XMVECTOR transformedPlane =
XMPlaneTransform(p, &invTransposeT);
```

C.4.6 평면 상의 주어진 점에 가장 가까운 점

공간 내 점  $p$ 가 주어졌을 때, 평면  $(\mathbf{n}, d)$  상에서  $p$ 에 가장 가까운 점  $q$ 를 찾고자 한다. 그림 C.9에서 다음과 같이 나타낼 수 있다.

$$\mathbf{q} = \mathbf{p} - \text{proj}_{\mathbf{n}}(\mathbf{p} - \mathbf{p}_0)$$

$\|\mathbf{n}\| = 1$  이므로  $\text{proj}_{\mathbf{n}}(\mathbf{p} - \mathbf{p}_0) = [(\mathbf{p} - \mathbf{p}_0) \cdot \mathbf{n}] \mathbf{n}$  이라고 가정하면, 이를 다음과 같이 다시 쓸 수 있습니다:

$$\begin{aligned} \mathbf{q} &= \mathbf{p} - [(\mathbf{p} - \mathbf{p}_0) \cdot \mathbf{n}] \mathbf{n} \\ &= \mathbf{p} - (\mathbf{p} \cdot \mathbf{n} - \mathbf{p}_0 \cdot \mathbf{n}) \mathbf{n} \\ &= \mathbf{p} - (\mathbf{p} \cdot \mathbf{n} + d) \mathbf{n} \end{aligned}$$

C.4.7 선분과 평면의 교점

주어진 선분  $\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{u}$ 와 평면 방정식  $\mathbf{n} \cdot \mathbf{p} + d = 0$ 가 주어졌을 때, 이 선분이 평면과 교차하는지 여부와 교점 위치를 알고자 한다. 이를 위해 선분을 평면 방정식에 대입하고 평면 방정식을 만족하는 매개변수  $t$ 를 구함으로써 교점을 결정하는 매개변수를 얻는다:

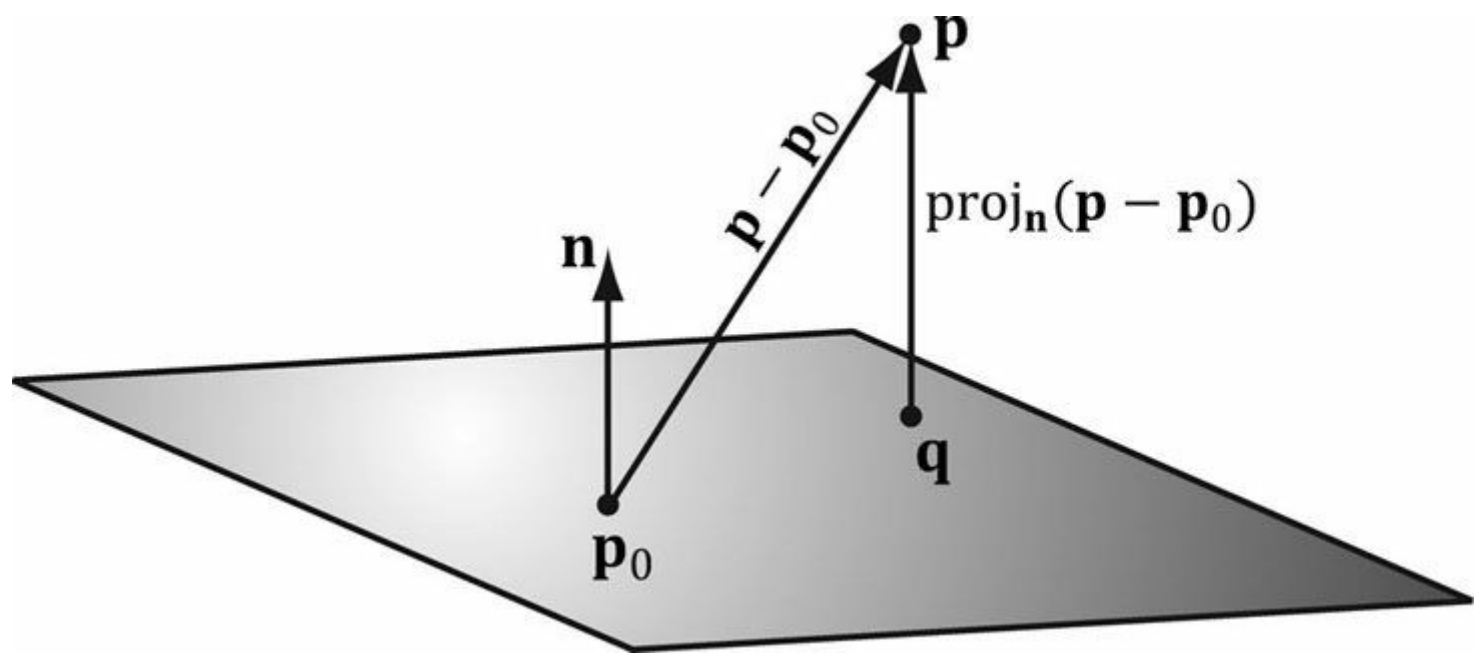


그림 C.9. 평면 상의 점 p에 가장 가까운 점. 점 p0는 평면 상의 점이다.

|                                                                        |                                               |
|------------------------------------------------------------------------|-----------------------------------------------|
| $\mathbf{n} \cdot \mathbf{p}(t) + d = 0$                               | 평면 방정식에 선분을 대입하라                              |
| $\mathbf{n} \cdot (\mathbf{p}_0 + t\mathbf{u}) + d = 0$                | 대입                                            |
| $\mathbf{n} \cdot \mathbf{p}_0 + t\mathbf{n} \cdot \mathbf{u} + d = 0$ | 분배법칙 적용                                       |
| $t\mathbf{n} \cdot \mathbf{u} = -\mathbf{n} \cdot \mathbf{p}_0 - d$    | 양변에 $-\mathbf{n} \cdot \mathbf{p}_0 - d$ 를 더함 |
|                                                                        |                                               |

|                                                                              |         |
|------------------------------------------------------------------------------|---------|
| $t = \frac{-\mathbf{n} \cdot \mathbf{p}_0 - d}{\mathbf{n} \cdot \mathbf{u}}$ | 에 대해 풀기 |
|------------------------------------------------------------------------------|---------|

$\mathbf{n} \cdot \mathbf{u} = 0$ 이면 광선은 평면과 평행하며, 해가 없거나 무한히 많은 해가 존재한다(광선이 평면과 일치할 경우 무한함).  $t$ 가 구간  $[0, \infty)$ 에 속하지 않으면 광선은 평면과 교차하지 않지만, 광선과 일치하는 직선은 평면과 교차한다.  $t$ 가 구간  $[0, \infty)$ 에 속하면 광선은 평면과 교차하며, 교점은 광선의 방정식을  $t$ 이서 광선 방정식을 평가하여 구합니다.

선분/평면 교차 테스트는 선분 정의점  $\mathbf{p}$ 와  $\mathbf{q}$ 를 사용하여 선분  $\mathbf{r}(t) = \mathbf{p} + t(\mathbf{q} - \mathbf{p})$ 을 형성함으로써 구현됩니다. 이 선분을 교차 테스트에 사용합니다  $t \in [0, 1]$ 일 때 선분은 평면과 교차하며, 그렇지 않으면 교차하지 않습니다. XNA Math 라이브러리는 다음 함수를 제공합니다:

```

XMVECTOR XMPlaneIntersectLine(XMVECTOR P,
 XMVECTOR LinePoint1,
 XMVECTOR LinePoint2);

```

### C.4.8 벡터 반사

벡터  $\mathbf{I}$ 를 주어졌을 때, 법선 벡터  $\mathbf{n}$ 을 가진 평면에 대해 이를 반사시키고자 한다. 벡터는 위치가 없으므로, 벡터 반사 시에는 평면의 법선만 관여한다. 그림 C.10은 기하학적 상황을 보여주며, 이를 통해 반사된 벡터는 다음과 같이 주어짐을 알 수 있다:

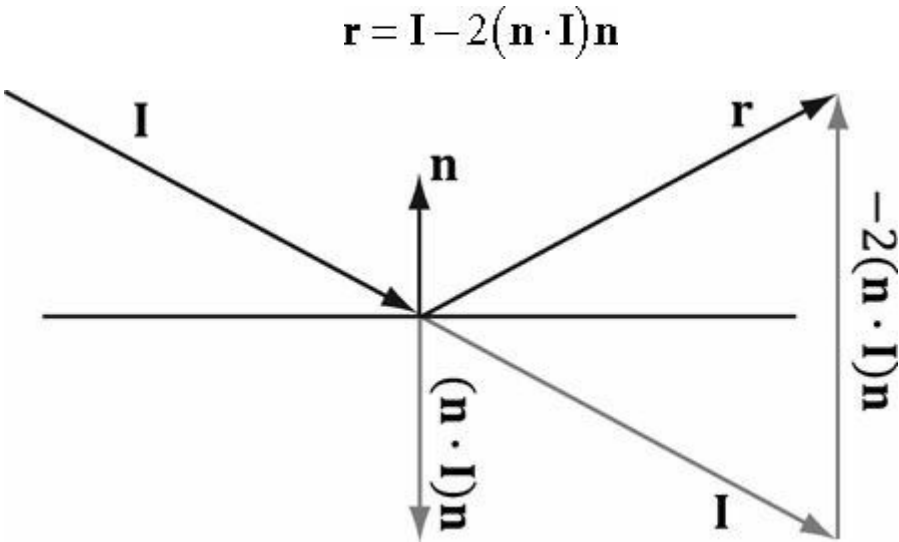


그림 C.10. 벡터 반사의 기하학적 구조.

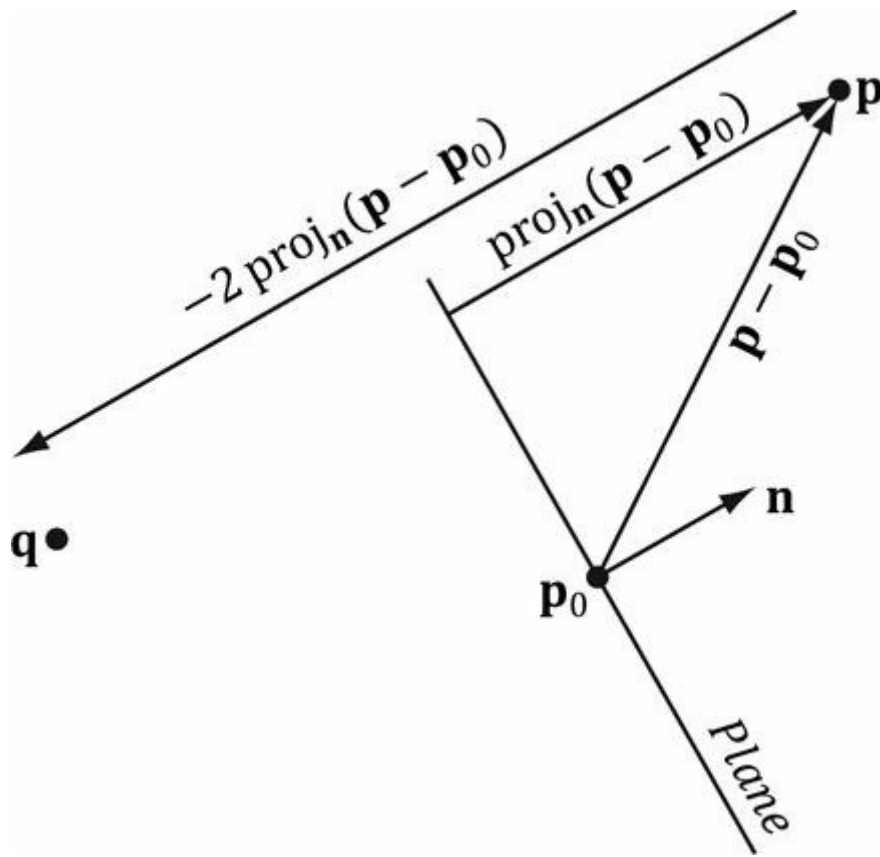


그림 C.11. 점 반사의 기하학적 구조.

### C.4.9 점의 대칭 반사

점은 위치가 있으므로 벡터와 달리 반사됩니다. 그림 C.11은 반사된 점  $\mathbf{q}$ 가 다음과 같이 주어짐을 보여줍니다:

$$\mathbf{q} = \mathbf{p} - 2\text{proj}_{\mathbf{n}}(\mathbf{p} - \mathbf{p}_0)$$

### C.4.10 반사 행렬

평면의 계수를  $(\mathbf{n}, d) = (n_x, n_y, n_z, d)$ 로 정의하자. 여기서  $d = -\mathbf{n} \cdot \mathbf{p}_0$ 이다. 그러면, 동차 좌표를 사용하여 단일 동차 반사 행렬( $4 \times 4$ )로 이 평면에 대해 점과 벡터를 모두 반사할 수 있다:

$$\mathbf{R} = \begin{bmatrix} 1 - 2n_x n_x & -2n_x n_y & -2n_x n_z & 0 \\ -2n_x n_y & 1 - 2n_y n_y & -2n_y n_z & 0 \\ -2n_x n_z & -2n_y n_z & 1 - 2n_z n_z & 0 \\ -2dn_x & -2dn_y & -2dn_z & 1 \end{bmatrix}$$

이 행렬은 평면이 정규화되어 있다고 가정합니다. 즉,

$$\begin{aligned} \text{proj}_{\mathbf{n}}(\mathbf{p} - \mathbf{p}_0) &= [\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0)] \mathbf{n} \\ &= [\mathbf{n} \cdot \mathbf{p} - \mathbf{n} \cdot \mathbf{p}_0] \mathbf{n} \\ &= [\mathbf{n} \cdot \mathbf{p} + d] \mathbf{n} \end{aligned}$$

이 행렬로 점을 곱하면 점의 대칭 반사 공식이 도출됩니다:

$$\begin{aligned}
& \begin{bmatrix} p_x, p_y, p_z, 1 \end{bmatrix} \begin{bmatrix} 1-2n_xn_x & -2n_xn_y & -2n_xn_z & 0 \\ -2n_xn_y & 1-2n_y n_y & -2n_y n_z & 0 \\ -2n_xn_z & -2n_y n_z & 1-2n_z n_z & 0 \\ -2dn_x & -2dn_y & -2dn_z & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_x-2p_xn_xn_x-2p_y n_xn_y-2p_z n_xn_z-2dn_x \\ -2p_xn_xn_y+p_y-2p_y n_y n_y-2p_z n_y n_z-2dn_y \\ -2p_xn_xn_z-2p_y n_y n_z+p_z-2p_z n_z n_z-2dn_z \\ 1 \end{bmatrix}^T \\
&= \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}^T + \begin{bmatrix} -2n_x(p_xn_x+p_y n_y+p_z n_z+d) \\ -2n_y(p_xn_x+p_y n_y+p_z n_z+d) \\ -2n_z(p_xn_x+p_y n_y+p_z n_z+d) \\ 0 \end{bmatrix}^T \\
&= \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}^T + \begin{bmatrix} -2n_x(\mathbf{n} \cdot \mathbf{p} + d) \\ -2n_y(\mathbf{n} \cdot \mathbf{p} + d) \\ -2n_z(\mathbf{n} \cdot \mathbf{p} + d) \\ 0 \end{bmatrix}^T \\
&= \mathbf{p} - 2[\mathbf{n} \cdot \mathbf{p} + d]\mathbf{n} \\
&= \mathbf{p} - 2\text{proj}_{\mathbf{n}}(\mathbf{p} - \mathbf{p}_0)
\end{aligned}$$

 행 벡터를 열 벡터로 바꾸기 위해 전치를 취합니다. 이는 단순히 표현을 깔끔하게 하기 위한 것으로, 그렇지 않으면 매우 긴 행 벡터가 생성될 것입니다.

마찬가지로, 벡터에 이 행렬을 곱하면 벡터 반사 공식이 도출됩니다:

$$\begin{aligned}
& \begin{bmatrix} v_x, v_y, v_z, 0 \end{bmatrix} \begin{bmatrix} 1-2n_xn_x & -2n_xn_y & -2n_xn_z & 0 \\ -2n_xn_y & 1-2n_y n_y & -2n_y n_z & 0 \\ -2n_xn_z & -2n_y n_z & 1-2n_z n_z & 0 \\ -2dn_x & -2dn_y & -2dn_z & 1 \end{bmatrix} \\
&= \mathbf{v} - 2(\mathbf{n} \cdot \mathbf{v})\mathbf{n}
\end{aligned}$$

다음 XNA Math 함수는 평면을 주어 이전 반사 행렬을 구성하는 데 사용할 수 있습니다:

`XMMATRIX XMMatrixReflect(XMVECTOR ReflectionPlane);`

## C.5 연습문제

- $\mathbf{p}(t) = (1, 1) + t(2, 1)$ 를 어떤 좌표계에 대한 광선이라고 하자.  $t = 0.0, 0.5, 1.0, 2.0, 5.0$ 에서의 광선 상의 점들을 그린다.
- $\mathbf{p}_0$ 와  $\mathbf{p}_1$ 이 선분의 양 끝점을 정의한다고 하자. 선분의 방정식이  $t \in [0, 1]$ 에 대해  $\mathbf{p}(t) = (1-t)\mathbf{p}_0 + t\mathbf{p}_1$ 로도 표현될 수 있음을 보여라.
- 각 부분에 대해, 두 점을 지나는 직선의 벡터 방정식을 구하라.

$$\mathbf{p}_1 = (2, -1), \mathbf{p}_2 = (4, 1)$$

$$\mathbf{p}_1 = (4, -2, 1), \mathbf{p}_2 = (2, 3, 2)$$

- $\mathbf{L}(t) = \mathbf{p} + t\mathbf{u}$ 가 3차원 공간에서 직선을 정의한다고 하자.  $\mathbf{q}$ 를 3차원 공간의 임의의 점이라 하자.  $\mathbf{q}$ 에서 직선까지의 거리가 다음과 같이 표현될 수 있음을 증명하라:

$$d = \frac{\|(\mathbf{q} - \mathbf{p}) \times \mathbf{u}\|}{\|\mathbf{u}\|}$$

다음 점들에서 직선까지의 거리를 구하라:

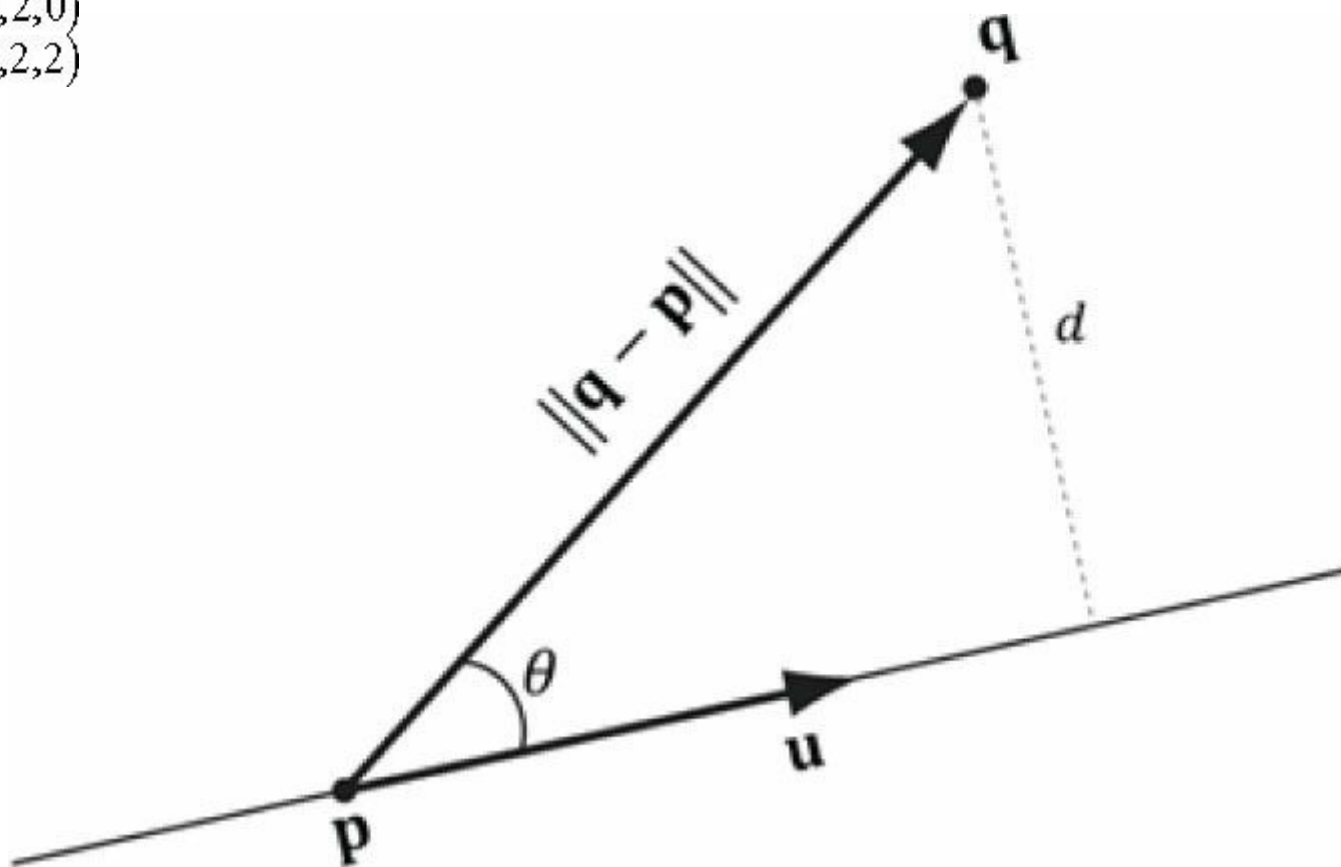
- 다음을  $\mathbf{L}(t) = (4, 2, 2) + t(1, 1, 1)$

그하시오.

$$\mathbf{q} = (0, 0, 0)$$

$$\mathbf{q} = (4, 2, 0)$$

$$\mathbf{q} = (0, 2, 2)$$



그림/C.12. 점  $\mathbf{q}$ 에서 직선까지의 거리.

- 다음과 같이 정의하라.  $\mathbf{p}_0 = (0, 1, 0)$ ,  $\mathbf{p}_1 = (-1, 3, 6)$  그리고  $\mathbf{p}_2 = (8, 5, 3)$ 가 세 점이라고 하자. 이 점들이 정의하는 평면을 구하라.

- 평면이  $\left(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, -5\right)$  평면이 주어졌다고 하자. 다음 점들의 평면에 대한 국소성을 정의하라.  $(3\sqrt{3}, 5\sqrt{3}, 0)$ ,  $(2\sqrt{3}, \sqrt{3}, 2\sqrt{3})$  그리고  $(\sqrt{3}, -\sqrt{3}, 0)$

8. 평면  $\left(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0, \frac{5}{\sqrt{2}}\right)$  평면이라 하자. 평면 상에서 점  $(0,1,0)$ 에 가장 가까운 점을 구하라.
9. 평면  $\left(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0, \frac{5}{\sqrt{2}}\right)$  평면이 되도록 하라. 점  $(0,1,0)$ 에 대한 평면에서의 대칭점을 구하라.
10. 평면  $\left(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, -5\right)$  평면이고,  $\mathbf{r}(t) = (-1, 1, -1) + t(1, 0, 0)$  이 빔이라고 하자. 빔이 평면과 교차하는 점을 구하라. 그런 다음 `XMPlaneIntersectLine` 함수를 사용하여 답을 검증하는 간단한 프로그램을 작성하라.

# 선별된 연습문제 해답

## 해답

교재 내 선별된 연습문제의 해답(도표 포함)은 부속 DVD에서 확인할 수 있습니다.

# 참고문헌 및

## 추가 참고문헌

[Angel00] Angel, Edward, *Interactive Computer Graphics: A Top-Down Approach with OpenGL, Second Edition*, Addison-Wesley, Boston, MA, 2000.

[BilodInteractive Computer Graphics: A Top-Down Approach with OpenGL, Second Edition, Downp-Down Approach with OpenGL, Second Editi OpenGL, Second Edition, Addison-Wesley, Boston, MA, 2000.

[Bilodeau10] Bilodeau, Bill. "효율적인 컴퓨터 셰이더 프로그래밍," 게임 개발자 컨퍼런스, AMD 슬라이드 발표, 2010. ([http://developer.amd.com/gpu\\_assets/Efficient%20Compute%20Shader%20Programming.pps](http://developer.amd.com/gpu_assets/Efficient%20Compute%20Shader%20Programming.pps))

[Bilodeau10b] 빌로도, 빌( ). "Direct3D 11 튜토리얼: 테셀레이션," 게임 개발자 컨퍼런스, AMD 슬라이드 프레젠테이션, 2010. ([http://developer.amd.com/gpu\\_assets/Direct3D%2011%20Tessellation%20Tutorial.ppsx](http://developer.amd.com/gpu_assets/Direct3D%2011%20Tessellation%20Tutorial.ppsx))

[Blinn78] Blinn, James F., and Martin E. Newell. "Clipping Using Homogeneous Coordinates." In *Computer Graphics (SIGGRAPH '78 Proceedings)*, pages 245–251, New York, 1978.

[Blinn96] Blinn, Jim, 『Jim Blinn의 코너: 그래픽스 파이프라인 탐구』, Morgan Kaufmann Publishers, Inc., 샌프란시스코 캘리포니아, 1996. [Boyd08] Boyd, Chas. "DirectX 11 컴퓨터 셰이더," Siggraph 슬라이드 발표 자료, 2008.

(<http://s08.idav.ucdavis.edu/boyd-dx11-compute-shader.pdf>)

[Boyd10] 보이드, 채스 "DirectCompute 강의 시리즈 101: 소개 DirectCompute DirectCompute," 2010. (<http://channel9.msdn.com/Blogs/gclassy/DirectCompute-Lecture-Series-101-Introduction-to-DirectCompute>)

[Brennan02] Brennan, Chris. "물체 거리 보정을 통한 정확한 반사 및 굴절," Direct3D ShaderX: 버텍스 및 픽셀 셰이더 팁과 트릭, Wordware Publishing Inc., Plano, TX, 2002.

[Burg10] Burg, 존 반 데어, "건축 an 고급 입자 시스템 구축," Gamasutra, 2000년 2000. ([http://www.gamasutra.com/features/20000623/vanderburg\\_01.htm](http://www.gamasutra.com/features/20000623/vanderburg_01.htm))

[De berg00] de Berg, M., M. van Kreveld, M. Overmars, and O. Schwarzkopf, *Computational Geometry: Algorithms and Applications*, Second Edition, Springer-Verlag Berlin Heidelberg, The Netherlands, 2000.

[Dietrich] Dietrich, Sim. "Texture Space Bump Maps," 2010. ([http://developer.nvidia.com/object/texture\\_space\\_bump\\_mapping.html](http://developer.nvidia.com/object/texture_space_bump_mapping.html)) [Dunlop03] Dunlop, Robert. "FPS 대 프레임 시간," 2003.

([http://www.mvps.org/directx/articles/fps\\_versus\\_frame\\_time.htm](http://www.mvps.org/directx/articles/fps_versus_frame_time.htm)) [DXSDK10] Microsoft DirectX 2010년 6월 SDK 문서, Microsoft Corporation, 2010.

[Eberly01] Eberly, David H. 『3D 게임 엔진 설계』, Morgan Kaufmann Publishers, Inc., 샌프란시스코, 캘리포니아, 2001.

[Engel02] Engel, Wolfgang (편집), *Direct3D ShaderX: 버텍스 및 픽셀 셰이더 팁과 트릭*, Wordware Publishing, Plano, TX, 2002. [Engel04] Engel, Wolfgang (편집), *ShaderX2: DirectX 9를 활용한 셰이더 프로그래밍 팁 & 트릭*, Wordware Publishing, Plano, TX, 2004. [Engel06] Engel, Wolfgang (편집), *ShaderX5: 셰이더 고급 렌더링 기법*, Charles River Media, Inc., Boston, MA, 2006. [Engel08] Engel, Wolfgang (편집), *ShaderX6: 셰이더 고급 렌더링 기법*, Charles River Media, Inc., Boston, MA, 2008. [Farin98] Farin, Gerald, and Dianne Hansford, 《기하학 도구 상자: 그래픽스와 모델링을 위한》, AK Peters, Ltd., Natick, MA, 1998. [Fernando03] Fernando, Randima, and Mark J. Kilgard, 《CG 튜토리얼: 프로그래밍 가능한 실시간 그래픽스의 결정판 가이드》, 애디슨-웨슬리, 매사추세츠 주 보스턴, 2003.

[Fraleigh95] Fraleigh, John B., and Raymond A. Beauregard, 《선형대수학》, 제3판, 애디슨-웨슬리, 매사추세츠 주 보스턴, 1995.

[Friedberg03] Friedberg, Stephen H., Arnold J. Insel, and Lawrence E. Spence, 《선형대수학》, 제4판, Pearson Education, Inc., 매사추세츠 주 보스턴, 2003.

[Fung10] 퐁, 제임스 "DirectCompute 강의 시리즈 210: GPU 최적화 및 성능," 2010. (<http://channel9.msdn.com/Blogs/gclassy/DirectCompute-Lecture-Series-210-GPU-Optimizations-and-Performance>)

[Halliday01] Halliday, David, Robert Resnick, and Jearl Walker, *Fundamentals of Physics*, Sixth Edition, John Wiley & Sons, Inc., Hoboken, NJ, 2001.

[Hausner98] Hausner, Melvin, *A Vector Space Approach to Geometry*, Dover Publications, Inc. (<http://www.doverpublications.com>),

Englewood Cliffs, NJ, 1998.

[Hoffmann75] Hoffmann, Banesh, 벡터에 관하여, Dover Publications, Inc. (<http://www.doverpublications.com>), 엔글우드 클리프스, 뉴저지, 1975. [Isidoro06] Isidoro, John R. "새도 매핑: GPU 기반 토폴과 기법," 게임 개발자 컨퍼런스, ATI 슬라이드 발표, 2006. ([http://developer.amd.com/media/gpu\\_assets/Isidoro-ShadowMapping.pdf](http://developer.amd.com/media/gpu_assets/Isidoro-ShadowMapping.pdf))

[Isidoro06b] Isidoro, John R. "새도 매핑 중 계산 클링을 위한 예지 마스킹 및 텍셀별 깊이 범위 전파," ShaderX 5: 고급 렌더링 기법, Charles River Media, 보스턴, 매사추세츠, 2007.

[Kilgard99] Kilgard, Mark J. "스텐실 버퍼를 이용한 반사 및 그림자 생성," 게임 개발자 컨퍼런스, NVIDIA 슬라이드 발표, 1999. (<http://developer.nvidia.com/docs/IO/1407/ATT/stencil.ppt>)

[Kilgard01] Kilgard, Mark J. "현재 OpenGL 하드웨어를 활용한 새도 매핑," 컴퓨터 엔터테인먼트 소프트웨어 협회 CEDEC, NVIDIA 발표, 2001. ([http://developer.nvidia.com/object/cedec\\_shadowmap.html](http://developer.nvidia.com/object/cedec_shadowmap.html))

[Kryachko05] Kryachko, Yuri. "현실적인 물 렌더링을 위한 정점 텍스처 범위 활용," GPU Gems 2: 고성능 그래픽스와 범용 컴퓨팅을 위한 프로그래밍 기법, Addison-Wesley, 보스턴, 매사추세츠, 2005.

[Kuipers99] Kuipers, Jack B. 『퀴터니언과 회전 시퀀스: 궤도, 항공우주 및 가상 현실에의 응용을 위한 입문서』, 프린스턴 대학교 출판부, 프린스턴, 뉴저지, 1999.

[Lengyel02] 랭겔, 에릭, 『3D 게임 프로그래밍 및 컴퓨터 그래픽스를 위한 수학』. 찰스 리버 미디어, 보스턴, 매사추세츠, 2002. [Möller08] 뮐러, 토마스, 에릭 헤인스 공저, 『실시간 렌더링』, 제3판, AK 피터스, 네이틱, 매사추세츠, 2008.

[Mortenson99] Mortenson, M.E. 『컴퓨터 그래픽스 응용을 위한 수학』, Industrial Press, Inc., 뉴욕, NY, 1999.

[NVIDIA05] 엔티앨리어싱 with 투명도 NVIDIA 코퍼레이션, 2005. ([ftp://download.nvidia.com/developer/SDK/Individual\\_Samples/DEMOS/Direct3D9/src/AntiAliasingWithTransparency/docs/AntiAliasingWi](ftp://download.nvidia.com/developer/SDK/Individual_Samples/DEMOS/Direct3D9/src/AntiAliasingWithTransparency/docs/AntiAliasingWi))

[NVIDIA08] GPU 프로그래밍 가이드 GeForce 8 및 9 시리즈 엔비디아 코퍼레이션, 2008. ([http://developer.download.nvidia.com/GPU\\_Programming\\_Guide/GPU\\_Programming\\_Guide\\_G80.pdf](http://developer.download.nvidia.com/GPU_Programming_Guide/GPU_Programming_Guide_G80.pdf))

[NVIDIA09] NVIDIA의 차세대 세대 CUDA 컴퓨트 아키텍처: 페르미, 엔비디아 코퍼레이션, 2009. ([http://www.nvidia.com/content/PDF/fermi\\_white\\_papers/NVIDIA\\_Fermi\\_Compute\\_Architecture\\_Whitepaper.pdf](http://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf))

[NVIDIA10] DirectCompute 프로그래밍 가이드 NVIDIA 코퍼레이션, 2007–2010. ([http://developer.download.nvidia.com/compute/DevZone/docs/html/DirectCompute/doc/DirectCompute\\_Programming\\_Guide.pdf](http://developer.download.nvidia.com/compute/DevZone/docs/html/DirectCompute/doc/DirectCompute_Programming_Guide.pdf))

[Oliveira10] 올리베이라, 구스타보 "디자인하기 빠른 크로스 플랫폼 SIMD 벡터 라이브러리 설계," 2010. ([http://www.gamasutra.com/view/feature/4248/designing\\_fast\\_crossplatform\\_simd\\_php](http://www.gamasutra.com/view/feature/4248/designing_fast_crossplatform_simd_php))

[Parent02] Parent, Rick, 컴퓨터 애니메이션: 알고리즘과 기법, Morgan Kaufmann Publishers (<http://www.mkp.com>), 매사추세츠 주 보스턴, 2002.

[Petzer04] 펠저, 커트 "수없이 흔들리는 풀의 렌더링하기," GPU 쟁스: 실시간 그래픽을 위한 프로그래밍 기법, 팀, 트릭, 애디슨-웨슬리, 매사추세츠 주 보스턴, 2004.

[Petzold99] Petzold, Charles, 『Windows 프로그래밍』, 제5판, Microsoft Press, 레드먼드, WA, 1999. [Prorise99] Prorise, Jeff. 『MFC를 이용한 Windows 프로그래밍』, 제2판, Microsoft Press, 레드먼드, WA, 1999. [Santrock 03] Santrock, John W. 『심리학 7』, The McGraw-Hill Companies, Inc., 매사추세츠 주 보스턴, 2003. [Savchenko00] Savchenko, Sergei. 『3D 그래픽스 프로그래밍: 게임과 그 이상』, Sams Publishing, 2000.

[Schneider03] 필립 J. 슈나이더( )와 데이비드 H. 에버리(David H. Eberly), 『컴퓨터 그래픽스를 위한 기하학적 도구』, 모건 카우프만 출판사(Morgan Kaufmann Publishers, <http://www.mkp.com>), 매사추세츠 주 보스턴, 2003.

[Snook 03] Snook, Greg, C++ 및 DirectX9를 이용한 실시간 3D 지형 엔진, Charles River Media, Inc., 매사추세츠 주 보스턴, 2003.

[Story10] Story, Jon, and Cem Cebenoyan. "테셀레이션 성능," 게임 개발자 컨퍼런스, NVIDIA 슬라이드 발표, 2010. ([http://developer.download.nvidia.com/presentations/2010/gdc/Tessellation\\_Performance.pdf](http://developer.download.nvidia.com/presentations/2010/gdc/Tessellation_Performance.pdf))

[Sutherland74] 서덜랜드, I. E., 그리고 G. W. 호지먼, 재진입 가능한 다각형 클리핑. ACM 통신, 17(1):32–42, 1974. [Tuft10] Tuft, David. "Cascaded Shadow Maps," 2010. (<http://msdn.microsoft.com/en-us/library/ee416307%28v=vs.85%29.aspx>)

[Uralsky05] Uralsky, Yuri. "효율적인 소프트웨어 옛지 색도 구현을 위한 픽셀 셰이더 분기 기법," GPU Gems 2: 고성능 그래픽스와 범용 컴퓨팅을 위한 프로그래밍 기법, Addison-Wesley, Boston, MA, 2005.

[Verth04] Verth, James M. van, and Lars M. Bishop, Essential Mathematics for Games & Interactive Applications: A Programmer's Guide. 모건 카우프만 출판사(<http://www.mkp.com>), 매사추세츠 주 보스턴, 2004.

[Vlachos01] Vlachos, Alex, Jörg Peters, Chas Boyd, Jason L. Mitchell, "곡선 PN 삼각형," ACM 대화형 3D 그래픽스 심포지엄 2001, pp. 159–166, 2001. (<http://alex.vlachos.com/graphics/CurvedPNTriangles.pdf>)

[Watt92] Watt, Alan, and Mark Watt, 《고급 애니메이션 및 렌더링 기법: 이론과 실습》, Addison-Wesley, Boston, MA, 1992. [Watt00] Watt, Alan, 《3D 컴퓨터 그래픽스》, 제3판, Addison-Wesley, Boston, MA, 2000.

[Watt01] Watt, Alan, Fabio Policarpo, 『3D 게임: 실시간 렌더링과 소프트웨어 기술』, Addison-Wesley, 매사추세츠 주 보스턴, 2001. [Weinreich98] Weinreich, Gabriel, 『기하학적 벡터』, 시카고 대학교 출판부, 일리노이 주 시카고, 1998.

[Whatley05] Whatley, David. "가상 식물학에서의 사실적 표현을 향하여," GPU Gems 2: 고성능 그래픽스와 범용 컴퓨팅을 위한 프로그래밍 기법, 애디슨-웨슬리, 매사추세츠 주 보스턴, 2005.

[Wloka03] Wloka, Matthias. "배치, 배치, 배치: 진짜 의미는 무엇인가?" 게임 개발자 컨퍼런스 발표, 2003. (<http://developer.nvidia.com/docs/IO/8230/BatchBatchBatch.pdf>)

[XNAMath10] DirectXMath 온라인 문서, Microsoft 코퍼레이션 (<http://msdn.microsoft.com/en-us/library/windows/desktop/hh437833%28v=VS.85%29.aspx>)

t 색상 구성 요소 149-50, 267

비트 심도 버퍼 678, 685

무부호 98, 372

비트 색상 값 150, 266

비트 부동 소수점 구성 요소 93, 313

-비트 색상 값 266

## A

BB (축 정렬 바운딩 박스) 515-20, 526-7, 530, 532, 546, 609, 616-17, 619-20

BB/평면 테스트 525

가속 447, 633-5

포인트, 기본값 106, 137-8

압축 모드 311, 332-5, 338, 342, 636

네 변환 59, 67-8, 70-1, 80, 86, 737

99-100, 674-5, 698

ha 채널 93, 352, 358-62, 381, 406, 422, 587, 591-2, 622, 639, 723

ha component 148, 183, 287, 339, 348-9, 361, 369, 574

ha-to-coverage 351, 400, 422-4

ha-to-coverage 혼합 상태 객체 422 ha 값 362, 370, 422, 424

ient-access 703, 711, 720

ient light 281, 284-5, 687, 701, 719, 721

ient map 711, 716, 718-19, 723

객체 재료 색상 281, 701

ient occlusion 496, 701-3, 705-7, 709, 711, 713, 715, 717, 719, 721-3

치아 폐색 데모 703, 705-6

차단 지도 706, 715, 719

관련 용어 281, 289, 296-8, 496, 701-2, 719-22

le

급성 11

둔각 11

le 제한 747

짜짓기 760, 776, 779, 789

정보 94, 116, 120-1, 127, 136, 175, 207, 344, 473, 477, 492, 496, 760-3, 776-8, 787-90

전체 프레임 94, 136

116, 118 프레임

94, 136의 주어진 프레임

94, 136의 다음 프레임

I 오버헤드 202, 510, 526, 530

응용 프로그램 코드 129-30, 206, 218, 224, 263, 301, 318, 526

응용 프로그램 정의 마스크 값 373 응용 프로그램 인스턴스 125-6

응용 프로그램 메시지 루프 119, 124, 126

애플리케이션 창, 메인 124, 126-7

y 58, 103, 105, 115-16, 199, 261, 263, 265, 268, 275, 295, 351-2, 416, 424, 566

y 최종 변환 행렬의 780, 792

ect 비율 126, 168-71, 174, 184-5, 187, 235, 522, 536-7

어셈블리 코드 222

부속 값 181

가중치 분포 19, 274-5, 364, 370, 450-1, 460, 779, 781

레이지 프레임 126, 128, 137 레이지된

픽셀 색상 100

-정렬된 바운딩 박스 참조: AABB 지침 548, 550

B

kface 컬링 179-80, 363, 475

kground 색상 361

IC 입출력 시스템(BIOS) 117 IC 테셀레이션 데모 493

icModel 클래스 732-4, 789

is, per-triangle 526

벡터 503, 507

표준 62, 65, 72

변환된 71-2

nstein 기본 함수 486, 493

ier 곡선 473, 484-7, 493 ier 패치 데모 494

ier 쿼드 패치 473

ier 표면 473, 487-8, 493-4

3차 488

2차 493-4

ier 삼각형 패치 480, 494 ier 삼각형 표면 패치

493 패치 489-90

ing 674, 676-7, 693

딩 115, 199, 318, 377, 435, 454, 467-8, 643-4, 686

S (기본 입출력 시스템) 117 '의 눈 406, 454-5, 629

nd 347-8, 354, 356-7, 368-70, 380, 413, 585, 621-2, 691, 735

nd 데모 362-4, 368-70, 393, 396-7, 408, 426

nd factor color 350 nd factors 349-52, 369

목적지 348, 350, 354-6, 397 nd 지도 용품 621

nd maps 620, 622, 627-8

그레이스케일 621

nd 작업 349, 351, 355, 370, 397

nd 연산자 350, 354-6

nd 매개 변수 638-9

nd 상태 264, 350-1, 353

기본값 353

nder 145, 735, 792

nding 90, 148, 182-3, 210, 347-9, 351-5, 357-9, 361-3, 369, 422, 620, 638-40, 682, 779-82, 791-2

결정 방정식 348-50, 354-6, 358, 369, 381, 397, 621, 638

결정 방법 357

연산자 찾기 351-2

투명도 찾기 569

eks 97, 371-4, 379, 392, 570, 601, 607, 702

r 430, 449-51, 453-7, 459-65, 469, 715, 718-19

보존 715-16, 718, 722

수직 453, 456

r 반경 451, 461-4

rred 결과 455-6 rredTex

436

링 450, 457-60, 723

링 알고리즘 450, 455-6

e 영향 770, 780-2, 792

e 공간 776

nding box 495, 517-19, 530, 540-1

nding sphere 515, 520, 530, 546, 682-3

nding volumes 509, 515, 520, 526, 530, 541, 546, 615-16, 732, 735

동적 인칭 289, 296-8

페어 94-8, 100-4, 108-16, 127, 130, 136, 195-201, 260-4, 347-8, 379-81, 440-3, 447-8, 454-5, 513-15, 643-4

32비트 부동 소수점 깊이 98, 372

할당됨 261

깊이 스텐실 566

전체 197, 562

부동 소수점 468

부동 소수점 깊이 678

노멀/깊이 707

오프스크린 371, 392

원시 442

두 번째 텍스처 457

정적 260, 262

fer 알파 값 349 fer 정의 230-1

fer 치수 130, 168

fer form 94, 136

요소의 fer 440, 467 fer 픽셀 색상 357

fer 픽셀 형식 108 fer 픽셀 348, 380

미러 스텐실 390

fer 리소스 468

텍스처 94

fer width 107, 126

페어링, 더블 94, 108, 136

## C

era 18, 163, 165-7, 211, 364-5, 406-10, 498-503, 505, 507, 549-50, 554-6, 563, 606-7, 626-8, 654-5

1인칭 495, 497, 499, 501-3, 505, 507

void 501, 503-4

시대 기준 벡터 500 시대 클래스

499, 505, 507

시대 좌표계 164-5, 498, 500, 506-7

era 데모 505-7

era 프러스텀 527-8

era 위치 165, 303, 364-5, 370, 502, 507

era 공간 163-4, 500

era system 495, 497

era vectors 504

t 24, 110, 116, 126, 134, 174, 220, 235-7, 257-8, 261-2, 313, 332, 506, 564-5, 664

t rays 703-5, 722

nge 상수 버퍼 변수 226 nge 표면 형식 340, 359

racter 769, 777-8, 781, 791

캐릭터 애니메이션 495-6, 760, 769, 771, 773, 775, 777, 779, 781, 783, 785, 787, 789, 791

캐릭터 메쉬 774

mp 148, 183, 237, 332-4, 350, 376-7, 439, 463-5, 592, 610, 626, 717

177, 361-2, 367, 412, 529, 584, 685, 690, 707, 776-8, 787, 789

함수 207, 361, 422

ping 176-8, 204, 388, 672, 678

또는

32비트 148-50, 183

128비트 148-9

파란색 146-7

경계 332

목적지 355-6, 638

안개 364, 366, 368, 370, 413, 585, 691

안개 깐 365-6

녹색 100, 147

인스턴스 511

lit 295-7, 366, 368, 413, 585, 691

새로움 146-7, 348

버텍스당 307, 311

픽셀의 557

채널 339, 352, 355, 585

또는  $c_k$  395

또는 구성 요소 146, 148, 150, 573, 575

추가 148, 183

또는 요소 191-2, 265

또는 이미지 574, 594

또는 정보 94-5, 101, 313, 422

또는 강도 397, 638

또는 지도 621-2

또는 작업 147-8, 150, 183

또는 픽셀 314

또는 텍스처링 678

또는 값 207, 209, 263

또는 벡터 146-8, 183, 313, 349, 352

단일 396

오레드 큐브 265-6

M (컴포넌트 객체 모델) 92-3, 135

M 인터페이스 92-3, 126, 135

M 객체 92

비교 함수 374, 681

파일러 197, 214, 216-17, 222

복소수 737-41, 766 복잡한 픽셀 셰이더 작업 684

구성 요소 객체 모델 COM 참조구성 요소

비트 부동 소수점 50 부동 소수점 94, 313, 636

노멀 275, 526

무부호 정수 93, 313

위치 70, 78, 81, 753

압축된 형식 340-1, 359, 435, 552

압축 텍스처 형식 339-40, 576

푸트 셰이더 90

상수 가속 447, 634-5, 661 상수 버퍼 값 383

상수 버퍼 변수 219, 332, 336, 438

상수 버퍼 205-7, 213, 218-20, 263-4, 286, 293, 318, 433-4, 511, 566, 588

생략 223, 225

상수 색상 182, 701

스텐트 힐 셰이더 473-4, 479, 492, 589, 617-18

상수 보간 320-1, 323

stant 벡터 23, 32

stant XMVECTOR 인스턴스 23 제어점 출력 476, 493, 591

제어점 156, 472-4, 476-7, 479-81, 484-5, 487-8, 491-4, 588, 591, 610

추가 476-7

버텍스 셰이더 입력 492

도입 165, 179, 181, 385

정점 구형 좌표를 직교 좌표로 변환 235, 302 버터 725-6

좌표 행렬 77-9

좌표 행렬, 좌표 변환 77-81, 85, 160-1, 163-4, 498, 507, 580

좌표 표현 6-7, 74 좌표

중심점 85, 480

부동 소수점 94, 313

ith 387

정상 526

정규화된 장치 170, 174, 177-8, 535, 667

객체의 160

u-텍스처 579

벡터의 6

ner to-far-plane 벡터 707, 711

e 데모 312, 326, 342-4

픽셀 347

팅 버텍스 버퍼 409, 646

장치 572, 585-6, 593, 705

e 맵 455, 547-54, 559-69, 699

동적 559-60, 564-5, 568, 699

e 맵 데모 551, 569-70

e 지도 열골 562-3, 568

e 지도 텍스처 551, 560, 562

e 매핑 496, 547-9, 551, 553, 555, 557, 559, 561, 563, 565, 567, 569, 582

e 텍스처 548, 553

DA 466-7

DA 코어 429, 432

임대 시간 가치 117, 136

## D

DX 라이브러리 19, 137, 213

a 요소 429, 448, 468

93, 135, 312의 배열

93, 312의 행렬

a 멤버 116, 126, 130, 204, 293, 415, 499

a 구조체 447, 725, 734

S 파일 323, 340-2, 553, 568

S 텍스처 이미지 형식 551

ug 출력 창 193-4, 222

th 바이어스 676-8, 693

th 버퍼 95-8, 100-2, 111-12, 114, 134-5, 144, 357-8, 371-5, 383, 392, 566, 673-4, 677-8, 684-5, 720-1

버퍼 범위 116, 535

버퍼링 89, 91, 95-6, 98, 136, 179, 375

th 버퍼링 알고리즘 172, 181 th 비교 테스트 720

복잡도 372, 394-6

좌표 172, 536

NDC 공간에서의 th 671, 687

설정 375, 393

스텐실 562, 566

th/스텐실 103, 115, 125, 127, 130

th/스텐실 버퍼 98-9, 103, 111-14, 125, 127, 132, 150, 314, 354, 372-3, 556, 640-1, 664, 666

메인 721

th/스텐실 버퍼링 372

th/스텐실 상태 264, 378, 382, 393

기본값 377

th/스텐실 상태 블록 374, 376-7, 393

th/스텐실 상태 객체 378, 391

th/스텐실 텍스처 113

th 테스트 97-8, 207, 357-8, 372, 374-7, 380-1, 383, 394, 396-7, 545, 556, 661, 673, 695

th 값 97-8, 172-3, 181, 207, 375, 533, 666, 673-4, 676, 678, 691, 695, 697, 707, 715

해당 97-8

정규화된 172, 556, 688 픽셀 셰이더 수정됨 396

대체, 부분 304, 488, 490, 494, 693 지정 혼합 계수 Fdst 352

지정 픽셀 색상 348

사용 조명 281, 283-4, 308, 558 사용 조명 색상 280

조명 사용 280, 285

조명 계산 사용 280

재료 사용 280, 285, 287, 300, 330, 359, 368-9, 413, 585, 691

재료 색상 사용 280

|                              |                                                                                 |
|------------------------------|---------------------------------------------------------------------------------|
| ct3D 버퍼                      | 195, 263                                                                        |
| ct3D 함수                      | 92, 134-6                                                                       |
| ct3D 리소스                     | 90, 201, 430-1                                                                  |
| 작용 점선                        | 628                                                                             |
| 점선 벡터                        | 287, 559                                                                        |
| 기능 조명                        | 90, 270, 287, 291, 299, 302, 306, 308, 390, 393, 699                            |
| 기능적 그림자 행렬                   | 387                                                                             |
| ctX SDK                      | 104, 213, 221, 341, 552, 576, 790                                               |
| ctX 텍스처 도구                   | 323, 340, 342, 547, 552-3, 568                                                  |
| 블러 픽셀 셰이더                    | 642, 653, 659                                                                   |
| 배치 매핑                        | 158, 469, 484, 496, 571, 573, 575, 577, 579, 581, 585, 587, 589-95, 610-11, 613 |
| 배치                           | 2, 4, 72, 338, 586-7, 593, 686, 721                                             |
| ance                         |                                                                                 |
| 안개 시작                        | 364, 370                                                                        |
| 기능                           | 289, 365, 368                                                                   |
| 계산                           | 588, 610                                                                        |
| 메인                           | 332, 338, 476-7, 480, 483-4, 591-2, 613, 668                                    |
| 정규화된 텍스처 공간                  | 337                                                                             |
| 메인 셰이더                       | 151, 431, 472, 474, 479-80, 483-4, 492-4, 585, 587, 591-3, 607, 613, 615, 627   |
| 제품                           | 10-12, 15, 27, 32, 40-1, 55, 68, 87, 147, 183, 228, 276, 507, 753               |
| 제품 속성                        | 34                                                                              |
| nload                        | 129, 594, 735                                                                   |
| er                           | 99, 104-6, 125, 314, 478                                                        |
| GI_FORMAT_D24_UNORM_S8_UINT  | 98, 113, 372, 392, 665                                                          |
| GI_FORMAT_R8G8B8A8_SINT      | 93, 191, 313                                                                    |
| GI_FORMAT_R8G8B8A8_SNORM     | 93, 313                                                                         |
| GI_FORMAT_R16G16B16A16_UNORM | 93, 313                                                                         |
| GI_FORMAT_R32_UINT           | 201, 236, 257                                                                   |
| GI_FORMAT_R32G32_UINT        | 93, 313                                                                         |
| amic 버퍼                      | 260, 262, 264, 514                                                              |

## E

|                |                                                                   |
|----------------|-------------------------------------------------------------------|
| e 벡터           | 577-8                                                             |
| 해당 텍스처 삼각형     | 577 es                                                            |
| 삼각형            | 34, 274                                                           |
| 상부 삼각형         | 625                                                               |
| cts 프레임워크      | 90, 189, 206, 212-13, 216-17, 223, 225-6, 264, 306, 368, 434, 441 |
| 멘션, 버텍스 셰이더 출력 | 204 t 위치/방향 650, 656                                              |
| 터 파티클          | 647, 654, 660                                                     |
| 단일             | 647-8                                                             |
| 환경 맵           | 496, 549-51, 554, 557-9, 568, 596, 687                            |
| 환경 매핑          | 495-6, 549-50, 558-9, 595, 727                                    |
| 매개 변수화         | 121, 493, 635                                                     |
| 위치             | 280, 282, 307, 533, 611, 632                                      |

## F

|           |                                                                                     |
|-----------|-------------------------------------------------------------------------------------|
| 평면        | 168, 173-4, 176-7, 184, 186-7, 364, 499, 502, 507, 521-3, 527, 555, 666-7, 697, 708 |
| 평면 값      | 184-5                                                                               |
| h, 정점 텍스처 | 262 힌지 텍스처 샘플 454                                                                   |
| d, 수평     | 168-9, 502, 551                                                                     |
| , fx      | 215-16, 318, 327                                                                    |
| 링         | 362, 680-1                                                                          |
| 이방성       | 323-5                                                                               |
| 선형        | 321-4, 480                                                                          |
| rs        | 93, 135, 312, 319, 323-5, 328, 334, 410, 418, 439, 549, 681, 688-9, 712, 717        |
| I 변환 행렬   | 780, 792                                                                            |

ers 6-7, 15-16  
hlight 290, 303, 308  
en 222, 296-8, 390, 452  
t  
16비트 605  
32비트 20, 605  
마지막 291-2  
룩업 벡터 549  
패치 센터 612  
무작위 636  
반환 229, 484, 489, 707  
정적 128, 295  
텍스처 712  
t 접근 711, 715  
페색 713, 715  
t 오프셋 638, 652, 657  
팅 포인트 높이 값 603  
팅 포인트 부정확성 30-1  
팅 포인트 번호 373  
팅 포인트 번호 30-1, 575, 680  
팅 포인트 값 31, 99, 148, 314  
es 2, 4, 8-9, 119  
매트 93-4, 98-9, 102, 111-12, 114, 191, 201, 312-14, 339-40, 359-60, 435-6, 601, 641, 725-7, 730  
custom 725-6  
깊이 버퍼 678, 685  
부동 소수점 442, 574  
가능한 442  
매트 DXGI, 32비트 픽셀 723  
매트 속성 442  
(초당 프레임 수) 119, 126, 128-9, 137, 269, 344, 529  
me 카메라 공간 499  
완료 94, 136  
76의 기원  
이전 119-20  
75-6의 y축  
z축 75-6, 83  
me F0 772  
me 행렬, 변경 77-8, 83  
참조 5-6, 74, 80, 83, 121  
me 비율 116, 129, 137, 368  
텍스처 344  
첫 번째 344  
다음 344  
초당 프레임 수 FPS 참조  
메쉬 105, 124, 127, 212, 369, 372  
벡터 대수 19  
메쉬 함수 124, 132  
가상 124  
메쉬 메소드 124, 126  
t 버퍼 94-5, 136  
tum 166-8, 170, 176-7, 184, 186-7, 499, 501-2, 509, 515, 521-7, 530-1, 533-4, 618-19, 627, 670-3  
프로젝터 672, 698  
텀 쿨링 475, 493, 495, 509, 511, 513-15, 517, 519, 521, 523-7, 529-31, 546, 567, 569, 615  
텀 평면 방정식 178, 522  
tum planes 177, 521, 524-5, 615, 619  
tum 속성 500-1  
ction  
코사인 282-3  
보간 321  
벡터 값 332, 579

속도 634

합수성, 벡텍스 텍스처 페치 262

G

meTimer 118, 121, 123, 266

러시안 블러 451, 453, 722

러시아 함수 451, 453

메트릭 해석 8, 12, 45, 71, 345, 739

미터법 기본 함수 152-3

메트릭 변환 37, 59, 72 메트릭

임의의 496, 669, 697

혼합 358

비혼합 357-8

객체의 159

미터법 버퍼 255

메트리 해상도, 투영 스크린 323

메트리 셰이더 156, 176, 206-7, 262, 399-403, 405, 407-11, 413-15, 423-7, 471-2, 565-8, 631-3, 640-2, 644-5, 647

선택적 213, 264

메트리 셰이더 본체 400, 402

메트리 셰이더 구현 401, 567

메트리 셰이더 입력 176, 399

메트리 셰이더 출력 399, 641

메트릭 셰이더 프로그램 319, 399

메트리 셰이더 시그니처 402, 415, 424

메트리 셰이더 스테이지 150, 176, 399, 423

구체 252

GPU 프로그래밍 429-431, 445

U (그래픽 처리 장치) 90, 112-13, 175, 195-7, 262-3, 340-1, 429-30, 445, 466, 530, 611, 640, 642-3, 645, 661

U 메모리 104, 112, 158, 196, 219-20, 262, 339-40, 342, 442, 640

U 리소스 112, 150-1, 196, 430-1, 442, 472, 640

ss 텍스처 336, 338, 620

en light 146-7, 271, 280, 308

, 삼각형 627를 사용하여

점 240, 260, 470, 623, 627

텍스처 좌표 생성 337 삼각형 243

정점 469, 598, 627

ij번째 241, 337

정점 241

und 평면 165, 390, 406

H

하드웨어 인스턴싱 509-10

하드웨어 테셀레이션 472, 492, 572, 597, 627 빛

적절한 607, 610, 627

웨이브 260

빛 함수 244, 623, 627

ght 값 585, 587, 599, 603, 614

ghtmap 574, 585-7, 591, 593-6, 598-605, 607-8, 613-14, 616, 622-3, 627-8

부동 소수점 603

ghtmap 셰이더 리소스 뷰 605

isphere 636, 702-3, 705, 707, 709, 714, 722

h 수준 셰이딩 언어 HLSL 참조h 품질 픽셀 셰이더 226

hlight 544-5, 565

143, 244, 246, 481, 598, 620, 627-8

SL (고수준 셰이딩 언어) 203-4, 206-7, 263, 284, 294, 296, 439-40, 467-8, 548, 568, 570 SL 함수 출력 295-7

SL 셰이더 컴파일러 229-32

모노겐 클립 공간 166, 174, 176-8, 186, 203-6, 209-10, 214-15, 300, 329, 366, 399, 404, 423, 470, 479

모노제닉 좌표 67-8, 77, 82, 161-2, 164, 171, 498

셰이더 151, 431, 472-3, 476-7, 479, 481, 492-3, 587-9, 591, 610

제어점 473, 476-7, 479, 492-3, 591 셰이더 상수 함수 611

I

다면체 252-3, 424-6

D10Blob 216-17, 238

D10Blob 데이터 구조 216 D11BlendState 210, 264, 351-3, 422-4

D11DepthStencilState 210, 264, 377, 382, 392

D11DepthStencilState 인터페이스 371, 374, 377, 393

D11device::createbuffer 195, 263

D11InputLayout 190, 193-5, 234, 263

D11RasterizerState 210-12, 264, 363, 385

D11RasterizerState 객체 211-12

D11Resource 260, 312, 340, 418, 420

D11Texture1D 312, 637

D11Texture2D 93, 110-11, 114, 125, 340, 418, 436, 454, 561-2, 606, 665

D11Texture2D 인터페이스 312, 415, 424, 568

D11Texture2D 객체 317, 415, 553 D11Texture2D 객체 저장소 553

D11UnorderedAccessView 436-7, 441, 459

DX11EffectPass 220

DX11EffectShaderResourceVariable 변수 435, 440

DX11EffectTechnique 194, 219-20, 234

엔티티 행렬 37, 43-4, 48, 52, 55, 68, 78, 163, 276

GISwapChain::Present 94, 127, 454

GISwapChain::Present 메서드 127, 454

ore, string 727-8, 730-1, 783-5, 787-8

ion 141-2, 338, 372, 406, 554-5, 586, 594, 596, 639-40, 656

지오

흐릿함 450-1, 456

그레이스케일 339, 359, 585, 691

ge borders 463-5

ge 색상 101

ge 데이터 90, 93-5, 135, 312, 339, 417

ge editors 598, 600-1

ge 파일 317, 323, 340, 342-3, 360, 580

utable 112, 196, 198, 200, 237-8, 245, 257, 481, 491, 609-10, 637

utable 리소스 112, 196

elementation 118, 120, 126-7, 130, 203, 248, 291, 306, 400-1, 450, 456-7, 502, 593, 604, 646

치과 의사 557, 570, 585, 691

R 376-8, 391-2, 394-5

ement 344, 376-7, 390, 394, 527, 789

ex, 선형 420-1

ex 버퍼 저장소 인덱스 200

ex 버퍼 200-2, 210, 237, 242, 245-6, 255-7, 259, 262-3, 427, 434, 513-14, 608, 731, 735

대형 202, 266

굴절의 예 569-70

오프셋 예 255

exing 447, 461

텍스처 추출 및 샘플링 437 매트릭스 팔레트 792

굴절의 예 570

직사각 272, 281, 289, 308, 687, 701, 721

스크린 공간을 사용한 실시간 정보 706  
알리제이션 코드 126-7

UT 어셈블러 151-2, 415, 423, 431, 510, 530, 640

ut 어셈블러 스테이지 150, 152, 201, 210, 414, 424, 511, 530

UT 버퍼 444, 448, 469, 644

ut 요소 512

ut 이미지 453, 456, 461

ut 레이아웃 190, 193-5, 210, 239, 263, 307, 511-12

사용 195

ut 레이아웃 설명 190, 193, 263, 301, 307

ut 매개 변수 193, 204, 401

ut 프리미티브 176, 399, 401-2, 405

UT 레지스터 194

UT 시그니처 193-4, 204-5, 230-2

ut 슬롯 191-2, 198-200, 265, 513, 644

싱글 191-2, 265

ut 텍스처 447, 456, 459

ut 텍스처 좌표 332 ut 텍스처 리소스 435

ut 삼각형 566

ut 버텍스 버퍼 바운드 644  
anced 버퍼 513-14,

527-30

동적 527

161, 254, 495, 509-13, 515, 517, 519, 521, 523, 525, 527, 529-531

춤, 고급 기법 192  
게르 지수 438

사용 438

거 테셀레이션 계수 값 477  
거, 부호 없음 94, 98, 201, 313,

372  
적색광의 밀도 146

변경 43, 55

중간점 485-6

회전 보간 496

텍스처 좌표 보간 404  
원근 평면 벡터 입력 보간 707  
보간

이차원 보간 320, 480, 484, 613, 678

반복 485-6

구형 756, 765

rsection, 가장 가까운 삼각형 540, 546

rsection points 177, 386, 388, 541-2, 703

rsection 테스트 527, 530-1, 539, 546

프러스텀/AABB 525

프러스텀/구체 524

레이/구 541

레이/삼각형 540-2, 546, 705

rsectRayTriangle 540, 542

좌표계 변환 및 좌표계 행렬의 역행렬 78  
정면 평면 524-5

에반트 코드 302-3

243-4, 248, 533, 540-1, 609

부분집합 729, 733

## K

프레임 760-3, 776

첫 번째 761

마지막 761

목록 760-1, 776, 785

텍스처링 주제 90

L  
버트의 코사인 법칙 278-80, 282, 285, 287 e PCF 커널 692-3

rs 364, 620-2, 627  
t  
활성 227  
291-2의 배열  
파란색 146-7, 270-1, 280, 302, 308  
첫 번째 687, 690  
들어오는 271, 278, 284, 496, 702, 722  
키 306

점 287, 299, 302, 306  
컬러 사용 302, 308  
t 색상 270, 302  
t 감도 270, 282, 289-92, 672  
t 혼합 146  
t 프로젝터 670, 672  
t 선 4, 148, 271-2, 278-9, 281, 287-8, 296, 386, 388  
t 수용체 271  
t 산란 280-1

t 소스 90, 144, 183, 270-2, 281, 287, 289, 292, 306, 389-90, 664, 673-4, 676-7, 687-8, 696-7  
t 소스 방출 283-4 t 공간 683, 694-7  
t 공간 깊이 695, 697  
t 타격 271, 273, 279-81, 283, 287, 308  
t 벡터 278-80, 284, 288, 290, 297-8, 594, 672  
t 벡터 목표 287, 296  
tDir 384, 683  
htHelper.fx 296-9, 328, 410, 581-2, 686, 688

팅  
직접 284  
동적 573  
경면 281, 285  
팅 계산 227, 273, 289, 301, 305, 307, 572, 581, 593-4  
팅 데모 299, 301, 308-9  
팅 방정식 147, 270, 281, 289, 301, 307, 496, 557-8, 686, 701, 719  
팅 모델 269, 272, 281, 721

실시간 701  
세그먼트 87, 274, 426, 485-6  
스트립 154, 156, 265, 402, 424  
AR 조합 37, 42, 56, 61, 71-2, 85  
AR 보관 182, 320-1, 323, 332, 623, 627, 756, 758-60  
ar 변환 45, 60-2, 65, 67-8, 70, 72, 83-4, 86, 187

현재 입자 642, 649  
업데이트된 입자 642

kull 데모 305-7, 345  
fogColor 364, 370  
foggedColor 364, 370

d 매쉬 725-6  
d 시간 340, 342

ds 3-element UINT 배열 21 tion, 기본 버텍스 203  
p 219, 250, 604, 682, 777, 779, 785-6  
레이/삼각형 교차 544  
강도 색상 397 er 삼각형 DCB  
626

M  
DLoader, void 728, 730-1, 783-5, 787

nification 319, 321-2, 324, 342, 422, 622  
nitude 4, 6, 9-10, 18, 31, 67, 83, 548, 739-41, 745, 758, 765, 792

, 노멀/덱스 707-8, 710, 715

버텍스 셰이더 출력 204, 208

s 프레임 83, 85

ch 버텍스 요소 204, 208

시리얼 매트릭스 296-8, 728

시리얼 값 285-6, 307

수학적 객체 496, 737 행렬 행 벡터 38

릭스

초기화 50

직교 65, 83, 87

행 460

제공 43-5, 48, 55

부모로 이동 772-3, 791

릭스 추가 39, 743

릭스 대수 2, 37, 39, 41, 43, 45, 47-9, 51, 53, 55, 57

릭스-열 벡터 순서 751 리크 요소 38, 752

릭스 방정식 48, 55, 59, 389, 577, 694, 751, 754

릭스 방정식 Ax 543, 754

릭스 역행렬 48

릭스 행렬 곱셈 59, 72-3, 78, 83

릭스 부식 45

릭스 곱셈 40-2, 44, 48, 55, 58, 68, 72-3, 78, 83, 85, 171, 205, 276, 387, 393

픽셀의 리크 99-100, 450-1

릭스 제품 55-6, 78, 742

릭스 표현 60-2, 65, 67-9, 77, 171, 187, 765

표준 83-4, 86

릭스 유형 50 메모리

런타임 133, 234

비디오 112, 196-7, 430

h

연속 779-81, 791-2

육지 304, 336, 338

두개골 259, 517, 527, 529, 725, 760

h 데이터 731-3, 735, 789

h 기하학 540, 572, 731

h 계층 구조 771, 791

h 인덱스 버퍼 545

h 표면 273, 287, 308

h 삼각형 540, 546, 705

h 버텍스 255

h 뷰어 데모 734 shGeometry 인스턴스 732, 735

sages 126-7, 129-31

ers 4-5, 628, 635

ification 322-4, 342

최대 깊이 버퍼 값 116 최소 점 63, 66, 69

맵 체인 318, 322-3, 342-3, 420, 681

맵 레벨 93, 111-12, 114, 312, 323, 342-3, 419-21, 437-8, 592-3, 638

지도 111-12, 114, 322-3, 420, 638, 678

또는 287, 292, 332-4, 371-2, 378-84, 397, 557

또는 데모 386, 390, 393, 397

또는 픽셀 382-3

가시적 383-4

또는 평면 379 de

디버그 105, 134, 137, 213, 216, 221

전체 화면 108, 110, 131-2

와이어프레임 189, 212, 262, 266, 545, 593, 629

del formats custom 725

표준 725

del 반사 547, 557

delers 145

엔테인즈. 먼 547, 550, 554

티패스 텍스처링 기법 213

곱셈 44, 58, 296, 332, 387, 389, 738-9, 742, 745, 747, 764

tiprocessor 432, 448-9, 467-8

tisampling 89, 91, 93, 101-2, 312, 351, 353

사용 353, 370

## N

e 27, 134, 216, 230-2, 361, 365, 371, 435, 777, 785, 789

상수 선체 셰이더 함수 478 espace std, using 25, 28, 30, 35, 53

C 좌표 171, 669-70, 672, 674

C 공간 171, 178, 186, 522, 530-1, 536-7, 667, 669-72, 687-8, 708, 712, 717

rest ray/triangle intersection 540

-XMVECTOR 매개변수 23

mal/depth 720-1

mal 매핑 93, 312, 495, 571, 573, 575, 577-85, 587, 589, 591, 593, 595, 614, 690, 727

mal maps 339, 571, 573-7, 580-2, 585-7, 591, 593-6, 615, 622, 723

mal 벡터 방향 585, 593

악성 벡터 시각화 기법 426

말 벡터 152, 179, 181, 249, 259, 273-4, 276-9, 304-5, 531-2, 570-1, 573, 577-8, 580-2, 585-6, 615

샘플링됨 581

변형된 276

최적화된 범위 146, 149, 172, 183, 587

텍스처 좌표 변환 438

픽셀당 맵스 593

깊이/스텐실 버퍼 642

픽셀 셰이더 685-6

랜덤 숫자 636, 651, 656

IDIA 하드웨어 432, 467 IDIA 텍스처 도

구 341-2

## O

B (방향성 바운딩 박스) 518-19, 530-2, 620

ect 공간 576-80

ect 공간 좌표 578-80 ect

애니메이션 559-60, 568

블렌드 상태 352-4

블렌딩됨 357-8

cbuffer 206

자식 770, 791

대상 리소스 420

먼 364

효과 219, 409

기하학적 515, 530

글로벌 일루미네이션 모델 조명 272 그룹 96, 143

키 560, 568

다중 206, 246, 254

출력 XMVECTOR 26, 28, 35

배치 623, 627

샘플러 324, 334, 681

반투명 347, 369, 640

단일 776

솔리드 180, 710

상태 212, 218, 307

상태 그룹 212

투명 270, 357

트리 160, 510

루드 180, 381, 383, 698, 702-3, 710-11, 713, 715

    전면 삼각형 180 루딩 포인트, 잠재적 710

환상 703, 705, 707, 710-11, 713, 715, 722

    평균 삼각형 704

et 변환 775, 777-8

    변환 775, 791

벡터 338, 710, 714 *참조*출력 병합기

    스테이지 454, 566, 568, 666, 684, 721

-by-one, 텍스처 애니메이션 프레임 344 도시 148, 183, 356, 621, 638-9, 654

que 356, 361, 370, 381, 639

que 장면 객체 136

rations 2-3, 19, 27, 37, 97, 147, 170, 175, 204, 264, 343, 357, 429, 436, 737

레이더 24, 26, 28, 35, 51-3, 296, 349, 369, 374, 392, 439, 467, 749

    오버로드 24, 27, 51

nted bounding box OBB *참조*

in 5, 76-7, 88, 159, 161-4, 168-9, 498-9, 507, 521-3, 537-8, 546, 548-9, 554, 623-4, 634-5

직교화 13

사각형화 13-14, 17-18

인구 통계학적 예측 663, 666-7, 672-3, 697-9

비정상 집합 13-15

put, 초기화 296-8

버퍼 넣기 443, 447-8, 468-9, 644 제어점 넣기 477

이미지 삽입 456

put merger 151, 182-3, 431

합병기(OM) 배치 103, 110, 114, 150-1, 182-3, 352, 377, 393, 396, 431

put 병합 단계 103, 110, 114, 150, 182, 352, 377, 393, 396

매개변수 204-5, 209, 401

프라이머리 삽입 399, 402, 423

리소스 저장 434-5

서명 저장 228, 230-2

텍스처 넣기 466

삼각형 리스트 삽입 402

정점 속성 329, 367, 470, 511, 583, 589, 611, 689, 706, 782 설정 정점 버퍼 바인딩 644 설정

창 105, 134, 137-8, 267 설정

XMVECTOR 53을 넣으세요

P

광선 287, 389-90, 673

동일한 광원 287-9, 389-90, 666, 673, 697

allelogram 34-5, 57, 72, 86-7

통과 미터 22, 32, 296 미터

    감쇠 270, 289

    첫 번째 110-11, 115-16, 128, 210, 220, 325, 377, 638, 641

    두 번째 111, 114-16, 201, 210, 220, 325, 350, 638, 641, 681

아이클 시스템 프레임워크 640, 645, 647

아이클 시스템 90, 260, 429, 495-6, 631, 633, 635, 637-43, 645, 647-53, 655, 657-61

아이클 VIN 652, 654, 658-9

아이클

지오메트리 셰이더 입력 642

지오메트리 셰이더 생성/파괴/업데이트 661 리빙 645, 661

새로 만들기 647, 652, 658

구조체 447, 633, 652, 657

위치 지정 476-7, 483, 493, 591

s 버텍스 컬러 203-5, 209-10, 214

XMVECTOR 매개 변수 사용 22, 32 ch

607-8의 그리드

테셀레이션 479, 493, 591

ch 제어점 473, 610, 616 ch 정점, 테셀레이션 493

sed 시간 120-3

121의 추적

(퍼센티지 클로저 필터링) 678, 680-2, 686, 692-3, 695, 698-9

커널 682, 693-4

-버텍스 재료 307

퍼센트 클로저 필터링 참조 PCF 성능, 피크 400

성능 카운터 89, 116-19, 136

퍼포먼스 타이머 116-17, 136

투영 투영 166, 627, 666-7, 672-3, 698-9

투영 행렬 173-4, 184-6, 507, 698

킹 6, 71, 495, 520, 533-7, 539-41, 543, 545-6, 698, 732, 735

킹 레이 533, 535-9, 541, 546

g-ponging 버텍스 버퍼 644

엘라인, 렌더링 89-90, 135, 141, 149-53, 155-7, 163-5, 181-3, 189, 313, 430-1, 454-5, 471-3, 492, 529-30, 641-2

엘라인 스테이지 98-9, 104, 110, 313-14 1 블록 컬러

100

l center 100-1, 422 l color

최종 150

lit 558

해결됨 100

l 색상 178, 265

객체 357

l 형식 89, 108

l 조각 182, 207, 264, 357, 371, 375-6, 392, 394, 396-7

l 가장 가까운 674

l Pij 450-1

l 셰이더 89-90, 150-1, 182-3, 203-5, 207-10, 213-14, 226, 263-4, 318-19, 361-2, 396-7, 422-4, 613-15, 686, 707-8

비싼 396

짧은 724

단순한 208

l 셰이더 알파 422 l 셰이더 바

디 415

l 셰이더 입력 207, 209

l 셰이더 입력 매개 변수 208 l 셰이더 명령어

362

l 셰이더 수정 208, 396

l 셰이더 단계 150, 182, 396, 399, 414-15, 423-4 ls

변경 693

클리핑 361-2

현재 361, 370, 707

버리기 362, 367, 412, 584, 690

예지 422, 604

추가 464

ijth 348, 371, 394, 450, 604

|                  |                                                                     |
|------------------|---------------------------------------------------------------------|
| less             | 457                                                                 |
| 이웃               | 603-4, 718                                                          |
| 인접한              | 100, 461, 604, 693                                                  |
| 불투명한             | 422                                                                 |
| 특정               | 95, 373, 392                                                        |
| 거절하다             | 373, 375, 392                                                       |
| 렌더링              | 361, 370                                                            |
| 단일               | 395, 632                                                            |
| 시간 처리            | 394                                                                 |
| 투명한              | 370, 685                                                            |
| 가시성              | 380, 383, 673-4, 698, 710, 720-1                                    |
| IShader          | 230, 232 ne                                                         |
| 바닥               | 521 복합 739-40                                                       |
| ne 방정식           | 521-2, 530-1, 616                                                   |
| 616              |                                                                     |
| yer              | 4, 116, 120-1, 455, 623, 626-7, 791                                 |
| 삼각형 및 캐틀-클라크 근사법 | 493 삼각형 스킴 476                                                      |
| nt 필터링           | 322-4, 678                                                          |
| nt 광원            | 288, 296, 388-9                                                     |
| nt 조명            | 90, 270, 288-91, 302-3, 308, 388-90, 699                            |
| nt 목록            | 153-4, 265, 402                                                     |
| nt 프리미티브         | 407, 426, 632                                                       |
| nt/백터            | 70, 74, 765                                                         |
| nts, 소멸          | 142-3, 166, 183                                                     |
| ar 표현            | 740-1, 746-7, 765                                                   |
| 다각형 기울기          | 677 다각형, 블록                                                         |
| 177 ition        |                                                                     |
| 주어진              | 563                                                                 |
| 초기               | 634-5, 645, 661                                                     |
| 입자               | 260, 447                                                            |
| 표준               | 5-6, 18, 31, 85                                                     |
| 버텍스 블렌딩          | 781                                                                 |
| 이온화 요소           | 191, 265, 397                                                       |
| 이동 함수            | 634                                                                 |
| 위치 백터            | 18, 67, 72, 83, 425, 506                                            |
| 이전 버텍스 셰이더       | 205-6, 782                                                          |
| 미티브 ID 매개변수      | 414-15, 424                                                         |
| 미티브 ID           | 90, 414-16, 424, 426-7, 647                                         |
| 미티브 토폴로지         | 152-3                                                               |
| 지오메트리 셰이더 출력     | 640 블렌, 깊이 버퍼 정밀도 173 덕                                             |
| 트, 백터-매트릭스       | 56                                                                  |
| 그램 지오메트리 셰이더     | 90, 400 그램 지오메트리 셰이                                                 |
| 더 400 정점 객선      | 253, 672                                                            |
| 투영               | 166-70, 172, 182, 206, 390, 507, 522, 533-4, 537, 666-7, 669, 697   |
| 투영 프러스텀          | 522-3                                                               |
| 투영 행렬            | 127, 171-2, 174, 185-6, 206-7, 235, 501, 505, 522, 538, 669-70, 682 |
| 직교               | 668-9, 698                                                          |
| 투영 평면            | 167, 169, 667, 670, 697                                             |
| 투사 변환            | 171, 206                                                            |
| 투영 창             | 166-72, 182, 395, 533-5, 537, 546, 669-70                           |
| 주관적 텍스처링         | 669, 697                                                            |
| 사용               | 674, 688 참조픽셀                                                       |
| 셰이더              |                                                                     |

Q

d 패치 90, 474-5, 479-81, 491, 493, 609

입방 배지어 484 d 패치 버텍

스 608 d 버텍스 660, 708

정밀도 수준 101-2, 108, 112, 125, 223-6

벡터 값을 가진 엔티티 4-5

삼원 곱셈 743-5, 765

삼원자 회전 연산자 749, 751, 765

삼원 공간 765

3원자 496, 519, 737-9, 741-5, 747, 749, 751, 753, 755-61, 763-7, 785

보간된 753, 756

0이 아닌 745, 765

ryPerformanceCounter 117, 119-20, 122

사용 117

ryPerformanceCounter 함수 117, 136

## R

입자 645, 655 dF, 정적 부동 소

수점 636

DOM 벡터 638, 709, 714

domValues 637

dUnitVec3 636, 638, 652-3, 657

ge 매개 변수 289

에리제이션, 깊이/스텐실 버퍼 비활성화 641 W 형식 601

캐스팅 702-3, 706, 709, 722

방향 538, 541-2

intersects 533, 540-2, 546

미스 540-2, 546

원점 538, 541-2, 703

삼각형 교차점 516, 542, 705

숫자 7, 30, 40, 45, 55, 737-8, 744-5, 753, 759, 765

33의 성질

부분 738, 742, 748, 764, 766

건축 213, 457, 500, 505

각도 88, 90, 178, 319, 616

가위 268

조명 146-7, 271-2, 302, 308, 557

참조 장치 104-5, 125

참조 벡터 64

참조 정점 184, 729

인덱턴스 281-2, 285, 290

절단 182, 282, 284, 287, 371-2, 378-81, 383-5, 557-8, 560, 562, 570, 596

동적 455, 560

경면 281-2

절단 벡터 281-2, 284, 557-9 절단 벡터 픽셀당 557

해석 94, 99, 110, 134, 220, 235-6, 257-8, 262, 313, 373, 444, 454, 528, 564-5, 721

텍스처에 대한 해석 450, 454, 664, 673

더링 효과 152, 212, 223, 263-4

더링 패스 219, 264, 661

초 720

더링 장면 깊이 664 표현, 축-각도 회전 759 해상도 텍스

처 321-2

소스, 단일 135

소스 바인딩 230-1

소스 형식 99, 314 ult, 짧은 거리 422

m 코드 134

B 구성 요소 349, 369

BA 색상 벡터 352

t-handed 좌표계 6-7, 15

전환 행렬 65, 70, 83

변환 행렬 72, 84, 87, 161, 519, 751, 759-60, 765-6

일반 84, 87

변환 71, 87

벡터-행렬 곱셈 742

벡터 40-2, 50, 65, 68, 71, 83, 162-3

S

ple

경계 463-5

무작위 709, 715-16

스레드 449-50

플레 코드 102, 118, 124

플레 카운트 102, 711

ple 프레임워크 108, 137, 423 ple 프레임워크

코드 91

ple 인접 값 694-5 ple 포인트, 무작위 707, 709-10

ple 스크린샷 394, 396

ple 텍스처 329, 362, 367, 412, 437, 584, 690

pler 324-5, 638, 671, 724

비교 681

pler 상태 325, 467

플링 텍스처 324, 437, 449, 680

lar 곱셈 7-8, 24, 28, 32-3, 39-40, 147-8, 744, 765

링 2-단위 88

링 행렬 62, 72, 81, 84-6, 161

변환 62, 277

전체 94, 136, 394, 554, 567-8, 673, 682

ne 깊이 664-5, 673-4, 698

ne geometry 246, 669

ne 조명 158, 307, 705

ne 객체 136, 272, 302, 527, 533, 539

네 버텍스 163, 498

en 노멀/깊이 텍스처 724 en 픽셀 322, 675

en point, clicked 533-5, 546 en point ps 536

en quad 455, 707-8

en 해상도 100-1, 112, 457, 707, 724

en space 181-2, 533, 535-6, 694-5, 697, 702, 707

en space ambient occlusion SSAO *참조* en space triangle

density 627

der

컴파일됨 216, 218, 239

특화 362, 368

버텍스/픽셀 706

코드 216, 223, 225, 289, 362, 366-7, 412, 448, 459, 463, 468, 584, 636, 645, 673

지형 픽셀 622

der 함수, 상수 선체 612, 627 der 생성기 메커니즘 226-7

der 입력 193, 665, 697, 719

제어점 선체 476, 493, 591

der 프로그램 212, 215, 220, 263, 284, 336, 455, 460, 580, 605, 762

der resource 98-9, 113, 313-14, 317-18, 327, 417, 430, 435, 553, 561-2, 568, 605, 607, 627, 665

해당하는 317-18

변형 225-6, 264

des 146, 148, 213-14, 494

평면 하강 90, 385-6, 388, 390, 496, 697

다우 여드름 674-5, 698-9

다우 지수 387, 389

다우 예지 681-2, 698

다우 팩터 686-7

다우 지도 664-6, 673-8, 681-2, 684-6, 688, 691, 693-9

다우 지도 데모 691-2

다우 맵 샘플 675, 679, 681-2

다우 맵 저장소 674, 697-8

다우 맵 테스트 675, 678, 688, 693-4, 696, 699

다우 맵 텍셀 674-5, 698 다우 맵 텍스처 공간

688 다우 맵 값 677, 679

다우 매핑 455, 495-6, 663-5, 667, 669, 671, 673, 675, 677, 679, 681-3, 685, 687, 689, 697

다우 매핑 알고리즘, 기본 663 다우 행렬 388, 390, 392

일반 390, 393

다우 평면 386, 388

다우 기법 223-4, 663

실시간 496, 697

레드 메모리 431-2, 448-50, 461-2, 468, 716

레드 메모리 어레이 449

res 101, 212, 275-6, 422, 550, 578-9, 581, 590, 703-4, 734

AR 변환 187

ll 입자 647, 661-2

D 19-22, 27, 32, 50-1, 432, 467

D 지침 19-20

, 전파면 432, 434, 467

d 삼각형 403

equal 252

정점 774

메쉬 데모 776, 790, 792

ll 145, 379-81, 383-4, 393, 397, 529, 705, 720, 760, 763-4

반사 372, 379, 381-3

es 247, 249-51

pe-scaled-bias 677, 698-9

AP 679-80, 686-7, 694, 698

ke 입자 639-40, 645

ke 텍스처 647

rce, 평행 광원 287, 386-7

rce 알파 구성 요소 356, 358, 621

rce 블렌드 팩터 348, 355-6

rce 데이터 420

rce 이미지 318, 340, 450

rce 픽셀 347-8, 354-7, 361, 369 ce

셀 623-4

homogeneous 177-8, 215, 388

homogenous 522-3

양의 절반 177, 521

부피 142, 166-7

ce 좌표계 499, 666

ce 좌표 500, 668

ce 깊이 706, 723

ce 노멀 706-7, 720

ce 위치 708, 710, 714

ce z축 501, 666-7, 697

ceship 144

DXGI 지정 108

텍스처 좌표 지정 326 구형 281, 292-3, 558, 727

특수 조명 270, 281-4, 308

|                                                                              |
|------------------------------------------------------------------------------|
| 양 281-3, 285, 292                                                            |
| 특정 용어 296-8, 719-20 ere                                                      |
| 조명 144, 269-70                                                               |
| 조명이 켜지지 않음 144, 269-70                                                       |
| 교차점 524                                                                      |
| 에어 정점 박스 정점 실린더 정점 202 tlight 90, 270, 290-2, 298-9, 302-3, 308, 672, 698-9  |
| tlight 원뿔 290-2, 672-3, 698                                                  |
| tlight factor 291, 298                                                       |
| are 63, 66, 69, 72, 84, 86, 162, 451, 549, 554                               |
| 행렬 48-9, 55                                                                  |
| C 350, 353-4, 356, 638, 651                                                  |
| 72, 161-2                                                                    |
| V 419, 435-6, 440-2, 456-7, 467-8, 562, 606, 638, 665                        |
| O (스크린 공간 앰비언트 오클루전) 455, 702, 706-7, 711, 715-16, 719-24                    |
| O 맵 707-8, 711, 715, 719-22, 724                                             |
| O 픽셀 셰이더 707-8                                                               |
| 2 19-20, 26, 28, 30, 36, 53                                                  |
| ks 247-51, 420, 666                                                          |
| ge, 클리핑 183, 526, 529-30, 615                                                |
| ncil 90, 99, 371, 373                                                        |
| ncil 버퍼 90, 98, 130, 134, 210, 371-3, 375-6, 379-83, 386, 390-5              |
| 8비트 376-7                                                                    |
| 업데이트됨 380                                                                    |
| ncil 버퍼 항목 376-7, 380, 390                                                   |
| 해당 381, 391, 394 ncil 버퍼 작업 395                                              |
| ncil 버퍼 픽셀 380, 390                                                          |
| ncil 버퍼 테스트 182, 207                                                         |
| 이중 블렌딩을 방지하기 위한 ncil 버퍼 390 ncil 버퍼 값 390                                    |
| ncil 테스트 210, 373-7, 380-1, 390-2                                            |
| kes 146-7, 270-2, 278, 281, 308, 673, 701                                    |
| ng 190, 216, 478, 641, 777, 787-8                                            |
| ps 154-5, 401-2, 405                                                         |
| cture                                                                        |
| 출력 정점 유형 400                                                                 |
| 특수 리소스 195, 200, 263 구조화된 버퍼 리소스                                             |
| 439                                                                          |
| 구성 버퍼 197, 439-40, 442-3, 447-8, 467-8                                       |
| 나누기 252-3, 402, 405, 425, 586, 611                                           |
| 픽셀 100-1, 112, 422                                                           |
| 가시성 100-1                                                                    |
| 직사각형 115-16, 139, 267                                                        |
| 재샘플링 100-1                                                                   |
| 면, 부드럽게 281, 284, 476, 493, 578                                              |
| 면 색상 301, 307                                                                |
| 면 법선 273, 307-8                                                              |
| 지정 571                                                                       |
| 면 점 278, 282, 285, 304-5, 307, 329, 362, 364-7, 370, 412, 554, 583, 672, 690 |
| 면 텍스처 476                                                                    |
| AP 103, 107-9, 132                                                           |
| p chain 94, 103, 107-10, 125, 136, 454                                       |
| tch 108, 131, 137, 212, 353, 362, 432, 449, 455-6, 467-8, 716                |
| 템                                                                            |
| 비 입자 655                                                                     |
| 삼점 조명 306                                                                    |
| 템 메모리 112, 196, 242, 261, 417, 430, 442-3, 540, 732, 735                     |
| 템 메모리 버퍼 442-3                                                               |
| tem 메모리 복사 514, 520, 540, 732, 735                                           |

tem 메모리 리소스 442 tem 값 SV 477, 566, 569  
tem 값 204, 468, 566

# T

gent 576, 578, 580, 614-15, 730, 781, 783

젠트 평면 273, 304, 307, 694

젠트 공간 577-81, 594, 614

벡터 276, 304-5, 326, 487, 576, 578-9, 581-2, 614-15, 694

정점별 581 변환된 276

et 배열 슬라이스 566, 569  
et point 165-6

N-베이스 576, 579-81, 593

기법

고급 93, 115, 176, 312, 352

빌보드 423

다중 패스 213, 264

다중 텍스처링 496

다중 223

텍스처로 렌더링 454

렌더링 189, 213, 264, 641-2, 648, 650, 661

기법 객체 219

에이전트 551, 569, 600-1, 628

메인 213, 240, 246, 299, 347, 495-6, 556, 597-603, 605-9, 611, 613, 615, 617, 619-23, 625-9  
void 601, 605, 608-9, 617

주요 색상, 현재 621 주요 데모 628

ain 그리드, 테셀레이션 607 ain 레이어

620

AIN 메쉬 지오메트리 623, 627

AIN 표면 304, 597, 599, 623-4, 626-8

AIN 테셀레이션 606, 627

AIN 버텍스 623, 627

AIN 버텍스 셰이더 610  
ellate 90, 471-2, 474-6, 478, 481, 483, 492-3, 572, 585, 588-9, 593, 607, 627, 686, 721

연결된 기하학 686, 721

연결 253, 425, 474-6, 481, 483, 493

연결 90, 175, 251-2, 264, 471-3, 475-81, 483-4, 492-3, 496, 571, 585, 587-9, 592, 611, 613

분수 477, 493

분리 인자 474-9, 481, 492-3, 588-91, 608, 611-12, 615, 627, 629

내부 475, 478-9, 590-1, 611

엘레이션 단계 90, 156, 175, 183, 399, 423, 471-9, 481, 483, 485, 487, 489, 491-3, 526, 530

엘레이터 480, 484, 492, 572, 592

레이/삼각형 540

가위 268

scene 397, 426

좌표 248, 469, 713, 730, 783

l 점 319-21

l 값 461-2

삼각형 헤더 731

정점 헤더 730, 783

ure alpha 362, 367, 412, 584, 690

애니메이션 311, 338

유레 어레이 90, 112, 400, 414-21, 424, 510-11, 548, 553, 560-1, 568-9, 622, 647, 651, 656

전체 565-6

ure 배열 인덱스 420

어레이 저장소 415, 424

ure 아티스트 316, 587

ure atlas 316, 319, 344

ure 버퍼 94, 456

ure 색상 323, 325, 639

ure 좌표 오프셋 값 637 ure 좌표계 314-15, 697

URE 좌표 변환 336 URE 좌표

압축된 575

해당하는 316, 577

생성된 669

생성 투영 669, 671

보간된 320, 326

정규화 672

정규화된 320, 438

쌍 316, 320

투영적 669, 671-2, 678, 697, 710, 719

각각 315, 345

변형 332

무한한 636

유일한 320

사용 438

ure 데이터 317, 327, 551, 639

유레 치수 314, 318

유레 왜곡 316, 578

ure 요소 415-18, 467

ure 파일 341, 576

ure 필터링 548, 622

ure 형식 101-2

ure 함수 332-3

ure 기능 262, 697

ure 기하학 436

ure gOutput 437

ure 이미지 437, 572, 574

전체 316, 326

ure map 316, 319, 322, 325, 369, 573, 575, 615, 637, 709

랜덤 벡터 710

요소 매핑 90, 286, 311, 316, 571, 576

우레 매트릭스 336, 671

물의 338

ure 객체 312, 318, 344

ure 출력 및 무순서 액세스 435 ure 평면 332, 335, 338, 574, 577

ure 해상도 333, 336

ure 리소스 98, 150, 260, 264, 313, 317-19, 434, 467, 734

ure 샘플 449, 680

감소 454

크기 438

우레 공간 315-16, 337, 576-9, 593, 669-71, 683

우레 공간 축 576-7

우레 공간 좌표 337 우레 공간 평면 338

유레 공간 v축 614 유레 저장소 93, 312, 415

ure 하위 리소스 420

ure 타일링 90, 338

URE 변환 671

ure 변환 행렬 338, 594 ure 변환 596

애니메이션 90

ure 삼각형 316, 576

요소 값 322, 449 요소 점점 316

uring 90, 112, 152, 241, 305, 322-3, 328, 331, 368, 548, 556, 602, 647, 651, 656

우링 좌표계 576 광고 그림 크기 432, 445, 459

광고 그림 432-4, 437, 443, 445-6, 448-9, 456-61, 464, 467-8

광고 ID 값 447

광고 ID 431, 437, 445-7, 468

광고 430, 432-4, 437, 443, 445-50, 458-62, 464-5, 467-8

외부 요소 458, 462, 466

메인 렌더링 106-7

작업자 106-7

mb 포인트 6-7, 16

116-23, 129-30, 136-7, 160-1, 222, 225-6, 259-61, 565-9, 633-5, 637, 643-7, 661, 760-1, 776-7, 785-6

현재 120-2

121, 266, 335-6, 338, 344, 425, 771의 함수

초기화 99, 212, 218, 254, 260, 313, 322, 325, 327, 420

실수 738, 744

총계 120-1, 123

구간 116, 137

위치 764, 785, 789-90

r 116, 121-2, 130

ar-평면 벡터 708

보간된 707-8

아렌트 변환 773, 791

sform 69-70, 178-9, 203-6, 209-10, 299-300, 329, 332, 366, 404, 527-8, 580-1, 593-4, 771-8, 781-2, 791

최종 776, 778-9

뿌리까지 775-6, 779

좌표 변형 2, 579

sform 기하학 772

sform quad 정점 412, 655

변환 행렬 59, 83, 158

구성 59

여러 세계 생성 246

변환 행렬 70, 80, 83, 86, 276, 336, 504, 546, 762, 772

아핀 68, 71, 185-6

네트 59, 72

단일 84

부모로 설정 773, 791

변환 아핀 80

변형 1-2, 59, 61, 63, 65, 67, 69-75, 77-88, 158-9, 164, 167, 171-3, 187, 277-8, 771-2

활성 80-1, 83

본체 71, 576

좌표계 변환 499, 770

최종 775, 780, 792

정체성 68

역전 164, 498

회전 텍스처 594

시퀀스 161-3

텍스처 공간 670

루트에 대한 775, 791-2

형성된 법선 벡터 n A 276 형성된 점 71-2, 781 형

성된 접선 벡터 u A 276 관련 276

정규화 노멀 벡터 276 정규화 노멀 781-2

위치, 부드러운 285, 308, 422, 597

변위 2, 37, 59, 67-8, 70-1, 73-4, 80-2, 85, 88, 164, 187, 278, 624-5, 761, 763

변환 행렬 69, 72, 81, 84-5, 87, 162, 187, 278

변환 68, 81

투명도 90, 180, 182, 270, 348, 356, 381, 384, 386, 390, 422, 640, 729

투명도 알파 620-1

spose 37, 43, 47, 54-6, 58, 65, 83, 580, 742, 751  
725, 769, 773, 775, 778-9  
빌보드 데모 408-9, 422-3  
s 143, 405, 416, 422-3, 727, 775, 778  
ngle centroid 591  
ngle clipping 475  
ngle count 90, 471, 481  
    낮음 175  
밀도 247, 627  
각도 중복 157  
ngle grid 469, 598  
ngle grid mesh 240, 260 ngle grid mesh  
vertices 260 ngle GS 566  
ngle 인덱스 730  
ngle 입력 425  
ngle 목록 145, 153, 155-7, 176, 265, 399, 402  
    사용 153, 156  
ngle mesh 145, 183, 273-5, 285, 287, 307-8, 473, 476, 541, 546, 587, 623, 627  
    임의 308, 579  
    임의 사용 581  
각도 메쉬 근사 145  
앵글 패치 473, 475-6, 480, 493, 588  
    3차 베지어 476 3차 베지어의 차수  
    476  
각 패치 구성 475  
각 패치 테셀레이션 예시 479 각 평면 586  
각도 공유 276  
앵글 스트립 154-5, 157, 265, 401-2, 412, 655  
    사용 153  
ngle strip 정점 411, 654  
ngle 세분화 479  
삼각형으로의 각도 변환 577  
각도 토폴로지 262 각도 정점 542  
각 꼭지점 179, 425, 577-8 각도  
    예각 316  
    추가 586  
    인접한 155-6, 591  
    후면 180, 211  
    바닥 590  
    클립 처리된 177, 530  
    해당하는 315  
    폐기물 526, 530  
    추가 175  
    맞대기 376-7  
    편평하게 390  
    정면 배치 211  
    그룹 728, 735  
    ith 275, 567  
    왼쪽 179  
    하단 624-5  
    매핑됨 576  
    인접 156  
    새로운 175, 483  
    보통 476  
    겹치는 390  
    선택된 540, 544-5  
    오른쪽 179

직각 316

조각 250

세분화 402

테셀레이션 475

상단 405, 590

이동 425

상부 624-5

사용 252

ngles form 145

버퍼링 94, 108

아이얼 액세스 기능 126

94, 99, 111, 114, 313-14, 665, 681

무제한 형식 94, 99, 135, 313-14

es CXMVECTOR 22

U

벡터-행렬 곱 42

V (무순서 액세스 뷰) 435-6, 440-2, 448, 456-7, 467-8

방향 벡터 635

길이 10, 12-14, 65, 83, 87, 249, 278, 305, 425, 504, 531, 541-2, 578-9, 723, 753

쿼터니언 496, 519, 522, 737, 745-7, 749-50, 753, 755, 757-8, 760, 765-6

구면 252-3, 404, 425, 636, 638, 652, 657, 753-4, 756-9

정렬된 접근 뷰 참조: UAV ORM 깊이 버퍼 형식 677,

ORM 형식 109, 138, 340-1, 678

oll 330, 367, 404-5, 412-13, 464-5, 584, 655, 687, 690, 695, 714

ating 112, 196, 207, 262, 408, 420, 645

삼각형 ABC 626

V

버텍스 버퍼 사용 644

d 깊이/스텐실 버퍼 형식 392 eys 246, 260, 597-8, 627

ues

근사 320

감소된 377

증가된 376-7

최대 376-7

무작위 637

스텐실 383-4

able mPickedTriangle 544

ables, 두 번째 상수 버퍼는 장면 빛을 저장함 207 tor 추가 7-8, 19, 24, 28, 32, 739

기하학적 해석 8, 625

토르 좌표 6, 67, 215

조회 551

토르 내적 함수 296 토르 함수 26

토르 행렬 곱셈 37, 42, 61, 72-3, 78, 205

토어 작업 18, 26, 32-3, 147, 183

토어 파트 747-8

상상 742, 744, 764

tor/point 76, 83

토어 속성 33, 67

tor 반사 함수 296 tor 뿔셈 8, 28

토르

32비트 부동 소수점 191

이접선 579

열 38, 40-1

좌표 61

변위 68, 71

분산 709-10

첫 번째 15-16, 295

사건 284, 569-70 들어오는 광선 570 i

번째 열 543, 754

j번째 열 40, 55

조회 548-9, 554, 557, 568

정규화 30, 448

직교 12, 17, 34 직교 집합 13-14 직교 정규 18, 507

직교 정규 축 531

반환됨 16, 27

오른쪽 502-4, 507

부호 있는 정수 191

텍스처 좌표 336

변환 278, 539

변환 82, 580, 751

변환 67, 785

무부호 정수 191

사용 18-19

토러스 및 좌표계 5

도시 2, 4, 447, 634-5, 641, 645, 652, 658, 661

ex

임의 275, 578-9

첫 번째 200, 203, 255

생성된 608, 613

ijth 337, 598, 603, 627

마지막 249-50

중간 275

객체의 160

특정 598, 627

좌상단 623-4

ex 배열 156, 255

ex 속성 181, 204, 705-6

ex basis 263, 285

예: 혼합 가중치 783

예: 버퍼 저장 481, 491

ex 버퍼 152-3, 156, 195-202, 210, 237, 242, 255-6, 261-3, 265, 513, 520, 640-5, 647-9, 661

199, 643 배열

연결된 255

해당하는 199

해당 로컬 201-2

동적 260, 262, 408, 645

전역 202-3

불변 197

초기화 647-8

ith 199

large 202, 266

특수 초기화 647

바인딩 시작 199

정적 408

using 199

using dynamic 262

ex 버퍼의 메모리 200

ex colors 197, 270, 285, 301

ex component 192, 263

ex 데이터 153, 156-7, 176, 190-1, 199, 242, 263, 326, 511-13, 643

예: 깊이 값 181

예제 요소 191, 204, 208, 244, 256, 512

ex 형식 152, 190, 307, 641, 730

사용자 정의 190, 263

ex 높이의 변경 262

ex ID 415, 424, 426-7

ex/index 버퍼 156, 261

정적 540

ex 인덱스 값 415, 424, 426-7

ex 입력 레이아웃 239, 644

ex 레벨 287, 301, 571

ex 조명 계산 273

예제 목록 157-8, 184, 193, 399, 520, 640, 644, 661

ex 노말 273-4, 276, 287, 305, 308, 426, 494, 578

예: 점 273-4, 305, 308

이전 위치 176, 204-5, 399, 480, 494, 586, 613, 682

첫 번째 구체 203

테셀레이션 479, 613

ex 셰이더, 간단함 203

ex 셰이더 액트 479, 493

ex 셰이더 바디 415

ex shader fragment 781

ex 셰이더     함수     158 ex 셰이더 입력 매개 변수 204

ex 셰이더 입력 시그니처 190, 193

ex 셰이더 출력 209

ex 셰이더 매개 변수 204, 208

ex 셰이더 시그니처 415, 424

ex 셰이더 단계 158

ex 셰이더 구조 순서, 대응 267 ex 셰이더, 논 213

구 구조 158, 190-4, 204, 223, 225, 263-4, 266-7, 285, 316, 408, 519, 579, 633, 780

사용자 정의 193, 204

지형 패치 608

ex 구조 순서 267

텍스처 좌표 316, 320, 345

ex type 198, 401

ex 정점 198, 237

이칼 필드 168-9, 174, 184-5, 187, 502, 522, 537

아이스

배열 195, 275, 401, 520

중복 157

생성된 90, 471, 481, 592, 607, 627

목록 152, 399, 401-2 최대 수 400 새 252, 262, 403, 424, 477

객체의 386, 388

발신 401, 414

테셀레이션 586

유일한 157-8, 247

버텍스 셰이더 입력 399 아이스 및 삼각형 인

덱스 730 아이스 이동 613, 615

아이스 출력 402, 640, 661

wer 94-8, 136, 142-3, 163, 179, 181, 271-2, 303, 423

wpoint 179, 280-1, 550, 563, 664-5, 673, 675

wport 103, 115-16, 125, 178-9, 267, 535, 562, 564, 664, 666, 721

wport 매트릭스 535-6

ual camera 18, 89, 141, 150, 163, 183

, 벡터-행렬 곱 82

W

좌표, 음수 388

1 272, 281, 320, 322, 335, 358, 372, 379-80, 383, 393

er 240, 303, 305, 336, 347, 369-70, 469, 570

기하학 336, 338

er 픽셀 347, 370

er texture 336, 338

es 260, 262, 303, 455, 469, 594-6

es 데모 261-2, 299, 336

es 버텍스 셰이더 469

ghts 85, 275, 366, 451-3, 558, 715, 718, 753, 780-2, 784

딩 순서 155-6, 181, 384, 477

프레임 211, 215, 425, 586

k

공통 226-7

계산 449-50

특화된 263-4

ld, 결합 206-7

ld 좌표계 160, 163-4, 184, 499, 507, 772, 774

ld 좌표 499, 507, 580, 632

ld 행렬 254, 514

ld 행렬 159, 161-4, 206, 246, 254, 388, 407, 499, 510-12, 530, 539, 577, 580, 654

두개골의 764

ld 공간 축 507, 549-50

ld 공간 좌표 163, 549, 580 ld 공간 부동 소수점 215

ld 변환 159-60, 163, 388, 511, 520, 773

ld 변환 행렬 266

ld-view-projection 행렬, 결합 220 ld y-축 165, 503

## X

FLOAT 21, 52, 295

FLOAT2 20-1, 51, 190-2, 208, 242-3, 264, 316, 408, 440, 443-4, 513, 519, 579, 608, 617

FLOAT3 20-1, 190-4, 198, 237, 241-4, 253, 291-2, 302-3, 481, 491, 500-1, 519-20, 563-4, 646, 763

FLOAT4 20-1, 35, 51, 53, 190, 193-4, 244-5, 286-7, 291-2, 302, 381, 391, 522-3, 637, 708-9

MATRIX toRoot 778-9

MATRIX ViewProj 257, 500, 506

MATRIX world 220, 236, 258, 383, 392

MatrixAffineTransformation 761-2

MatrixDeterminant 53, 277, 528

MatrixIdentity 53, 234, 254

MatrixInverse 53, 277, 523, 528, 539

MatrixLookAtLH 166, 235, 683

MatrixMultiply 52, 254, 528, 539, 778-9

MatrixPerspectiveFovLH 174, 235, 457, 502

MatrixRotationAxis 함수 503

MatrixRotationY 503, 683

MatrixScaling 254, 338-9

MatrixTranslation 254-5, 339, 392

QuaternionRotationAxis 759, 763

StoreFloat 26, 28, 35, 53, 249, 253-4, 303, 305, 384, 503-4, 518, 616, 683, 710, 763

StoreFloat4x4 234-5, 254-5, 338-9, 457, 502, 683, 761-2, 778-9

A collision library 517, 519, 521-2, 524, 526, 541

A collision library functions 519-20, 541

수학 라이브러리 20-1, 25-6, 50, 81, 759

수학 라이브러리 1, 19-20, 24, 30-2, 52, 59, 148-50, 166, 183, 390, 531, 759

A 수학 도서관의 벡터 함수 3

수학 미지원 26, 28, 30, 36, 53

A math 벡터 함수 148, 183 A math XMVECTOR 를

래스 32

A 모델 클래스 725

# 목차

|                                 |    |
|---------------------------------|----|
| 하프 타이틀                          | 2  |
| 라이선스                            | 5  |
| 표지                              | 3  |
| 저작권                             | 4  |
| 헌사                              | 6  |
| 목차                              | 7  |
| 감사의 글                           | 18 |
| 서론                              | 19 |
| 대상 독자                           | 19 |
| 필수 조건                           | 19 |
| 필수 개발 도구 및 하드웨어                 | 19 |
| D3DX 라이브러리 사용                   | 20 |
| DirectX SDK 문서 및 SDK 샘플 사용      | 20 |
| 명확성                             | 22 |
| 샘플 프로그램 및 온라인 보충 자료             | 22 |
| Visual Studio 2010에서 데모 프로젝트 설정 | 23 |
| Win32 프로젝트 만들기                  | 23 |
| DirectX 라이브러리 연결                | 25 |
| 검색 경로 설정                        | 27 |
| 소스 코드 추가 및 프로젝트 빌드              | 28 |
| 제1부 수학적 선행 지식                   | 32 |
| 제1장 벡터 대수학                      | 33 |
| 1.1 벡터                          | 33 |
| 1.1.1 벡터와 좌표계                   | 34 |
| 1.1.2 왼손 좌표계 대 오른손 좌표계          | 35 |
| 1.1.3 기본 벡터 연산                  | 36 |
| 1.2 길이 및 단위 벡터                  | 37 |
| 1.3 내적                          | 38 |
| 1.3.1 직교화                       | 40 |
| 1.4 외적                          | 43 |
| 1.4.1 의사 2차원 외적                 | 44 |
| 1.4.2 외적에 의한 직교화                | 44 |
| 1.5 점                           | 45 |
| 1.6 XNA 수학 벡터                   | 47 |
| 1.6.1 벡터 유형                     | 47 |
| 1.6.2 로딩 및 저장 방법                | 48 |
| 1.6.3 매개변수 전달                   | 49 |
| 1.6.4 상수 벡터                     | 49 |
| 1.6.5 중복 정의된 연산자                | 50 |
| 1.6.6 기타                        | 51 |

|       |                    |    |
|-------|--------------------|----|
| 1.6.7 | 세터 함수              | 51 |
| 1.6.8 | 벡터 함수              | 52 |
| 1.6.9 | 부동 소수점 오류          | 56 |
| 1.7   | 요약                 | 57 |
| 1.8   | 연습 문제              | 58 |
| 제2장   | 행렬 대수              | 62 |
| 2.1   | 정의                 | 62 |
| 2.2   | 행렬 곱셈              | 63 |
| 2.2.1 | 정의                 | 64 |
| 2.2.2 | 벡터-행렬 곱셈           | 65 |
| 2.2.3 | 결합법칙               | 65 |
| 2.3   | 행렬의 전치             | 65 |
| 2.4   | 단위 행렬              | 66 |
| 2.5   | 행렬의 행렬식            | 67 |
| 2.5.1 | 행렬의 소행렬            | 67 |
| 2.5.2 | 정의                 | 68 |
| 2.6   | 행렬의 동반 행렬          | 69 |
| 2.7   | 행렬의 역행렬            | 69 |
| 2.8   | XNA 행렬             | 71 |
| 2.8.1 | 행렬 유형              | 71 |
| 2.8.2 | 매트릭스 함수            | 72 |
| 2.8.3 | XNA 행렬 샘플 프로그램     | 73 |
| 2.9   | 요약                 | 76 |
| 2.10  | 연습 문제              | 76 |
| 제3장   | 변환                 | 79 |
| 3.1   | 선형 변환              | 79 |
| 3.1.1 | 정의                 | 79 |
| 3.1.2 | 행렬 표현              | 80 |
| 3.1.3 | 스케일링               | 80 |
| 3.1.4 | 회전                 | 83 |
| 3.2   | 아핀 변환              | 85 |
| 3.2.1 | 동차 좌표              | 85 |
| 3.2.2 | 정의와 행렬 표현          | 85 |
| 3.2.2 | 평행 이동              | 86 |
| 3.2.3 | 확대 및 회전을 위한 아핀 행렬  | 88 |
| 3.2.4 | 아핀 변환 행렬의 기하학적 해석. | 88 |
| 3.3   | 변환의 합성             | 90 |
| 3.4   | 좌표 변환의 변화          | 92 |
| 3.4.1 | 벡터                 | 92 |
| 3.4.2 | 점                  | 93 |

|       |                |    |
|-------|----------------|----|
| 3.4.3 | 행렬 표현          | 94 |
| 3.4.4 | 결합성과 좌표 행렬의 변환 | 95 |
| 3.4.5 | 역행렬과 좌표계 변환 행렬 | 95 |

|                         |     |
|-------------------------|-----|
| 3.5 변환 행렬 대 좌표계 변환 행렬   | 97  |
| 3.6 XNA 수학 변환 함수        | 97  |
| 3.7 요약                  | 98  |
| 3.8 연습 문제               | 99  |
| 제2부 직접 3D 기초            | 104 |
| 제4장 Direct3D 초기화        | 105 |
| 4.1 준비 작업               | 105 |
| 4.1.1 Direct3D 개요       | 105 |
| 4.1.2 COM               | 105 |
| 4.1.3 텍스처 및 데이터 리소스 형식  | 105 |
| 4.1.4 스왑 체인과 페이지 플리핑    | 106 |
| 4.1.5 깊이 버퍼링            | 108 |
| 4.1.6 텍스처 리소스 뷰         | 110 |
| 4.1.7 멀티샘플링 이론          | 110 |
| 4.1.8 Direct3D에서의 멀티샘플링 | 112 |
| 4.1.9 기능 레벨             | 112 |
| 4.2 Direct3D 초기화        | 113 |
| 4.2.1 장치 및 컨텍스트 생성      | 113 |
| 4.2.2 4X MSAA 품질 지원 확인  | 115 |
| 4.2.3 스왑 체인 설명          | 116 |
| 4.2.4 스왑 체인 생성          | 117 |
| 4.2.5 렌더 타겟 뷰 생성        | 118 |
| 4.2.6 깊이/스텐실 버퍼 및 뷰 생성  | 118 |
| 4.2.7 뷰를 출력 병합 단계에 바인딩  | 120 |
| 4.2.8 뷰포트 설정            | 121 |
| 4.3 타이밍 및 애니메이션         | 122 |
| 4.3.1 성능 타이머            | 122 |
| 4.3.2 게임 타이머 클래스        | 123 |
| 4.3.3 프레임 간 경과 시간       | 123 |
| 4.3.4 총 시간              | 125 |
| 4.4 데모 애플리케이션 프레임워크     | 128 |
| 4.4.1 D3DApp            | 128 |
| 4.4.2 비 프레임워크 메서드       | 129 |
| 4.4.3 프레임워크 메서드         | 129 |
| 4.4.4 프레임 통계            | 130 |
| 4.4.5 메시지 핸들러           | 131 |
| 4.4.6 전체 화면으로           | 133 |
| 4.4.7 "Direct3D 초기화" 데모 | 133 |
| 4.5 Direct3D 애플리케이션 디버깅 | 135 |
| 4.6 요약                  | 135 |

|               |     |
|---------------|-----|
| 4.7 연습 문제     | 137 |
| 제5장 렌더링 파이프라인 | 139 |
| 5.1 3D 환상     | 139 |

|         |                          |     |
|---------|--------------------------|-----|
| 5.2     | 모델 표현                    | 142 |
| 5.3     | 기본 컴퓨터 색상                | 143 |
| 5.3.1   | 색상 작업                    | 144 |
| 5.3.2   | 128비트 색상                 | 145 |
| 5.3.3   | 32비트 색상                  | 145 |
| 5.4     | 렌더링 파이프라인 개요             | 147 |
| 5.5     | 입력 어셈블러 단계               | 149 |
| 5.5.1   | 버텍스                      | 149 |
| 5.5.2   | 프리티비브 토폴로지               | 149 |
| 5.5.2.1 | 점 목록                     | 150 |
| 5.5.2.2 | 선 스트립                    | 150 |
| 5.5.2.3 | 라인 리스트                   | 150 |
| 5.5.2.4 | 삼각형 스트립                  | 151 |
| 5.5.2.5 | 삼각형 목록                   | 152 |
| 5.5.2.6 | 인접성을 가진 원시형              | 152 |
| 5.5.2.7 | 제어점 패치 목록                | 152 |
| 5.5.3   | 인덱스                      | 153 |
| 5.6     | 버텍스 셰이더 단계               | 154 |
| 5.6.1   | 로컬 공간과 월드 공간             | 154 |
| 5.6.2   | 뷰 스페이스                   | 159 |
| 5.6.3   | 투영 및 동차 클립 공간            | 161 |
| 5.6.3.1 | 프러스텀 정의                  | 162 |
| 5.6.3.2 | 정점 투영                    | 165 |
| 5.6.3.3 | 정규화된 장치 좌표(NDC)          | 167 |
| 5.6.3.4 | 행렬을 이용한 투영 방정식 작성        | 167 |
| 5.6.3.5 | 정규화된 깊이 값                | 168 |
| 5.6.3.6 | XMMatrixPerspectiveFovLH | 170 |
| 5.7     | 테셀레이션 단계                 | 170 |
| 5.8     | 지오메트리 셰이더 단계             | 171 |
| 5.9     | 클리핑                      | 171 |
| 5.10    | 래스터화 단계                  | 173 |
| 5.10.1  | 뷰포트 변환                   | 173 |
| 5.10.2  | 백페이스 클리닝                 | 174 |
| 5.10.3  | 정점 속성 보간                 | 175 |
| 5.11    | 픽셀 셰이더 단계                | 177 |
| 5.12    | 출력 병합 단계                 | 177 |
| 5.13    | 요약                       | 177 |
| 5.14    | 연습 문제                    | 177 |
| 제6장     | Direct3D에서의 그리기          | 182 |
| 6.1     | 정점 및 입력 레이아웃             | 182 |

|                 |     |
|-----------------|-----|
| 6.2 버텍스 버퍼      | 186 |
| 6.3 인덱스와 인덱스 버퍼 | 189 |
| 6.4 버텍스 셰이더 예시  | 192 |

|                                 |     |
|---------------------------------|-----|
| 6.5 상수 버퍼                       | 194 |
| 6.6 예제 픽셀 셰이더                   | 195 |
| 6.7 렌더 상태                       | 197 |
| 6.8 효과                          | 199 |
| 6.8.1 효과 파일                     | 199 |
| 6.8.2 셰이더 컴파일                   | 201 |
| 6.8.3 C++ 애플리케이션에서 효과와 인터페이스하기  | 203 |
| 6.8.4 이펙트를 사용한 그리기              | 204 |
| 6.8.5 빌드 시점에 효과 컴파일하기           | 205 |
| 6.8.6 효과 프레임워크를 "셰이더 생성기"로 활용하기 | 206 |
| 6.8.7 어셈블리의 모습                  | 210 |
| 6.9 박스 데모                       | 214 |
| 6.10 HiIs 데모                    | 220 |
| 6.10.1 그리드 정점 생성                | 222 |
| 6.10.2 그리드 인덱스 생성               | 224 |
| 6.10.3 높이 함수 적용                 | 225 |
| 6.11 모양 데모                      | 227 |
| 6.11.1 원통형 메쉬 생성                | 227 |
| 6.11.1.1 원통 측면 형상               | 227 |
| 6.11.1.2 캡 형상                   | 230 |
| 6.11.2 구 메쉬 생성                  | 231 |
| 6.11.3 지오스피어 메쉬 생성              | 232 |
| 6.11.4 데모 코드                    | 234 |
| 6.12 파일에서 지오메트리 로드하기            | 238 |
| 6.13 동적 버텍스 버퍼                  | 239 |
| 6.14 요약                         | 241 |
| 6.15 연습 문제                      | 242 |
| 제7장 조명                          | 245 |
| 7.1 빛과 물질의 상호작용                 | 245 |
| 7.2 법선 벡터                       | 247 |
| 7.2.1 법선 벡터 계산                  | 248 |
| 7.2.2 법선 벡터 변환                  | 250 |
| 7.3 램베르트 코사인 법칙                 | 252 |
| 7.4 확산 조명                       | 254 |
| 7.5 주변 조명                       | 255 |
| 7.6 반사 조명                       | 255 |
| 7.7 간략한 요약                      | 259 |
| 7.8 재료 지정                       | 259 |
| 7.9 파라렐 조명                      | 261 |
| 7.10 포인트 라이트                    | 262 |

|        |        |     |
|--------|--------|-----|
| 7.10.1 | 감쇠     | 262 |
| 7.10.2 | 범위     | 263 |
| 7.11   | 스포트라이트 | 263 |

|                        |     |
|------------------------|-----|
| 7.12 구현                | 264 |
| 7.12.1 조명 구조물          | 264 |
| 7.12.2 구조물 패킹          | 266 |
| 7.12.3 방향성 조명 구현       | 267 |
| 7.12.4 포인트 라이트 구현      | 268 |
| 7.12.5 스포트라이트 구현       | 269 |
| 7.13 조명 데모             | 270 |
| 7.13.1 효과 파일           | 270 |
| 7.13.2 C++ 애플리케이션 코드   | 272 |
| 7.13.3 일반 계산           | 274 |
| 7.14 Lit Skull 데모      | 276 |
| 7.15 요약                | 277 |
| 7.16 연습 문제             | 278 |
| 제8장 텍스처링               | 280 |
| 8.1 텍스처와 리소스 요약        | 281 |
| 8.2 텍스처 좌표             | 282 |
| 8.3 텍스처 생성 및 활성화       | 285 |
| 8.4 필터                 | 287 |
| 8.4.1 확대               | 287 |
| 8.4.2 축소               | 290 |
| 8.4.2.1 mip맵 생성        | 290 |
| 8.4.3 이방성 필터링          | 291 |
| 8.5 텍스처 샘플링            | 291 |
| 8.6 텍스처와 재료            | 292 |
| 8.7 크레이트 데모            | 292 |
| 8.7.1 텍스처 좌표 지정        | 292 |
| 8.7.2 텍스처 생성           | 293 |
| 8.7.3 텍스처 설정           | 293 |
| 8.7.4 업데이트된 기본 효과      | 294 |
| 8.8 주소 모드              | 297 |
| 8.9 텍스처 변환             | 301 |
| 8.10 텍스처 처리된 고랑과 물결 데모 | 302 |
| 8.10.1 그리드 텍스처 좌표 생성   | 302 |
| 8.10.2 텍스처 타일링         | 304 |
| 8.10.3 텍스처 애니메이션       | 304 |
| 8.11 압축 텍스처 형식         | 305 |
| 8.12 요약                | 307 |
| 8.13 연습 문제             | 307 |
| 제9장 블렌딩                | 311 |
| 9.1 혼합 방정식             | 312 |
| 9.2 혼합 연산              | 312 |

|            |     |
|------------|-----|
| 9.3 블렌딩 팩터 | 312 |
| 9.4 블렌드 상태 | 313 |
| 9.5 예시     | 316 |

|                               |     |
|-------------------------------|-----|
| 9.5.1 색상 없음 쓰기                | 316 |
| 9.5.2 더하기/빼기                  | 316 |
| 9.5.3 곱하기                     | 317 |
| 9.5.4 투명도                     | 317 |
| 9.5.5 블렌딩과 덱스 버퍼              | 318 |
| 9.6 알파 채널                     | 319 |
| 9.7 클리핑 픽셀                    | 322 |
| 9.8 안개                        | 325 |
| 9.9 요약                        | 330 |
| 9.10 연습 문제                    | 331 |
| 제10장 스텐실링                     | 332 |
| 10.1 깊이/스텐실 형식 및 지우기          | 332 |
| 10.2 스텐실 테스트                  | 333 |
| 10.3 깊이/스텐실 상태 블록             | 334 |
| 10.3.1 깊이 설정                  | 334 |
| 10.3.2 스텐실 설정                 | 334 |
| 10.3.3 깊이/스텐실 상태 생성 및 바인딩     | 336 |
| 10.3.4 효과 파일 내의 깊이/스텐실 상태     | 336 |
| 10.4 평면 거울 구현                 | 337 |
| 10.4.1 미러 개요                  | 337 |
| 10.4.2 거울 깊이/스텐실 상태 정의        | 339 |
| 10.4.3 장면 그리기                 | 340 |
| 10.4.4 와인딩 순서와 반사             | 341 |
| 10.5 평면 그림자 구현                | 342 |
| 10.5.1 파라렐 라이트 색도             | 343 |
| 10.5.2 포인트 라이트 색도우            | 345 |
| 10.5.3 일반 그림자 매트릭스            | 346 |
| 10.5.4 스텐실 버퍼를 사용하여 이중 블렌딩 방지 | 347 |
| 10.5.5 색도 코드                  | 348 |
| 10.6 요약                       | 348 |
| 10.7 연습 문제                    | 349 |
| 제11장 지오메트리 셰이더                | 354 |
| 11.1 지오메트리 셰이더 프로그래밍          | 354 |
| 11.2 나무 게시판 데모                | 358 |
| 11.2.1 개요                     | 358 |
| 11.2.2 정점 구조                  | 362 |
| 11.2.3 효과 파일                  | 362 |
| 11.2.4 SV_PrimitiveID         | 367 |
| 11.3 텍스처 배열                   | 368 |

|                    |     |
|--------------------|-----|
| 11.3.1 개요          | 368 |
| 11.3.2 텍스처 배열 샘플링  | 368 |
| 11.3.3 텍스처 배열 로드하기 | 369 |

|        |                                  |     |
|--------|----------------------------------|-----|
| 11.4   | 11.4.1. p4hTa-eTtxtou-rCe 844gms | 372 |
| 11.5   | 요약                               | 374 |
| 11.6   | 연습 문제                            | 374 |
| 제12장   | 컴퓨터 셰이더                          | 377 |
| 12.1   | 스레드와 스레드 그룹                      | 379 |
| 12.2   | 간단한 컴퓨터 셰이더                      | 380 |
| 12.3   | 데이터 입력 및 출력 리소스                  | 381 |
| 12.3.1 | 텍스처 입력                           | 381 |
| 12.3.2 | 텍스처 출력 및 무순서                     | 381 |
| 12.3.3 | 인덱싱 및 텍스처 샘플링                    | 383 |
| 12.3.4 | 구조화된 버퍼 리소스                      | 385 |
| 12.3.5 | CS 결과 복사하기 시스템 메모리로              | 387 |
| 12.4   | 스레드 식별 시스템 값                     | 389 |
| 12.5   | 버퍼 추가 및 소비                       | 391 |
| 12.6   | 공유 메모리 및 동기화                     | 392 |
| 12.7   | 블러 데모                            | 393 |
| 12.7.1 | 블러링 이론                           | 393 |
| 12.7.2 | 텍스처로 렌더링                         | 396 |
| 12.7.3 | 블러 구현 개요                         | 398 |
| 12.7.4 | 컴퓨터 셰이더 프로그램                     | 402 |
| 12.8   | 추가 자료                            | 407 |
| 12.9   | 요약                               | 407 |
| 12.10  | 연습 문제                            | 408 |
| 제13장   | 테셀레이션 단계                         | 410 |
| 13.1   | 테셀레이션 기본 유형                      | 411 |
| 13.1.1 | 테셀레이션과 버텍스 셰이더                   | 412 |
| 13.2   | 후 셰이더                            | 412 |
| 13.2.1 | 상수 Hull 셰이더                      | 412 |
| 13.2.2 | 제어점 Hull 셰이더                     | 413 |
| 13.3   | 테셀레이션 단계                         | 414 |
| 13.3.1 | 쿼드 패치 테셀레이션 예시                   | 415 |
| 13.3.2 | 삼각형 패치 테셀레이션 예시                  | 415 |
| 13.4   | 도메인 셰이더                          | 416 |
| 13.5   | 사각형 테셀레이션                        | 417 |
| 13.6   | 입방 베지어 쿼드 패치                     | 420 |
| 13.6.1 | 베지어 곡선                           | 421 |
| 13.6.2 | 3차 베지어 표면                        | 423 |
| 13.6.3 | 3차 베지어 곡면 평가 코드                  | 424 |
| 13.6.4 | 패치 기하 구조 정의                      | 425 |
| 13.7   | 요약                               | 427 |

|                  |     |
|------------------|-----|
| 13.8 연습 문제       | 427 |
| 제3부 주제           | 429 |
| 제14장 1인칭 카메라 구축, | 430 |

|          |                  |     |
|----------|------------------|-----|
| 14.1     | 뷰 트랜스폼 복습        | 430 |
| 14.2     | 카메라 클래스          | 431 |
| 14.3     | 선택된 메서드 구현       | 433 |
| 14.3.1   | XMVECTOR 반환 변형   | 433 |
| 14.3.2   | SetLens          | 433 |
| 14.3.3   | 파생된 프러스텀 정보      | 434 |
| 14.3.4   | 카메라 변환           | 434 |
| 14.3.5   | 뷰 매트릭스 구축        | 435 |
| 14.4     | 카메라 데모 코멘트       | 436 |
| 14.5     | 요약               | 437 |
| 14.6     | 연습 문제            | 438 |
| 제15장     | 인스턴싱과 프러스텀 커팅    | 440 |
| 15.1     | 하드웨어 인스턴싱        | 440 |
| 15.1.1   | 버텍스 셰이더          | 440 |
| 15.1.2   | 스트리밍 인스턴스화된 데이터  | 441 |
| 15.1.3   | 인스턴스화된 데이터 그리기   | 442 |
| 15.1.4   | 인스턴스화된 버퍼 생성     | 443 |
| 15.2     | 경계 볼륨 및 프러스텀     | 444 |
| 15.2.1   | XNA 충돌           | 444 |
| 15.2.2   | 상자               | 444 |
| 15.2.2.1 | 회전 및 축 정렬 바운딩 박스 | 447 |
| 15.2.3   | 구체               | 449 |
| 15.2.4   | 프러스텀             | 449 |
| 15.2.4.1 | 별뿔면 평면 구성        | 449 |
| 15.2.4.2 | 프러스텀/구체 교차       | 452 |
| 15.2.4.3 | 프러스텀/AABB 교차     | 452 |
| 15.3     | 프러스텀 커링          | 453 |
| 15.4     | 요약               | 456 |
| 15.5     | 연습 문제            | 457 |
| 제16장     | 픽킹               | 460 |
| 16.1     | 스크린에서 투영 창으로의 변환 | 461 |
| 16.2     | 월드/로컬 공간 픽킹 레이   | 464 |
| 16.3     | 레이/메쉬 교차점        | 465 |
| 16.3.1   | 레이/AABB 교차       | 466 |
| 16.3.2   | 레이/구체 교차         | 466 |
| 16.3.3   | 선삼각형 교차          | 467 |
| 16.4     | 데모 애플리케이션        | 469 |
| 16.5     | 요약               | 470 |
| 16.6     | 연습 문제            | 471 |
| 제17장     | 큐브 매핑            | 472 |

|                                |     |
|--------------------------------|-----|
| 17.1 큐브 매핑                     | 472 |
| 17.2 환경 맵                      | 473 |
| 17.2.1 Direct3D에서 큐브 맵 로딩 및 사용 | 477 |

|        |                        |     |
|--------|------------------------|-----|
| 17.3   | 하늘 텍스처링                | 477 |
| 17.4   | 반사 모델링                 | 480 |
| 17.5   | 동적 큐브 맵                | 482 |
| 17.5.1 | 큐브 맵 및 렌더 타깃 뷰 구축      | 483 |
| 17.5.2 | 깊이 버퍼 및 뷰포트 구축         | 484 |
| 17.5.3 | 큐브 맵 카메라 설정            | 485 |
| 17.5.4 | 큐브 맵에 그리기              | 486 |
| 17.6   | 지오메트리 셰이더를 사용한 동적 큐브 맵 | 487 |
| 17.7   | 요약                     | 489 |
| 17.8   | 연습 문제                  | 489 |
| 제18장   | 노멀 매핑과 변위 매핑 .....     | 492 |
| 18.1   | 동기 부여                  | 492 |
| 18.2   | 노멀 맵                   | 494 |
| 18.3   | 텍스처/탄젠트 공간             | 496 |
| 18.4   | 정점 접선 공간               | 499 |
| 18.5   | 접선 공간과 객체 공간 간의 변환     | 500 |
| 18.6   | 노멀 매핑 셰이더 코드           | 500 |
| 18.7   | 변위 매핑                  | 505 |
| 18.8   | 변위 매핑 셰이더 코드           | 506 |
| 18.8.1 | 프리미티브 유형               | 506 |
| 18.8.2 | 버텍스 셰이더                | 506 |
| 18.8.3 | Hull 셰이더               | 508 |
| 18.8.4 | 도메인 셰이더                | 509 |
| 18.9   | 요약                     | 511 |
| 18.10  | 연습 문제                  | 511 |
| 제19장   | 지형 렌더링                 | 514 |
| 19.1   | 하이트맵                   | 515 |
| 19.1.1 | 하이트맵 생성하기              | 516 |
| 19.1.2 | RAW 파일 로드하기            | 517 |
| 19.1.3 | 스무딩                    | 519 |
| 19.1.4 | 하이트맵 셰이더 리소스 뷰         | 521 |
| 19.2   | 지형 테셀레이션               | 522 |
| 19.2.1 | 그리드 구축                 | 522 |
| 19.2.2 | 지형 버텍스 셰이더             | 525 |
| 19.2.3 | 테셀레이션 요소               | 526 |
| 19.2.4 | 변위 매핑                  | 527 |
| 19.2.5 | 접선 및 법선 벡터 추정          | 528 |
| 19.3   | 프러스텀 커링 패치             | 529 |
| 19.4   | 텍스처링                   | 534 |
| 19.5   | 지형 높이                  | 536 |

|                             |     |
|-----------------------------|-----|
| 19.6 요약                     | 539 |
| 19.7 연습 문제                  | 540 |
| 제20장 입자 시스템과 스트림 아웃         | 542 |
| 20.1 입자 표현                  | 542 |
| 20.2 입자 운동                  | 543 |
| 20.3 무작위성                   | 545 |
| 20.4 블렌딩 및 파티클 시스템          | 547 |
| 20.5 스트림 아웃                 | 548 |
| 20.5.1 스트림 아웃용 지오메트리 셰이더 생성 | 549 |
| 20.5.2 스트림 아웃 전용            | 550 |
| 20.5.3 스트림 아웃용 버텍스 버퍼 생성    | 550 |
| 20.5.4 SO 스테이지에 바인딩         | 551 |
| 20.5.5 스트림 아웃 스테이지에서 바인딩 해제 | 551 |
| 20.5.6 자동 그리기               | 551 |
| 20.5.7 버텍스 버퍼의 핑퐁           | 551 |
| 20.6 GPU 기반 파티클 시스템         | 552 |
| 20.6.1 입자 효과                | 552 |
| 20.6.2 입자 시스템 클래스           | 552 |
| 20.6.3 이미터 파티클              | 553 |
| 20.6.4 초기화 버텍스 버퍼           | 554 |
| 20.6.5 업데이트/드로우 메서드         | 554 |
| 20.7 발사                     | 556 |
| 20.8 비                      | 561 |
| 20.9 요약                     | 565 |
| 20.10 연습                    | 565 |
| 제21장 색도 매핑                  | 567 |
| 21.1 씬 깊이 렌더링               | 567 |
| 21.2 직교 투영                  | 569 |
| 21.3 투영 텍스처 좌표              | 571 |
| 21.3.1 코드 구현                | 573 |
| 21.3.2 프러스텀 외부 점            | 573 |
| 21.3.3 직교 투영                | 574 |
| 21.4 색도 매핑                  | 574 |
| 21.4.1 알고리즘 설명              | 574 |
| 21.4.2 바이어싱 및 에일리어싱         | 575 |
| 21.4.3 PCF 필터링              | 579 |
| 21.4.4 색도 맵 구축              | 582 |
| 21.4.5 색도우 팩터               | 586 |
| 21.4.6 색도우 맵 테스트            | 587 |
| 21.4.7 색도 맵 렌더링             | 590 |

|          |                            |     |
|----------|----------------------------|-----|
| 21.5     | 대형 PCF 커널                  | 591 |
| 21.5.1   | DDX 및 DDY 함수               | 592 |
| 21.5.2   | 대형 PCF 커널 문제의 해결책          | 592 |
| 21.5.3   | 대형 PCF 커널 문제에 대한 대안적 해결책.. | 594 |
| 21.6     | 요약                         | 595 |
| 21.7     | 연습문제                       | 596 |
| 제22장     | 앰비언트 오클루전                  | 597 |
| 22.1     | 레이 캐스팅을 통한 앰비언트 오클루전       | 597 |
| 22.2     | 스크린 스페이스 앰비언트 오클루전         | 601 |
| 22.2.1   | 노멀 및 깊이 패스 렌더링             | 601 |
| 22.2.2   | 앰비언트 오클루전 패스               | 601 |
| 22.2.2.1 | 뷰 스페이스 위치 재구성              | 602 |
| 22.2.2.2 | 랜덤 샘플 생성                   | 603 |
| 22.2.2.3 | 잠재적 가림점 생성                 | 604 |
| 22.2.2.4 | 페색 테스트 수행                  | 604 |
| 22.2.2.5 | 계산 완료                      | 605 |
| 22.2.2.6 | 구현                         | 605 |
| 22.2.3   | 블러 패스                      | 608 |
| 22.2.4   | 앰비언트 오클루전 맵 사용             | 612 |
| 22.3     | 요약                         | 614 |
| 22.4     | 연습 문제                      | 614 |
| 제23장     | 메쉬                         | 617 |
| 23.1     | m3d 형식                     | 617 |
| 23.1.1   | 헤더                         | 617 |
| 23.1.2   | 재료                         | 618 |
| 23.1.3   | 하위 집합                      | 619 |
| 23.1.4   | 정점 및 삼각형 인덱스               | 621 |
| 23.2     | 메쉬 지오메트리                   | 622 |
| 23.3     | 기본 모델                      | 622 |
| 23.4     | 메쉬 뷰어 데모                   | 624 |
| 23.5     | 요약                         | 624 |
| 23.6     | 연습 문제                      | 625 |
| 제24장     | 사원수                        | 626 |
| 24.1     | 복소수 복습                     | 626 |
| 24.1.1   | 정의                         | 626 |
| 24.1.2   | 기하학적 해석                    | 627 |
| 24.1.3   | 극좌표 표현과 회전                 | 628 |
| 24.2     | 쿼터니언 대수                    | 630 |
| 24.2.1   | 정의와 기본 연산                  | 630 |
| 24.2.2   | 특수 제품                      | 631 |

|        |                            |     |
|--------|----------------------------|-----|
| 24.2.3 | 속성                         | 631 |
| 24.2.4 | 변환                         | 632 |
| 24.2.5 | 공액과 노름                     | 632 |
| 24.2.6 | 역함수                        | 633 |
| 24.2.7 | 극좌표 표현                     | 633 |
| 24.3   | 단위 쿼터니언과 회전                | 635 |
| 24.3.1 | 회전 연산자                     | 635 |
| 24.3.2 | 쿼터니언 회전 연산자를 행렬로 변환        | 637 |
| 24.3.3 | 행렬을 쿼터니언 회전 연산자로 변환        | 638 |
| 24.3.4 | 합성                         | 640 |
| 24.4   | 쿼터니언 보간                    | 640 |
| 24.5   | XNA 수학 쿼터니언 함수             | 645 |
| 24.6   | 회전 데모                      | 646 |
| 24.7   | 요약                         | 650 |
| 24.8   | 연습 문제                      | 651 |
| 제25장   | 캐릭터 애니메이션                  | 653 |
| 25.1   | 프레임 계층 구조                  | 653 |
| 25.1.1 | 수학적 공식화                    | 654 |
| 25.2   | 스킨 메쉬                      | 656 |
| 25.2.1 | 정의                         | 656 |
| 25.2.2 | 뼈대-루트 변환 재구성               | 657 |
| 25.2.3 | 오프셋 변환                     | 657 |
| 25.2.4 | 골격 애니메이션                   | 658 |
| 25.2.5 | 최종 변환 계산                   | 659 |
| 25.3   | 버텍스 블렌딩                    | 660 |
| 25.4   | .m3d 애니메이션 데이터 로드          | 664 |
| 25.4.1 | 스킨 처리된 버텍스 데이터             | 664 |
| 25.4.2 | 본 오프셋 변환                   | 665 |
| 25.4.3 | 계층 구조                      | 665 |
| 25.4.4 | 애니메이션 데이터                  | 666 |
| 25.4.5 | M3DLoader                  | 668 |
| 25.5   | 캐릭터 애니메이션 데모               | 669 |
| 25.6   | 요약                         | 670 |
| 25.7   | 연습 문제                      | 671 |
| 부록 A:  | Windows 프로그래밍 소개           | 673 |
| A.1    | 개요                         | 673 |
| A.1.1  | 참고 자료                      | 673 |
| A.1.2  | 이벤트, 메시지 큐, 메시지, 메시지 루프... | 673 |
| A.1.3  | GUI                        | 675 |
| A.1.4  | 유니코드                       | 676 |

|       |                      |     |
|-------|----------------------|-----|
| A.2   | 기본 Windows 응용 프로그램   | 676 |
| A.3   | 기본 Windows 애플리케이션 설명 | 680 |
| A.3.1 | 포함, 전역 변수 및 프로토타입    | 680 |
| A.3.2 | WinMain              | 680 |
| A.3.3 | WNDCLASS 및 등록        | 681 |
| A.3.4 | 창 생성 및 표시            | 682 |
| A.3.5 | 메시지 루프               | 684 |
| A.3.6 | 창 프로시저               | 685 |
| A.3.7 | 메시지 상자 함수            | 686 |
| A.4   | 더 나은 메시지 루프          | 686 |
| A.5   | 요약                   | 687 |
| A.6   | 연습 문제                | 687 |

|                                |     |
|--------------------------------|-----|
| 스칼라 유형                         | 688 |
| 벡터 유형                          | 688 |
| 스위즐                            | 688 |
| 매트릭스 유형                        | 689 |
| 배열                             | 690 |
| 구조체                            | 690 |
| typedef 키워드                    | 690 |
| 변수 접두사                         | 691 |
| 형 변환                           | 691 |
| 키워드와 연산자                       | 691 |
| 키워드                            | 691 |
| 연산자                            | 692 |
| 프로그램 흐름                        | 693 |
| 함수                             | 693 |
| 사용자 정의 함수                      | 693 |
| 내장 함수                          | 695 |
| 부록 C: 일부 해석 기하학                | 697 |
| C.1 광선, 직선 및 선분                | 697 |
| C.2 평행선                        | 698 |
| C.3 삼각형                        | 698 |
| C.4 평면                         | 699 |
| C.4.1 XNA 수학 평면                | 701 |
| C.4.2 점/평면의 공간적 관계             | 701 |
| C.4.3 구성                       | 702 |
| C.4.4 평면의 정규화                  | 702 |
| C.4.5 평면 변환                    | 702 |
| C.4.6 주어진 점에 대한 평면 상의 가장 가까운 점 | 703 |
| C.4.7 선분과 평면의 교점               | 703 |
| C.4.8 벡터 반사                    | 704 |
| C.4.9 점의 반사                    | 705 |
| C.4.10 반사 행렬                   | 705 |
| C.5 연습문제                       | 707 |
| 부록 D: 연습문제 해답                  | 709 |
| 참고문헌 및 추가 자료                   | 710 |
| 색인                             | 713 |